

Git & GitHub Mastery

From Zero → Internals → Production → Senior Engineer

Volume 4 of 4: Scale, professional practice, production, and reference

Baseline: Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Every Git transcript in this book is real output from the lab scripts under `labs/`, reproducible with `labs/verify-all.sh`. GitHub-side behavior is described from GitHub's documentation and is marked as such.

Contents of this volume

- Chapter 22: Git LFS
- Chapter 23: Submodules and Subtrees
- Chapter 24: Monorepos
- Chapter 25: Worktrees
- Chapter 26: Performance
- Chapter 27: Open source and team workflows
- Chapter 28: AI/ML workflows
- Chapter 29: Production Troubleshooting
- Chapter 30: Incident Response
- Chapter 31: Interview Preparation
- Chapter 32: Capstone
- Chapter 33: Cheat Sheets
- Chapter 33 (continued): Emergency recovery, one page
- Chapter 33 (continued): Command safety
- Chapter 34: Glossary
- Chapter 35: References

Chapter 22: Git LFS

Baseline. Git 2.55.0 and git-lfs 3.7.1 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch22/`.

22.1 Why this matters

Questions that reach a CTO's desk in an AI/ML team:

1. "The push to GitHub was rejected because of a model file. I deleted the file and committed. It is still rejected. Why?"
2. "The clone takes twenty minutes and the repository holds four hundred lines of Python."
3. "The training job in CI crashed while loading the weights: the file is 132 bytes and starts with the word `version`."
4. "Our storage bill for this repository keeps growing, and nobody has added a model in months."

The first two are Git doing what it was designed to do with content it was not designed for. The third is Git LFS working as designed on a machine that does not have it. The fourth is how GitHub accounts for LFS. To answer any of them you need to know what LFS stores where.

Git LFS (Large File Storage) is not part of Git. It is a separate program, `git-lfs`, that plugs into Git through two mechanisms you already know from Chapter 14C: a clean and smudge filter selected by `.gitattributes`, and hooks. This chapter does not repeat how filters and attributes work; it shows what this particular filter does.

The project is `transcriber`, a speech-to-text service: a little code and one acoustic model file, `models/acoustic.onnx`. The lab uses stand-in files of 1.2 to 1.3 MB so that the demos run in a second; real models are a thousand times larger and behave the same.

What was run, and what was not. The remote in this chapter is a bare repository reached through a file path, which git-lfs 3.7.1 supports with a built-in transfer adapter. Tracking, pushing, cloning, fetching, pruning and migrating are therefore real transcripts; four demos are marked volatile, for reasons given where they appear. Anything that needs an LFS server could not be run: the HTTP API, authentication, file locking, and all GitHub limits and billing. Those parts are described from the documentation and labelled.

22.2 Why LFS exists

In one sentence. Git keeps every version of every file forever and gives every clone all of them, which is right for source code and ruinous for large binaries that change.

See it. Three versions of the model were committed as ordinary files:

```
$ git log --oneline
8b13104 Retrain acoustic model with accents (v3)
1ddbfc0 Resample input to 16 kHz
65c2188 Retrain acoustic model on noisy audio (v2)
f9e56b8 Add acoustic model v1
c66100a Add transcription entry point
$ wc -c models/acoustic.onnx
1300000 models/acoustic.onnx
# Every blob that any commit reaches, with its size in bytes:
$ git rev-list --objects --all | git cat-file --batch-check="%(<objecttype> %(<objectsize>) %(<rest>)" | grep blob
blob 68 README.md
blob 1300000 models/acoustic.onnx
blob 14 models/vocab.txt
blob 231 transcribe.py
blob 1250000 models/acoustic.onnx
blob 198 transcribe.py
blob 1200000 models/acoustic.onnx
```

The working tree has one model of 1.3 MB. The repository has three blobs, one per version, each stored whole. Chapter 3 showed that packfiles store similar objects as deltas; that helps when versions share long runs of bytes. Retrained weights, compressed archives, images and audio share almost nothing from one version to the next, so each version costs its full size, and zlib cannot shrink data that is already dense.

```
$ git push origin main
To $LAB/ch22/why-lfs/remotes/transcriber.git
c66100a..8b13104 main -> main
$ cd ..
$ git clone remotes/transcriber.git ravi-transcriber
Cloning into 'ravi-transcriber'...
done.
$ cd ravi-transcriber
# One model file in the working tree, three in the repository:
$ ls models
```

```
acoustic.onnx
vocab.txt
$ git cat-file --batch-all-objects --batch-check="%(objecttype) %(objectsize)" | grep -c -E "blob [0-9]{7}"
3
```

A teammate's clone downloads all three, to check out one. The cost grows with the number of versions, and it is paid again by every clone and every CI job.

```
$ git rm --quiet models/acoustic.onnx
$ git commit -m "Remove the model from the repository"
[main 09c8ef8] Remove the model from the repository
1 file changed, 0 insertions(+), 0 deletions(-)
delete mode 100644 models/acoustic.onnx
$ git rev-list --objects --all | git cat-file --batch-check="%(objecttype) %(objectsize) %(rest)" | grep onnx
blob 1300000 models/acoustic.onnx
blob 1250000 models/acoustic.onnx
blob 1200000 models/acoustic.onnx
```

Deleting the file adds a commit. The three blobs are still reachable from the earlier commits, which is the answer to question 1: a hosting platform that rejects a large blob rejects it because it is in a commit being pushed, not because it is in the latest tree. Only a history rewrite removes it (section 22.9, and Chapter 13 for rewrites in general).

LFS changes one thing: Git stores a small text file in place of the binary, and the binary goes to a separate store that is downloaded per version, on demand.

22.3 The pointer file and the two filters

In one sentence. For a path that is tracked by LFS, the blob in Git is a three-line pointer containing the SHA-256 and the size of the real content, and the real content lives in an LFS object store, locally under `.git/lfs/objects` and remotely beside the Git repository.

Analogy. A coat check. You hand over the coat and carry a numbered ticket; the ticket goes wherever you go, and the coat stays in the cloakroom until someone presents the ticket. Git history carries tickets. The analogy breaks in two places: the number on the ticket is computed from the coat itself, so the same content always gets the same ticket, and a ticket can be copied to a place that has no cloakroom attendant, where it is only a piece of paper.

Precisely. Two filter commands do the exchange ([specification](#)):

- **clean**, run by `git add`: reads the file content, computes its SHA-256, stores the content in `.git/lfs/objects/<first two hex digits>/<next two>/<full hash>`, and hands Git the pointer text. Git hashes and stores the pointer as a blob.
- **smudge**, run by checkout: reads the pointer from the blob, finds the object in the local store, downloads it from the remote if it is not there, and writes the real content into the working tree.

The pointer format is fixed by the specification: UTF-8 text, one `key value` pair per line, `version` first and the other keys in alphabetical order, less than 1024 bytes in total. The required keys are `version`, `oid` (with the hash method as prefix) and `size`.

Set it up. `git lfs install` normally writes the filter definition into your global configuration and installs hooks in the current repository. In this book it is always run with `--local`, which writes the definition into the repository's own `.git/config` and nothing anywhere else:

```
$ git lfs version
git-lfs/3.7.1 (GitHub; darwin arm64; go 1.25.3)
$ git lfs install --local
Updated Git hooks.
Git LFS initialized.
$ git config list --local | grep -e ^filter -e ^lfs | sort
filter.lfs.clean=git-lfs clean -- %f
filter.lfs.process=git-lfs filter-process
filter.lfs.required=true
filter.lfs.smudge=git-lfs smudge -- %f
lfs.repositoryformatversion=0
$ ls .git/hooks | grep -v sample
post-checkout
post-commit
post-merge
pre-push
```

`filter.lfs.required=true` is the setting Chapter 14C described for filters whose stored form is unusable by itself: if the filter fails, the Git command fails. `filter.lfs.process` is the long-running variant that handles all files of one command in a single process. The four hooks are covered in section 22.5. (The lines are sorted for the transcript; git-lfs writes the four `filter` keys in an order that changes from run to run.)

The filter definition says how to convert. `.gitattributes` says which paths get converted. `git lfs track` writes that line:

```
$ git lfs track "*.onnx"
Tracking "*.onnx"
$ cat .gitattributes
```

```

*.onnx filter=lfs diff=lfs merge=lfs -text
$ git lfs track
Listing tracked patterns
  *.onnx (.gitattributes)
Listing excluded patterns
$ git check-attr filter diff merge text -- models/acoustic.onnx
models/acoustic.onnx: filter: lfs
models/acoustic.onnx: diff: lfs
models/acoustic.onnx: merge: lfs
models/acoustic.onnx: text: unset
$ git check-attr filter -- models/vocab.txt
models/vocab.txt: filter: unspecified

```

See it. Add a model of 1,200,000 bytes and commit it together with `.gitattributes`:

```

$ wc -c models/acoustic.onnx
1200000 models/acoustic.onnx
$ shasum -a 256 models/acoustic.onnx
02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de  models/acoustic.onnx
$ git add .gitattributes models/acoustic.onnx
$ git commit -m "Add acoustic model v1, tracked with Git LFS"
[main 26caf87] Add acoustic model v1, tracked with Git LFS
2 files changed, 4 insertions(+)
create mode 100644 .gitattributes
create mode 100644 models/acoustic.onnx

```

```

# What Git stored for the path:
$ git cat-file -p HEAD:models/acoustic.onnx
version https://git-lfs.github.com/spec/v1
oid sha256:02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
size 1200000
$ git cat-file -s HEAD:models/acoustic.onnx
132
# What is in the working tree:
$ wc -c models/acoustic.onnx
1200000 models/acoustic.onnx

```

The blob that Git stored is 132 bytes of text. Its `oid` is the SHA-256 that `shasum` printed for the file, and its `size` is the file's size. The working tree still has the real file: the clean filter changes what goes into the repository, not what is on your disk.

Inside `.git`.

```

$ find .git/lfs/objects -type f
.git/lfs/objects/02/69/02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
$ git lfs ls-files
0269885262 * models/acoustic.onnx
$ git lfs ls-files --long --size
02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de * models/acoustic.onnx (1.2 MB)
$ git lfs pointer --file=models/acoustic.onnx
Git LFS pointer for models/acoustic.onnx

version https://git-lfs.github.com/spec/v1
oid sha256:02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
size 1200000

```

The content is in `.git/lfs/objects/`, in a file named by its own SHA-256. It is not a Git object: `git cat-file`, `git fsck`, `git gc` and `git push` know nothing about it. `git lfs ls-files` lists the LFS-tracked paths of the current commit; the `*` means the working tree file holds the real content, and a `-` would mean it holds a pointer. `git lfs pointer --file` computes the pointer for a file without storing anything, which is a quick way to check what a file's pointer should be.

Picture.



In production. A repository holds an evaluation harness and three reference checkpoints of a few hundred megabytes that change twice a year. With LFS, a clone downloads the code and the pointers at once and each checkpoint only when a checkout needs it, and a CI job that does not need the checkpoints can skip them (section 22.7).

State table for `git add` and `git commit` of an LFS-tracked file:

Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
unchanged (real content)	entry for the path names a pointer blob	new commit on git commit	advanced on git commit	pointer blob in .git/objects; real content in .git/lfs/objects	unchanged until pushed	unchanged until pushed

22.4 The .gitattributes line, and what tracking does not do

```
*.onnx filter=lfs diff=lfs merge=lfs -text
```

Attribute	Effect
filter=lfs	Run the <code>lfs</code> clean and smudge filter for matching paths. This is the attribute that matters.
diff=lfs	Use a diff driver named <code>lfs</code> . <code>git lfs install</code> defines none, so <code>git diff</code> shows the difference between pointers (section 22.5).
merge=lfs	Use a merge driver named <code>lfs</code> . None is defined either, so Git merges the pointers as text. When two branches each commit a new version, the merge stops with conflict markers around the <code>oid</code> and <code>size</code> lines of the pointer; <code>git checkout --theirs <path></code> or <code>--ours</code> , then <code>git add</code> , picks one version (run for this chapter, no transcript printed).
-text	Never apply line-ending conversion (Chapter 14C, section 14C.5).

Rules that follow from this being an ordinary attributes file:

- **Commit .gitattributes**. It is the only part of the setup that travels with the repository. The filter definition is local configuration and the hooks are local files; a clone gets neither (Chapter 14C, section 14C.4).
- **Patterns follow gitignore syntax**. Quote them in the shell ("`*.onnx`"), or the shell expands the pattern to the files that exist today and `git lfs track` records those names.
- **Tracking applies from the next git add**. `git lfs track` edits a text file. It does not convert a file that is already in the index or in history; section 22.10 shows the symptom, and section 22.9 the conversion.

22.5 Day to day: status, diff, and the hooks

A new version of the model, not yet staged:

```
$ git status --short
M models/acoustic.onnx
$ git diff
diff --git a/models/acoustic.onnx b/models/acoustic.onnx
index 1097a4b..366f6ac 100644
--- a/models/acoustic.onnx
+++ b/models/acoustic.onnx
@@ -1,3 +1,3 @@
 version https://git-lfs.github.com/spec/v1
-oid sha256:02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
-size 1200000
+oid sha256:f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
+size 1250000
$ git lfs status
On branch main
Objects to be pushed to origin/main:

    models/acoustic.onnx (02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de)

Objects to be committed:

Objects not staged for commit:

    models/acoustic.onnx (LFS: 0269885 -> File: f8719e1)
```

`git status` reports a modified file as usual: Git ran the clean filter on the working tree file, got a different pointer, and compared it with the index. `git diff` shows the change between the two pointers, which tells a reviewer that the file changed and by how much it grew, and nothing about its content. 🟢 `git lfs status` adds the LFS view: which objects a push would upload, and for changed files whether the index and working tree hold LFS content.

`git lfs install` wrote four hooks. Each is two lines:

```
$ cat .git/hooks/pre-push
#!/bin/sh
command -v git-lfs >/dev/null 2>&1 || { printf >&2 "\n%s\n\n" "This repository is configured for Git LFS but 'git-lfs' was not found on your path. If you no longer wish to use Git LFS, remove this hook by deleting the 'pre-push' file in the hooks directory (set by 'core.hookspath'; usually '.git/hooks')." ; exit 2; }
git lfs pre-push "$@"
$ cat .git/hooks/post-checkout
```

```
#!/bin/sh
command -v git-lfs >/dev/null 2>&1 || { printf >&2 "\n%s\n\n" "This repository is configured for Git LFS but 'git-lfs' was not
found on your path. If you no longer wish to use Git LFS, remove this hook by deleting the 'post-checkout' file in the hooks
directory (set by 'core.hookspath'; usually '.git/hooks')."; exit 2; }
git-lfs post-checkout "$@"
```

`pre-push` is the important one: `git-lfs pre-push` uploads the LFS objects referenced by the commits about to be pushed, before Git sends the commits. The other three exist for file locking. The first line of each hook is the source of a message people meet on machines without the client: "This repository is configured for Git LFS but 'git-lfs' was not found on your path."

Two consequences of the upload living in a hook (Chapter 14C, section 14C.11): `git push --no-verify` skips it, and a repository whose `core.hookspath` points elsewhere does not run it unless the LFS hook was installed there.

22.6 Pushing, and where the objects go

The LFS client derives its server from the Git remote. For an HTTPS or SSH remote it appends `/info/lfs` to the repository URL and talks to an HTTP API there ([server discovery](#)); `lfs.url`, or a committed `.lfsconfig` file, can point it elsewhere. For the lab's path remote it resolves a `file://` endpoint and uses its built-in `lfs-standalone-file` adapter, which copies objects into an `lfs/objects` directory inside the bare repository:

```
$ git remote get-url origin
$LAB/ch22/lfs-push-volatile/remotes/transcriber.git
$ git lfs env | grep -e ^Endpoint -e Transfers
Endpoint=file://$LAB/ch22/lfs-push-volatile/remotes/transcriber.git (auth=none)
ConcurrentTransfers=8
TusTransfers=false
BasicTransfersOnly=false
DownloadTransfers=basic,lfs-standalone-file,ssh
UploadTransfers=basic,lfs-standalone-file,ssh
```

```
$ git lfs status
On branch main
Objects to be pushed to origin/main:

    models/acoustic.onnx (f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65)

Objects to be committed:

Objects not staged for commit:

$ git push origin main
Uploading LFS objects: 100% (1/1), 0 B | 0 B/s, done.
To $LAB/ch22/lfs-push-volatile/remotes/transcriber.git
1690072..98ef746  main -> main
```

The line `Uploading LFS objects` is printed by the pre-push hook, before Git's own output. This demo is marked volatile for that line alone: with `git-lfs 3.7.1` and a file remote the progress meter printed nothing in about one push out of ten in our runs, although the upload happened each time. The `0 B` is also an artifact of the file adapter, which does not report bytes.

```
# The remote Git repository received a pointer blob:
$ git -C ../remotes/transcriber.git cat-file -s main:models/acoustic.onnx
132
# The bytes are in the LFS store beside it, one file per version, named by SHA-256:
$ find ../remotes/transcriber.git/lfs/objects -type f | sort
../remotes/transcriber.git/lfs/objects/02/69/02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
../remotes/transcriber.git/lfs/objects/f8/71/f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
```

The remote now has two stores, like your clone: Git objects with a 132-byte blob for the path, and an LFS directory with one full file per version. On GitHub the second store is a separate storage service with its own quota (section 22.11).

State table for `git push` in a repository with the LFS hook:

Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
unchanged	unchanged	unchanged	unchanged	remote-tracking branch updated	LFS objects uploaded first, then commits with pointer blobs	Each uploaded object counts against the repository owner's LFS storage

What a file remote cannot do. File locking (`git-lfs lock`, and `git-lfs track --lockable`, which makes files read-only until locked) is a server feature:

```
$ git-lfs lock models/acoustic.onnx

hint: The remote resolves to a file:// URL, which can only work with a
```

```
hint: standalone transfer agent. See section "Using a Custom Transfer Type
hint: without the API server" in custom-transfers.md for details.
Locking models/acoustic.onnx failed: missing protocol: "file://$LAB/ch22/lfs-lock/remotes/transcriber.git"
[exit status: 2]
```

Locking exists because binary files cannot be merged: two people who edit the same model or design file in parallel produce a conflict that only one of them can win. It was not run for this book.

22.7 Cloning: what arrives without the client, and with it

The lab's configuration has no LFS filter, which makes it behave like a laptop, a container or a CI runner where `git lfs install` was never run:

```
# The lab configuration has no LFS filter, like a machine where git-lfs was never set up:
$ git config get filter.lfs.smudge
[exit status: 1]
$ git clone remotes/transcriber.git ravi-transcriber
Cloning into 'ravi-transcriber'...
done.
$ cd ravi-transcriber
$ cat models/acoustic.onnx
version https://git-lfs.github.com/spec/v1
oid sha256:f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
size 1250000
$ git status --short --branch
## main...origin/main
```

The clone succeeds, `git status` is clean, and the model file is the pointer: 132 bytes of text. Nothing is wrong from Git's point of view. `.gitattributes` names a filter called `lfs`, no such filter is defined in this configuration, and an undefined filter is a no-op. This is question 3 of section 22.1. GitHub's documentation says the same about collaborators without the client: they "will only fetch the pointer files, and won't have access to any of the actual data" ([collaboration with Git LFS](#)).

Root cause. A program that crashes on a file beginning with `version https://git-lfs.github.com/spec/v1` is reading an LFS pointer. The first diagnostic is `head -c 200 <file>`; the second is `git lfs ls-files`, where a `-` marks a path whose working tree file is still a pointer.

```
$ git lfs ls-files
f8719e1ac8 - models/acoustic.onnx
$ git lfs pull
Skipping object checkout, Git LFS is not installed for this repository.
Consider installing it with 'git lfs install'.
$ find .git/lfs -type f
.git/lfs/objects/f8/71/f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
```

`git lfs pull` downloaded the object into the local store and then declined to touch the working tree, because the repository has no LFS filter configured. After `git lfs install --local` the pieces can be run one at a time. 🟢 `git lfs fetch` downloads the objects that the current commit needs; 🟢 `git lfs checkout` replaces pointer files in the working tree with content from the local store, and never overwrites a modified file, so nothing is lost and no ref moves. `git lfs pull` is the two together.

```
$ git lfs install --local
Updated Git hooks.
Git LFS initialized.
$ git lfs fetch
Fetching reference refs/heads/main
$ find .git/lfs/objects -type f
.git/lfs/objects/f8/71/f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
$ wc -c models/acoustic.onnx
  132 models/acoustic.onnx
$ git lfs checkout
Checking out LFS objects: 100% (1/1), 1.3 MB | 0 B/s, done.
$ wc -c models/acoustic.onnx
1250000 models/acoustic.onnx
$ git lfs ls-files
f8719e1ac8 * models/acoustic.onnx
$ git status --short
```

On a machine where the filter is configured before the clone, which is what a global `git lfs install` arranges, none of these commands is needed: the smudge filter downloads each object as the checkout writes each file. The same happens on any later checkout that needs a version that is not in the local store yet:

```
# With the filter configured, a checkout that needs another version downloads it:
$ git switch --detach HEAD~1
HEAD is now at 1690072 Add acoustic model v1
$ git lfs ls-files --size
```

```
0269885262 * models/acoustic.onnx (1.2 MB)
$ find .git/lfs/objects -type f | sort
.git/lfs/objects/02/69/02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
.git/lfs/objects/f8/71/f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
$ git switch -
Previous HEAD position was 1690072 Add acoustic model v1
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

`GIT_LFS_SKIP_SMUDGE`. With the environment variable set to `1`, the smudge filter passes pointers through untouched. That is how to clone or switch quickly when you do not need the large files, and how a CI job avoids downloading them:

```
$ GIT_LFS_SKIP_SMUDGE=1 git switch --detach HEAD~1
HEAD is now at 1690072 Add acoustic model v1
$ cat models/acoustic.onnx
version https://git-lfs.github.com/spec/v1
oid sha256:02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
size 1200000
$ git lfs ls-files
0269885262 - models/acoustic.onnx
$ git status --short
# git lfs pull replaces the pointers of the current commit with content:
$ git lfs pull
$ git lfs ls-files
0269885262 * models/acoustic.onnx
$ git switch -
Previous HEAD position was 1690072 Add acoustic model v1
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
```

`git status` is clean with a pointer in the working tree: the clean filter recognises a pointer and passes it through unchanged, so the file matches the index. `git lfs install --skip-smudge` makes this the permanent behavior of a configuration, and `git lfs pull --include "<paths>"` (or `lfs.fetchinclude` and `lfs.fetchexclude`) fetches a subset.

GitHub Actions, not Git. `actions/checkout` downloads LFS files only when its `lfs` input is `true`; the default is `false` ([action.yml](#)). A job with the default therefore sees pointers, and a job with `lfs: true` downloads objects, which GitHub's billing page counts as bandwidth: "If GitHub Actions downloads a 500 MB file that is tracked with Git LFS, it will use 500 MB of the repository owner's bandwidth" ([Git LFS billing](#)). Not run for this book.

22.8 The local store: `ls-files`, `prune`, `fetch --all`, `fsck`

Every version you have added or checked out stays in `.git/lfs/objects`. `git gc` does not touch it.

```
$ git lfs ls-files --all --size
f8719e1ac8 * models/acoustic.onnx (1.3 MB)
0269885262 - models/acoustic.onnx (1.2 MB)
$ find .git/lfs/objects -type f | sort
.git/lfs/objects/02/69/02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
.git/lfs/objects/f8/71/f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
```

🟡 `git lfs prune` deletes local objects that the current checkout does not need, that are not "recent", and that have been pushed. `--dry-run` `--verbose` `previews`; `--verify-remote` asks the remote whether it really has each object before deleting the local copy:

```
$ git lfs prune --dry-run --verbose
2 local objects, 1 retained, done.

* 02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de (1.2 MB), done.
$ git lfs prune --verify-remote
2 local objects, 1 retained, 1 verified with remote, done.
Deleting objects: 100% (1/1), done.
$ find .git/lfs/objects -type f | sort
.git/lfs/objects/f8/71/f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
```

"Recent" is defined by settings such as `lfs.fetchrecentrefsdays` (7 days) plus `lfs.pruneoffsetdays` (3 days), measured against the real clock. The lab's commits are dated 7 September 2026, so by the time you run this they are never recent; in a repository with fresh commits, `prune` keeps more than this transcript suggests.

A pruned object is not lost as long as the remote has it. 🟢 `git lfs fetch --all` downloads every object referenced by any commit, which is what a backup or a migration to another host needs, and `git lfs fsck` verifies that the files in the local store match their hashes:

```
# The pruned version is still on the remote. Anything that needs it downloads it again:
$ git lfs fetch --all
2 objects found, done.
Fetching all references...
```

```
$ find .git/lfs/objects -type f | sort
.git/lfs/objects/02/69/02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
.git/lfs/objects/f8/71/f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
$ git lfs fsck
Git LFS fsck OK
```

`prune` refuses to delete what exists only on your machine. Two more versions, committed and not pushed:

```
# Two more versions, committed and not pushed. v3 is no longer checked out:
$ git status --short --branch
## main...origin/main [ahead 2]
$ find .git/lfs/objects -type f | sort
.git/lfs/objects/02/69/02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
.git/lfs/objects/ca/75/ca75f5b31e55e3e1e51533c80135181b6d1ea30a82a2c1ab28ddd120c1a8a4f2
.git/lfs/objects/eb/60/eb606cc995d11405ec792073ee6b8668916a61d453a17686d3178c86ffa022bd
.git/lfs/objects/f8/71/f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
$ git lfs prune
4 local objects, 2 retained, done.
Deleting objects: 100% (2/2), done.
$ find .git/lfs/objects -type f | sort
.git/lfs/objects/ca/75/ca75f5b31e55e3e1e51533c80135181b6d1ea30a82a2c1ab28ddd120c1a8a4f2
.git/lfs/objects/eb/60/eb606cc995d11405ec792073ee6b8668916a61d453a17686d3178c86ffa022bd
```

The two pushed versions were deleted; the two unpushed ones stayed, including the one that is not checked out. The help text states one exception that matters after a reset or an amend: "The reflog is not considered, only commits. Therefore LFS objects that are only referenced by orphaned commits are always deleted." A model version that only an abandoned commit refers to is not protected by Git's reflog safety net.

22.9 `git lfs migrate`: files that are already in history

`git lfs track` works for files you have not added yet. For files that are already committed there is `git lfs migrate`, with three modes.

`info` reads history and reports sizes by file extension. It changes nothing:

```
$ git status --short --branch
## main...origin/main [ahead 4]
$ git lfs migrate info
Fetching remote refs: ..., done.
Sorting commits: ..., done.
Examining commits: 100% (4/4), done.
*.onnx 3.8 MB 3/3 files 100%
*.py 429 B 2/2 files 100%
*.md 68 B 1/1 file 100%
*.txt 14 B 1/1 file 100%
$ git lfs migrate info --above=1mb
Fetching remote refs: ..., done.
Sorting commits: ..., done.
Examining commits: 100% (4/4), done.
*.onnx 3.8 MB 3/3 files 100%
```

Note the scope it chose by itself: "Examining commits: 100% (4/4)". By default all three modes look only at the current branch and only at commits that are on no remote. `main` is four commits ahead of `origin/main`, so four commits were examined.

`import` is  a history rewrite: it replaces matching blobs by pointers in every commit in scope, adds the `.gitattributes` line, and gives every rewritten commit and all of its descendants a new ID.

```
$ git log --oneline
8b13104 Retrain acoustic model with accents (v3)
1ddbfc0 Resample input to 16 kHz
65c2188 Retrain acoustic model on noisy audio (v2)
f9e56b8 Add acoustic model v1
c66100a Add transcription entry point
$ git lfs install --local
Updated Git hooks.
Git LFS initialized.
$ git lfs migrate import --include="*.onnx"
Fetching remote refs: ..., done.
Sorting commits: ..., done.
Rewriting commits: 100% (4/4), done.
Updating refs: ..., done.
Checkout: ..., done.
$ git log --oneline
d1e8e7d Retrain acoustic model with accents (v3)
654d53c Resample input to 16 kHz
53cb32a Retrain acoustic model on noisy audio (v2)
b1b921b Add acoustic model v1
c66100a Add transcription entry point
```

The commit that was already on the remote, `c66100a`, kept its ID. The four above it are new commits with the old messages, authors and dates:

```
$ git show --stat --format="%h %s" HEAD~3
b1b921b Add acoustic model v1

.gitattributes      | 1 +
models/acoustic.onnx | 3 +++
2 files changed, 4 insertions(+)
$ cat .gitattributes
*.onnx filter=lfs diff=lfs merge=lfs -text
$ git rev-list --objects main | git cat-file --batch-check="%(<objecttype>) %(<objectsize>) %(<rest>)" | grep onnx
blob 132 models/acoustic.onnx
blob 132 models/acoustic.onnx
blob 132 models/acoustic.onnx
$ git lfs migrate info
Fetching remote refs: ..., done.
Sorting commits: ..., done.
Examining commits: 100% (4/4), done.
*.py          429 B   2/2 files   100%
*.md           68 B   1/1 file    100%
*.gitattributes 43 B   1/1 file    100%
*.txt          14 B   1/1 file    100%

LFS Objects    3.8 MB  3/3 files   100%
```

The branch now reaches three pointer blobs of 132 bytes where it reached three blobs of 1.2 to 1.3 MB, and `migrate info` reports the 3.8 MB as LFS objects.

Two things about the state it leaves surprise people. First, the working tree file is a pointer afterwards, even with the filter installed. The command's own help says this is normal and names the remedy, `git lfs checkout`:

```
$ wc -c models/acoustic.onnx
132 models/acoustic.onnx
$ git lfs ls-files
ca75f5b31e - models/acoustic.onnx
$ git lfs checkout
Checking out LFS objects: 100% (1/1), 1.3 MB | 0 B/s, done.
$ wc -c models/acoustic.onnx
1300000 models/acoustic.onnx
$ git lfs ls-files
ca75f5b31e * models/acoustic.onnx
$ git status --short --branch
## main...origin/main [ahead 4]
```

Second, the repository did not get smaller. The old commits are off the branch and still in the reflog, so the old blobs are still in the object database until the reflog entries expire and a garbage collection runs (Chapter 13). Here the expiry is forced, with the two cut-offs that do not depend on a clock:

```
# The old commits are no longer on any branch, but the reflog still holds them:
$ git reflog -2
d1e8e7d HEAD@{0}:
8b13104 HEAD@{1}: commit: Retrain acoustic model with accents (v3)
$ git cat-file --batch-all-objects --batch-check="%(<objecttype>) %(<objectsize>)" | grep -c -E "blob [0-9]{7}"
3
$ git reflog expire --expire=now --all
$ git gc --quiet --prune=now
$ git cat-file --batch-all-objects --batch-check="%(<objecttype>) %(<objectsize>)" | grep -c -E "blob [0-9]{7}"
0
$ find .git/lfs/objects -type f | sort
.git/lfs/objects/02/69/02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
.git/lfs/objects/ca/75/ca75f5b31e55e3e1e51533c80135181b6d1ea30a82a2c1ab28ddd120c1a8a4f2
.git/lfs/objects/f8/71/f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
```

 `git reflog expire --expire=now --all` followed by `git gc --prune=now` removes the only way back to the pre-migration commits. Run them when you have verified the result, not before. The bytes have not left your disk either: they are in `.git/lfs/objects` now, once per version.

Scope is the dangerous option. The default scope, unpushed commits of the current branch, is the safe one: nothing that anyone else has is rewritten. Name further branches as arguments to include their unpushed commits. `--everything` means every commit reachable from every local and remote ref, including commits that are already on the remote. In Lab 15.4 a second local branch was missed by the first import, and `--everything` looked like the fix:

```
# The other branch was not part of the migration:
$ git rev-list --objects origin/main..experiment/quantized | git cat-file --batch-check="%(<objecttype>) %(<objectsize>) %(<rest>)" |
grep onnx
blob 600000 models/acoustic-int8.onnx
blob 1250000 models/acoustic.onnx
blob 1200000 models/acoustic.onnx
# A flag that promises to catch everything looks like the answer:
$ git lfs migrate import --include="*.onnx" --everything
Sorting commits: ..., done.
Rewriting commits: 100% (8/8), done.
```

```
Updating refs: ..., done.
Checkout: ..., done.
$ git status --short --branch
## main...origin/main [ahead 5, behind 1]
```

```
$ git log --graph --oneline --decorate --all
* f074e3b (experiment/quantized) Add 8-bit quantized acoustic model
| * ca28864 (HEAD -> main) Retrain acoustic model with accents (v3)
| * 106939f Resample input to 16 kHz
|/
* c96fc1c Retrain acoustic model on noisy audio (v2)
* 8a8e13b Add acoustic model v1
* e6c7da2 Add transcription entry point
* c66100a (origin/main, origin/HEAD) Add transcription entry point
$ git show --stat --format="%h %s" $(git rev-list --max-parents=0 main)
e6c7da2 Add transcription entry point

.gitattributes | 1 +
README.md     | 3 +++
models/vocab.txt | 4 ++++
transcribe.py | 12 ++++++++
4 files changed, 20 insertions(+)
```

Observed behavior : After migrate import --everything, main is "ahead 5, behind 1" of origin/main and the log shows the first commit twice, as e6c7da2 and as c66100a.

Git state : origin/main still names c66100a. Local main descends from e6c7da2, a rewritten copy of it. The two histories share no commit.

Mechanism : --everything put the pushed root commit in scope. The .gitattributes line was added in the first commit in scope, which changed its tree and therefore its ID, and the ID of every descendant.

Root cause : A history rewrite that included published commits.

Why Git does this : A commit ID is a hash of the commit's content, parents included (Chapter 6). A different first commit cannot have the same descendants.

Correct fix : Not pushed yet: return both branches to their previous positions from their reflogs and migrate with the branch named instead (Lab 15.4).

Prevention : Run migrate info with the same scope options first and read the commit count. Use --everything only for a planned rewrite of published history.

When the large files are already on the remote, a rewrite of published history is what you need, and it is an operation with a plan, not a command: everyone stops pushing, one person rewrites and pushes with `--force-with-lease`, everyone else re-clones or resets (Chapter 9 for the push, Chapter 21B for the general procedure). Old clones that push again bring the blobs back.

`export` is the reverse rewrite: pointers become ordinary blobs again in the commits in scope. It needs at least one `--include` pattern.

```
$ git lfs migrate export --include="*.onnx"
Fetching remote refs: ..., done.
Sorting commits: ..., done.
Rewriting commits: 100% (4/4), done.
Updating refs: ..., done.
Checkout: ..., done.
3 local objects, 0 retained, done.

ca75f5b31e55e3e1e51533c80135181b6d1ea30a82a2c1ab28ddd120c1a8a4f2 (1.3 MB), done.
Deleting objects: 100% (3/3), done.
```

```
$ git log --oneline
082366e Retrain acoustic model with accents (v3)
a021a3b Resample input to 16 kHz
0b68d28 Retrain acoustic model on noisy audio (v2)
6ecc8cc Add acoustic model v1
c66100a Add transcription entry point
$ cat .gitattributes
*.onnx filter=lfs diff=lfs merge=lfs -text
*.onnx !text !filter !merge !diff
$ git lfs ls-files
$ git rev-list --objects main | git cat-file --batch-check="%(<objecttype> %(<objectsize> %(<rest>)" | grep onnx
blob 1300000 models/acoustic.onnx
blob 1250000 models/acoustic.onnx
blob 1200000 models/acoustic.onnx
$ find .git/lfs/objects -type f | wc -l
0
```

Export does not delete the tracking line; it appends a line that switches the attributes off for the pattern. It ends by pruning the local LFS store, and that is why this demo is volatile: when the prune removes several objects, git-lfs 3.7.1 prints the name of one of them, and not always the same one.

`git lfs migrate import --no-rewrite <files>` converts files in one new commit without touching earlier ones. It is equivalent to the `git add --renormalize` fix of the next section and has the same limit: the old blobs stay in history, so it does not help with a push that is rejected for a large file. (Run for this chapter without a transcript: it refuses a working tree that is not clean unless `--yes` is given.)

22.10 Common errors and their root causes

The file was committed before the pattern was tracked.

```
$ git add samples/hello.wav
$ git commit -m "Add a sample recording"
[main e6d55e2] Add a sample recording
1 file changed, 0 insertions(+), 0 deletions(-)
create mode 100644 samples/hello.wav
$ git lfs track "*.wav"
Tracking "*.wav"
$ git add .gitattributes
$ git commit -m "Track WAV files with Git LFS"
[main 6a6a9f2] Track WAV files with Git LFS
1 file changed, 1 insertion(+)
```

```
$ git lfs ls-files
0269885262 * models/acoustic.onnx
$ git cat-file -s HEAD:samples/hello.wav
48000
$ git status --short
M samples/hello.wav
$ git lfs fsck --pointers
pointer: unexpectedGitObject: "samples/hello.wav" (treeish 6a6a9f24bfc3f0cbac37c4bce72464747916d954) should have been a pointer
but was not
[exit status: 1]
```

Three symptoms of one cause. `git lfs ls-files` does not list the recording. The blob in HEAD is 48,000 bytes, the raw file. And `git status` reports the file as modified although nobody edited it: the attributes now say "clean this path", the clean filter turns the working tree file into a pointer, and the pointer differs from the raw blob in the index. `git lfs fsck --pointers` names the problem. The forward fix is `git add --renormalize`, which runs the path through the filter again and stages the result:

```
# Nothing was pushed with the plain blob as its only form, and a new commit is acceptable:
$ git add --renormalize samples/hello.wav
$ git diff --cached --stat
samples/hello.wav | Bin 48000 -> 130 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
$ git commit -m "Store the sample recording in Git LFS"
[main 365b8a5] Store the sample recording in Git LFS
1 file changed, 0 insertions(+), 0 deletions(-)
$ git lfs ls-files
0269885262 * models/acoustic.onnx
8a2b8ee542 * samples/hello.wav
$ git status --short
```

```
$ git rev-list --objects --all | git cat-file --batch-check="%(<objecttype>) %(<objectsize>) %(<rest>)" | grep wav
blob 130 samples/hello.wav
blob 48000 samples/hello.wav
```

The raw blob is still in the earlier commit. For a 48 KB file that is acceptable. For a file that a host would reject, use `git lfs migrate import` on the unpushed commits instead (Lab 15.3 does).

A commit made without the filter. On a machine with no LFS filter, such as a CI job that regenerates a file or a colleague's fresh laptop, `git add` stores whatever is in the working tree:

```
$ cd ci-transcriber
$ git config get filter.lfs.clean
[exit status: 1]
$ cat .gitattributes
*.onnx filter=lfs diff=lfs merge=lfs -text
$ git commit -am "Retrain acoustic model on noisy audio (v2)"
[main 28ed764] Retrain acoustic model on noisy audio (v2)
1 file changed, 0 insertions(+), 0 deletions(-)
$ git cat-file -s HEAD:models/acoustic.onnx
1250000
$ git push origin main
To $LAB/ch22/lfs-error-not-pointer/remotes/transcriber.git
1690072..28ed764 main -> main
```

The full 1,250,000 bytes went into Git as a blob, on a path that `.gitattributes` assigns to LFS. Everyone who does have the filter gets a warning on their next pull and a file that is modified the moment it is checked out:

```
$ cd ../asha-transcriber
$ git pull
From $LAB/ch22/lfs-error-not-pointer/remotes/transcriber
1690072..28ed764 main -> origin/main
Updating 1690072..28ed764
Fast-forward
```



```
models/acoustic.onnx | Bin 132 -> 1250000 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
Encountered 1 file that should have been a pointer, but wasn't:
    models/acoustic.onnx
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   models/acoustic.onnx

no changes added to commit (use "git add" and/or "git commit -a")
```

```
$ git lfs ls-files
$ git lfs fsck --pointers
pointer: unexpectedGitObject: "models/acoustic.onnx" (treeish 28ed7643fddd4cc06c6c7309d7426f7d09f86d64) should have been a
pointer but was not
[exit status: 1]
$ git diff --stat
models/acoustic.onnx | Bin 1250000 -> 132 bytes
1 file changed, 0 insertions(+), 0 deletions(-)
```

The mechanism is the same as in the first error, reached from the other side. The fix is the same too, and so is its limit: the large blob stays in the history that was pushed.

```
$ git add --renormalize models/acoustic.onnx
$ git commit -m "Store acoustic model v2 as an LFS pointer"
[main 6bbd888] Store acoustic model v2 as an LFS pointer
1 file changed, 0 insertions(+), 0 deletions(-)
$ git lfs ls-files --size
f8719e1ac8 *models/acoustic.onnx (1.3 MB)
$ git status --short
```

Prevention is a server-side or CI check, because the failing machine is by definition one where the local setup is missing: a job that runs `git lfs fsck --pointers` and fails on a non-zero exit status.

The pointer was pushed and the object was not. Here the hook was skipped with `--no-verify`; an interrupted upload or a hooks directory without the LFS hook gives the same state. The pointer is in the remote's Git history; the object exists only in the author's `.git/lfs/objects`. A teammate with a correctly installed client pulls:

```
$ git lfs pull
error transferring "f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65": [0] remote missing object
f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
Failed to fetch some objects from 'file://$LAB/ch22/lfs-error-missing-object-volatile/remotes/transcriber.git'
[exit status: 2]
$ git lfs ls-files
f8719e1ac8 - models/acoustic.onnx
```

```
# The same failure through the smudge filter, when a checkout needs the file:
$ rm models/acoustic.onnx
$ git restore models/acoustic.onnx
Downloading models/acoustic.onnx (1.3 MB)
Error downloading object: models/acoustic.onnx (f8719e1): Smudge error: Error downloading models/acoustic.onnx
(f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65): error transferring
"f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65": [0] remote missing object
f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65

Errors logged to '$LAB/ch22/lfs-error-missing-object-volatile/asha-transcriber/.git/lfs/logs/20261002T103300.970995.log'.
Use `git lfs logs last` to view the log.
error: external filter 'git-lfs filter-process' failed
fatal: models/acoustic.onnx: smudge filter lfs failed
[exit status: 128]
$ git status --short
D models/acoustic.onnx
```

Because the filter is `required`, a failing smudge fails the checkout: the file is absent, not silently wrong. (This demo is volatile because the log file name in the message contains the wall-clock time.) The structure is the one of the unpushed submodule commit in Chapter 23, section 23.8: a reference was published without the thing it refers to.

```
$ git cat-file -p HEAD:models/acoustic.onnx
version https://git-lfs.github.com/spec/v1
oid sha256:f8719e1ac8fb4d5c1d84bffc340e58ce46e7aba4ee423b2d752d6eb7d0d68a65
size 1250000
$ find ../remotes/transcriber.git/lfs/objects -type f | sort
../remotes/transcriber.git/lfs/objects/02/69/02698852622172279f75a9ccc7b8505d411bf4e26c48dba8010e6c447439e8de
```

```
# Whoever still has the object uploads it:
$ cd ../transcriber
# A plain "git lfs push origin main" sends nothing: the commit is already on origin/main,
# so the client assumes its objects are too. --all sends every object the branch references.
$ git lfs push origin main
$ find ../remotes/transcriber.git/lfs/objects -type f | wc -l
1
$ git lfs push --all origin main
Uploading LFS objects: 100% (2/2), 0 B | 0 B/s, done.
$ find ../remotes/transcriber.git/lfs/objects -type f | wc -l
2
$ cd ../asha-transcriber
$ git lfs pull
$ git lfs ls-files
f8719e1ac8 *models/acoustic.onnx
$ git status --short
```

A plain `git lfs push origin main` uploads nothing, because the client skips objects of commits that the remote-tracking branch already contains. `git lfs push --all origin main` sends every object the branch references. If nobody has the object any more, the pointer is permanently dangling, and the only repair is a new commit with a file that exists.

22.11 GitHub: file limits and metered LFS billing

GitHub, not Git. Everything in this section is platform behavior, quoted from GitHub's documentation as read on 2 October 2026. None of it can be reproduced in the lab.

Without LFS. "If you attempt to add or update a file that is larger than 50 MiB, you will receive a warning from Git", and "GitHub blocks files larger than 100 MiB". A file added through the browser "can be no larger than 25 MiB". Repositories should "remain small, ideally less than 1 GB, and less than 5 GB is strongly recommended" ([about large files](#)). The Phase 0 report adds a cap of 2 GB for a single push ([repository limits](#)). The block applies to any commit in the push that contains such a blob, which is why deleting the file in a later commit does not help (section 22.2).

With LFS. The maximum size of one LFS file depends on the plan: 2 GB on GitHub Free and Pro, 4 GB on Team, 5 GB on Enterprise Cloud. LFS "cannot be used with GitHub Pages sites" and "cannot be used with template repositories" ([about Git LFS](#)).

Billing ([Git LFS billing](#)):

Fact	Consequence
Included per month: 10 GiB of storage and 10 GiB of bandwidth on Free and Pro; 250 GiB of each on Team and Enterprise Cloud	Beyond that, usage is metered and billed
"Previously, Git LFS billing used pre-paid data packs. These have been removed and replaced with metered billing."	Advice to "buy a data pack" is out of date
Every pushed version is stored in full: "If you make a 1 byte change and push the file again, you'll use another 500 MB of storage"	Storage grows with the number of versions, not with the size of the latest one
Downloads use bandwidth; uploads do not. Downloads by GitHub Actions count	A CI matrix that pulls the same model in every job multiplies bandwidth
"Bandwidth and storage usage always count against the repository owner's account"; pushes to forks count "against the parent repository's bandwidth and storage quotas"	The owner pays for what contributors and forks do
When the quota is used up and no payment method is on file, clones "will only retrieve the pointer files" and you "will not be able to push new files back up"	An exhausted quota looks exactly like a machine without the client
After removing files from LFS, "the Git LFS objects still exist on the remote storage and will continue to count toward your Git LFS storage quota"; to remove them, "delete and recreate the repository", or contact Support (removing files)	Question 4 of section 22.1: storage does not shrink when history does

Unverified. Per-GiB prices for metered LFS usage are not stated on the billing page that was read, and the Phase 0 report records a conflict between GitHub's pricing page and its billing documentation about data packs. This book gives no prices.

In a pull request, GitHub "does not render some Git LFS objects"; reviewers see the pointer ([collaboration with Git LFS](#)).

22.12 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A program fails to parse a file that is about 130 bytes and starts with <code>version https://git-lfs...</code>	<code>git lfs ls-files</code> shows <code>-</code> ; <code>git config get filter.lfs.smudge</code> prints nothing	<code>git lfs install</code> (in this book: <code>--local</code>), then <code>git lfs pull</code>	Install the client in images and CI; in Actions, <code>lfs: true</code> on checkout

Symptom	Diagnosis	Fix	Prevention
<code>git lfs ls-files</code> does not list a file that should be in LFS; <code>git status</code> shows it modified right after checkout; or the warning "Encountered N file(s) that should have been a pointer, but wasn't"	<code>git lfs fsck --pointers</code> ; <code>git cat-file -s HEAD:<path></code> is the full size	<code>git add --renormalize <path></code> and commit; or <code>migrate import</code> if unpushed	Track before adding; a CI job running <code>git lfs fsck --pointers</code>
Smudge error, "remote missing object", <code>smudge filter lfs failed</code>	Compare the oid in <code>git cat-file -p HEAD:<path></code> with the remote's object store	Whoever has the object: <code>git lfs push --all <remote> <branch></code>	Never <code>--no-verify</code> in LFS repositories; keep the pre-push hook when adopting a hook manager
"This repository is configured for Git LFS but 'git-lfs' was not found on your path"	The hooks exist, the program is not installed	Install <code>git-lfs</code> ; or delete the hooks if LFS is no longer used	Provision the client with the toolchain
A push is still rejected for a large file after <code>git lfs track</code> and a new commit	<code>git rev-list --objects <remote>/<branch>.. <branch></code> piped to <code>git cat-file --batch-check</code> still lists the big blob	<code>git lfs migrate import</code> on the unpushed commits	<code>migrate info</code> before the first push of new binary types
After <code>migrate import</code> , the branch is "ahead N, behind M"	<code>--everything</code> rewrote published commits (22.9)	Restore from the branch reflogs; migrate with named branches	Read the commit count that <code>migrate info</code> prints for the same scope
Merge conflict markers inside a pointer file	Two branches changed the same LFS file	<code>git checkout --ours</code> or <code>--theirs <path></code> , <code>git add</code>	Locking on a server that supports it; one owner per binary
LFS storage on GitHub grows although files were deleted	Objects stay until the repository is deleted or Support purges them	See 22.11	Decide what belongs in LFS before pushing it

22.13 When not to use it, and what to use instead

LFS fits a modest number of mid-sized binary files that change rarely and must be versioned together with the code: reference checkpoints for tests, small fixtures, images and design assets. The Phase 0 report's assessment of where it is the wrong tool, with the alternatives it lists:

Situation	Why LFS is wrong	Use instead
Files larger than the plan's per-file cap (2 to 5 GB on GitHub)	The upload is refused	Object storage or a model registry; keep the URI and a checksum in Git
Training data and model weights that change with every run	Each version is stored whole and billed; nothing is ever freed	DVC, which keeps small metafiles in Git and the data in a remote you control; or a model registry, with the Git commit recorded beside each model version
Data that must be deletable (personal data, licensed data)	Removing a file from LFS on GitHub does not delete the object	Storage with retention and deletion controls
Models and datasets published for others	Consumers need the client, and your bandwidth pays for their downloads	The Hugging Face Hub, pinning the <code>revision</code> you download; or GitHub Releases for binaries that accompany a tag
Contributors or CI that cannot be relied on to have the client	Pointer files in place of content, silently	Any of the above; or a download step with a checksum

The versions, licences and stewardship of these tools changed during 2025 and 2026; Chapter 28 gives the current facts with their sources.

Dangerous edge cases:

- **Rewriting published history with** `migrate import --everything`, or any `migrate` followed by a forced push, changes commit IDs for everyone (22.9).
- `git lfs prune` **ignores the reflog**. An LFS object referenced only by a commit you reset away from can be deleted locally although `git reflog` still shows the commit (22.8).

22.14 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git lfs ls-files</code> , <code>status</code> , <code>env</code> , <code>fsck</code> , <code>pointer --file</code> , <code>migrate info</code> , <code>track</code> (no argument)	● SAFE	Nothing (<code>fsck</code> moves corrupt local objects aside)	not needed	not needed
<code>git lfs install --local</code>	● SAFE	<code>filter.lfs.*</code> in <code>.git/config</code> ; four hooks in the hooks directory. Without <code>--local</code> the filter keys go into your global configuration (●)	<code>git config list --show-origins</code>	<code>git lfs uninstall --local</code>
<code>git lfs track "<pattern>"</code> , <code>untrack</code>	● SAFE	A line in <code>.gitattributes</code>	<code>git lfs track --dry-run</code>	Edit the file

Command	Label	What it changes	Preview	Recovery
<code>git lfs fetch</code> , <code>fetch --all</code> , <code>pull</code> , <code>checkout</code>	● SAFE	<code>fetch</code> downloads into <code>.git/lfs/objects</code> ; <code>checkout</code> and <code>pull</code> replace pointer files in the working tree and never overwrite modified files	<code>git lfs fetch --dry-run</code> ; <code>git lfs ls-files</code>	<code>git lfs prune</code> for the downloads
<code>git lfs push</code> , and <code>git push</code> with the hook	● CAUTION	Uploads objects; on GitHub they are billed and cannot be deleted by you	<code>git lfs push --dry-run <remote> <ref></code> ; <code>git lfs status</code>	None on GitHub short of deleting the repository (22.11)
<code>git lfs prune</code>	● CAUTION	Deletes local LFS objects that are pushed and not needed	<code>--dry-run --verbose</code> ; <code>--verify-remote</code>	<code>git lfs fetch</code> , if the remote has them
<code>git lfs migrate import</code> , <code>export</code> (default scope or named branches)	● CAUTION	Rewrites unpushed commits; working tree left with pointers after import	<code>migrate info</code> with the same options	Branch reflogs: <code>git reset --hard <branch>@{1}</code>
<code>git lfs migrate import --everything</code> , then a forced push	● DANGEROUS	Rewrites published history; destroys nothing locally, breaks every other clone	<code>migrate info --everything</code> ; rehearse in a fresh clone	Before the push: reflogs. After: the old history from another clone
<code>git reflog expire --expire=now --all</code> and <code>git gc --prune=now</code>	● DANGEROUS	Deletes unreachable commits and the reflog that led to them	<code>git reflog</code> ; <code>git fsck --unreachable</code>	None

22.15 Version notes

Version note. Older behavior: GitHub sold prepaid LFS "data packs". Current behavior: metered billing with an included monthly quota per plan. Since: the billing documentation says the packs "have been removed"; the Phase 0 report dates the change to the enhanced billing platform of February and March 2025 ([changelog](#)). Recommended: budget by stored versions and by downloads, including CI.

- The client used here is git-lfs 3.7.1 (17 October 2025). The newest release is 3.8.0 (28 August 2026), which adds zstd-compressed object downloads and requires macOS 13 or later ([releases](#)); the report records no security content in it.
- The help text of git-lfs 3.7.1 states that with Git 2.42.0 or later, `git lfs pull` and `git lfs checkout` act only on files that are in the index and match an LFS filter attribute, which matters in sparse or partial clones.

Outdated advice. "Use `git lfs clone`, it is faster than `git clone`." The installed client prints a warning when the command is run: "`git lfs clone` is deprecated and will not be updated with new flags from `git clone`", and adds that `git clone` "has been updated in upstream Git to have comparable speeds". Use `git clone`.

Unverified. The git-lfs release that introduced the built-in `lfs-standalone-file` adapter is not stated in the documentation that was read. It works in 3.7.1, as the transcripts show.

22.16 Practice

- Lab 15.3 (inspect an LFS pointer) and Lab 15.4 (migrate an already-committed large file) in the Module 15 lab manual.
- Replay any transcript with `labs/run ch22/<demo>`, for example `labs/run ch22/lfs-clone`.
- A drill: in the sandbox of `ch22/lfs-pointer`, compute by hand what the pointer of `models/vocab.txt` would be if you tracked `*.txt` (`shasum -a 256, wc -c`), then check with `git lfs pointer --file`.

22.17 Interview questions

- Why does deleting a large file in a new commit not make a rejected push succeed?
- What does Git store for an LFS-tracked path, what does LFS store, and where is each on your machine and on the remote?
- Which two Git mechanisms does LFS use, and which parts of the setup travel with a clone?
- A CI job fails because a model file is 132 bytes. Give the root cause, two diagnostics, and the fix for GitHub Actions.
- `git status` shows a tracked binary as modified immediately after a fresh checkout. Explain the mechanism and the two possible histories that lead to it.
- A teammate gets "Smudge error ... remote missing object". What happened, who can fix it, and with which command? Why does a plain `git lfs push` not do it?
- What does `git lfs migrate import` do to commit IDs, to the working tree, and to the size of `.git`? What is the default scope, and what does `--everything` add?
- Your LFS bill grows although nobody adds models. Name three mechanisms that can cause it.
- When is LFS the wrong tool for model weights, and what would you use?

10. How does `git lfs prune` decide what it may delete, and in which case does Git's reflog not protect you?

22.18 Sources

Primary sources

- Git LFS: the [specification](#), [server discovery](#) and [custom transfers](#) in the project's repository, read on 2 October 2026; the help texts of the installed client 3.7.1 (`git lfs <command> --help`), which the transcripts were checked against; [releases](#).
- GitHub Docs, read on 2 October 2026: [About large files on GitHub](#), [About Git Large File Storage](#), [Git LFS billing](#), [Collaboration with Git LFS](#), [Removing files from Git LFS](#), [Repository limits](#).
- [gitattributes](#) for filter drivers; `actions/checkout` [action.yml](#) at v7.0.1.

Secondary sources

- The Phase 0 report of this course: section 2 (the large-files paragraph), section 13 ("Scale, submodules and large files") and section 15 ("Data and model versioning"), with the research notes on AI/ML workflows.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- glich.stream, "[Chad level git: advanced concepts \(2025\)](#)" (1 h 14 min): hands-on, includes Git LFS. Current.
- Vikash Das, "[MLOps: Day 4 - Data Versioning using DVC](#)" (1 h 25 min, 2024, Hindi): why Git alone does not suit data, and DVC beside Git. Mostly current; DVC CLI versions not checked.

Further reading

- Chapter 28 for data and model versioning beyond LFS; Chapter 26 for partial clone, which is Git's own way of not downloading every blob.

Chapter 23: Submodules and Subtrees

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch23/`.

23.1 Why this matters

Four reports from one week on a team that vendors a shared library:

1. "I cloned the service and the `vendor/textsplit` directory is empty. The imports fail."
2. "`git pull` worked for everyone yesterday. Today it dies with `not our ref`."
3. "I pulled, changed one line in `ingest.py`, committed, and the reviewer says my commit downgrades the library."
4. "CI is green on my branch and red on `main`, and the only difference is a directory I never touched."

A CTO who hears these asks two things: why does this mechanism fail so often, and should we be using it at all? Both questions have precise answers, and both start from one fact that the Phase 0 report puts in a sentence: **the superproject records a commit ID, not a branch, and Git's defaults do not keep the nested checkout in sync with that record**. Every standard failure in this chapter is a gap between three things that should agree: the commit ID recorded in the superproject, the commit checked out in the submodule's working tree, and the commits that exist in the submodule's shared repository.

The projects: `doc-qa`, a document question-answering service owned by your team, and `textsplit`, a small text-chunking library that Asha maintains in a repository of its own. Ravi is your teammate on `doc-qa`. The second half of the chapter solves the same problem with a subtree and compares both with a package manager.

23.2 What a submodule is: a gitlink, `.gitmodules`, and a repository under `.git/modules`

In one sentence. A submodule is another repository checked out in a subdirectory of yours, of which your repository records exactly one thing in its history: the ID of the commit that should be checked out there.

Analogy. A recipe that says "use the stock from page 212 of the other book, third printing". Your book does not contain the stock recipe; it contains a precise reference to one version of it. Anyone cooking from your book needs the other book as well, opened at that page. Where the analogy breaks: the other book keeps being reprinted, your reference does not update itself, and the reader can scribble in their copy of the other book without your book noticing until they compare.

Precisely. Three pieces, in three different places ([gitsubmodules](#)):

Piece	Where	Versioned in the superproject?	What it holds
The gitlink	A tree entry (and index entry) with mode <code>160000</code> at the submodule path	Yes	The commit ID the superproject expects there
<code>.gitmodules</code>	A file at the top of the superproject's working tree	Yes	For each submodule: a name, the path, the URL to clone from, optional settings such as <code>branch</code>
The submodule's repository	<code>.git/modules/<name>/</code> inside the superproject's Git directory, plus a <code>.git</code> file in the submodule's working tree that points to it	No	The submodule's own objects, refs, HEAD, index, configuration

A fourth, local piece connects them: `git submodule init` copies the URL from `.gitmodules` into the superproject's `.git/config` as `submodule.<name>.url`. From then on Git uses the copy. That is why a changed URL does not reach existing clones by itself (section 23.11).

Chapter 3 listed mode `160000` among the five entry modes of a tree. This chapter is about what that entry does not contain.

See it. In your clone of `doc-qa`, add the library. The option before `submodule` is explained in section 23.4; the lab remotes are directories, and Git is cautious about those.

```
$ git -c protocol.file.allow=always submodule add ../textsplit.git vendor/textsplit
Cloning into 'LAB/ch23/submodule-add/doc-qa/vendor/textsplit'...
done.
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
    new file:   .gitmodules
    new file:   vendor/textsplit
```

The command cloned the library into `vendor/textsplit` and staged two things. First, `.gitmodules` and the copy of the URL in the local configuration:

```
$ cat .gitmodules
[submodule "vendor/textsplit"]
    path = vendor/textsplit
    url = ../textsplit.git
$ git config list --local | grep ^submodule
submodule.vendor/textsplit.url=$LAB/ch23/submodule-add/remotes/textsplit.git
submodule.vendor/textsplit.active=true
```

The versioned file holds the relative URL `../textsplit.git`. A URL that starts with `./` or `../` is resolved against the superproject's own remote, here `remotes/doc-qa.git`, which gives its sibling `remotes/textsplit.git`. The local configuration holds the resolved, absolute result. Relative URLs are worth using when both repositories live on the same host: a clone made over HTTPS fetches the submodule over HTTPS, a clone made over SSH uses SSH, and no protocol is written into a versioned file.

Second, the gitlink in the index:

```
$ git ls-files --stage
100644 9f740c3c709ae97192a6f851e624b01da96b7224 0      .gitmodules
100644 09f9084395224cf2f392b6a549a62c2cdc28d7dd 0      README.md
100644 b5c3d8b6ef195753d164433b23610b16e4fbd62 0      answer.py
100644 eb72b3ec2e343a1cc0d789f693ab705990af7f35 0      ingest.py
160000 e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 0      vendor/textsplit
$ git -C vendor/textsplit log --oneline --decorate -1
e216665 (HEAD -> main, origin/main, origin/HEAD) Add overlap between neighbouring chunks
```

Four blobs with mode `100644` and one entry with mode `160000`. Its ID, `e216665`, is the commit that the library's `main` pointed at when you ran the command. Nothing in the entry says "main".

```
$ cat vendor/textsplit/.git
gitdir: ../../.git/modules/vendor/textsplit
$ ls .git/modules/vendor/textsplit
config
description
HEAD
hooks
index
info
logs
objects
packed-refs
refs
$ git -C vendor/textsplit rev-parse --git-dir
$LAB/ch23/submodule-add/doc-qa/.git/modules/vendor/textsplit
$ git -C vendor/textsplit config get core.worktree
../../../../../vendor/textsplit
```

The submodule's working tree has a `.git` file, the same device a linked worktree uses (Chapter 25), and the real Git directory sits under the superproject's `.git/modules/`. Keeping it there means the repository survives when the working tree directory disappears, for example when you switch to a commit of the superproject from before the submodule existed.

```
$ git diff --cached -- vendor/textsplit
diff --git a/vendor/textsplit b/vendor/textsplit
new file mode 160000
index 0000000..e216665
--- /dev/null
+++ b/vendor/textsplit
@@ -0,0 +1 @@
+Subproject commit e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4
```

To the superproject a submodule is a one-line file whose content is `Subproject commit <ID>`. That is how every later change to it appears in diffs and in review.

```
$ git commit -m "Vendor textsplit as a submodule"
[main bf751ba] Vendor textsplit as a submodule
2 files changed, 4 insertions(+)
create mode 100644 .gitmodules
create mode 160000 vendor/textsplit
$ git ls-tree HEAD
100644 blob 9f740c3c709ae97192a6f851e624b01da96b7224      .gitmodules
100644 blob 09f9084395224cf2f392b6a549a62c2cdc28d7dd      README.md
100644 blob b5c3d8b6ef195753d164433b23610b16e4fbd62      answer.py
100644 blob eb72b3ec2e343a1cc0d789f693ab705990af7f35      ingest.py
040000 tree 341fc987ffe5b926508788b966cbd3b8e0f19927      vendor
$ git ls-tree HEAD vendor/
160000 commit e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4      vendor/textsplit
$ git submodule status
e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit (v0.1.0-1-ge216665)
```

`git ls-tree` prints the type `commit` for the entry. `git submodule status` prints the checked-out commit, the path, and the output of `git describe` for it. And the point that the rest of the chapter depends on:

```
# The tree entry names a commit, but the commit object is not in this repository:
$ git cat-file -t HEAD:vendor/textsplit
fatal: git cat-file: could not get object info
[exit status: 128]
# It lives in the object database of the submodule:
$ git -C vendor/textsplit cat-file -t HEAD
commit
$ git -C vendor/textsplit rev-parse HEAD
e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4
```

The superproject's object database does not contain commit `e216665`. A tree entry of mode `160000` is the only place in Git where an object ID names something that the repository is not required to have. Git does not check that the ID exists anywhere when you commit it or when you push it. Everything that makes the entry useful (where to get the object, whether it has been fetched, whether it is checked out) is arranged outside the commit.

Picture.

```
doc-qa (superproject)                                textsplit (its own repository)
+-----+-----+                                     +-----+
| commit bf751ba                                     | ceaafef1 --- e216665  main |
| tree                                               | (v0.1.0)               |
| 100644 blob .gitmodules ----- url -->| remotes/textsplit.git |
| 100644 blob ingest.py                           |                       |
| 040000 tree vendor                               |                       |
| 160000 commit e216665 textsplit ----- "check this out" -----+
| .git/config      submodule.vendor/textsplit.url = <absolute URL> (local, not versioned)
| .git/modules/vendor/textsplit/      the clone of textsplit (local, not versioned)
| vendor/textsplit/.git (file) -> ../../.git/modules/vendor/textsplit
+-----+-----+                                     +-----+
```

In production. A team keeps prompt templates, a tokenizer vocabulary or protocol definitions in one repository that five services consume. A submodule gives each service an exact, reviewable pin: the pull request that moves the pin shows `-Subproject commit` and `+Subproject commit`, and a release of the service is reproducible from one commit ID of the superproject. The price is the set of failures below.

State table for `git submodule add <url> <path>`:

Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code><path>/</code> created and populated; <code>.gitmodules</code> created or extended	<code>.gitmodules</code> and a gitlink staged	unchanged	unchanged	<code>.git/modules/<name>/</code> created (a clone); <code>submodule.<name>.url</code> and <code>.active</code> in <code>.git/config</code>	unchanged (a fetch from the submodule's remote)	unchanged

23.3 What a teammate receives: clone, init, update

A plain clone transfers the superproject's objects. The gitlink arrives; the commit it names does not.

```
$ git clone remotes/doc-qa.git ravi-doc-qa
Cloning into 'ravi-doc-qa'...
done.
$ cd ravi-doc-qa
$ ls -A vendor/textsplit
$ git status
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
$ git submodule status
-e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit
```

Read the three outputs together. The directory exists and is empty. `git status` says "nothing to commit, working tree clean": an uninitialized submodule is not a modification, so nothing warns Ravi. Only `git submodule status` tells him, with the leading `-`, that the submodule is not initialized. This is report 1 from section 23.1.

Two steps fill the directory.  `git submodule init` registers the submodule: it copies the URL into `.git/config`.

```
$ git config get submodule.vendor/textsplit.url
[exit status: 1]
$ git submodule init
Submodule 'vendor/textsplit' ($LAB/ch23/submodule-clone/remotes/textsplit.git) registered for path 'vendor/textsplit'
$ git config list --local | grep ^submodule
```



```
submodule.vendor/textsplit.active=true
submodule.vendor/textsplit.url=$LAB/ch23/submodule-clone/remotes/textsplit.git
```

git submodule update then clones the repository into .git/modules/, and checks out the recorded commit:

```
$ git -c protocol.file.allow=always submodule update
Cloning into 'LAB/ch23/submodule-clone/ravi-doc-qa/vendor/textsplit'...
done.
Submodule path 'vendor/textsplit': checked out 'e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4'
$ ls -A vendor/textsplit
.git
README.md
splitter.py
$ git submodule status
e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit (v0.1.0-1-ge216665)
```

git submodule update --init does both steps, and --recursive repeats them for submodules inside submodules. For a fresh clone, one command does everything:

```
$ git -c protocol.file.allow=always clone --recurse-submodules remotes/doc-qa.git asha-doc-qa
Cloning into 'asha-doc-qa'...
done.
Submodule 'vendor/textsplit' ($LAB/ch23/submodule-clone/remotes/textsplit.git) registered for path 'vendor/textsplit'
Cloning into 'LAB/ch23/submodule-clone/asha-doc-qa/vendor/textsplit'...
done.
Submodule path 'vendor/textsplit': checked out 'e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4'
$ git -C asha-doc-qa submodule status
e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit (v0.1.0-1-ge216665)
```

The table of prefixes that git submodule status prints is worth memorising, because it is the fastest diagnosis in this chapter:

Prefix	Meaning
(space)	Initialized, and the checked-out commit equals the recorded one
-	Not initialized
+	The checked-out commit differs from the one recorded in the superproject's index
U	The submodule path has a merge conflict in the superproject

23.4 Why the lab needs protocol.file.allow, and what that says about security

Without the extra option, the very first command of this chapter fails:

```
$ git remote -v
origin $LAB/ch23/submodule-add/remotes/doc-qa.git (fetch)
origin $LAB/ch23/submodule-add/remotes/doc-qa.git (push)
$ git submodule add ../textsplit.git vendor/textsplit
Cloning into '$LAB/ch23/submodule-add/doc-qa/vendor/textsplit'...
fatal: transport 'file' not allowed
fatal: clone of '$LAB/ch23/submodule-add/remotes/textsplit.git' into submodule path '$LAB/ch23/submodule-add/doc-qa/vendor/textsplit' failed
[exit status: 128]
```

So does a recursive clone, at the moment it starts cloning the submodule by itself:

```
$ cd ..
$ git clone --recurse-submodules remotes/doc-qa.git asha-doc-qa
Cloning into 'asha-doc-qa'...
done.
Submodule 'vendor/textsplit' ($LAB/ch23/submodule-clone/remotes/textsplit.git) registered for path 'vendor/textsplit'
Cloning into '$LAB/ch23/submodule-clone/asha-doc-qa/vendor/textsplit'...
fatal: transport 'file' not allowed
fatal: clone of '$LAB/ch23/submodule-clone/remotes/textsplit.git' into submodule path '$LAB/ch23/submodule-clone/asha-doc-qa/vendor/textsplit' failed
Failed to clone 'vendor/textsplit'. Retry scheduled
Cloning into '$LAB/ch23/submodule-clone/asha-doc-qa/vendor/textsplit'...
fatal: transport 'file' not allowed
fatal: clone of '$LAB/ch23/submodule-clone/remotes/textsplit.git' into submodule path '$LAB/ch23/submodule-clone/asha-doc-qa/vendor/textsplit' failed
Failed to clone 'vendor/textsplit' a second time, aborting
[exit status: 1]
```

```
# The clone of the superproject worked; only the clone that Git started by itself was refused.
$ git -C asha-doc-qa submodule status
-e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit
# The default policy for the file transport is "user". This variable is how Git marks
# a transfer that the user did not type:
$ GIT_PROTOCOL_FROM_USER=0 git clone remotes/textsplit.git probe
```

```
Cloning into 'probe'...
fatal: transport 'file' not allowed
[exit status: 128]
```

Mechanism. Git has a per-transport policy, `protocol.<name>.allow`, with three values: `always`, `never` and `user`. The local manual gives the defaults: `http`, `https`, `git` and `ssh` are `always`; `ext` is `never`; everything else, the `file` transport included, is `user`. Under `user` a transport may be used only when the environment variable `GIT_PROTOCOL_FROM_USER` is unset or `1`, that is, when a person typed the clone, fetch or push. The `git submodule` command sets the variable to `0` before it does anything (it is the second statement of the `git-submodule` script in Git's exec path), because the URLs it will use come from `.gitmodules`, a file written by whoever authored the repository. A recursive clone runs the same code. The second half of the last transcript reproduces the refusal with an ordinary `git clone` and that variable.

Why the default is what it is. The value `user` for `file` dates from the fix for CVE-2022-39253, shipped in Git 2.38.1 and the maintenance releases of the same day. The release notes describe two changes: local clones no longer dereference symbolic links, and "the value of `protocol.file.allow` is changed to be "user" by default" ([release notes 2.38.1](#), [protocol configuration](#)). The idea behind the policy is in the manual's wording: a protocol that is fine when you name it yourself should not be usable "by commands which execute clone/fetch/push commands without user input, e.g. recursive submodule initialization".

What the lab does. The lab's "servers" are bare repositories in a directory, so the submodule URL is a file path. Each command that makes Git clone or fetch a submodule on its own initiative is therefore run with `-c protocol.file.allow=always`, for that one command. Nothing is written to any configuration file. With a real remote over HTTPS or SSH you never need the option.

Outdated advice. "Run `git config --global protocol.file.allow always` to fix submodule errors." That switches a security default off for every repository you will ever clone. Use `-c` for a single command on repositories you created yourself, and leave the setting alone otherwise. The Phase 0 report's rule set says the same.

The wider security note. A recursive clone is the one common Git operation in which a repository you have not inspected decides what else gets cloned and where it is written. That is where the recent client vulnerabilities have been:

CVE	Fixed in	What a recursive clone of a hostile repository could do
CVE-2024-32002 (critical)	2.45.1 and backports, May 2024	On a case-insensitive filesystem with symbolic links (the default on macOS), write a hook into <code>.git/</code> and run it during the clone (advisory)
CVE-2025-48384 (high)	2.50.1 and backports, July 2025	Through a submodule path ending in a carriage return, check content out to an unintended location where a hook could run (advisory)

Both are fixed in the Git 2.55.0 this book uses and in Apple's 2.50.1. The second advisory's own workaround is the durable rule: do not recursively clone submodules of repositories you do not trust. Clone without recursion, read `.gitmodules`, and then decide. `git submodule init` also refuses, for the same reason, to copy a `submodule.<name>.update` setting that is a custom command from `.gitmodules` into your configuration.

23.5 The detached HEAD after `update`, and why it is correct

```
$ git -C vendor/textsplit status
HEAD detached at e216665
nothing to commit, working tree clean
$ git -C vendor/textsplit branch --all
* (HEAD detached at e216665)
  main
remotes/origin/HEAD -> origin/main
remotes/origin/main
```

Why. The superproject asked for commit `e216665`. It did not ask for a branch, and it cannot: the gitlink has no room for one. `git submodule update` therefore does what `git switch --detach <commit>` does. A local `main` exists in the submodule because the clone created it; HEAD is not on it. This is the detached HEAD of Chapter 7, reached by one more route, and it is the correct state for a dependency you only consume.

It becomes a trap the moment you edit the library in place. Ravi fixes something inside `vendor/textsplit` and commits:

```
$ cd vendor/textsplit
$ git status --short --branch
## HEAD (no branch)
$ git commit -am "Add line splitter"
[detached HEAD 679aa20] Add line splitter
 1 file changed, 5 insertions(+)
$ git log --oneline --decorate -2
679aa20 (HEAD) Add line splitter
e216665 (origin/main, origin/HEAD, main) Add overlap between neighbouring chunks
$ cd ../..
```

The commit `679aa20` is on no branch. Meanwhile you have moved the pointer to the library's 0.2.0 release and pushed. Ravi pulls and updates:

```
$ git pull
From $LAB/ch23/submodule-detached/remotes/doc-qa
 907dbd3..3719d28  main    -> origin/main
Fetching submodule vendor/textsplit
From $LAB/ch23/submodule-detached/remotes/textsplit
 e216665..b3c86ce  main    -> origin/main
* [new tag]         v0.2.0    -> v0.2.0
Updating 907dbd3..3719d28
Fast-forward
 vendor/textsplit | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git submodule update
Submodule path 'vendor/textsplit': checked out 'b3c86ceee1901d9cfff1a21920bd863a64ffb738'
$ git -C vendor/textsplit log --oneline --decorate -2
b3c86ce (HEAD, tag: v0.2.0, origin/main, origin/HEAD) Reject an overlap that is not smaller than the chunk size
e216665 (main) Add overlap between neighbouring chunks
# Which branch, local or remote-tracking, contains the commit Ravi made? None:
$ git -C vendor/textsplit branch --all --contains 679aa20
```

`git submodule update` checked out the commit that the superproject now records and said nothing about the commit it left behind. No branch contains it. It is not lost: the submodule is a repository with its own reflog (Chapter 13).

```
# No branch ever pointed at the commit. The submodule has a reflog of its own:
$ git -C vendor/textsplit reflog -3
b3c86ce HEAD@{0}: checkout: moving from 679aa200d3e14163fc2fc5764cdd69de4c189dbd to b3c86ceee1901d9cfff1a21920bd863a64ffb738
679aa20 HEAD@{1}: commit: Add line splitter
e216665 HEAD@{2}: checkout: moving from main to e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4
$ git -C vendor/textsplit branch line-splitter 'HEAD@{1}'
$ git -C vendor/textsplit log --oneline --decorate -1 line-splitter
679aa20 (line-splitter) Add line splitter
```

The prevention is a habit: before you change anything inside a submodule, get on a branch there.

```
# The way to work in a submodule: get on a branch first.
$ git -C vendor/textsplit switch line-splitter
Previous HEAD position was b3c86ce Reject an overlap that is not smaller than the chunk size
Switched to branch 'line-splitter'
$ git -C vendor/textsplit status --short --branch
## line-splitter
$ git status --short
 M vendor/textsplit
$ git submodule status
+679aa200d3e14163fc2fc5764cdd69de4c189dbd vendor/textsplit (v0.1.0-2-g679aa20)
```

With the submodule on a branch, `git submodule update --merge` or `--rebase` integrates the recorded commit into that branch and leaves HEAD attached; `submodule.<name>.update` makes either the default for one submodule. They are for people who develop the library inside the superproject. For consumers, the default checkout is right.

23.6 Moving the pointer: `update --remote`, `status`, `diff`, `commit`

Asha has released textsplit 0.2.0. Nothing changes in `doc-qa` until someone moves the gitlink. 🟡 `git submodule update --remote` fetches in the submodule and checks out the tip of its remote-tracking branch instead of the recorded commit:

```
$ git submodule status
e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit (v0.1.0-1-ge216665)
$ git -c protocol.file.allow=always submodule update --remote
From $LAB/ch23/submodule-update/remotes/textsplit
 e216665..b3c86ce  main    -> origin/main
* [new tag]         v0.2.0    -> v0.2.0
Submodule path 'vendor/textsplit': checked out 'b3c86ceee1901d9cfff1a21920bd863a64ffb738'
$ git submodule status
+b3c86ceee1901d9cfff1a21920bd863a64ffb738 vendor/textsplit (v0.2.0)
```

Which branch? By default the one the remote's HEAD points at. `git submodule set-branch --branch <name> <path>` records another in `.gitmodules` as `submodule.<name>.branch`, and `git submodule add -b <name>` does so at creation. That setting is read by `update --remote` and by nothing else: it does not make the submodule "follow" a branch, and the gitlink is still a commit ID.

The submodule's working tree is now at `b3c86ce`, the superproject still records `e216665`, and `git submodule status` marks the difference with `+`. The superproject sees a modified one-line file:

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
```

```
modified: vendor/textsplit (new commits)

no changes added to commit (use "git add" and/or "git commit -a")
$ git diff
diff --git a/vendor/textsplit b/vendor/textsplit
index e216665..b3c86ce 160000
--- a/vendor/textsplit
+++ b/vendor/textsplit
@@ -1,1 @@
-Subproject commit e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4
+Subproject commit b3c86ceee1901d9cfff1a21920bd863a64ffb738
$ git diff --submodule=log
Submodule vendor/textsplit e216665..b3c86ce:
    > Reject an overlap that is not smaller than the chunk size
```

(new commits) in `git status` means "the submodule's HEAD differs from the gitlink in the index". `git diff --submodule=log` replaces the two IDs by the list of commits between them, which is what a reviewer wants to read; `git config set diff.submodule log` makes it the default.

Recording the new pointer is an ordinary add and commit:

```
$ git add vendor/textsplit
$ git commit -m "Update textsplit to 0.2.0"
[main 0d51dbc] Update textsplit to 0.2.0
1 file changed, 1 insertion(+), 1 deletion(-)
$ git push origin main
To $LAB/ch23/submodule-update/remotes/doc-qa.git
 907dbd3..0d51dbc main -> main
```

State table for the two commands of this section:

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git submodule update</code> (with or without <code>--remote</code>)	Files inside the submodule path change; the superproject's own files do not	unchanged	unchanged; the submodule's HEAD is detached at the target commit	unchanged	<code>.git/modules/<name>/</code> : new objects after a fetch, its HEAD and HEAD reflog	unchanged (fetch only)	unchanged
<code>git add <path></code> then <code>git commit</code>	unchanged	gitlink updated	new commit	advanced	reflogs	unchanged until pushed	unchanged until pushed

Treat the resulting commit like any dependency bump: one purpose, its own pull request, CI on the result.

23.7 Keeping clones in step: the stale submodule and the silent rollback

You pushed the pointer change. Ravi pulls:

```
$ cd ../ravi-doc-qa
$ git pull
From $LAB/ch23/submodule-update/remotes/doc-qa
 907dbd3..0d51dbc main -> origin/main
Fetching submodule vendor/textsplit
From $LAB/ch23/submodule-update/remotes/textsplit
 e216665..b3c86ce main -> origin/main
* [new tag]      v0.2.0 -> v0.2.0
Updating 907dbd3..0d51dbc
Fast-forward
 vendor/textsplit | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
    modified:   vendor/textsplit (new commits)

no changes added to commit (use "git add" and/or "git commit -a")
```

`git pull` fetched the superproject, noticed that the new commits change a gitlink, and fetched inside the submodule as well ("Fetching submodule vendor/textsplit"; this on-demand fetch is the default of `fetch.recurseSubmodules`). Then it fast-forwarded the superproject. It did **not** check out the new commit in the submodule. Ravi made no change, and `git status` reports one:

```
$ git submodule status
+e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit (v0.1.0-1-ge216665)
$ git diff --submodule=log
Submodule vendor/textsplit b3c86ce..e216665 (rewind):
< Reject an overlap that is not smaller than the chunk size
```

The `+` and the word `(rewind)` are the evidence. The superproject now records `b3c86ce`; the submodule's working tree is still at `e216665`. Seen from the superproject, the working tree differs from the index by going one library commit backwards.

```
Observed behavior : After a pull that changed nothing of his own, git status shows
                    "modified: vendor/textsplit (new commits)".
Git state          : HEAD and index of the superproject record gitlink b3c86ce. The submodule's HEAD is
                    still e216665. The library commit b3c86ce has been fetched into .git/modules/.
Mechanism          : pull = fetch + merge. The merge updates the gitlink in the index. Checking out a
                    superproject commit does not move the HEAD of a submodule unless recursion is on.
Root cause         : Two repositories, two HEADs, and no default that ties the second to the first.
Why Git does this  : The submodule may hold work in progress; Git will not move it without being asked.
Correct fix        : git submodule update      (then git status is clean)
Prevention         : git config set submodule.recurse true, or git pull --recurse-submodules.
```

If Ravi does not look closely, the stale state turns into a commit. He edits one file and uses `git commit -a`, which stages everything that is modified:

```
# Ravi does not look closely. He edits a file and commits everything that is modified:
$ git commit -am "Use larger chunks"
[main b98f829] Use larger chunks
 2 files changed, 2 insertions(+), 2 deletions(-)
$ git show --stat --format="%h %an: %s"
b98f829 Ravi Menon: Use larger chunks

 ingest.py      | 2 +-
 vendor/textsplit | 2 +-
 2 files changed, 2 insertions(+), 2 deletions(-)
$ git ls-tree HEAD vendor/
160000 commit e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit
```

Two files changed. His commit moved the gitlink back to `e216665`. This is report 3 from section 23.1, and after a push it is report 4 for everyone else: the service silently runs the old library again. A reviewer sees it only as one changed line, `vendor/textsplit | 2 +-`, in a commit that claims to be about chunk sizes.

The commit has not been pushed, so it can be repaired in place: check out the right library commit, stage the gitlink, amend.

```
# Not pushed yet, so repair the commit: put the submodule on the commit that main records
# upstream, stage it, amend.
$ git -C vendor/textsplit checkout --quiet $(git rev-parse origin/main:vendor/textsplit)
$ git add vendor/textsplit
$ git commit --amend --no-edit
[main 2b86667] Use larger chunks
 Date: Mon Sep 7 10:35:00 2026 +0530
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git show --stat --format="%h %an: %s"
2b86667 Ravi Menon: Use larger chunks

 ingest.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git submodule status
b3c86ceee1901d9cfff1a21920bd863a64ffb738 vendor/textsplit (v0.2.0)
```

`origin/main:vendor/textsplit` is the gitlink as the remote-tracking branch records it; `git rev-parse` returns the ID even though the object is not in the superproject. If the bad commit had already been pushed, the fix is a new commit that moves the pointer forward again.

The cure for the whole section is to make the second step automatic. After a pull, the manual sequence is two commands:

```
$ git pull
From $LAB/ch23/submodule-recurse/remotes/doc-qa
 907dbd3..3719d28 main    -> origin/main
Fetching submodule vendor/textsplit
From $LAB/ch23/submodule-recurse/remotes/textsplit
 e216665..b3c86ce main    -> origin/main
* [new tag]          v0.2.0    -> v0.2.0
Updating 907dbd3..3719d28
Fast-forward
 vendor/textsplit | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git submodule status
+e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit (v0.1.0-1-ge216665)
$ git submodule update
Submodule path 'vendor/textsplit': checked out 'b3c86ceee1901d9cfff1a21920bd863a64ffb738'
$ git submodule status
b3c86ceee1901d9cfff1a21920bd863a64ffb738 vendor/textsplit (v0.2.0)
```

```
$ git status --short --branch
## main...origin/main
```

The same staleness appears with every command that moves the superproject's HEAD:

```
# An older commit of doc-qa records the older library commit. Without recursion:
$ git switch --detach HEAD~1
HEAD is now at 907dbd3 Vendor textsplit as a submodule and chunk documents
M vendor/textsplit
$ git submodule status
+b3c86ceee1901d9cfff1a21920bd863a64ffb738 vendor/textsplit (v0.2.0)
$ git switch -
Previous HEAD position was 907dbd3 Vendor textsplit as a submodule and chunk documents
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ git status --short
```

With `submodule.recurse` set, `checkout`, `fetch`, `grep`, `pull`, `push`, `read-tree`, `reset`, `restore` and `switch` behave as if `--recurse-submodules` were given (the list is from `git help -m config`), and the submodule follows:

```
$ git config set submodule.recurse true
$ git switch --detach HEAD~1
HEAD is now at 907dbd3 Vendor textsplit as a submodule and chunk documents
$ git submodule status
e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit (v0.1.0-1-ge216665)
$ git switch -
Previous HEAD position was 907dbd3 Vendor textsplit as a submodule and chunk documents
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ git submodule status
b3c86ceee1901d9cfff1a21920bd863a64ffb738 vendor/textsplit (v0.2.0)
$ git config get submodule.recurse
true
```

`git clone` is the exception: it needs its own `--recurse-submodules`, and so does `git ls-files`. The manual's workflow section recommends `submodule.recurse true` for repositories that are split into submodules ([git submodules](#)). The setting is local or global configuration, so it cannot be shipped with the repository: every clone, and every CI job, has to set it or pass the option.

23.8 Push order: the commit your teammate cannot fetch

You improve the library from inside the service. You are on the submodule's `main`, as section 23.5 advised:

```
$ cd vendor/textsplit
$ git switch main
Already on 'main'
Your branch is up to date with 'origin/main'.
$ git commit -am "Add line splitter"
[main 17d7857] Add line splitter
1 file changed, 5 insertions(+)
$ cd ../../
$ git status --short
M vendor/textsplit
$ git commit -am "Use the line splitter from textsplit"
[main ee30f63] Use the line splitter from textsplit
1 file changed, 1 insertion(+), 1 deletion(-)
```

There are now two unpushed commits in two repositories: `17d7857` in the library and `ee30f63` in the superproject, whose tree names `17d7857`. Pushing the superproject alone is accepted, because a gitlink is only an ID and the server does not check it:

```
# Without the guard, the push of the superproject succeeds:
$ git push origin main
To $LAB/ch23/submodule-push-order/remotes/doc-qa.git
907dbd3..ee30f63 main -> main
```

Ravi pulls:

```
$ cd ../ravi-doc-qa
$ git pull
From $LAB/ch23/submodule-push-order/remotes/doc-qa
907dbd3..ee30f63 main -> origin/main
Fetching submodule vendor/textsplit
fatal: git upload-pack: not our ref 17d78577459b6c2e681d54d4cfac2b350aaf7b2c
fatal: remote error: upload-pack: not our ref 17d78577459b6c2e681d54d4cfac2b350aaf7b2c
Errors during submodule fetch:
vendor/textsplit
[exit status: 1]
```

Root cause. The superproject on the server records library commit 17d7857. That commit exists in exactly one place:

`.git/modules/vendor/textsplit` on your machine. The server-side `git upload-pack` of the library was asked for it by ID and answered not our ref.

In this transcript two processes report the same refusal: the client (`remote error: upload-pack: not our ref`) and `git upload-pack` itself, which runs on your own machine when the remote is a path. Their two lines can appear in either order, which is why `labs/verify-all.sh` treats this demo as volatile. Over HTTPS or SSH you see only the `remote error` line.

Note what state the failure left. The fetch of the superproject succeeded and the merge did not run:

```
$ git status --short --branch
## main...origin/main [behind 1]
$ git ls-tree origin/main vendor/
160000 commit 17d78577459b6c2e681d54d4cfac2b350aaf7b2c vendor/textsplit
$ git ls-remote ../remotes/textsplit.git
e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 HEAD
e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 refs/heads/main
f8fdbba471fcee4fa0c9a10d59be41c10c86f79ec refs/tags/v0.1.0
ceaafe1cb2284284c7aa91a8ac843268d1dad25e refs/tags/v0.1.0^{}
```

```
$ git -C vendor/textsplit cat-file -t 17d7857
fatal: Not a valid object name 17d7857
[exit status: 128]
$ git -C ../doc-qa/vendor/textsplit branch --all --contains 17d7857
* main
```

The diagnosis in three questions. Which library commit does the superproject want? `git ls-tree origin/main vendor/`. Does the library's shared repository have it? `git ls-remote` shows `main` still at `e216665`, and no ref leads to `17d7857`. Who has it? The author of the superproject commit, on a local branch.

The fix is on the author's side: push the library commit. Then the teammate's commands work:

```
$ cd ../doc-qa
$ git -C vendor/textsplit push origin main
To $LAB/ch23/submodule-push-order/remotes/textsplit.git
 e216665..17d7857 main -> main
$ cd ../ravi-doc-qa
$ git pull
Updating 907dbd3..ee30f63
Fast-forward
 vendor/textsplit | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git -c protocol.file.allow=always submodule update
From $LAB/ch23/submodule-push-order/remotes/textsplit
 e216665..17d7857 main -> origin/main
Submodule path 'vendor/textsplit': checked out '17d78577459b6c2e681d54d4cfac2b350aaf7b2c'
$ git submodule status
17d78577459b6c2e681d54d4cfac2b350aaf7b2c vendor/textsplit (v0.1.0-2-g17d7857)
```

Git has a guard for this, and it is off by default. 🟢 `git push --recurse-submodules=check` verifies, before anything is sent, that every submodule commit named by the commits being pushed is reachable from a remote-tracking branch of that submodule:

```
$ git push --recurse-submodules=check origin main
The following submodule paths contain changes that can
not be found on any remote:
 vendor/textsplit

Please try

    git push --recurse-submodules=on-demand

or cd to the path and use

    git push

to push them to a remote.

fatal: Aborting.
fatal: the remote end hung up unexpectedly
[exit status: 128]
```

`--recurse-submodules=on-demand` goes one step further and pushes the submodules first:

```
$ cd ../doc-qa
$ git -C vendor/textsplit commit -am "Add sentence splitter"
[main 0dd831a] Add sentence splitter
1 file changed, 4 insertions(+)
```

```
$ git commit -am "Use the sentence splitter from textsplit"
[main 9b2d81f] Use the sentence splitter from textsplit
1 file changed, 1 insertion(+), 1 deletion(-)
$ git push --recurse-submodules=on-demand origin main
Pushing submodule 'vendor/textsplit'
To $LAB/ch23/submodule-push-order/remotes/textsplit.git
17d7857..0dd831a main -> main
To $LAB/ch23/submodule-push-order/remotes/doc-qa.git
ee30f63..9b2d81f main -> main
```

```
$ git config set push.recurseSubmodules check
$ git config get push.recurseSubmodules
check
```

`push.recurseSubmodules=check` is the setting to put in every developer's configuration for a repository with submodules. Its limits: `check` trusts your remote-tracking branches, so it is satisfied by a commit that sits on a library branch you pushed and that someone later deletes; and `on-demand` needs push permission on the library, which consumers often do not have. In that case the library change goes through the library's own review first, and the superproject bumps the pointer afterwards.

23.9 Dirty submodules and switching branches

A submodule's working tree can have uncommitted changes of its own. The superproject reports them and cannot record them:

```
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
  (commit or discard the untracked or modified content in submodules)
        modified:   vendor/textsplit (modified content, untracked content)

no changes added to commit (use "git add" and/or "git commit -a")
$ git submodule status
e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit (v0.1.0-1-ge216665)
```

```
$ git diff
diff --git a/vendor/textsplit b/vendor/textsplit
--- a/vendor/textsplit
+++ b/vendor/textsplit
@@ -1,1 @@
-Subproject commit e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4
+Subproject commit e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4-dirty
$ git diff --submodule=diff
Submodule vendor/textsplit contains modified content
diff --git a/vendor/textsplit/splitter.py b/vendor/textsplit/splitter.py
index c0dea76..6f4648f 100644
--- a/vendor/textsplit/splitter.py
+++ b/vendor/textsplit/splitter.py
@@ -2,3 +2,4 @@ def split(text, size=200, overlap=0):
    """Cut text into pieces of at most `size` characters; neighbours share `overlap` characters."""
    step = size - overlap
    return [text[i:i + size] for i in range(0, len(text), step)]
+# debug: print every chunk
```

(modified content, untracked content) means the submodule's HEAD is the recorded commit and its working tree is not clean; `git submodule status` therefore shows no `+`. The diff appends `-dirty` to the ID. A gitlink can only name a commit, so there is nothing for the superproject to stage:

```
$ git commit -am "Save my work"
On branch main
Your branch is up to date with 'origin/main'.

Changes not staged for commit:
  (use "git add <file>..." to update what will be committed)
  (use "git restore <file>..." to discard changes in working directory)
  (commit or discard the untracked or modified content in submodules)
        modified:   vendor/textsplit (modified content, untracked content)

no changes added to commit (use "git add" and/or "git commit -a")
[exit status: 1]
```

The work has to be committed inside the submodule first (on a branch), and then the new commit ID can be staged in the superproject. To look into all submodules at once, `git submodule foreach` runs a shell command in each. The lower-case `m` and `?` in short status are the superproject's markers for modified and untracked content inside a submodule, and `--ignore-submodules` chooses how much of that is shown:


```
$ git submodule foreach "git status --short"
Entering 'vendor/textsplit'
M splitter.py
?? notes.txt
$ git status --short
m vendor/textsplit
$ git status --short --ignore-submodules=untracked
m vendor/textsplit
$ git status --short --ignore-submodules=dirty
```

To get back to exactly the recorded state,  `git submodule update --force` checks the files out again and discards edits to tracked files; untracked files need `git clean` inside the submodule (Chapter 4), previewed with `-n`:

```
# update does nothing: the checked-out commit already is the recorded one.
$ git submodule update
$ git status --short
m vendor/textsplit
# --force checks the files out again, which discards the edit to the tracked file:
$ git submodule update --force
Submodule path 'vendor/textsplit': checked out 'e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4'
$ git status --short
? vendor/textsplit
$ git -C vendor/textsplit clean -n
Would remove notes.txt
$ git -C vendor/textsplit clean -f
Removing notes.txt
$ git status --short
```

Switching to a commit without the submodule leaves its directory behind, because Git will not delete a directory that contains a repository's working tree:

```
$ git log --oneline
907dbd3 Vendor textsplit as a submodule and chunk documents
bf78eb9 Add keyword answerer
1d93b09 Add document loader
$ git switch --detach HEAD~1
warning: unable to rmdir 'vendor/textsplit': Directory not empty
HEAD is now at bf78eb9 Add keyword answerer
$ git status
HEAD detached at bf78eb9
Untracked files:
  (use "git add <file>..." to include in what will be committed)
  vendor/

nothing added to commit but untracked files present (use "git add" to track)
$ ls -A vendor/textsplit
.git
README.md
splitter.py
```

The stray `vendor/` is untracked on that commit. A careless `git add -A` there stages it as a new gitlink, with the warning `adding embedded git repository: vendor/textsplit`, and `git clean -f -d` answers `Skipping repository vendor/textsplit` (both run for this chapter). With recursion, the checkout removes and restores the submodule's working tree together with the superproject's files; the repository itself stays in `.git/modules/` either way:

```
$ git switch --recurse-submodules --detach HEAD~1
HEAD is now at bf78eb9 Add keyword answerer
$ git status --short
$ ls vendor 2>&1
ls: vendor: No such file or directory
$ git switch --recurse-submodules main
Previous HEAD position was bf78eb9 Add keyword answerer
Switched to branch 'main'
Your branch is up to date with 'origin/main'.
$ git submodule status
e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit (v0.1.0-1-ge216665)
```

23.10 Pointer conflicts

Two branches of `doc-qa` move the gitlink. What a merge does depends on how the two library commits relate, and Git can look that up only in the submodule's repository.

```
$ git log --graph --oneline --decorate --all
* 7a1b409 (HEAD -> use-tokens) Use the token splitter of textsplit
| * e5a77ed (use-release) Use textsplit 0.2.0
|/
* 907dbd3 (origin/main, main) Vendor textsplit as a submodule and chunk documents
* bf78eb9 Add keyword answerer
```

```
* 1d93b09 Add document loader
$ git ls-tree use-release vendor/
160000 commit de689e20c2a64f6ebe2a0f3ec7c98efc09459cb9 vendor/textsplit
$ git ls-tree use-tokens vendor/
160000 commit cd2a8a45f5da8ab7bfedfe413228a50b91874c72 vendor/textsplit
$ git -C vendor/textsplit log --graph --oneline --decorate v0.2.0 origin/feature/tokens
* de689e2 (tag: v0.2.0, origin/main, origin/HEAD) Reject an overlap that is not smaller than the chunk size
| * cd2a8a4 (HEAD, origin/feature/tokens) Add token splitter
|/
* e216665 (main) Add overlap between neighbouring chunks
* ceaafel (tag: v0.1.0) Add fixed-size splitter
```

`use-release` records `de689e2` (the 0.2.0 release) and `use-tokens` records `cd2a8a4` (a feature branch of the library). The two library commits have diverged: neither is an ancestor of the other.

```
$ git merge use-release
hint: Recursive merging with submodules currently only supports trivial cases.
hint: Please manually handle the merging of each conflicted submodule.
hint: This can be accomplished with the following steps:
hint:   - go to submodule (vendor/textsplit), and either merge commit de689e2
hint:     or update to an existing commit which has merged those changes
hint:   - come back to superproject and run:
hint:
hint:       git add vendor/textsplit
hint:
hint:   to record the above merge or update
hint:   - resolve any other conflicts in the superproject
hint:   - commit the resulting index in the superproject
hint:
hint: Disable this message with "git config set advice.submoduleMergeConflict false"
Failed to merge submodule vendor/textsplit
CONFLICT (submodule): Merge conflict in vendor/textsplit
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
```

```
$ git status --short
UU vendor/textsplit
$ git ls-files --unmerged
160000 e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 1      vendor/textsplit
160000 cd2a8a45f5da8ab7bfedfe413228a50b91874c72 2      vendor/textsplit
160000 de689e20c2a64f6ebe2a0f3ec7c98efc09459cb9 3      vendor/textsplit
$ git submodule status
U0000000000000000000000000000000000000000000000000000000000000000 vendor/textsplit
$ git diff
diff --cc vendor/textsplit
index cd2a8a4,de689e2..0000000
--- a/vendor/textsplit
+++ b/vendor/textsplit
```

The index has the three stages of Chapter 8, each a gitlink: stage 1 the merge base's pointer, stage 2 ours, stage 3 theirs. There are no conflict markers, because there is no file to put them in. The superproject cannot produce the answer: the correct pointer is a library commit that contains both lines of development, and only the library can create it. Git's hint lists the two ways. One is to merge inside the submodule yourself; that creates a merge commit which exists only in your clone and must be pushed to the library before anyone else can use it (section 23.8). The other, used here, is to wait for the library to merge and then point at the result:

```
$ git -C vendor/textsplit fetch origin
From $LAB/ch23/submodule-conflict/remotes/textsplit
  de689e2..5a1935a  main      -> origin/main
$ git -C vendor/textsplit log --graph --oneline --decorate -4 origin/main
*   5a1935a (origin/main, origin/HEAD) Merge feature/tokens
|\
| * cd2a8a4 (HEAD, origin/feature/tokens) Add token splitter
* | de689e2 (tag: v0.2.0) Reject an overlap that is not smaller than the chunk size
|/
* e216665 (main) Add overlap between neighbouring chunks
```

```
$ git -C vendor/textsplit checkout --quiet origin/main
$ git add vendor/textsplit
$ git status --short
M vendor/textsplit
$ git commit --no-edit
[use-tokens c927444] Merge branch 'use-release' into use-tokens
$ git ls-tree HEAD vendor/
160000 commit 5a1935ae4e615fa75f14284cab1eb1906c8cdf33 vendor/textsplit
$ git submodule status
5a1935ae4e615fa75f14284cab1eb1906c8cdf33 vendor/textsplit (v0.2.0-2-g5a1935a)
```

`git add vendor/textsplit` stages whatever commit the submodule has checked out, which resolves the path; `git commit` concludes the merge.

When one of the two library commits is an ancestor of the other, Git resolves the pointer by itself and takes the descendant. It says so in one line:

```
# A new branch from main that records the library merge commit. use-release records v0.2.0,
# an ancestor of that merge. Both branches moved the pointer away from the one main has.
$ git ls-tree main vendor/
160000 commit e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit
$ git ls-tree use-merged vendor/
160000 commit 5a1935ae4e615fa75f14284cab1eb1906c8cdf33 vendor/textsplit
$ git ls-tree use-release vendor/
160000 commit de689e20c2a64f6ebe2a0f3ec7c98efc09459cb9 vendor/textsplit
$ git merge use-release
Note: Fast-forwarding submodule vendor/textsplit to 5a1935ae4e615fa75f14284cab1eb1906c8cdf33
Merge made by the 'ort' strategy.
[exit status: 0]
$ git ls-tree HEAD vendor/
160000 commit 5a1935ae4e615fa75f14284cab1eb1906c8cdf33 vendor/textsplit
```

That automatic resolution needs both commits to be present in the submodule's local repository. In a clone where the submodule is not checked out, Git cannot determine ancestry and reports `Failed to merge submodule vendor/textsplit (not checked out)` and a conflict instead. `-X theirs` does not help either: the merge stops with the same submodule conflict. (Both were run for this chapter; no transcript is printed.)

23.11 Removing a submodule, and a submodule whose URL changes

There are two different "removals".  `git submodule deinit <path>` is local: it empties the working directory and removes the `submodule.` `<name>` section from your `.git/config`. The superproject's history is untouched, and `git submodule update --init` undoes it.

```
$ git submodule deinit vendor/textsplit
Cleared directory 'vendor/textsplit'
Submodule 'vendor/textsplit' (../textsplit.git) unregistered for path 'vendor/textsplit'
$ git submodule status
-e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4 vendor/textsplit
$ ls -A vendor/textsplit
$ git status --short
$ git config list --local | grep ^submodule
$ ls .git/modules/vendor
textsplit
```

 `git rm <path>` removes the submodule from the project: it deletes the gitlink and the section in `.gitmodules`, and you commit that.

```
$ git rm vendor/textsplit
rm 'vendor/textsplit'
$ git status
On branch main
Your branch is up to date with 'origin/main'.

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   .gitmodules
        deleted:    vendor/textsplit

$ git diff --cached
diff --git a/.gitmodules b/.gitmodules
index 9f740c3..e69de29 100644
--- a/.gitmodules
+++ b/.gitmodules
@@ -1,3 +0,0 @@
-[submodule "vendor/textsplit"]
-    path = vendor/textsplit
-    url = ../textsplit.git
diff --git a/vendor/textsplit b/vendor/textsplit
deleted file mode 160000
index e216665..0000000
--- a/vendor/textsplit
+++ /dev/null
@@ -1 +0,0 @@
-Subproject commit e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4
```

```
$ git commit -m "Stop vendoring textsplit"
[main 4adef58] Stop vendoring textsplit
 2 files changed, 4 deletions(-)
 delete mode 160000 vendor/textsplit
$ ls -A
.git
.gitmodules
answer.py
ingest.py
```

```
README.md
vendor
```

Three leftovers remain, and each causes a question later. An empty `.gitmodules` is still tracked (delete it with `git rm .gitmodules` if no submodule is left). The section in `.git/config` and the repository under `.git/modules/` are still on your disk:

```
$ git config list --local | grep ^submodule
submodule.vendor/textsplit.active=true
submodule.vendor/textsplit.url=$LAB/ch23/submodule-remove/remotes/textsplit.git
$ ls .git/modules/vendor
textsplit
```

Git keeps the repository on purpose, so that old commits which still contain the gitlink can be checked out without a new clone. The cost shows when someone adds a submodule under the same name again:

```
$ git -c protocol.file.allow=always submodule add ../textsplit.git vendor/textsplit
fatal: A git directory for 'vendor/textsplit' is found locally with remote(s):
  origin          $LAB/ch23/submodule-remove/remotes/textsplit.git
If you want to reuse this local git directory instead of cloning again from
  $LAB/ch23/submodule-remove/remotes/textsplit.git
use the '--force' option. If the local git directory is not the correct repo
or you are unsure what this means choose another name with the '--name' option.
[exit status: 128]
```

To remove a submodule completely from your clone, delete the leftovers. 🚫 The first command deletes a repository; if the submodule held unpushed commits, they are gone.

```
$ rm -rf .git/modules/vendor/textsplit
$ git config remove-section submodule.vendor/textsplit
$ git -c protocol.file.allow=always submodule add ../textsplit.git vendor/textsplit
Cloning into '$LAB/ch23/submodule-remove/doc-qa/vendor/textsplit'...
done.
$ git status --short
M .gitmodules
A vendor/textsplit
```

History is not rewritten by any of this. A checkout of a commit from before the removal has the gitlink again, and the directory is empty until the next `git submodule update --init`.

A changed URL shows the role of the local copy in `.git/config`. The library's repository has moved. You record the new URL with `git submodule set-url`, which edits `.gitmodules` and synchronizes your own configuration:

```
$ git submodule set-url vendor/textsplit ../chunking.git
Synchronizing submodule url for 'vendor/textsplit'
$ git diff
diff --git a/.gitmodules b/.gitmodules
index 9f740c3..7902bc0 100644
--- a/.gitmodules
+++ b/.gitmodules
@@ -1,3 +1,3 @@
 [submodule "vendor/textsplit"]
   path = vendor/textsplit
-  url = ../textsplit.git
+  url = ../chunking.git
$ git config get submodule.vendor/textsplit.url
$LAB/ch23/submodule-url/remotes/chunking.git
$ git -c protocol.file.allow=always submodule update --remote
From $LAB/ch23/submodule-url/remotes/chunking
 e216665..4f00a5b  main    -> origin/main
 * [new tag]         v0.2.0    -> v0.2.0
Submodule path 'vendor/textsplit': checked out '4f00a5b8bb1f8ba700d273d9ec2f453be23efc75'
$ git commit -am "textsplit moved to chunking.git; update to 0.2.0"
[main 109e5d9] textsplit moved to chunking.git; update to 0.2.0
 2 files changed, 2 insertions(+), 2 deletions(-)
$ git push --recurse-submodules=check origin main
To $LAB/ch23/submodule-url/remotes/doc-qa.git
 907dbd3..109e5d9  main -> main
```

Ravi pulls the new `.gitmodules`. His configuration still has the URL that `init` copied months ago:

```
$ cd ../ravi-doc-qa
$ git pull --no-recurse-submodules
From $LAB/ch23/submodule-url/remotes/doc-qa
 907dbd3..109e5d9  main    -> origin/main
Updating 907dbd3..109e5d9
Fast-forward
 .gitmodules      | 2 +-
 vendor/textsplit | 2 +-
 2 files changed, 2 insertions(+), 2 deletions(-)
$ cat .gitmodules
```

```
[submodule "vendor/textsplit"]
    path = vendor/textsplit
    url = ../chunking.git
$ git config get submodule.vendor/textsplit.url
$LAB/ch23/submodule-url/remotes/textsplit.git
$ git -c protocol.file.allow=always submodule update
fatal: '$LAB/ch23/submodule-url/remotes/textsplit.git' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
Unable to fetch in submodule path 'vendor/textsplit'; trying to directly fetch 4f00a5b8bb1f8ba700d273d9ec2f453be23efc75:
fatal: '$LAB/ch23/submodule-url/remotes/textsplit.git' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
fatal: Fetched in submodule path 'vendor/textsplit', but it did not contain 4f00a5b8bb1f8ba700d273d9ec2f453be23efc75. Direct
fetching of that commit failed.
[exit status: 128]
```

● `git submodule sync` copies the URLs from `.gitmodules` into `.git/config` and into the `origin` remote of each initialized submodule:

```
$ git submodule sync
Synchronizing submodule url for 'vendor/textsplit'
$ git config get submodule.vendor/textsplit.url
$LAB/ch23/submodule-url/remotes/chunking.git
$ git -C vendor/textsplit remote get-url origin
$LAB/ch23/submodule-url/remotes/chunking.git
$ git -c protocol.file.allow=always submodule update
From $LAB/ch23/submodule-url/remotes/chunking
 e216665..4f00a5b  main      -> origin/main
* [new tag]         v0.2.0    -> v0.2.0
Submodule path 'vendor/textsplit': checked out '4f00a5b8bb1f8ba700d273d9ec2f453be23efc75'
```

23.12 CI consequences

A CI job is a fresh clone made by a script, so every default of this chapter applies to it, and nobody is there to read `git status`.

GitHub Actions, not Git. `actions/checkout` does not check out submodules unless told to: its `submodules` input defaults to `false`, and the README describes the values as "`true` to checkout submodules or `recursive` to recursively checkout submodules" ([action.yml](#), [README](#)). The job's `github.token` is scoped to the repository that triggered the run, so a private submodule in another repository needs a token or an SSH key with access to it, passed through the action's `token` or `ssh-key` input. The README also notes that, without `ssh-key`, URLs beginning with `git@github.com:` are converted to HTTPS. None of this was run for this book.

What follows from the mechanics:

- **An empty directory is not an error to Git.** A job without submodule checkout fails later and elsewhere: an import error, a missing file in a Docker build context, a test that is silently skipped. Add an explicit step such as `git submodule status` and fail if any line starts with `-` or `+`.
- **The job tests the recorded commit**, not the library's latest. A library fix reaches the service only through a pointer commit.
- **The push-order failure of section 23.8 appears in CI first**, as "not our ref" or "did not contain" during checkout, on the commit of the person who forgot to push the library.
- **Trust.** A workflow that recursively checks out submodules of a pull request from a fork lets the fork's `.gitmodules` choose what is cloned onto the runner. Combine that with section 23.4 before enabling recursion for untrusted code (Chapter 21A).
- **Docker and archives.** `git archive` does not include submodule contents: the archive of this chapter's superproject contains the entry `vendor/textsplit/` and nothing below it (run for this chapter, no transcript printed). A build that starts from an archive needs the submodules fetched separately.

23.13 Subtrees: the dependency's files inside your own history

In one sentence. ● `git subtree add` copies another project's files into a subdirectory of your repository as ordinary tracked files, and records in commit messages where they came from, so that later commands can merge newer upstream versions in or extract your changes back out.

Analogy. Instead of a reference to page 212 of the other book, you photocopy the page and bind the copy into yours, with a note in the margin saying which printing it was copied from. Every reader of your book has the recipe. The margin note is the only link to the original: if somebody copies a newer printing over it without updating the note, the next update has nothing to go by.

Precisely. `git subtree` is a script from Git's `contrib/` directory. The Homebrew Git on this Mac installs it. It has no page in `git help`; its documentation is in the Git source tree, and `git subtree -h` prints the usage. That documentation describes the contrast with submodules

itself: subtrees "do not need any special constructions (like `.gitmodules` files or gitlinks) be present in your repository, and do not force end-users of your repository to do anything special or to understand how subtrees work" ([git-subtree](#)). Do not confuse the command with the `subtree merge` strategy of `git merge -s subtree`, which it builds on.

See it. Start again from `doc-qa` without any dependency and add the library under the same path:

```
$ git subtree add --prefix=vendor/textsplit ../remotes/textsplit.git main --squash
git fetch ../remotes/textsplit.git main
From ../remotes/textsplit
 * branch      main      -> FETCH_HEAD
Added dir 'vendor/textsplit'
$ git log --graph --format="%h %an: %s"
* fb692e1 Lab User: Merge commit 'c77d389e2c498a974e85fe327f712348bbe6f573' as 'vendor/textsplit'
|\
| * c77d389 Lab User: Squashed 'vendor/textsplit/' content from commit e216665
* bf78eb9 Lab User: Add keyword answerer
* 1d93b09 Lab User: Add document loader
```

`--squash` produced two commits. `c77d389` is a commit with no parent in your history whose tree is the library's tree at upstream commit `e216665`; `fb692e1` merges it into `main` and places that tree at `vendor/textsplit`. Without `--squash` the merge would bring the library's entire history into yours.

```
$ git ls-tree HEAD vendor/
040000 tree bb3d20df131c55a48b035cace5cd1812d87d5332 vendor/textsplit
$ git ls-files vendor
vendor/textsplit/README.md
vendor/textsplit/splitter.py
$ git log -1 --format=%B HEAD^2
Squashed 'vendor/textsplit/' content from commit e216665

git-subtree-dir: vendor/textsplit
git-subtree-split: e2166657d4daa7b174a5fb83da9e2f8ecca2c9e4

$ git status --short
$ ls -A
.git
answer.py
ingest.py
README.md
vendor
```

Inside `.git`. Nothing special at all. The entry at `vendor/textsplit` has mode `040000`: a tree like any other directory. The files are blobs in your object database. There is no `.gitmodules`, no `.git/modules`, no configuration. The whole mechanism is the two trailer lines in the squash commit's message: `git-subtree-dir` and `git-subtree-split`, the upstream commit that the content corresponds to.

For a teammate there is nothing to learn and nothing to forget:

```
$ git push origin main
To $LAB/ch23/subtree/remotes/doc-qa.git
bf78eb9..fb692e1 main -> main
$ cd ..
$ git clone remotes/doc-qa.git ravi-doc-qa
Cloning into 'ravi-doc-qa'...
done.
$ ls ravi-doc-qa/vendor/textsplit
README.md
splitter.py
$ cd doc-qa
```

A plain clone, by a person or a CI job, has the library. No `init`, no `update`, no second fetch, no second set of credentials.

Updating. 🟡 `git subtree pull` fetches the upstream branch, builds a squash commit for the difference since the recorded `git-subtree-split`, and merges it:

```
# Asha has published textsplit 0.2.0.
$ git subtree pull --prefix=vendor/textsplit ../remotes/textsplit.git main --squash
From ../remotes/textsplit
 * branch      main      -> FETCH_HEAD
Merge made by the 'ort' strategy.
 vendor/textsplit/splitter.py | 2 ++
 1 file changed, 2 insertions(+)
$ git log --graph --format="%h %an: %s"
* 7241fca Lab User: Merge commit 'c2dc1b7050d0f35c26e98632a0a8916c9f9b61e3'
|\
| * c2dc1b7 Lab User: Squashed 'vendor/textsplit/' changes from e216665..4980f0b
* | fb692e1 Lab User: Merge commit 'c77d389e2c498a974e85fe327f712348bbe6f573' as 'vendor/textsplit'
|\|
| * c77d389 Lab User: Squashed 'vendor/textsplit/' content from commit e216665
* bf78eb9 Lab User: Add keyword answerer
* 1d93b09 Lab User: Add document loader
```

```
$ git log -1 --format=%B HEAD^2
Squashed 'vendor/textsplit/' changes from e216665..4980f0b

4980f0b Reject an overlap that is not smaller than the chunk size

git-subtree-dir: vendor/textsplit
git-subtree-split: 4980f0b1f99fa205fd4882634ce93e661ede41fd
```

The new squash commit's message lists the upstream commits it covers and records the new split point. It is an ordinary merge: if you changed the vendored files yourself and upstream changed the same lines, you resolve a normal content conflict in normal files.

The option has to be used consistently. A repository whose subtree was added with `--squash` has none of the upstream commits in its history, so a pull without `--squash` tries to merge two histories with no common ancestor:

```
# Update the vendored copy, and forget --squash:
$ git subtree pull --prefix=vendor/textsplit ../remotes/textsplit.git main
From ../remotes/textsplit
 * branch          main      -> FETCH_HEAD
fatal: refusing to merge unrelated histories
[exit status: 128]
$ git status --short --branch
## main...origin/main
```

Contributing back. Because the files are yours, one commit can change the library and the application together, which a submodule cannot do:

```
# A change of your own that touches the library and the application in one commit:
$ git commit -am "Split documents by paragraph"
[main e3827e6] Split documents by paragraph
 2 files changed, 9 insertions(+)
$ git show --stat --format="%h %s"
e3827e6 Split documents by paragraph

 ingest.py | 4 ++++
 vendor/textsplit/splitter.py | 5 +++++
 2 files changed, 9 insertions(+)
```

● `git subtree split` walks your history and synthesizes a history that contains only what happened under the prefix, with the prefix removed from the paths:

```
$ git subtree split --quiet --prefix=vendor/textsplit -b textsplit-export
ee9c8f1aeeb272a682e93f0137db86efd7c142e8
$ git log --graph --format="%h %an: %s" textsplit-export
* ee9c8f1 Lab User: Split documents by paragraph
* 4980f0b Asha Rao: Reject an overlap that is not smaller than the chunk size
* e216665 Asha Rao: Add overlap between neighbouring chunks
* ceaafe1 Asha Rao: Add fixed-size splitter
$ git ls-tree --name-only textsplit-export
README.md
splitter.py
$ git show --stat --format="%h %s" textsplit-export
ee9c8f1 Split documents by paragraph

 splitter.py | 5 +++++
 1 file changed, 5 insertions(+)
```

Look at the IDs. The synthetic branch ends in `ee9c8f1`, your commit reduced to its library part (`ingest.py` is gone from it), and below it are `4980f0b`, `e216665` and `ceaafe1`: the upstream commits themselves, with their original IDs and Asha as author. The split found them through the `git-subtree-split` trailers, so the result is a branch that upstream can merge or review as a normal contribution. `git subtree push` is a split followed by a push of the result:

```
$ git subtree push --quiet --prefix=vendor/textsplit ../remotes/textsplit.git docqa/paragraphs
git push using: ../remotes/textsplit.git docqa/paragraphs
To ../remotes/textsplit.git
 * [new branch]      ee9c8f1aeeb272a682e93f0137db86efd7c142e8 -> docqa/paragraphs
$ git ls-remote --heads ../remotes/textsplit.git
ee9c8f1aeeb272a682e93f0137db86efd7c142e8      refs/heads/docqa/paragraphs
4980f0b1f99fa205fd4882634ce93e661ede41fd      refs/heads/main
```

`--rejoin` merges the synthetic history back into yours so that, in the words of its documentation, "future splits can search only the part of history that has been added since the most recent `--rejoin`".

In production. Vendoring a small, slowly changing dependency into a service where every CI job and every Docker build must work from a plain clone. The price: the repository grows with every vendored version, the link to upstream lives in commit messages that a rebase or a squash-merge of the pull request can destroy, and everyone who updates the copy must use the same command with the same options.

State table:

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git subtree add --squash, git subtree pull --squash</code>	Files under the prefix created or updated	updated to match	two new commits (a squash commit and a merge)	advanced	new objects; <code>FETCH_HEAD</code>	unchanged (a fetch)	unchanged
<code>git subtree split -b <branch></code>	unchanged	unchanged	unchanged	unchanged	a new branch with synthetic commits	unchanged	unchanged
<code>git subtree push</code>	unchanged	unchanged	unchanged	unchanged	new synthetic commits	a branch created or updated in the library's repository	as for any push

23.14 Submodule, subtree, or a package manager

Question	Submodule	Subtree	Package manager (pip, npm, Maven, Go modules)
What the consuming repository records	A commit ID (gitlink) and a URL in <code>.gitmodules</code>	The files themselves, plus trailers in commit messages	A name and a version, in a manifest and a lock file
Where the dependency's content lives	In the dependency's repository; cloned beside yours	In your repository's own objects	In a registry or artifact store; installed at build time
After a plain <code>git clone</code>	An empty directory	Complete	Manifest only; an install step is required
Updating	<code>update --remote</code> , then commit the pointer	<code>git subtree pull --squash</code>	Change the version, regenerate the lock file
Changing the dependency from inside the consumer	Commit in the submodule, push it, then commit the pointer: two repositories, in order	Edit and commit normally; <code>split</code> and <code>push</code> to offer it upstream	Not possible in place; publish a new version or use the tool's local-override mechanism
Access control and licensing separation	Kept: the dependency keeps its own repository and permissions	Lost: whoever can read your repository reads the copy	Kept, through the registry
Typical failure	The states of this chapter: uninitialized, stale, unpushed, detached	Lost trailers; mixed <code>--squash</code> usage; local edits that conflict on every update	Version conflicts; a registry that is unavailable; unpinned versions
Fits	A dependency developed in step with the consumer by people who have access to both; large or access-restricted content	A small dependency that must be present in every clone and changes rarely	Anything that is published with versions, which is most libraries

How to decide. If the dependency is published as a package, use the package manager; it solves versioning, transitive dependencies and security advisories, which neither Git mechanism attempts. If it is not a package and must be present after a plain clone, use a subtree. If it must remain a separate repository (its own access rules, its own release cadence, a size you do not want in your history) and the team will adopt the configuration of section 23.16, use a submodule. If the real wish is "one atomic commit across both", the two projects may belong in one repository (Chapter 24).

23.15 What can go wrong

Symptom	Diagnosis	Fix	Prevention
The submodule directory is empty after a clone	<code>git submodule status</code> shows <code>-</code>	<code>git submodule update --init --recursive</code>	<code>git clone --recurse-submodules</code> ; document it in the README; check it in CI
<code>modified: <path></code> (new commits) after a pull, with no edits of your own	<code>git submodule status</code> shows <code>+</code> ; <code>git diff --submodule=log</code> shows (rewind)	<code>git submodule update</code>	<code>submodule.recurse=true</code> ; <code>git pull --recurse-submodules</code>
A commit about something else changes <code><path> 2 +-</code>	<code>git show --stat <commit></code> ; the gitlink moved backwards	Unpushed: check out the right commit in the submodule, <code>git add</code> , amend. Pushed: a new commit that restores the pointer	Update before committing; avoid <code>git commit -a</code> in superprojects; review gitlink changes
<code>not our ref <id></code> , or "did not contain . Direct fetching of that commit failed"	<code>git ls-tree origin/main <path></code> against <code>git ls-remote <library URL></code> : the recorded commit is on no branch of the library's remote	The author pushes the library commit; or the pointer is moved to a published commit	<code>push.recurseSubmodules=check</code> ; CI on a fresh recursive clone
Work done inside the submodule vanished after <code>git submodule update</code>	<code>git -C <path> reflog</code> : a commit made on a detached HEAD	<code>git -C <path> branch <name> <id></code>	Switch to a branch inside the submodule before editing
<code>modified: <path></code> (modified content) and	<code>git submodule foreach 'git status --short'</code>	Commit inside the submodule, or discard there	Keep consumed submodules clean

Symptom	Diagnosis	Fix	Prevention
<code>git commit -a</code> commits nothing			
CONFLICT (submodule)	<code>git ls-files -u</code> ; compare the two commits inside the submodule	Check out a library commit that contains both, <code>git add <path>, commit</code>	Move pointers in dedicated, short-lived branches
does not appear to be a git repository on update after the library moved	<code>git config get submodule.<name>.url</code> shows the old URL	<code>git submodule sync</code> , then update	Announce URL changes; relative URLs
<code>git subtree pull</code> ends in refusing to merge unrelated histories	The subtree was added with <code>--squash</code> and pulled without	Repeat the pull with <code>--squash</code>	A script or alias for the update command

23.16 When not to use it, and dangerous edge cases

Do not use a submodule:

- **For a library that is available as a package.** You would rebuild a worse package manager.
- **When contributors cannot be expected to learn it**, for example an open-source project with many occasional contributors.
- **For content from sources you do not trust**, combined with recursive clones (23.4).

Do not use a subtree when the vendored content is large or changes often (every version stays in your history, Chapter 22 explains why that matters for binaries), or when its access rules must differ from those of the consuming repository.

A configuration that removes most of the submodule failures. It is local configuration, so each clone and each CI job needs it:

```
git config set submodule.recurse true      # switch, pull, checkout, reset follow the gitlink
git config set push.recurseSubmodules check # refuse to push a pointer to an unpublished commit
git config set diff.submodule log         # diffs list the library commits
git config set status.submoduleSummary true # status shows them too
```

Edge cases:

-  `git submodule update --force` and `git submodule deinit --force` discard uncommitted changes inside the submodule. Preview with `git submodule foreach 'git status --short'`.
-  Deleting `.git/modules/<name>` deletes a repository, including commits that were never pushed.
- **A pointer to a commit on a library branch that is later rebased or deleted.** The commit becomes unreachable on the server and will eventually be unobtainable. Record only commits that are on a protected branch or a tag of the library.
- `update --remote` **in CI.** A job that runs it tests a library commit that no superproject commit records, so the run cannot be reproduced. Use it in a scheduled job that opens a pull request with the pointer change.
- **Linked worktrees** of a superproject have incomplete submodule support according to the worktree manual (Chapter 25, section 25.7).
- **Rewriting a subtree's history.** Squash-merging or rebasing the commits that `git subtree` created discards or rewrites the trailers that the next pull depends on.

23.17 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git submodule status, summary, foreach '<read-only command>'</code> , <code>git diff --submodule=log</code> , <code>git push --recurse-submodules=check</code> , <code>git subtree split</code>	 SAFE	Nothing in your branches, index or files (<code>split</code> adds synthetic commits and, with <code>-b</code> , a branch)	not needed	not needed
<code>git submodule add</code>	 SAFE	Clones; stages <code>.gitmodules</code> and a gitlink; writes <code>.git/config</code> and <code>.git/modules/</code>	<code>git ls-remote <url></code>	<code>git rm -f <path></code> before committing, and remove the leftovers (23.11)
<code>git submodule init, sync, set-url, set-branch</code>	 SAFE	<code>.git/config</code> ; <code>set-url</code> and <code>set-branch</code> also edit <code>.gitmodules</code>	<code>cat .gitmodules</code> ; <code>git config list --local</code>	Edit the values back
<code>git submodule update</code> (default checkout)	 CAUTION	Moves the submodule's HEAD (detached) and its files; refuses to overwrite local edits to files it must change	<code>git submodule status</code> ; <code>git -C <path> status</code>	The submodule's reflog: <code>git -C <path> reflog</code>

Command	Label	What it changes	Preview	Recovery
<code>git submodule update --remote</code>	⚠ CAUTION	As above, to the tip of the remote-tracking branch; the superproject then shows a modified gitlink	<code>git -C <path> fetch && git -C <path> log --oneline HEAD..origin/HEAD</code>	<code>git submodule update</code> returns to the recorded commit
<code>git submodule update --force</code>	🔴 DANGEROUS	Overwrites modified tracked files in the submodule	<code>git submodule foreach 'git status --short'</code>	None for uncommitted edits
<code>git submodule deinit <path></code>	⚠ CAUTION	Empties the working directory; removes the section from <code>.git/config</code> ; refuses if there are local modifications unless <code>--force</code>	<code>git -C <path> status</code>	<code>git submodule update --init</code>
<code>git rm <submodule path></code>	⚠ CAUTION	Removes the gitlink, the <code>.gitmodules</code> section and the working directory	<code>git status</code>	<code>git restore --staged --worktree .gitmodules <path></code> , then <code>git submodule update --init</code>
<code>git push --recurse-submodules=on-demand</code>	⚠ CAUTION	Pushes in the submodules, then the superproject	<code>git submodule foreach 'git status --short --branch'</code>	As for any push
<code>git subtree add</code> , <code>git subtree pull</code>	⚠ CAUTION	New commits on the current branch (a merge); needs a clean working tree	<code>git ls-remote <repository> <ref></code>	<code>git reset --hard ORIG_HEAD</code> if not pushed (🔴 for uncommitted work, Chapter 11); <code>git revert -m 1</code> if pushed
<code>git subtree push</code>	⚠ CAUTION	Creates or updates a branch in the library's repository	<code>git subtree split first</code> and inspect the result	Delete or reset that branch in the library's repository

23.18 Version notes

Version note. Older behavior: submodule commands could clone from local paths without restriction. Current behavior: transfers that Git starts by itself over the `file` transport are refused unless `protocol.file.allow` permits them. Since: Git 2.38.1 and the maintenance releases issued with it ([release notes](#)). Recommended: leave the default; use `-c protocol.file.allow=always` on single commands in local test setups.

- The local manual of Git 2.55.0 documents `git submodule add --ref-format`, `set-branch`, `set-url` and `absorbgitdirs`. The last one moves a `.git` directory that is embedded in a submodule's working tree into the superproject's `.git/modules/`.

Unverified. The release in which `git submodule update` began to fetch a missing commit directly by ID ("trying to directly fetch"), and the release that introduced `submodule.recurse`, were not looked up for this chapter. Both behaviors were run on Git 2.55.0 and are shown above.

Outdated advice. "After cloning, run `git submodule init` and then `git submodule update`." It still works. `git clone --recurse-submodules`, or `git submodule update --init --recursive` in an existing clone, does the same in one step and also handles nested submodules.

23.19 Practice

- Lab 15.1 (a submodule that breaks for a teammate) and Lab 15.2 (the same dependency as a subtree) in the Module 15 lab manual.
- Replay any transcript with `labs/run ch23/<demo>`, for example `labs/run ch23/submodule-update`.
- Two drills. In the sandbox of `ch23/submodule-conflict`, resolve the conflict the other way: merge inside the submodule yourself, and then list everything that must be pushed, and in which order, before a teammate can use your merge. In the sandbox of `ch23/subtree`, find with `git log --grep` the commit that records the current upstream version of the vendored library, and say what would happen to the next `git subtree pull` if that commit's message were edited.

23.20 Interview questions

- What exactly does a superproject store about a submodule, and what does it not store? Where is each piece?
- A colleague says "our submodule tracks the library's `main`". Correct the statement and explain what `submodule.<name>.branch` does.

3. Why is HEAD detached in a submodule after `git submodule update`? When does that lose work, and how do you get it back?
4. After `git pull`, `git status` shows a modified submodule that you never touched. Explain the state of the three relevant commit IDs, the risk, and the fix.
5. A teammate's pull fails with `not our ref`. Give the root cause, the three commands that prove it, the fix, and the setting that prevents it.
6. Why does Git refuse `file` URLs for submodules by default, and why is a recursive clone of an untrusted repository a security decision?
7. Two branches moved the same gitlink and the merge conflicts. Why can Git not resolve it, when does it resolve it by itself, and what does a correct resolution look like?
8. Explain how `git subtree split` can reproduce the original upstream commit IDs. What would break that?
9. Submodule, subtree, package manager: decide for (a) an internal protocol-definition repository used by eight services, (b) a 300-line utility copied from an open-source project, (c) a tokenizer library published on a package index.
10. You are asked to introduce submodules for a team of thirty. Which four configuration settings and which CI check do you require, and why?

23.21 Sources

Primary sources

- [git-submodule](#), [gitsubmodules](#), [gitmodules](#), [git-push](#) (`--recurse-submodules`). The local copies (`git help -m submodule`, `git help -m gitsubmodules`, `git help -m config`) are the Git 2.55.0 text that the transcripts were checked against.
- [git-subtree](#) in Git's `contrib/` directory, read on 2 October 2026; locally, `git subtree -h`.
- [Protocol configuration](#); release notes [2.38.1](#) and [2.30.6](#).
- Security advisories [GHSA-8h77-4q3w-gfgv](#) (CVE-2024-32002) and [GHSA-vwqx-4fm8-6qc9](#) (CVE-2025-48384).
- `actions/checkout` [action.yml](#) and [README](#) at v7.0.1, the README read on 2 October 2026.

Secondary sources

- Pro Git, [Git Tools: Submodules](#), which documents the failure modes of sections 23.5 to 23.10 from the user's side.
- The Phase 0 report of this course, section 12 (the misconception table), section 13 ("Scale, submodules and large files") and section 14 ("The Git client").

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- Tobias Günther for freeCodeCamp, ["Advanced Git Tutorial"](#) (34 min, 2021): includes a submodules segment. Concepts current; mixes `master` and `main` and uses `checkout`.

Further reading

- [gitrepository-layout](#) for `modules/`; Chapter 24 for the alternative of one repository.

Chapter 24: Monorepos

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch24/`.

24.1 Why this matters

Four questions a CTO can ask in one planning meeting:

1. "Every cross-service change is five pull requests in forty repositories that must merge in order. Should we move to one repository?"
2. "Google and Meta run monorepos. Why does `git status` take seconds in ours?"
3. "A new engineer on the ranking team needs four directories out of nine hundred. Does she have to download and check out everything?"
4. "A required check has been 'expected' for two days on a documentation pull request. What is broken?"

The answers, in order. A monorepo exchanges coordination across repositories for tooling inside one (section 24.2). The repositories those two companies are known for are not Git repositories, and stock Git needs features switched on at that size (sections 24.2 and 24.3, and Chapter 26). No: a partial clone with a cone-mode sparse checkout downloads and checks out only what she works on (sections 24.4 to 24.7). And nothing is broken in Git: a workflow that a path filter kept from starting never reports, so the requirement cannot be met (section 24.9).

A monorepo is not a Git feature. It is a decision about where the boundary of a repository lies, and Git stores it like any repository: one object database, one set of refs, one index per working tree. This chapter covers the decision and the working-tree side; Chapter 26 covers history, storage and transfer.

Both chapters use `orbit`, a small search platform in one repository: three services, two libraries, two pipelines, documentation and tooling, 33 files, 94 commits on `main`. It is small so that you can read every transcript to the end. The mechanisms are the same at ten thousand times the size.

24.2 Monorepo versus polyrepo

In one sentence. A monorepo keeps many separately deployed projects in one repository so that one commit can change all of them; a polyrepo layout gives each project its own history, access rules and release cadence.

Analogy. One shared kitchen against a row of food stalls. In the kitchen a change to the menu is one conversation, and one cook's mess slows everyone. The stalls never collide, and agreeing on anything takes a meeting. The analogy breaks at cost: a kitchen gets crowded by people, a repository gets slow by files, commits and refs, whoever made them.

Precisely. The layouts differ in where a change crosses a boundary.

Concern	Monorepo	Polyrepo
A change to a library and its users	One commit, one review. In <code>orbit</code> , commit <code>100bb99</code> changes a schema, the gateway and the ingest worker together	One pull request per repository, merged in order, with a published version between them
Internal dependency versions	One: what is in the tree	Each consumer pins; consumers drift
Access control	Git has no per-directory read permission: who can clone reads everything	Per repository
Ownership and review	Needs path-based rules (section 24.8)	Follows the repository
Continuous integration	Must compute what a change affects (section 24.9)	Each repository builds itself; integration is tested late
Git performance	Degrades with total size unless tuned	Rarely a concern
Tooling investment	A dependency-aware build system and its maintainers	Release and dependency-update automation
Failure radius	A broken <code>main</code> blocks everyone	A broken <code>main</code> blocks one team

Neither column wins. Google's paper on its monorepo lists the benefits (one source of truth, shared code, atomic changes, large-scale refactoring) and the costs: heavy investment in tooling and in code health ([Potvin and Levenberg, CACM 2016](#)).

The largest monorepos are not Git.

- **Google** keeps its main codebase in Piper, a centralized system it built itself, because it found no existing version control system able to host about one billion files and 35 million commits (figures of January 2015, same paper).
- **Meta** serves a repository of tens of millions of files from its own server and virtual file system, with Sapling as the client. Sapling can also work with Git repositories; Meta's main repository is not one ([Engineering at Meta, 2022](#)).
- **Microsoft** did move a very large codebase to Git: Windows, about 3.5 million files and 300 GB in 2017, first with a virtual file system and then with Scalar, whose techniques were contributed to Git ([Brian Harry, 2017](#); [The Story of Scalar](#)). Sections 24.4 to 24.7 come from that

work.

Git monorepos of ordinary company size are documented by the people who run them: Canva with about half a million files in 2022 ([Canva](#)), Dropbox shrinking its monorepo from 87 GB to 20 GB in 2026 ([Dropbox](#)), Uber running Git operations on its Go monorepo as an internal service ([Uber](#)). Each figure is that company's own measurement and no benchmark.

In production. For an AI/ML platform, split by coupling. Feature code, training pipeline, serving code and shared schemas change together and benefit from one repository. Components with other owners or other access rules are separate repositories, because Git cannot hide a directory. Model weights and datasets belong in neither (Chapter 22, Chapter 28). No primary study of this choice for ML platforms was found for this course; treat it as a trade-off, not a rule.

24.3 What Git has to carry

Size has independent dimensions, and each slows different commands. Sorting a complaint into the right row is the first step of a diagnosis.

Dimension	Commands that feel it	What addresses it
Tracked files (index entries, <code>lsstat</code> calls)	<code>git status</code> , <code>git add</code> , <code>git switch</code>	Sparse-checkout, sparse index (sections 24.4 to 24.6); untracked cache, file-system monitor (Chapter 26, section 26.9)
Commits and trees	<code>git log -- <path></code> , <code>git blame</code> , <code>git merge-base</code>	Commit-graph (26.6)
Blobs in all versions	<code>git clone</code> , <code>git fetch</code> , disk	Partial clone (24.7, 26.12); keeping large binaries out (Chapter 22)
New objects and packs per day	Everything, gradually	Maintenance (26.3 to 26.5)
Refs	<code>git fetch</code> , <code>git push</code>	Narrow refsspecs, pruning (Chapter 12)

The file `labs/ch24/fixture-orbit.bash` builds `orbit` for every demo and lab of both chapters, with the same commit IDs each time.

24.4 Sparse-checkout in cone mode

In one sentence. `git sparse-checkout` makes Git write only chosen directories into the working tree, while the repository, its history and its index still contain every file.

Analogy. A library with closed stacks: the catalogue lists every book, and your desk holds the shelves you asked for. The analogy breaks in one place: in a normal clone the stacks are in your building too, since every object is on your disk. Section 24.7 moves them out.

Precisely. The definition is a list of patterns in `$GIT_DIR/info/sparse-checkout`. For every index entry the patterns do not select, Git sets the **skip-worktree** bit, does not write the file, and does not report it as deleted. In **cone mode**, the default, you name directories: each is included with everything below it, and each of its parents with the files directly inside it, the top level always included. That restriction lets Git decide inclusion by hash lookup instead of matching every path against every pattern, and it is the precondition for the sparse index. The older pattern mode is deprecated in the manual's own words.

The manual carries this warning: "THIS COMMAND IS EXPERIMENTAL. ITS BEHAVIOR, AND THE BEHAVIOR OF OTHER COMMANDS IN THE PRESENCE OF SPARSE-CHECKOUTS, WILL LIKELY CHANGE IN THE FUTURE." The command dates from Git 2.25 and cone mode is the default since 2.37; the label is still there in 2.56 ([git-sparse-checkout](#)).

The subcommands: `set <dirs>` 🟡 switches the feature on and replaces the list; `add <dirs>` 🟡 extends it; `list` 🟢 prints it; `reapply` 🟡 enforces it again; `disable` 🟡 switches it off; `init --cone` 🟡 is the older spelling of `set` with no directories, which the manual calls deprecated; `clean` 🔴 is in section 24.5.

See it. A full clone of `orbit`, by project:

```
$ ls
docs
libs
pipelines
pyproject.toml
README.md
services
tools
$ git ls-files | wc -l
33
$ git ls-files | cut -d/ -f1-2 | sort | uniq -c
1 .github/CODEOWNERS
1 .gitignore
1 docs/architecture.md
3 docs/runbooks
7 libs/schemas
2 libs/tokenizer
2 pipelines/eval
```

```

2 pipelines/training
1 pyproject.toml
1 README.md
4 services/gateway
3 services/ingest
4 services/ranker
1 tools/ci

```

Older tutorials start with `init --cone`. It still works and shows where every cone starts: the top-level files.

```

$ git sparse-checkout init --cone
$ ls -A
.git
.gitignore
pyproject.toml
README.md
$ git status
On branch main
Your branch is up to date with 'origin/main'.

You are in a sparse checkout with 10% of tracked files present.

nothing to commit, working tree clean
$ git sparse-checkout list
[exit status: 0]

```

Inside `.git`. Three things changed, and no object or ref did:

```

# The definition is a file of patterns, and the switches live in a per-worktree config file:
$ cat .git/info/sparse-checkout
/*
!/*/
$ cat .git/config.worktree
[core]
    sparseCheckout = true
    sparseCheckoutCone = true
$ git config get extensions.worktreeConfig
true

```

The two patterns read "everything directly at the top level, but no directory". The switches are not in `.git/config` but in `.git/config.worktree`, which applies to one working tree and is read because `extensions.worktreeConfig` is now `true`: each linked working tree (Chapter 25) can have its own cone.

Now name the directories you work in. `set` alone would have done what `init` did:

```

$ git sparse-checkout set services/ranker libs/tokenizer
$ git sparse-checkout list
libs/tokenizer
services/ranker
$ find . -path ./git -prune -o -type f -print | sort
./gitignore
./libs/tokenizer/tokenizer.py
./libs/tokenizer/vocab.txt
./pyproject.toml
./README.md
./services/ranker/config.yaml
./services/ranker/features.py
./services/ranker/model.py
./services/ranker/tests/test_model.py
$ git status | sed -n 4p
You are in a sparse checkout with 28% of tracked files present.

```

```

$ cat .git/info/sparse-checkout
/*
!/*/
/libs/
!libs/*/
/services/
!services/*/
libs/tokenizer/
services/ranker/
# Every tracked file is still in the index. Those outside the cone carry the skip-worktree bit (S):
$ git ls-files | wc -l
33
$ git ls-files -t | cut -c1 | sort | uniq -c
9 H
24 S
$ git ls-files -t services
S services/gateway/app.py
S services/gateway/config.yaml
S services/gateway/routes.py
S services/gateway/tests/test_routes.py
S services/ingest/settings.py

```

```
S services/ingest/tests/test_worker.py
S services/ingest/worker.py
H services/ranker/config.yaml
H services/ranker/features.py
H services/ranker/model.py
H services/ranker/tests/test_model.py
```

Read the pattern file in pairs: `/libs/` with `!/libs/*/` includes `libs` without its subdirectories (a parent), `/libs/tokenizer/` includes one subdirectory with everything below (recursive). You never edit this file in cone mode. The index still has 33 entries; `git ls-files -t` ● marks ordinary entries `H` and skip-worktree entries `S`. The gateway is in the index, in `HEAD` and in the object database. It is not on disk.

```
$ git sparse-checkout add docs/runbooks
$ git sparse-checkout list
docs/runbooks
libs/tokenizer
services/ranker
# The parent directory arrives with its own files, but not with its other subdirectories:
$ ls docs docs/runbooks
docs:
architecture.md
runbooks

docs/runbooks:
gateway-latency.md
ingest-backlog.md
ranker-rollback.md
$ git status | sed -n 4p
You are in a sparse checkout with 40% of tracked files present.
```

`docs/runbooks` was requested and `docs/architecture.md` came with it, because `docs` became a parent and parents bring their own files. There is no switch for that.

```
$ git sparse-checkout disable
$ ls
docs
libs
pipelines
pyproject.toml
README.md
services
tools
$ git ls-files -t | cut -c1 | sort | uniq -c
33 H
$ git status | sed -n 1,4p
On branch main
Your branch is up to date with 'origin/main'.

nothing to commit, working tree clean
# The switch is off. The old definition stays on disk, unused:
$ cat .git/config.worktree
[core]
    sparseCheckout = false
    sparseCheckoutCone = false
[index]
    sparse = false
$ wc -l < .git/info/sparse-checkout
11
```

`disable` writes every file back and sets the switches to `false`; the pattern file stays on disk, unused.

Picture.

HEAD (tree of 100bb99)	index: 33 entries	working tree: 9 files
<pre>+-----+ README.md docs/... libs/schemas/... libs/tokenizer/... services/gateway/... services/ranker/... +-----+</pre>	<pre>+-----+ H README.md S docs/... S libs/schemas/... H libs/tokenizer/... S services/gateway/... H services/ranker/... +-----+</pre>	<pre>+-----+ README.md libs/tokenizer/... services/ranker/... +-----+</pre>
every commit complete	S = skip-worktree	only the H entries

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code>git sparse-checkout set <dirs></code>	Files outside the cone removed, inside written	Same entries; skip-worktree bits changed	unchanged	unchanged	<code>info/sparse-checkout</code> , <code>config.worktree</code> , <code>extensions.worktreeConfig</code>	unchanged	unchanged

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git sparse-checkout add <dirs></code>	Files of the added directories written	Bits cleared for them	unchanged	unchanged	<code>info/sparse-checkout</code> extended	unchanged	unchanged
<code>git sparse-checkout disable</code>	Every tracked file written	All bits cleared	unchanged	unchanged	Switches <code>false</code> ; pattern file kept	unchanged	unchanged

In production. A sparse checkout is a property of one working tree. It is not committed and not pushed. A team that wants every member to start with the same cone puts the `set` line into its onboarding script. Canva used the feature to leave out generated translation files, about 70% of all files in its repository (article cited in 24.2).

24.5 Living in a sparse checkout

One rule explains every difference: a path with the skip-worktree bit is in the index and in history, and Git does not look at it on disk.

```
# History and objects are complete. Only the working tree is narrow:
$ git log --oneline -2 -- services/gateway
100bb99 schemas, gateway, ingest: add the tenant field in one change
cb65af7 gateway: add search route 12
$ git show HEAD:services/gateway/config.yaml
timeout_ms: 800
upstream: ranker
# A search of the working tree sees only what is checked out; --cached searches the index:
$ git grep -l tenant
[exit status: 1]
$ git grep -l --cached tenant
libs/schemas/events.py
services/gateway/app.py
services/ingest/worker.py
```

Commands that read commits, trees and blobs see the whole repository. `git grep` without options searches the working tree, finds nothing in files that are not there, and exits with "no match": a wrong answer without a warning. `git grep --cached` searches the index.

The cone pays off in work per command. The pipeline is explained in Chapter 26, section 26.2; it prints counters that Git reports about itself, not times:

```
# The work of one status, counted (Chapter 26, section 26.2 explains the pipeline):
$ GIT_TRACE2_PERF=1 git status 2>&1 >/dev/null | awk -F'|' '$4 ~ /data/ {gsub(/[.]/, "", $NF); gsub(/ /, "", $(NF-1)); print $(NF-1), $NF}' | grep -e read/cache_nr -e sum_lstat -e visited
index read/cache_nr:33
index refresh/sum_lstat:9
read_directo directories-visited:6
read_directo paths-visited:15
```

The index still has 33 entries to read, but `git status` asked the file system about nine files instead of 33 and walked six directories instead of twenty.

Files outside the cone. A working tree gets them in two everyday ways. The first: you or a tool create one.

```
# A new file in a directory outside the cone:
$ mkdir -p services/gateway
$ printf '# Gateway notes\n\nRoutes are versioned under /v1.\n' > services/gateway/NOTES.md
$ git add services/gateway/NOTES.md
The following paths and/or pathspecs matched paths that exist
outside of your sparse-checkout definition, so will not be
updated in the index:
services/gateway/NOTES.md
hint: If you intend to update such entries, try one of the following:
hint: * Use the --sparse option.
hint: * Disable or modify the sparsity rules.
hint: Disable this message with "git config set advice.updateSparsePath false"
[exit status: 1]
$ git status --short
?? services/gateway/NOTES.md
```

```
$ git add --sparse services/gateway/NOTES.md
$ git commit -q -m 'gateway: add notes on route versioning'
$ git ls-files -t services/gateway
H services/gateway/NOTES.md
S services/gateway/app.py
S services/gateway/config.yaml
S services/gateway/routes.py
S services/gateway/tests/test_routes.py
# The file is committed and still on disk although its directory is outside the cone:
```



```
$ ls services/gateway
NOTES.md
$ git sparse-checkout reapply
$ ls services
ranker
$ git ls-files -t services/gateway/NOTES.md
S services/gateway/NOTES.md
```

`git add` refuses with exit status 1 and names the ways forward. With `--sparse` the file is staged and committed, and it stays on disk as an H entry among S entries until `git sparse-checkout reapply` removes it. The second way, in the manual's description of `reapply`: a merge or rebase may "materialize paths to do their work (e.g. in order to show you a conflict)". The same `reapply` cleans up after the resolution.

Untracked and ignored files. Narrowing a cone removes tracked files. What about the rest?

```
# Widen the cone, leave an untracked file behind, and narrow it again:
$ git sparse-checkout add pipelines/eval
$ printf 'recall@10 = 0.83\n' > pipelines/eval/results.tmp
$ git sparse-checkout set services/ranker libs/tokenizer
warning: directory 'pipelines/' contains untracked files, but is not in the sparse-checkout cone
$ find pipelines -type f
pipelines/eval/results.tmp
$ git status --short
?? pipelines/eval/results.tmp
```

```
# Preview first: the cleanup removes the whole directory, the untracked file included.
$ git sparse-checkout clean --dry-run
Would remove pipelines/
$ git sparse-checkout clean
fatal: for safety, refusing to clean without one of --force or --dry-run
[exit status: 128]
$ git sparse-checkout clean -f
Removing pipelines/
$ ls pipelines
ls: pipelines: No such file or directory
[exit status: 1]
```

Git kept the untracked file and its directory and warned. `git sparse-checkout clean` (Git 2.52 or later) removes such directories. What it changes: the working tree. What it can destroy: untracked files there, here `results.tmp`, which were never in Git, so no reflog and no `git fsck` returns them. Preview: `--dry-run`, with `--verbose` for every file. Recovery: none from Git. Appropriate: after reading the preview and moving away what you need.

Ignored files get no warning. The manual states the rule: if everything untracked in a directory outside the cone is ignored, "then the directory will be deleted".

```
# The same sequence with a file that .gitignore covers (build/ is ignored in this repository):
$ git sparse-checkout add pipelines/eval
$ mkdir pipelines/eval/build && printf 'cached features\n' > pipelines/eval/build/features.bin
$ git status --short --ignored
!! pipelines/eval/build/
$ git sparse-checkout set services/ranker libs/tokenizer
$ ls pipelines
ls: pipelines: No such file or directory
[exit status: 1]
```

Exit status 0, and the cache is gone. For a build cache that is a convenience. For an ignored file you cannot regenerate, such as a local `.env` or a downloaded evaluation set, narrowing the cone is a  operation whose only preview is `git status --short --ignored`.

```
Observed behavior : a search or a file-listing script reports that most of the repository is gone,
                    in one engineer's clone only.
Git state          : core.sparseCheckout=true in .git/config.worktree; most index entries carry the
                    skip-worktree bit; "git status" prints "You are in a sparse checkout".
Mechanism          : commands that read the working tree see the cone; commands that read the index
                    or commits see everything.
Root cause         : a working-tree question was asked where a repository question was meant.
Why Git does this  : the purpose of the feature is that the files are not on disk.
Correct fix        : ask Git: "git grep --cached", "git ls-files", "git ls-tree -r HEAD".
Prevention         : scripts that must see all files read from Git, not from the directory listing.
```

24.6 The sparse index

In one sentence. A sparse index replaces all entries of a directory outside the cone by one entry that names the directory's tree, so the index shrinks with the cone.

Analogy. A table of contents that lists every section of the chapters you read and one line for each other chapter. Whoever needs a section of a collapsed chapter must expand the line first, and the expansion is the slow part. The analogy breaks at who decides: a reader chooses to expand, while Git expands whenever a command asks for a path that the collapsed entry cannot answer.

Precisely. Section 24.5 left `read/cache_nr:33` untouched: every command still reads, and often rewrites, an index proportional to the whole repository. With `index.sparse=true`, written by `git sparse-checkout set --sparse-index` 🟡, the index may hold **sparse directory entries**: mode `040000`, a tree ID, the skip-worktree bit, a path ending in a slash ([design notes](#)). It needs cone mode. The manual marks it separately: "This feature is still experimental", and warns that external tools may not understand such an index.

See it.

```
# A sparse checkout with an ordinary index: 33 entries for 7 files on disk.
$ git ls-files --sparse | wc -l
    33
$ git ls-files -t | cut -c1 | sort | uniq -c
    7 H
   26 S
```

```
$ git sparse-checkout set --sparse-index services/ranker
$ cat .git/config.worktree
[core]
    sparseCheckout = true
    sparseCheckoutCone = true
[index]
    sparse = true
$ git ls-files --sparse | wc -l
   14
```

```
# A whole directory outside the cone is now a single entry that names a tree object:
$ git ls-files --sparse --stage
040000 a3f81eb96031c5886197d2da2d1aeb5d17e4c646 0      .github/
100644 982c4febe4ff8d8dd6daee80ed5d8a282f228d44 0      .gitignore
100644 b47224884eaf3c9f9ac2bfb32638072b748d8b7 0      README.md
040000 1e616b23f37f85ce3b07a9157ade65588054b583 0      docs/
040000 ac165d9f7472f790a4477c93edf90960e84774a7 0      libs/
040000 01bedcc4c53928618088e5c7bf68ca94f1757667 0      pipelines/
100644 897065b8f51bfbd091c267898cef63ea44564ee3 0      pyproject.toml
040000 74cd101987d4a8b085ecf364708437364befbf28 0      services/gateway/
040000 710e8eb227d465fde35a63d622e4040521e6f339 0      services/ingest/
100644 b89d0b7fac309a1df9cf9c4c1e57cc77e37ba8f6 0      services/ranker/config.yaml
100644 2e52099a797292009b44e5b9d721e5b37aeb7197 0      services/ranker/features.py
100644 fa20c90bd0927af350059ca40e00a7d2e390193b 0      services/ranker/model.py
100644 dd50a7eb0825765c50f09d9d205ff43c89f15916 0      services/ranker/tests/test_model.py
040000 c2505f39bb5212640238d272ed2c9f50aec4e0d1 0      tools/
```

Fourteen entries instead of 33. `docs/`, `libs/`, `pipelines/` and `tools/` are one entry each; `services/` is not collapsed because the cone reaches into it.

Inside .git. The index file is rewritten, and `index.sparse = true` joins the switches in `config.worktree`. The tree IDs are those of `HEAD`, so Git can compare such an entry with a commit without opening it:

```
# The entry for docs/ holds exactly the ID of the docs tree in HEAD:
$ git rev-parse HEAD:docs
1e616b23f37f85ce3b07a9157ade65588054b583
$ git ls-files --sparse --stage docs/
040000 1e616b23f37f85ce3b07a9157ade65588054b583 0      docs/
$ git status
On branch main
Your branch is up to date with 'origin/main'.

You are in a sparse checkout.

nothing to commit, working tree clean
```

```
$ GIT_TRACE2_PERF=1 git status 2>&1 >/dev/null | awk -F'|' '{ $4 ~ /data/ { gsub(/[.]/, "", $NF); gsub(/ /, "", $(NF-1)); print $(NF-1), $NF }' | grep -e read/cache_nr -e sum_lstat
index read/cache_nr:14
index refresh/sum_lstat:7
```

Both counters now follow the cone. On `orbit` that saves nothing you can feel. GitHub's engineers reported an index shrinking from about 180 MB to under 10 MB on a test repository of two million files ([GitHub Blog, 2021](#)); the figure is theirs.

The cost. A command that needs paths inside a collapsed directory expands the index in memory first:

```
# A command that asks for every path expands the index in memory, and says so:
$ git ls-files | wc -l
hint: The sparse index is expanding to a full index, a slow operation.
hint: Your working directory likely has contents that are outside of
hint: your sparse-checkout patterns. Use 'git sparse-checkout list' to
hint: see your sparse-checkout definition and compare it to your working
hint: directory contents. Cleaning up any merge conflicts or staged
hint: changes before running 'git sparse-checkout clean' or 'git
hint: sparse-checkout reapply' may assist in this cleanup.
```

```

hint: Disable this message with "git config set advice.sparseIndexExpanded false"
33
# The file on disk stays sparse:
$ git ls-files --sparse | wc -l
14

```

`git ls-files` without `--sparse` lists every path, so it expands, and Git says that this is "a slow operation". The hint's guess at the cause is wrong here. A tool that runs such a command on every prompt or save cancels the benefit. `git sparse-checkout set --no-sparse-index <dirs>` goes back, and the transcript `ch24/sparse-index/07-back` shows it.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git sparse-checkout set --sparse-index <dirs></code>	As set	Rewritten: one tree entry per directory outside the cone	unchanged	unchanged	<code>index.sparse=true</code> in <code>config.worktree</code>	unchanged	unchanged

In production. Use it when the index itself is the bottleneck, which means hundreds of thousands of tracked files, and when the tools that share the working tree (editor integration, prompt, hook framework, an older Git) cope with it.

24.7 Partial clone plus sparse-checkout, and what `scalar clone` configures

In one sentence. A blobless partial clone leaves file contents on the server until something needs them, a sparse checkout limits that "something" to your cone, and `scalar clone` sets up both together with settings and scheduled maintenance.

Precisely. A sparse checkout alone still downloaded every version of every file. `git clone --filter=blob:none` asks the server for commits and trees without blobs, and Git fetches a blob when a command first needs it (Chapter 26, section 26.12). With `--sparse`, the checkout needs only the blobs of the cone.

See it. The layout that `scalar clone` creates, an enlistment directory with the working tree in `src`, built by hand:

```

$ git clone --filter=blob:none --sparse "file://$PWD/server/orbit.git" orbit-dev/src
Cloning into 'orbit-dev/src'...
$ cd orbit-dev/src
$ ls -A
.git
.gitignore
pyproject.toml
README.md
$ git cat-file --batch-all-objects --batch-check='%(<objecttype>)' 2>/dev/null | sort | uniq -c
  3 blob
 96 commit
  5 tag
284 tree
$ git status | sed -n 4p
You are in a sparse checkout with 10% of tracked files present.

```

Three blobs for three files, and all 96 commits and 284 trees. On one machine this needs a `file://` URL and `uploadpack.allowFilter` on the serving repository; section 26.12 shows what happens without them.

Inside .git. Two independent mechanisms in two files: the promise of the remote belongs to the repository, the cone to the working tree.

```

# What makes it partial is in .git/config; what makes it sparse is in .git/config.worktree:
$ git config list --local --show-origin | grep -i -e promisor -e partialclone -e worktreeconfig
file:.git/config      remote.origin.promisor=true
file:.git/config      remote.origin.partialclonefilter=blob:none
file:.git/config      extensions.worktreeconfig=true
$ git config list --worktree --show-origin
file:.git/config.worktree core.sparsecheckout=true
file:.git/config.worktree core.sparsecheckoutcone=true

```

```

# Widening the cone downloads the blobs of those directories, and only those:
$ git sparse-checkout set services/ranker libs/tokenizer
$ git cat-file --batch-all-objects --batch-check='%(<objecttype>)' 2>/dev/null | sort | uniq -c
  9 blob
 96 commit
  5 tag
284 tree
$ git rev-list --objects --all --missing=print | grep -c '^?'
113
$ git status | sed -n 4p
You are in a sparse checkout with 28% of tracked files present.

```

Widening the cone fetched six more blobs, the current versions of six files. 113 objects that the history refers to are not here, and nothing is wrong.

```
# History of the cone, fetched in one batch instead of one request per commit:
$ git backfill --sparse
$ git cat-file --batch-all-objects --batch-check='%(objecttype)' 2>/dev/null | sort | uniq -c
33 blob
96 commit
5 tag
284 tree
$ git log -p --format=%s -1 -- services/ranker/features.py | sed -n 1,12p
ranker: add ranking signal 12

diff --git a/services/ranker/features.py b/services/ranker/features.py
index 04b3cf5..2e52099 100644
--- a/services/ranker/features.py
+++ b/services/ranker/features.py
@@ -11,4 +11,5 @@ FEATURES = [
     "signal_09",
     "signal_10",
     "signal_11",
+    "signal_12",
 ]
```

`git backfill --sparse` (experimental, Git 2.49 or later) downloads the historical versions of the files in the cone in batches, so that `git log -p` and `git blame` there stop fetching blob by blob.

What `scalar clone` does. `scalar` ships with Git since 2.38 and is not labelled experimental. `git help scalar` describes `scalar clone` in four parts:

1. A partial clone: "By default, only commit and tree objects are cloned", with the working tree in `<enlistment>/src`.
2. A cone-mode sparse checkout with only the top-level files, unless `--full-clone`.
3. Configuration "to optimize for large repositories", each value with its reason in the manual: among others `commitGraph.changedPaths=true`, `core.untrackedCache=true`, `index.version=4`, `fetch.unpackLimit=1`, `gc.auto=0`, `status.aheadBehind=false`, `fetch.showForcedUpdates=false`, `pack.useBitmaps=false`, `log.excludeDecoration=refs/prefetch/*`.
4. Background maintenance, the scheduler of `git maintenance start` (section 26.3), unless `--no-maintenance`.

Why there is no `scalar clone` transcript here. The lab rules forbid demos that install schedulers or start daemons. A contained test for this chapter (Git 2.55.0, macOS, a sandbox with its own `HOME`, the scheduler replaced by a logging stub) found: with `--no-maintenance` the scheduler was not called; the enlistment was registered in the global configuration as `scalar.repo`; and the repository received `core.fsmonitor=true` while `scalar` ran `git fsmonitor--daemon start`, which left a daemon running. `--no-maintenance` does not prevent that, and the 2.55 manual page does not list the setting. The source does ([scalar.c](#)).

On your own machine `scalar clone` is a reasonable start for a large repository if you know what it leaves behind: a registration in your global configuration, scheduler entries (on macOS under `~/Library/LaunchAgents`, per `git help maintenance`) and one monitor process per enlistment. `scalar unregister` undoes the first two for a repository, `scalar delete <enlistment>` also removes the directory, and `git fsmonitor--daemon stop` inside the working tree stops the daemon.

The hand-built clone has the same data layout without those side effects. What remains is settings and maintenance:

```
# A few of the settings that scalar would write, set by hand (the manual lists them all):
$ git config set commitGraph.changedPaths true
$ git config set status.aheadBehind false
$ git config set fetch.showForcedUpdates false
$ git config set advice.fetchShowForcedUpdates false
$ git config set log.excludeDecoration "refs/prefetch/*"
$ git config set maintenance.auto false
$ git config set maintenance.strategy incremental
```

```
# What the scheduler would run every hour, run once in the foreground:
$ GIT_TRACE="$PWD/./maintenance.log" git maintenance run --task=prefetch --task=commit-graph
$ sed -n 's/.trace: run_command: git //p' ../maintenance.log | grep -v -e '^maintenance' -e '^pack-objects' -e '^index-pack'
fetch origin --prefetch --prune --no-tags --no-write-fetch-head --recurse-submodules=no --quiet
commit-graph write --split --reachable --no-progress
$ git for-each-ref --format='%(<refname>)' refs/prefetch
refs/prefetch/remotes/origin/feature/rerank-cache
refs/prefetch/remotes/origin/main
# The prefetched refs do not move your remote-tracking branches and stay out of the log:
$ git log --oneline --decorate -1
100bb99 (HEAD -> main, tag: schemas/v1.1.0, origin/main, origin/HEAD) schemas, gateway, ingest: add the tenant field in one change
```

The `prefetch` task fetches into `refs/prefetch/`. Your remote-tracking branches do not move, and when you run `git fetch` yourself the objects are already local. Lab 18.4 builds this clone and then cuts the connection to the server.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code>git clone --filter=blob:none --sparse</code>	Top-level files	All entries; skip-worktree below the	Default branch	Created	Promisor configuration and packs; sparse switches	unchanged	unchanged

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<url>		top level					
git backfill --sparse	unchanged	unchanged	unchanged	unchanged	New promisor packs	Read only	unchanged
git maintenance run --task=prefetch	unchanged	unchanged	unchanged	unchanged	Objects; refs/prefetch/*	Read only	unchanged

24.8 Ownership at scale: CODEOWNERS

GitHub, not Git. Git stores `.github/CODEOWNERS` as an ordinary file and gives it no meaning.

In a monorepo, ownership is a function of the path, and on GitHub that function is the `CODEOWNERS` file. `orbit` maps `/services/gateway/` to `@orbit/gateway`, `/services/ingest/` to `@orbit/data`, `/libs/` to `@orbit/platform`, and so on. Chapter 19 teaches the file. Four documented facts decide whether it works at this scale ([About code owners](#)):

- The **last matching pattern** wins, so a broad rule below a specific one replaces it silently.
- The file is read from the pull request's **base branch**: a pull request cannot change who reviews it.
- The file only requests reviews. Requiring them is a ruleset or branch protection setting (Chapter 18).
- A line with invalid syntax is skipped, and the pull request does not fail because of it.

Commit `100bb99` touches `libs/schemas`, `services/gateway` and `services/ingest`: three teams are asked. That is the trade-off of section 24.2 in one pull request.

24.9 Continuous integration at scale

A monorepo's CI must first answer: which projects does this change affect? Building everything is correct and unaffordable; building too little lets a broken `main` through. Git's part of the answer is a diff against the merge base.

```
$ cd orbit
$ git log --oneline --graph -4 main feature/rerank-cache
* 100bb99 schemas, gateway, ingest: add the tenant field in one change
| * 88222b7 ranker: make the cache size configurable
| * 9c9fd21 ranker: cache scores per query and document
|/
* d259a34 docs: record architecture change 12
# What the branch changed since it forked (three dots: diff from the merge base):
$ git diff --name-only main...feature/rerank-cache
services/ranker/config.yaml
services/ranker/model.py
$ git diff --name-only main...feature/rerank-cache | cut -d/ -f1-2 | sort -u
services/ranker
# Two dots compare the two tips, and blame the branch for what main did meanwhile:
$ git diff --name-only main..feature/rerank-cache | cut -d/ -f1-2 | sort -u
libs/schemas
services/gateway
services/ingest
services/ranker
$ cd ..
```

Three dots mean "from the merge base to the second commit": what the branch did since it forked (Chapter 14A, section 14A.2). Two dots compare the tips and charge the branch with what `main` did meanwhile, so a pipeline built on them runs, and blames, jobs for projects the pull request never touched. A build system (section 24.10) then extends the list by dependency, which Git cannot know.

The shallow-clone trap. The merge base is a commit in the past, and a job that fetches one commit per branch does not have it:

```
# The CI job clones the branch with depth 1 and fetches the tip of main the same way:
$ git clone -q --depth 1 --branch feature/rerank-cache "file://$PWD/server/orbit.git" ci-shallow
$ cd ci-shallow
$ git fetch -q --depth 1 origin main:refs/remotes/origin/main
$ git log --oneline --all
100bb99 schemas, gateway, ingest: add the tenant field in one change
88222b7 ranker: make the cache size configurable
$ git diff --name-only origin/main...HEAD
fatal: origin/main...HEAD: no merge base
[exit status: 128]
$ git merge-base origin/main HEAD
[exit status: 1]
$ cd ..
```

Observed behavior : the "what changed" step fails with "no merge base", or a script that swallows the error concludes that nothing changed and skips every test.

```

Git state      : a shallow repository; "git log --all" shows two commits with no connection.
Mechanism     : each fetch with --depth 1 delivered one commit and recorded it as a boundary.
Root cause    : a history question asked of a clone that was told to have no history.
Why Git does this : a shallow clone is "these commits and nothing behind them" (Chapter 26, 26.11).
Correct fix    : a full-depth fetch, or a blobless clone: every commit and tree, no file content
                 that is not checked out.
Prevention     : jobs that build one snapshot may be shallow; jobs that compare, describe or
                 compute versions may not.

```

```

# A blobless clone without a checkout has every commit and tree and not a single file:
$ git clone -q --filter=blob:none --no-checkout --branch feature/rerank-cache "file://$PWD/server/orbit.git" ci-blobless
$ cd ci-blobless
$ git cat-file --batch-all-objects --batch-check='%(<objecttype>)' 2>/dev/null | sort | uniq -c
  96 commit
   5 tag
 284 tree
$ git merge-base origin/main HEAD
d259a34f3117e4516fb2f7c071b2dd9c8edc9e5b
$ git diff --name-only origin/main...HEAD | cut -d/ -f1-2 | sort -u
services/ranker
# Then check out only what the job needs:
$ git sparse-checkout set services/ranker libs/tokenizer
$ git checkout -q
$ find . -path ./git -prune -o -type f -print | sort
./gitignore
./libs/tokenizer/tokenizer.py
./libs/tokenizer/vocab.txt
./pyproject.toml
./README.md
./services/ranker/config.yaml
./services/ranker/features.py
./services/ranker/model.py
./services/ranker/tests/test_model.py

```

96 commits, 284 trees and no blob are enough for `git merge-base` and for `git diff --name-only`, which compares trees. The job then checks out the affected project and its dependencies: nine files.

GitHub Actions, not Git. By default `actions/checkout` fetches one commit and no tags, which is the state above; its README documents inputs for depth, a partial-clone filter and sparse-checkout ([README at v7.0.1](#); Chapter 20A). Not run here.

The path-filter and required-check trap. Saving CI time with a `paths` filter and protecting `main` by requiring that workflow's check do not combine. A workflow that a path filter, a branch filter or `[skip ci]` keeps from starting never reports, so its required check stays pending and the pull request cannot merge ([skipping workflow runs](#)). A job that starts and is skipped by a condition reports success ([troubleshooting required status checks](#)). That is question 4 of section 24.1. The design that follows is an inference of this course, not a GitHub recipe: start one workflow on every pull request, compute the affected projects inside it, skip jobs by condition, and require one final job that fails if any job that ran has failed (Chapters 20A and 20B).

24.10 Build systems, by name and purpose

Git answers "which files changed". A monorepo also needs "which targets depend on them, and which results can be reused". That is the work of a build system that models the dependency graph: it rebuilds and retests only what a change can affect and caches results by the content of their inputs. This book teaches none of them. Know the names: [Bazel](#) ("build and test your multi-language, multi-platform projects"), [Buck2](#) ("an open-source large-scale build system from Meta"), [Pants](#) ("currently focused on Python, Go, Java, Scala, Kotlin, Shell, and Docker"), [Nx](#) and [Turborepo](#) (both describe themselves as build systems for monorepos), and [Gradle](#) ("for Java, Android, and Kotlin developers"). The quotations are from the tools' own pages as fetched on 2 October 2026 and will change. For Git, two things matter: these tools write large ignored output directories, the very files that narrowing a cone deletes, and their notion of "affected" starts from the diff of section 24.9.

24.11 Versioning and releases in a monorepo

In one sentence. Tags name commits of the whole repository, so a monorepo that releases projects separately puts the project into the tag name and into every question about "the last release".

Precisely. There is one `refs/tags/` namespace per repository (Chapter 14B). The usual convention is a prefix per project; the slash is only a character to Git, and patterns can match it.

```

$ git for-each-ref --format='%(<refname:short>) %(<objecttype>) -> %(<*objectname:short>) %(<*subject>)' refs/tags
gateway/v1.0.0 tag -> 153c828 gateway: add search route 3
gateway/v1.1.0 tag -> f5915c9 gateway: add search route 9
ranker/v0.9.0 tag -> 52ee19b ranker: add ranking signal 6
schemas/v1.0.0 tag -> 6f3c9a4 docs: record architecture change 2
schemas/v1.1.0 tag -> 100bb99 schemas, gateway, ingest: add the tenant field in one change
$ git tag -l 'gateway/*'

```

```
gateway/v1.0.0
gateway/v1.1.0
```

schemas/v1.0.0 points at a documentation commit: a tag names a state of the entire tree, here whatever `main` was when the release was cut.

```
# Without a pattern, describe answers with the nearest tag of any project:
$ git describe main
schemas/v1.1.0
# With a pattern it answers for one project:
$ git describe --match 'gateway/v*' main
gateway/v1.1.0-28-g100bb99
$ git describe --match 'ranker/v*' main
ranker/v0.9.0-48-g100bb99
```

`git describe` without a pattern answers with the nearest tag of any project. A build that stamps the gateway with `schemas/v1.1.0` is wrong, and nothing fails. `--match` restricts the candidates.

```
# The range between two gateway releases contains everybody else too:
$ git log --oneline gateway/v1.0.0..gateway/v1.1.0 | wc -l
42
# Limit it to the paths that go into the gateway:
$ git log --oneline gateway/v1.0.0..gateway/v1.1.0 -- services/gateway libs/schemas
f5915c9 gateway: add search route 9
b8f00f5 gateway: add search route 8
6df5fa6 gateway: add search route 7
bc833b0 gateway: add search route 6
f29a33b gateway: add search route 5
aa79006 gateway: add search route 4
# Has anything that the gateway ships changed since its last release?
$ git log --oneline gateway/v1.1.0..main -- services/gateway libs/schemas
100bb99 schemas, gateway, ingest: add the tenant field in one change
cb65af7 gateway: add search route 12
5f20f82 gateway: add search route 11
47dda3a gateway: add search route 10
$ git diff --quiet gateway/v1.1.0 main -- services/gateway libs/schemas
[exit status: 1]
$ git diff --quiet ranker/v0.9.0 gateway/v1.0.0 -- libs/schemas
[exit status: 0]
```

Of the 42 commits between two gateway releases, six concern the gateway. A changelog, and the question "does this project need a release", must be limited to the paths the project ships, its libraries included. `git diff --quiet` with paths turns the question into an exit status.

In production. Tag-triggered automation must filter on the prefix, or a ranker release deploys the gateway. A shallow clone has no tags behind its boundary, so version stamping fails there (section 26.11). On GitHub a release is attached to one tag (Chapter 15), so the release list mixes all projects. Whether projects share one version number is a product decision that Git neither supports specially nor prevents.

24.12 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A search or linter "cannot find" code that exists	<code>git status</code> says "sparse checkout"; <code>git ls-files -t</code> shows <code>S</code>	<code>git grep --cached</code> , or widen the cone	Scripts read the file list from Git
<code>git add</code> : "outside of your sparse-checkout definition"	<code>git sparse-checkout list</code>	<code>git add --sparse</code> , or add the directory	Keep what you write to inside the cone
Tracked files outside the cone stay on disk	<code>H</code> entries among <code>S</code> entries	<code>git sparse-checkout reapply</code>	<code>reapply</code> after conflicts outside the cone
"contains untracked files, but is not in the sparse-checkout cone"	Untracked, not ignored, files there	Move or commit them; <code>clean --dry-run</code> , then <code>-f</code>	Scratch output outside the repository
A cache or local file vanished after narrowing	It was ignored; ignored content outside the cone is deleted	None from Git	<code>git status --short --ignored first</code>
A tool is slow or fails after <code>--sparse-index</code>	The "expanding to a full index" hint	<code>set --no-sparse-index</code>	Test the tools first
<code>could not fetch <id> from promisor remote</code>	Partial clone, server unreachable	Reconnect and repeat	<code>git backfill --sparse</code> before going offline
CI fails with <code>no merge base</code>	<code>git rev-parse --is-shallow-repository</code> is <code>true</code>	Full depth or a blobless clone	Shallow only for snapshot jobs
A required check stays pending	The workflow has a path filter and never started	Start it always; decide inside	Require one aggregate job
A build carries another project's version	<code>git describe</code> without <code>--match</code>	Add the pattern	One release script per project
An unexplained daemon and scheduler entries	<code>scalar list</code> ; <code>git fsmonitor--daemon status</code>	<code>scalar unregister</code> ; <code>git fsmonitor--daemon stop</code>	Section 24.7

24.13 When not to use a monorepo, and dangerous edge cases

When a monorepo is the wrong tool.

- **Different readers.** Git cannot restrict reading to part of a repository. If some people may see some projects and not others, those are separate repositories. A review rule protects writes, not reads.
- **Unrelated lifecycles.** Projects that never change together gain nothing from atomic commits and pay for the shared history, CI queue and failure radius.
- **Large binary content.** Sparse checkout and partial clone postpone a download; they do not shrink it (Chapter 22).
- **No one to own the tooling.** Without affected-target CI and ownership rules, a monorepo becomes a slow repository with a red `main`.
- **Code you do not control** belongs in a package manager, a subtree or a submodule (Chapter 23).

Dangerous edge cases.

- Narrowing a cone deletes ignored files without a warning, and `clean -f` deletes untracked ones (section 24.5).
- `git commit -a` "will not record paths outside the sparse-checkout directories/patterns as deleted" (the manual). Intended, and a stray file out there is left alone too.
- A pattern file edited by hand can switch cone mode off: Git then behaves "as though `core.sparseCheckoutCone` was false", and the sparse index stops working.
- Changing the cone neither removes nor initialises submodules (`git help sparse-checkout`).
- The experimental label is real. Pin the Git version in automation that depends on these details (Chapter 14D).
- Merging repositories into one, or splitting one, creates new commit IDs (`git subtree`, Chapter 23; `git filter-repo`, Chapter 21B). Plan it as a migration with a freeze.

24.14 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git sparse-checkout list</code> , <code>git ls-files -t</code> , <code>git ls-files --sparse</code>	●	nothing	not needed	not needed
<code>git sparse-checkout set</code> , <code>add</code> , <code>init --cone</code>	●	Working tree, skip-worktree bits, pattern file, <code>config.worktree</code> ; deletes ignored files in directories that leave the cone	<code>git status --short --ignored</code>	<code>set</code> again or <code>disable</code> ; ignored files are gone
<code>git sparse-checkout reapply</code> , <code>disable</code>	●	Removes, or writes, tracked files	<code>git ls-files -t</code>	<code>set</code> or <code>add</code>
<code>git sparse-checkout set --sparse-index</code>	●	Rewrites the index in a form some tools cannot read	not needed	<code>--no-sparse-index</code>
<code>git sparse-checkout clean -f</code>	●	Deletes directories outside the cone with untracked files	<code>--dry-run</code> , <code>--verbose</code>	None
<code>git add --sparse</code>	●	Stages a path outside the cone	<code>git status</code>	<code>git restore --staged</code>
<code>git clone --filter=blob:none --sparse</code> , <code>git backfill</code> , <code>git maintenance run --task=prefetch</code>	●	Add objects; <code>refs/prefetch/*</code>	not needed	not needed
<code>scalar clone</code>	●	Also writes global configuration, installs a scheduler unless <code>--no-maintenance</code> , starts a monitor daemon	<code>git help scalar</code>	<code>scalar unregister</code> or <code>delete</code> ; <code>git fsmonitor--daemon stop</code>
<code>git describe --match</code> , <code>git diff --quiet</code> , <code>git diff --name-only A...B</code>	●	nothing	not needed	not needed

24.15 Version notes

Version note. Older behavior: `git sparse-checkout init --cone` followed by `set`; without `--cone`, pattern mode. Current behavior: cone mode is the default, `set` does everything, the manual calls `init` deprecated. Since: the command in Git 2.25, the default in 2.37 ([GitHub Blog](#); Phase 0 report, section 1). Recommended: `git sparse-checkout set <dirs>`.

Version note. Older behavior: Scalar was a separate application from Microsoft, preceded by VFS for Git, whose README now recommends Scalar ([VFS for Git](#)). Current behavior: `scalar` is a command in Git. Since: Git 2.38 ([The Story of Scalar](#)). Recommended: read `git help scalar` for the version you run; the list of settings changes between releases.

Version note. `git sparse-checkout clean` needs Git 2.52 ([release notes](#)) and `git backfill` Git 2.49. `git backfill`, the sparse index, and `git sparse-checkout` as a whole are labelled experimental in Git 2.56.

24.16 Practice

Labs 18.2 and 18.4 in `lab-manual/m18-transfer-scale.md`: a cone-mode sparse checkout with work that lands outside the cone, and the clone that `scalar clone` would build, made by hand and then cut off from its server. Replay any transcript with `labs/run ch24/<demo> (sparse-cone, sparse-behaviour, sparse-index, sparse-partial, affected-projects, project-tags)`; the sandbox stays in place. A drill: in `ch24/affected-projects/ci-shallow`, make the three-dot diff work with the smallest fetch you can find.

24.17 Interview questions

1. We are considering one repository for all services. Name three costs we would stop paying and three we would start paying. Which are Git's and which are tooling's?
2. "Google uses a monorepo, so Git can handle ours." What is wrong with that sentence?
3. Explain a cone-mode sparse checkout in terms of the index. What is on disk, in the index, in `HEAD`?
4. A search finds nothing in a colleague's clone although the code exists. Mechanism, and the command that searches everything?
5. What happens to untracked and to ignored files when a cone is narrowed?
6. What does the sparse index change, why only in cone mode, and what makes a command slower with it?
7. What does `scalar clone` set up, which parts live outside the repository, and how do you remove each?
8. Our CI computes affected projects with `git diff main..HEAD`. What is wrong, and why does the fix fail in a shallow clone?
9. A required check has been pending for two days on a documentation pull request. Cause, and the design that avoids it?
10. How do you tag and describe releases of twenty projects in one repository?
11. A team asks for a directory that only they can read. What do you answer?

24.18 Sources

Primary sources. The local manual of Git 2.55.0: `git help sparse-checkout` with its "Internals" sections, `git help scalar`, `git help clone`, `git help backfill`. Online: [git-sparse-checkout](#), [scalar](#), [git-backfill](#), [partial-clone](#), [git-describe](#), the [sparse-index design notes](#) and [scalar.c](#) at the 2.55.0 tag. Potvin and Levenberg, [Why Google Stores Billions of Lines of Code in a Single Repository](#) (2016; not Git). Engineering at Meta, [Sapling](#) (2022). GitHub Docs as linked in sections 24.8 and 24.9.

Secondary sources. GitHub Blog: [sparse-checkout](#) (2020), [sparse index](#) (2021), [The Story of Scalar](#) (2022), [Git's database internals V](#) (2022) on ways to split a repository. [Software Engineering at Google, chapter 16](#). The practitioner reports linked in section 24.2, each from the publisher's own environment. The Phase 0 report of this course, sections 1, 12 and 13.

Videos (optional; assessments from the Phase 0 report). Derrick Stolee, ["Git at Scale for Everyone"](#) (32 min, 2020): partial clone and a cone-mode demo; concepts current, the Scalar packaging is the one of 2020. Scott Chacon, ["So You Think You Know Git"](#) (FOSDEM 2024, 47 min): a survey, with about a minute of product pitch. Stolee's [Scalar talk](#) (2021) and ["Scaling Git at Microsoft"](#) (2017) describe superseded designs: history only.

Further reading. Chapter 26 for maintenance, the commit-graph and clone shapes; Chapter 25 for per-worktree configuration.

Chapter 25: Worktrees

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch25/`.

25.1 Why this matters

Three situations, each of which ends with someone asking you what to do:

1. "Production rejects nothing when the question is empty and the retriever falls over. I need a fix in ten minutes." You are in the middle of a rebase that has stopped at a conflict. `git switch main` is refused and `git stash` fails.
2. "Can you look at Asha's pull request now? She is blocked." Your working tree holds two hours of half-finished edits and a warm build cache.
3. "We want four coding agents working on four tasks in this repository at the same time. Do we need four clones?"

The usual answers are a second clone, or a stash and a branch switch. A second clone costs a full copy of the object database and has to be fetched and kept in sync separately. A stash and a switch disturb the one working tree you have, and during a conflicted rebase they are not available at all. Git's own answer is a **linked worktree**: one more working tree, with its own HEAD and its own index, attached to the repository you already have.

The project in this chapter is `rag-api`, a retrieval-augmented question-answering service. `main` is the release line and `feature/rerank` is your topic branch. Chapter 7 created one worktree from a tag to show a detached HEAD; this chapter covers the mechanism, the commands, and the ways it goes wrong.

25.2 What a linked worktree is

In one sentence. `git worktree add <path> <branch>` creates a second directory of checked-out files that belongs to the same repository: the same objects and the same refs, with a HEAD and an index of its own.

Analogy. One library, several desks. Every desk has its own open book and its own bookmark (the checked-out files, HEAD, the index). The shelves (objects) and the catalogue (refs) are shared, so a book that one desk adds to the shelves is on the shelves for all of them. The analogy breaks at one rule that libraries do not have: Git allows a given branch to be open at one desk only (section 25.4).

Precisely. The manual distinguishes the **main worktree**, the one created by `git init` or `git clone`, from **linked worktrees** created by `git worktree add`. A repository has one main worktree, unless it is bare, and any number of linked ones. The word "worktree" means the working tree together with its per-worktree metadata ([git-worktree](#)).

Inside .git. Two things are created. In the new directory there is a `.git` file, not a directory, that contains one line pointing back into the repository. In the repository there is a private directory `.git/worktrees/<id>/` that holds what belongs to this worktree alone: `HEAD`, `index`, `logs/HEAD`, `ORIG_HEAD` and the other root refs, plus two small files, `gitdir` (where the working tree is) and `commondir` (where the shared part is, relative to this directory).

See it. You are on `main`; `feature/rerank` exists and is not checked out anywhere:

```
$ git log --graph --oneline --decorate --all
* 323e8f1 (HEAD -> main, origin/main) Document deployment
* 0a09820 Retrieve eight passages and drop short ones
| * 110124a (feature/rerank) Document reranking
| * 6c9c90c Rerank retrieved passages
| * 6dd09ed Add keyword-overlap reranker
|/
* 168d50a (tag: v1.3.0) Add check script
* 6f8ad00 Add question endpoint and retriever
$ git worktree add ../rag-api-rerank feature/rerank
Preparing worktree (checking out 'feature/rerank')
HEAD is now at 110124a Document reranking
$ git worktree list
$LAB/ch25/worktree-basics/rag-api          323e8f1 [main]
$LAB/ch25/worktree-basics/rag-api-rerank 110124a [feature/rerank]
```

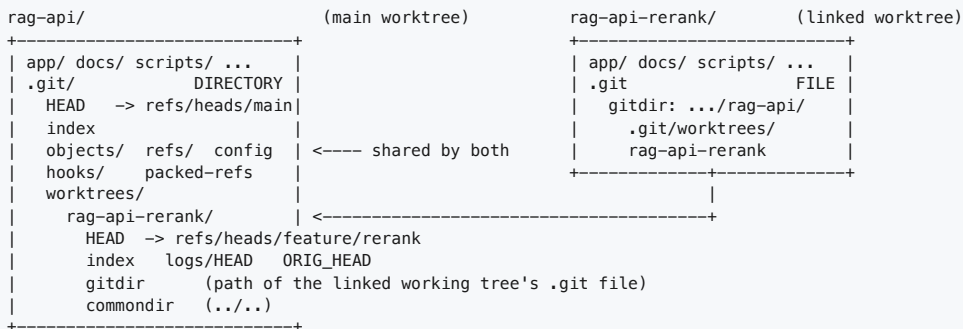
Preparing worktree (checking out 'feature/rerank') is an ordinary checkout into another directory. No objects were copied. `git worktree list` prints one line per worktree: path, the commit its HEAD resolves to, and the branch in brackets. The main worktree is always first.

```
# The linked worktree has a .git FILE, not a directory:
$ cat ../rag-api-rerank/.git
gitdir: $LAB/ch25/worktree-basics/rag-api/.git/worktrees/rag-api-rerank
```

```
# It points at a private directory inside the one repository:
$ ls .git/worktrees/rag-api-rerank
commondir
gitdir
HEAD
index
logs
ORIG_HEAD
refs
$ cat .git/worktrees/rag-api-rerank/HEAD
ref: refs/heads/feature/rerank
$ cat .git/worktrees/rag-api-rerank/gitdir
$LAB/ch25/worktree-basics/rag-api-rerank/.git
$ cat .git/worktrees/rag-api-rerank/commondir
../..
```

Read the three small files as a pair of pointers. The `.git` file in the working tree says where this worktree's private Git directory is. `gitdir` inside that directory points back at the working tree, which is how `git worktree list` and `git worktree prune` find out whether the working tree still exists. `commondir` contains `../..`: two levels up from `.git/worktrees/rag-api-rerank` is `.git`, the shared part.

Picture.



In production. A model-evaluation job runs for forty minutes against the files of a release tag. With one working tree nobody can touch the repository until it finishes. With `git worktree add --detach ../rag-api-v1.3.0 v1.3.0` the job gets a directory of its own, and you keep working.

25.3 What is shared and what is not

Inside a linked worktree Git works with two directories. `$GIT_DIR` is the private one; `$GIT_COMMON_DIR` is the repository's `.git`. Every path under "the Git directory" belongs to one of the two, and `git rev-parse --git-path` tells you which:

```
$ cd ../rag-api-rerank
$ git rev-parse --git-dir
$LAB/ch25/worktree-basics/rag-api/.git/worktrees/rag-api-rerank
$ git rev-parse --git-common-dir
$LAB/ch25/worktree-basics/rag-api/.git
# Per worktree:
$ git rev-parse --git-path HEAD --git-path index --git-path logs/HEAD --git-path ORIG_HEAD
$LAB/ch25/worktree-basics/rag-api/.git/worktrees/rag-api-rerank/HEAD
$LAB/ch25/worktree-basics/rag-api/.git/worktrees/rag-api-rerank/index
$LAB/ch25/worktree-basics/rag-api/.git/worktrees/rag-api-rerank/logs/HEAD
$LAB/ch25/worktree-basics/rag-api/.git/worktrees/rag-api-rerank/ORIG_HEAD
# Shared:
$ git rev-parse --git-path objects --git-path refs/heads/main --git-path config --git-path hooks
$LAB/ch25/worktree-basics/rag-api/.git/objects
$LAB/ch25/worktree-basics/rag-api/.git/refs/heads/main
$LAB/ch25/worktree-basics/rag-api/.git/config
$LAB/ch25/worktree-basics/rag-api/.git/hooks
```

Per worktree	Shared by all worktrees
The working tree files	The object database (<code>objects/</code>)
<code>HEAD</code> , and therefore "the current branch"	All refs under <code>refs/</code> : branches, tags, remote-tracking branches, <code>refs/stash</code>
The index (staging area)	<code>packed-refs</code>
<code>logs/HEAD</code> (the <code>HEAD</code> reflog)	Reflogs of branches
<code>ORIG_HEAD</code> , <code>MERGE_HEAD</code> , <code>CHERRY_PICK_HEAD</code> , <code>FETCH_HEAD</code> and the other root refs	<code>config</code> (unless <code>extensions.worktreeConfig</code> is on, section 25.7)
The state of an operation in progress: <code>rebase-merge/</code> , <code>MERGE_MSG</code> , the sequencer	<code>hooks/</code> , <code>info/exclude</code>

Per worktree	Shared by all worktrees
refs/bisect/*, refs/worktree/*, refs/rewritten/*	Remotes and their URLs

The manual's rule is short: refs that start with `refs/` are shared, with the three exceptions in the last row of the left column, and root refs such as `HEAD` are per worktree. The manual also warns not to guess: "do not make any assumption about whether a path belongs to `$GIT_DIR` or `$GIT_COMMON_DIR`"; ask `git rev-parse --git-path`.

Two consequences can be seen at once. First, a commit made in one worktree exists in the other the moment it is created, because the commit object and the branch ref are shared:

```
$ git commit -am "Document tie-breaking"
[feature/rerank ab93222] Document tie-breaking
1 file changed, 2 insertions(+)
$ cd ../rag-api
# No fetch, no push: the branch ref and the new objects are already here.
$ git log --oneline -2 feature/rerank
ab93222 Document tie-breaking
110124a Document reranking
$ git branch -v
+ feature/rerank ab93222 Document tie-breaking
* main          323e8f1 Document deployment
```

In `git branch -v` the `*` marks the branch checked out here and the `+` marks a branch checked out in another worktree. Second, staging is private. A file staged in the main worktree does not appear in the status of the linked one:

```
$ git add docs/deploy.md
$ git status --short --branch
## main...origin/main
M docs/deploy.md
$ git -C ../rag-api-rerank status --short --branch
## feature/rerank
```

Because root refs are per worktree, the name `HEAD` means something different in each directory. To name another worktree's `HEAD`, prefix it: `main-worktree/HEAD` for the main one and `worktrees/<id>/HEAD` for a linked one, where `<id>` is the name of the directory under `.git/worktrees/`:

```
# From any worktree you can name the HEAD of another one:
$ git log -1 --format="%h %s" main-worktree/HEAD
323e8f1 Document deployment
$ git log -1 --format="%h %s" worktrees/rag-api-rerank/HEAD
ab93222 Document tie-breaking
$ git -C ../rag-api-rerank log -1 --format="%h %s" main-worktree/HEAD
323e8f1 Document deployment
```

State table for `git worktree add ../rag-api-rerank feature/rerank`, run in the main worktree:

Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
unchanged here; a new working tree is written at the given path	unchanged here; a new index is written in <code>.git/worktrees/<id>/</code>	unchanged here; the new worktree gets its own <code>HEAD</code>	unchanged	<code>.git/worktrees/<id>/</code> created; with <code>-b</code> , a new branch ref; no objects copied	unchanged	unchanged

25.4 One branch, one worktree

In one sentence. A branch can be checked out in at most one worktree, and Git refuses every command that would break this.

See it. `main` is checked out in the main worktree and `feature/rerank` in the linked one:

```
$ git worktree add ../rag-api-second main
Preparing worktree (checking out 'main')
fatal: 'main' is already used by worktree at '$LAB/ch25/worktree-one-branch/rag-api'
[exit status: 128]
$ cd ../rag-api-rerank
$ git switch main
fatal: 'main' is already used by worktree at '$LAB/ch25/worktree-one-branch/rag-api'
[exit status: 128]
$ cd ../rag-api
```

The same rule protects a checked-out branch from being deleted or moved from elsewhere:

```
$ git branch -D feature/rerank
error: cannot delete branch 'feature/rerank' used by worktree at '$LAB/ch25/worktree-one-branch/rag-api-rerank'
```

```
[exit status: 1]
$ git branch -f feature/rerank main
fatal: cannot force update the branch 'feature/rerank' used by worktree at '$LAB/ch25/worktree-one-branch/rag-api-rerank'
[exit status: 128]
```

Why. A branch is one ref. A worktree that has the branch checked out assumes that its index and its files correspond to the commit the ref names. If two worktrees shared the ref, a commit in one would move the ref under the other, whose index and files would stay where they were. `--force` lets you build exactly that situation, so you can see it once in a sandbox:

```
# What the rule prevents. Force a second checkout of main:
$ git worktree add --force ../rag-api-dup main
Preparing worktree (checking out 'main')
HEAD is now at 323e8f1 Document deployment
$ git commit -am "Point the README at the deployment notes"
[main 7e00138] Point the README at the deployment notes
 1 file changed, 2 insertions(+)
# Nobody touched the other worktree, and yet:
$ git -C ../rag-api-dup status
On branch main
Your branch is ahead of 'origin/main' by 1 commit.
  (use "git push" to publish your local commits)

Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
```

Nobody touched `rag-api-dup`, and its status reports a staged modification. The diff shows which one:

```
$ git -C ../rag-api-dup diff --cached
diff --git a/README.md b/README.md
index e038a61..a2cce02 100644
--- a/README.md
+++ b/README.md
@@ -1,5 +1,3 @@
 # rag-api

 Retrieval-augmented question answering.
 -
 -See docs/deploy.md for releases.
```

Observed behavior : A worktree that nobody edited shows "Changes to be committed"; the staged diff removes the lines that the latest commit added.

Git state : Both worktrees have HEAD -> refs/heads/main. The ref was advanced by a commit made in the other worktree. This worktree's index and files still match the old commit.

Mechanism : `git status` compares HEAD with the index. HEAD now resolves to the new commit; the index describes the old tree. The difference, read from HEAD to index, is the inverse of the new commit.

Root cause : One ref, two indexes. Only the worktree that makes a commit updates its own index.

Why Git does this : The index is per worktree by design; Git has no mechanism that updates the index and files of another worktree when a ref moves.

Correct fix : In the stale worktree, if it has no work of its own: `git reset --hard`. If it has: commit there would revert the other commit, so stash first, reset, pop.

Prevention : Do not use `--force` with `git worktree add`. Use `--detach` or a new branch.

```
# The branch moved under that worktree; its index and files are one commit behind.
# It had no work of its own, so bring it up to date:
$ git -C ../rag-api-dup reset --hard
HEAD is now at 7e00138 Point the README at the deployment notes
$ git -C ../rag-api-dup status --short --branch
## main...origin/main [ahead 1]
```

`git commit` in the stale worktree before the reset would have recorded the old tree on top of the new commit: a silent revert of a colleague's, or your own, work. That is the failure the rule exists to prevent.

When you need the files of a branch that is checked out elsewhere, and you do not intend to commit to it, detach:

```
$ git worktree add --detach ../rag-api-readonly main
Preparing worktree (detached HEAD 323e8f1)
HEAD is now at 323e8f1 Document deployment
$ git worktree list
$LAB/ch25/worktree-one-branch/rag-api          323e8f1 [main]
$LAB/ch25/worktree-one-branch/rag-api-readonly 323e8f1 (detached HEAD)
$LAB/ch25/worktree-one-branch/rag-api-rerank   110124a [feature/rerank]
```

A detached worktree holds a commit, not a branch, so any number of them can sit on the same commit.

Two things count as "checked out" that are easy to miss. A branch that is in the middle of a rebase or a bisect in some worktree is in use by that worktree, although `git worktree list` shows that worktree as (detached HEAD) (section 25.5 shows the refusal). And `git rebase --`

`update-refs` skips branches that are checked out in a worktree: the manual states that such branches "are not updated in this way" (Chapter 9).

25.5 Use cases

A hotfix while a rebase is stopped

You started `git rebase main` on `feature/rerank` and it stopped at a conflict. Then the incident arrives:

```
$ git status --short --branch
## HEAD (no branch)
UU app/retriever.py
$ git branch
* (no branch, rebasing feature/rerank)
  feature/rerank
  main
$ git switch main
fatal: cannot switch branch while rebasing
Consider "git rebase --quit" or "git worktree add".
[exit status: 128]
$ git stash
error: could not write index
app/retriever.py: needs merge
[exit status: 1]
```

Both escape routes are closed. `git switch` refuses during a rebase, and Git's own message names the two alternatives: give up the rebase state, or add a worktree. `git stash` cannot write a tree from an index that has unmerged entries. Aborting the rebase would work and would throw away the conflict resolutions you have already done. The worktree costs nothing:

```
$ git worktree add -b hotfix/empty-question ../rag-api-hotfix main
Preparing worktree (new branch 'hotfix/empty-question')
HEAD is now at 323e8f1 Document deployment
$ git worktree list
$LAB/ch25/worktree-hotfix/rag-api          337498d (detached HEAD)
$LAB/ch25/worktree-hotfix/rag-api-hotfix 323e8f1 [hotfix/empty-question]
```

`-b hotfix/empty-question` creates the branch at `main` and checks it out in the new directory. `git worktree list` shows the main worktree as `(detached HEAD)`: that is the rebase, which works on a detached HEAD until it finishes (Chapter 9).

```
$ cd ../rag-api-hotfix
$ sh scripts/check.sh
checks failed: app/api.py accepts an empty question
$ git diff
diff --git a/app/api.py b/app/api.py
index 1e6749d..452ddaf 100644
--- a/app/api.py
+++ b/app/api.py
@@ -2,5 +2,7 @@ from app.retriever import retrieve

def ask(index, question):
+   if not question or not question.strip():
+       raise ValueError("question must not be empty")
    passages = retrieve(index, question)
    return {"question": question, "passages": passages}
$ sh scripts/check.sh
checks passed
$ git commit -am "Reject empty questions"
[hotfix/empty-question dd34256] Reject empty questions
1 file changed, 2 insertions(+)
$ git push -u origin hotfix/empty-question
To $LAB/ch25/worktree-hotfix/remotes/rag-api.git
 * [new branch]      hotfix/empty-question -> hotfix/empty-question
branch 'hotfix/empty-question' set up to track 'origin/hotfix/empty-question'.
```

The fix was written, checked, committed and pushed from the second directory. Then the directory is removed and you are back where you were:

```
$ cd ../rag-api
$ git worktree remove ../rag-api-hotfix
$ git worktree list
$LAB/ch25/worktree-hotfix/rag-api 337498d (detached HEAD)
# The rebase is exactly where you left it, and the hotfix branch is in the shared refs:
$ git status --short
UU app/retriever.py
$ git branch -vv
* (no branch, rebasing feature/rerank) 337498d Add keyword-overlap reranker
  feature/rerank                      110124a Document reranking
```

hotfix/empty-question	dd34256 [origin/hotfix/empty-question] Reject empty questions
main	323e8f1 [origin/main] Document deployment

The rebase state lives in `.git/rebase-merge/`, which is per worktree, so nothing that happened next door could disturb it. The hotfix branch is an ordinary shared ref: it is still there after its worktree is gone, with its upstream configured. Lab 14.1 walks through this case by hand, including the cleanup mistake that people make afterwards.

Reviewing a pull request

You have uncommitted work on `feature/rerank`. A teammate's branch arrives on the remote:

```
$ git status --short --branch
## feature/rerank
M app/rerank.py
$ git fetch origin
From $LAB/ch25/worktree-review/remotes/rag-api
* [new branch]      feature/citations -> origin/feature/citations
```

```
$ git worktree add --detach ../rag-api-review origin/feature/citations
Preparing worktree (detached HEAD 00daaec)
HEAD is now at 00daaec Number the passages so answers can cite them
$ cd ../rag-api-review
$ git log --oneline main..HEAD
00daaec Number the passages so answers can cite them
$ git diff --stat main..HEAD
 app/api.py | 3 ++-
1 file changed, 2 insertions(+), 1 deletion(-)
$ cd ../rag-api
$ git worktree remove ../rag-api-review
$ git status --short --branch
## feature/rerank
M app/rerank.py
```

A detached worktree at the remote-tracking branch gives you the files to read, run and test, without creating a local branch that you would have to delete later, and your own modification is untouched throughout. If you intend to push commits to the branch, name it instead. When no local branch of that name exists and exactly one remote has one, `git worktree add` creates a tracking branch, the same convenience that `git switch` offers:

```
# If you intend to push commits to the branch, name it. No local branch feature/citations
# exists; exactly one remote has one, so Git creates a tracking branch:
$ git worktree add ../rag-api-citations feature/citations
Preparing worktree (new branch 'feature/citations')
branch 'feature/citations' set up to track 'origin/feature/citations'.
HEAD is now at 00daaec Number the passages so answers can cite them
$ git -C ../rag-api-citations status --short --branch
## feature/citations...origin/feature/citations
$ git worktree list
$LAB/ch25/worktree-review/rag-api          110124a [feature/rerank]
$LAB/ch25/worktree-review/rag-api-citations 00daaec [feature/citations]
```

GitHub, not Git. A pull request from a fork has no branch in your remote. GitHub publishes its head as the ref `refs/pull/<number>/head`, which you can fetch with `git fetch origin pull/<number>/head` and then check out from `FETCH_HEAD` in a detached worktree ([checking out pull requests locally](#)). The Phase 0 report records that GitHub CLI releases after the 2.88.1 installed here added `gh pr checkout --worktree`; it was not run for this book.

Parallel work, by people or by AI agents

Each task gets a branch and a directory:

```
git worktree add -b agent/retry-policy ../wt/retry-policy main
git worktree add -b agent/citation-tests ../wt/citation-tests main
git worktree add -b agent/latency-budget ../wt/latency-budget main
git worktree list
```

Compared with separate clones, the worktrees share one object database, so there is no second copy of history and nothing to fetch between them: a branch committed in one directory can be diffed, merged or cherry-picked from any other immediately. The one-branch rule gives each agent exclusive ownership of its branch for free. What you have to supply yourself:

- **Dependencies and build output.** Ignored files are not part of any commit, so a new worktree has no virtual environment, no `node_modules`, no `.env` (section 25.7). Each worktree needs its own setup step.
- **Ports, caches and local databases** are outside Git's view. Two worktrees that start the same service collide exactly as two clones would.

- **Shared state inside `.git`.** Stashes, configuration and hooks are common to all worktrees (section 25.7). An agent that runs `git stash pop` may pop another agent's stash. Tell agents to commit to their branch and never to stash.
- **Integration is still a merge.** Parallel branches that touch the same lines conflict when they meet, however they were produced.
- **Cleanup.** `git worktree remove` for each directory and `git branch -d` for each merged branch, or the list grows without bound.

25.6 The life of a worktree: add, list, remove, lock, move, prune, repair

`git worktree add` has three common forms:

```
# Path only: a new branch named after the last path component.
$ git worktree add ../spike-cache
Preparing worktree (new branch 'spike-cache')
HEAD is now at 323e8f1 Document deployment
# An explicit new branch and a starting point.
$ git worktree add -b fix/top-k ../rag-api-fix v1.3.0
Preparing worktree (new branch 'fix/top-k')
HEAD is now at 168d50a Add check script
# No branch at all.
$ git worktree add --detach ../rag-api-v1.3.0 v1.3.0
Preparing worktree (detached HEAD 168d50a)
HEAD is now at 168d50a Add check script
```

With only a path, Git creates a branch named after the last path component, here `spike-cache`, starting at HEAD. That default surprises people who meant "give me a copy of what I have": they get a new branch they did not ask for. `--orphan` starts a worktree on a new branch with no history and an empty index, and `--no-checkout` creates the worktree without writing files, for example to set up a sparse checkout first.

```
$ git worktree list
$LAB/ch25/worktree-lifecycle/rag-api      323e8f1 [main]
$LAB/ch25/worktree-lifecycle/rag-api-fix  168d50a [fix/top-k]
$LAB/ch25/worktree-lifecycle/rag-api-v1.3.0 168d50a (detached HEAD)
$LAB/ch25/worktree-lifecycle/spike-cache  323e8f1 [spike-cache]
$ git worktree list --porcelain | head -8
worktree $LAB/ch25/worktree-lifecycle/rag-api
HEAD 323e8f143a79eac653c3f09a2981e88ef090051d
branch refs/heads/main

worktree $LAB/ch25/worktree-lifecycle/rag-api-fix
HEAD 168d50a8eb883a9cddffbd6a29347766e81da70e
branch refs/heads/fix/top-k
```

Use `--porcelain` (with `-z` if paths may contain newlines) in scripts; the manual promises that this format stays stable.

🟡 `git worktree remove` deletes the working tree directory and its entry under `.git/worktrees/`. It refuses when the tree has modified or untracked files. 🔴 `--force` overrides the refusal and deletes those files; nothing in Git can bring back files that were never added.

```
$ git worktree remove ../spike-cache
fatal: '../spike-cache' contains modified or untracked files, use --force to delete it
[exit status: 128]
$ git worktree remove --force ../spike-cache
# The worktree is gone. Its branch is not:
$ git branch --list "spike*"
spike-cache
$ git branch -d spike-cache
Deleted branch spike-cache (was 323e8f1).
```

Removing a worktree does not delete its branch. The commits are safe in the shared repository; the branch stays until you delete it.

`git worktree lock` protects a worktree from `prune`, `move` and `remove`. Its purpose in the manual is a working tree on a removable disk or a network share that is not always mounted; it serves equally to mark a directory where a long job is running:

```
$ git worktree lock --reason "benchmark of 1.3.0 running until Friday" ../rag-api-v1.3.0
$ git worktree list --verbose
$LAB/ch25/worktree-lifecycle/rag-api      323e8f1 [main]
$LAB/ch25/worktree-lifecycle/rag-api-fix  168d50a [fix/top-k]
$LAB/ch25/worktree-lifecycle/rag-api-v1.3.0 168d50a (detached HEAD)
      locked: benchmark of 1.3.0 running until Friday
$ git worktree remove ../rag-api-v1.3.0
fatal: cannot remove a locked working tree, lock reason: benchmark of 1.3.0 running until Friday
use 'remove -f -f' to override or unlock first
[exit status: 128]
$ cat .git/worktrees/rag-api-v1.3.0/locked
benchmark of 1.3.0 running until Friday
$ git worktree unlock ../rag-api-v1.3.0
```

```
$ git worktree move ../rag-api-v1.3.0 ../bench-1.3.0
$ git worktree list
$LAB/ch25/worktree-lifecycle/rag-api      323e8f1 [main]
```



```
$LAB/ch25/worktree-lifecycle/bench-1.3.0 168d50a (detached HEAD)
$LAB/ch25/worktree-lifecycle/rag-api-fix 168d50a [fix/top-k]
```

`git worktree move` relocates a linked worktree and updates both pointers. It cannot move the main worktree, nor a worktree that contains submodules.

Deleted by hand. If someone removes the directory with `rm -rf`, the entry under `.git/worktrees/` remains, and with it the claim on the branch:

```
$ rm -rf ../rag-api-fix
$ git worktree list
$LAB/ch25/worktree-lifecycle/rag-api 323e8f1 [main]
$LAB/ch25/worktree-lifecycle/bench-1.3.0 168d50a (detached HEAD)
$LAB/ch25/worktree-lifecycle/rag-api-fix 168d50a [fix/top-k] prunable
$ git branch -d fix/top-k
error: cannot delete branch 'fix/top-k' used by worktree at '$LAB/ch25/worktree-lifecycle/rag-api-fix'
[exit status: 1]
$ git worktree prune --dry-run --verbose
Removing worktrees/rag-api-fix: gitdir file points to non-existent location
$ git worktree prune --verbose
Removing worktrees/rag-api-fix: gitdir file points to non-existent location
$ git branch -d fix/top-k
Deleted branch fix/top-k (was 168d50a).
```

prunable in the listing and "gitdir file points to non-existent location" say the same thing: the admin entry points at a working tree that is gone.

 `git worktree prune` removes such entries; `--dry-run --verbose` shows first what it would remove. Git also prunes stale entries by itself:

`git gc` calls `git worktree prune --expire 3.months.ago` (the default of `gc.worktreePruneExpire`), and the automatic maintenance of Git 2.54 and later includes a `worktree-prune` task (Chapter 26).

Moved by hand. If the directory was moved with `mv`, one of the two pointers is stale. `git worktree repair` rewrites them:

```
$ mv ../bench-1.3.0 ../bench-old-release
$ git worktree list
$LAB/ch25/worktree-lifecycle/rag-api 323e8f1 [main]
$LAB/ch25/worktree-lifecycle/bench-1.3.0 168d50a (detached HEAD) prunable
$ git worktree repair ../bench-old-release
repair: gitdir incorrect: $LAB/ch25/worktree-lifecycle/rag-api/.git/worktrees/rag-api-v1.3.0/gitdir
$ git worktree list
$LAB/ch25/worktree-lifecycle/rag-api 323e8f1 [main]
$LAB/ch25/worktree-lifecycle/bench-old-release 168d50a (detached HEAD)
```

The name in the repair message, `worktrees/rag-api-v1.3.0`, is the worktree's ID: the directory name it was created with. The ID does not change when the working tree moves, which matters when you write `worktrees/<id>/HEAD`.

Moving or renaming the main worktree breaks the other direction: every linked worktree's `.git` file now points at a path that no longer exists.

```
# Now the main worktree itself is renamed. The linked one loses its way home:
$ cd .. && mv rag-api rag-api-main && cd rag-api-main
$ git -C ../bench-old-release status --short --branch
fatal: not a git repository: (null)
[exit status: 128]
$ git worktree repair
repair: .git file broken: $LAB/ch25/worktree-lifecycle/bench-old-release
$ git -C ../bench-old-release status --short --branch
## HEAD (no branch)
```

Run `git worktree repair` in the main worktree and it fixes the `.git` file of every linked worktree it knows. If both ends have moved, run it in the main worktree with the new paths of the linked ones as arguments.

25.7 Pitfalls

Ignored files do not come along. A worktree is a checkout of a commit. What is not in the commit is not in the new directory:

```
# The virtual environment is ignored, so it is not in any commit:
$ git status --short --ignored
!! .venv/
$ ls -A
.git
.gitignore
.venv
app
docs
README.md
scripts
$ ls -A ../rag-api-rerank
.git
.gitignore
```

```
app
docs
README.md
scripts
```

Stashes are shared. `refs/stash` is under `refs/`, so there is one stash list for the whole repository:

```
$ cd ../rag-api-rerank
$ git stash push -m "rerank: cache idea"
Saved working directory and index state On feature/rerank: rerank: cache idea
$ cd ../rag-api
# refs/stash lives under refs/, and refs/ is shared:
$ git stash list
stash@{0}: On feature/rerank: rerank: cache idea
$ git rev-parse --git-path refs/stash
.git/refs/stash
```

A `git stash pop` in the main worktree would apply an entry that was made on another branch in another directory. In a repository with several worktrees, prefer a commit on the branch to a stash.

Configuration and hooks are shared. `git config set` without a scope writes `.git/config`, which every worktree reads:

```
$ git -C ../rag-api-rerank config set user.email rerank-bot@example.com
$ git config get user.email
rerank-bot@example.com
$ git config list --show-origin --local | grep user.email
file:.git/config          user.email=rerank-bot@example.com
```

Per-worktree configuration needs an extension that has to be switched on first. With it, `git config set --worktree` writes `config.worktree` in the private directory:

```
$ git -C ../rag-api-rerank config set --worktree user.email rerank-bot@example.com
fatal: --worktree cannot be used with multiple working trees unless the config
extension worktreeConfig is enabled. Please read "CONFIGURATION FILE"
section in "git help worktree" for details
[exit status: 128]
$ git config set extensions.worktreeConfig true
$ git -C ../rag-api-rerank config set --worktree user.email rerank-bot@example.com
$ git -C ../rag-api-rerank config get user.email
rerank-bot@example.com
$ git config get user.email
you@example.com
$ git -C ../rag-api-rerank rev-parse --git-path config.worktree
$LAB/ch25/worktree-shared-pitfalls/rag-api/.git/worktrees/rag-api-rerank/config.worktree
```

The manual notes that older Git versions refuse to work in a repository that has `extensions.worktreeConfig` set, and that `core.bare` and `core.worktree`, if present, must then be moved to the main worktree's `config.worktree`. Hooks live in the shared `hooks/` directory: a `pre-commit` hook runs for commits in every worktree, with the worktree's directory as its working directory.

Garbage collection respects other worktrees. A commit that is reachable only from the detached HEAD of another worktree is not unreachable. `git gc` takes the HEAD of every worktree into account:

```
$ cd ../rag-api-try
$ git commit -m "Try twelve passages"
[detached HEAD 69669bf] Try twelve passages
1 file changed, 1 insertion(+)
 create mode 100644 app/settings.py
$ cd ../rag-api
# That commit is on no branch. Only the HEAD of the other worktree holds it.
$ git branch --contains worktrees/rag-api-try/HEAD
$ git gc --quiet --prune=now
$ git cat-file -t worktrees/rag-api-try/HEAD
commit
```

That protection lasts as long as the worktree's admin entry does. After `git worktree remove` or `prune`, a commit that was made on a detached HEAD there is held by nothing, because the worktree's HEAD reflog went with the entry. Put work on a branch before you remove a detached worktree.

From the manual's BUGS section. "Multiple checkout in general is still experimental, and the support for submodules is incomplete. It is NOT recommended to make multiple checkouts of a superproject." In practice: a linked worktree of a repository with submodules starts with empty submodule directories and needs its own `git submodule update --init`; once a submodule is checked out in it, `git worktree move` refuses ("working trees containing submodules cannot be moved or removed") and `git worktree remove` refuses with the same message until you pass `--force`, as the manual says. The empty directory and the two refusals were run for this chapter; no transcript is printed (Chapter 23).

Other limits worth knowing. `git refs migrate`, which converts a repository between the files and refable ref formats, cannot migrate a repository that has worktrees (Chapter 3). Editors, language servers and file watchers treat each worktree as a separate project and index it

separately. Disk use is the checked-out files per worktree, not the history.

25.8 What can go wrong

Symptom	Diagnosis	Fix	Prevention
fatal: '<branch>' is already used by worktree at '<path>'	git worktree list: the branch is checked out, or being rebased or bisected, there	Work in that worktree; or <code>--detach</code> ; or a new branch with <code>-b</code>	One branch per task
error: cannot delete branch '' used by worktree at '<path>', and the path no longer exists	git worktree list shows <code>prunable</code>	git worktree prune, then delete the branch	git worktree remove, never <code>rm -rf</code>
fatal: not a git repository inside a linked worktree	The main worktree was moved or renamed; <code>cat .git</code> shows the old path	git worktree repair in the main worktree	Decide the directory layout before adding worktrees; or relative links (section 25.11)
A listing shows <code>prunable</code> for a worktree that exists	It was moved with <code>mv</code>	git worktree repair <new path>	git worktree move
Staged changes in a worktree nobody edited	Two worktrees on one branch (<code>--force</code>); section 25.4	git reset --hard in the stale one, after saving any real work	Never force a second checkout of a branch
"It works in the other directory": imports fail, tests cannot find the environment	git status --ignored in the main worktree lists what the new one lacks	Create the environment in the new worktree	A setup script per worktree
A stash from another task appears in <code>git stash list</code>	refs/stash is shared	git stash show -p stash@{n} before applying; apply in the right worktree	Commit work in progress to its branch
A setting made "for this worktree" changed all of them	git config list --show-origin shows <code>.git/config</code>	extensions.worktreeConfig and <code>--worktree</code>	Know the scope before writing
git worktree remove refuses	Modified or untracked files, a lock, or submodules	Commit or clean; <code>unlock</code> ; <code>--force</code> only after looking	Keep worktrees clean; lock with a reason
Commits made in a detached worktree cannot be found after it was removed	The worktree's HEAD reflog was deleted with it	git fsck --lost-found in the main worktree, before the next pruning (Chapter 13)	Create a branch before removing

25.9 When not to use it, and dangerous edge cases

Do not reach for a worktree:

- **For a two-minute look at another branch** when your working tree is clean. `git switch` is enough.
- **To get isolation.** Worktrees share refs, stashes, configuration and hooks. For an untrusted experiment, a destructive history rewrite you want to rehearse, or a different set of remotes and credentials, use a separate clone.
- **For a superproject with submodules**, unless you have tested your exact workflow; the manual advises against it.
- **As a substitute for branches.** A worktree is where a branch is checked out. Ten worktrees on ten stale branches are ten stale branches.

Edge cases:

-  `git worktree remove --force` deletes untracked and modified files in that worktree. Preview with `git -C <path> status --short --ignored`.
-  `git worktree add --force <path> <branch>` for a branch that is checked out elsewhere creates the two-index situation of section 25.4.
- **A linked worktree placed inside the main working tree** shows up there as an untracked directory and is visible to `git clean -d`. Put worktrees beside the repository, not inside it.
- **Deleting the main worktree's directory deletes the repository**, and with it the history of every linked worktree. Linked worktrees are not backups.
- **A bare repository can have linked worktrees and no main one.** Some teams keep a bare clone and add every branch as a worktree next to it. That layout works; note that a bare clone made with `git clone --bare` has no remote-tracking branches configured, which changes how `fetch` behaves (Chapter 12).

25.10 Command safety

Command	Label	What it changes	Preview	Recovery
git worktree list, git rev-parse --git-path, git rev-parse --git-common-dir	 SAFE	Nothing	not needed	not needed

Command	Label	What it changes	Preview	Recovery
<code>git worktree add <path> <branch>, --detach, -b</code>	● SAFE	A new directory, an entry in <code>.git/worktrees/</code> , with <code>-b</code> a new branch	<code>git worktree list; git branch --list <name></code>	<code>git worktree remove; git branch -d</code>
<code>git worktree add --force</code> (branch already checked out)	● DANGEROUS	Two indexes on one ref	none; do not use	<code>git reset --hard</code> in the stale worktree after saving its work
<code>git worktree lock, unlock</code>	● SAFE	A <code>locked</code> file in the entry	<code>git worktree list --verbose</code>	the opposite command
<code>git worktree move</code>	● CAUTION	The directory and both pointers	<code>git worktree list</code>	move it back
<code>git worktree remove</code>	● CAUTION	Deletes a clean working tree and its entry, including its HEAD reflog	<code>git -C <path> status --short; git -C <path> log --oneline -3</code>	<code>git worktree add</code> again; detached commits via <code>git fsck --lost-found</code>
<code>git worktree remove --force</code>	● DANGEROUS	As above, and deletes uncommitted and untracked files	<code>git -C <path> status --short --ignored</code>	none for files that were never committed
<code>git worktree prune</code>	● CAUTION	Deletes entries whose working tree is missing, including their HEAD reflogs	<code>git worktree prune --dry-run --verbose</code>	<code>git worktree add</code> again; a moved tree: <code>repair</code> before pruning
<code>git worktree repair</code>	● SAFE	Rewrites the two pointer files	<code>git worktree list</code>	run it again with the right paths

25.11 Version notes

Version note. Older behavior: worktrees are linked with absolute paths, so moving the repository or mounting it at another path (a container, a network share) breaks the links until `git worktree repair`. Current behavior: the same by default; `git worktree add --relative-paths`, or `worktree.useRelativePaths=true`, writes relative links. Since: Git 2.48 ([release notes](#)). Recommended: leave the default unless the repository and its worktrees are moved together; relative links set a repository extension that older Git versions refuse to read.

- `git worktree` has been part of Git since 2.5. It is not labelled experimental, and its manual still carries the BUGS note quoted in section 25.7 ([git-worktree](#)).
- Since Git 2.54 the default automatic maintenance removes worktree entries that cannot be located any more; before that, `git gc --auto` did so through `gc.worktreePruneExpire` ([maintenance configuration](#)).
- Git 2.56 adds an `includeIf` condition on the worktree location, so that a configuration file can be included for some worktrees only (not run here; [release notes](#)).

Outdated advice. "Clone the repository a second time when you need two branches at once." It still works, and it is the right choice when you need isolation. For parallel work on one project it costs a second object database and a second set of remotes to keep current, which is what worktrees were added to avoid.

25.12 Practice

- Lab 14.1 in the Module 14 worktree lab: a hotfix in a second worktree while a rebase is stopped in the first, a cleanup that goes wrong, and the repair.
- Replay any transcript with `labs/run ch25/<demo>`, for example `labs/run ch25/worktree-lifecycle`.
- Two drills in the sandbox of `ch25/worktree-basics`: predict, then check with `git rev-parse --git-path`, whether `MERGE_HEAD`, `refs/tags/v1.3.0`, `info/exclude` and `refs/bisect/bad` are per worktree or shared; and start a `git bisect` in the linked worktree, then show from the main worktree that its HEAD did not move.

25.13 Interview questions

- What exactly does `git worktree add` create on disk, in the new directory and in the repository?
- Which parts of a repository are shared between worktrees and which are per worktree? How do you find out for a path you are unsure about?
- Why does Git refuse to check out one branch in two worktrees? Describe what would go wrong, in terms of refs and indexes.
- `git branch -d` says a branch is used by a worktree at a path that does not exist. What happened, and what are the two commands that diagnose and fix it?
- A rebase has stopped at a conflict and an urgent fix is needed on `main`. Compare three options: abort the rebase, a second clone, a worktree.
- A teammate ran `git stash pop` and got changes they had never seen. The repository has four worktrees. Explain.

7. Is a commit that exists only on the detached HEAD of a linked worktree safe from `git gc`? When does that stop being true?
8. Your team wants several AI coding agents to work in one repository at once. What do worktrees give you, and what do they not give you?
9. After moving a project directory to another disk, every linked worktree reports "not a git repository". What broke, how do you repair it, and how could it have been avoided?
10. When is a separate clone the better tool?

25.14 Sources

Primary sources

- [git-worktree](#): commands, the REFS, CONFIGURATION FILE, DETAILS and BUGS sections. The local copy (`git help -m worktree`) is the Git 2.55.0 text that the transcripts were checked against. Source of the current manual: [git-worktree.adoc](#).
- `gc.worktreePruneExpire`, `extensions.worktreeConfig`, `worktree.guessRemote`, `worktree.useRelativePaths`, `checkout.defaultRemote`: `git help -m config`.
- [git-rebase](#) for `--update-refs` and branches checked out in a worktree; [git-refs](#) for the migration limitation.
- Release notes: [2.48.0](#), [2.56.0](#).

Secondary sources

- The Phase 0 report of this course, section 1 (the table of newer commands and the maintenance defaults) and section 13.

Videos (optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- Scott Chacon, "[So You Think You Know Git Part 2 - DevWorld 2024](#)" (23 min, 2024): worktrees among other topics. Current.
- Edward Thomson, "[You Don't Know Git](#)", NDC London 2025 (1 h 02 min): `reflog`, `rerere`, worktrees, rebase variants. Current.
- glich.stream, "[Chad level git: advanced concepts \(2025\)](#)" (1 h 14 min): hands-on, includes worktrees and Git LFS. Current.

Further reading

- [gitrepository-layout](#) for the `worktrees/` directory and the `commondir` and `gitdir` files.

Chapter 26: Performance

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch26/`. They show counts of objects, packs and work done. They show no timings, because a time measured on one machine on one day is not a fact about Git.

26.1 Why this matters

Four questions a CTO can ask:

1. "git status takes four seconds and git log -- path takes twenty. Is the repository broken, or too big?"
2. "CI clones 3 GB for every job. Someone proposed --depth 1 everywhere. What breaks?"
3. "A consultant told us to put git gc --aggressive into a nightly cron job. Should we?"
4. "The air-gapped training cluster needs our code every week. How do we ship it, and how do we ship only what is new?"

The answers. Neither: slow commands map to specific kinds of size, and each kind has a remedy that changes no history (sections 26.2 to 26.9). History questions break, silently: versions, blame, merge bases; a partial clone is usually what was wanted (26.11 to 26.13). No: since 2.54 Git's automatic maintenance uses a strategy that avoids exactly that kind of full rewrite, and git gc does not even take maintenance's lock (26.3 to 26.5). As bundles, with an incremental bundle whose prerequisite names what the other side must already have (26.14).

Everything in this chapter is derived data or a way of leaving data out. A commit-graph, a multi-pack-index, a bitmap, an untracked cache can be deleted and rebuilt; a shallow or partial clone holds fewer objects, not different ones. No feature here changes a commit ID. That is why they are safe to adopt, and why a slow repository is rarely a reason to rewrite history.

The repository is `orbit` from Chapter 24: 33 files, 94 commits on `main`, a feature branch, five tags.

26.2 What makes a repository slow

In one sentence. A Git command is slow when the amount of data it must touch is large, so the diagnosis is to find which data a command touches and then to make that amount smaller or indexed.

Precisely. "Large" has dimensions that do not depend on each other (Chapter 24, section 24.3): tracked files, commits and trees, blob content in all versions, loose objects and packs, refs. Measure them before you tune anything:

```
# History: how many commits, and how many objects of each type?
$ git rev-list --count --all
96
$ git cat-file --batch-all-objects --batch-check='%(objecttype)' | sort | uniq -c
122 blob
 96 commit
   5 tag
284 tree
# Working tree: how many tracked files does every status have to consider?
$ git ls-files | wc -l
33
# Refs: how many names does every fetch have to advertise and compare?
$ git for-each-ref | wc -l
7
# Storage: loose objects and packs.
$ git count-objects -v | grep -e "^count" -e in-pack -e "^packs"
count: 507
in-pack: 0
packs: 0
```

```
# The largest blob of each path in the whole history, five largest paths first:
$ git rev-list --objects --all | git cat-file --batch-check='%(objecttype) %(objectsize) %(rest)' | grep '^blob' | sort -k2 -n -r | awk '!seen[$3]++' | head -5
blob 43334 pipelines/eval/cases.jsonl
blob 19407 libs/tokenizer/vocab.txt
blob 814 docs/architecture.md
blob 401 services/gateway/routes.py
blob 352 tools/ci/affected.py
# How many versions of the largest file does the history hold?
$ git log --oneline -- pipelines/eval/cases.jsonl | wc -l
13
```

The second pipeline is the one to remember. `git rev-list --objects --all` 🟢 lists every reachable object with a path that leads to it, and `git cat-file --batch-check` adds type and size. One 43 KB file with 13 versions is where the bytes of `orbit` are. In a real repository this list

usually shows a few committed archives, datasets or generated files (Chapter 22). `git repo structure` (experimental, Chapter 14D) and the external tool `git-sizer` summarise the same dimensions.

Counting work. To see where one command spends its effort, ask Git. `GIT_TRACE2_PERF=1`  makes any command print a table of what it did (`api-trace2`). Most columns are times. The rows of type `data` carry counters, and counters are reproducible:

```
# What one "git status" does, in counted work. GIT_TRACE2_PERF prints a table; the awk
# program keeps the rows that carry a counter and drops the columns that carry times.
$ GIT_TRACE2_PERF=1 git status 2>&1 >/dev/null | awk -F'|' ' $4 ~ /data/ {gsub(/[.]/, "", $NF); gsub(/ /, "", $(NF-1)); print
$(NF-1), $NF}' | grep -e read/cache_nr -e sum_lstat -e visited
index read/cache_nr:33
index refresh/sum_lstat:33
read_directo directories-visited:20
read_directo paths-visited:53
```

One `git status` in a repository of 33 files read 33 index entries, made 33 `lstat` calls to compare them with the working tree, and visited 20 directories and 53 paths to look for untracked files. Scale those three numbers to 500,000 files and you have the four seconds of question 1. The remedies follow from the counters: fewer files to examine (sparse-checkout, Chapter 24), a smaller index (sparse index), or not examining at all (section 26.9). On your own machine, read the time columns too: `GIT_TRACE2_PERF=1 git status 2>&1 >/dev/null | less`.

Slow command	What it touches	Remedy	Section
<code>git status</code> , <code>git add</code> , <code>git switch</code>	Every index entry and tracked file; every directory	Sparse-checkout, sparse index, untracked cache, file-system monitor	Chapter 24; 26.9
<code>git log -- <path></code> , <code>git blame</code>	Every commit's tree	Changed-path filters in the commit-graph	26.6
<code>git log --graph</code> , <code>git merge-base</code> , <code>git branch --contains</code>	Commit objects, one by one	Commit-graph	26.6
Any object lookup	One index per pack	Repacking, multi-pack-index	26.3 to 26.7
<code>git clone</code> , <code>git fetch</code>	Everything reachable	Partial clone; bitmaps on the server	26.8, 26.12

26.3 `git gc` and `git maintenance`

In one sentence. `git gc` is one command that repacks everything and deletes what has expired, `git maintenance` runs a configurable set of smaller tasks, and since Git 2.54 the maintenance that Git starts on its own no longer runs `git gc`.

Analogy. `git gc` closes the warehouse for a full inventory and reshelving. `git maintenance` sends a clerk through one aisle at a time while the warehouse stays open. The analogy breaks at deletion: a full inventory is also the moment when unclaimed goods are thrown away, and the clerk's rounds postpone that moment (section 26.5).

Precisely. Chapter 3, section 3.7 showed what a pack is, and Chapter 13 what `git gc` deletes. `git maintenance run`  executes **tasks**. The manual lists them: `gc`, `commit-graph`, `prefetch`, `loose-objects`, `incremental-repack`, `pack-refs`, `reflog-expire`, `rerere-gc`, `worktree-prune`. Git 2.55 also accepts `geometric-repack`, which the task list of its manual page omits. Which tasks run without `--task` is decided by `maintenance.strategy`:

Strategy	Tasks	Deletes data?	When it is the default
<code>geometric</code>	<code>commit-graph</code> , <code>geometric-repack</code> , <code>pack-refs</code> , <code>reflog-expire</code> , <code>rerere-gc</code> , <code>worktree-prune</code>	Reflog entries past their expiry; unreachable objects only at an all-into-one repack (26.5)	Manual and automatic maintenance, since Git 2.54
<code>gc</code>	<code>gc</code>	Yes: what <code>git gc</code> deletes	Before Git 2.54
<code>incremental</code>	Hourly <code>prefetch</code> , <code>commit-graph</code> ; daily <code>loose-objects</code> , <code>incremental-repack</code> ; weekly <code>pack-refs</code>	No	Set by <code>git maintenance register</code> for scheduled runs
<code>none</code>	None	No	Scheduled maintenance, until a strategy is configured

The table is assembled from `git help maintenance` and, for the task lists, from `builtin/gc.c` at the 2.55.0 tag. The manual page of 2.55 is not consistent with itself: it calls `geometric` "the default strategy for manual maintenance" and still says in two places that "by default, only `maintenance.gc.enabled` is true". The trace below settles it.

Maintenance runs in three ways. **Automatic:** commands that write objects finish by starting `git maintenance run --auto`, detached, and each task runs only if its threshold is met (`maintenance.auto`, default true). **Manual:** you run it. **Scheduled:** `git maintenance start`  registers the repository in your global configuration, installs entries in the system scheduler (launchd on macOS, with files in `~/Library/LaunchAgents`; cron or systemd timers on Linux) that run `git maintenance run --schedule=hourly|daily|weekly`, sets the incremental strategy, and sets `maintenance.auto=false` in the repository. This book does not run `start`: it writes outside the sandbox. It is described here from `git help maintenance`.

See it. The trigger, in a new repository:

```
# Commands that write objects end by asking for maintenance. In a new repository:
$ git init -q probe
$ echo 'probe' > probe/file.txt && git -C probe add file.txt
$ GIT_TRACE=1 git -C probe commit -q -m 'Add a file' 2>&1 | sed -n 's/.*trace: run_command: //p'
git maintenance run --auto --quiet --detach
# In the repository we are going to study, nothing may run behind our back. Switch the trigger off:
$ cd orbit
$ git config set maintenance.auto false
```

`orbit` has never been maintained: 507 loose objects. `git maintenance is-needed` (Git 2.53 or later) answers with its exit status whether an automatic run would act:

```
$ git count-objects -v | grep -e "^count" -e in-pack -e "^packs"
count: 507
in-pack: 0
packs: 0
# Would an automatic run do anything? Exit status 0 means yes.
$ git maintenance is-needed --auto
[exit status: 0]
# Task by task:
$ git maintenance is-needed --auto --task=geometric-repack
[exit status: 0]
$ git maintenance is-needed --auto --task=commit-graph
[exit status: 1]
$ git maintenance is-needed --auto --task=gc
[exit status: 1]
```

The repack is needed, because more than about 100 loose objects exist. The commit-graph is not (fewer than 100 commits are missing from it), and the old `gc` threshold of about 6,700 loose objects is far away. The loose count is an estimate: Git counts the files in one of the 256 object directories, `objects/17`, and multiplies ([odb/source-loose.c](https://source-lore.com/odb/source-loose.c)).

```
# Run maintenance in the foreground and write the trace into a file beside the repository:
$ GIT_TRACE="$PWD/../trace-1.log" git maintenance run
# The commands that maintenance started, in order:
$ sed -n 's/.*trace: run_command: git //p' ../trace-1.log | grep -v -e '^pack-objects' -e '^multi-pack-index'
pack-refs --all --prune
reflog expire --all
repack -d -l --cruft --cruft-expiration=2.weeks.ago --quiet --write-midx
commit-graph write --split --reachable --no-progress
worktree prune --expire 3.months.ago
rerere gc
# The commands that the repack started in turn (process ID and temporary file name masked):
$ sed -n 's/.*trace: run_command: git //p' ../trace-1.log | grep -e '^pack-objects' -e '^multi-pack-index' | sed -e 's/tmp-[0-9]*-pack/tmp-PID-pack/' -e 's/pack-[0-9a-f]*\.pack/pack-ID.pack/' -e 's/--refs-snapshot=[^]*/--refs-snapshot=FILE/' | fold -s - w 100
pack-objects --local --quiet --delta-base-offset --honor-pack-keep .git/objects/pack/.tmp-PID-pack
--keep-true-parents --non-empty --all --reflog --indexed-objects
pack-objects --local --quiet --delta-base-offset --honor-pack-keep .git/objects/pack/.tmp-PID-pack
--cruft --cruft-expiration=2.weeks.ago --non-empty
multi-pack-index write --no-progress --preferred-pack=pack-ID.pack --refs-snapshot=FILE
--stdin-packs
```

`GIT_TRACE`  with a file name appends the trace to that file. Six commands, and no `git gc` among them. The repack line says `--cruft` with no `--geometric`, because there was no pack yet and so everything went into one. Below it, `repack` started `pack-objects` twice, once for reachable objects and once for a cruft pack that turned out empty, and then wrote a multi-pack-index.

Inside `.git`.

```
$ git count-objects -v | grep -e "^count" -e in-pack -e "^packs"
count: 0
in-pack: 507
packs: 1
# Every file under objects/ that is not a loose object (40-digit names masked):
$ find .git/objects -type f | grep -v '^[0-9a-f]{40}$' | sed 's/[0-9a-f]\{40\}/ID/' | sort
.git/objects/info/commit-graphs/commit-graph-chain
.git/objects/info/commit-graphs/graph-ID.graph
.git/objects/info/packs
.git/objects/pack/multi-pack-index
.git/objects/pack/pack-ID.idx
.git/objects/pack/pack-ID.pack
.git/objects/pack/pack-ID.rev
# The refs moved into packed-refs:
$ head -3 .git/packed-refs
# pack-refs with: peeled fully-peeled sorted
88222b7043b92ba564af0aefc4acd25af9ef15e2 refs/heads/feature/rerank-cache
100bb993157cac87afe81008782cd27cd39f8c9d refs/heads/main
$ find .git/refs -type f | wc -l
0
$ git maintenance is-needed --auto
[exit status: 1]
```


One pack with its index and reverse index, a multi-pack-index (26.7), a commit-graph stored as a chain (26.6), and every ref moved from its own file into `packed-refs` (Chapter 3, section 3.9). For comparison, the same repository maintained by the `gc` task:

```
# The same repository, maintained the older way:
$ GIT_TRACE="$PWD/../trace-3.log" git maintenance run --task=gc
$ sed -n 's/.*trace: run_command: git //p' ../trace-3.log | grep -v -e '^pack-objects'
pack-refs --all --prune
gc --quiet --no-detach --skip-foreground-tasks
repack -d -l -q --cruft --cruft-expiration=2.weeks.ago
prune --expire 2.weeks.ago --no-progress
worktree prune --expire 3.months.ago
rerere gc
$ for p in $(find .git/objects/pack/pack-*.idx; do git show-index < "$p" | wc -l; done | sort -rn
611
$ find .git/objects -type f | grep -v '/[0-9a-f][0-9a-f]/' | sed 's/[0-9a-f]\{40\}/ID/' | sort
.git/objects/info/commit-graph
.git/objects/info/packs
.git/objects/pack/pack-ID.idx
.git/objects/pack/pack-ID.pack
.git/objects/pack/pack-ID.rev
```

Here `git gc` appears, with `git prune`, which deletes loose unreachable objects past the grace period. It leaves one pack of all 611 objects (the repository had grown by then), a single commit-graph file, and no multi-pack-index.

The tasks of the `incremental` strategy, which a scheduler would spread over hours and days, can be run by name:

```
# The tasks of the "incremental" strategy, which a scheduler would run hourly and daily,
# run here once in the foreground (prefetch is left out: this repository has no remote):
$ GIT_TRACE="$PWD/../trace-4.log" git maintenance run --task=commit-graph --task=loose-objects --task=incremental-repack
$ sed -n 's/.*trace: run_command: git //p' ../trace-4.log
commit-graph write --split --reachable --no-progress
prune-packed --quiet
multi-pack-index write --no-progress
multi-pack-index expire --no-progress
multi-pack-index repack --no-progress --batch-size=1
```

None of these deletes an object that is not stored elsewhere. `prune-packed` removes loose copies of objects that are already in a pack, and the three `multi-pack-index` steps index all packs, drop packs that have become redundant, and merge small ones. The manual describes the tasks that `register` enables as "safe for running in the background without disrupting foreground processes".

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git maintenance run</code>	unchanged	unchanged	unchanged	Same value, now in <code>packed-refs</code>	Loose objects packed; multi-pack-index and commit-graph chain written; reflog entries past expiry removed; <code>rr-cache</code> and stale worktree records pruned	unchanged	unchanged
<code>git maintenance run --task=gc, git gc</code>	unchanged	unchanged	unchanged	Same value, now in <code>packed-refs</code>	All packs rewritten into one; unreachable objects into a cruft pack, or deleted when past the grace period	unchanged	unchanged

In production. Leave automatic maintenance on for ordinary repositories; it is the reason you rarely think about any of this. For a very large repository use `git maintenance start` on developer machines, which is what `scalar` does, and know what it installs (Chapter 24, section 24.7). Do not put `git gc` into cron next to it: the manual says "`git gc` modifies the object database but does not take the lock in the same way as `git maintenance run`. If possible, use `git maintenance run --task=gc` instead of `git gc`." And question 3: `--aggressive` discards all existing deltas and recomputes them. The manual's own verdict is that it is "probably not worth it" without benchmarks for your repository.

26.4 Geometric repacking

In one sentence. Geometric repacking keeps packs in a sequence where each is at least twice as large as the next, by merging only the small ones, so that regular maintenance never rewrites the large pack.

Precisely. `git repack --geometric=<factor>`  "determines a 'cut' of packfiles that need to be repacked into one in order to ensure a geometric progression", leaving as many of the larger packs intact as possible (`git help repack`). Size is counted in objects. The cost of a run is then proportional to the new data, not to the repository. `git gc` and `git repack -a` rewrite everything every time, and on a large repository that difference decides whether maintenance can run daily.

See it. Six commits later, the second run uses the geometric mode:

```
# Six more commits, then a second run:
$ git count-objects -v | grep -e '^count' -e in-pack -e "packs"
count: 24
```

```

in-pack: 507
packs: 1
$ GIT_TRACE="$PWD/../trace-2.log" git maintenance run
$ sed -n 's/.*/trace: run_command: git //p' ../trace-2.log | grep -v -e '^pack-objects' -e '^multi-pack-index'
pack-refs --all --prune
reflog expire --all
repack -d -l --geometric=2 --quiet --write-midx
commit-graph write --split --reachable --no-progress
worktree prune --expire 3.months.ago
rerere gc
$ git count-objects -v | grep -e '^count' -e in-pack -e '^packs'
count: 0
in-pack: 531
packs: 2

```

```

# Objects per pack, largest first:
$ for p in .git/objects/pack/pack-*.idx; do git show-index < "$p" | wc -l; done | sort -rn
507
24
# Six more commits and a third run. The new pack is not rolled into the old small one yet:
$ git maintenance run
$ for p in .git/objects/pack/pack-*.idx; do git show-index < "$p" | wc -l; done | sort -rn
507
24
24
# Fourteen more commits and a fourth run. Now the two small packs and the new objects merge:
$ git maintenance run
$ for p in .git/objects/pack/pack-*.idx; do git show-index < "$p" | wc -l; done | sort -rn
507
104
# The commit-graph grew as a chain of files, one line per file:
$ wc -l < .git/objects/info/commit-graphs/commit-graph-chain
2

```

After the second run: 507 and 24. After the third: 507, 24 and 24. The split is computed over the packs that exist, 507 and 24, which are in progression, so nothing is merged and the new loose objects become a pack of their own. At the fourth run the existing packs are 507, 24 and 24; the two small ones break the rule, so they merge with the new objects into one pack of 104, and 507 is at least twice 104. The large pack was never touched.

Picture.

run 2: [507] [24]	new loose objects become a pack
run 3: [507] [24] [24]	507 >= 2 x 24: nothing existing is merged
run 4: [507] [104]	24 < 2 x 24: the small packs and 56 new objects merge
never rewritten	

In production. GitHub described the move to geometric repacking on its servers in 2021, with the multi-pack-index and bitmaps that make many packs cheap to read ([Scaling monorepo maintenance](#)). On your side the consequence is that several packs are the normal state of a healthy repository. A count of packs is not a problem to fix; a count in the hundreds with no multi-pack-index is (26.16).

26.5 Cruft packs

In one sentence. A cruft pack is a pack that holds only unreachable objects, with a companion file recording when each was last touched, so that garbage can wait out its grace period without becoming thousands of loose files.

Precisely. Chapter 13, section 13.13 covered the retention rules and the destructive pair of commands. The storage side is new here. Unreachable objects must be kept for a grace period (`gc.pruneExpire`, two weeks), because a running command may be about to reference them. A cruft pack stores them together with a `.mtimes` file of per-object timestamps (`git help gitformat-pack`, "Cruft packs"). Under the geometric strategy a cruft pack is written only when the repack is all-into-one; an ordinary geometric run does not look at reachability at all.

See it. Start from one pack, delete an unmerged branch and expire the reflogs:

```

# One pack, everything reachable:
$ for p in .git/objects/pack/pack-*.idx; do git show-index < "$p" | wc -l; done | sort -rn
507
$ git fsck --unreachable --no-reflogs | wc -l
0
# Delete the unmerged feature branch. Its two commits were made on that branch, so its reflog
# went with it, but the HEAD reflog still names them:
$ git branch -D feature/rerank-cache
Deleted branch feature/rerank-cache (was 88222b7).
$ git fsck --unreachable | wc -l
0
# Remove that protection too (Chapter 13). Now the objects are unreachable:
$ git reflog expire --expire=now --all
$ git fsck --unreachable | sort
unreachable blob 0aced5d065592ae1a158a66931131e377b97dbbe
unreachable blob c2cf087c511ab0c6fbcf5667654692ea854df5ff

```

```

unreachable commit 88222b7043b92ba564af0aefc4acd25af9ef15e2
unreachable commit 9c9fd21b4c3b6d3699faf92edca795c05502285d
unreachable tree 1daf6eb53180a27301435609d474f8052dc5e3aa
unreachable tree 251b5335442f8401b0fc4c2dd70755de905d2062
unreachable tree 57751408afb309db0f6d6cbee79823581a949c
unreachable tree 5a983380960468216ce59f604efbd7fe29996b8a
unreachable tree dfdea423ad6498c15fc99a910f4fcaa4729fdebf
unreachable tree fe9a2ed70859ef92d48e7dfb7d4d5a302c977f1e

```

```

# One new commit, then a maintenance run of the default strategy. It packs the new loose
# objects and deletes nothing:
$ printf '\nSee docs/runbooks for the on-call procedures.\n' >> README.md
$ git commit -q -am 'readme: point to the runbooks'
$ git maintenance run
$ for p in .git/objects/pack/pack-*.idx; do git show-index < "$p" | wc -l; done | sort -rn
507
3
$ ls .git/objects/pack | cut -d. -f2 | sort | uniq -c
2 idx
1 multi-pack-index
2 pack
2 rev
$ git fsck --unreachable | wc -l
10

```

The default strategy packed three new objects and left the ten unreachable ones where they were: inside the large pack, which a geometric run does not rewrite.

```

# The gc task repacks everything into one pack and separates the unreachable objects:
$ git maintenance run --task=gc
$ for p in .git/objects/pack/pack-*.idx; do git show-index < "$p" | wc -l; done | sort -rn
500
10
$ ls .git/objects/pack | cut -d. -f2 | sort | uniq -c
2 idx
1 mtimes
2 pack
2 rev
# The pack that has an .mtimes file is the cruft pack. Its objects are still readable:
$ git cat-file -t 88222b7
commit
$ git log --oneline -2 88222b7
88222b7 ranker: make the cache size configurable
9c9fd21 ranker: cache scores per query and document

```

The `gc` task separates them: 498 reachable objects in one pack, ten in a cruft pack with its `.mtimes` file. The deleted branch is still readable by ID.

```

# The cruft pack waits for the grace period (two weeks by default). With no grace period:
$ git gc --prune=now
$ ls .git/objects/pack | cut -d. -f2 | sort | uniq -c
1 idx
1 pack
1 rev
$ git cat-file -t 88222b7
fatal: Not a valid object name 88222b7
[exit status: 128]
$ git fsck --unreachable | wc -l
0

```

`git gc --prune=now`  ends the grace period (Chapter 13 gives the five facts for this command). The transcript uses `now` because a cut-off that depends on the clock could not be reproduced.

In production. Two consequences. A repository under the default strategy can hold unreachable objects for a long time, which matters after a secret has been removed from history: rewriting and force-pushing does not delete the objects from your clone, and the purge is the deliberate procedure of Chapter 21B. And `count-objects` no longer shows garbage as loose files, so "the repository is large although the history is small" is diagnosed with `git fsck --unreachable | wc -l` and by looking for `.mtimes` files.

26.6 The commit-graph

In one sentence. The commit-graph is a file that stores, for every commit, its parents, its root tree, its date and its distance from the roots of history, and optionally a compact summary of the paths it changed, so that history walks stop decompressing commit objects and stop comparing trees that did not change.

Analogy. A railway timetable printed from the day's train records. Asking the timetable is faster than reading the records, and it is wrong the moment the records change without a reprint. The analogy breaks in Git's favour: Git notices commits that are missing from the file and reads those from the objects, so a stale commit-graph is slower, not wrong. A damaged one is another matter (Lab 16.4).

Precisely. Two things are stored ([commit-graph design](#); [Git's database internals II](#) and [III](#)):

- **The graph with generation numbers.** A commit's generation is larger than that of every ancestor. A walk that asks "is A an ancestor of B" can stop as soon as it reaches commits whose generation is below A's, instead of walking to the root. This is what `git merge-base`, `git branch --contains`, `git tag --contains` and `git log --graph` gain from.
- **Changed-path Bloom filters** (`--changed-paths`). For each commit, a small probabilistic set of the paths that differ from its first parent. Asked "did this commit touch `services/ranker`?", a filter answers "definitely not" or "maybe". Only the "maybe" commits need their trees compared. This is what `git log --<path>` and `git blame` gain from.

`git commit-graph write --reachable --changed-paths`  writes the file for all commits reachable from refs; `git commit-graph verify`  checks it against the object database. `git gc` writes one (`gc.writeCommitGraph`, default true), the `commit-graph` maintenance task writes one incrementally, and `fetch.writeCommitGraph` does it after fetches. Filters are written only if requested once, or with `commitGraph.changedPaths=true` (Git 2.52 or later).

See it.

```
# No commit-graph yet. A path-limited log has to compare trees commit by commit:
$ ls .git/objects/info
[exit status: 0]
$ git rev-list --count main
94
$ git log --oneline -- services/ranker | wc -l
13
# Git reports statistics about changed-path filters through trace2. Without a file there are none:
$ GIT_TRACE2_PERF=1 git log --oneline -- services/ranker 2>&1 >/dev/null | grep -c 'statistics:'
0
```

Without the file, `git log -- services/ranker` compares the tree of each of the 94 commits with its parent's to find the 13 that changed the path.

```
$ git commit-graph write --reachable --changed-paths
$ ls .git/objects/info
commit-graph
$ git commit-graph verify
[exit status: 0]
# The file starts with the signature CGPH and a table of chunks:
$ xxd -l 96 .git/objects/info/commit-graph
00000000: 4347 5048 0101 0600 4f49 4446 0000 0000  CGPH....OIDF....
00000010: 0000 005c 4f49 444c 0000 0000 0000 045c  ...\\OIDL.....\\
00000020: 4344 4154 0000 0000 0000 0bdc 4744 4132  CDAT.....GDA2
00000030: 0000 0000 0000 195c 4249 4458 0000 0000  ....\\BIDX....
00000040: 0000 1adc 4244 4154 0000 0000 0000 1c5c  ....BDAT.....\\
00000050: 0000 0000 0000 0000 0000 1e07 0000 0001  .....
```

Inside `.git`. One file, `objects/info/commit-graph`. Its header is the signature `CGPH`, and the chunk table that follows names the parts: `OIDF` and `OIDL` (the commit IDs, sorted, with a fan-out table), `CDAT` (tree, parents, generation and date per commit), `GDA2` (generation data), `BIDX` and `BDAT` (the Bloom filters) ([gitformat-commit-graph](#)).

```
# The same question again. One filter per commit was consulted:
$ GIT_TRACE2_PERF=1 git log --oneline -- services/ranker 2>&1 >/dev/null | sed -n 's/.*statistics://p'
{"filter_not_present":0,"maybe":13,"definitely_not":81,"false_positive":0}
$ git log --oneline -- services/ranker | wc -l
13
# A path that few commits touch:
$ GIT_TRACE2_PERF=1 git log --oneline -- libs/schemas 2>&1 >/dev/null | sed -n 's/.*statistics://p'
{"filter_not_present":0,"maybe":2,"definitely_not":92,"false_positive":0}
$ git log --oneline -- libs/schemas
100bb99 schemas, gateway, ingest: add the tenant field in one change
00eae68 schemas: add the event types and their JSON schemas
```

The statistics come from Git itself. For `services/ranker`, 81 of 94 commits were dismissed by their filter without a tree comparison and 13 said "maybe"; all 13 were real, so `false_positive` is 0. For `libs/schemas`, 92 were dismissed. The answer did not change, and the work fell from 94 comparisons to 13 and to 2. That ratio, not a stopwatch, is why the feature matters: in a monorepo most commits do not touch any given path.

```
# A commit-graph is a snapshot. New commits are not in it until it is written again:
$ printf '\nSee docs/runbooks for the on-call procedures.\n' >> README.md
$ git commit -q -am 'readme: point to the runbooks'
$ git commit-graph verify
[exit status: 0]
# Maintenance adds a small file on top instead of rewriting the large one:
$ git commit-graph write --reachable --changed-paths --split
$ find .git/objects/info -type f | sed 's/[0-9a-f]{40}/ID/' | sort
.git/objects/info/commit-graphs/commit-graph-chain
.git/objects/info/commit-graphs/graph-ID.graph
.git/objects/info/commit-graphs/graph-ID.graph
$ wc -l < .git/objects/info/commit-graphs/commit-graph-chain
```

```
2
$ git commit-graph verify
[exit status: 0]
```

A new commit does not invalidate the file. With `--split`, the form maintenance uses, a small file is added on top and `commit-graph-chain` lists the layers; layers are merged when the upper one grows large relative to the one below (`git help commit-graph`).

```
# A replacement object changes what the history is, so Git stops trusting the file:
$ git replace --graft HEAD HEAD~2
$ GIT_TRACE2_PERF=1 git log --oneline -- services/ranker 2>&1 >/dev/null | grep -c 'statistics:'
0
$ git replace -d HEAD
Deleted replace ref '80e1432b3813ec4352a4131e441d28a312b981ca'
$ GIT_TRACE2_PERF=1 git log --oneline -- services/ranker 2>&1 >/dev/null | grep -c 'statistics:'
1
# The switch that bypasses the file for one command:
$ GIT_TRACE2_PERF=1 git -c core.commitGraph=false log --oneline -- services/ranker 2>&1 >/dev/null | grep -c 'statistics:'
0
```

The manual's caveat, observed: "The existence of replace objects or commit grafts turns off reading or writing to the commit-graph." The same holds in a shallow repository, as section 26.11 shows. `core.commitGraph=false` bypasses the file for one command, which is the diagnostic to remember.

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git commit-graph write --reachable --changed-paths</code>	unchanged	unchanged	unchanged	unchanged	<code>objects/info/commit-graph</code> written, or a layer under <code>objects/info/commit-graphs/</code> with <code>--split</code>	unchanged	unchanged

In production. An evaluation team asks "which commits changed the scoring rubric" many times a day. On a repository with a long history that question is the twenty seconds of section 26.1, and changed-path filters are the fix. Microsoft published the effect on the Linux and Windows repositories when the feature was introduced in Git 2.18 ([Supercharging the Git Commit Graph](#)); the numbers there are theirs, on their machines.

26.7 The multi-pack-index

In one sentence. The multi-pack-index is one sorted index over the objects of many packs, so that finding an object costs one lookup however many packs there are.

Precisely. Each pack has its own `.idx` (Chapter 3, section 3.7). With N packs, a lookup may search N indexes. Geometric repacking, fetches and partial clones all produce many packs on purpose, so the cost has to be removed elsewhere: `objects/pack/multi-pack-index` maps every object ID to a pack and an offset ([git-multi-pack-index](#)). Git reads it when `core.multiPackIndex` is true, the default.

See it. A clone that has fetched three times with `fetch.unpackLimit=1`, the setting `scalar` uses so that every fetch keeps its pack:

```
# A clone that has fetched three times since it was made holds four packs, each with its own index:
$ cd dev
$ ls .git/objects/pack | cut -d. -f2 | sort | uniq -c
 4 idx
 4 pack
 4 rev
$ for p in .git/objects/pack/pack-*.idx; do git show-index < "$p" | wc -l; done | sort -rn
507
 4
 4
 4
```

```
# One index over all packs:
$ git multi-pack-index write
$ ls .git/objects/pack | cut -d. -f2 | sort | uniq -c
 4 idx
 1 multi-pack-index
 4 pack
 4 rev
$ git multi-pack-index verify
[exit status: 0]
# Signature MIDX, version, hash version, number of chunks, number of base files, number of packs:
$ xxd -l 12 .git/objects/pack/multi-pack-index
00000000: 4d49 4458 0101 0400 0000 0004          MIDX.....
# Objects are found exactly as before:
$ git cat-file -t HEAD
commit
```

```
$ git rev-list --count --objects --all
519
```

`git multi-pack-index write`  created one file; the last byte of the header shown says four packs. Nothing else changed, and nothing you can observe through Git changed. Maintenance writes this file as part of its repack (`--write-midx` in the trace of 26.3), and the `incremental-repack` task uses it to merge small packs without a full rewrite.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git multi-pack-index write</code>	unchanged	unchanged	unchanged	unchanged	objects/pack/multi-pack-index written; packs untouched	unchanged	unchanged

In production. You do not run this command by hand. You recognise the file, you know it is derived data that `git multi-pack-index verify` checks, and you know that many packs without it are a sign that maintenance has not run (26.16).

26.8 Reachability bitmaps

In one sentence. A bitmap file stores, for selected commits, one bit per object in the pack saying whether that commit reaches it, so that a server can answer "which objects does this client need" without walking the graph.

Precisely. Serving a clone or fetch starts with "enumerating objects": everything reachable from the wanted commits minus everything reachable from the ones the client has. A walk does that commit by commit and tree by tree. With bitmaps it is a few bitwise operations, and stretches of the existing pack can be copied to the client verbatim ([bitmap-format](#); [Git's database internals IV](#)). `repack.writeBitmaps` "defaults to true on bare repos, false otherwise": bitmaps are a server feature. `scalar` sets `pack.useBitmaps=false` on clients, with the reason that bitmaps "are optimized for server-side use, not client-side use".

See it.

```
$ cd ..
# The bare repository that plays the server was packed by "git gc". Beside the pack:
$ ls server/orbit.git/objects/pack | cut -d. -f2 | sort | uniq -c
 1 bitmap
 1 idx
 1 pack
 1 rev
# The non-bare repository after the same command:
$ ls orbit/.git/objects/pack | cut -d. -f2 | sort | uniq -c
 1 idx
 1 pack
 1 rev
# Why the difference: the default of repack.writeBitmaps depends on the kind of repository.
$ git -C server/orbit.git config get repack.writeBitmaps
[exit status: 1]
$ git -C server/orbit.git rev-parse --is-bare-repository
true
```

The same `git gc`, and only the bare repository got a `.bitmap`. Now serve one clone from it and read the counters that `git pack-objects` on the serving side reports:

```
# Serve one full clone and ask the serving side how it filled the pack:
$ GIT_TRACE2_PERF=1 git clone -q --bare "file://$PWD/server/orbit.git" with-bitmap.git 2>&1 | awk -F'|' '$4 ~ /data/ {gsub(/[\.\/, ""], $NF); gsub(/ /, "", $(NF-1)); print $(NF-1), $NF}' | grep -e 'written:' -e 'reused:' -e 'pack-reused:'
pack-objects written:519
pack-objects reused:0
pack-objects pack-reused:519
# Remove the bitmap and serve the same clone again:
$ rm server/orbit.git/objects/pack/*.bitmap
$ GIT_TRACE2_PERF=1 git clone -q --bare "file://$PWD/server/orbit.git" without-bitmap.git 2>&1 | awk -F'|' '$4 ~ /data/ {gsub(/[\.\/, ""], $NF); gsub(/ /, "", $(NF-1)); print $(NF-1), $NF}' | grep -e 'written:' -e 'reused:' -e 'pack-reused:'
pack-objects written:519
pack-objects reused:519
pack-objects pack-reused:0
# Both clones are complete:
$ git -C with-bitmap.git rev-list --count --objects --all
519
$ git -C without-bitmap.git rev-list --count --objects --all
519
```

With the bitmap, all 519 objects were `pack-reused`: copied from the existing pack as one stretch, with no per-object work. Without it, the same 519 objects were found by a walk and reused one at a time. The clones are identical.

In production. This is the hosting provider's concern, and yours if you run a Git server or a mirror for CI: a bare repository that is repacked as a whole gets its bitmap without further configuration. As a user you meet the feature in the first phase of a clone, which the configuration manual calls "counting objects": that is the phase a bitmap shortens.

26.9 The working tree: untracked cache and file-system monitor

In one sentence. The untracked cache remembers which directories contained no untracked files, the file-system monitor tells Git which paths changed since the last command, and together they replace the scan of the working tree by a question.

Precisely. Section 26.2 counted two kinds of work in `git status`: an `lstat` call per tracked file, and a walk of every directory in search of untracked files.

- **Untracked cache** (`core.untrackedCache`). The index records the modification time of each directory; a directory whose time has not changed is not read again. It depends on the file system updating directory times, which `git update-index --test-untracked-cache` checks. The default is `keep`; `feature.manyFiles=true` and `scalar` switch it on.
- **File-system monitor** (`core.fsmonitor=true`). A daemon, `git fsmonitor--daemon`, subscribes to the operating system's change notifications for one working tree. `git status` asks it for the changed paths and calls `lstat` only on those. With the configuration set, "commands, such as `git status`, will ask the daemon for changes and automatically start it (if necessary)" (`git help fsmonitor--daemon`). The manual lists caveats: it does not know about submodules, and it refuses network-mounted repositories by default.

See it. This book starts no daemon, so the transcript shows only that both are off by default and how to ask:

```
# Neither accelerator of the working-tree scan is on by default:
$ git config get core.fsmonitor
[exit status: 1]
$ git config get core.untrackedCache
[exit status: 1]
# Asking for the status of the daemon does not start one:
$ git fsmonitor--daemon status
fsmonitor-daemon is not watching '$LAB/ch26/scale-dimensions/orbit'
[exit status: 1]
$ git version --build-options | grep feature
feature: fsmonitor--daemon
```

`git fsmonitor--daemon status`  exits 1 and starts nothing. The untracked cache is not demonstrated with counters either: its effect depends on directory modification times, and a transcript that depends on file ages is not reproducible.

Unverified. The first release with a working built-in daemon: the manual page exists at the 2.36.0 tag, and a GitHub engineering post dates the feature to Git 2.37 ([GitHub Blog](#)). Git 2.55 added the daemon on Linux according to its release notes, while the configuration manual of 2.55 still names only Windows and macOS.

In production. These two matter when `sum_lstat` and `directories-visited` are in the hundreds of thousands. Canva reported `git status` going from 10 seconds to about 3 with the monitor, the untracked cache, scheduled maintenance and sparse checkout together (their measurement; article linked in Chapter 24). Both features belong to developer machines. A CI job that runs `git status` once gains nothing from a daemon that must first be started.

26.10 The fetch conversation

In one sentence. A fetch is a short dialogue in which the server advertises what it has, the client says what it wants and what it already has, and the server answers with one pack of the difference.

Precisely. Chapter 12, section 12.13 printed a whole trace. Four steps are enough to reason about every clone shape in this chapter ([gitprotocol-v2](#)):

1. **Advertise.** The server names the protocol version and its capabilities. The client asks for refs with `ls-refs`.
2. **Want.** The client names the tips it wants.
3. **Have.** The client names commits it has; the server acknowledges those it knows. This negotiation finds the common boundary, and `fetch.negotiationAlgorithm` controls how the client picks what to offer.
4. **Pack.** The server enumerates the objects on its side of the boundary (26.8) and sends them as one packfile.

See it.

```
# What the serving side says it can do for a fetch:
$ GIT_TRACE_PACKET=1 git ls-remote "file://$PWD/server/orbit.git" HEAD 2>&1 | sed -n 's/.*packet: *ls-remote< //p' | sed -n '1,/^\0000/p'
version 2
agent=git/2.55.0-Darwin
ls-refs=unborn
fetch=shallow wait-for-done filter
server-option
object-format=sha1
0000
```


The line `fetch=shallow wait-for-done filter` is the server saying which extras its `fetch` command supports. `filter` appears only because the lab server sets `uploadpack.allowFilter`.

```
# A full clone, with the conversation written to a file beside it:
$ GIT_TRACE_PACKET="$PWD/full.trace" git clone -q "file://$PWD/server/orbit.git" full
# The request names one tip per ref it wants, has nothing to offer, and says so:
$ sed -n 's/.*packet: *clone> //p' full.trace | sed -n '/command=fetch/, $p' | grep -c '^want'
8
$ sed -n 's/.*packet: *clone> //p' full.trace | sed -n '/command=fetch/, $p' | grep -c '^have'
0
$ sed -n 's/.*packet: *clone> //p' full.trace | sed -n '/command=fetch/, $p' | grep -v -e '^want' | tr '\n' ' '
command=fetch agent=git/2.55.0-Darwin object-format=sha1 0001 thin-pack no-progress ofs-delta done 0000
# The answer is one section, the pack:
$ sed -n 's/.*packet: *clone< //p' full.trace | grep -e acknowledgments -e shallow-info -e packfile
packfile
```

```
# A shallow clone adds one line to the request, and the answer starts with the boundary:
$ GIT_TRACE_PACKET=1 git clone --depth 1 "file://$PWD/server/orbit.git" shallow 2>&1 | sed -n 's/.*packet: *\(\(clone[<>]\)\)/\1/p'
| grep -e deepen -e shallow
clone< fetch=shallow wait-for-done filter
clone> deepen 1
clone< shallow-info
clone< shallow 100bb993157cac87afe81008782cd27cd39f8c9d
# A partial clone adds one line as well:
$ GIT_TRACE_PACKET=1 git clone --filter=blob:none "file://$PWD/server/orbit.git" blobless 2>&1 | sed -n 's/.*packet: *\
(clone[<>]\)/\1/p' | grep -e 'filter '
clone> filter blob:none
```

A clone has nothing to offer, so it sends wants and `done`. Each special clone is one more line in that request. `deepen 1` makes the server stop after one commit per tip, and the answer begins with a `shallow-info` section naming the commits where history was cut. `filter blob:none` makes the server leave out objects the filter rejects. Both are decisions of the server's enumeration; the client receives an ordinary pack either way.

```
# Later, the server has two new commits. The client names what it wants and what it has:
$ cd full
$ GIT_TRACE_PACKET=1 git fetch 2>&1 | sed -n 's/.*packet: *\(\(fetch[<>]\)\)/\1/p' | sed -n '/command=fetch/, $p' | grep -e want -e
have -e done -e ACK -e ready -e packfile | cut -c1-60
fetch> want 9ea7ebd6450f44a34cc11c227de15f7d2837a510
fetch> have 100bb993157cac87afe81008782cd27cd39f8c9d
fetch> have 88222b7043b92ba564af0aefc4acd25af9ef15e2
fetch> have f5915c95c140a7bfddf0cc4b9da258aa5bebe3a2
fetch> have 52ee19bf6e6ff1d7594abe2f2ecc1ac5523a2061
fetch> have 153c8285ef8f669100822c87e96585b181a79b09
fetch< ACK 100bb993157cac87afe81008782cd27cd39f8c9d
fetch< ACK 88222b7043b92ba564af0aefc4acd25af9ef15e2
fetch< ACK f5915c95c140a7bfddf0cc4b9da258aa5bebe3a2
fetch< ACK 52ee19bf6e6ff1d7594abe2f2ecc1ac5523a2061
fetch< ACK 153c8285ef8f669100822c87e96585b181a79b09
fetch< ready
fetch< packfile
$ git log --oneline -3 origin/main
9ea7ebd docs: add note 2
12adbc3 docs: add note 1
100bb99 schemas, gateway, ingest: add the tenant field in one change
```

An ordinary fetch names one wanted tip and offers the tips it has; the server acknowledges them, says `ready`, and sends a pack of the two new commits with their trees and blobs.

In production. When a fetch is slow, this model tells you where to look: a long wait before any progress is the server enumerating (bitmaps, 26.8); a long transfer is the pack (blob content, 26.2); a long pause after the transfer is the client indexing the pack and running maintenance. `GIT_TRACE_PACKET=1` and `GIT_TRACE2_PERF=1` separate the three (Chapter 29).

26.11 Shallow clones and their limits

In one sentence. A shallow clone contains the commits within a given distance of the requested tips and a file that tells Git to pretend the history ends there.

Analogy. The last page of a ledger, photocopied. It is enough to see today's balance and useless for an audit, and if you forget that it is a photocopy of one page you will conclude that the business started yesterday. The analogy breaks in one direction: unlike a photocopy, a shallow clone can be deepened later.

Precisely. `git clone --depth <n>`  sends `deepen <n>`. The server sends the commits within that distance with their complete trees and blobs. The client writes the IDs of the boundary commits into `$GIT_DIR/shallow`, and Git treats those commits as if they had no parents ([shallow](#)). `--depth` implies `--single-branch` unless `--no-single-branch` is given, and tags that point behind the boundary do not arrive. Related options: `--shallow-since=<date>` and `--shallow-exclude=<ref>` on clone and fetch, `git fetch --deepen=<n>`, `git fetch --unshallow`.

See it.

```
$ git clone --depth 1 "file://$PWD/server/orbit.git" shallow
Cloning into 'shallow'...
$ git -C shallow cat-file --batch-all-objects --batch-check='%(objecttype)' | sort | uniq -c
 33 blob
   1 commit
   1 tag
  20 tree
$ git -C shallow log --oneline
100bb99 schemas, gateway, ingest: add the tenant field in one change
$ git -C shallow tag -l | wc -l
1
```

One commit, its 20 trees and 33 blobs: one complete snapshot.

Inside `.git`.

```
$ git rev-parse --is-shallow-repository
true
$ cat .git/shallow
100bb993157cac87afe81008782cd27cd39f8c9d
# The commit object is unchanged and still names its parent. Git has been told to stop there:
$ git cat-file -p HEAD | sed -n 1,2p
tree f2e47b39bb31b7f1afa42bfb51fa6919b4aa3842
parent d259a34f3117e4516fb2f7c071b2dd9c8edc9e5b
$ git log --format='%h parents: [%p] %s'
100bb99 parents: [] schemas, gateway, ingest: add the tenant field in one change
$ git cat-file -t HEAD~1
fatal: Not a valid object name HEAD~1
[exit status: 128]
$ git config get --all remote.origin.fetch
+refs/heads/main:refs/remotes/origin/main
```

The commit object still names its parent `d259a34`. An object cannot be altered without changing its ID. What changed is Git's reading: the ID in `.git/shallow` makes every walk stop there, `%p` prints no parent, and `HEAD~1` does not resolve.

```
# Questions about history get short or wrong answers, without a warning:
$ git rev-list --count HEAD
1
$ git log --oneline -- services/ranker | wc -l
1
$ git blame -s services/gateway/routes.py | sed -n 1,3p
^100bb99 1) ROUTES = [
^100bb99 2)     ("GET", "/healthz"),
^100bb99 3)     ("POST", "/v1/search/1"),
$ git shortlog -sn HEAD
1 Lab User
$ git describe --match 'gateway/v*'
fatal: No names found, cannot describe anything.
[exit status: 128]
# A commit-graph cannot be written while the history is cut off. Git says nothing:
$ git commit-graph write --reachable
[exit status: 0]
$ ls .git/objects/info
[exit status: 0]
```

This is the danger. Every command exits with status 0 and a plausible answer: one commit in history, one commit that ever touched the ranker, every line of the file blamed on the boundary commit (the `^` marks it), one author. Only `git describe` fails loudly, because the gateway tags lie behind the boundary. And `git commit-graph write` writes nothing, without a message.

Observed behavior	: a version string, a changelog or a "who changed this" report produced in CI is wrong or empty, while the same command on a laptop is right.
Git state	: "git rev-parse --is-shallow-repository" prints true; <code>.git/shallow</code> exists.
Mechanism	: history walks stop at the boundary commits and report what they saw.
Root cause	: the job needed history and was given a snapshot.
Why Git does this	: a shallow repository is defined as complete up to its boundary. Git cannot know that your question reaches further.
Correct fix	: "git fetch --unshallow", or enough depth plus tags; or a blobless clone.
Prevention	: shallow clones only where the job reads one snapshot and is thrown away.

```
$ git fetch --deepen=5
$ git rev-list --count HEAD
6
$ cat .git/shallow
8d17b5e4d7bcbb52ee1b37a94a03d6704394b4db
$ git log --oneline -- services/ranker | wc -l
1
```

```

$ git fetch --unshallow
From file:///LAB/ch26/shallow-limits/server/orbit
* [new tag]      gateway/v1.0.0 -> gateway/v1.0.0
* [new tag]      gateway/v1.1.0 -> gateway/v1.1.0
* [new tag]      ranker/v0.9.0 -> ranker/v0.9.0
* [new tag]      schemas/v1.0.0 -> schemas/v1.0.0
$ git rev-parse --is-shallow-repository
false
$ ls .git/shallow
ls: .git/shallow: No such file or directory
[exit status: 1]
$ git rev-list --count HEAD
94
$ git describe --match 'gateway/v*'
gateway/v1.1.0-28-g100bb99
# The refspec is still the narrow one that --depth implied (Chapter 12, section 12.12):
$ git config get --all remote.origin.fetch
+refs/heads/main:refs/remotes/origin/main
$ git branch -r
origin/HEAD -> origin/main
origin/main

```

`--deepen=5` moved the boundary five commits back; `--unshallow` 🟢 removed it and brought the tags. The refspec stays narrow: the clone still follows `main` only (Chapter 12, section 12.12 widens it).

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code>git clone --depth <n> <url></code>	Full checkout of the tip	Full	The cloned branch	Created	Objects of <code>n</code> commits per tip; <code>.git/shallow</code> ; a refspec for one branch	unchanged	unchanged
<code>git fetch --deepen=<n>, --unshallow</code>	unchanged	unchanged	unchanged	unchanged	Older commits added; <code>.git/shallow</code> rewritten or removed; tags that became reachable created	Read only	unchanged

In production. GitHub's own guidance is that shallow clones belong in builds that are thrown away, because later fetches into a shallow repository are expensive for the server and history commands break ([GitHub Blog](#)). `actions/checkout` produces a shallow, tagless clone by default (Chapter 24, section 24.9). If a repository must not be cloned from a shallow source, `git clone --reject-shallow` and `clone.rejectShallow` refuse it.

26.12 Partial clone and promisor remotes

In one sentence. A partial clone has all of the history's structure and leaves out objects of a chosen kind, which a designated remote has promised to deliver when a command needs them.

Analogy. A book whose footnotes are kept at the publisher. The text is complete, every footnote number is there, and when you look one up it is sent to you and stays in your copy. Read offline, the footnotes you never looked up are unavailable. The analogy breaks at who looks things up: here it is any Git command, without asking you.

Precisely. `git clone --filter=<spec>` 🟢 sends `filter <spec>` (`git help rev-list` lists the forms):

- `blob:none`, **blobless**: all commits and trees, no blobs except those the checkout needs.
- `tree:0`, **treeless**: all commits, no trees or blobs except those the checkout needs.
- `blob:limit=<n>`: omit blobs of at least that size.

The clone records `remote.<name>.promisor=true` and `remote.<name>.partialclonefilter`. Packs from that remote get a `.promisor` file beside them, and an object that is missing but referenced from such a pack counts as promised, not as corruption ([partial-clone](#)). When a command needs a missing object, Git fetches it from the promisor remote.

See it.

```

$ git clone --filter=blob:none "file://$PWD/server/orbit.git" blobless
Cloning into 'blobless'...
$ git -C blobless cat-file --batch-all-objects --batch-check='%(objecttype)' | sort | uniq -c
warning: This repository uses promisor remotes. Some objects may not be loaded.
 33 blob
 96 commit
  5 tag
284 tree
$ git -C blobless rev-list --count --all
96
$ git -C blobless ls-files | wc -l
33

```

All 96 commits, all 284 trees, and 33 blobs: the files of the checkout. `git cat-file` warns that it lists only what is local.

Inside .git.

```
$ git config get --all --show-names --regexp "^remote\.origin\.(promisor|partialclonefilter)$"
remote.origin.promisor true
remote.origin.partialclonefilter blob:none
# Two packs arrived: commits and trees at clone time, then the blobs of the checkout.
$ ls .git/objects/pack | cut -d. -f2 | sort | uniq -c
  2 idx
  2 pack
  2 promisor
  2 rev
$ for p in .git/objects/pack/pack-*.idx; do git show-index < "$p" | wc -l; done | sort -rn
 385
  33
# Objects that the history refers to and this repository does not hold:
$ git rev-list --objects --all --missing=print | grep -c '^?'
89
# Git does not call that damage:
$ git fsck
[exit status: 0]
```

89 objects that the history refers to are absent, and `git fsck` exits 0.

```
# A command that needs an old version of a file fetches it, without being asked:
$ GIT_TRACE=1 git show HEAD~20:services/ranker/features.py 2>&1 >/dev/null | sed -n 's/.*trace: run_command: git //p' | grep -v
-e '^pack-objects' -e '^index-pack' -e '^maintenance'
-c fetch.negotiationAlgorithm=noop fetch origin --no-tags --no-write-fetch-head --recurse-submodules=no --filter=blob:none --
stdin
$ ls .git/objects/pack/*.pack | wc -l
  3
# Commands that only read commits and trees fetch nothing:
$ git log --oneline -- services/ranker | wc -l
 13
$ git diff --name-status HEAD~10 HEAD -- services | wc -l
  5
$ ls .git/objects/pack/*.pack | wc -l
  3
# Counting changed lines needs file contents. Git asks for all of them in one request:
$ GIT_TRACE=1 git diff --stat HEAD~10 HEAD -- services 2>&1 | grep -c 'run_command: git .*fetch'
  1
$ ls .git/objects/pack/*.pack | wc -l
  4
```

`git show` of an old version triggered a `git fetch` for one object, with negotiation switched off (`noop`). Commands that read only commits and trees (`git log --<path>`, `git diff --name-status`, `git merge-base`) fetched nothing. `git diff --stat` needed contents and fetched them in one request.

```
# A patch needs both versions of the file for every commit shown: one request per commit.
$ GIT_TRACE=1 git log -p --format=%s -- services/ingest/settings.py 2>&1 >/dev/null | grep -c 'run_command: git .*fetch'
11
# Blame walks one version at a time as well:
$ GIT_TRACE=1 git blame pipelines/training/config.yaml 2>&1 >/dev/null | grep -c 'run_command: git .*fetch'
12
$ ls .git/objects/pack/*.pack | wc -l
 27
$ git rev-list --objects --all --missing=print | grep -c '^?'
60
```

This is the cost. `git log -p` on one file made 11 separate requests and `git blame` made 12, each a round trip to the server, and the repository now holds 27 packs. The manual of `git backfill` describes it in the same terms: such commands "become very slow as they download the missing blobs in single-blob requests".

```
# git backfill (experimental) asks for the missing blobs of the current branch in batches:
$ GIT_TRACE=1 git backfill 2>&1 | grep -c 'run_command: git .*fetch'
 1
$ git rev-list --objects --all --missing=print | grep -c '^?'
 2
# What is still missing belongs to another branch:
$ git rev-list --objects HEAD --missing=print | grep -c '^?'
 0
$ ls .git/objects/pack/*.pack | wc -l
 28
```

```
# Many small packs are the price of on-demand fetching. Maintenance merges them:
$ git maintenance run
$ ls .git/objects/pack | cut -d. -f2 | sort | uniq -c
  1 idx
  1 multi-pack-index
  1 pack
  1 promisor
  1 rev
```

`git backfill`  (experimental, Git 2.49 or later) fetched everything reachable from `HEAD` in one batch, grouped by path; the two objects still missing belong to the other branch. Maintenance then merged 28 packs into one promisor pack.

When the promise cannot be kept.

```
# A second blobless clone, and the server becomes unreachable:
$ cd ..
$ git clone -q --filter=blob:none "file://$PWD/server/orbit.git" laptop
$ mv server server-offline
$ cd laptop
# What is local still works:
$ git log --oneline -2
100bb99 schemas, gateway, ingest: add the tenant field in one change
d259a34 docs: record architecture change 12
$ git status --short --branch
## main...origin/main
# What needs a missing blob does not:
$ git show HEAD~20:services/ranker/features.py
fatal: '$LAB/ch26/partial-clone/server/orbit.git' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
fatal: could not fetch 3b82e45dc30fa47e6f6aa66193090e2b590f7da5 from promisor remote
[exit status: 128]
$ git switch feature/rerank-cache
fatal: '$LAB/ch26/partial-clone/server/orbit.git' does not appear to be a git repository
fatal: Could not read from remote repository.

Please make sure you have the correct access rights
and the repository exists.
fatal: could not fetch 03ada008385944bed720efa38fd6346db17f4169 from promisor remote
[exit status: 128]
$ git status --short --branch
## main...origin/main
$ mv ../server-offline ../server
$ git switch feature/rerank-cache
Switched to a new branch 'feature/rerank-cache'
branch 'feature/rerank-cache' set up to track 'origin/feature/rerank-cache'.
```

Without the server, everything local works and anything that needs a missing blob fails with `could not fetch ... from promisor remote`. `git switch` failed before touching the working tree: `git status` afterwards still shows `main`, clean.

A filter that is silently not applied.

```
# A path instead of a URL: the local transport copies files and ignores the filter.
$ git clone --filter=blob:none server/orbit.git by-path
Cloning into 'by-path'...
warning: --filter is ignored in local clones; use file:// instead.
done.
$ git -C by-path cat-file --batch-all-objects --batch-check='%(objecttype)' 2>/dev/null | sort | uniq -c
 122 blob
   96 commit
    5 tag
  284 tree
# A server that does not allow filters (uploadpack.allowFilter is false unless set):
$ git -C server/orbit.git config set uploadpack.allowFilter false
$ git clone --filter=blob:none "file://$PWD/server/orbit.git" refused
Cloning into 'refused'...
warning: filtering not recognized by server, ignoring
$ git -C refused cat-file --batch-all-objects --batch-check='%(objecttype)' 2>/dev/null | sort | uniq -c
 122 blob
   96 commit
    5 tag
  284 tree
# Both clones are complete, and both are nevertheless configured as partial clones:
$ git -C refused config get remote.origin.partialclonefilter
blob:none
$ git -C server/orbit.git config set uploadpack.allowFilter true
```

Two traps. A local path makes `git clone` copy files, so the filter is ignored; use a `file://` URL. And a server without `uploadpack.allowFilter` ignores the filter. In both cases you get a warning, a complete clone, and a repository that is still configured as partial.

Treeless.

```
$ git clone --filter=tree:0 "file://$PWD/server/orbit.git" treeless
Cloning into 'treeless'...
$ git -C treeless cat-file --batch-all-objects --batch-check='%(objecttype)' 2>/dev/null | sort | uniq -c
  33 blob
   96 commit
    5 tag
   20 tree
```

```
$ git -C treeless rev-list --count --all
96
# A path-limited log needs the trees of every commit, and asks for them one commit at a time:
$ GIT_TRACE=1 git -C treeless log --oneline -- services/ranker 2>&1 >/dev/null | grep -c 'run_command: git .*fetch'
93
$ ls treeless/.git/objects/pack/*.pack | wc -l
96
```

Only the 20 trees of the checkout are present, so a walk that looks at paths must fetch trees: one `git log -- services/ranker` sent 93 requests and left 96 packs behind. The GitHub post cited in 26.11 recommends blobless clones for developers and treeless clones only for builds that need the commit history and nothing else.

State table.

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git clone --filter=blob:none <url></code>	Full checkout	Full	Default branch	Created	Promisor configuration; .promisor packs; blobs outside the checkout absent	unchanged	unchanged
Any command that needs a missing object	As that command	As that command	As that command	As that command	One more promisor pack	Read only	unchanged
<code>git backfill</code>	unchanged	unchanged	unchanged	unchanged	Promisor packs with the missing blobs	Read only	unchanged

GitHub, not Git. Whether a filter is honoured is the server's decision. GitHub supports partial clone, and its engineers wrote the guidance cited above. A self-hosted server needs `uploadpack.allowFilter`.

26.13 --single-branch, and choosing a clone

`--single-branch` narrows the refspec to one branch (Chapter 12, section 12.3). It limits history by reachability, not by depth:

```
$ git clone "file://$PWD/server/orbit.git" full
Cloning into 'full'...
$ git -C full cat-file --batch-all-objects --batch-check='%<objecttype>' | sort | uniq -c
122 blob
96 commit
5 tag
284 tree
$ git -C full for-each-ref --format='%<refname>' | wc -l
9
```

```
$ git clone --single-branch "file://$PWD/server/orbit.git" single
Cloning into 'single'...
$ git -C single cat-file --batch-all-objects --batch-check='%<objecttype>' | sort | uniq -c
120 blob
94 commit
5 tag
278 tree
$ git -C single branch -r
origin/HEAD -> origin/main
origin/main
```

The saving is the two commits of the other branch with their trees and blobs. It pays off when a repository has long-lived branches with large unshared histories, and hardly at all otherwise.

The five shapes of `orbit`, from the transcripts of this chapter:

Clone	Commits	Trees	Blobs	Tags	History questions	Works offline
Full	96	284	122	5	Right	Yes
<code>--single-branch</code>	94	278	120	5	Right, for that branch	Yes
<code>--depth 1</code>	1	20	33	1	Wrong or failing, silently	Yes, for the snapshot
<code>--filter=blob:none</code>	96	284	33	5	Right; contents fetched on demand	Only for what was fetched
<code>--filter=tree:0</code>	96	20	33	5	Right; trees and contents fetched on demand	Only for what was fetched

Situation	Choice
Developer on a large repository	Blobless, with sparse-checkout if the tree is wide (Chapter 24)
CI job that builds one snapshot and is discarded	Shallow
CI job that computes versions, changelogs or affected projects	Blobless, or full depth

Situation	Choice
Mirror, backup, server	Full
Machine without a route to the server	Full, or a bundle (26.14)

26.14 Bundles

In one sentence. A bundle is a file that holds a pack and the refs that go with it, and an incremental bundle is one that names commits the receiver must already have.

Precisely. Chapter 12, section 12.13 and Chapter 13, section 13.14 created and cloned bundles. Two things remain. A bundle made from a range, `git bundle create <file> <old>..<new>` , contains only the objects that the range adds, and records `<old>` as a **prerequisite**: `git bundle verify` and every fetch from the bundle refuse a repository that lacks it. And `git clone --bundle-uri=<uri>` takes the bulk of a clone from a bundle, for example from a CDN, and only the remainder from the server.

See it.

```
$ git bundle create ../orbit-full.bundle --all
$ git bundle verify ../orbit-full.bundle | tail -2
../orbit-full.bundle is okay
The bundle records a complete history.
The bundle uses this hash algorithm: sha1
# Two more commits and a release tag, then a bundle of only what is new since the last one:
$ git log --oneline schemas/v1.1.0..main
6d3aa59 gateway: retry the upstream twice
aa0428a gateway: lower the upstream timeout to 600 ms
$ git bundle create ../orbit-update.bundle schemas/v1.1.0..main gateway/v1.2.0
$ git bundle verify ../orbit-update.bundle
../orbit-update.bundle is okay
The bundle contains these 2 refs:
6d3aa5958cd87cc93b3d598f9d92f1c1289f1204 refs/heads/main
b796e9ea912be02a0f9a0e2471070693a83a6efa refs/tags/gateway/v1.2.0
The bundle requires this ref:
100bb993157cac87afe81008782cd27cd39f8c9d
The bundle uses this hash algorithm: sha1
```

Inside the file.

```
# A bundle is a short text header followed by a pack. The header of the incremental bundle:
$ head -n 4 ../orbit-update.bundle
# v2 git bundle
-100bb993157cac87afe81008782cd27cd39f8c9d schemas, gateway, ingest: add the tenant field in one change
6d3aa5958cd87cc93b3d598f9d92f1c1289f1204 refs/heads/main
b796e9ea912be02a0f9a0e2471070693a83a6efa refs/tags/gateway/v1.2.0
$ head -n 2 ../orbit-full.bundle | cut -c1-70
# v2 git bundle
88222b7043b92ba564af0aefc4acd25af9ef15e2 refs/heads/feature/rerank-cac
```

A signature line, one line per prerequisite starting with `-`, one line per ref, an empty line, and then a pack ([gitformat-bundle](#)). The prerequisite is the commit `100bb99`, which was the tip when the full bundle was made.

```
$ cd ..
# A repository that was cloned from the full bundle has the prerequisite commit:
$ git clone -q orbit-full.bundle site-b
$ git -C site-b bundle verify --quiet ../orbit-update.bundle
../orbit-update.bundle is okay
[exit status: 0]
$ git -C site-b fetch ../orbit-update.bundle 'refs/heads/*:refs/remotes/origin/*' 'refs/tags/*:refs/tags/*'
From ../orbit-update.bundle
   100bb99..6d3aa59  main       -> origin/main
   * [new tag]       gateway/v1.2.0 -> gateway/v1.2.0
$ git -C site-b merge --ff-only origin/main
Updating 100bb99..6d3aa59
Fast-forward
 services/gateway/config.yaml | 3 +-
 1 file changed, 2 insertions(+), 1 deletion(-)
# An empty repository does not:
$ git init -q empty
$ git -C empty bundle verify ../orbit-update.bundle
error: Repository lacks these prerequisite commits:
error: 100bb993157cac87afe81008782cd27cd39f8c9d
[exit status: 1]
$ git clone orbit-update.bundle from-update
Cloning into 'from-update'...
error: Repository lacks these prerequisite commits:
error: 100bb993157cac87afe81008782cd27cd39f8c9d
fatal: remote transport reported error
[exit status: 128]
```

The site that has 100bb99 verifies and fetches the update; the empty repository is refused twice with the name of the missing commit. An incremental bundle is thin on purpose, and its deltas may be based on objects it does not contain.

```
# A clone that takes the bulk from a bundle and only the rest from the server:
$ GIT_TRACE_PACKET="$PWD/seeded.trace" git clone -q --bundle-uri="$PWD/orbit-full.bundle" "file://$PWD/server/orbit.git" seeded
$ git -C seeded for-each-ref --format='%(refname)' | sed 's,^\(refs/[a-z]*\)/.*,\1,' | sort | uniq -c
  7 refs/bundles
  1 refs/heads
  3 refs/remotes
  6 refs/tags
# The request to the server offers what the bundle brought:
$ sed -n 's/.*packet: *clone> //p' seeded.trace | grep -c '^have'
5
$ sed -n 's/.*packet: *clone> //p' seeded.trace | grep -c '^want'
3
$ git -C seeded log --oneline -3
6d3aa59 gateway: retry the upstream twice
aa0428a gateway: lower the upstream timeout to 600 ms
100bb99 schemas, gateway, ingest: add the tenant field in one change
```

The seeded clone first unpacked the bundle, kept its refs in a namespace of their own, and then told the server what it had: five `have` lines and three `want` lines, where the plain clone of 26.10 sent eight wants and no have. On this machine the namespace is `refs/bundles/`; the manual of `git clone` writes `refs/bundle/`. Servers can advertise bundle URLs themselves, and clients ignore that unless `transfer.bundleURI` is set ([bundle-uri](#)).

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git bundle create <file> <range></code>	unchanged	unchanged	unchanged	unchanged	unchanged; a file is written outside	unchanged	unchanged
<code>git fetch <bundle> <refspec></code>	unchanged	unchanged	unchanged	unchanged	Objects added; the refs named by the refspec updated	unchanged	unchanged

In production. For question 4 of section 26.1: one full bundle to start, then an incremental bundle per week whose basis is a tag that records what was sent last (the pattern in the `EXAMPLES` of `git help bundle`). Keep the bundles in order; Lab 18.3 loses one and recovers. A bundle carries no reflogs, hooks or configuration (Chapter 13).

26.15 When each feature matters

Feature	Matters when	Who has reported it (their own figures)
Automatic maintenance	Always; leave it on	Git's default since 2.54
Scheduled maintenance with prefetch	Large repositories on developer machines	Microsoft's Scalar (Introducing Scalar)
Commit-graph with changed-path filters	Long history and path-limited queries	Microsoft (commit-graph post)
Geometric repack, multi-pack-index, bitmaps	Servers and very large clones	GitHub (Scaling monorepo maintenance)
File-system monitor, untracked cache	Hundreds of thousands of tracked files	Canva; Dropbox in 2020 (Dropbox)
Sparse-checkout, sparse index	A wide tree of which each person needs little	Canva; GitHub Blog (Chapter 24)
Partial clone	Much blob content in history	GitHub Blog (26.11)
Shallow clone	Throwaway builds of one snapshot	GitHub Blog (26.11)
Checking what the pack contains	A repository much larger than its content explains	Dropbox in 2026

The last row deserves a paragraph. Dropbox reported in 2026 that its monorepo shrank from 87 GB to 20 GB through repacking, not through deleting history. The cause was the heuristic by which Git chooses delta candidates, which paired translation files of different languages because of the directory layout; the fix was a repack on the server with a larger window and depth ([Dropbox](#)). Section 3.7 of Chapter 3 said that a delta is "between two objects chosen for similarity"; here the choice went wrong at scale. The lesson of both chapters is the same: measure the dimension that is large before choosing the feature.

26.16 What can go wrong

Symptom	Diagnosis	Fix	Prevention
Hundreds of packs or thousands of loose objects; everything slow	<code>git count-objects -v</code> ; <code>git maintenance is-needed --auto</code> exits 0	<code>git maintenance run</code>	Leave <code>maintenance.auto</code> on, or schedule maintenance
Maintenance "runs" and nothing changes	<code>git maintenance run --no-quiet</code> warns that a lock file exists; <code>.git/objects/maintenance.lock</code>	Check that no maintenance process runs for the repository, then remove the lock (Lab 16.3)	Do not kill maintenance; do not run <code>git gc</code> beside it

Symptom	Diagnosis	Fix	Prevention
History commands fail with invalid parent position or give answers that differ between clones	<code>git commit-graph verify</code> ; the same command with <code>-c core.commitGraph=false</code>	Delete <code>objects/info/commit-graph</code> or <code>commit-graphs/</code> and write it again (Lab 16.4)	Treat derived files as disposable; never copy them between repositories by hand
<code>git log --<path></code> slow although a commit-graph exists	The statistics line shows <code>filter_not_present</code> above zero, or does not appear	<code>git commit-graph write --reachable --changed-paths</code>	<code>commitGraph.changedPaths=true</code>
Versions, blame or changelogs wrong in CI	<code>git rev-parse --is-shallow-repository</code>	<code>git fetch --unshallow</code> , or a blobless clone	Shallow only for snapshot jobs
could not fetch <id> from promisor remote	Partial clone; server unreachable or the object deleted there	Restore access; <code>git backfill</code>	Backfill before going offline; keep the promisor remote alive
<code>git blame</code> or <code>git log -p</code> crawls in a partial clone	The trace shows one fetch per blob	<code>git backfill</code> , then <code>git maintenance run</code>	Backfill what you work on
A "partial" clone is as large as a full one	The clone printed "filtering not recognized by server" or "--filter is ignored in local clones"	Clone again from a server that allows filters, by URL	Read the output of <code>git clone</code>
Repository lacks these prerequisite commits	An incremental bundle arrived without its predecessor	Apply the missing bundle, or cut a new one from what the site has (Lab 18.3)	Number bundles; record the basis in a tag
The repository stays large after history was cleaned	<code>git fsck --unreachable; .mtimes</code> files in <code>objects/pack</code>	The purge of Chapter 21B	Know that geometric maintenance postpones deletion

26.17 When not to use it, and dangerous edge cases

- **Do not tune a small repository.** Every feature here is configuration that the next person must understand. Automatic maintenance is enough until a counter of section 26.2 is large.
- `git gc --aggressive` recomputes every delta and rarely pays for itself; `git gc --prune=now` and `git prune` remove the grace period that protects running commands and your own mistakes (Chapter 13).
- `git maintenance start` and scalar **change your machine, not only the repository:** scheduler entries, global configuration and, for `scalar`, a daemon. Remove them with `git maintenance unregister`, `git maintenance stop` and `scalar unregister` when a repository goes away.
- **A shallow clone gives wrong answers with exit status 0.** It is safe only where nobody asks it about history.
- **A partial clone is not a backup.** It cannot be completed without its promisor remote. If the server loses the repository, or an object is removed there by a history rewrite, the promised objects are gone.
- **Not every combination works.** `git repack --filter` cannot write a bitmap (`git help repack`), and `--bundle-uri` is incompatible with `--depth` (`git help clone`). Read the manual of the version you run before combining transfer options.
- **Derived files can be wrong.** A damaged commit-graph changed the answer of `git rev-list --count` in Lab 16.4, and Git does not verify its checksum on every read. When two clones disagree about history, bypass the derived data first.
- `fetch.unpackLimit=1` **without maintenance** produces one pack per fetch for ever. The settings of `scalar` are a set; copying one of them is not a tuning.

26.18 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git count-objects -v</code> , <code>git maintenance is-needed</code> , <code>git commit-graph verify</code> , <code>git multi-pack-index verify</code> , <code>git bundle verify</code> , <code>git fsmonitor--daemon status</code>	●	nothing	not needed	not needed
<code>GIT_TRACE</code> , <code>GIT_TRACE_PACKET</code> , <code>GIT_TRACE2_PERF</code>	●	nothing; a trace file if a path is given	not needed	not needed
<code>git commit-graph write</code> , <code>git multi-pack-index write</code>	●	Derived files	not needed	Delete the file
<code>git maintenance run</code>	●	Packs, <code>packed-refs</code> , <code>commit-graph</code> , <code>multi-pack-index</code> ; expires relog entries by the configured periods	<code>git maintenance is-needed</code> ; <code>GIT_TRACE=1</code>	None for expired relog entries
<code>git maintenance run --task=gc</code> , <code>git gc</code>	●	The same, in one pack; deletes unreachable objects past the grace period	<code>git count-objects -v</code> ; <code>git prune -n</code>	None for what was deleted

Command	Label	What it changes	Preview	Recovery
<code>git gc --prune=now</code>		Deletes all unreachable objects	<code>git fsck --unreachable</code>	None in this repository
<code>git maintenance start, register</code>		Global configuration and the system scheduler	<code>git help maintenance</code>	<code>git maintenance stop, unregister</code>
<code>git clone --depth, --filter, --single-branch, --bundle-uri</code>		Creates a repository	not needed	Delete the directory
<code>git fetch --deepen, --unshallow, git backfill</code>		Adds objects; moves or removes the shallow boundary	not needed	not needed
<code>git bundle create</code>		Writes a file	<code>git bundle verify, list-heads</code>	Delete the file
<code>git repack --geometric=2 -d</code>		Rewrites packs	<code>git count-objects -v</code>	Not needed for reachable objects
<code>git repack -a -d</code>		Rewrites everything reachable into one pack and deletes the packs it replaces. What it can destroy: the unreachable objects in those packs, which are what <code>git fsck</code> recovery depends on (Chapter 3, section 3.17)	<code>git fsck --unreachable; git count-objects -v</code>	Another clone or a backup only. Appropriate after a deliberate history rewrite, in a repository you have copied first

26.19 Version notes

Version note. Older behavior: porcelain commands ran `git gc --auto`, triggered by about 6,700 loose objects or 50 packs. Current behavior: they run `git maintenance run --auto` with the geometric strategy, and `git gc` is not involved. Since: `git maintenance` in Git 2.29, the geometric strategy in 2.52, the default in 2.54 ([maintenance configuration](#)). Recommended: nothing to configure. Read "automatic gc" in older texts, and in `git help gc` itself, as "automatic maintenance".

Version note. `git maintenance is-needed` needs Git 2.53, `git backfill` 2.49 (experimental), `commitGraph.changedPaths` 2.52, `--bundle-uri` 2.38. The `commit-graph` dates from Git 2.18. Sources: the release table of the Phase 0 report, section 1, and the posts cited above.

Version note. Git 2.56 adds `git repack --drop-filtered` and makes path-walk repacking work with bitmaps ([release notes](#); [Highlights from Git 2.56](#)). Not run here.

Outdated advice. "Run `git gc` regularly" and "use `--depth 1` to make clones fast" both predate the features of this chapter. The first is now automatic and incremental. The second trades correctness for speed where a blobless clone would give both.

Unverified. Whether `gc.auto=0` suppresses all automatic maintenance under the geometric strategy was not tested for this course. `maintenance.auto=false` is the documented switch and the one the transcripts use.

26.20 Practice

Labs 16.3 and 16.4 in `lab-manual/m16-maintenance.md`: trace a maintenance run and diagnose one that silently does nothing; write, verify, damage and rebuild a commit-graph. Labs 18.1 and 18.3 in `lab-manual/m18-transfer-scale.md`: one repository cloned three ways, and a bundle round trip with a lost update. Replay any transcript with `labs/run ch26/<demo>`. A drill: in `ch26/partial-clone/dev`, predict how many objects `git rev-list --objects --all --missing=print` still reports after `git backfill feature/rerank-cache`, then check.

26.21 Interview questions

1. `git status` is slow in one repository and `git log -- path` in another. How do you find out what each command is doing, without a stopwatch?
2. What does Git 2.55 run when a commit triggers automatic maintenance? How does that differ from `git gc`, and what does each delete?
3. Explain geometric repacking with three packs of 500, 30 and 30 objects. What happens at the next run, and what is never rewritten?
4. What is a cruft pack, and why can a repository under the default strategy hold unreachable objects for a long time?

5. What two kinds of data does a commit-graph hold, and which commands does each help? What happens when the file is stale, and when it is damaged?
6. Why are bitmaps a server feature? What does the counter `pack-reused` tell you?
7. Describe a fetch as a conversation. Where do `--depth` and `--filter` enter it?
8. A CI job reports the wrong version and an empty changelog, with exit status 0. Diagnose it.
9. Compare shallow, blobless and treeless clones: what each holds, what each answers correctly, and what each costs later.
10. A partial clone fails with `could not fetch ... from promisor remote`. What are the two possible causes, and how do you make a laptop independent of the server for a week?
11. How do you ship weekly updates to a site with no network route to the server? What goes wrong if one shipment is lost?
12. A consultant recommends `git gc --aggressive` every night. What do you answer?

26.22 Sources

Primary sources. The local manual of Git 2.55.0: `git help maintenance`, `gc`, `repack`, `commit-graph`, `multi-pack-index`, `fsmonitor--daemon`, `update-index`, `clone`, `fetch`, `rev-list`, `backfill`, `bundle`, `config`. Online: [git-maintenance](#), [git-gc](#), [git-repack](#), [git-commit-graph](#), [gitformat-commit-graph](#), [git-multi-pack-index](#), [bitmap-format](#), [gitformat-pack](#), [git-fsmonitor--daemon](#), [shallow](#), [partial-clone](#), [git-backfill](#), [git-bundle](#), [bundle-uri](#), [gitprotocol-v2](#), [api-trace2](#). Git source at the 2.55.0 tag: [builtin/gc.c](#) (strategies, tasks and thresholds) and [odb/source-loose.c](#) (the loose-object estimate).

Secondary sources. Derrick Stolee, Git's database internals, GitHub Blog, 2022: [I, the packed object store](#), [II, commit history queries](#), [III, file history queries](#), [IV, distributed synchronization](#). [Get up to speed with partial clone and shallow clone](#) (2020). Taylor Blau, [Scaling monorepo maintenance](#) (2021). Jeff Hostetler, [the file system monitor](#) (2022). The practitioner reports of section 26.15: each from the publisher's own environment and date, not comparable with one another. The Phase 0 report of this course, sections 1, 12 and 13.

Videos (optional; assessments from the Phase 0 report). Derrick Stolee, ["Git Internals: a Database Perspective"](#) (Git Merge 2022, 27 min): pack indexes, the multi-pack-index, bitmaps, `git maintenance start`; current, and the best compact talk on this layer. Derrick Stolee, ["Git at Scale for Everyone"](#) (32 min, 2020): partial clone and background maintenance; the Scalar packaging shown is the one of 2020.

Further reading. Chapter 3, section 3.7, for packs and deltas; Chapter 13, sections 13.4 and 13.13, for retention; Chapter 24 for the working-tree side; Chapter 29 for traces in diagnosis.

Chapter 27: Open source and team workflows

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch27/`.

27.1 Why this matters

Four questions a CTO can ask, none of which is about a command:

1. "We ship one hosted version today. Next quarter two enterprise customers will run our gateway on their own hardware and stay a version behind. Does our branching model survive that?"
2. "A hotfix went out as 1.4.1. Why did 1.5.0 ship the same bug again?"
3. "Somebody on the team says the research proves trunk-based development is best. Does it?"
4. "Our pull request to an open-source dependency has been ignored for three weeks. What did we do wrong?"

Every answer in this chapter comes from two facts you already have. A branch is a ref that names a commit (Chapter 7: Branches), so a "branching strategy" is nothing but an agreement about which refs exist, who may move them, and in which direction commits travel between them. And a release is a tag that names a commit (Chapter 14B: Configuration, tags and signing), so "what did we ship" is always a question you can put to the commit graph.

The second question has a precise answer: the fix was committed on the release branch and never reached `main`, and nobody ran the one-line check that would have listed it (section 27.10). The third: no, and section 27.13 says what the research does show. The first and the fourth are decisions, and the chapter gives you the vocabulary and the evidence for them.

The demos use one project, `promptgate`: a gateway between product teams and LLM providers that routes requests to a model, enforces per-tenant limits and streams answers back.

A note on layers. Branches, tags, merges and cherry-picks are Git. Forks, pull requests, merge queues, rulesets and releases are GitHub. A workflow is a convention that people hold.

27.2 The fork workflow, end to end

In one sentence. In the fork workflow you never push to the project's repository: you push to a server-side copy that you own, and you ask the project to take your commits through a pull request.

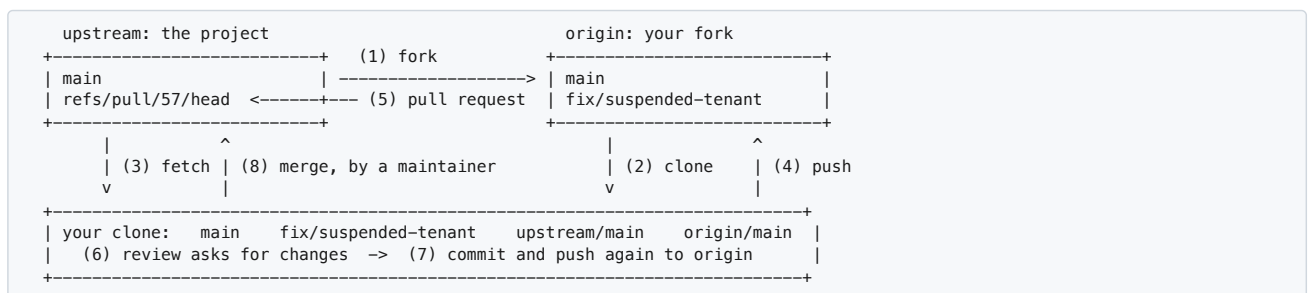
Analogy. A journal submission. You do not edit the journal; you send a manuscript, reviewers ask for changes, you send a revision, and an editor decides. The analogy breaks at storage: on GitHub your copy and the journal's share one object store (a fork network), so what you push to your fork can be reached from the upstream repository by commit ID, as Chapter 17: Pull requests, section 17.15 documents.

Precisely. Three repositories are involved and each has its own refs:

Repository	Remote name in your clone	You can	You use it to
The project (upstream)	<code>upstream</code>	fetch	learn what the project's <code>main</code> is now
Your fork	<code>origin</code>	fetch and push	publish branches for pull requests
Your clone	none: it is the local repository	everything	do the work

The mechanics of two remotes, `remote.pushDefault` and `@{push}` were built in Chapter 12: Remote operations, section 12.10, and the platform side (the `refs/pull/N/head` ref, what a maintainer can push to, what a workflow run from a fork may see) in Chapter 17, section 17.15.

Picture.



The ten steps. Steps 1, 5, 6 and 8 happen on GitHub and are given from the documentation ([forks reference](#), [GitHub flow](#)); the commands were checked with `gh <command> --help` and are not run here.

```
# 1. Fork on GitHub, and 2. clone the fork. gh does both, and names the remotes origin and upstream.
gh repo fork example-org/promptgate --clone
cd promptgate

# 3. Start from the project's current main, not from your fork's copy of it.
git fetch upstream
git switch -c fix/suspended-tenant upstream/main

# ... edit, test ...
git commit -am "Fix limit 0 being treated as unlimited"

# 4. Publish the branch on your fork.
git push -u origin fix/suspended-tenant

# 5. Open the pull request against the upstream repository.
gh pr create --repo example-org/promptgate --base main --fill

# 6 and 7. Review asks for changes: commit and push again. The pull request follows the branch.
git commit -am "Test that a limit of 0 blocks the tenant"
git push

# 8. A maintainer merges. 9. Bring main up to date. 10. Delete the branch in both places.
```

See it. No network is available in the lab, so two bare repositories play the two servers: `upstream/promptgate.git` is the project, maintained by Asha, and `fork/promptgate.git` is your fork. `git clone --bare` is used in the fixture where GitHub would fork.

```
# fork/promptgate.git plays your fork on the platform. Clone it, then name the upstream:
$ git clone -q fork/promptgate.git you/promptgate
$ cd you/promptgate
$ git remote add upstream ../../upstream/promptgate.git
$ git fetch -q upstream
$ git remote -v
origin    ../../fork/promptgate.git (fetch)
origin    ../../fork/promptgate.git (push)
upstream  ../../upstream/promptgate.git (fetch)
upstream  ../../upstream/promptgate.git (push)
$ git branch -a -vv
* main                4ebb785 [origin/main] Add gateway with tenant limits
remotes/origin/HEAD   -> origin/main
remotes/origin/main   4ebb785 Add gateway with tenant limits
remotes/upstream/HEAD -> upstream/main
remotes/upstream/main 4ebb785 Add gateway with tenant limits
```

Four refs name one commit, `4ebb785`. `origin/main` is your clone's record of the fork and `upstream/main` its record of the project. They will drift apart.

```
$ git switch -c fix/suspended-tenant upstream/main
Switched to a new branch 'fix/suspended-tenant'
branch 'fix/suspended-tenant' set up to track 'upstream/main'.
$ git commit -q -am "Fix limit 0 being treated as unlimited"
$ git push -u origin fix/suspended-tenant
To ../../fork/promptgate.git
 * [new branch]      fix/suspended-tenant -> fix/suspended-tenant
branch 'fix/suspended-tenant' set up to track 'origin/fix/suspended-tenant'.
```

Read the two "set up to track" lines. `git switch -c <branch> upstream/main` made `upstream/main` the upstream of the new branch, which is the right base for comparison. `git push -u origin` then replaced it with `origin/fix/suspended-tenant`, which is the right destination for pushes. After this command `git status` compares you with your fork, not with the project. Compare with the project explicitly: `git log --oneline upstream/main..HEAD`.

Inside .git. Nothing is new. `refs/remotes/upstream/*` and `refs/remotes/origin/*` are two sets of remote-tracking refs filled by two fetch refsspecs in `.git/config`. The pull request does not exist in your repository at all.

Step	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git fetch upstream</code>	unchanged	unchanged	unchanged	unchanged	<code>refs/remotes/upstream/*</code> move; <code>FETCH_HEAD</code> written	unchanged	unchanged
<code>git switch -c fix/... upstream/main</code>	files of <code>upstream/main</code>	matches it	symbolic ref to the new branch	created at <code>upstream/main</code>	<code>branch.<name>.remote</code> and <code>.merge</code> written	unchanged	unchanged
<code>git push -u origin fix/...</code>	unchanged	unchanged	unchanged	unchanged	<code>refs/remotes/origin/fix/...</code> created; upstream configuration rewritten	the fork gains a branch	the fork shows the branch; no pull request yet

Step	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
gh pr create	unchanged	unchanged	unchanged	unchanged	unchanged	unchanged	a pull request object and refs/pull/N/head in upstream

In production. Forks are not only for open source: an organization that employs contractors who must not have write access uses the same triangle internally. The cost is that secrets are withheld from workflows triggered by pull requests from forks, which matters for LLM evaluation jobs (Chapter 28: AI/ML workflows, section 28.11).

27.3 When upstream moves: three ways to sync the fork, two ways to update the branch

In one sentence. Your fork's `main` is a copy that nothing updates for you, and your feature branch is based on a commit that is getting older; bringing each up to date is a separate operation.

Precisely. There are two different things to keep current, and people mix them up:

- **The fork's default branch.** It should be an exact copy of the project's `main`. If you never commit on it, updating it is always a fast-forward.
- **Your feature branch.** It contains your commits on top of an old upstream commit. Updating it means either replaying your commits on the new tip (rebase) or merging the new tip into the branch.

GitHub documents three ways to do the first ([syncing a fork](#)). Two run on the server and are described from the documentation; the third is plain Git and is run below.

Way	Where it runs	What it does	What it leaves for you
The Sync fork control on the fork's page (label from the documentation; the interface changes)	GitHub	updates the fork's branch from upstream	your clone is now behind <code>origin: git pull --ff-only</code>
gh repo sync <owner>/<fork>	GitHub, through the API	fast-forwards the fork's default branch from its parent; <code>--force</code> hard-resets it instead	the same
git fetch upstream, fast-forward, git push origin main	your clone	moves your local <code>main</code> , then the fork's	nothing: the clone is already current

gh repo sync without an argument updates the local repository instead: the destination is the argument (gh repo sync --help).

See it. While your pull request waits, Asha merges somebody else's work into the project:

```
# While your pull request waits for review, the maintainer merges other work:
$ git fetch upstream
From ../..upstream/promptgate
 4ebb785..7de9633 main -> upstream/main
$ git rev-list --left-right --count upstream/main...origin/main
1
0
$ git log --oneline --graph --all
* 7de9633 Stream tokens to the client
| * a83a713 Fix limit 0 being treated as unlimited
|/
* 4ebb785 Add gateway with tenant limits
```

1 0: one commit on the left side (upstream/main) that the right side (origin/main) lacks, and none the other way. That second zero is what makes the sync safe. The way with plain Git:

```
# Bring the main branch of the fork up to date. Your local main is only a relay:
$ git switch -q main
$ git merge --ff-only upstream/main
Updating 4ebb785..7de9633
Fast-forward
 gateway/stream.py | 3 +++
 1 file changed, 3 insertions(+)
 create mode 100644 gateway/stream.py
$ git push origin main
To ../..fork/promptgate.git
 4ebb785..7de9633 main -> main
$ git rev-list --left-right --count upstream/main...origin/main
0
0
```

`--ff-only` is the guard. If you had committed on `main` by mistake, the merge would stop instead of creating a merge commit that exists only in your fork and that every later pull request would carry.

Now the feature branch. **Way 1, rebase:**

```
# Update the feature branch, way 1: replay it on the new upstream tip.
$ git switch -q fix/suspended-tenant
$ git rebase upstream/main
Rebasing (1/1)
Successfully rebased and updated refs/heads/fix/suspended-tenant.
$ git push
To ../../fork/promptgate.git
 ! [rejected]        fix/suspended-tenant -> fix/suspended-tenant (non-fast-forward)
error: failed to push some refs to ' ../../fork/promptgate.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
$ git push --force-with-lease --force-if-includes
To ../../fork/promptgate.git
+ a83a713...3560caf fix/suspended-tenant -> fix/suspended-tenant (forced update)
$ git log --oneline --graph --all
* 3560caf Fix limit 0 being treated as unlimited
* 7de9633 Stream tokens to the client
* 4ebb785 Add gateway with tenant limits
```

The rebase replaced commit `a83a713` with `3560caf`: same change, new parent, new ID (Chapter 9: Rebase). The fork still had the old commit, so the plain push was rejected as a non-fast-forward, and the push that succeeded was a forced one. 🚫 DANGEROUS: `git push --force-with-lease --force-if-includes`. What it changes: the branch on the fork, to a commit that does not descend from the old one. What it can destroy: commits on that server branch that are not in your clone, for example a fix a maintainer pushed to your pull request branch. Preview: `git fetch origin`, then `git log HEAD..origin/fix/suspended-tenant` lists what would be dropped. Recovery: push the old tip back from any clone that still has it; your own old tip is in the branch reflog. When it is appropriate: on a branch that only you push to, after a rebase. The lease makes the push succeed only if the fork's branch is where your clone last saw it, and `--force-if-includes` only if you have integrated what you last saw; Chapter 12, section 12.8 explains both conditions and how a background fetch defeats the first.

Way 2, merge:

```
# Upstream moved again. Way 2: merge upstream into the branch. No commit is rewritten,
# so a plain push is enough:
$ git fetch -q upstream
$ git merge -m "Merge upstream main into fix/suspended-tenant" upstream/main
Merge made by the 'ort' strategy.
 gateway/fallback.py | 5 +++++
 1 file changed, 5 insertions(+)
 create mode 100644 gateway/fallback.py
$ git push
To ../../fork/promptgate.git
 3560caf..44d750c fix/suspended-tenant -> fix/suspended-tenant
$ git log --oneline --graph fix/suspended-tenant
* 44d750c Merge upstream main into fix/suspended-tenant
|\
| * bb1ddd2 Fall back to a second model on timeout
| * | 3560caf Fix limit 0 being treated as unlimited
|/
* 7de9633 Stream tokens to the client
* 4ebb785 Add gateway with tenant limits
```

No commit changed its ID, so the push is an ordinary fast-forward. The price is the merge commit `44d750c` in the middle of your branch.

	Rebase onto upstream/main	Merge upstream/main into the branch
Your commit IDs	change	stay
Push	forced	plain
Branch history	linear, reads as if written today	has "merge upstream" commits
Review comments already made	may lose their anchor; the reviewer needs <code>git range-diff</code> to see what changed (Chapter 9, section 9.14)	stay attached
Conflicts	resolved once per replayed commit	resolved once, in the merge

Which to use is the project's decision: read its contribution guide. Where it is silent, rebase freely until review starts and add commits afterwards, because a reviewer who has read a commit should be able to trust that it has not changed.

The review round, and the squash merge. The reviewer asks for a test. You add a commit, push, and the pull request updates because it follows the branch:

```
# The reviewer asks for a test. Changes after review start are added, not rewritten:
$ mkdir -p tests && printf 'from gateway.limits import LIMITS, allowed\n\n\ndef test_suspended():\n    LIMITS["acme"] = 0\n\nassert not allowed("acme", 0)\n' > tests/test_limits.py
$ git add tests && git commit -q -m "Test that a limit of 0 blocks the tenant"
$ git push
To ../../fork/promptgate.git
```

```

44d750c..147ed67  fix/suspended-tenant -> fix/suspended-tenant
$ git log --oneline upstream/main..HEAD
147ed67 Test that a limit of 0 blocks the tenant
44d750c Merge upstream main into fix/suspended-tenant
3560caf Fix limit 0 being treated as unlimited

```

Then the maintainer squash-merges. The lab imitates the merge button with `git merge --squash` in Asha's clone; GitHub documents the result as one new commit on the base branch ([merge methods](#)).

```

# The maintainer squash-merges the pull request. Upstream main has one new commit:
$ git fetch upstream
From ../..upstream/promptgate
bb1ddd2..16e9e75  main      -> upstream/main
$ git log --oneline -2 upstream/main
16e9e75 Fix limit 0 being treated as unlimited (#57)
bb1ddd2 Fall back to a second model on timeout
$ git switch -q main && git merge -q --ff-only upstream/main && git push -q origin main
# Your commits are not ancestors of main: the squash made one new commit with a new ID.
$ git merge-base --is-ancestor fix/suspended-tenant main
[exit status: 1]
# Compare content instead. No output means main has everything the branch has:
$ git diff --stat main fix/suspended-tenant
$ git branch -d fix/suspended-tenant
warning: deleting branch 'fix/suspended-tenant' that has been merged to
        'refs/remotes/origin/fix/suspended-tenant', but not yet merged to HEAD
Deleted branch fix/suspended-tenant (was 147ed67).
$ git push origin --delete fix/suspended-tenant
To ../..fork/promptgate.git
- [deleted]          fix/suspended-tenant

```

```

Observed behavior : after a squash merge, "is my branch merged?" gets three different answers.
Git state         : upstream main has one new commit, 16e9e75. Your branch tip is 147ed67.
                   147ed67 is not an ancestor of 16e9e75.
Mechanism         : "merged" in Git means "reachable from". git merge-base --is-ancestor and
                   git branch --merged both test reachability, and a squash creates a commit
                   with one parent and no link to your commits.
Root cause        : the merge method discarded the ancestry and kept the content.
Why Git does this : a commit's parents are part of its ID. Git cannot record "these commits were
                   folded into that one" without a second parent, and a second parent is
                   exactly what a squash omits.
Correct fix       : test content, not ancestry: an empty "git diff main <branch>" here, or the
                   merge-tree comparison of Lab 34.1 when main has moved further.
Prevention        : delete the branch right after the merge, on the fork and locally, and never
                   reuse it. Chapter 17, section 17.12 shows what reuse does to the next pull request.

```

One line of that transcript contradicts a rule you may have read: `git branch -d` deleted a branch that was not merged into `main`, with a warning. The manual explains it: `-d` requires the branch to be "fully merged in its upstream branch, or in `HEAD` if no upstream was set". The upstream here is `origin/fix/suspended-tenant`, which has every commit. On a branch that was never pushed with `-u`, the same command refuses, and that is the case older tutorials describe.

27.4 Etiquette, and the maintainer's side

In one sentence. A maintainer's scarce resource is attention, and every convention of open-source contribution is a way of spending less of it per accepted change.

Before you write code.

- **Read the contribution guide.** GitHub shows a repository's `CONTRIBUTING` file when someone opens an issue or pull request, and since 7 August 2025 in the repository's tab bar and sidebar ([setting guidelines for contributors](#), [changelog](#)). It tells you the branch to target, the merge method, the test command, the commit message convention and whether a sign-off or a contributor agreement is required. Its templates are part of the same contract ([templates](#)).
- **Issue first for anything large.** A pull request nobody asked for forces the maintainer to choose between reviewing work that does not fit the roadmap and rejecting work somebody spent a week on. An issue costs both sides ten minutes.

When you open the pull request.

- **One logical change.** Martin Fowler's catalogue of branching patterns notes that open-source maintainers integrate work from occasional contributors they do not yet know, which is why pre-merge review of a self-contained branch fits open source so well ([Patterns for Managing Source Code Branches](#)). A small, self-contained change is one a stranger can evaluate.
- **Say what you tested.** For an ML change: which evaluation set, which metric, which commit.
- **Leave "allow edits by maintainers" on** for a fork you own; it lets a maintainer push a small fix to your branch instead of asking for another round (Chapter 17, section 17.15).
- **Expect checks to behave differently.** Workflows on a pull request from a fork run without the project's secrets, and in a public repository the default is that first-time contributors need a maintainer's approval before workflows run ([Actions settings](#)). A red check that needs a secret is not your bug; say so in the pull request instead of pushing guesses.

While it is in review.

- Answer every comment, and do not force-push over a review in progress unless the project asks for it.
- Do not open a second pull request because the first is quiet. Ask once, after a reasonable wait, in the place the project names.

The maintainer's side. The same economics apply in reverse, and most of the tools are platform features covered elsewhere in this book:

Maintainer need	Tool	Where
Tell contributors the rules once	CONTRIBUTING.md , templates, default community health files	this section
Never run untrusted code with secrets	<code>pull_request</code> instead of <code>pull_request_target</code> ; approval for outside contributors	Chapter 21A: GitHub Actions security
Route review to the right people	CODEOWNERS	Chapter 19: CODEOWNERS
Keep <code>main</code> releasable	required checks, required review, merge queue	Chapter 18: Branch protection and rulesets
Keep credit when squashing	read the proposed squash message before confirming; who GitHub records as author and co-authors is flagged as unverified in Chapter 17	Chapter 17, section 17.8

27.5 Branch names are conventions, not laws

In one sentence. `main`, `develop`, `feature/*`, `bugfix/*`, `hotfix/*` and `release/*` are names that people agreed on; Git gives none of them a meaning.

Precisely. To Git, `refs/heads/release/1.4` is a ref whose name happens to contain a slash. The slash matters in exactly two technical ways. It lets patterns select groups of branches (`git branch --list 'release/*'`, a ruleset target such as `release/**`, a workflow filter such as `branches: ['release/**']`). And, in the files backend, it makes `release` a directory, so a branch named `release` and a branch named `release/1.4` cannot both exist (Chapter 7: Branches).

Name	Usual meaning	What gives it force
<code>main</code>	the integration branch; the default branch	the repository's default-branch setting; a ruleset
<code>develop</code>	Git Flow's integration branch, with <code>main</code> reserved for released code	only the team's agreement
<code>feature/<topic></code>	work in progress on one change	nothing; often a naming rule
<code>bugfix/<topic></code>	a fix that is not urgent, taking the normal path	nothing
<code>hotfix/<topic></code>	an urgent fix to something already released	nothing in Git; sometimes a faster review rule
<code>release/<version></code>	the line from which one version and its patches are tagged	a ruleset; deployment workflows that trigger on the pattern

The lab configuration sets `init.defaultBranch=main`; unconfigured Git 2.55 still creates `master`. Neither name does anything the other does not.

In production. The convention earns its keep when rules are attached to it. "Nobody pushes to `release/**` except through a pull request with two approvals" is one ruleset with one pattern, and it works only if every release branch is named that way. A name that carries the author (`asha/rerun`) says the branch is private and may be rebased. Choose a scheme, write it in `CONTRIBUTING.md`, and enforce the parts that matter with a ruleset (Chapter 18: Branch protection and rulesets).

27.6 GitHub Flow

In one sentence. One long-lived branch that is always deployable, short branches off it, a pull request for each, and deployment right after the merge.

Precisely. GitHub's documentation describes six steps: create a branch, make changes, create a pull request, address review comments, merge, delete the branch ([GitHub flow](#)). Scott Chacon's 2011 essay, where the name comes from, adds the rules that give it its character: anything on the main branch is deployable, and you deploy immediately after merging; he also gives the reason GitHub did not use Git Flow, which is that GitHub deployed many times a day while Git Flow is organised around releases ([GitHub Flow](#)). Fowler's reading is that the model assumes a single production version, so release branches and hotfix branches disappear: a hotfix is one more short branch ([Fowler](#)).

Picture.




```
| feature/batch-api merged
tag v1.4.0, deploy
```

What it assumes. That there is one live version, that `main` can be deployed at any commit, and that a broken deployment is repaired by rolling forward or back quickly. Microsoft's description of its own practice records where the strict form stops scaling: when a repository completes more than 200 pull requests a day, deploying each one before or after merge turns into a queue, so Microsoft batches deployments into sprint releases instead ([How Microsoft develops with DevOps](#)).

What it does not say. How long a feature branch may live, or what to do when a customer cannot take the newest version. Section 27.9 shows the consequence with a real history.

27.7 Git Flow, and its author's 2020 note

In one sentence. Two long-lived branches, `main` for released code and `develop` for integration, plus three kinds of supporting branch: feature, release and hotfix.

Precisely. Vincent Driessen published "A successful Git branching model" on 5 January 2010. Feature branches start from and merge into `develop`, with `--no-ff` so that each feature stays visible as a unit. A release branch is cut from `develop`, receives only stabilization commits, and is merged into `main` (and tagged) and back into `develop`. A hotfix branch is cut from `main`, and is merged into `main` (and tagged) and into `develop` ([nvie.com](#)).

See it. The same project and the same changes as in the other demos, arranged as Git Flow prescribes:

```
$ git log --oneline --graph --decorate --all
* 24124ab (HEAD -> develop) Merge hotfix/1.4.1 into develop
|\
* \ f2b87da Merge pull request #43 from feature/batch-api
|\ \
| * | b713222 Add batch endpoint
|/ /
* | 989ca76 Merge release/1.4.0 back into develop
|\ \
| | | * a3de55c (tag: v1.4.1, main) Hotfix 1.4.1
| | | |\
| | | |/
| | | |/
| | | |/
| | * | 1607dca Fix limit 0 being treated as unlimited
| | |/
| | * 2a0387c (tag: v1.4.0) Release 1.4.0
| | |\
| | |/
| | |/
| * | 7237206 Bump version to 1.4.0
|/ /
* | 7d9c521 Merge pull request #42 from feature/tenant-limits
|\ \
| * | c3cc79e Read tenant limits with a default
| |/
* | e27adc8 Merge pull request #41 from feature/streaming
|\ \
| |/
|/ |
| * 39a039a Stream tokens to the client
|/
* b6e2f58 (tag: v1.3.0) Add per-tenant rate limits
* 9df3d07 Add gateway skeleton
```

Find the hotfix commit `1607dca`. It has two children: the merge `a3de55c` on `main`, tagged `v1.4.1`, and the merge `24124ab` on `develop`. That double merge is the model's answer to "how does a fix reach both lines", and it is a merge-upward answer (section 27.10).

```
$ git log --oneline --no-merges v1.4.0..v1.4.1
1607dca Fix limit 0 being treated as unlimited
$ git log --oneline --first-parent main
a3de55c Hotfix 1.4.1
2a0387c Release 1.4.0
b6e2f58 Add per-tenant rate limits
9df3d07 Add gateway skeleton
$ git rev-list --count --merges v1.3.0..develop
6
$ git branch --contains v1.4.1^2
* develop
main
```

The patch release contains the fix and nothing else, because `main` receives only releases and hotfixes. `git log --first-parent main` is a release log, one line per release. And the history between `v1.3.0` and `develop` contains six merge commits for three features, one release and one hotfix.

The 2020 note. On 5 March 2020 the author added a "note of reflection" at the top of the article. The report summarizes it this way: the model was conceived for explicitly versioned software that may need several versions supported in the wild; a team doing continuous delivery of a web application should adopt a simpler workflow such as GitHub Flow; and readers should weigh their own context, because no model is a cure-all ([nvie.com](#)). Fowler adds that Git Flow says nothing about how long feature branches live, and Atlassian's tutorial now labels Gitflow a legacy workflow ([Atlassian](#)).

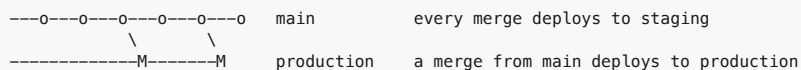
Outdated advice. "Use Git Flow" as a default for every project. The author of the model restricts it to explicitly versioned software with several supported versions. For a continuously deployed service it adds a second long-lived branch and two merges per release, and the tags already record what was released.

When it still fits. Installed software, firmware, SDKs and libraries with several maintained versions, and organizations where a release is a scheduled, audited event. Even there, the part that does the work is the release branch, which you can have without `develop` (section 27.8).

27.8 GitLab Flow, trunk-based development, release branches and Release Flow

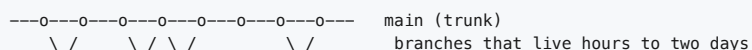
All four keep one integration branch and differ in what, if anything, sits downstream of it.

GitLab Flow. GitLab defines it as a simplified strategy that works directly with `main` and adds, where needed, a production branch, environment branches such as staging and production, or release branches for software shipped in versions ([What is GitLab Flow?](#)). Among its eleven published rules are: fix bugs in `main` first and release branches second; releases are based on tags; pushed commits are never rebased ([best practices](#)).

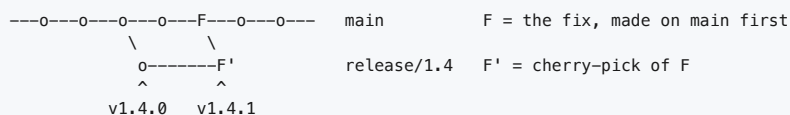


Unverified. GitLab's original 2014 "GitLab Flow" document is no longer reachable on docs.gitlab.com; the phrase "upstream first" for its fix direction was read only through a mirror, according to the research notes. The two pages linked above are current.

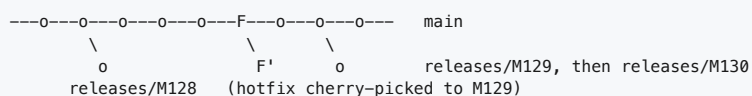
Trunk-based development. Developers integrate into one branch, the trunk, and avoid other long-lived development branches. Very small teams may commit straight to the trunk; larger teams use short-lived feature branches, which should last no more than a couple of days, belong to one developer or a pair, and be deleted after the merge; long-running changes are handled with feature flags and branch by abstraction instead of long branches ([trunkbaseddevelopment.com](#), [short-lived feature branches](#)). Fowler treats it as close to a synonym for continuous integration. The difference from GitHub Flow is one of degree, not of kind: both have one long-lived branch, and trunk-based development adds a limit on branch lifetime and, in its strictest form, drops the pre-merge review gate.



Release branches. A release branch is cut from the trunk shortly before a release, receives only fixes, and is the place the release and its patches are tagged. The trunk-based development site states the rules: cut late; the branch may start from a commit older than the trunk's head; it is never merged back; it is deleted when the version is no longer supported; bugs are reproduced and fixed on the trunk first and cherry-picked to the release branch ([branch for release](#)).



Microsoft's Release Flow. Short-lived topic branches are merged to `main` through pull requests with branch policies. At the end of each three-week sprint a release branch is created, named for the sprint (the document's example is `releases/M129`), and deployed in rings. A hotfix is made in `main` first and then cherry-picked to the release branch through its own pull request. Release branches never merge back to `main`, and the old one is abandoned after the next sprint ships ([How Microsoft develops with DevOps](#)).



Release Flow is trunk-based development plus a release branch per sprint plus the "fix on main first" rule. The names differ more than the graphs do.

27.9 One release and one hotfix, under two strategies

In one sentence. The same three features and the same one-line fix produce two different answers to "what does the patch release contain", and the difference is where the release tag lives.

The scenario: `promptgate` 1.3.0 is released. Two pull requests are merged, and 1.4.0 is released. A third pull request, a batch endpoint, is merged afterwards. Then production reports that suspended tenants, whose limit is 0, are not blocked, and a fix must ship as 1.4.1.

Strategy A: GitHub Flow, releases are tags on `main`.

```
# Production reports that suspended tenants (limit 0) are not blocked.
$ git switch -c hotfix/suspended-tenant main
Switched to a new branch 'hotfix/suspended-tenant'
$ git diff
diff --git a/gateway/limits.py b/gateway/limits.py
index bf5d3ec..18f6e2c 100644
--- a/gateway/limits.py
+++ b/gateway/limits.py
@@ -2,5 +2,5 @@ LIMITS = {"default": 60}

def allowed(tenant, used):
-    limit = LIMITS.get(tenant) or LIMITS["default"]
+    limit = LIMITS.get(tenant, LIMITS["default"])
    return used < limit
$ git commit -q -am "Fix limit 0 being treated as unlimited"
$ git switch -q main
$ git merge --no-ff -m "Merge pull request #44 from hotfix/suspended-tenant" hotfix/suspended-tenant
Merge made by the 'ort' strategy.
 gateway/limits.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git branch -d hotfix/suspended-tenant
Deleted branch hotfix/suspended-tenant (was 9a75273).
$ git tag -a v1.4.1 -m "promptgate 1.4.1"
```

The bug is the `or`: a limit of `0` is falsy in Python, so the suspended tenant silently got the default limit. The fix is one line. Now the history, and the question that matters:

```
$ git log --oneline --graph --decorate
* c1702e7 (HEAD -> main, tag: v1.4.1) Merge pull request #44 from hotfix/suspended-tenant
|\
| * 9a75273 Fix limit 0 being treated as unlimited
|/
* a1351be Merge pull request #43 from feature/batch-api
|\
| * 5928b76 Add batch endpoint
|/
* 518d961 (tag: v1.4.0) Merge pull request #42 from feature/tenant-limits
|\
| * 4806775 Read tenant limits with a default
* | a7ccb4b Merge pull request #41 from feature/streaming
|\ \
| | /
| | /
| * b00109d Stream tokens to the client
|/
* b6e2f58 (tag: v1.3.0) Add per-tenant rate limits
* 9df3d07 Add gateway skeleton
```

```
# What does a customer get when moving from v1.4.0 to v1.4.1?
$ git log --oneline --no-merges v1.4.0..v1.4.1
9a75273 Fix limit 0 being treated as unlimited
5928b76 Add batch endpoint
$ git diff --stat v1.4.0 v1.4.1
 gateway/batch.py | 2 ++
 gateway/limits.py | 2 +-
 2 files changed, 3 insertions(+), 1 deletion(-)
$ git branch --list
* main
```

Version 1.4.1 contains the fix **and the batch endpoint**. Under GitHub Flow that is by design: `main` is what you ship, and everything merged since the last tag ships with the next one. For a hosted service with one live version this is fine, and usually desirable. For a customer who pinned 1.4 and expects a patch release to contain only fixes, it is a breach of what the version number promised ([Semantic Versioning](#) reserves the third number for backward-compatible bug fixes).

Strategy B: a release branch, fix on `main` first, cherry-pick down.

```
# Then the one commit is copied to the release branch and released from there:
$ git switch -q release/1.4
$ git cherry-pick -x bc59804
[release/1.4 996796d] Fix limit 0 being treated as unlimited
Date: Mon Sep 7 10:24:00 2026 +0530
 1 file changed, 1 insertion(+), 1 deletion(-)
```

```
$ git tag -a v1.4.1 -m "promptgate 1.4.1"
$ git log -1 --format=%B
Fix limit 0 being treated as unlimited
(cherry picked from commit bc5980493f22fc60a53dcc923eb9941b88e13284)
```

⚠ CAUTION: `git cherry-pick -x <commit>` creates a new commit on the current branch with the same change as `<commit>` and moves the branch to it. The `-x` adds the "(cherry picked from commit ...)" line, which is the only durable record that 996796d on the release branch and bc59804 on main are the same fix. Chapter 10: Cherry-pick explains the three-way merge behind the command.

```
$ git log --oneline --graph --decorate --all
* 996796d (HEAD -> release/1.4, tag: v1.4.1) Fix limit 0 being treated as unlimited
| * 956fbb2 (main) Merge pull request #44 from hotfix/suspended-tenant
| | \
| | * bc59804 Fix limit 0 being treated as unlimited
| | /
| * e6db0b4 Merge pull request #43 from feature/batch-api
| /
| /
| * 7a47553 Add batch endpoint
| /
| * 518d961 (tag: v1.4.0) Merge pull request #42 from feature/tenant-limits
| \
| * 4806775 Read tenant limits with a default
| * | a7ccb4b Merge pull request #41 from feature/streaming
| \ \
| | /
| | /
| * b00109d Stream tokens to the client
| /
| * b6e2f58 (tag: v1.3.0) Add per-tenant rate limits
| * 9df3d07 Add gateway skeleton
```

```
# What does a customer get when moving from v1.4.0 to v1.4.1?
$ git log --oneline --no-merges v1.4.0..v1.4.1
996796d Fix limit 0 being treated as unlimited
$ git diff --stat v1.4.0 v1.4.1
 gateway/limits.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
```

Version 1.4.1 contains the fix and nothing else. The batch endpoint will ship in 1.5.0.

Inside .git. Strategy A created two tag objects and moved `refs/heads/main`. Strategy B created the same two tags, one extra ref `refs/heads/release/1.4`, and one extra commit object. A "release" in both is `refs/tags/v1.4.1` naming an annotated tag object that names a commit.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git switch -c release/1.4 main</code>	unchanged	unchanged	symbolic ref to <code>release/1.4</code>	new ref at the commit of <code>main</code>	reflog for the new branch	unchanged until pushed	unchanged until pushed
<code>git tag -a v1.4.0 -m ...</code>	unchanged	unchanged	unchanged	unchanged	a tag object and <code>refs/tags/v1.4.0</code>	unchanged until <code>git push origin v1.4.0</code>	no tag and no release until pushed
<code>git cherry-pick -x <fix></code>	the fix applied	matches the new commit	unchanged (still symbolic)	moves to the new commit	new commit object; reflog entries; <code>CHERRY_PICK_HEAD</code> only if it stops on a conflict	unchanged until pushed	unchanged until pushed
<code>git push origin release/1.4 v1.4.1</code>	unchanged	unchanged	unchanged	unchanged	<code>refs/remotes/origin/release/1.4</code> updated	branch and tag created or moved	the tag appears; a release object exists only if you create one (gh release create)

The comparison.

Question	A: GitHub Flow with tags	B: release branch
Long-lived refs	<code>main</code>	<code>main</code> and one <code>release/x.y</code> per supported version
Contents of a patch release	everything merged to <code>main</code> since the previous tag	only what was picked onto the release branch
Commits per fix	one	two (one per line), linked by patch ID and the <code>-x</code> line

Question	A: GitHub Flow with tags	B: release branch
Can you patch 1.4 after 1.5 exists?	no: there is no line for 1.4 to stand on, short of creating a branch at the old tag	yes
Extra failure mode	a half-finished feature on <code>main</code> blocks an urgent release unless it is behind a flag	a fix that reaches one line and not the other (section 27.10)
CI cost	one branch	every supported release branch needs its own pipeline

Strategy A can become strategy B on the day it is needed: `git switch -c release/1.4 v1.4.0` creates the missing line at the tag, after the fact. That is an argument for starting with the simpler model and adding release branches when a second live version appears.

GitHub, not Git. A GitHub release is an object layered on a Git tag. Creating one in the web interface or with `gh release create` can create the tag on the server, and your clone does not have it until you fetch tags. With immutable releases enabled, the tag and its assets are locked and the tag name cannot be reused ([about releases](#), [immutable releases](#)).

27.10 Which way does a fix travel?

In one sentence. There are two legitimate conventions and they are opposites: commit the fix on the oldest branch that needs it and merge upward, or commit it on `main` first and cherry-pick it down.

Analogy. A textbook in print in two editions. You can correct the older edition and let the correction flow into the newer one, or correct the current manuscript and copy the correction by hand into the older edition. The analogy breaks because Git can do the first mechanically (a merge) only while the newer line still contains the older one as an ancestor.

Precisely.

- **Merge upward.** The Git project's own workflow document states it: "Always commit your fixes to the oldest supported branch that requires them. Then (periodically) merge the integration branches upwards into each other." (`git help workflows`, also at [gitworkflows](#)). The fix is one commit with one ID, and the merge commit is, in that document's words, a "promise" that everything from the older branch is included in the newer one.
- **Fix on main first, cherry-pick down.** The trunk-based development site, Google's published practice, Microsoft's Release Flow and GitLab's rules all prescribe this. The stated reason is the failure it prevents: a fix made on the release branch can be forgotten on the trunk, and the next release then regresses ([branch for release](#)).

See it: merge upward. The starting state is the same in all three parts of this demo: `release/1.4` at the 1.4.0 tag, and `main` one feature ahead.

```
# Convention A: commit the fix on the oldest branch that needs it...
$ git switch -q release/1.4
$ git commit -q -am "Fix limit 0 being treated as unlimited"
$ git tag -a v1.4.1 -m "promptgate 1.4.1"
# ...then merge that branch upward into main:
$ git switch -q main
$ git merge -m "Merge branch release/1.4 into main" release/1.4
Merge made by the 'ort' strategy.
 gateway/limits.py | 2 +-
 1 file changed, 1 insertion(+), 1 deletion(-)
$ git log --oneline --graph --decorate --all -6
*   1a91aff (HEAD -> main) Merge branch release/1.4 into main
|\
| * 64610e3 (tag: v1.4.1, release/1.4) Fix limit 0 being treated as unlimited
* |   a1351be Merge pull request #43 from feature/batch-api
| \
|  | /
| /|
| * 5928b76 Add batch endpoint
|/
*   518d961 (tag: v1.4.0) Merge pull request #42 from feature/tenant-limits
|\
| * 4806775 Read tenant limits with a default
```

```
# One commit, one ID, contained in both lines:
$ git branch --contains v1.4.1
* main
  release/1.4
$ git log --oneline main..release/1.4
# Empty: everything on the release branch is in main. That is an invariant you can test:
$ git merge-base --is-ancestor release/1.4 main
[exit status: 0]
```

One commit, `64610e3`, contained in both branches. The invariant "the release branch is an ancestor of `main`" holds, and `git merge-base --is-ancestor release/1.4 main` tests it with an exit status that a CI job can check.

See it: fix on main, cherry-pick down.

```
$ cd ../pick-down
# Convention B: commit the fix on main first...
$ git commit -q -am "Fix limit 0 being treated as unlimited"
$ git switch -q release/1.4
# ...then copy it down to the release branch:
$ git cherry-pick -x main
[release/1.4 a2d7a91] Fix limit 0 being treated as unlimited
Date: Mon Sep 7 10:36:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git tag -a v1.4.1 -m "promptgate 1.4.1"
$ git log --oneline --graph --decorate --all -6
* a2d7a91 (HEAD -> release/1.4, tag: v1.4.1) Fix limit 0 being treated as unlimited
| * d9474f7 (main) Fix limit 0 being treated as unlimited
| * a1351be Merge pull request #43 from feature/batch-api
|/|
| * 5928b76 Add batch endpoint
|/
* 518d961 (tag: v1.4.0) Merge pull request #42 from feature/tenant-limits
|\
| * 4806775 Read tenant limits with a default
```

```
# Two commits, two IDs. The release branch is never merged into main:
$ git merge-base --is-ancestor release/1.4 main
[exit status: 1]
# The check that replaces the ancestry test compares patches, not IDs:
$ git log --oneline --cherry-pick --right-only --no-merges main...release/1.4
# Empty: every change on the release branch has an equivalent on main.
```

Two commits, `d9474f7` and `a2d7a91`. The ancestry test fails by design, because the release branch is never merged back. What replaces it is a comparison of patches: `git log --cherry-pick --right-only main...release/1.4` lists the commits on the release branch that have no equivalent change on `main`. The manual describes `--cherry-pick` as omitting "any commit that introduces the same change as another commit on the 'other side' of a symmetric difference. Git decides "the same change" by patch ID, a hash of the diff without line numbers and whitespace (Chapter 10).

See it: the failure both conventions exist to prevent.

```
$ cd ../forgotten
# The failure both conventions exist to prevent: a fix made only on the release branch.
$ git switch -q release/1.4
$ git commit -q -am "Fix limit 0 being treated as unlimited"
$ git tag -a v1.4.1 -m "promptgate 1.4.1"
# Weeks later release 1.5 is cut from main:
$ git switch -q main
$ git tag -a v1.5.0 -m "promptgate 1.5.0"
$ git show v1.5.0:gateway/limits.py | grep "limit ="
    limit = LIMITS.get(tenant) or LIMITS["default"]
$ git show v1.4.1:gateway/limits.py | grep "limit ="
    limit = LIMITS.get(tenant, LIMITS["default"])
```

```
# v1.5.0 ships the bug that v1.4.1 fixed. Either check finds it before the tag:
$ git merge-base --is-ancestor release/1.4 main
[exit status: 1]
$ git log --oneline --cherry-pick --right-only --no-merges main...release/1.4
70188ed Fix limit 0 being treated as unlimited
```

Observed behavior	: v1.5.0 ships a bug that v1.4.1 fixed.
Git state	: commit 70188ed is reachable from release/1.4 and from tag v1.4.1 only. main and v1.5.0 do not contain it and contain no commit with the same patch.
Mechanism	: a commit on one branch is on another branch only if somebody merges or cherry-picks it. Nothing in Git propagates a fix between branches.
Root cause	: the fix was committed on the release branch, and the step that carries it to main (a merge upward, or a port) was a human step that nobody took.
Why Git does this	: branches are independent refs. "This fix belongs everywhere" is knowledge held by the team, not by the repository.
Correct fix	: port the commit to main (<code>git cherry-pick -x 70188ed</code> , or merge release/1.4 into main if your convention is merge-upward), then release 1.5.1. Do not move the v1.5.0 tag: people already have it.
Prevention	: make the check a release gate. Either "release/x is an ancestor of main" (merge-upward), or " <code>git log --cherry-pick --right-only main...release/x</code> is empty" (pick-down). Both are one command with a testable result.

The trade-off, stated plainly.

	Merge upward	Fix on main, pick down
Identity of the fix	one commit, one ID	one commit per line; linked by patch ID and the <code>-x</code> line
"Is the fix in release X?"	<code>git branch --contains <id></code> answers for every line	must search by patch or by message on each line
What a forgotten step causes	the fix is missing from <code>main</code> until the next upward merge, which brings it along with everything else	nothing dangerous if the pick is forgotten: the old release lacks a fix it never had
Extra baggage	merging the release branch upward brings everything on it, including version bumps and release-only changes, which then conflict or must be neutralized	the patch-ID check breaks when the port needed conflict resolution, because the diffs then differ
Fits	projects with several maintained lines and maintainers who curate merges (Git itself)	trunk-based teams with late-cut, short-lived release branches

Two limits of the checks. A cherry-pick that needed a conflict resolution has a different patch ID, so `--cherry-pick` reports it as missing although a human ported it; the `-x` line is then the evidence. And an upward merge resolved by discarding the release branch's side satisfies the ancestry test while the fix is absent; `git log --remerge-diff` shows what the resolver changed (Chapter 8: Merge).

In production. Pick one direction and write it down. Here is one repository in which the first fix was picked down and a second fix, on the next line of the same function, is then merged upward:

```
# ...and somebody carries it to main by merging the release branch upward:
$ git switch -q main
$ git merge release/1.4
Auto-merging gateway/limits.py
CONFLICT (content): Merge conflict in gateway/limits.py
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ git diff
diff --cc gateway/limits.py
index 18f6e2c,ca849ed..0000000
--- a/gateway/limits.py
+++ b/gateway/limits.py
@@@ -3,4 -3,4 +3,8 @@@ LIMITS = {"default": 60

    def allowed(tenant, used):
        limit = LIMITS.get(tenant, LIMITS["default"])
++<<<<<< HEAD
+     return used < limit
++=====
+     return max(used, 0) < limit
++>>>>>> release/1.4
$ git merge --abort
```

The upward merge brought the picked copy of the first fix with it. Relative to the merge base, `main` changed one line and the release branch changed that line identically plus its neighbour, so the two changes overlap. Git dropped the identical line from the conflict and left the neighbouring line for a human, although `main` never touched it. A repository that mixes the directions satisfies neither check of this section and pays for it in conflicts like this one.

27.11 Feature flags

In one sentence. A feature flag is a condition in the code that keeps unfinished or unreleased behavior switched off, so that the code can be merged long before the feature is released.

Analogy. A new wing of a building, built and inspected behind a locked door: part of the building from day one, open to the public later. The analogy breaks because both sides of the door run in the same production process, so a mistake behind the door can still bring the building down.

Precisely. A flag replaces a long-lived branch with a short-lived condition. Instead of keeping `feature/batch-api` unmerged for three weeks, you merge it in small pieces behind `if flags.enabled("batch_api")`, and `main` contains the code from the first day. Pete Hodgson's taxonomy distinguishes release toggles (ship incomplete code dark), experiment toggles, ops toggles and permission toggles, which differ in how long they live and how dynamically they change, and it names the cost: added complexity, combinations to test, and "toggle debt" when flags are not removed ([Feature Toggles](#)). Microsoft describes flags as what lets its developers avoid long-running feature branches and separate deployment from exposure.

What it changes in Git. Compare the two graphs. With the flag, the feature never exists as a diverged line, so it never needs a large merge and never blocks a release:

```
Long-lived branch:  ---o---o---o---o---o---o---M---  main
                    \                               /
                    o---o---o---o---o---o---o         feature/batch-api (3 weeks, one big merge)
```

Behind a flag: ---o---b1---o---b2---o---b3---o--- main b1..b3 = small merged pieces,
switched off until release

This is what removes the "extra failure mode" of GitHub Flow in the table of section 27.9: a half-finished feature on `main` does not block an urgent release if it is dark.

When not to use one. Fowler's caveat is that with flags the unfinished code ships inside the product, so the approach demands strong automated tests to keep the main line healthy. A flag is the wrong tool for a change that cannot be made conditional (a schema migration, a dependency upgrade), and a flag that guards a security boundary must fail closed. A flag is configuration, and configuration that changes behavior belongs in version control or in a system with its own audit trail; DORA's version-control capability lists configuration among the things to keep under version control ([DORA: version control](#)).

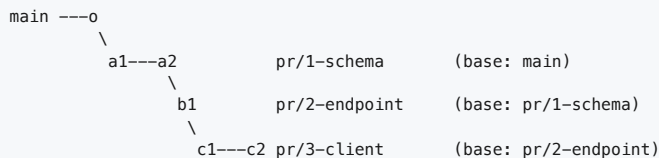
In production. For an LLM application the natural flags are the model name, the prompt version and the retrieval settings. A new prompt can then be merged and evaluated on a fraction of traffic before it becomes the default, and "which prompt answered this ticket" becomes a question about a flag value at a time, not only about a commit (Chapter 28, section 28.7).

27.12 Merge queues and stacked changes

In one sentence. A merge queue and a stack attack the two costs of reviewing before merging: a busy base branch that keeps invalidating finished work, and an author who is blocked while a large change waits for review.

Merge queues. Two pull requests can each pass their checks against an older `main` and break `main` together; requiring every branch to be up to date before merging fixes that and creates a race, because every merge makes every other pull request stale. A merge queue runs the required checks on temporary branches that contain the base plus the queued changes, in order, and merges what passes. On GitHub those checks run on the `merge_group` event, and the merge method is fixed by the queue instead of being chosen per pull request. Chapter 17: Pull requests, section 17.11 reproduces the mechanism with plain Git. The published experience comes from large repositories: GitHub reports about 2,500 pull requests a month from more than 500 engineers landing through its own queue, with the average wait to ship down 33% ([GitHub Blog](#)), and Shopify described running CI on a predicted post-merge branch and ejecting failures to keep its main branch green ([Shopify Engineering](#)). Those numbers describe those companies; a team that merges five pull requests a day gets the correctness guarantee and little else.

Stacked changes. A stack is a chain of small changes, each reviewed separately and each based on the one below. The practice comes from tools with per-commit review, such as Phabricator at Facebook and chained changelists at Google ([Jackson Gabbard](#), [The Pragmatic Engineer](#)). The stated benefits are an unblocked author and small reviews; the stated costs are the rebasing skill and tooling it needs, and little gain for a small team. GitHub's native stacked pull requests have been in public preview since 30 July 2026 ([changelog](#)). Chapter 17, section 17.14 shows the plain-Git mechanics.



Neither is a branching strategy. A merge queue makes any model with pre-merge checks safer at volume, and stacks make short-lived branches practical for a change too large for one review.

27.13 What the evidence shows, and what it does not

In one sentence. The research associates short-lived branches and small batches with better delivery performance; it does not rank named workflows, and it does not establish cause.

What DORA reports. DORA defines trunk-based development as each developer dividing work into small batches and merging into the trunk at least once a day. Its capability page reports, from research conducted in 2016 and 2017, that higher delivery performance is associated with three or fewer active branches, merging to trunk at least daily, and no code freezes or integration phases; it lists heavyweight, slow code review and skipping automated tests before commit as pitfalls ([DORA: trunk-based development](#)). A companion capability says that working in small batches predicts delivery and organizational performance ([DORA: working in small batches](#)).

What that evidence is.

Property of the evidence	Consequence for how you may cite it
Survey-based: respondents describe their own practices and outcomes	it measures reported practice, not observed repositories
Correlational	"is associated with" and "predicts", in the statistical sense the authors use; not "causes"

Property of the evidence	Consequence for how you may cite it
Expressed in branch lifetime, number of active branches and batch size	it says nothing about "Git Flow" or "GitHub Flow" by name; a GitHub Flow team with one-day branches satisfies it, and a nominally trunk-based team with week-long branches does not
The branch findings date from 2016 and 2017	later reports were not read for this course: the research notes record that the 2024 and 2025 report PDFs were not read and that no numeric effect sizes were captured

Unverified. Any specific number for how much trunk-based development improves delivery performance. The notes behind this course contain none, and you should not quote one from memory.

What other sources add. Fowler, who accepts this research, still lists real advantages of feature branching: a feature can be assessed as a unit, code enters the product only when complete, and it suits teams that cannot yet keep a main line healthy and open-source projects with occasional contributors who are not yet trusted (Fowler). An interview-and-survey study of Brazilian developers published in 2025 concludes that trunk-based workflows suit fast-paced projects with experienced, smaller teams, and branch-based workflows suit less experienced and larger teams despite their management overhead (arXiv 2507.08943); it is one study of one population. The company case studies come from organizations with very large investments in CI, flags and tooling, and Google and Meta do not use stock Git for their main repositories, so they support the principle more than any Git command sequence. DORA's recent reports describe AI as an amplifier of an organization's existing strengths and weaknesses and keep small batches and version control among the capabilities that matter (DORA 2025 report).

How to say it to a CTO. "The research does not prove that trunk-based development is best. It shows that teams reporting short-lived branches and small batches also report better delivery performance. The variable we can act on is how long our branches live and how large our changes are, under whatever name we give the model."

27.14 Choosing by context

No strategy is best. The reputable sources tie the choice to context, and two questions come before all others: **how many versions are live at once**, and **how often do you release**.

Context	What it pushes you toward	Why	What to watch
One live version, deployed continuously (a hosted service)	one long-lived branch: GitHub Flow or trunk-based development	there is nothing for a release branch to hold	branch lifetime; flags for unfinished work; fast rollback
One live version, released on a schedule (weekly, per sprint)	trunk plus a late-cut release branch per release (Release Flow)	the branch isolates stabilization without freezing <code>main</code>	fix direction; retire old release branches
Several supported versions (SDK, on-premises product, mobile app with old versions in the field)	long-lived <code>release/x.y</code> branches; Git Flow is one historical arrangement of them	each supported version needs a line to receive fixes	the port check of section 27.10 as a release gate; CI per supported line
Small, experienced team with strong automated tests	shorter branches, lighter pre-merge gates	the cost of a bad merge is low and quickly repaired	do not drop review silently: decide it
Large team, or many less experienced contributors	feature branches with required review and checks; a merge queue at volume	pre-merge gates protect a main line the team cannot yet keep healthy by habit	review latency becoming the bottleneck
Open source with outside contributors	fork workflow, pull requests, maintainers merge	contributors are not yet trusted with write access	secrets and <code>pull_request_target</code> (Chapter 21A)
Regulated environment (segregation of duties, audit trail, change approval)	protected branches with required review by someone other than the author, signed tags for releases, release branches where a release is an audited event	the audit asks who approved what and what exactly shipped	make bypasses visible: rulesets and Rule Insights (Chapter 18); compliance comes from enforced, logged rules, not from the name of the model
Model or prompt releases that must be reproducible	tags on the exact commit, plus the data and model versions recorded with it	"which code produced this model" must have one answer	Chapter 28, section 28.7

The rows combine: a regulated company with a hosted product and an on-premises edition is in three rows at once. Changing the model later is cheap in Git (branches are refs) and expensive in habits, pipelines and rulesets, so start with the simplest model that answers the two questions and add a branch only when you can name the version or the audit requirement that needs it.

27.15 Practices, each with its reason

A practice you cannot justify is a superstition, and it will be dropped under pressure. Each row gives the mechanism that makes the practice worth its cost.

Practice	The reason, in terms of mechanism	Where it is taught
Small commits, one logical change each	a commit is the unit of <code>revert</code> , <code>cherry-pick</code> , <code>bisect</code> and review; a commit that does three things can only be undone, ported or blamed as three things	this section; Chapter 6, Chapter 14A
Meaningful messages	the message is the only part of a commit that records why; the diff already records what	section 27.18
A protected main branch	a ref anyone can move is a ref anyone can break or rewind; a rule turns "we agreed not to" into "the server refuses"	Chapter 18
Review before merge	a second reader catches what the author's model of the change hides, and spreads knowledge of the code	section 27.17
Required CI checks	a check that is not required is advice; and a check must run on the merged result to mean anything about <code>main</code>	Chapter 17, sections 17.6 and 17.11
No secrets in Git	history is copied to every clone and fork and cannot be recalled; deleting the file adds a commit and removes nothing	Chapter 21B: Repository security; Chapter 28, section 28.14
<code>--force-with-lease</code> , never bare <code>--force</code>	a bare force sets the server's ref with no check; a lease makes the push conditional on the ref being where you last saw it	Chapter 12, section 12.8
Signed commits and tags where identity matters	author and committer fields are text that anyone can set; a signature is evidence that a key holder made the object	Chapter 14B
A branch naming scheme	rules, workflow triggers and cleanup scripts select branches by pattern	section 27.5
Rebase private branches	rebasing rewrites commit IDs; on a branch nobody else has, nobody else can be holding the old IDs	Chapter 9
No history rewriting on shared branches	everyone who fetched the old commits now has a history that diverges from the server, and the next careless merge brings the removed commits back	Chapter 9; Chapter 12, section 12.8
Inspect before you merge	a merge brings every commit reachable from the other side, not only the ones you had in mind	this section

See it: inspect before merging. Three commands answer "what would this merge bring, and will it merge cleanly", and none of them changes anything:

```
# Before merging feature/fallback: which commits would arrive, and what do they change?
$ git log --oneline main..feature/fallback
dd57a51 chore: raise default limit for load test
4ea69d0 feat(router): fall back when the primary model is unhealthy
8df1ca6 feat(router): add fallback table
$ git diff --stat main..feature/fallback
gateway/fallback.py | 5 ++++
gateway/limits.py   | 2 +-
gateway/router.py   | 7 ++++--
3 files changed, 11 insertions(+), 3 deletions(-)
# Would it merge cleanly? Ask without touching the working tree, the index or any ref:
$ git merge-tree --write-tree --name-only main feature/fallback
4a26aedd8171757262dc58e3a1d20dbaf20c118b
[exit status: 0]
```

The two-dot `git log` lists the commits you would gain. The three-dot `git diff` shows the change since the merge base, which is what a pull request shows (Chapter 17, section 17.3). `git merge-tree --write-tree` merges in memory and prints the ID of the resulting tree; exit status 0 means no conflicts. The first list contains a surprise: a commit that raises a limit "for load test".

See it: why small commits pay. Suppose it was merged anyway, with a merge commit that preserves the three commits:

```
# The load-test limit reached production. Because it is a commit of its own, it can be
# undone alone, and the fallback feature stays:
$ git revert --no-edit dd57a51
[main f3624e2] Revert "chore: raise default limit for load test"
Date: Mon Sep 7 10:17:00 2026 +0530
1 file changed, 1 insertion(+), 1 deletion(-)
$ git show --stat --format="%h %s" HEAD
f3624e2 Revert "chore: raise default limit for load test"

gateway/limits.py | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
$ grep default gateway/limits.py
LIMITS = {"default": 60}
return used < LIMITS.get(tenant, LIMITS["default"])
```

🟡 **CAUTION:** `git revert` adds a commit, which moves the branch, and removes nothing (Chapter 11: Reset, revert, restore). One commit undone, one file touched, the feature intact. Now the same branch merged as one squashed commit:

```
# The same branch as one squashed commit (rebuilt here from the same content):
$ git show --stat --format="%h %s" HEAD
9a0bfe3 Add model fallback (#51)

gateway/fallback.py | 5 ++++
gateway/limits.py   | 2 +-

```

```

gateway/router.py | 7 +++++--
3 files changed, 11 insertions(+), 3 deletions(-)
# Reverting it removes the feature together with the mistake:
$ git revert --no-edit HEAD
[main 108204a] Revert "Add model fallback (#51)"
Date: Mon Sep 7 10:25:00 2026 +0530
3 files changed, 3 insertions(+), 11 deletions(-)
delete mode 100644 gateway/fallback.py
$ git show --stat --format="%h %s" HEAD
108204a Revert "Add model fallback (#51)"

gateway/fallback.py | 5 -----
gateway/limits.py   | 2 +-
gateway/router.py   | 7 +-----
3 files changed, 3 insertions(+), 11 deletions(-)

```

The only revert available removes the fallback feature together with the mistake. Squash merging is a legitimate choice, and this is its price: the unit of undo becomes the pull request. The practice that follows is not "never squash" but "keep pull requests as small as the commits you would have wanted".

27.16 Anti-patterns, each with its root cause

An anti-pattern is rarely stupidity. Each one below is the reasonable result of a wrong mental model, and the model is what you correct.

Anti-pattern	Root cause: the model behind it	What it costs	The correction
Giant commits	"a commit is a save point for my day", not a unit of change	review that cannot be done; <code>bisect</code> that ends on 2,000 lines; reverts that take the good with the bad	stage by hunk (<code>git add -p</code> , Chapter 5); one logical change per commit
Committed secrets	"the repository is private", or "I will delete it in the next commit"	every clone and fork has it; deletion adds a commit; on GitHub the old commits stay reachable by ID	rotate first, then clean up (Chapter 21B); push protection
Committed generated files	"everything needed to run should be in the repository", confusing sources with outputs	conflicts on every merge; diffs nobody reads; a repository that grows with each build	ignore outputs; commit the inputs and the lock file (Chapter 28, section 28.9)
Rebasing shared branches	"rebase tidies history", without the fact that it creates new commits	colleagues' branches now contain both versions; duplicated commits after their next merge	rebase only what nobody else has; merge on shared branches
Blind force-push	"the push was rejected, so force it": the rejection is read as an obstacle and not as information	colleagues' commits removed from the server	read the rejection; fetch; <code>--force-with-lease</code> <code>--force-if-includes</code> ; forbid force pushes on shared branches by rule
Meaningless messages (<code>wip</code> , <code>fix</code> , <code>update</code>)	"the diff explains itself"	history that cannot be searched; nobody knows why a line exists	section 27.18
<code>git reset --hard</code> without understanding state	"reset means undo"	uncommitted work destroyed; the reflog records commits, so it cannot bring back work that was never committed	know the three trees (Chapter 11); <code>git stash</code> or commit first
Merging without inspecting history	"merging a branch brings my change", not "everything reachable from it"	unrelated or unreviewed commits arrive on <code>main</code>	section 27.15: <code>git log A..B</code> before every merge
Ignoring CI	"it is flaky", or "it passed locally"	a broken main line; red becomes normal, and a real failure hides in it	make checks required; fix or delete flaky tests; know why CI differs from your machine (Chapter 20B)
Outdated authentication	a tutorial from before 13 August 2021, when GitHub stopped accepting account passwords for Git operations; or one long-lived token with full scope shared by a team	pushes that fail; or a leaked token that reaches everything	Chapter 16: Authentication
Huge binaries in Git	"Git stores my project", without the fact that every clone carries every version of every file	slow clones for everyone, forever; hard limits on the server	pointers and external storage (Chapter 22: Git LFS; Chapter 28, section 28.6)
Treating GitHub as Git	"it is on GitHub, so it is backed up and it is the truth"	surprise when a force push removes work, when a pull request is not in the clone, or when a ruleset blocks a valid Git operation	label the layer: which behavior is Git's, which is the platform's (Chapter 15: GitHub)

`riskscore`, the repository of Lab 34.1, contains several of these on purpose, and the lab finds each of them from the command line: an anti-pattern you can measure is one you can put in front of a CTO.

27.17 Code review as a practice

In one sentence. Review is where a team's knowledge of its code is exchanged; finding defects is the smaller part of its value, and a review that takes days costs more than it finds.

The mechanics of review on GitHub are in Chapter 17, and its section 17.17 lists what an author and a reviewer should do with them. This section adds the practice around them.

Size decides everything else. Every technique in this chapter that shortens branches (flags, stacks, small commits) is also a review technique.

Speed is part of quality. DORA lists heavyweight and slow review as a pitfall (section 27.13). A change that waits two days makes its author start something else. Agree on a time to first response.

Say what kind of comment it is: blocking, a question, or a preference. Rouan Wilsenach's "Ship / Show / Ask" lets the author choose per change whether to merge directly, to open a pull request and merge without waiting, or to wait for discussion; it presupposes trust, good CI and feature flags ([Ship / Show / Ask](#)).

Review the right things. A human reviewer's attention is best spent on what a machine cannot check: whether the change does what the issue asked, whether it is in the right place, what it does on failure, what it does to data. Formatting and import order belong to a tool. In an ML repository add three questions: which data and which evaluation support the claim in the description, whether a prompt or configuration change was evaluated or only edited, and whether the pull request changes anything under `.github/workflows/`, a lock file or a `Dockerfile`, which deserve a second reader.

What review does not replace. Tests, required checks and rules. "Request changes" does not block a merge, and `CODEOWNERS` does not enforce review, unless a ruleset says so (Chapter 18, Chapter 19).

27.18 Commit message conventions

In one sentence. The subject line says what the commit does, the body says why, and trailers carry facts that tools read.

Precisely. Git itself imposes one piece of structure: the first line, up to the first blank line, is the subject, which is what `git log --oneline`, `git shortlog` and `git format-patch` use. Everything else is convention. The most cited one is Chris Beams's seven rules: separate subject from body with a blank line; limit the subject to about 50 characters; capitalize it; do not end it with a period; use the imperative mood; wrap the body at 72 characters; use the body to explain what and why, not how ([How to Write a Git Commit Message](#)).

Conventional Commits. Conventional Commits 1.0.0 adds a machine-readable prefix: `<type>[optional scope]: <description>`, with an optional body and footers. `fix` corresponds to a PATCH release in Semantic Versioning, `feat` to a MINOR one, and a `BREAKING CHANGE` footer (or a `!` after the type) to a MAJOR one; other types such as `docs` or `chore` are allowed ([specification](#)). The point is that release tooling can compute the next version number and a changelog from the history.

```
$ git log --format="%h %s" v1.3.0..HEAD
108204a Revert "Add model fallback (#51)"
9a0bfe3 Add model fallback (#51)
f09559c docs: say how to run the tests
# With a convention, the history answers questions:
$ git log --oneline --grep="^feat" v1.3.0..feature/fallback
4ea69d0 feat(router): fall back when the primary model is unhealthy
8df1ca6 feat(router): add fallback table
$ git log --format="%s" v1.3.0..feature/fallback | sed "s/[[:].*/]" | sort | uniq -c
  1 chore
  2 feat
$ git shortlog -sn v1.3.0..feature/fallback
  3 Lab User
```

`--grep="^feat"` and the `sort | uniq -c` pipeline work only because the prefix is regular. `Add model fallback (#51)` is a GitHub-style squash subject, with the pull request number as the link back to the discussion.

Trailers. Lines of the form `Key: value` at the end of the message are trailers: `Co-authored-by`, `Signed-off-by` (added by `git commit -s`), `Reviewed-by`, or an issue reference. `git interpret-trailers` parses them, and `git log --format='%(trailers:key=Co-authored-by)'` prints them.

What the convention cannot do. It does not make a message true. `fix: typo` on a commit that changes a rate limit passes every commit-message linter. A convention is enforced the same way as any other local check: a `commit-msg` hook for fast feedback (Chapter 14C, section 14C.10), and a required check or the squash-merge title for the guarantee.

27.19 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A new release ships a bug an older patch release fixed	<code>git log --oneline --cherry-pick --right-only</code> <code>--no-merges main...release/x.y</code> lists the unported fix	port it to <code>main</code> , release a new patch; do not move the tag	the port check as a release gate (section 27.10)

Symptom	Diagnosis	Fix	Prevention
A patch release contains a feature nobody announced	<code>git log --oneline --no-merges vA..vB</code>	none after the fact; say so in the release notes	a release branch, or flags for unfinished work
The fork's <code>main</code> cannot be fast-forwarded	<code>git rev-list --left-right --count upstream/main...origin/main</code> shows a second number above zero	move the stray commits to a branch, then reset the fork's <code>main</code> to upstream	never commit on the fork's <code>main</code> ; <code>--ff-only</code>
"Is this branch merged?" gives contradictory answers	the branch was squash-merged: ancestry is gone, content is present	compare content: <code>git diff</code> , or <code>git merge-tree</code> against the tree of <code>main</code> (Lab 34.1)	delete branches at merge time
A revert removed more than intended	the pull request was squashed; the unit of undo is the whole pull request	revert, then re-apply the wanted part as a new commit	smaller pull requests
A pull request from a fork has red checks the author cannot fix	the workflow needs secrets, which forks do not get	a maintainer runs the privileged part after review	split workflows: untrusted build and test, trusted evaluation (Chapter 28, section 28.11)

27.20 When not to use it, and dangerous edge cases

- **Do not adopt a model by name.** "We do Git Flow" tells nobody how long branches live or which way fixes travel. Write down the refs, the rules and the fix direction.
- **Do not cherry-pick between two long-lived branches as a routine way of integrating.** Occasional ports are fine. Routine picking in both directions leaves two histories that no check can reconcile.
- **A moved tag is worse than a wrong tag.** Clones do not replace a tag they already have unless forced (Chapter 14B), so half your users keep the old meaning. Release a new patch version instead.
- `gh repo sync --force` **hard-resets the fork's branch.** If you committed there, those commits become unreachable on the fork. Preview with the `rev-list --left-right --count` command of section 27.3.
- **Feature flags that never die.** A flag whose both paths are still in the code a year later is an untested branch living inside `main`.
- **Merge queues change what your workflows must listen to.** A required check that does not run on `merge_group` never reports, and the queue stalls (Chapter 17, section 17.11).

27.21 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git log A..B</code> , <code>git log --cherry-pick --right-only A...B</code> , <code>git cherry</code> , <code>git diff A...B</code> , <code>git merge-base --is-ancestor</code> , <code>git merge-tree --write-tree</code>	🟢 SAFE	nothing (<code>merge-tree</code> adds unreferenced objects)	not needed	not needed
<code>git tag -a <name></code>	🟢 SAFE	adds a tag object and a ref	<code>git tag --list</code>	<code>git tag -d <name></code> before it is pushed
<code>git cherry-pick -x <commit></code>	🟡 CAUTION	adds a commit to the current branch, which moves it; updates index and working tree	<code>git show <commit></code>	<code>git reset --keep HEAD^</code> before pushing; <code>git revert</code> after
<code>git revert <commit></code>	🟡 CAUTION	adds a commit, which moves the branch; updates index and working tree	<code>git show <commit></code>	revert the revert
<code>git merge --ff-only upstream/main</code> , <code>git merge --no-ff <branch></code> , <code>git merge <release branch></code>	🟡 CAUTION	moves the current branch; updates index and working tree	<code>git log HEAD..<other></code> ; <code>git merge-tree --write-tree</code>	<code>git reset --keep ORIG_HEAD</code> before pushing
<code>git rebase upstream/main</code>	🟡 CAUTION	replaces the branch's commits with new ones	<code>git log upstream/main..HEAD</code>	<code>git reset --keep ORIG_HEAD</code> , or the branch reflog
<code>git branch -d <branch></code>	🟡 CAUTION	deletes a ref and its reflog, after a merge check against the upstream or HEAD	<code>git branch -vv</code>	<code>git branch <name> <id></code> from the ID it prints
<code>git push origin --delete <branch></code>	🔴 DANGEROUS	deletes a ref on the server	<code>git ls-remote origin</code>	push the commit again from a clone that has it

Command	Label	What it changes	Preview	Recovery
<code>git push --force-with-lease --force-if-includes</code>	 DANGEROUS	replaces a server branch if it is where you last saw it	<code>git fetch; git log HEAD..@{u}</code>	section 27.3; Chapter 12, section 12.8
<code>gh repo sync --force</code>	 DANGEROUS	hard-resets a branch of the fork to its parent	<code>git rev-list --left-right --count upstream/main...origin/main</code>	push the old commits back from a clone that has them

27.22 Version notes

Version note. Older behavior: a push forced with a lease could still discard work fetched in the background. Current behavior: `--force-if-includes` additionally checks that you integrated what you last fetched. Since: Git 2.30. Recommended: use both options together.

Version note. Older behavior: testing a merge meant performing it and aborting. Current behavior: `git merge-tree --write-tree` merges without touching the working tree, the index or any ref. Since: Git 2.38. Recommended: use it in scripts and release gates.

Version note. Older behavior: stacked and dependent pull requests were maintained by hand or with third-party tools. Current behavior: native stacked pull requests on GitHub, in public preview. Since: 30 July 2026. Recommended: learn the plain-Git mechanics in Chapter 17 first.

Version note. Older behavior: Git Flow presented as a general default. Current behavior: its author's note restricts it to explicitly versioned software with several supported versions. Since: 5 March 2020. Recommended: choose by the table in section 27.14.

Unverified. The release that introduced `%(ahead-behind:<ref>)` in `git for-each-ref`, used in Lab 34.1, was not confirmed for this chapter (Chapter 7 carries the same flag). It runs on Git 2.55.

27.23 Practice

- Lab 32.1: one release and one hotfix under two strategies, a fix forgotten on `main`, and its repair.
- Lab 34.1: the Level 8 design review. Measure a company's repository, then design and defend its branching model, governance, CI and security policy.
- Revisit Lab 7.5 in the Module 7 labs for the mechanics of the triangular setup, and the Module 21 labs for pull requests from forks.
- Run the demos: `labs/run ch27/fix-direction`, `labs/run ch27/git-flow`, `labs/run ch27/fork-sync`.

27.24 Interview questions

1. Your company ships one hosted version today and will support two on-premises versions next year. What changes in your branching model, and on which day do you change it?
2. A patchy release went out without a fix that the previous patch release contained. Walk through how you find the cause from the repository alone, and what you add to the release process afterwards.
3. State the two conventions for the direction of a fix. For each, give the invariant you can test and one way the test can mislead you.
4. A colleague says the DORA research proves trunk-based development is best. What does it show, how was it measured, and what would you say instead?
5. Under GitHub Flow, a customer asks for a release that contains only a security fix. What do you tell them, and what would you have needed to have in place?
6. After a squash merge, `git branch --merged` does not list your branch. Why, and how do you establish that its work is on `main`?
7. What does a merge queue guarantee that "require branches to be up to date" does not, and what does it cost?
8. Give the mechanism behind three of your team's practices: why small commits, why no force push on shared branches, why inspect before merging.
9. You maintain an open-source project and receive a 3,000-line pull request from a stranger. What do you do, and what in your repository should have prevented it?
10. Your fork's `main` is two commits ahead of and forty behind upstream. How did that happen, and how do you repair it without losing the two commits?

27.25 Sources

Primary sources

- Git 2.55 manual pages: `git help workflows` ([gitworkflows](#)), `git help cherry-pick`, `git help rev-list` (`--cherry-pick`, `--left-right`), `git help branch`, `git help merge-tree`, `git help for-each-ref`.
- GitHub Docs: [GitHub flow](#), [Forks](#), [Syncing a fork](#), [About merge methods](#), [Managing a merge queue](#), [Setting guidelines for repository contributors](#), [About releases](#).
- GitHub CLI 2.88.1, local help: `gh repo fork --help`, `gh repo sync --help`, `gh pr create --help`.
- The authors of the models: Vincent Driessen, [A successful Git branching model](#) with the 2020 note; Scott Chacon, [GitHub Flow](#); GitLab, [What is GitLab Flow?](#); [trunkbaseddevelopment.com](#); Microsoft, [How Microsoft develops with DevOps](#).
- DORA capability pages: [trunk-based development](#), [working in small batches](#), [version control](#).
- Conventions: [How to Write a Git Commit Message](#), [Conventional Commits 1.0.0](#), [Semantic Versioning 2.0.0](#), [Keep a Changelog 1.1.0](#).

Secondary sources

- The Phase 0 report of this course, sections 6, 12 and 13, and section 6 of its notes on books, documentation and misconceptions.
- Martin Fowler, [Patterns for Managing Source Code Branches](#); Pete Hodgson, [Feature Toggles](#); Rouan Wilsenach, [Ship / Show / Ask](#).
- Lopes, Accioly, Borba and Menezes, interview-and-survey study of branching workflows, [arXiv 2507.08943](#).
- Vendor and company material, linked where used, each describing its own environment: GitHub and Shopify on merge queues, Atlassian on Gitflow.

Videos (from the report's tables, with its caveats)

- [3 Git Workflows Every Developer Should Know](#), TechWorld with Nana, 13 January 2026, 32 minutes: a neutral comparison of Git Flow, GitHub Flow and trunk-based development. Caveat: it has a sponsored segment.
- [Continuous Integration vs Feature Branch Workflow](#), Dave Farley, 6 January 2021, 18 minutes: the argued case for trunk. Caveat: deliberately one-sided advocacy; watch it after the first, and with section 27.13 beside you.
- [Implementing a Strong Code-Review Culture](#), Derek Prior, RailsConf 2015, 38 minutes: review as teaching and context-giving. Tool-independent and still valid.

Further reading

- Jackson Gabbard, [Stacked Diffs Versus Pull Requests](#), and Gergely Orosz, [Stacked Diffs](#).
- *Accelerate* (Forsgren, Humble, Kim, 2018), the book-length statement of the DORA research.
- Chapter 28: AI/ML workflows applies this chapter to repositories that also hold notebooks, data pointers and evaluation runs.

Chapter 28: AI/ML workflows

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch28/`.

28.1 Why this matters

Four questions from a CTO to the engineer who owns the model repository:

1. "The evaluation dashboard says accuracy 0.90 for the run we showed the board. Check out the commit it names and show me 0.90."
You check it out and get 0.80.
2. "The clone takes eleven minutes. Why?"
3. "Security found an API key in a notebook. Nobody committed a key."
4. "The pre-commit hooks block large files. How did a 900 kB dump reach `main`?"

Each has a cause you can name by the end of this chapter. The run was made from a working tree with an uncommitted change, and the tracker recorded only the commit (section 28.7). A checkpoint was committed two years ago and "deleted", and every clone still carries it (section 28.9). The key was printed by a cell, and the notebook file stores what cells print (section 28.3). And a hook that is not installed does not run (section 28.10).

The rule that organizes the chapter: **keep code and pointers in Git, and keep data, weights and outputs out of it**. An ML project differs from a backend service in that most of its bytes are not source code, and most of what determines a result (the data, the weights, the resolved configuration, the environment) is not in a commit unless you put a reference to it there.

Two limits of this chapter, stated once. The tools of this field (DVC, nbstripout, nbdime, jupyter, pre-commit, promptfoo, MLflow and others) are not installed in the lab, so their commands are given from their documentation, in blocks without output, with the versions and dates the Phase 0 report recorded on 1 October 2026. What is run here is the mechanism under each tool, written with the Python standard library: a clean filter, a pointer file with a checksum, a function that records the commit and the dirty state. The scripts are in `labs/ch28/files/`; each is short enough to read in full, and reading them is the point.

The demos use `docqa`, a small project that classifies support tickets and answers them from documentation.

28.2 What belongs in Git, and what does not

In one sentence. Git should hold what a human writes and reviews, plus small references to everything else; it should not hold what a program produces or what is large.

Analogy. A lab notebook and a freezer. The notebook records the protocol, the settings and the label of every sample; the samples live in the freezer. Nobody tapes a sample into the notebook, and nobody runs a lab from the freezer alone. The analogy breaks in one useful way: a Git "notebook" can verify the freezer, because the label it records can be a checksum of the contents.

Precisely. Three properties of Git decide the question. Every clone carries the full history, so a file committed once is downloaded by everyone, in every version, for as long as the history exists (Chapter 3: Git internals). Objects cannot be recalled: removing a file adds a commit and removes nothing. And diff, merge and blame work on lines of text. On top of that the platform sets limits: GitHub warns about files larger than 50 MiB, blocks files larger than 100 MiB, recommends that repositories stay ideally under 1 GB and strongly under 5 GB, and states that Git is not designed to serve as a backup tool ([about large files on GitHub](#)).

In Git	Not in Git; referenced from Git
source code and tests	raw and processed data sets
configuration, including evaluation settings	checkpoints and exported weights
prompts and evaluation specifications	experiment-tracker stores (<code>mlruns/</code> , <code>mlflow.db</code> , <code>wandb/</code>)
lock files	virtual environments and caches
Dockerfiles and CI workflows	build outputs, logs, run directories
notebooks without outputs	notebook outputs
pointer and metadata files (a checksum, a model ID with a pinned revision)	secrets, in any form

This split is the report's inference from the limits above, not a published standard. The less obvious rows are prompts and pointers. A prompt is source code for an LLM application: a one-word change alters behavior, so it deserves a diff, a review and a commit. A pointer is what makes the right-hand column reproducible: Git cannot hold the data, but it can hold forty or sixty-four hexadecimal characters that identify exactly one version of it.

In production. The same split is a security boundary. The left column is what you are prepared to show every engineer, contractor and CI job with read access. Customer data, weights you licensed and credentials need access control that a Git repository does not give you per file.

28.3 Notebooks: a JSON file that stores code, outputs and counters together

In one sentence. An `.ipynb` file is one JSON document in which each code cell carries its source, the outputs of its last run and a counter, so running a notebook changes the file even when no code changed.

Precisely. The notebook format defines a top-level object with `metadata`, `nbformat`, `nbformat_minor` and `cells`. A code cell stores `source`, a list of `outputs` and an `execution_count`; an output can be a stream (what was printed), a result, an error with its traceback, or display data such as a base64-encoded `image/png` ([nbformat](#)).

See it. One code cell of a committed notebook. The lab cannot run Jupyter, so `labs/ch28/files/make_notebook.py` writes the file Jupyter would save after run number N: the same sources every time, and different counters, printed text and chart bytes.

```
# One code cell of the committed notebook: source, outputs and a counter in one object.
$ sed -n '10,29p' notebooks/01-error-analysis.ipynb
},
{
  "cell_type": "code",
  "execution_count": 4,
  "id": "load",
  "metadata": {},
  "outputs": [
    {
      "name": "stdout",
      "output_type": "stream",
      "text": [
        "using key sk-demo-not-a-real-key-0001\n"
      ]
    }
  ],
  "source": [
    "import os\n",
    "key = os.environ[\"LLM_API_KEY\"]\n",
    "print(\"using key\", key)"
  ]
}
```

Source, output and counter sit in one object. That one design decision causes three separate problems.

Problem 1: a rerun is a change. Run all cells again without editing anything:

```
# Run all cells again without editing any code (simulated: the file Jupyter would save):
$ git diff --stat
notebooks/01-error-analysis.ipynb | 12 +++++-----
1 file changed, 6 insertions(+), 6 deletions(-)
$ git diff | grep '^[-+]' | cut -c1-76
- "execution_count": 4,
+ "execution_count": 7,
-   "using key sk-demo-not-a-real-key-0001\n"
+   "using key sk-demo-not-a-real-key-0002\n"
- "execution_count": 5,
+ "execution_count": 8,
-   "execution_count": 5,
+   "execution_count": 8,
-   "execution_count": 6,
+   "execution_count": 9,
-   "image/png": "AAECAwQFBgcICQoLDA00DxAREhMUFRYXGBkaGxwdHh8gISiJJCUMjYg
+   "image/png": "AAIEBggKDA4QEhQWGBocHiAiJCyoKiwuMDI0Njg6PD5AQkRGSEpMTlB
```

Six lines changed and none of them is code: three counters (one of them stored twice), one printed line, one image. A reviewer who opens this diff learns nothing, and after a few such commits nobody opens notebook diffs at all.

Problem 2: outputs carry data and credentials into history.

```
# The first cell printed a credential. It is now in two commits:
$ git grep -n "sk-demo" $(git rev-list HEAD) -- notebooks | cut -c1-110
5e95f6c14957c3a89416fb5587e9913a0c04dcde:notebooks/01-error-analysis.ipynb:21:   "using key sk-demo-not-a-r
ac8532561836542f31df0ed0c49a3213816c29ea:notebooks/01-error-analysis.ipynb:21:   "using key sk-demo-not-a-r
$ wc -c < notebooks/01-error-analysis.ipynb
3558
$ python3 -c "import json,sys; nb=json.load(open(sys.argv[1])); print(sum(len(json.dumps(c['outputs'])) for c in nb['cells'] if
c['cell_type']=='code'), 'bytes of outputs')" notebooks/01-error-analysis.ipynb
2429 bytes of outputs
```

The source of the cell reads a key from the environment, which is the right way to handle a key. The cell also printed it, and what a cell prints is part of the file. The value (a fake one here) is now in two commits. GitGuardian's write-up of notebook leaks names this path: print statements,

error messages and variable inspection display secrets that then persist in the notebook's JSON ([GitGuardian](#)). The same holds for a data frame preview that shows customer rows: no secret scanner has a pattern for "a table that should not be here". In this small notebook 2,429 of 3,558 bytes are outputs; with real charts the proportion is far higher.

Unverified. How often secrets leak through notebook outputs specifically. The report found mechanisms, not numbers.

Problem 3: a line-based merge can produce a file that is not a notebook.

```
# Two people rerun the same notebook on two branches:
$ git merge asha/rerun
Auto-merging notebooks/01-error-analysis.ipynb
CONFLICT (content): Merge conflict in notebooks/01-error-analysis.ipynb
Automatic merge failed; fix conflicts and then commit the result.
[exit status: 1]
$ grep -c '^<<<<<<' notebooks/01-error-analysis.ipynb
6
$ python3 -m json.tool notebooks/01-error-analysis.ipynb > /dev/null
Expecting property name enclosed in double quotes: line 13 column 1 (char 185)
[exit status: 1]
```

Two people reran the same notebook on two branches. Git merged line by line, as it does for any text file, and wrote conflict markers into the middle of a JSON document. The file no longer parses; Jupyter cannot open it to let you resolve the conflict.

```
Observed behavior : a notebook nobody edited shows as modified, conflicts on merge, and
                    contains a credential nobody typed.
Git state          : the blob for the .ipynb holds source, outputs and execution counts.
Mechanism          : Jupyter saves the result of the last run into the same file as the code.
                    Git stores and compares the file as lines of text.
Root cause         : one file mixes what a human wrote with what a program produced.
Why Git does this  : Git has no knowledge of file formats. Content awareness is added through
                    filter, diff and merge drivers, and those must be configured.
Correct fix        : keep outputs out of the blob (a clean filter, section 28.4), or review a
                    paired text file instead (jupyter, section 28.5).
Prevention         : enforce the choice in CI, because the filter lives in each clone's
                    configuration and a clone does not have it.
```

GitHub, not Git. GitHub renders .ipynb files as static HTML. Its rich notebook diff is still the feature preview announced on 1 March 2023, in which you cannot comment on lines; no general-availability announcement was found ([changelog](#)). **Unverified:** whether the redesigned "Files changed" page renders notebook rich diffs; the report did not check a signed-in interface. Treat it as a viewer, not as the basis of a review process.

28.4 A clean filter that strips outputs

In one sentence. A clean filter rewrites a notebook on its way into the repository so that the blob has no outputs, while the file in your working tree keeps them.

Analogy. A mail room that removes attachments from outgoing letters and files the plain letter. Your desk copy still has the attachments. Where it breaks: every desk has its own mail room, and a new employee's desk has none until somebody sets it up.

Precisely. Chapter 14C: Stash internals, rerere, attributes and hooks, section 14C.8 defines filter drivers: .gitattributes names a driver for a path pattern, and filter.<name>.clean in the Git configuration defines the command, which reads the file on standard input and writes the cleaned form to standard output. That chapter showed a minimal notebook filter. This section goes further in the directions an ML team needs: what "required" changes, how git status behaves, what a clone has, and the form of the check that CI runs.

The real tool. nbstripout (0.9.1, February 2026) is the maintained implementation. From its documentation, not run here ([nbstripout](#)):

```
nbstripout --install          # filter into this clone's .git/config, attributes into .git/info/attributes
nbstripout --install --attributes .gitattributes # attributes into a committed file instead
nbstripout --status           # is the filter installed in this clone?
nbstripout --verify notebooks/*.ipynb          # exit non-zero if anything would be stripped: the CI form
```

Its README states the limitation that shapes everything below: there is no way to have Git set up the filter automatically when someone clones a repository, by design, so that a clone never executes arbitrary code.

How such a tool works. The core of tools/nbstrip.py, standard library only:

```
$ sed -n '10,31p' tools/nbstrip.py
DROP_CELL_METADATA = ("collapsed", "scrolled", "execution")
```

```
def strip(nb):
    """Remove everything that running the notebook adds. Returns the same object."""
    for cell in nb.get("cells", []):
        if cell.get("cell_type") == "code":
            cell["outputs"] = []
            cell["execution_count"] = None
        for key in DROP_CELL_METADATA:
            cell.get("metadata", {}).pop(key, None)
    nb.get("metadata", {}).pop("widgets", None)
    return nb

def dump(nb):
    # The layout Jupyter itself writes: one-space indent, keys in file order, final newline.
    return json.dumps(nb, indent=1, ensure_ascii=False) + "\n"

def main(argv):
    if argv[1] == ["--verify"]:
```

Parse the JSON, empty `outputs`, set `execution_count` to null, drop the metadata that execution adds, and write the document back in the layout Jupyter itself uses. The last part matters: if the filter reformatted the file, every notebook would show as changed the first time it passed through. Cleaning cleaned content must change nothing.

See it. The attribute is already committed in `.gitattributes`; the driver is defined in this clone's configuration:

```
$ git check-attr filter -- notebooks/01-error-analysis.ipynb
notebooks/01-error-analysis.ipynb: filter: nbstrip
$ git config set filter.nbstrip.clean "python3 tools/nbstrip.py"
$ git config set filter.nbstrip.smudge cat
$ git config set filter.nbstrip.required true

$ git add tools/nbstrip.py notebooks
$ git commit -q -m "Add notebook filter and the error-analysis notebook"
checks: 2 file version(s) examined, 0 problem(s)
# The file on disk still has its outputs. The blob in the repository does not:
$ grep -c output_type notebooks/01-error-analysis.ipynb
3
$ git cat-file -p HEAD:notebooks/01-error-analysis.ipynb | grep -c output_type
0
$ git cat-file -p HEAD:notebooks/01-error-analysis.ipynb | sed -n '10,20p'
},
{
  "cell_type": "code",
  "execution_count": null,
  "id": "load",
  "metadata": {},
  "outputs": [],
  "source": [
    "import os\n",
    "key = os.environ[\"LLM_API_KEY\"]\n",
    "print(\"using key\", key)"
$ wc -c < notebooks/01-error-analysis.ipynb
3558
$ git cat-file -s HEAD:notebooks/01-error-analysis.ipynb
967
```

The file on disk has three outputs and 3,558 bytes. The blob has none and 967 bytes. The `checks:` line is the project's pre-commit hook reporting (section 28.10).

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in <code>.git</code>	Remote	GitHub
<code>git config set filter.nbstrip.clean ...</code>	unchanged	unchanged	unchanged	unchanged	<code>.git/config</code> gains the driver	unchanged	unchanged
<code>git add notebook.ipynb</code> with the filter	unchanged: outputs stay	entry points at a blob of the cleaned content	unchanged	unchanged	one new blob	unchanged	unchanged
<code>git add --renormalize .</code>	unchanged	every tracked file re-cleaned; entries change where the cleaned form differs	unchanged	unchanged	new blobs	unchanged	unchanged
<code>git restore <notebook></code> , or <code>git switch</code> to a commit where the notebook differs	file rewritten from the cleaned blob (smudge is <code>cat</code>): local outputs are gone	updated	as usual	as usual	as usual	unchanged	unchanged

The last row is the price: the outputs exist only in your working tree, and any command that rewrites the file from the repository discards them.

Rerunning is no longer a change.

```
# Rerun the notebook (simulated). The file changes; what Git would store does not:
$ grep -c 'not-a-real-key-0002' notebooks/01-error-analysis.ipynb
1
$ git status --short
$ git diff --stat
```

The file contains the second run's printed key, and `git status` is silent, because Git compares the cleaned form of the file with the index. A change to the code of a cell still shows, as a one-line diff a reviewer can read:

```
# A change to the code of a cell is still a change:
$ sed -i.bak 's/plot_confusion(errors)/plot_confusion(errors, normalize=True)/' notebooks/01-error-analysis.ipynb && rm
notebooks/01-error-analysis.ipynb.bak
$ git status --short
M notebooks/01-error-analysis.ipynb
$ git diff
diff --git a/notebooks/01-error-analysis.ipynb b/notebooks/01-error-analysis.ipynb
index ed9abfc..42ce0ef 100644
--- a/notebooks/01-error-analysis.ipynb
+++ b/notebooks/01-error-analysis.ipynb
@@ -38,7 +38,7 @@
     "metadata": {},
     "outputs": [],
     "source": [
-      "plot_confusion(errors)"
+      "plot_confusion(errors, normalize=True)"
     ]
   }
 ],
```

What `required` changes. A filter is optional by default. The manual says a missing driver definition, or a driver that exits with an error, "is not an error but makes the filter a no-op passthru". For a filter whose purpose is to keep outputs out of history, a silent pass-through is the worst possible failure:

```
# required = true: a filter that cannot run stops the operation instead of passing content through.
$ git config set filter.nbstrip.clean nbstrip-not-installed
$ git add notebooks/01-error-analysis.ipynb
error: cannot run nbstrip-not-installed: No such file or directory
error: cannot fork to run external filter 'nbstrip-not-installed'
error: external filter 'nbstrip-not-installed' failed
fatal: notebooks/01-error-analysis.ipynb: clean filter 'nbstrip' failed
[exit status: 128]
$ git status --short
M notebooks/01-error-analysis.ipynb
# The same with required unset: the outputs go in, with only a warning.
$ git config unset filter.nbstrip.required
$ git add notebooks/01-error-analysis.ipynb
error: cannot run nbstrip-not-installed: No such file or directory
error: cannot fork to run external filter 'nbstrip-not-installed'
error: external filter 'nbstrip-not-installed' failed
[exit status: 0]
$ git cat-file -p :notebooks/01-error-analysis.ipynb | grep -c output_type
3
```

With `required = true`, `git add` fails with status 128 and nothing is staged. With it unset, the same three error lines appear, the exit status is 0, and the notebook is staged with its three outputs. `nbstripout` declares its filter required for this reason.

What a clone has.

```
# A clone gets the attribute and the script, not the configuration:
$ git clone -q ../server.git ../docqa-asha && cd ../docqa-asha
$ git check-attr filter -- notebooks/01-error-analysis.ipynb
notebooks/01-error-analysis.ipynb: filter: nbstrip
$ git config get filter.nbstrip.clean
[exit status: 1]
# Asha reruns the notebook and commits. Nothing strips it and nothing warns her:
$ git commit -q -am "Rerun error analysis" && git push -q
$ git cat-file -p HEAD:notebooks/01-error-analysis.ipynb | grep -c output_type
3
```

The attribute travelled, because `.gitattributes` is a tracked file. The driver did not, because it lives in `.git/config`, and Git never copies configuration from a clone source: if it did, cloning a repository would let its author run commands on your machine. `required = true` did not help either, since that setting is part of the same missing configuration. Asha's notebook went in with outputs and nothing warned her.

The check that does travel.

```
# What CI runs on a fresh checkout of each commit:
$ python3 tools/nbstrip.py --verify $(git ls-files "*.ipynb")
```

```
not stripped: notebooks/01-error-analysis.ipynb
[exit status: 1]
$ git switch -q --detach HEAD~1
$ python3 tools/nbstrip.py --verify $(git ls-files "*.ipynb")
[exit status: 0]
```

`--verify` strips each committed notebook in memory and fails if the result differs from the file. On a fresh checkout the files are the blobs, so this is a check of what is in history. It fails on Asha's commit and passes on the one before. Run as a required check, it is the control; the filter on each laptop is a convenience that keeps people from tripping it.

In production. Stripping outputs is also a data-protection control, as the report notes: outputs hold query results and personal data that no secret-scanning pattern matches. A team that needs some outputs in review (a chart that is the result) has three honest options: commit them deliberately with a tool that supports review of outputs (section 28.5), export the figure to a tracked report file, or keep the outputs in the experiment tracker and link the run.

28.5 nbtime, jupyterxtext, ReviewNB: tools that complement the filter

The maintained tools are complementary, not alternatives. Commands are from each tool's documentation and are not run here.

Tool (version on 1 October 2026)	What it controls	Mechanism in Git terms
nbstripout (0.9.1)	what enters history	a clean filter, section 28.4
nbtime (4.0.4)	how notebooks are diffed and merged locally	a diff driver, a merge driver and a merge tool
jupyterxtext (1.19.5)	what gets reviewed	pairs the notebook with a plain-text file
ReviewNB (commercial GitHub app)	review of notebooks with outputs	comments on rendered notebook diffs in pull requests

nbtime understands the notebook structure, so it can show a cell-level diff and can merge two reruns by resolving generated values such as execution counters instead of writing conflict markers into JSON ([nbtime](#), [Git integration](#)):

```
nbtime config-git --enable          # register the diff driver, merge driver and merge tool for this repository
nbdiff-web main feature notebook.ipynb
git mergetool --tool nbtime -- *.ipynb
```

The `.gitattributes` lines it relies on are `*.ipynb diff=jupyterxtext` and `*.ipynb merge=jupyterxtext`. Like every driver, these are definitions in each clone's configuration (Chapter 14C, sections 14C.6 and 14C.7): a colleague who has not run the command gets Git's line-based behavior, with no warning.

jupyterxtext takes the other road: do not review the `.ipynb` at all ([jupyterxtext](#)).

```
jupyterxtext --set-formats ipynb,py:percent notebook.ipynb # pair the notebook with notebook.py
jupyterxtext --sync notebook.py                          # bring both files up to date from the newer one
jupyterxtext --to ipynb notebook.py                      # rebuild the notebook from the text file
```

The `.py` file contains only the cell inputs, so a pull request shows an ordinary Python diff. The README's own recipe is to commit the text file and, unless you want outputs versioned, to keep the `.ipynb` out of version control.

Choosing.

Situation	Reasonable choice
Notebooks are exploration; nobody reviews outputs	strip outputs (nbstripout), verify in CI
Notebooks contain logic that is reviewed like code	pair with jupyterxtext and review the <code>.py</code> ; ignore or strip the <code>.ipynb</code>
Outputs are the deliverable and must be discussed	commit them on purpose, and use a tool built to review them (ReviewNB, or nbtime's web viewers locally)
Notebook logic keeps growing	move it into the package under <code>src/</code> and import it; the notebook becomes a thin caller

The last row is the durable fix. A function in `src/docqa/` has tests, a diff and a blame history. A cell has none of them.

28.6 Data and model versioning: Git holds the reference, another system holds the bytes

In one sentence. You version a large file by committing a small file that names it by checksum, and by keeping the bytes in a store that is addressed by that checksum.

Analogy. A coat-check ticket. The ticket is small and travels with you; the coat stays in the cloakroom. The analogy breaks in two places that matter: this ticket is derived from the coat (it is a hash of the contents, so a different coat can never be handed back for it), and a ticket is worthless if the cloakroom has been emptied.

Precisely. Every tool in this section is a variation on one pattern, which you met as Git LFS in Chapter 22: Git LFS:

1. Compute a cryptographic hash of the file.
2. Store the file outside Git under that hash (content-addressed storage).
3. Commit a small pointer that records the hash and the size.
4. Tell Git to ignore the real file, or let a filter swap pointer and file.

Git then versions the pointer: `git log`, `git diff`, branches, tags and pull requests all work on it, and "which data did this commit use" has an exact answer.

See it. `tools/dataref.py` is that pattern in about a hundred lines of standard-library Python. The store is a directory beside the repository; in production it would be a bucket.

```
$ wc -l < data/raw/tickets.csv
11
$ git status --short --ignored data
!! data/raw/
$ python3 tools/dataref.py add data/raw/tickets.csv
stored data/raw/tickets.csv as sha256:2226b23c455b (432 bytes)
commit the pointer: git add data/raw/tickets.csv.ref
$ cat data/raw/tickets.csv.ref
{
  "path": "tickets.csv",
  "sha256": "2226b23c455b63565c71f01555c41bdd3661598f70cd2c989b0ddcd0164a898e",
  "size": 432
}
$ git status --short data
?? data/raw/
$ git add data/raw/tickets.csv.ref && git commit -q -m "Version the ticket data set by reference"
checks: 1 file version(s) examined, 0 problem(s)
```

The data file was ignored before and stays ignored (`!! data/raw/` in the first status). What becomes visible to Git is the pointer: five lines of JSON for a file of any size.

Picture.

Git repository (cloned by everyone)	data store (a bucket; access-controlled)
data/raw/tickets.csv.ref	sha256/22/26b23c45... (432 bytes)
{ "sha256": "2226b23c45...",	sha256/4e/fbc43184... (606 bytes)
"size": 432 }	
.gitignore: /data/** !*.ref	one object per version, named by hash

working tree: data/raw/tickets.csv is present, ignored, and checked against the pointer

A new version of the data is a one-line diff.

```
# Four labelled tickets arrive. The data file changes; Git sees nothing until the pointer changes:
$ git status --short
$ python3 tools/dataref.py verify
differs data/raw/tickets.csv (pointer sha256:2226b23c455b)
[exit status: 1]
$ python3 tools/dataref.py add data/raw/tickets.csv
stored data/raw/tickets.csv as sha256:4efbc43184d3 (606 bytes)
commit the pointer: git add data/raw/tickets.csv.ref
$ git diff
diff --git a/data/raw/tickets.csv.ref b/data/raw/tickets.csv.ref
index e27856a..0dd411a 100644
--- a/data/raw/tickets.csv.ref
+++ b/data/raw/tickets.csv.ref
@@ -1,5 +1,5 @@
 {
   "path": "tickets.csv",
-  "sha256": "2226b23c455b63565c71f01555c41bdd3661598f70cd2c989b0ddcd0164a898e",
-  "size": 432
+  "sha256": "4efbc43184d3a7e7fbed75dd9f07d1c89633dd493cbb53392340d5541a9a0681",
+  "size": 606
 }
$ git commit -q -am "Add four labelled tickets to the data set"
checks: 1 file version(s) examined, 0 problem(s)
```

Read the first two commands together. Four rows were appended to the data, and `git status` printed nothing: Git does not watch an ignored file. The tool's own `verify` reports the mismatch. This is the central weakness of versioning by reference, and Lab 33.2 turns it into a failure scenario: **the pointer and the file can disagree, and Git will not tell you.**

Going back is two steps.

```
# Going back is two steps: Git moves the pointer, the tool moves the bytes.
$ git switch -q --detach HEAD~1
$ python3 tools/dataref.py verify
differs data/raw/tickets.csv (pointer sha256:2226b23c455b)
```

```
[exit status: 1]
$ python3 tools/dataref.py checkout
restored data/raw/tickets.csv from sha256:2226b23c455b
$ wc -l < data/raw/tickets.csv
11
$ git switch -q main && python3 tools/dataref.py checkout
restored data/raw/tickets.csv from sha256:4efbc43184d3
```

Git moved the pointer; the working file still held the newer data until the tool restored the version the pointer names. DVC has the same two steps (`git checkout`, then `dvc checkout`). Git LFS hides the second step inside a smudge filter, which is more convenient and is why a clone without the LFS client silently gets pointers.

A clone has pointers, not data; and the store is a dependency.

```
$ git clone -q . ../docqa-ravi && cd ../docqa-ravi
$ ls data/raw
tickets.csv.ref
$ python3 tools/dataref.py checkout
restored data/raw/tickets.csv from sha256:4efbc43184d3
$ python3 tools/dataref.py verify
ok data/raw/tickets.csv (pointer sha256:4efbc43184d3)
```

```
# The pointer is only as good as the store behind it:
$ rm -r ../datastore/sha256 && rm data/raw/tickets.csv
$ python3 tools/dataref.py checkout
NOT IN STORE data/raw/tickets.csv sha256:4efbc43184d3
[exit status: 1]
```

A pointer whose object is gone identifies a data set precisely and cannot produce it. The store needs what the repository has: backups, access control, and a retention rule that never deletes an object some commit still points to.

The real tools. The table is the report's, with its dates and flags.

Tool	Status on 1 October 2026	Mechanism	When it fits, and caveats
Git LFS	client 3.8.0; GitHub caps a file at 2 to 5 GB by plan; metered billing (LFS documentation)	a three-line pointer in Git, swapped by a required filter; objects in LFS storage	a modest number of mid-sized, rarely changing binaries. Wrong when files exceed the cap, change often (each version is stored whole), must be deletable, or when CI and contributors lack the client (the report's inference; Chapter 22, section 22.13)
DVC	3.67.1 (31 March 2026); Apache-2.0; stewardship moved to lakeFS on 18 November 2025 (DVC blog , releases)	a <code>.dvc</code> metafile per tracked path; the data path is added to <code>.gitignore</code> ; data moves with <code>dvc push</code> and <code>dvc pull</code>	Git-based versioning for small to medium projects. Maintained; no release in the six months after March 2026, and its roadmap is unstated
lakeFS	v1.88.0; Business Source License from v1.87.0 (22 September 2026); earlier releases remain Apache 2.0 (lakeFS blog)	Git-like branches over object storage	platform-scale data. Do not call it open source without the version caveat; read the licence text before embedding it in a product
git-annex	10.20260901 (git-annex)	manages large files with Git without storing their contents in Git	a niche alternative
Hugging Face Hub	all repositories migrated from Git LFS storage to Xet by October 2025; the LFS pointer format is kept for compatibility (Hugging Face , legacy LFS)	Git repositories whose large files are routed through <code>.gitattributes</code> ; byte-level deduplication	pin what you download
Model registries	MLflow 3.16.1; W&B SDK 0.30.0, documentation now under docs.coreweave.com (MLflow , W&B)	versioned models with lineage to the producing run; mutable aliases	record the Git commit beside the model version

Unverified. The Hugging Face "all repositories migrated" claim dates from October 2025 and was not independently verified for 2026. No statement of DVC's roadmap was found. Both flags are carried from the report.

DVC, from its documentation ([get started](#)); not run here, because DVC is not installed. In a repository where DVC has been initialized as the guide describes:

```
dvc add data/data.xml # writes data/data.xml.dvc and adds the data path to data/.gitignore
git add data/data.xml.dvc data/.gitignore # Git versions the metafile, not the data
git commit -m "Add raw data"
dvc remote add -d myremote <location> # where the bytes go (a bucket or a directory)
dvc push # upload the data
dvc pull # in another clone: download what the metafiles name
git checkout <revision> # move the metafile ...
dvc checkout # ... then make the data match it
```

Compare each line with the transcript above: `dvc add is dataref.py add` plus the ignore rule; the `.dvc` metafile is the `.ref` file (the guide shows it holding an `md5` and a `path`); `dvc checkout is dataref.py checkout`. The DVC guide says of itself that DVC "is technically not a version control system": it manipulates metafiles that Git versions.

Pinning a model from a hub. A model ID such as `org/name` names whatever the publisher's default branch points at today. The Hugging Face download guide documents a `revision` parameter that accepts a branch, a tag or a commit hash, and says a commit hash must be the full-length hash, not a seven-character abbreviation ([download guide](#)). Not run here:

```
from huggingface_hub import snapshot_download

snapshot_download(repo_id="example-org/ticket-embedder",
                  revision="<the full-length commit hash you evaluated>",
                  allow_patterns=["*.safetensors", "*.json"])
```

A branch or tag in `revision` is a moving reference, exactly as a Git branch is. Commit the hash you evaluated; that line of configuration is the model's pointer file.

In production. Registry aliases such as `champion` are mutable by design, which is what makes promotion a one-step operation and what makes an alias useless as a record. Deploy by alias if you like; log the resolved version number and the Git commit of the training code beside it.

28.7 Reproducibility: the commit alone does not identify what ran

In one sentence. A result is identified by the commit, the state of the working tree relative to that commit, the data and model versions, the resolved configuration and the environment; record all of them or you have recorded a guess.

Analogy. A chain of custody. A sealed evidence bag with a label is evidence; an open bag with the same label is not. The commit ID is the label; the dirty flag says whether the bag was open. Where it breaks: unlike a seal, a dirty tree can be reconstructed exactly if you also keep the patch.

Precisely. The report's list of what must be recorded together: the Git commit and whether the tree was dirty; a committed lock file; an immutable container reference; the resolved configuration; the data and model versions. It also records that the tools capture less than people assume:

- **MLflow.** The classic run context sets `mlflow.source.git.commit`, the branch and the repository URL when the entry point is inside a Git repository, and does not inspect or record uncommitted changes. A newer, opt-in function marked experimental records the dirty state and the diff ([system tags](#), [git context source](#), [genai versioning source](#)).
- **Weights & Biases.** It generates a patch of uncommitted changes when code is logged, but code saving is disabled by default for all teams ([code saving](#)).
- **Hydra.** It writes the resolved configuration and the command-line overrides into a `.hydra` directory for every run ([Hydra](#)).

Unverified. The report flags a conflict about MLflow's branch tag: the documentation table says it is set for MLflow Projects only, the source code sets it for any run inside a repository, and the source is the more reliable description. The opt-in function is described from source code, not from a documentation page. W&B's exact default for commit capture when code saving is off was not tested, and a bug report about missing patch files exists, so verify capture instead of assuming it. Hydra's behavior was read from the development documentation, not re-verified against release 1.3.7.

The consequence, in the report's words: with classic tracking, two different code states can share one commit ID.

See it: a function that records the state. Sixteen lines of standard library:

```
$ sed -n '19,34p' tools/runinfo.py
"""Return a dict that identifies the code state of the current repository."""
commit = _git("rev-parse", "HEAD").decode().strip()
# Porcelain v1 with NUL separators: stable across Git versions and safe for odd file names.
status = [e for e in _git("status", "--porcelain=v1", "-z", "--untracked-files=all").decode().split("\0") if e]
tracked = [e for e in status if not e.startswith("??")]
untracked = [e[3:] for e in status if e.startswith("??")]
diff = _git("diff", "HEAD", "--binary") if tracked else b""
return {
    "commit": commit,
    "describe": _git("describe", "--always", "--dirty", "--tags").decode().strip(),
    "dirty": bool(tracked),
    "changed": tracked,
    "untracked": untracked,
    "diff_sha256": hashlib.sha256(diff).hexdigest() if diff else None,
}
```

Four Git commands do the work. `git rev-parse HEAD` gives the commit. `git status --porcelain=v1 -z` gives a stable, machine-readable list of what differs; entries starting with `??` are untracked files. `git diff HEAD --binary` is the complete uncommitted change to tracked files,

and its SHA-256 identifies that change. `git describe --always --dirty --tags` gives a human-readable name.

```
$ python3 tools/runinfo.py
{
  "changed": [],
  "commit": "aa18ad2fb32ebcc4c1603d60d9aa1b6f3540b80e",
  "describe": "v0.1.0",
  "diff_sha256": null,
  "dirty": false,
  "untracked": []
}
```

A recorded run. The evaluation script calls the function and writes the record beside the metrics:

```
$ python3 evals/run_eval.py baseline
run baseline: accuracy 0.8000 on 10 examples (commit aa18ad2)
$ cat runs/baseline/run.json
{
  "code": {
    "changed": [],
    "commit": "aa18ad2fb32ebcc4c1603d60d9aa1b6f3540b80e",
    "describe": "v0.1.0",
    "diff_sha256": null,
    "dirty": false,
    "untracked": []
  },
  "config_sha256": "b315d1f3d160e1022d366b19e4189868df102c829bf0c826b85473da1045e1fa",
  "data": {
    "path": "data/raw/tickets.csv",
    "sha256": "2226b23c455b63565c71f01555c41bdd3661598f70cd2c989b0ddcd0164a898e"
  },
  "lock_sha256": "085aa13c19c2ff263dc4c94771b99cb7dc6ec535f40dded48e1b6c54f06980cb",
  "metrics": {
    "accuracy": 0.8,
    "examples": 10
  },
  "resolved_config": {
    "data": "data/raw/tickets.csv",
    "default": "general",
    "keywords": {
      "billing": [
        "invoice",
        "refund",
        "charged"
      ],
      "technical": [
        "error",
        "crash",
        "timeout"
      ]
    }
  }
}
```

Five identifiers in one file: the commit `aa18ad2` with `dirty: false`; the checksum of the data actually read (compare it with the pointer in section 28.6); a hash of the lock file; a hash of the configuration; and the resolved configuration itself, so that the record can be read without checking anything out.

Two results, one commit. Now try an idea without committing it:

```
# Try an idea without committing it: one more billing keyword.
$ sed -i.bak 's/"charged"/"charged", "card"/' configs/eval.json && rm configs/eval.json.bak
$ git status --short
M configs/eval.json
$ python3 evals/run_eval.py tuned
refusing a tracked run from a dirty tree: M configs/eval.json
[exit status: 1]
$ python3 evals/run_eval.py tuned --allow-dirty
run tuned: accuracy 0.9000 on 10 examples (commit aa18ad2, dirty)
```

The script refuses a tracked run from a dirty tree, which is the first defensible policy. With `--allow-dirty` it runs and records what it can:

```
# Two results, one commit ID. Only the dirty flag and the diff hash tell them apart:
$ grep -h -e "\"commit\"" -e "\"dirty\"" -e "\"diff_sha256\"" -e "\"accuracy\"" runs/baseline/run.json runs/tuned/run.json
  "commit": "aa18ad2fb32ebcc4c1603d60d9aa1b6f3540b80e",
  "diff_sha256": null,
  "dirty": false,
  "accuracy": 0.8,
  "commit": "aa18ad2fb32ebcc4c1603d60d9aa1b6f3540b80e",
  "diff_sha256": "71195d1fdbe1681494c2eca7772d5753adaeb9e886818b92607d602d432790e7",
  "dirty": true,
  "accuracy": 0.9,
$ ls runs/tuned
```

```
metrics.json
run.json
uncommitted.patch
$ git diff HEAD --binary | shasum -a 256
71195d1fdb1681494c2eca7772d5753adaeb9e886818b92607d602d432790e7 -
```

Accuracy 0.8 and accuracy 0.9, both "at commit `aa18ad2`". A tracker that stores only the commit shows two runs of the same code with different results. This record tells them apart: `dirty: true`, a hash of the uncommitted diff, and the diff itself saved as `uncommitted.patch`. The last command proves the hash is of that diff. That is the second defensible policy: if dirty runs are allowed, log the status and the diff as artifacts. Lab 33.3 reproduces the 0.9 from exactly these three things.

```
Observed behavior : checking out the commit named by a run gives a different metric.
Git state         : the commit is intact. The run's working tree had a modified tracked file.
Mechanism         : "git rev-parse HEAD" names the last commit; it says nothing about the
                   index or the working tree. Python imported the files on disk.
Root cause        : the tracker recorded HEAD and not the difference between HEAD and what ran.
Why Git does this : a commit is a snapshot that was taken; uncommitted work is, by definition,
                   in no snapshot.
Correct fix       : if the patch was saved, apply it to the commit and rerun (Lab 33.3). If
                   not, the result cannot be reproduced; say so and rerun from a commit.
Prevention        : refuse tracked runs from a dirty tree, or record status and diff; treat
                   untracked files the same way.
```

Untracked files are a third state.

```
# An untracked file is not in "git diff", and it can still change what runs:
$ printf 'KEYWORD_OVERRIDES = {} \n' > src/docqa/local_settings.py
$ git status --short
?? src/docqa/local_settings.py
$ python3 tools/runinfo.py | grep -e dirty -e untracked -A1
"dirty": false,
"untracked": [
  "src/docqa/local_settings.py"
]
$ python3 tools/runinfo.py --require-clean
refusing to run: the working tree does not match commit aa18ad2
untracked: src/docqa/local_settings.py
[exit status: 1]
```

`dirty` is false, because no tracked file changed, and `git diff HEAD` is empty. Yet a new module on the import path can change behavior. The function reports untracked files separately and `--require-clean` refuses both. The MLflow opt-in function described above does not count untracked files as dirty, according to the report's reading of its source.

28.8 Lock files, container digests and resolved configuration

The record of section 28.7 contains hashes of three things that live in the repository. Each has a rule.

Lock files are committed. A dependency range such as `transformers>=4` is resolved to exact versions at install time, so two installs a month apart differ. `uv` (0.12.21) documents `uv.lock` as a cross-platform lock file that should be checked into version control, and `--locked` as the option that raises an error instead of updating a lock file that is out of date ([project layout](#), [locking and syncing](#)). GitHub's Python ignore template recommends committing lock files for applications ([template](#)). From the `uv` documentation, not run here:

```
uv lock          # resolve and write uv.lock; commit the result
uv sync --locked # install exactly what uv.lock says; fail if it is out of date (the CI form)
```

A lock file is generated, and it is still committed, because it is an input to every later build. The test for "generated files do not belong in Git" is whether the file can be regenerated identically from what is in Git; a lock file cannot, since the package index changes.

Container images are pinned by digest. An image tag is a mutable reference, like a branch: the publisher can move it. Docker's guidance is that pinning to a digest guarantees the same image, at the price of opting out of automatic fixes, which it suggests offsetting with Dependabot updating the pin on a schedule ([Docker best practices](#)). The parallel with Git is exact: tag is to digest as branch is to commit ID.

Configuration is recorded resolved. A configuration built from defaults, files, environment variables and command-line overrides exists in its final form only at run time. Record that final form with the run, as `resolved_config` does above and as Hydra does in `.hydra/config.yaml`.

Versions derived from tags need the tags. `git describe` names a commit by the most recent reachable annotated tag, and packaging plugins such as `setuptools-scm` (10.3.4) and `hatch-vcs` (0.5.0) derive the package version from it. In CI the default `actions/checkout` fetches one commit and no tags, so the derived version fails or falls back; set `fetch-depth: 0` in the jobs that need it ([git-describe](#), [setuptools-scm](#)).

Identifier	Mutable reference (do not record)	Immutable identifier (record)
Code	branch name, <code>HEAD</code>	full commit ID, plus dirty flag and diff hash
Dependencies	version ranges in <code>pyproject.toml</code>	the committed lock file, or its hash
Environment	image tag	image digest

Identifier	Mutable reference (do not record)	Immutable identifier (record)
Data	file path, "latest"	checksum from the pointer file
Model	model name, registry alias, hub branch	registry version number; hub commit hash
Configuration	defaults plus overrides	the resolved configuration

28.9 .gitignore and .gitattributes for ML projects

In one sentence. Ignore rules keep generated and large files from being added by accident, attributes say how tracked files are converted and merged, and neither is a security control.

Assembled, not authoritative. The report found no authoritative, maintained .gitignore template for ML projects. GitHub's Python template covers interpreter, packaging, environment and tool caches and contains nothing ML-specific. The file below is assembled from that template and from each tool's documentation of where it writes output. Read it as a starting point and adjust it to the tools you run.

```
$ cat .gitignore
# Python (a subset of GitHub's Python template)
__pycache__/
*.py[cod]
.venv/
.pytest_cache/
.ruff_cache/
build/
dist/
*.egg-info/

# Secrets and machine-local settings
.env
.env.*
!.env.example

# Data is versioned by reference: only pointers and the README are tracked
/data/**
!/data/**/
!/data/**/*.ref
!/data/README.md

# Model weights live in the model store, whatever directory they land in
/models/**
!/models/README.md
!/models/*.ref
*.pt
*.pth
*.ckpt
*.safetensors
*.onnx
*.gguf

# Experiment trackers and run outputs
/runs/
wandb/
mlruns/
mlflow.db
outputs/
multirun/

# Notebook autosave copies
.ipynb_checkpoints/
```

Why each group exists: the data directory, because data is versioned by reference (Cookiecutter Data Science's generated ignore file likewise ignores /data/); weight formats, because one stray `git add .` after training would otherwise commit a checkpoint; `wandb/`, `mlruns/` and `mlflow.db`, because the trackers write there by default; `outputs/` and `multirun/`, because Hydra creates a directory per run; `.env`, because that is where local credentials go. Editor and operating-system files (`.DS_Store`, `.idea/`) are deliberately absent: they belong in each developer's global ignore file (`core.excludesFile`), not in every project.

Which rule matched? `git check-ignore -v` answers with the file, line and pattern:

```
$ git check-ignore -v data/raw/tickets.csv data/processed/train.parquet models/v1/model.safetensors checkpoints/epoch-3.ckpt
runs/r1/metrics.json wandb/debug.log .env
.gitignore:17:/data/** data/raw/tickets.csv
.gitignore:17:/data/** data/processed/train.parquet
.gitignore:23:/models/** models/v1/model.safetensors
.gitignore:28:*.ckpt checkpoints/epoch-3.ckpt
.gitignore:34:/runs/ runs/r1/metrics.json
.gitignore:35:wandb/ wandb/debug.log
.gitignore:12:.env .env
# The pointer, the README and the example file are not ignored: no output, exit status 1.
```

```
$ git check-ignore data/raw/tickets.csv.ref data/README.md .env.example
[exit status: 1]
# With -v, Git names the last matching pattern even when it is a negation (it starts with "!"):
$ git check-ignore -v data/raw/tickets.csv.ref data/README.md .env.example
.gitignore:19:!/data/**/*.*ref data/raw/tickets.csv.ref
.gitignore:20:!/data/README.md data/README.md
.gitignore:14:!.env.example .env.example
```

The data rules use a pattern worth understanding. `/data/**` ignores everything below `data/`. `!/data/**/` re-includes the directories, which is necessary because Git does not look inside an ignored directory, so no later rule could re-include a file in it. `!/data/**/*.*ref` then re-includes the pointer files. Note the last command: with `-v`, `check-ignore` prints the matching pattern even when it is a negation, and exits 0; a line whose pattern starts with `!` means "not ignored".

Rule 1: ignore patterns never affect files that are already tracked.

```
# A checkpoint was committed before anybody wrote a rule for it:
$ git add classifier.ckpt && git commit -q -m "Add trained classifier"
$ printf '*.ckpt\n' > .gitignore && git add .gitignore && git commit -q -m 'Ignore checkpoints'
$ git ls-files
.gitattributes
.gitignore
README.md
classifier.ckpt
pyproject.toml
requirements.lock
src/docqa/__init__.py
src/docqa/classify.py
tests/test_classify.py
$ git check-ignore -v classifier.ckpt
[exit status: 1]
$ git check-ignore -v --no-index classifier.ckpt
.gitignore:1:*.*ckpt classifier.ckpt
# The rule matches the path, and Git keeps tracking the file: retraining shows up as a change.
$ git status --short
M classifier.ckpt
```

The pattern matches the path (`--no-index` shows that), and Git goes on tracking the file, because the manual defines ignore files as specifying "intentionally untracked files": the index decides what is tracked, not `.gitignore`.

```
$ git rm --cached classifier.ckpt
rm 'classifier.ckpt'
$ git commit -q -m "Stop tracking the checkpoint"
$ git status --short --ignored
!! classifier.ckpt
# Untracked from now on. History is unchanged: every clone still downloads the blob.
$ git log --oneline -- classifier.ckpt
04154c0 Stop tracking the checkpoint
a4fc927 Add trained classifier
$ git rev-list --objects --all | git cat-file --batch-check='%(objecttype) %(objectsize) %(rest)' | grep ckpt
blob 262144 classifier.ckpt
```

● CAUTION: `git rm --cached <path>` removes the path from the index and leaves the file on disk; the next commit records a deletion, and a colleague who pulls that commit has the file deleted from their working tree. And the history is untouched: the 262,144-byte blob is still reachable, so every clone still downloads it. Removing it from history is a rewrite with `git filter-repo`, with everything that implies for a shared repository (Chapter 21B: Repository security; Chapter 30: Incident response). For a secret, rotation comes first.

Rule 2: drivers are named in `.gitattributes` and defined in configuration.

```
$ cat .gitattributes
# Line endings: normalize text to LF in the repository on every platform
* text=auto
*.sh text eol=lf

# Notebooks enter the repository without outputs (driver: tools/nbstrip.py, see README)
*.ipynb filter=nbstrip

# Pointer files must never be converted or merged line by line
*.ref text eol=lf merge=binary
$ git check-attr -a notebooks/01-error-analysis.ipynb data/raw/tickets.csv.ref
notebooks/01-error-analysis.ipynb: text: auto
notebooks/01-error-analysis.ipynb: filter: nbstrip
data/raw/tickets.csv.ref: merge: binary
data/raw/tickets.csv.ref: text: set
data/raw/tickets.csv.ref: eol: lf
```

The attributes file, like the ignore file, is assembled. Three decisions are in it. `* text=auto` normalizes line endings in the repository for every contributor; the manual's way to apply it to an existing repository is `git add --renormalize`. (Chapter 14C, section 14C.5). `*.ipynb filter=nbstrip` names the notebook filter; a committed attribute that names a driver nobody configured is a silent no-op, or a hard failure

when the filter is required, which is section 28.4 in one sentence. And `merge=binary` on pointer files tells Git never to combine two pointers line by line: a pointer whose hash came from one side and whose size came from the other names nothing.

Rule 3: an ignore file is not a security control. It prevents accidents by people who use `git add` without `-f`. It does nothing about a file committed before the rule existed, a file added with `--force`, or a secret pasted into a tracked file.

28.10 The pre-commit framework, and why local hooks must be backed by CI

In one sentence. A hook gives the author feedback in seconds; only a check that runs where the author cannot skip it is a control.

The framework. `pre-commit` (4.6.2, August 2026) manages hooks declared in a committed file: `pre-commit install` installs the Git hook into the clone, `pre-commit run --all-files` is the documented CI usage, and `rev` must be an immutable tag or commit (`pre-commit`). A typical configuration for an ML repository, assembled from the tools' READMEs with the versions in the report, not run here:

```
# .pre-commit-config.yaml
repos:
- repo: https://github.com/pre-commit/pre-commit-hooks
  rev: v6.0.0
  hooks:
    - id: check-added-large-files    # default threshold 500 kB
    - id: detect-private-key
- repo: https://github.com/astrol-sh/ruff-pre-commit
  rev: v0.16.9
  hooks:
    - id: ruff-check
    - id: ruff-format
- repo: https://github.com/kynan/nbstripout
  rev: 0.9.1
  hooks:
    - id: nbstripout
```

Add one secret scanner. The report notes that the gitleaks README now describes the project as feature complete, with future releases limited to security patches, so check the state of whichever scanner you choose. `prek` (v0.5.4) is a Rust reimplement that reads the same configuration (`prek`); the report collected no benchmarks for it.

```
pre-commit install      # once per clone: writes .git/hooks/pre-commit
pre-commit run --all-files  # what CI runs
```

Why "once per clone" is the whole problem. The report lists the ways a local hook fails open: hooks are not installed by cloning, they are skipped with one flag, and they are bypassed by commits made through the web interface, the API, or an agent's own Git client. Chapter 14C, sections 14C.11 to 14C.13 give the mechanics. Here is each failure with the checks of this project, which are a standard-library stand-in for the hooks above (`tools/checks.py`: file size, private keys, environment files, weight files, notebooks with outputs).

```
$ cat hooks/pre-commit
#!/bin/sh
# Shared pre-commit hook. Enable it once per clone: git config set core.hooksPath hooks
exec python3 tools/checks.py --staged
$ git config get core.hooksPath
hooks
$ git switch -q -c feature/deploy-script
$ mkdir deploy && printf -- '-----BEGIN OPENSSH PRIVATE KEY-----\nnot-a-real-key\n-----END OPENSSH PRIVATE KEY-----\n' >
  deploy/id_deploy
$ git add deploy
$ git commit -m "Add deploy key for the staging box"
index deploy/id_deploy: contains a private key
checks: 1 file version(s) examined, 1 problem(s)
[exit status: 1]
```

The hook did its job: the commit was refused before the key reached any history.

```
# One flag skips the hook:
$ git commit -q --no-verify -m "Add deploy key for the staging box"
[exit status: 0]
$ git push -q -u origin feature/deploy-script
```

`--no-verify` is not a loophole; it is a documented option, and there are honest uses for it. A control that the controlled person can switch off is not a control.

```
$ git clone -q ../server.git ../docqa-asha && cd ../docqa-asha
$ git config get core.hooksPath
[exit status: 1]
$ git switch -q -c feature/eval-dump
$ wc -c < evals/dump.json
900011
```

```
$ git add evals/dump.json && git commit -q -m "Add evaluation dump for debugging"
$ git push -q -u origin feature/eval-dump
```

Asha never skipped anything. Her clone has no `core.hooksPath`, because configuration does not travel, so no hook ran. This is the answer to the fourth question of section 28.1.

The same checks, where they cannot be skipped. `ci/check.sh` runs the identical script over what was pushed:

```
# CI checks out each pushed branch and runs the same script over the range it adds:
$ cat ci/check.sh
#!/bin/sh
# Everything CI runs, runnable locally: sh ci/check.sh <base>
# <base> is the commit the branch started from (default: origin/main).
status=0
# 1. Every file version the branch adds, including ones a later commit removed again.
python3 tools/checks.py --range "${1:-origin/main}..HEAD" || status=1
# 2. The whole snapshot that would be merged.
python3 tools/checks.py --tree HEAD || status=1
if python3 -m unittest discover -s tests > /dev/null 2>&1; then echo "tests: ok"; else echo "tests: FAILED"; status=1; fi
exit $status
$ sh ci/check.sh origin/main
27e79a4 evals/dump.json: larger than 500000 bytes (900011)
checks: 1 file version(s) examined, 1 problem(s)
HEAD evals/dump.json: larger than 500000 bytes (900011)
checks: 25 file version(s) examined, 1 problem(s)
tests: ok
[exit status: 1]
$ git switch -q feature/deploy-script
$ sh ci/check.sh origin/main
8ca3417 deploy/id_deploy: contains a private key
checks: 1 file version(s) examined, 1 problem(s)
HEAD deploy/id_deploy: contains a private key
checks: 25 file version(s) examined, 1 problem(s)
tests: ok
[exit status: 1]
$ git switch -q main
$ sh ci/check.sh origin/main~3
checks: 11 file version(s) examined, 0 problem(s)
checks: 24 file version(s) examined, 0 problem(s)
tests: ok
[exit status: 0]
```

Two design points. The hook and CI run the **same code** (`tools/checks.py`), so they cannot disagree about the rules. And the CI script checks two things: every file version the branch **adds** (`--range`), which catches a file that was committed and removed again within the branch and is therefore still in history, and the final snapshot (`--tree`). Both read blobs from Git, never the working tree, so they judge what is in history.

Layer	Runs where	Can the author skip it?	Role
Editor and local hook	the author's clone	yes: not installed, <code>--no-verify</code> , another client	fast feedback
CI job as a required check	GitHub Actions	no, if a ruleset requires it and bypass is restricted	the control for the branch
Push protection and push rulesets	GitHub, at push time	only through a logged bypass	stops secrets and oversized files before they are stored (push protection)

GitHub, not Git. A CI check catches a secret after it has been pushed: the commit is already on the server, and the credential must be rotated. Only a server-side control at push time (push protection, a push ruleset, or a `pre-receive` hook on a server you run) prevents the bytes from arriving. Use CI for policy, and push-time controls for secrets and size.

28.11 CI for ML projects: matrices, caches, GPU runners and LLM evaluations

In one sentence. The CI of an ML repository is ordinary until it needs a GPU, a multi-gigabyte model or a paid API key, and each of those three collides with a default of GitHub Actions.

The ordinary part. A Python version matrix, installs driven by the lock file, and a dependency cache. uv's guide pins `astral-sh/setup-uv` to a full commit SHA and installs with `uv sync --locked` ([uv with GitHub Actions](#)). This course's `workflows/04-python-tests.yml` is that job, with the action pins of 1 October 2026; Chapter 20A: GitHub Actions fundamentals explains every line. For `docqa`, one more step runs `sh ci/check.sh` with the base of the pull request, and the checkout needs `fetch-depth: 0` so that the range exists.

Three pressure points are specific to ML. All facts are GitHub's, from the report.

GPU capacity. GitHub-hosted GPU runners come in one shape, a 4-vCPU machine with a single T4. Larger runners, GPU included, are available only to organizations on Team and Enterprise Cloud plans and are always billed per minute, with no included minutes ([larger runners](#)).

That is enough for a smoke test of a small model, not for training. Teams that need more run self-hosted GPU runners, which is a security decision before it is a capacity decision (below).

Caches. Caches are free up to 10 GB per repository, and anyone who can open a pull request can read them, so a cache is not a place for anything sensitive. They suit package caches and small test models, not multi-gigabyte checkpoints. Fetch large models by pinned reference from the model store inside the job, or bake them into a runner image.

LLM evaluations. An evaluation that calls a model provider needs an API key in CI. `promptfoo`'s GitHub Action runs a before-and-after evaluation of edited prompts on pull requests, and `DeepEval` integrates with `pytest` and fails the build when a metric falls below a threshold; `Inspect AI`, `Ragas` and the `LangSmith SDK` are also maintained ([promptfoo](#), [DeepEval](#)). `CML`, once the usual way to post model metrics on pull requests, has had no release since October 2024; the report infers from repository activity that it is dormant, and no official statement says so ([CML releases](#)).

Such evaluations differ from unit tests in three ways the report names: they are non-deterministic, they cost money per run, and they need secrets. The first two shape how you use them:

Property	Consequence for the workflow
Non-deterministic	compare against a threshold or a baseline with a tolerance, never for equality; record model version, prompt version and seed with the result (section 28.7)
Costs money per run	trigger on paths that matter (<code>prompts/**</code> , <code>configs/**</code> , <code>evals/**</code>), cap the data set for pull requests, run the full suite on a schedule or on <code>main</code>
Needs secrets	see below

The collision with the fork model. Secrets are not passed to workflows triggered by `pull_request` from a fork (Chapter 27: Open source and team workflows, section 27.2). So the evaluation job fails on exactly the contributions an open-source LLM project most wants to evaluate. The tempting repair is to switch the trigger to `pull_request_target`, which runs with the base repository's secrets, and then check out the contributor's code. That combination executes untrusted code with your API keys and your token. The report identifies it as the pattern behind several compromises, some of them in ML projects (`PyTorch`'s self-hosted runners, `Ultralytics`, `LiteLLM`), and Chapter 21A: GitHub Actions security takes it apart.

pull_request from a fork	pull_request_target + checkout of the fork's code
+-----+ contributor's code runs no secrets, read-only token -> evaluation cannot call the model provider +-----+	+-----+ contributor's code runs WITH the base repository's secrets and token -> evaluation works, and so does "print the API key" in a changed test file +-----+

Defensible designs, from least to most machinery:

1. **Do not evaluate fork pull requests automatically.** Run cheap, secret-free checks on `pull_request` (lint, unit tests, `ci/check.sh`, evaluation against a local stub or recorded responses). A maintainer triggers the paid evaluation after reading the diff.
2. **Evaluate after merge.** Run the evaluation on `main` and alert or revert on regression.
3. **Separate data from code.** If the change is only to prompts or configuration, a trusted workflow can read those files as data and run the base repository's own evaluation code on them. This is safe only if nothing from the pull request is executed, which includes test files, `confptest.py`, package scripts and anything a prompt template can make the harness import.

Self-hosted GPU runners. They concentrate risk because they are expensive, long-lived and hold cached models and cloud credentials. The configuration the report calls defensible: private repositories or approval for all outside contributors, ephemeral runners that perform one job, runner groups scoped to named repositories, and OIDC instead of long-lived keys on the host. GitHub's own guidance is that self-hosted runners should almost never be used for public repositories.

In production. Give the evaluation job its own provider key with a spend cap, separate from production's. When that key leaks through a log or a compromised action, the damage is a bill with a ceiling, not production traffic.

28.12 A professional AI project repository

In one sentence. A good layout makes the rules of this chapter the path of least resistance: there is an obvious place for everything that belongs in Git and an ignored place for everything that does not.

Assembled, not authoritative. No single authoritative layout exists. Two reference points are well documented: `Cookiecutter Data Science v2` generates `data/` split into raw, interim, processed and external, a `notebooks/` directory with a naming convention, `models/`, `reports/` and an importable source package, and its ignore file ignores `/data/` ([CCDS](#)); and the `Python Packaging Authority` describes the trade-off between the flat and `src/` layouts without a blanket recommendation ([PyPA](#)). The tree below is this course's assembly for an LLM application, built from those and from the mechanisms above.

See it. `docqa`, as Lab 33.1 builds it from an empty folder. Everything Git tracks:

```
$ git ls-files
.dockerignore
.gitattributes
.gitignore
Dockerfile
README.md
ci/check.sh
configs/eval.json
data/README.md
data/raw/tickets.csv.ref
evals/run_eval.py
hooks/pre-commit
models/README.md
models/embedder.bin.ref
notebooks/01-error-analysis.ipynb
prompts/answer.v1.txt
pyproject.toml
requirements.lock
src/docqa/__init__.py
src/docqa/classify.py
tests/test_classify.py
tools/checks.py
tools/dataref.py
tools/nbstrip.py
tools/runinfo.py
```

And what is on disk and deliberately not tracked:

```
# On disk but deliberately not in Git:
$ git status --short --ignored
!! data/raw/tickets.csv
!! models/embedder.bin
!! runs/
$ git check-ignore -v data/raw/tickets.csv models/embedder.bin runs/baseline/run.json
.gitignore:17:/data/** data/raw/tickets.csv
.gitignore:23:/models/** models/embedder.bin
.gitignore:34:/runs/ runs/baseline/run.json
```

The tree, with every component explained.

docqa/	
-- .gitignore	what must never be added by accident (section 28.9)
-- .gitattributes	line endings, the notebook filter, pointers merged as binary
-- README.md	what the project is, and the commands a new clone must run once
-- pyproject.toml	package metadata and dependency ranges
-- requirements.lock	exact versions (uv.lock in a real project); committed (section 28.8)
-- Dockerfile	the serving image; base image passed in pinned by digest
-- .dockerignore	keeps .git, .env, data and runs out of the build context
-- src/docqa/	the importable package: all logic that deserves tests lives here
-- tests/	unit tests; fast, no network, no secrets
-- configs/	evaluation and model settings; reviewed like code
-- prompts/	prompt templates, one file per version; reviewed like code
-- evals/	the evaluation entry point; writes a run record (section 28.7)
-- notebooks/	exploration; committed without outputs (section 28.4)
-- data/	README and *.ref pointers only; the data itself is ignored
-- models/	README and *.ref pointers only; weights are ignored
-- tools/	repository tooling: filter, checks, pointer tool, run recorder
-- hooks/	shared Git hooks, enabled per clone with core.hooksPath
-- ci/	the one script CI runs, runnable locally
-- runs/	local run outputs; ignored (a tracker holds the durable copy)

Files the lab does not create and a real repository on GitHub should have: `.github/workflows/` for CI (section 28.11), `.github/CODEOWNERS` so that changes to workflows, lock files, prompts and the `Dockerfile` get a named reviewer (Chapter 19: CODEOWNERS), `.github/dependabot.yml` (section 28.13), `SECURITY.md`, a licence, and `.env.example` listing the variable names a developer must set, with no values.

The order of the history matters as much as the tree.

```
$ git log --oneline --decorate
aa18ad2 (HEAD -> main, tag: v0.1.0) Add serving image definition and model pointer
8a681ec Add evaluation config, prompt and run recorder
1117838 Version the ticket data set by reference
6be6418 Add notebook filter and the error-analysis notebook
ef4b558 Add repository checks, shared hook and CI entry point
8484ac0 Add package skeleton, lock file and tests
426268a Add ignore rules and attributes before any content
```

Read it from the bottom. The ignore rules and attributes are the **first** commit, before any content exists that they should have caught. The checks and the hook arrive before the first notebook. The notebook filter arrives in the same commit as the first notebook. The pointer tool

arrives with the first data. Each safeguard precedes the thing it guards, so there is no window in which a mistake can enter history. A repository that adds `.gitignore` in commit forty has thirty-nine commits to audit.

What a clone must do.

```
# What a clone has, and what it does not have:
$ git clone -q . ../docqa-asha && cd ../docqa-asha
$ git config get filter.nrstrip.clean
[exit status: 1]
$ git config get core.hooksPath
[exit status: 1]
$ python3 tools/dataref.py verify
missing data/raw/tickets.csv (pointer sha256:2226b23c455b)
missing models/embedder.bin (pointer sha256:7daca2095d04)
[exit status: 1]
```

No filter, no hook path, no data. This is why the README of `docqa` opens with the five commands a new clone runs once, and why CI repeats the checks. A `make setup` target or a bootstrap script that runs those commands is worth having; it is a convenience and still not a control.

Monorepo or one repository per model? The report found no ML-specific primary study, so this is a trade-off, not a finding. One repository gives atomic changes across training code, serving code and shared libraries and needs CI that runs only what changed (Chapter 24: Monorepos). Separate repositories give each team its own release rhythm and access control, and turn every shared library into a versioned dependency.

28.13 Model-serving repositories, Docker, and the Java and backend side

Serving. The report found no authoritative layout for model-serving repositories; what it found are three sourced building blocks, and the `Dockerfile` of `docqa` is shaped by them.

- **Secrets never enter the image through build arguments or environment variables.** Docker's documentation says both persist in the final image, and provides secret mounts for the purpose ([build secrets](#)). From that documentation, not run here:

```
docker build --secret id=aws,src=$HOME/.aws/credentials .
# in the Dockerfile: RUN --mount=type=secret,id=aws <command that needs the credentials>
```

- **Weights are fetched by pinned reference, never committed.** At build or start time the image downloads the revision or checksum recorded in the repository (section 28.6).
- **The base image is pinned by digest** (section 28.8), and `.dockerignore` keeps `.git`, `.env`, data and run outputs out of the build context, which is the Docker analogue of `.gitignore` and matters for the same reason: a `COPY . .` with a credential file in the context bakes it into a layer.

The link back to Git is the image label. Record the commit in the image (the OCI annotation `org.opencontainers.image.revision` exists for it) and tag images by commit ID or release tag, so that "which code is serving" has one answer.

Unverified. The annotation name `org.opencontainers.image.revision` is given from the author's knowledge of the OCI image specification; it is not in the report or its notes. Confirm it in the specification before you depend on it.

Java and backend notes. An ML platform is rarely Python alone: the service that calls the model is often Java. Three facts from the report matter.

- **Ignore rules.** GitHub's templates ignore `target/`, `build/`, `.gradle` and `*.class`. They treat the two build wrappers differently: the Maven wrapper JAR is ignored, while the Gradle template explicitly un-ignores `gradle-wrapper.jar`, which Gradle's documentation says is expected to be committed ([Maven template](#), [Gradle template](#), [Gradle Wrapper](#)).
- **The committed JAR is a review blind spot.** It is the one place in a typical Java repository where a pull request can change executable code that a reviewer cannot read in a diff. That is why `gradle/actions/wrapper-validation` exists, and why `setup-gradle` validates wrappers automatically since its v4 ([wrapper validation](#)). The parallel in this chapter: weights are also unreadable in a diff, which is one more reason they travel as checksums.
- **Dependabot needs one entry per ecosystem.** A polyglot repository lists `pip` or `uv`, `maven` or `gradle`, `github-actions`, and `docker` separately; `uv` has had version updates since 13 March 2025 and security updates since 16 December 2025, and `docker` and `pre-commit` receive version updates only ([supported ecosystems](#)).

Unverified. Maven's own documentation on distributing the wrapper was not fetched for the report; the statement above rests on GitHub's template.

28.14 Secrets in AI repositories

In one sentence. AI repositories leak the same way as others and through three extra paths: notebook outputs, agent and tool configuration files, and evaluation logs.

The scale, from the report. GitGuardian counted 28,649,024 new secrets on public GitHub in 2025, of which 1,275,105 were tied to AI services, up 81%; 24,008 unique secrets sat in MCP configuration files; and commits co-authored by Claude Code leaked secrets at roughly twice the baseline rate ([State of Secrets Sprawl 2026](#)). These are one vendor's measurements of public repositories.

Why an LLM key is worse than a bill. The report's example is an xAI key, hardcoded and pushed in 2025, that gave access to at least 60 private and fine-tuned models and was still valid two months after the first alert ([Krebs on Security](#)). What a provider key exposes depends on the provider and the key's scope; assume it is more than usage charges.

Leak path specific to AI work	Mechanism	Control
Notebook outputs	a cell prints a key or a data frame; the file stores it (section 28.3)	strip outputs; verify in CI
<code>.env</code> files	created for local runs, added by <code>git add .</code>	ignore rule; <code>.env.example</code> without values; push protection
Agent and tool configuration (MCP server files, editor agent settings)	a token pasted into a JSON or YAML file that looks like configuration, not like a credential	keep tokens in the environment or a secret manager; scan these paths
Evaluation logs and traces	request headers or full prompts logged and committed as "results"	keep run outputs out of Git (<code>runs/</code> is ignored); redact at the source
Prompts and fixtures	real customer text pasted into a test case	review <code>prompts/</code> and <code>tests/</code> like code; synthetic fixtures
CI	a key printed by a debugging step, or exfiltrated by untrusted code (section 28.11)	least privilege, spend caps, no secrets on fork pull requests

The rule that does not change. A commit that deletes a secret adds a snapshot and leaves the old one in history, in every clone and fork. The order is: revoke or rotate first, then find out what the key could reach, then clean up. Chapter 21B: Repository security has the full procedure. `git rm --cached .env` followed by a commit is not a response to a leak.

28.15 AI coding agents as participants in the workflow

In one sentence. An agent that writes commits is a contributor with a different Git client, unusual speed and no memory of your conventions, so the controls that matter are the ones that do not depend on the contributor.

What the report establishes. By October 2026 agents are first-class actors on GitHub: Copilot can review pull requests and, in preview, approve them, and the Copilot cloud agent opens pull requests and signs its commits. DORA's 2025 research describes AI as an amplifier of an organization's existing strengths and weaknesses, lists version control and working in small batches among the capabilities that matter, and recommends small, frequent commits for AI-generated code ([DORA 2025 report](#), [DORA: version control](#)).

What follows for the repository, as the report's inference:

Property of an agent	Consequence	Control
It may use its own Git client or the API	local hooks do not run	every hook that matters is also a required check (section 28.10)
It produces large changes quickly	review becomes the bottleneck, or stops being real	small pull requests; a human approval that an automated approval cannot replace, enforced by a ruleset
It reads whatever is in the repository and in the task	text in an issue, a pull request title or a file can steer it (prompt injection)	do not run agents automatically on untrusted contributions; no write token unless required
It holds credentials while it works	whatever it can read, an attacker who steers it can read	a dedicated low-privilege, spend-capped key; narrowly scoped tokens
It does not know your conventions	inconsistent branch names, messages, generated files committed	conventions in a committed instruction file, and checks that enforce them anyway

Two incidents from the report show the third and fourth rows are not hypothetical: an automated account ran a campaign in February and March 2026 that combined `pull_request_target` abuse, injection through branch names and filenames, and prompt injection against an AI reviewer ([StepSecurity](#)); and researchers reported in April 2026 that AI coding agents run as GitHub Actions could be steered by text in pull-request titles, issue bodies or comments into revealing CI secrets ([Techzine](#)).

Unverified. The April 2026 research was verified for the report only through that one secondary article.

Attribution. Decide how agent-written commits are marked: a Co-authored-by trailer, a dedicated bot account, or a signature. Then "which changes did an agent write" is a `git log` query (Chapter 27, section 27.18), and when a model version turns out to have introduced a class of bug you can find its commits.

The practical conclusion. Nothing in this chapter becomes less important when an agent writes the commits, and most of it becomes more important, because the volume goes up and the author cannot be asked what it meant.

28.16 What can go wrong

Symptom	Diagnosis	Fix	Prevention
A notebook nobody edited shows as modified on every run	outputs and counters are in the blob: <code>git cat-file -p HEAD:<nb> grep -c output_type</code>	configure the filter; <code>git add --renormalize .</code> ; commit	filter plus <code>--verify</code> as a required check
Outputs in history although "we use nbstripout"	the committing clone had no filter: <code>git config get filter.<name>.clean</code> exits 1 there	strip and commit; if a secret was printed, rotate it	setup script; CI check; <code>required = true</code>
<code>git add</code> fails with "clean filter ... failed"	the filter is required and its program is missing or broken	install the tool or fix the path in the clone's configuration	document setup in the README; do not unset <code>required</code> to make the error go away
Checking out a run's commit gives another metric	the run was made from a dirty tree, or with other data	apply the saved patch; restore the data version the record names	refuse dirty runs; record data checksum and diff
The metric changed and no commit did	the data file was edited in place: <code>dataref.py verify</code> reports "differs"	record the new version, or restore the old one	the evaluation refuses data that does not match its pointer
A pointer cannot be resolved ("NOT IN STORE")	the object was deleted from the store, or never pushed	restore from backup; otherwise the version is lost	retention rules; push data before pushing the pointer
The clone is slow and <code>.git</code> is large	a big blob in history: the <code>rev-list --objects</code> pipeline of section 28.9	history rewrite, coordinated; or accept and use partial clone (Chapter 26: Performance)	size checks in CI; a push ruleset on file size
A large or secret file reached <code>main</code> "despite the hooks"	the clone had no hook, or <code>--no-verify</code>	remove; rotate if secret	the same checks as a required CI job; push protection
The evaluation job fails on every fork pull request	secrets are withheld from fork runs	secret-free checks on <code>pull_request</code> ; a maintainer-triggered evaluation	design the split up front (section 28.11)
The package version in CI is <code>0.0.0</code> or the job fails in <code>git describe</code>	shallow checkout without tags	<code>fetch-depth: 0</code>	set it in jobs that derive versions

28.17 When not to use it, and dangerous edge cases

- **Do not strip outputs when the outputs are the deliverable.** A report notebook whose charts are the result needs the outputs; commit them on purpose and review them with a tool that can.
- **A clean filter makes `git status` lie by design.** The working file and the blob differ and Git reports no change. Anyone who copies the working file elsewhere (an email, a bucket, a Docker build context) copies the outputs too. Keep `notebooks/` out of the build context.
- **Pointers are not backups.** Versioning by reference moves the durability requirement to the store. A bucket with a lifecycle rule that expires old objects silently destroys old versions.
- **Do not use Git LFS for data that must be deletable.** The report notes that LFS objects are hard to remove from the server; personal data with a deletion obligation belongs in a store with real deletion.
- `dataref.py checkout`, like `dvc checkout`, **overwrites the working data file.** Unrecorded edits to the data are lost, with no reflog to recover them. Run `verify` first.
- **A dirty-tree record with a patch still omits untracked files.** A new module that was never added is not in `git diff HEAD`.
- `git add -f` **defeats every ignore rule**, and `git add .` after `git rm --cached` of a file that is no longer ignored re-adds it.
- **Renormalizing a large repository touches many files in one commit.** Do it in a commit of its own, announced, so that it does not hide in a feature diff and so that `git blame --ignore-rev` can skip it (Chapter 14A: History investigation).
- **This chapter's scripts are teaching instruments.** `nbstrip.py`, `dataref.py` and `checks.py` show mechanisms. In a real project use the maintained tools, which handle the cases forty lines cannot.

28.18 Command safety

Command	Label	What it changes	Preview	Recovery
<code>git check-ignore -v</code> , <code>git check-attr</code> , <code>git status --ignored</code> , <code>git</code>	🟢 SAFE	nothing	not needed	not needed

Command	Label	What it changes	Preview	Recovery
<code>cat-file -p <rev>:<path>, git describe</code>				
<code>git config set filter.<name>.clean ... git config set core.hooksPath hooks</code>	⚠ CAUTION	how later <code>git add</code> and <code>git commit</code> behave in this clone	<code>git config list --local</code>	<code>git config unset <key></code>
<code>git add <notebook> with a clean filter</code>	✅ SAFE	stages the cleaned form	<code>git diff</code> (compares cleaned forms)	<code>git restore --staged <path></code>
<code>git add --renormalize .</code>	⚠ CAUTION	re-cleans and restages every tracked file	<code>git status</code> afterwards, before committing	<code>git restore --staged .</code>
<code>git restore <notebook>, git switch</code> across a notebook change	🚫 DANGEROUS for local outputs	rewrites the file from the stripped blob	<code>git status</code> ; copy the file first	none: outputs were never in Git; rerun the notebook
<code>git rm --cached <path></code>	⚠ CAUTION	removes the path from the index; the next commit deletes it for everyone who pulls	<code>git status</code>	<code>git restore --staged <path></code> before committing
<code>git commit --no-verify</code>	⚠ CAUTION	commits without the pre-commit and commit-msg hooks	run the hook by hand: <code>hooks/pre-commit</code>	<code>git commit --amend</code> after fixing; CI is the backstop
<code>git worktree add --detach <dir> <commit></code>	✅ SAFE	a new directory and an entry under <code>.git/worktrees/</code>	<code>git worktree list</code>	<code>git worktree remove <dir></code>
<code>git apply <patch></code>	⚠ CAUTION	working tree files	<code>git apply --check <patch>; git apply --stat <patch></code>	<code>git restore <paths></code>
<code>python3 tools/dataref.py checkout</code> (and <code>dvc checkout</code>)	🚫 DANGEROUS for unrecorded data edits	overwrites data files with the versions the pointers name	<code>python3 tools/dataref.py verify</code>	none for unrecorded edits; record them first with <code>add</code>

For the two 🚫 rows: what they change and destroy is in the table; they are appropriate whenever the working copy holds nothing that exists only there, which is the normal case after `verify` reports "ok" or after outputs have been exported.

28.19 Version notes

Version note. Older behavior: `git config filter.x.clean ...` and `git config --unset`. Current behavior: `git config set`, `git config get`, `git config unset`, `git config list`. Since: Git 2.46. Recommended: the subcommands; the old forms still work.

Version note. Older behavior: Git LFS data packs on GitHub. Current behavior: metered billing (Chapter 22: Git LFS, section 22.11). Recommended: check the billing page before putting model files in LFS.

Version note. Older behavior: Hugging Face Hub repositories stored large files in Git LFS storage. Current behavior: Xet storage, with the LFS pointer format kept for compatibility. Since: migration reported complete by October 2025 (flagged above as not independently verified). Recommended: pin downloads by full commit hash either way.

Version note. Older behavior: DVC maintained by Iterative; lakeFS under Apache 2.0. Current behavior: DVC stewarded by lakeFS since 18 November 2025 and still Apache-2.0; lakeFS under the Business Source License from v1.87.0 (22 September 2026). Recommended: state the version when you describe either licence.

Version note. Older behavior: CML as the usual way to post model metrics on pull requests. Current behavior: no release since October 2024. Recommended: post results from your evaluation tool or a small script; do not build new pipelines on it without checking its status.

Tool versions quoted in this chapter (nbstripout 0.9.1, nbtime 4.0.4, jupyter 1.19.5, DVC 3.67.1, pre-commit 4.6.2, prek v0.5.4, uv 0.12.21, MLflow 3.16.1, W&B SDK 0.30.0, setuputils-scm 10.3.4, hatch-vcs 0.5.0, Git LFS client 3.8.0) are those the report recorded on 1 October 2026. None of the tools was run for this chapter.

28.20 Practice

- Lab 33.1: build `docqa` from an empty folder, in the order that keeps mistakes out of history; then commit a notebook from a clone without the filter, catch it, and repair it.
- Lab 33.2: version a data set by reference, move between versions, restore it in a clone, and catch an unrecorded edit.
- Lab 33.3: reproduce a past result from its recorded identifiers, including one made from a dirty tree.
- Lab 14.4 in the Module 14 labs is the smaller filter exercise this chapter builds on; the Module 22 labs cover Git LFS.
- Run the demos: `labs/run_ch28/notebook-filter`, `labs/run_ch28/run-record`, `labs/run_ch28/hooks-vs-ci`.

28.21 Interview questions

1. A run in the tracker names a commit. You check it out and get a different metric. List every cause you can think of, in the order you would test them, and the evidence for each.
2. Why does cloning a repository not install its clean filters and hooks? What would be possible for an attacker if it did?
3. Explain what a clean filter marked `required` does differently from one that is not, and why that matters for a filter that strips notebook outputs.
4. Your team "uses nbstripout", and a notebook with outputs is on `main`. How did it get there, and what do you change?
5. Compare Git LFS, DVC and a pointer file you maintain yourself. What is identical in all three, and what differs?
6. `.gitignore` lists `*.ckpt`, and `git status` shows `model.ckpt` as modified. Explain, fix it, and say what your fix did not fix.
7. Which files in an ML repository are generated and must be committed anyway, and what is the test that separates them from generated files that must not be?
8. An open-source LLM project wants evaluations on pull requests from forks. Describe the collision, the dangerous workaround, and two designs you would defend.
9. What does a local pre-commit hook guarantee? Name three ways a commit reaches the server without passing it, and the control for each.
10. An agent opens forty pull requests a day in your repository. Which of your existing controls still work, which do not, and what do you add?
11. A colleague deleted a leaked provider key in a follow-up commit and considers the incident closed. What is still true, and what is the first action?
12. Why is `merge=binary` a sensible attribute for a pointer file?

28.22 Sources

Primary sources

- Git 2.55 manual pages: `git help attributes` (filters, text, merge), `git help ignore`, `git help check-ignore`, `git help describe`, `git help status` (porcelain format), `git help worktree`, `git help apply`.
- Notebooks: [nbformat](#), [nbstripout](#), [nbdime](#) and its [Git integration](#), [jupyter](#), [ReviewNB](#), GitHub's [rich notebook diff preview](#).
- Data and models: [About large files on GitHub](#), [About Git LFS](#), [DVC: get started](#), [DVC joins lakeFS](#), [lakeFS licence change](#), [Hugging Face download guide](#), [MLflow model registry](#).
- Reproducibility: [MLflow system tags](#), [W&B code saving](#), [Hydra output directory](#), [uv project layout](#) and [locking](#), [Docker build best practices](#), [git-describe](#).
- Hooks and CI: [pre-commit](#), [pre-commit-hooks](#), [ruff-pre-commit](#), [uv with GitHub Actions](#), [larger runners](#), [push protection](#), [promptfoo GitHub Action](#), [DeepEval in CI](#).
- Structure, Docker and Java: [Cookiecutter Data Science](#), [PyPA on src and flat layouts](#), [Docker build secrets](#), [GitHub's Python ignore template](#), [Gradle Wrapper](#), [Dependabot ecosystems](#).

Secondary sources

- The Phase 0 report of this course, sections 14 and 15, and its notes on AI/ML workflows and security. Every version, date, licence and status in this chapter comes from there; its flags are carried as "Unverified" callouts.
- GitGuardian, [How to handle secrets in Jupyter notebooks](#) and [State of Secrets Sprawl 2026](#): one vendor's measurements.
- DORA, [2025 report](#) and [version control capability](#).
- StepSecurity and Techzine on agents in workflows, linked in section 28.15.

Videos (from the report's tables, with its caveats)

- [MLOps: Day 4 - Data Versioning using DVC](#), Vikash Das, in Hindi, 7 August 2024, 1 h 25 min: why Git alone does not suit data, DVC beside Git, an S3 remote, `dvc push` and `dvc pull`. Caveats: DVC CLI versions were not checked; no pipelines or CI.
- [How to build a ML project using MLOps](#), CampusX, in Hindi, 25 November 2024, 1 h 21 min. Use it as a concept map only: the report notes no workflow YAML, no `dvc push` or `dvc pull`, and no remote storage.

Further reading

- Chapter 14C for filters, drivers and hooks in full; Chapter 22 for Git LFS; Chapter 25: Worktrees for the worktree used in Lab 33.3.
- Chapter 27: Open source and team workflows for branching, release tags and the fork model that section 28.11 collides with.

Chapter 29: Production Troubleshooting

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/ch29/`.

29.1 Why this matters

A CTO does not ask "which command fixes this". The questions are: what happened, how do you know, what will the fix change, and how do we stop it from happening again. Every earlier chapter gave you one mechanism. This chapter is the method that selects the right one when all you have is a sentence from a colleague: "my commits are gone", "the push is rejected".

Three facts shape the method.

1. **Most damage is done after the incident, by the first repair attempt.** The original problem in a Git repository is rarely destructive: Git keeps unreachable commits for weeks (Chapter 13, section 13.4). The second command, typed before the state was understood, is what removes the uncommitted change or rewrites the published branch.
2. **A symptom names no layer.** "CI is red" may be a Git fact (the commit lacks a file), a GitHub fact (the check ran on the merge ref) or a GitHub Actions fact (the runner image changed). Evidence decides, not the wording of the complaint.
3. **Experience does not protect you.** In 2020, 40.0% of the people asking Git-command questions on Stack Exchange had been registered for more than five years, against 21.2% of all askers ([author preprint](#)). A fixed procedure is what replaces confidence.

This chapter does not repeat the mechanisms. It applies the root-cause framework of Chapter 1, section 1.10 to two new cases, reads interrupted operations from `git status` and `.git`, preserves evidence, lists what GitHub records, and ends with a symptom catalog that points to the chapter of each mechanism. The field version is the troubleshooting playbook.

29.2 The method

In one sentence. Diagnosis is a read-only search for the one fact about the state that explains the symptom, followed by the smallest change that repairs that fact.

Analogy. An accident investigator secures the site, records everything, reconstructs the sequence, and only then states a cause. The analogy breaks in one useful way: a repository can be copied in seconds, so you can rehearse the repair on the copy.

Precisely. The eleven steps of section 1.10 fall into three phases with different permissions.

PHASE 1: READ-ONLY		PHASE 2: PRESERVE	PHASE 3: CHANGE
SYMBOL			
OBSERVE	<code>git status</code>	record the output	SELECT LOWEST-RISK FIX
COLLECT EVIDENCE	the ten commands	backup ref	EXECUTE (preview, one step)
UNDERSTAND STATE	five places	copy of the repository	VERIFY (same commands)
FORM HYPOTHESES	at least three	bundle	PREVENT
TEST HYPOTHESES	one command each	(adds refs and files;	(moves refs, rewrites files,
IDENTIFY ROOT CAUSE	name the layer	destroys nothing)	talks to the server)

Phase 1 uses only  commands. Phase 2 adds refs and files and removes nothing. Phase 3 is the first moment a ref moves or a file is overwritten.

"Understand state" means being able to fill in this table for the repository in front of you. Each cell has a command that answers it.

Place	Question	Command
Working tree	Which files differ from the index? Which are untracked or ignored?	<code>git status</code> , <code>git diff</code> , <code>git status --ignored</code>
Index	What would the next commit record? Are there conflict stages?	<code>git diff --cached</code> , <code>git ls-files -s</code> , <code>git ls-files -u</code>
HEAD	A branch, or a raw commit ID? Is an operation in progress?	<code>git status</code> , <code>cat .git/HEAD</code> , <code>git symbolic-ref HEAD</code>
Refs	Where does each branch, tag and remote-tracking branch point, and what is each upstream?	<code>git branch -vv</code> , <code>git for-each-ref</code>
History of the refs	How did they get there?	<code>git reflog</code> , <code>git reflog show <branch></code>
The server	What does it hold now, as opposed to at the last fetch?	<code>git ls-remote origin</code>
GitHub	What did the platform record: pushes, rule evaluations, checks?	Section 29.8

A hypothesis is a mechanism plus a prediction: "if the branch was reset, the branch reflog has a `reset: moving to entry`". Write at least three before testing any. With one hypothesis you look for confirmation; with three you look for the command whose output separates them.

In production. Ask the reporter for the exact text on the screen, the last three commands they remember, and whether anything was done since. The scrollbar often contains the commit ID you will need (Chapter 13, section 13.7).

29.3 Worked case 1: the push that had nothing to push

The repository is `scoring-api`, a service that scores feature vectors with a ranking model. A bare repository on disk plays the server. The script is `labs/ch29/case-unfinished-rebase.sh`.

SYMPTOM. A developer reports: "I added two commits to my pull request branch `feature/latency-budget` this week. The pull request does not show them. I pushed again and Git says there is nothing to push."

```
$ git push origin feature/latency-budget
Everything up-to-date
```

"Everything up-to-date" is a statement about one ref: the local and the server's `feature/latency-budget` name the same commit. It says nothing about where the two new commits are.

OBSERVE and COLLECT EVIDENCE. The ten commands, in order. For each: what it reveals, and what to look for.

1. `git status`. Look at the first line before anything else. It names the current branch, or says that HEAD is detached, or names an operation in progress.

```
$ git status
interactive rebase in progress; onto 4b30d7e
Last command done (1 command done):
  pick 60d09a1 # Add latency budget to config
Next commands to do (2 remaining commands):
  pick cd8451c # Enforce latency budget in scorer
  pick 4a03014 # Document latency budget
(use "git rebase --edit-todo" to view and edit)
You are currently editing a commit while rebasing branch 'feature/latency-budget' on '4b30d7e'.
(use "git commit --amend" to amend the current commit)
(use "git rebase --continue" once you are satisfied with your changes)

nothing to commit, working tree clean
```

The first line is not "On branch". A rebase of `feature/latency-budget` onto `4b30d7e` was started and never finished: one command done, two remaining. The working tree is clean, so nothing uncommitted is at stake.

2. `git branch -vv`. Look for the star, for the upstream in brackets, and for `ahead`, `behind` and `gone`.

```
$ git branch -vv
* (no branch, rebasing feature/latency-budget) 075407e Log budget violations
  feature/latency-budget                      4a03014 [origin/feature/latency-budget] Document latency budget
  main                                         4b30d7e [origin/main] Raise worker count to 8
```

The star is on `(no branch, rebasing feature/latency-budget)`. The branch `feature/latency-budget` still points at `4a03014`, with no `ahead`: it equals its upstream. HEAD is on `075407e`, a commit that no branch names.

3. `git remote -v`. Look for more than one remote, and for fetch and push URLs that differ.

```
$ git remote -v
origin ../server.git (fetch)
origin ../server.git (push)
```

One remote, one URL.

4. `git log --graph --decorate --oneline --all`. Look for where each name sits, and for commits with no name beside them.

```
$ git log --graph --decorate --oneline --all
* 075407e (HEAD) Log budget violations
* 96a7b55 Add p95 latency metric
* 9f9e4d7 Add latency budget to config
* 4b30d7e (origin/main, main) Raise worker count to 8
| * 4a03014 (origin/feature/latency-budget, feature/latency-budget) Document latency budget
| * cd8451c Enforce latency budget in scorer
| * 60d09a1 Add latency budget to config
|/
* 8bd6e7d Add request schema
* 05ba901 Add scoring service skeleton
```


Two lines of history leave 8bd6e7d. The lower one is the published branch, ending at 4a03014. The upper one starts at 4b30d7e on main and holds a second "Add latency budget to config" followed by the two commits the developer is looking for. Only HEAD decorates it.

5. `git reflog`. Read it from the bottom up: it is the sequence of events. Look for `rebase`, `reset`, `checkout` and `commit (amend)` entries.

```
$ git reflog
075407e HEAD@{0}: commit: Log budget violations
96a7b55 HEAD@{1}: commit: Add p95 latency metric
9fbe4d7 HEAD@{2}: commit: Add latency budget to config
4b30d7e HEAD@{3}: rebase (start): checkout main
4a03014 HEAD@{4}: checkout: moving from main to feature/latency-budget
4b30d7e HEAD@{5}: commit: Raise worker count to 8
8bd6e7d HEAD@{6}: checkout: moving from feature/latency-budget to main
4a03014 HEAD@{7}: commit: Document latency budget
cd8451c HEAD@{8}: commit: Enforce latency budget in scorer
60d09a1 HEAD@{9}: commit: Add latency budget to config
8bd6e7d HEAD@{10}: checkout: moving from main to feature/latency-budget
8bd6e7d HEAD@{11}: commit: Add request schema
05ba901 HEAD@{12}: commit (initial): Add scoring service skeleton
```

HEAD@{3} is `rebase (start): checkout main`. There is no `rebase (finish)` line above it. The three entries above the start are plain `commit` entries: one for the conflict resolution, two for new work.

6. `git rev-parse`. Use it to turn the names of the previous outputs into exact IDs, and to ask whether HEAD is a branch.

```
$ git rev-parse HEAD feature/latency-budget origin/feature/latency-budget
075407e71d9b34006eccefd0d2072bfb51d2850b
4a030148e74b6b4394d1056737434790a34ae612
4a030148e74b6b4394d1056737434790a34ae612
$ git rev-parse --abbrev-ref HEAD
HEAD
$ git symbolic-ref HEAD
fatal: ref HEAD is not a symbolic ref
[exit status: 128]
```

The local branch and its remote-tracking branch are the same object, which is why the push had nothing to send. `--abbrev-ref HEAD` prints HEAD, and `git symbolic-ref HEAD` fails: HEAD holds a commit ID, not the name of a branch.

7. `git show --stat HEAD`. Look at the author, the date and the list of files: is this the commit the reporter means?

```
$ git show --stat HEAD
commit 075407e71d9b34006eccefd0d2072bfb51d2850b
Author: Lab User <you@example.com>
Date:   Mon Sep 7 10:21:00 2026 +0530

    Log budget violations

 app/audit.py | 7 ++++++
 1 file changed, 7 insertions(+)
```

8 and 9. `git diff` and `git diff --cached`. Look for work that exists in no commit. It is the work that a wrong fix destroys without a trace.

```
$ git diff
$ git diff --cached
```

Both are empty. Everything of value here is committed.

10. `git config list --show-origin --show-scope`. Look for the `branch.<name>.remote` and `branch.<name>.merge` pairs, for `pull.*`, `push.*` and `rebase.*` settings, and for the scope each one comes from.

```
$ git config list --show-origin --show-scope
global file:$LAB/ch29/case-unfinished-rebase/home/.gitconfig user.name=Lab User
global file:$LAB/ch29/case-unfinished-rebase/home/.gitconfig user.email=you@example.com
global file:$LAB/ch29/case-unfinished-rebase/home/.gitconfig init.defaultbranch=main
global file:$LAB/ch29/case-unfinished-rebase/home/.gitconfig gc.reflogexpire=never
global file:$LAB/ch29/case-unfinished-rebase/home/.gitconfig gc.reflogexpireunreachable=never
local file:./git/config core.repositoryformatversion=0
local file:./git/config core.filemode=true
local file:./git/config core.bare=false
local file:./git/config core.logallrefupdates=true
local file:./git/config core.ignorecase=true
local file:./git/config core.precomposeunicode=true
local file:./git/config remote.origin.url=../server.git
local file:./git/config remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
local file:./git/config branch.main.remote=origin
local file:./git/config branch.main.merge=refs/heads/main
local file:./git/config branch.feature/latency-budget.remote=origin
local file:./git/config branch.feature/latency-budget.merge=refs/heads/feature/latency-budget
```


The upstream of `feature/latency-budget` is the branch of the same name on `origin`. No setting changes the behavior of push or rebase.

11. `git ls-files`. Look for the files the reporter talks about: are they tracked at the current commit?

```
$ git ls-files
README.md
app/audit.py
app/metrics.py
app/schema.py
app/score.py
config/service.yaml
```

`app/metrics.py` and `app/audit.py`, the files of the two new commits, are in the index. The work exists.

UNDERSTAND STATE.

scoring-api (your clone)	server.git
9fbe4d7---96a7b55---075407e (HEAD, no name)	
8bd6e7d---4b30d7e main, origin/main	8bd6e7d---4b30d7e main
60d09a1---cd8451c---4a03014 feature/latency-budget origin/feature/...	60d09a1---cd8451c--- 4a03014 feature/...
.git/rebase-merge/ 1 done, 2 to do	

FORM HYPOTHESES and TEST HYPOTHESES.

Hypothesis	Evidence that would confirm it	Result
H1: the commits were made on another branch	<code>git branch -a --contains HEAD</code> lists a branch	Rejected: it lists only the detached HEAD
H2: a rebase stopped; HEAD is detached; the commits were made on the detached HEAD; the branch ref does not move until the rebase finishes	<code>.git/rebase-merge/</code> exists and names the branch; the commits are in <code>feature/latency-budget..HEAD</code>	Confirmed
H3: the commits were pushed and the pull request page is stale	<code>git ls-remote</code> shows a newer ID than the local branch	Rejected: the server holds <code>4a03014</code>
H4: the push went to another repository	A second remote, or a push URL that differs	Rejected by output 3

A rebase keeps its state in a directory, and the directory answers H2 without interpretation:

```
$ cat .git/HEAD
075407e71d9b34006eccefd0d2072bfb51d2850b
$ ls .git/rebase-merge
author-script
done
drop_redundant_commits
end
git-rebase-todo
git-rebase-todo.backup
head-name
interactive
message
msgnum
no-reschedule-failed-exec
onto
orig-head
patch
stopped-sha
$ cat .git/rebase-merge/head-name
refs/heads/feature/latency-budget
$ cat .git/rebase-merge/orig-head
4a030148e74b6b4394d1056737434790a34ae612
$ cat .git/rebase-merge/onto
4b30d7e307a39954609eb40c2b4c2a1f63c74857
$ cat .git/rebase-merge/done
pick 60d09a1ac0fe504f55aaa293966229a4a3ea7d44 # Add latency budget to config
$ cat .git/rebase-merge/git-rebase-todo
pick cd8451cbbf3435915b50dd0e6db7a492777fbdd9 # Enforce latency budget in scorer
pick 4a030148e74b6b4394d1056737434790a34ae612 # Document latency budget
```

`head-name` is the branch that will be moved when the rebase ends, `orig-head` is where it was when the rebase began, `onto` is the new base, `done` and `git-rebase-todo` are the two halves of the plan. The branch ref is updated in one step at the end (Chapter 9, section 9.4).

```
# H2: which commits does HEAD have that the branch does not?
$ git log --oneline feature/latency-budget..HEAD
075407e Log budget violations
96a7b55 Add p95 latency metric
```

```

9fbe4d7 Add latency budget to config
4b30d7e Raise worker count to 8
# H4: what does the server hold?
$ git ls-remote origin
4b30d7e307a39954609eb40c2b4c2a1f63c74857      HEAD
4a030148e74b6b4394d1056737434790a34ae612      refs/heads/feature/latency-budget
4b30d7e307a39954609eb40c2b4c2a1f63c74857      refs/heads/main
# Where is each new commit reachable from?
$ git branch -a --contains HEAD
* (no branch, rebasing feature/latency-budget)

```

IDENTIFY ROOT CAUSE.

```

Observed behavior : git push prints "Everything up-to-date"; two commits are missing from the pull request.
Git state         : HEAD is detached at 075407e. .git/rebase-merge exists: 1 of 3 picks done.
                   refs/heads/feature/latency-budget = origin/feature/latency-budget = 4a03014.
Mechanism         : git rebase detaches HEAD, replays commits on it, and moves the branch ref only
                   when the last pick succeeds. The rebase stopped at a conflict. The conflict was
                   resolved with git commit, and git rebase --continue was never run. Later
                   commits extended the detached HEAD. git push <remote> <branch> sends the ref,
                   and the ref had not moved.
Root cause        : An unfinished rebase. The developer worked for days inside it.
Why Git does this : Moving the branch only at the end is what makes git rebase --abort possible:
                   the original branch stays intact until the whole replay has succeeded.
Correct fix        : Anchor the detached commits, abort the rebase, add the two new commits to the
                   published branch, push. Nothing published is rewritten.
Prevention        : Read the first line of git status before committing and before pushing.
                   Show the Git state in the shell prompt.

```

The layer is Git. GitHub displayed the branch it was given.

PRESERVE. Before any change, give the unnamed commits a name. `git branch` 🟢 adds one ref and touches nothing else; section 29.7 adds the heavier layers.

```

$ git branch rescue/latency-wip HEAD
$ git log --oneline -4 rescue/latency-wip
075407e Log budget violations
96a7b55 Add p95 latency metric
9fbe4d7 Add latency budget to config
4b30d7e Raise worker count to 8

```

SELECT LOWEST-RISK FIX.

Option	What it changes	Risk	Verdict
A. <code>git rebase --continue</code> , then <code>git push --force-with-lease</code>	Finishes the replay: the branch gets six new commit IDs on top of <code>main</code>	🟡 then 🔴 : rewrites a published branch with an open pull request; review comments become outdated (Chapter 17, section 17.12); two more picks may conflict	Rejected for now. It is a valid choice only if the team rebases pull request branches, and then as a separate, announced step
B. <code>git rebase --abort</code> and nothing else	Returns HEAD and the branch to <code>4a03014</code>	🔴 for the new work: the three commits lose their only anchor and live in the reflog alone	Rejected
C. Anchor, abort, <code>git cherry-pick</code> the two new commits, push	Two new commits on top of the published branch	🟡 : only adds; the push is a fast-forward	Chosen

Option C does not bring `main` into the branch. That is a separate decision (Chapter 9, section 9.18); `rescue/latency-wip` keeps the conflict resolution for it.

EXECUTE. One change at a time. `git rebase --abort` 🟡 resets the index and the working tree to the original branch tip, so it is run only because outputs 8 and 9 were empty.

```

$ git rebase --abort
$ git status
On branch feature/latency-budget
Your branch is up to date with 'origin/feature/latency-budget'.

nothing to commit, working tree clean
$ git log --oneline -3
4a03014 Document latency budget
cd8451c Enforce latency budget in scorer
60d09a1 Add latency budget to config

```

```

$ git cherry-pick rescue/latency-wip~1 rescue/latency-wip
[feature/latency-budget c54eb8c] Add p95 latency metric
Date: Mon Sep 7 10:20:00 2026 +0530
1 file changed, 3 insertions(+)
create mode 100644 app/metrics.py
[feature/latency-budget 30e8fe4] Log budget violations

```

```
Date: Mon Sep 7 10:21:00 2026 +0530
1 file changed, 7 insertions(+)
create mode 100644 app/audit.py
$ git log --oneline origin/feature/latency-budget..HEAD
30e8fe4 Log budget violations
c54eb8c Add p95 latency metric
```

`git cherry-pick`  created `c54eb8c` and `30e8fe4`: the same changes as `96a7b55` and `075407e` with a different parent, and therefore different IDs (Chapter 10, section 10.3). Preview the push, then push:

```
$ git push --dry-run origin feature/latency-budget
To ../server.git
4a03014..30e8fe4 feature/latency-budget -> feature/latency-budget
$ git push origin feature/latency-budget
To ../server.git
4a03014..30e8fe4 feature/latency-budget -> feature/latency-budget
```

`4a03014..30e8fe4` with two dots is a fast-forward. A forced update would print a plus sign and three dots.

VERIFY. Run the commands that showed the problem and check that each output changed for the predicted reason.

```
$ git status
On branch feature/latency-budget
Your branch is up to date with 'origin/feature/latency-budget'.

nothing to commit, working tree clean
$ git rev-parse HEAD origin/feature/latency-budget
30e8fe42deeb7bc3f85ed0183c53b888716f22a6
30e8fe42deeb7bc3f85ed0183c53b888716f22a6
$ git ls-remote origin feature/latency-budget
30e8fe42deeb7bc3f85ed0183c53b888716f22a6 refs/heads/feature/latency-budget
$ git cherry -v HEAD rescue/latency-wip
+ 4b30d7e307a39954609eb40c2b4c2a1f63c74857 Raise worker count to 8
+ 9f9be4d7d5ad2de0af7c53ab650f5128730a19457 Add latency budget to config
- 96a7b550d08f1c63c7d6f57c14c2463a471a390e Add p95 latency metric
- 075407e71d9b34006eccefd0d2072bfb51d2850b Log budget violations
$ ls .git | grep -c rebase
0
```

The first line of `git status` is "On branch". `HEAD`, the remote-tracking branch and the server agree on `30e8fe4`. `git cherry -v HEAD rescue/latency-wip` compares by patch: the minus sign marks the two rescued commits as present on the branch in equivalent form; the plus sign marks the commit of `main` and the conflict resolution `9f9be4d7`, which were deliberately left out. No rebase directory remains.

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git rebase --abort</code> 	Reset to the original branch tip; uncommitted changes to tracked files are discarded	Reset	Re-attached to the branch named in <code>head-name</code>	Left at <code>orig-head</code>	<code>.git/rebase-merge/</code> removed; reflog entry <code>rebase (abort)</code>	unchanged	unchanged
<code>git cherry-pick <a></code> <code></code> 	Updated	Updated	Moves with the branch	Advances by two new commits	Reflog entries; <code>CHERRY_PICK_HEAD</code> and <code>.git/sequencer/</code> during a conflict	unchanged	unchanged

PREVENT. Make a stopped operation visible where people look. Git ships a prompt script, `git-prompt.sh`, whose `__git_ps1` function prints the branch and the operation in progress (for example `|REBASE 1/3`). On this Mac, Homebrew's Git installs it as `/opt/homebrew/etc/bash_completion.d/git-prompt.sh`. The habit that needs no tool: the first line of `git status` before every commit and every push.

29.4 Worked case 2: "I pulled, and the push is still rejected"

The repository is `embed-jobs`, a batch job that computes embeddings. The script is `labs/ch29/case-wrong-upstream.sh`. The case is shorter on purpose: once the ritual is a habit, you read the same outputs faster.

SYMPTOM. "My push is rejected. Git tells me to pull. I pull, Git says everything is up to date, and the push is rejected again."

```
$ git push origin feature/batch-size
To ../server.git
! [rejected] feature/batch-size -> feature/batch-size (fetch first)
error: failed to push some refs to '../server.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
```

```
$ git pull
From ../server
 * [new branch]      feature/batch-size -> origin/feature/batch-size
Already up to date.
$ git push origin feature/batch-size
To ../server.git
 ! [rejected]        feature/batch-size -> feature/batch-size (non-fast-forward)
error: failed to push some refs to '../server.git'
hint: Updates were rejected because the tip of your current branch is behind
hint: its remote counterpart. If you want to integrate the remote changes,
hint: use 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
[exit status: 1]
```

The first rejection says (fetch first): the server's branch has an object this clone has never seen. The pull prints * [new branch] feature/batch-size -> origin/feature/batch-size: this clone learns of the server's branch for the first time, and then says "Already up to date". The second rejection says (non-fast-forward): now the clone has the server's commit and can see that the local branch does not contain it. The pull fetched the commit and did not integrate it.

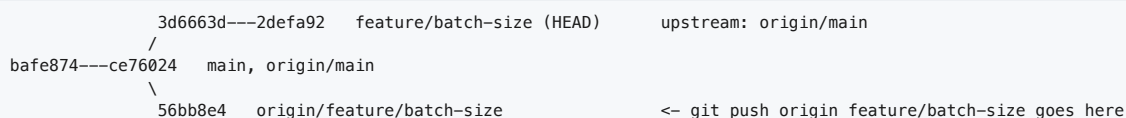
COLLECT EVIDENCE.

```
$ git status
On branch feature/batch-size
Your branch is ahead of 'origin/main' by 2 commits.
  (use "git push" to publish your local commits)

nothing to commit, working tree clean
$ git branch -vv
* feature/batch-size 2defa92 [origin/main: ahead 2] Read batch size from the command line
  main                ce76024 [origin/main] Write vectors to the daily bucket
$ git log --graph --decorate --oneline --all
* 56bb8e4 (origin/feature/batch-size) Add batch_size to the job config
| * 2defa92 (HEAD -> feature/batch-size) Read batch size from the command line
| * 3d6663d Embed in batches
|/
* ce76024 (origin/main, origin/HEAD, main) Write vectors to the daily bucket
* bafe874 Add embedding job
```

```
$ git config list --show-origin --show-scope | grep -E "(branch|remote)[.]"
local file:./git/config      remote.origin.url=../server.git
local file:./git/config      remote.origin.fetch=+refs/heads/*:refs/remotes/origin/*
local file:./git/config      branch.main.remote=origin
local file:./git/config      branch.main.merge=refs/heads/main
local file:./git/config      branch.feature/batch-size.remote=origin
local file:./git/config      branch.feature/batch-size.merge=refs/heads/main
$ git reflog -4
2defa92 HEAD@{0}: commit: Read batch size from the command line
3d6663d HEAD@{1}: commit: Embed in batches
ce76024 HEAD@{2}: checkout: moving from main to feature/batch-size
ce76024 HEAD@{3}: clone: from $LAB/ch29/case-wrong-upstream/server.git
```

UNDERSTAND STATE. The status line compares the branch with origin/main, not with origin/feature/batch-size. git branch -vv shows the same in brackets. The configuration holds the cause in one line: branch.feature/batch-size.merge=refs/heads/main. The graph shows two lines of work leaving ce76024: two local commits, and one commit by a teammate on the server's branch of the same name.



FORM HYPOTHESES and TEST HYPOTHESES.

Hypothesis	Evidence that would confirm it	Result
H1: the server's branch has a commit the local branch lacks, and git pull integrates a different branch	HEAD..origin/feature/batch-size is not empty; branch.<name>.merge names another branch	Confirmed
H2: a rule on the server rejects the push	The line would read ! [remote rejected] with remote: lines (Chapter 12, section 12.15)	Rejected: ! [rejected] is written by the local Git before any rule is consulted
H3: someone rewrote the server's branch	The fetch would print + ... (forced update); the reflog of the remote-tracking branch would show the old tip	Rejected: one reflog entry, a first fetch
H4: the pull failed	An error from git pull	Rejected: it succeeded, on another branch

```
# H1: the server branch has a commit that the local branch lacks.
$ git log --oneline HEAD..origin/feature/batch-size
56bb8e4 Add batch_size to the job config
```

```
# H1: and git pull integrates another branch.
$ git config get branch.feature/batch-size.merge
refs/heads/main
# H3: was the server branch rewritten? Its remote-tracking reflog has one entry, a first fetch.
$ git reflog show origin/feature/batch-size
56bb8e4 refs/remotes/origin/feature/batch-size@{0}: pull: storing head
# Would the two lines of work conflict? A test merge that touches nothing:
$ git merge-tree --write-tree --name-only HEAD origin/feature/batch-size
e2386b3e50aabb62956465df6e66d79d2d11040
[exit status: 0]
```

The last command is a test merge with `git merge-tree` (Chapter 8, section 8.17). It writes a tree object and touches neither the index nor the working tree. Exit status 0 with no file names means the two lines of work combine without conflict.

IDENTIFY ROOT CAUSE.

```
Observed behavior : push rejected; git pull says "Already up to date"; push rejected again.
Git state         : feature/batch-size is 2 ahead of origin/main and 2 ahead, 1 behind
                   origin/feature/batch-size. branch.feature/batch-size.merge = refs/heads/main.
Mechanism         : git switch -c <name> origin/main records origin/main as the upstream, because
                   the starting point is a remote-tracking branch (branch.autoSetupMerge=true).
                   git pull without arguments integrates the upstream. git push origin <name>
                   targets the server branch <name>. Pull and push address different branches.
Root cause       : The upstream of the local branch is not the branch it is pushed to.
Why Git does this : Tracking the starting point is right for a local copy of a remote branch
                   (git switch -c main origin/main). Git cannot know that this time the
                   starting point was meant only as a base.
Correct fix       : Point the upstream at origin/feature/batch-size, replay the two unpublished
                   commits on top of it, push. No force.
Prevention       : Create feature branches with --no-track, or from the local main. Read the
                   brackets in git branch -vv before the first push.
```

SELECT LOWEST-RISK FIX.

Option	Risk	Verdict
<code>git push --force origin feature/batch-size</code>	🔴 replaces the server's branch and discards the teammate's commit 56bb8e4 (Chapter 12, section 12.8)	Rejected. A rejected push is information, not an obstacle
<code>git pull origin feature/batch-size</code>	🟡 a one-off integration; stops with "Need to specify how to reconcile divergent branches" unless a mode is given; leaves the wrong upstream in place, so the problem returns	Rejected: treats the state, not the cause
Set the upstream, then <code>git rebase</code> onto it, then <code>git push</code>	🟡 rewrites two commits that exist nowhere else; adds to the server's branch	Chosen

Rebasing is acceptable here because the two commits are unpublished: no other repository has them (Chapter 9, section 9.15).

EXECUTE. A backup ref first; then the configuration change; then the rebase. `git rebase` without arguments uses the upstream.

```
$ git branch backup/batch-size-before-rebase
$ git branch --set-upstream-to=origin/feature/batch-size
branch 'feature/batch-size' set up to track 'origin/feature/batch-size'.
$ git status -sb
## feature/batch-size...origin/feature/batch-size [ahead 2, behind 1]
$ git rebase
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feature/batch-size.
$ git log --graph --oneline -4
* aca1b2f Read batch size from the command line
* 097fe6f Embed in batches
* 56bb8e4 Add batch_size to the job config
* ce76024 Write vectors to the daily bucket
```

After the upstream change, `git status -sb` already tells the truth: `[ahead 2, behind 1]`. No object moved; only the comparison did.

```
$ git push --dry-run
To ../server.git
 56bb8e4..aca1b2f feature/batch-size -> feature/batch-size
$ git push
To ../server.git
 56bb8e4..aca1b2f feature/batch-size -> feature/batch-size
```

VERIFY.

```
$ git status
On branch feature/batch-size
Your branch is up to date with 'origin/feature/batch-size'.

nothing to commit, working tree clean
```

```
$ git branch -vv
backup/batch-size-before-rebase 2defa92 Read batch size from the command line
* feature/batch-size              aca1b2f [origin/feature/batch-size] Read batch size from the command line
main                             ce76024 [origin/main] Write vectors to the daily bucket
$ git rev-parse HEAD @{upstream}
aca1b2fe23a5fed73fb916e3c34ed1ba7095dec0
aca1b2fe23a5fed73fb916e3c34ed1ba7095dec0
$ git ls-remote origin feature/batch-size
aca1b2fe23a5fed73fb916e3c34ed1ba7095dec0      refs/heads/feature/batch-size
$ git range-diff origin/main backup/batch-size-before-rebase HEAD
-: ----- > 1: 56bb8e4 Add batch_size to the job config
1: 3d6663d = 2: 097fe6f Embed in batches
2: 2defa92 = 3: aca1b2f Read batch size from the command line
$ git pull
Already up to date.
```

The brackets name the right branch, HEAD equals the upstream and the server, and `git range-diff` (Chapter 9, section 9.14) shows the two commits carried over unchanged (=) on top of one new commit. `git pull` says "Already up to date" again, this time about the branch you push to.

PREVENT.

```
$ git branch -D backup/batch-size-before-rebase
Deleted branch backup/batch-size-before-rebase (was 2defa92).
# A new branch from origin/main that does not take origin/main as its upstream:
$ git switch -c feature/shard-output --no-track origin/main
Switched to a new branch 'feature/shard-output'
$ git branch -vv
feature/batch-size  aca1b2f [origin/feature/batch-size] Read batch size from the command line
* feature/shard-output ce76024 Write vectors to the daily bucket
main               ce76024 [origin/main] Write vectors to the daily bucket
$ git push
fatal: The current branch feature/shard-output has no upstream branch.
To push the current branch and set the remote as upstream, use

    git push --set-upstream origin feature/shard-output

To have this happen automatically for branches without a tracking
upstream, see 'push.autoSetupRemote' in 'git help config'.

[exit status: 128]
```

With `--no-track` the new branch has no upstream, and the first `git push` stops and asks; `push.autoSetupRemote=true` makes that push create the upstream under the branch's own name. The manual for Git 2.55 also documents `branch.autoSetupMerge=simple`: automatic setup "only when the starting point is a remote-tracking branch and the new branch has the same name as the remote branch".

Operation	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git rebase</code> (onto the upstream) 🟡	Updated to the rebased tip	Updated	Detached during the replay, re-attached at the end	Set to the last replayed commit	<code>ORIG_HEAD</code> written; relog entries; <code>.git/rebase-merge/</code> during the operation	unchanged	unchanged

29.5 The extended toolbox

The ten commands are the fixed opening. The commands below are the ones you reach for when testing a hypothesis. All are 🟢. Each row names the question it answers and the chapter that teaches it.

Question	Command	Chapter
Branch, upstream and counts in one line	<code>git status -sb</code>	4, 4.4
Every ref with its upstream and distance	<code>git for-each-ref --format='% (refname) %(objectname:short) % (upstream:short) %(upstream:track) '</code>	3, 3.9
What is my upstream; where would a push go	<code>git rev-parse --abbrev-ref @{upstream}, @{push}</code>	12, 12.5
How far apart are two commits	<code>git rev-list --left-right --count A...B</code>	7, 7.8
Where two histories split	<code>git merge-base A B</code>	8, 8.2
Which branches or tags contain a commit	<code>git branch -a --contains <id>, git tag --contains <id></code>	12, 12.14
Is the same change present under another ID	<code>git cherry -v <upstream> <branch>, git range-diff</code>	10, 10.10; 9, 9.14
What the server holds right now	<code>git ls-remote origin</code>	12, 12.14

Question	Command	Chapter
When and how a ref moved	<code>git reflog show --date=iso <ref></code>	13, 13.3
What the index holds, with stages	<code>git ls-files -s</code> , <code>git ls-files -u</code>	5, 5.11, 5.13
Why a path is ignored	<code>git check-ignore -v <path></code> , <code>git status --ignored</code>	4, 4.5
Where one setting comes from	<code>git config get --show-origin --show-scope --all <key></code>	14B, 14B.3
Would a merge conflict, without touching anything	<code>git merge-tree --write-tree --name-only A B</code>	8, 8.17
Is the object store intact; what is unreachable	<code>git fsck</code> , <code>git fsck --lost-found</code> (the second writes files under <code>.git/lost-found/</code>)	13, 13.6
Stashes and other worktrees that hold work	<code>git stash list</code> , <code>git worktree list</code>	11, 11.11; 25, 25.2
What Git executes and sends	<code>GIT_TRACE=1</code> , <code>GIT_TRACE2_EVENT</code> , <code>GIT_CURL_VERBOSE=1</code> , <code>ssh -v</code>	14B, 14B.7; 16, 16.17

See it. Four of them on the repository of case 2, before the fix. The questions are "which refs exist and what do they follow" and "how do the two branches relate".

```
$ git status -sb
## feature/batch-size...origin/main [ahead 2]
$ git for-each-ref --format=%(refname) %(objectname:short) %(upstream:short) %(upstream:track)'
refs/heads/feature/batch-size 2defa92 origin/main [ahead 2]
refs/heads/main ce76024 origin/main
refs/remotes/origin/HEAD ce76024
refs/remotes/origin/feature/batch-size 56bb8e4
refs/remotes/origin/main ce76024
$ git rev-parse --abbrev-ref @{upstream}
origin/main
$ git rev-parse --abbrev-ref @{push}
fatal: cannot resolve 'simple' push to a single destination
[exit status: 128]
```

The `for-each-ref` line prints the whole ref table in a form that can be compared across machines. The last command fails, and the failure is evidence: with `push.default=simple`, a branch whose upstream has another name has no single push destination, so `@{push}` cannot be resolved. The same condition makes a bare `git push` stop with the message shown in Chapter 12, section 12.5.

```
$ git merge-base HEAD origin/feature/batch-size
ce760245c837e01a35b747b62b701ae9d5cb1593
$ git rev-list --left-right --count HEAD...origin/feature/batch-size
2      1
$ git log --oneline --left-right HEAD...origin/feature/batch-size
> 56bb8e4 Add batch_size to the job config
< 2defa92 Read batch size from the command line
< 3d6663d Embed in batches
$ git branch -a --contains origin/feature/batch-size
remotes/origin/feature/batch-size
```

2 1 reads: two commits only on the left side (HEAD), one only on the right. `<` and `>` mark the sides commit by commit.

```
$ git ls-remote origin
ce760245c837e01a35b747b62b701ae9d5cb1593      HEAD
56bb8e470535998cc128a14369042cc79c693292      refs/heads/feature/batch-size
ce760245c837e01a35b747b62b701ae9d5cb1593      refs/heads/main
$ git reflog show --date=iso origin/feature/batch-size
56bb8e4 refs/remotes/origin/feature/batch-size@{2026-09-07 10:18:00 +0530}: pull: storing head
$ git remote show origin
* remote origin
  Fetch URL: ../server.git
  Push URL: ../server.git
  HEAD branch: main
  Remote branches:
    feature/batch-size tracked
    main                tracked
  Local branches configured for 'git pull':
    feature/batch-size merges with remote main
    main                merges with remote main
  Local refs configured for 'git push':
    feature/batch-size pushes to feature/batch-size (local out of date)
    main                pushes to main                (up to date)
```

`git ls-remote` and `git remote show origin` contact the server; the other commands in this section answer from local data, which is as old as the last fetch. `git remote show origin` states the mismatch in words: the branch "merges with remote main" and "pushes to feature/batch-size (local out of date)".

Git, not GitHub. `git fetch` is the one command of the opening that changes something: it moves remote-tracking refs and adds objects. It never touches your branches, index or working tree, and each remote-tracking ref has a reflog, so it is treated as part of evidence collection. Record `git rev-parse origin/main` first if the stale value is itself evidence, as it is after a suspected force push (Chapter 13, section 13.10).

29.6 Operations in progress: what `git status` and `.git` tell you

In one sentence. Merge, rebase, cherry-pick, revert and bisect can stop half-way, and each leaves named files in `.git` that say which operation it is, where it started, and what remains.

Analogy. A surgeon's instrument count: the list on the wall says what is still inside. The analogy breaks because Git's list is also the undo record. The same files that describe the operation are what `--abort` reads to restore the starting point.

Precisely. Git has no process that "is" a rebase. An operation in progress is nothing but files: a pseudoref such as `MERGE_HEAD` that holds a commit ID, and for multi-step operations a directory with a todo list (Chapter 3, section 3.10). `git status` reads those files and prints its first lines from them. Three consequences follow. A stopped operation survives a reboot and can be weeks old. Deleting the files by hand ends the operation without undoing it. And you can diagnose a repository you were handed by listing `.git`.

See it. The script is `labs/ch29/ops-in-progress.sh`. It builds the same small repository six times, an output guard for an LLM service, and stops a different operation in five of them. A repository at rest first, as the reference:

```
$ cd clean
$ git log --graph --oneline --all
* 9f0e328 Add README
* 19b52fd Raise guard threshold to 0.65
| * 6f503d0 Lower max_tokens to 128
| * 7c39cc7 Add blocklist
| * 5c28c34 Raise guard threshold to 0.80
|/
* e432e9d Count tokens, not characters
* b039fd7 Add output guard
$ git status
On branch main
nothing to commit, working tree clean
$ ls .git
COMMIT_EDITMSG
config
description
HEAD
hooks
index
info
logs
objects
refs
$ cat .git/HEAD
ref: refs/heads/main
```

`.git/HEAD` contains `ref: refs/heads/main`. Any file in capitals that is not in this listing is a sign of activity. (`ORIG_HEAD` and `FETCH_HEAD` are records of past commands, not of an operation in progress.)

Merge.

```
$ cd ../merging
$ git status
On branch main
You have unmerged paths.
  (fix conflicts and run "git commit")
  (use "git merge --abort" to abort the merge)

Changes to be committed:
  new file:   blocklist.py

Unmerged paths:
  (use "git add <file>..." to mark resolution)
  both modified:  guard.yaml
```

`git status` does not print the word "merging". It says "You have unmerged paths" and offers `git merge --abort`; after the last conflict is staged, the text becomes "All conflicts fixed but you are still merging".

```
$ ls .git
AUTO_MERGE
COMMIT_EDITMSG
config
description
```



```

HEAD
hooks
index
info
logs
MERGE_HEAD
MERGE_MODE
MERGE_MSG
objects
ORIG_HEAD
refs
$ cat .git/MERGE_HEAD
a469d78723e151187e6d8cefe905e898fa0e1495
$ cat .git/MERGE_MODE
$ cat .git/MERGE_MSG
Merge branch 'feature/strict-guard'

# Conflicts:
#     guard.yaml
$ cat .git/ORIG_HEAD
3e2ce98834b9eccea6b02a664d0dfc63c75e0e4b
$ git ls-files -u
100644 248eaf478810ec8dbbcac312051603ba86b05f30 1      guard.yaml
100644 ea5b34d7cbddccfd80d8ac18aa35c90353b7249 2      guard.yaml
100644 07eef246dd9ec18304b69ded66e99c7edf039bf0 3      guard.yaml

```

`MERGE_HEAD` is the commit being merged in; it becomes the second parent. `MERGE_MSG` is the prepared message. `ORIG_HEAD` is the tip before the merge. `AUTO_MERGE` is a tree with the conflict markers as Git wrote them. `HEAD` is still attached to `main`. The index holds three stages for `guard.yaml` (Chapter 5, section 5.13).

Rebase.

```

$ cd ../rebasng
$ git status
interactive rebase in progress; onto 65032d6
Last command done (1 command done):
  pick 93eab48 # Raise guard threshold to 0.80
Next commands to do (2 remaining commands):
  pick bbb4ee3 # Add blocklist
  pick 7aaefa2 # Lower max_tokens to 128
(use "git rebase --edit-todo" to view and edit)
You are currently rebasing branch 'feature/strict-guard' on '65032d6'.
(fix conflicts and then run "git rebase --continue")
(use "git rebase --skip" to skip this patch)
(use "git rebase --abort" to check out the original branch)

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   guard.yaml

no changes added to commit (use "git add" and/or "git commit -a")

```

```

$ ls .git
AUTO_MERGE
COMMIT_EDITMSG
config
description
HEAD
hooks
index
info
logs
MERGE_MSG
objects
ORIG_HEAD
REBASE_HEAD
rebase-merge
refs
$ cat .git/HEAD
65032d66238b96ad4cde8f0540fa079fdc81db17
$ cat .git/REBASE_HEAD
93eab48d915310771a87dde80923ef6b9dac5f19
$ cat .git/rebase-merge/head-name
refs/heads/feature/strict-guard
$ cat .git/rebase-merge/onto
65032d66238b96ad4cde8f0540fa079fdc81db17
$ cat .git/rebase-merge/orig-head
7aaefa26c2cece48f6e468ddf7c5487f381121ca
$ cat .git/rebase-merge/msgnum .git/rebase-merge/end
1
3
$ cat .git/rebase-merge/git-rebase-todo
pick bbb4ee363e9d43a807c1ef1409e1f503f0904373 # Add blocklist
pick 7aaefa26c2cece48f6e468ddf7c5487f381121ca # Lower max_tokens to 128

```

HEAD is a raw commit ID. REBASE_HEAD is the commit whose replay stopped. rebase-merge/head-name is the branch that will be moved at the end, orig-head its tip at the start, onto the new base, msgnum and end the position (1 of 3), git-rebase-todo the remaining picks. The older apply backend uses a directory rebase-apply with the same role (Chapter 9, section 9.4).

Cherry-pick.

```
$ cd ../picking
$ git status
On branch main
You are currently cherry-picking commit a19fbd4.
  (fix conflicts and run "git cherry-pick --continue")
  (use "git cherry-pick --skip" to skip this patch)
  (use "git cherry-pick --abort" to cancel the cherry-pick operation)

Unmerged paths:
  (use "git add <file>..." to mark resolution)
    both modified:   guard.yaml

no changes added to commit (use "git add" and/or "git commit -a")
```

```
$ ls .git
AUTO_MERGE
CHERRY_PICK_HEAD
COMMIT_EDITMSG
config
description
HEAD
hooks
index
info
logs
MERGE_MSG
objects
refs
sequencer
$ cat .git/CHERRY_PICK_HEAD
a19fbd43d80f8f8214d7c1ba1bcd2b486660a821
$ ls .git/sequencer
abort-safety
head
todo
$ cat .git/sequencer/head
a2ba40c745eeaae544b12a748b049ae496e1a449
$ cat .git/sequencer/todo
pick a19fbd4 Raise guard threshold to 0.80
pick f559020 Add blocklist
pick c036209 Lower max_tokens to 128
```

CHERRY_PICK_HEAD is the commit being applied. HEAD stays on the branch. The sequencer directory exists because several commits were requested: head is where the branch was before the first pick, and todo lists the current pick first and then those not yet applied. A cherry-pick of a single commit has no sequencer directory (Chapter 10, section 10.6).

Revert.

```
$ cd ../reverting
$ git status
On branch feature/strict-guard
You are currently reverting commit e3f95f7.
  (fix conflicts and run "git revert --continue")
  (use "git revert --skip" to skip this patch)
  (use "git revert --abort" to cancel the revert operation)

Unmerged paths:
  (use "git restore --staged <file>..." to unstage)
  (use "git add <file>..." to mark resolution)
    both modified:   guard.yaml

no changes added to commit (use "git add" and/or "git commit -a")
```

```
$ ls .git
AUTO_MERGE
COMMIT_EDITMSG
config
description
HEAD
hooks
index
info
logs
MERGE_MSG
objects
refs
REVERT_HEAD
```

```
$ cat .git/REVERT_HEAD
e3f95f738e757958ffb420c74a4472a591051e58
$ cat .git/MERGE_MSG
Revert "Raise guard threshold to 0.80"

This reverts commit e3f95f738e757958ffb420c74a4472a591051e58.

# Conflicts:
#     guard.yaml
```

REVERT_HEAD is the commit being undone; the rest is the same machinery as cherry-pick with the roles of base and "theirs" exchanged (Chapter 10, section 10.11).

Bisect.

```
$ cd ../bisecting
$ git status
HEAD detached at 71a6bc1
You are currently bisecting, started from branch 'feature/strict-guard'.
  (use "git bisect reset" to get back to the original branch)

nothing to commit, working tree clean
```

```
$ ls .git
BISECT_ANCESTORS_OK
BISECT_EXPECTED_REV
BISECT_LOG
BISECT_NAMES
BISECT_START
BISECT_TERMS
COMMIT_EDITMSG
config
description
HEAD
hooks
index
info
logs
objects
refs
$ cat .git/HEAD
71a6bc165b04d0be0ad6f1c8310d5d37693a39f9
$ cat .git/BISECT_START
feature/strict-guard
$ cat .git/BISECT_TERMS
bad
good
$ cat .git/BISECT_LOG
# bad: [6ed2fda3394fb715dd93eb1690d7cd51e05deac7] Lower max_tokens to 128
# good: [307506a4f8bc8b31589e49db034dd1b16728206f] Add output guard
git bisect start 'HEAD' 'main~3'
# good: [3164fc1d8b8dbf03fed13bfa4deccd261c8fd254] Raise guard threshold to 0.80
git bisect good 3164fc1d8b8dbf03fed13bfa4deccd261c8fd254
$ git for-each-ref refs/bisect
6ed2fda3394fb715dd93eb1690d7cd51e05deac7 commit refs/bisect/bad
307506a4f8bc8b31589e49db034dd1b16728206f commit refs/bisect/good-307506a4f8bc8b31589e49db034dd1b16728206f
3164fc1d8b8dbf03fed13bfa4deccd261c8fd254 commit refs/bisect/good-3164fc1d8b8dbf03fed13bfa4deccd261c8fd254
```

A bisect has no conflict and a clean working tree, which is why it is the operation most often forgotten. HEAD is detached on the commit under test. BISECT_START is the branch to return to, BISECT_LOG the replayable record, and the marks are refs under refs/bisect/ (Chapter 14A, section 14A.20).

Picture. One table to keep:

Operation	First lines of git status	State in .git	HEAD	Continue	Leave and restore	Leave and keep the current state
Merge	"You have unmerged paths" or "All conflicts fixed but you are still merging"	MERGE_HEAD, MERGE_MODE, MERGE_MSG, AUTO_MERGE	on the branch	git commit or git merge --continue	git merge --abort	git merge --quit
Rebase	"interactive rebase in progress; onto ..." or "rebase in progress"	rebase-merge/ or rebase-apply/, REBASE_HEAD	detached	git rebase --continue	git rebase --abort	git rebase --quit
Cherry-pick	"You are currently cherry-picking commit ..."	CHERRY_PICK_HEAD; sequencer/ for several commits	on the branch	git cherry-pick --continue	git cherry-pick --abort	git cherry-pick --quit
Revert	"You are currently reverting commit ..."	REVERT_HEAD; sequencer/ for several commits	on the branch	git revert --continue	git revert --abort	git revert --quit
Bisect	"You are currently bisecting, started from branch ..."	BISECT_LOG, BISECT_START, BISECT_TERMS, refs/bisect/*	detached	git bisect good or bad	git bisect reset	git bisect reset HEAD

All of the "Leave and restore" commands are 🟡: each resets the index and the working tree for tracked files, so uncommitted edits made during the operation are lost. Section 29.7 shows that loss in a transcript.

The same question for six repositories at once:

```
# One read-only question for any repository: which state files exist?
$ for d in clean merging rebasing picking reverting bisecting; do echo "== $d"; ls ../$d/.git | grep -E "_HEAD$|rebase-
|sequencer|BISECT_LOG"; done
== clean
MERGE_HEAD
ORIG_HEAD
== rebasing
ORIG_HEAD
REBASE_HEAD
rebase-merge
== picking
CHERRY_PICK_HEAD
sequencer
== reverting
REVERT_HEAD
== bisecting
BISECT_LOG
```

What Git refuses while an operation is open. These refusals are often the reported symptom:

```
$ cd ../merging
$ git switch feature/strict-guard
fatal: cannot switch branch while merging
Consider "git merge --quit" or "git worktree add".
[exit status: 128]
$ git cherry-pick feature/strict-guard~1
error: Cherry-picking is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: cherry-pick failed
[exit status: 128]
$ git merge feature/strict-guard
error: Merging is not possible because you have unmerged files.
hint: Fix them up in the work tree, and then use 'git add/rm <file>'
hint: as appropriate to mark resolution and make a commit.
fatal: Exiting because of an unresolved conflict.
[exit status: 128]
$ cd ../rebasing
$ git rebase main
fatal: It seems that there is already a rebase-merge directory, and
I wonder if you are in the middle of another rebase. If that is the
case, please try
    git rebase (--continue | --abort | --skip)
If that is not the case, please
    rm -fr ".git/rebase-merge"
and run me again. I am stopping in case you still have something
valuable there.

[exit status: 128]
```

The last message proposes `rm -fr ".git/rebase-merge"`. That deletes the record of where the branch was and what remains to be replayed. It is appropriate only when you have confirmed that no rebase is wanted and have anchored HEAD first. `git rebase --quit` does the same with Git's own bookkeeping.

The ways out.

```
$ cd ../merging && git merge --abort && git status -sb && ls .git | grep -c MERGE
## main
0
$ cd ../rebasing && git rebase --abort && git status -sb
## feature/strict-guard
$ cd ../picking && git cherry-pick --abort && git status -sb
## main
$ cd ../reverting && git revert --abort && git status -sb
## feature/strict-guard
$ cd ../bisecting && git bisect reset && git status -sb
Previous HEAD position was 71a6bc1 Add blocklist
Switched to branch 'feature/strict-guard'
## feature/strict-guard
```

In production. A colleague's laptop is handed over with "Git is broken, it will not let me switch branches". The diagnosis is one `git status`: an earlier `git pull` stopped at a conflict and was left. The decision that follows is not automatic: continue (if the operation was intended and the resolution is known), abort (if nothing done inside it matters), or anchor and then abort (case 1). Ask what was intended before choosing.

29.7 Preserving evidence before acting

In one sentence. Before the first state-changing command, put what you might need later somewhere that the fix cannot reach.

Analogy. A database administrator takes a snapshot before a migration. The analogy breaks only in cost: a backup ref takes one second, so time never justifies skipping it.

Precisely. There are four layers. Each holds more than the previous one and costs more.

Layer	Command	Holds	Does not hold
The recorded output	redirect the ten commands to a file; <code>git diff > file.patch</code>	What you saw, including reflow lines that later commands will push down; uncommitted changes as patches	Objects
A backup ref 	<code>git branch rescue/<what> <id></code> or <code>git update-ref refs/backup/<name> <id></code>	One commit and everything it reaches, kept from garbage collection for as long as the ref exists	Reflows, the index, uncommitted files, operation state
A copy of the repository 	<code>cp -Rp . ../evidence/<name>-copy</code>	Everything: objects, refs, reflogs, index, working tree, stashes, hooks, configuration, operation state	Nothing local. It does not hold what only the server has
A bundle 	<code>git bundle create <file> --all</code>	Every ref and HEAD with all objects they reach, in one file that can be verified, moved and fetched from	Reflows, unreachable objects, the index, uncommitted files, configuration, hooks

A ref under `refs/backup/` is not shown by `git branch`; a branch under `rescue/` is, which is better when a colleague continues the work. Both keep the commit alive.

See it. The script is `labs/ch29/preserve-evidence.sh`. It uses the repository of case 1 with two additions that case 1 did not have: one staged and one unstaged edit.

```
$ mkdir ../evidence
$ { git status; git branch -vv; git log --graph --decorate --oneline --all; git reflow; } > ../evidence/state.txt 2>&1
$ git diff > ../evidence/unstaged.patch
$ git diff --cached > ../evidence/staged.patch
$ git status --short
M README.md
M config/service.yaml
$ grep -c "" ../evidence/state.txt ../evidence/unstaged.patch ../evidence/staged.patch
../evidence/state.txt:46
../evidence/unstaged.patch:10
../evidence/staged.patch:9
```

```
$ git branch rescue/latency-wip HEAD
$ git update-ref refs/backup/latency-budget feature/latency-budget
$ git for-each-ref refs/heads/rescue refs/backup
4a030148e74b6b4394d1056737434790a34ae612 commit refs/backup/latency-budget
075407e71d9b34006eccefd0d2072bfb51d2850b commit refs/heads/rescue/latency-wip
```

```
$ cp -Rp . ../evidence/scoring-api-copy
$ git -C ../evidence/scoring-api-copy status | head -4
interactive rebase in progress; onto 4b30d7e
Last command done (1 command done):
  pick 60d09a1 # Add latency budget to config
Next commands to do (2 remaining commands):
$ git -C ../evidence/scoring-api-copy status --short
M README.md
M config/service.yaml
$ git -C ../evidence/scoring-api-copy reflow -3
075407e HEAD@{0}: commit: Log budget violations
96a7b55 HEAD@{1}: commit: Add p95 latency metric
9f6e4d7 HEAD@{2}: commit: Add latency budget to config
```

The copy is a complete repository: the same rebase in progress, the same staged and unstaged files, the same reflow. `-p` keeps modification times, which spares Git a re-read of every file (Chapter 5, section 5.14).

```
$ git bundle create ../evidence/scoring-api.bundle --all
$ git bundle verify ../evidence/scoring-api.bundle
../evidence/scoring-api.bundle is okay
The bundle contains these 7 refs:
4a030148e74b6b4394d1056737434790a34ae612 refs/backup/latency-budget
4a030148e74b6b4394d1056737434790a34ae612 refs/heads/feature/latency-budget
4b30d7e307a39954609eb40c2b4c2a1f63c74857 refs/heads/main
075407e71d9b34006eccefd0d2072bfb51d2850b refs/heads/rescue/latency-wip
4a030148e74b6b4394d1056737434790a34ae612 refs/remotes/origin/feature/latency-budget
4b30d7e307a39954609eb40c2b4c2a1f63c74857 refs/remotes/origin/main
075407e71d9b34006eccefd0d2072bfb51d2850b HEAD
```

```
The bundle records a complete history.
The bundle uses this hash algorithm: sha1
```

The bundle lists seven refs, among them the backup refs created a moment ago and the detached HEAD. A bundle holds what refs and HEAD reach at the moment it is made. A commit that only a relog knows is not in it, which is the reason to anchor first and bundle second (Chapter 26, section 26.14 covers the format).

Inside .git. Now the proof. The script does on purpose what a panicked sequence of "fixes" does by accident:

```
# The worst afternoon: the rebase is aborted, the anchors are deleted, the relogs are
# emptied and the unreachable objects are pruned. Do not run this on a repository you need.
$ git rebase --abort
$ git branch -D rescue/latency-wip
Deleted branch rescue/latency-wip (was 075407e).
$ git update-ref -d refs/backup/latency-budget
$ git relog expire --expire=now --all
$ git gc --quiet --prune=now
$ git cat-file -t 075407e
fatal: Not a valid object name 075407e
[exit status: 128]
$ git status --short
```

git rebase --abort printed nothing, and the last command shows that the staged and the unstaged edit are gone with it. With the anchors deleted, the relogs emptied and the objects pruned, 075407e is no longer an object in this repository. This is the point of no return of Chapter 13, section 13.13, reached in five commands. 🚫 git relog expire --expire=now --all followed by git gc --prune=now removes the safety net itself: it has no preview and no undo, and it is appropriate only for removing sensitive data from a clone after the secret has been rotated.

```
$ git fetch ../evidence/scoring-api.bundle 'refs/heads/rescue/*:refs/heads/rescue/*'
From ../evidence/scoring-api.bundle
 * [new branch]      rescue/latency-wip -> rescue/latency-wip
$ git log --oneline -3 rescue/latency-wip
075407e Log budget violations
96a7b55 Add p95 latency metric
9fbe4d7 Add latency budget to config
$ git relog show rescue/latency-wip
075407e rescue/latency-wip@{0}: fetch ../evidence/scoring-api.bundle refs/heads/rescue/*:refs/heads/rescue/*: storing head
```

The bundle returns the three commits. The new relog has one line: the history of how the ref moved is not in a bundle.

```
$ cd ../evidence/scoring-api-copy
$ git status --short
M README.md
M config/service.yaml
$ git relog -3
075407e HEAD@{0}: commit: Log budget violations
96a7b55 HEAD@{1}: commit: Add p95 latency metric
9fbe4d7 HEAD@{2}: commit: Add latency budget to config
$ cat .git/rebase-merge/head-name
refs/heads/feature/latency-budget
$ git diff --cached --stat
config/service.yaml | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

The copy returns everything, including the two uncommitted edits and the rebase state.

None of the four layers changes the working tree, the index, HEAD or any existing ref: a backup ref adds one ref, and the copy and the bundle are written outside the repository.

In production. A ref is about to move: a backup ref, always. Uncommitted work or an operation in progress is involved: a copy of the whole directory. The disk is suspect or the evidence must leave the machine: a bundle, stored elsewhere. Someone else will ask what happened: the recorded output.

A copy of a repository contains everything the repository contains, including credentials in .git/config URLs and any secret in history. Treat it with the same care, and delete it when the incident is closed.

29.8 GitHub-side evidence

In one sentence. Your clone records what you did; GitHub records what reached the server, who was authenticated when it arrived, and which rules were evaluated.

GitHub, not Git. Nothing in this section is part of Git. Git's relogs are local and are not shared with remotes, and GitHub exposes no server-side relog (Chapter 13, section 13.15). Everything below is described from GitHub's documentation as read on 1 October 2026;

none of the commands was executed here, and user-interface labels change.

Precisely. Five sources, each with a different reach and a different lifetime.

Source	What it records	Reach and limits	Who can read it
Activity view of a repository	"pushes, merges, force pushes, and branch changes", associated "with commits and authenticated users"; filters for branch, activity type (direct pushes, pull request merges, force pushes, branch creations, branch deletions), user and time period; "Compare changes" on each entry (documentation)	The before and after commit of each ref update. Retention is not stated in the page	Not stated in the page (flag below)
Pull request timeline	The events of one pull request in order. The documented event types include <code>committed</code> , <code>head_ref_force_pushed</code> ("The pull request's HEAD branch was force pushed"), <code>head_ref_deleted</code> , <code>head_ref_restored</code> and <code>base_ref_changed</code> (issue event types). A closed pull request offers "Restore branch" for a deleted head branch (documentation)	One pull request. The head commits stay fetchable through <code>refs/pull/N/head</code> (Chapter 17, section 17.2)	Anyone who can read the pull request
Events API	<code>PushEvent</code> with <code>ref</code> , <code>before</code> ("the SHA of the most recent commit on <code>ref</code> before the push") and <code>head</code> (after the push) (event types)	"up to 300 events", only those "created within the past 30 days"; latency "anywhere from 30s to 6h"; since 7 October 2025 push events no longer carry commit summaries (events API , changelog)	Anyone who can read the repository
Rule Insights	Every ref update evaluated by a ruleset: passed, failed or bypassed, and what would have happened in Evaluate mode; the <code>rule-suites</code> REST endpoint returns the same data (documentation)	Rulesets only, not classic branch protection. An exempt actor skips enforcement without the signals a bypass generates (Chapter 18, section 18.5). The insights dashboard is for Team and Enterprise Cloud plans	Repository administrators
Audit log	Organization: events of the last 180 days, for owners, exportable. Enterprise: also Git events such as <code>git.push</code> , retained for seven days and available "only via the REST API, audit log streaming, or JSON/CSV exports"; <code>protected_branch.policy_override</code> when an administrator overrides a classic rule (organization audit log , enterprise audit log)	Organization and enterprise accounts only. A personal repository has no audit log of this kind	Organization owners; enterprise owners

The commands that read these sources, shown without output:

```
# Activity: the documented REST endpoint behind the Activity view
gh api "repos/OWNER/REPO/activity?activity_type=force_push&ref=refs/heads/main"

# One pull request: its state, its head commit, its timeline
gh pr view 42 --json headRefId,baseRefName,mergeable,mergeStateStatus,statusCheckRollup
gh api repos/OWNER/REPO/issues/42/timeline --paginate

# Events: before and head of recent pushes
gh api repos/OWNER/REPO/events --jq '.[] | select(.type=="PushEvent") | [.created_at, .payload.ref, .payload.before, .payload.head] | @tsv'

# Rules: every evaluation, and what applies to one branch
gh api repos/OWNER/REPO/rulesets/rule-suites
gh ruleset check main
```

The activity endpoint is documented as `GET /repos/{owner}/{repo}/activity` with the `activity_type` values `push`, `force_push`, `branch_creation`, `branch_deletion`, `pr_merge` and `merge_queue_merge`, and items that carry `before`, `after`, `ref`, `timestamp` and `actor` ([REST: repositories](#)).

How the two sides combine. Git evidence answers "what is the state and which local command produced it". GitHub evidence answers "which authenticated account moved the server's ref, when, from which commit to which, and did a rule allow it". Your clone cannot know who pushed. The author field of a commit does not say either: anyone can set it (Chapter 14B, section 14B.18).

The flags that travel with these facts (from the Phase 0 report, section 13):

Unverified. A "restore" action inside the Activity view: only filtering and "Compare changes" are documented there. The access requirement and the retention period of the Activity view are not stated in the documentation that was read.

Unverified. How long GitHub keeps commits that no ref reaches. No retention period is published. The recovery recipe (find the old ID, then create a ref at it through the references API, Chapter 13, section 13.15) is an inference from documented endpoints, not a documented procedure, and nothing guarantees that it succeeds.

In production. `main` moved backwards on Friday evening. `git reflog show origin/main` in any clone that fetched before and after gives the two IDs. The Activity view filtered by force pushes names the account and the time. Rule Insights says whether a ruleset evaluated that update and whether the actor bypassed it. The audit log shows whether a rule was edited shortly before.

29.9 Choosing the lowest-risk fix

In one sentence. List every fix that would repair the root cause, rank them by what each can destroy and how each is undone, and take the one that only adds.

Precisely. Rank by five questions, in this order.

1. **Does it destroy uncommitted work?** That work has no object and no reflog. Anything that can remove it (`git reset --hard`, `git restore`, `git clean`, `git stash drop`, every `--abort`) ranks last unless the working tree is clean or copied.
2. **Does it rewrite commits that another repository has?** Rewriting published history turns one person's problem into every clone's problem (Chapter 9, section 9.15).
3. **Does it change the server?** A local change is undone locally. A push, a deleted remote branch or a changed rule is seen by others and may start workflows.
4. **Is it undone by one command?** A new commit is undone by a revert; a moved ref by moving it back; a pruned object by nothing.
5. **Can it be previewed?** `--dry-run`, `git merge-tree`, `git diff <target>`, `git log <range>`, or a rehearsal in a copy.

The resulting ladder, from least to most risk:

1	add a ref	<code>git branch</code> , <code>git tag</code> , <code>git update-ref</code>	nothing can be lost
2	add a commit	<code>git commit</code> , <code>git revert</code> , <code>git cherry-pick</code> , <code>merge</code>	undone by a revert or by moving the ref back
3	move a local ref	<code>git reset --keep</code> , <code>git branch -f</code> , <code>git rebase</code>	undone through the reflog or a backup ref
4	overwrite the work tree	<code>git reset --hard</code> , <code>git restore</code> , <code>git clean</code>	uncommitted work is unrecoverable
5	rewrite the server	<code>git push --force-with-lease=<ref>:<expected></code>	others must repair their clones
6	remove the safety net	<code>git reflog expire</code> , <code>git gc --prune=now</code>	nothing undoes it

Prefer the lowest rung that repairs the cause. In case 1 that was rungs 1 and 2. In case 2 it was rung 3 on commits that existed nowhere else, with rung 1 first.

Three rules of thumb follow from the ladder.

- **On a shared branch, undo with** `git revert`, **not with** `git reset`. A revert adds a commit; everyone's next pull is a fast-forward (Chapter 11, section 11.8).
- **When a ref must move back, use** `git reset --keep`. It refuses when a file with local changes would be overwritten, where `--hard` overwrites it (Chapter 11, section 11.6).
- **When a force push is the correct fix, state the expected old value:** `git push --force-with-lease=<branch>:<expected-id>`. The bare form is defeated by anything that fetches in the background (Chapter 12, section 12.8).

29.10 Verification

In one sentence. A fix is verified when the commands that showed the problem now show its absence, for the reason you predicted, in every place the problem existed.

Precisely. Four checks, in this order.

1. **The symptom.** Repeat the command or page that showed it: the rejected push succeeds; the pull request lists the expected commits.
2. **The state.** Repeat the diagnosis commands that exposed the cause. The first line of `git status` changed; the brackets in `git branch -vv` changed; `.git` has no state files.
3. **Every copy.** A fix in your clone is not a fix on the server. Compare IDs, not names: `git rev-parse HEAD @{upstream}` and `git ls-remote origin <branch>` must print the same ID. Then the colleague's clone, the pull request, the pipeline run for that exact commit.
4. **Nothing else changed.** `git diff <backup> HEAD` or `git range-diff` shows that the result differs from the preserved state only where intended. `git fsck` reports no damage. `git status` shows no leftovers.

Write the prediction before you run the check: in case 1, "`git cherry` will mark the two new commits with a minus sign and the two others with a plus sign".

Then, and only then, remove what the investigation added: the rescue branch (`git branch -D` prints the ID, which is one more safety line), the evidence directory, the bundle.

What a pull request needs after a fix is platform behavior: a push to the head branch reruns workflows and can dismiss approvals; a forced push marks review comments as outdated (Chapter 17, sections 17.5 and 17.12). Verification includes reading the pull request page again.

29.11 The symptom catalog

Twenty-six symptoms; the playbook's compact catalog adds six shorter ones. Each row gives the likely causes in the order worth testing, the commands that separate them, and the chapter and section with the mechanism. Chapters 20A and 20B are the GitHub Actions chapters.

History and refs

#	Symptom	Likely causes	Commands that distinguish them	Mechanism
1	A commit is missing	(a) it is on another branch; (b) it was made on a detached HEAD or inside a stopped operation; (c) a reset, rebase or amend took it off the branch; (d) it was never pushed, or pushed to another branch; (e) it was never made: the change was not staged	<code>git branch -a --contains <id>; git status (first line); git reflog, git reflog show <branch>; git ls-remote origin; git log --all --oneline -- <path></code>	13, 13.7, 13.8; 12, 12.14; section 29.3; 1, 1.12
2	A branch disappeared	(a) it exists only as a remote-tracking branch and was never created locally; (b) deleted locally; (c) deleted on the server and removed here by a pruning fetch; (d) renamed; (e) you are in another clone or worktree	<code>git branch -a; git reflog (search for moving from <branch>); git ls-remote origin; git fsck --lost-found; git worktree list; on GitHub the Activity view filtered by branch deletions</code>	13, 13.8, 13.15; 12, 12.11; 7, 7.10
3	"HEAD detached at ..."	(a) a tag, a commit ID or a remote-tracking branch was checked out; (b) a rebase or bisect is in progress; (c) you are inside a submodule; (d) a CI checkout	<code>git status; cat .git/HEAD; ls .git; git reflog -5; git rev-parse --show-toplevel</code>	7, 7.7; section 29.6; 23, 23.5
4	Commits are duplicated: the same subject twice with different IDs	(a) a published branch was rebased and then merged with its old copy; (b) cherry-picked commits were later merged; (c) a pull request was squashed or rebase-merged and the branch reused	<code>git log --oneline --graph --all; git cherry -v <upstream> <branch>; git range-diff A...B; git log --cherry-mark --left-right A...B</code>	9, 9.3, 9.15; 10, 10.10; 17, 17.9
5	A merge brought nothing: "Already up to date", yet the code is absent	(a) the branch was merged before and the merge was reverted; (b) the commits are ancestors already and a later commit removed the change; (c) merged with <code>-s ours</code> ; (d) you merged a stale local branch instead of <code>origin/<branch></code>	<code>git merge-base --is-ancestor <branch> HEAD; git log --oneline --merges --grep=Revert; git log -S<string> --oneline; git rev-parse <branch> origin/<branch></code>	11, 11.9; 8, 8.3, 8.6
6	A merge conflict nobody expected	(a) both sides changed adjacent lines; (b) the merge base is older than assumed: squash-merged branch reused, or a criss-cross; (c) one side renamed or reformatted the file; (d) line endings differ; (e) "ours" and "theirs" are exchanged in a rebase	<code>git merge-base A B; git diff A...B --stat; git ls-files -u; git diff --ignore-cr-at-eol; git log --merge --oneline</code>	8, 8.7, 8.5, 8.11; 9, 9.11; 17, 17.12
7	Code vanished after a merge and no commit removes it	(a) a conflict was resolved by taking one side; (b) history simplification hides the commit in <code>git log -- <path></code> ; (c) a clean merge that is wrong in meaning	<code>git log --full-history --oneline --<path>; git show --remerge-diff <merge>; git log -S<string> --oneline</code>	8, 8.15, 8.16; 14A, 14A.10, 14A.11
8	A tag points somewhere else on another machine	(a) the tag was moved on the server after you fetched it: fetch does not update existing tags; (b) a local tag of the same name was never pushed; (c) lightweight on one side, annotated on the other	<code>git rev-parse <tag>^(commit); git ls-remote --tags origin <tag>; git cat-file -t <tag>; git for-each-ref refs/tags/<tag></code>	14B, 14B.10, 14B.11; 12, 12.15
9	<code>git branch -d</code> refuses: "not fully merged", after the pull request was merged	(a) squash or rebase merge created new IDs, so the branch is not an ancestor; (b) the branch has commits made after the merge; (c) the comparison is with the upstream, which is gone	<code>git cherry -v origin/main <branch>; git log --oneline origin/main...<branch>; git branch -vv (gone)</code>	17, 17.9; 7, 7.5

Working tree and index

#	Symptom	Likely causes	Commands that distinguish them	Mechanism
10	Git does not recognize a file: it is on disk and <code>git status</code> does not list it	(a) an ignore rule in <code>.gitignore</code> , <code>.git/info/exclude</code> or the global excludes file; (b) it is inside a nested repository; (c) the directory is empty; (d) outside the sparse-checkout cone; (e) you are in another repository	<code>git check-ignore -v <path>; git status --ignored; git rev-parse --show-toplevel; git ls-files -s <path> (mode 160000 is a submodule); git sparse-checkout list</code>	4, 4.3, 4.5, 4.11; 23, 23.2; 24, 24.4
11	A file keeps showing as modified	(a) it is tracked and also listed in <code>.gitignore</code> : ignore rules do not apply to tracked files; (b) line-ending conversion; (c) the executable bit; (d) two names that differ only in case; (e) an LFS or other filter; (f) a tool rewrites it	<code>git ls-files <path>; git diff --stat, git diff --ignore-cr-at-eol; git ls-files --eol <path>; git diff --summary (mode change); git ls-files piped to sort -f and uniq -di; git check-attr -a -- <path></code>	4, 4.6, 4.10, 4.12; 14C, 14C.5; 22, 22.5
12	A change is in the working tree and not in the commit	(a) edited and never staged; (b) staged, then edited again; (c) <code>git commit <path></code> or <code>-a</code> bypassed what was staged	<code>git status; git diff; git diff --cached; git show --stat HEAD</code>	1, 1.12; 5, 5.4, 5.10
13	Uncommitted work vanished	(a) <code>git reset --hard</code> , <code>git restore</code> , <code>git checkout -- <path></code> or an <code>--abort</code> ; (b) it is in a stash; (c) <code>git clean</code> ; (d) another worktree holds it	<code>git stash list; git reflog -10; git fsck --lost-found (staged content survives as blobs); git worktree list</code>	11, 11.5, 11.10; 13, 13.9, 13.12
14	Git refuses to switch, pull, merge or commit: "you need to resolve your current index first", "cannot switch branch while merging"	An operation is in progress	<code>git status; ls .git; section 29.6</code>	Section 29.6; 8, 8.10

Remotes, access and identity

#	Symptom	Likely causes	Commands that distinguish them	Mechanism
15	A push is rejected	(a) ! [rejected] (fetch first) or (non-fast-forward) : the server's branch has commits you lack; (b) the pull integrates another branch than the push targets; (c) ! [remote rejected] : a rule, a hook, push protection or a file-size limit, named in the remote: lines; (d) no permission	Read the bracket and the reason in the push output; git fetch, git status -sb; git branch -vv; gh ruleset check <branch>	12, 12.7, 12.15; section 29.4; 18, 18.17
16	A pull refuses	(a) "Need to specify how to reconcile divergent branches"; (b) "Not possible to fast-forward" with pull.ff=only; (c) local changes would be overwritten; (d) no upstream; (e) an operation is in progress; (f) "refusing to merge unrelated histories"	git status -sb; git config get --show-origins --all pull.ff, pull.rebase; git branch -vv; git merge-base HEAD @{upstream}	12, 12.6, 12.15; 8, 8.18
17	"Everything up-to-date", and the commit is not on the server	(a) the commit is not on the branch that was pushed: detached HEAD, stopped rebase, another branch; (b) the push went to another remote or branch	git branch -a --contains <id>; git status; git remote -v; git ls-remote origin	12, 12.14; section 29.3
18	"Already up to date" after a pull, and a teammate's commit is absent	(a) the upstream is another branch; (b) the teammate did not push, or pushed elsewhere; (c) a narrow fetch refspec or single-branch clone	git branch -vv; git ls-remote origin; git config get --all remote.origin.fetch	Section 29.4; 12, 12.4, 12.12
19	"Permission denied (publickey)"	(a) no key offered: agent empty, wrong IdentityFile; (b) the key belongs to no account; (c) the user in the URL is not git; (d) a key of another account answers first	ssh -T git@github.com; ssh -vT git@github.com; ssh-add -l; ssh -G github.com; git remote -v	16, 16.18, 16.9, 16.10
20	"Repository not found" for a repository that exists	(a) the credential belongs to an account without access: GitHub answers "not found" for private resources; (b) a typo or a renamed repository; (c) a fine-grained token not granted this repository; (d) SSO authorization missing	git remote -v; gh auth status; ssh -T git@github.com; git config get --show-origins --all credential.helper	16, 16.19, 16.15
21	The commit is attributed to the wrong person, or to nobody, on GitHub	(a) user.email comes from an unexpected scope or a conditional include; (b) the address is not registered on the account; (c) the author was set by environment variables	git log -1 --format='%an <ae> / %cn <ce>'; git config get --show-origins --show-scope --all user.email	14B, 14B.2, 14B.4; 6, 6.5, 6.11

Pull requests, checks and CI

#	Symptom	Likely causes	Commands that distinguish them	Mechanism
22	A pull request shows hundreds of changes	(a) the wrong base branch; (b) the head branch was reused after a squash merge; (c) the branch was rebased or force-pushed; (d) every line changed: line endings or a formatter; (e) a lock file or generated file	git log --oneline <base>...<head>; git diff --stat <base>...<head>; git merge-base <base> <head>; git diff --ignore-cr-at-eol --stat <base>...<head>	17, 17.12, 17.3; 14C, 14C.5
23	CI fails only on the pull request and passes on the branch and locally	(a) the pull request run builds refs/pull/N/merge, the head merged into the current base, which contains base changes you do not have; (b) secrets are withheld from fork pull requests; (c) a shallow clone without tags; (d) a case-sensitive filesystem on the runner; (e) an untracked or ignored file exists only on your machine	git fetch origin, then git merge-tree --write-tree origin/<base> HEAD, or merge the base locally and run the tests; git status --ignored; git ls-files; gh run view <id> --log-failed	17, 17.2, 17.6; Chapters 20A and 20B; 26, 26.11; 4, 4.12
24	A required check never finishes: it stays pending and blocks the merge	(a) the workflow was skipped by a path filter, a branch filter or [skip ci] and never reports; (b) the required name matches no job; (c) the run was triggered by an event that does not count; (d) with a merge queue, the workflow lacks the merge_group trigger	gh pr checks <n> --required; gh run list --commit <id>; compare the required names with the job names	18, 18.8; 17, 17.11

Submodules, LFS and scale

#	Symptom	Likely causes	Commands that distinguish them	Mechanism
25	A submodule is at the wrong commit, or its directory is empty	(a) the superproject moved its pointer and git submodule update was not run; (b) not initialized after clone; (c) the pointer names a commit that was never pushed; (d) work was committed on the submodule's detached HEAD	git submodule status (prefix +, -, U); git diff --submodule; git ls-tree HEAD <path>; git -C <path> status	23, 23.3, 23.5, 23.7, 23.8
26	Small text files beginning with version https://git-lfs.github.com/spec/v1 where data should be	(a) Git LFS is not installed or git lfs install was not run, so the smudge filter did not run; (b) the clone skipped the download; (c) the object was never uploaded; (d) the file was committed before the pattern was tracked, or the reverse	git lfs ls-files; git lfs status; git lfs env; git check-attr filter -- <path>; git config get --all filter.lfs.smudge	22, 22.3, 22.7, 22.10

The pattern across the catalog is the content of this book in one line: nearly every symptom is a difference between two of the places of section 29.2 that someone assumed were equal. The working tree and the commit (12). The branch and HEAD (1, 3, 17). The local branch and its upstream (15, 18). The remote-tracking branch and the server (2, 8). The branch and the merge ref (23). The diagnosis is finding which two.

29.12 What can go wrong

The method itself fails in recognisable ways.

What goes wrong	How to recognise it	Fix	Prevention
A "fix" is typed before the state is known	The relog shows <code>reset</code> , <code>checkout</code> or <code>rebase (abort)</code> entries made after the report	Stop. Anchor HEAD and every candidate ID, then restart at OBSERVE	The phases of section 29.2: nothing but 🟢 commands until the root cause is written down
The diagnosis uses stale remote-tracking refs	<code>git status</code> says "up to date" and <code>git ls-remote origin</code> shows another ID	<code>git fetch</code> , after recording the old value if it is evidence	Treat <code>origin/<branch></code> as "the server at the last fetch" (Chapter 12, section 12.4)
The wrong layer is blamed	"GitHub lost my commits" while <code>git ls-remote</code> shows the server never had them	Name the layer in the root-cause box and give the evidence for it	The label "Git, GitHub or Actions" is a required line of every explanation
The evidence is destroyed by the investigation	The relog was expired, a branch was deleted "to tidy up", <code>git gc</code> was run	Recover from a copy, a bundle or another clone (Chapter 13, section 13.10)	Section 29.7 before phase 3; never run <code>git gc</code> or <code>git prune</code> during an incident

29.13 When not to use it, and dangerous edge cases

When not to use the full method. A message that names its own remedy and whose remedy is 🟢 needs no hypothesis table: "The current branch has no upstream branch" with the command printed beneath it, or a typo in a branch name. Run `git status`, do what the message says, and move on. The full method is for symptoms that contradict what the reporter believes about the state. The opening ritual is never skipped: it takes less than a minute.

When to stop and escalate instead. If the evidence shows a credential in history, stop diagnosing and start the secret response: revoke first, investigate second (Chapter 21B). If the repository reports corrupt or missing objects, do not continue working in it: copy it and follow Chapter 13, section 13.11. If production is down, restore service first and diagnose afterwards on preserved evidence (Chapter 30).

Dangerous edge cases.

- **Diagnostic commands that are not read-only.** `git fsck --lost-found` writes files under `.git/lost-found/`. `git fetch` moves remote-tracking refs, and with `fetch.prune=true` or `--prune` it deletes those whose server branch is gone, together with their relogs: the last local record of a deleted branch (Chapter 12, section 12.11). Use `git ls-remote` when you must not change anything. `git status` may rewrite the index to refresh cached stat data; in a repository you must not touch at all, use `git --no-optional-locks status`, which the Git manual documents for background tools.
- **A stopped operation with new commits inside it** (case 1). `--abort` returns the branch and silently discards the commits and any uncommitted edit. `--continue` may be right or wrong. Anchor first; then every choice can be undone.
- **GitHub-side evidence that expires** (section 29.8). Export it on the first day.
- **Deleting state files by hand.** Removing `.git/MERGE_HEAD` or `.git/rebase-merge` ends the bookkeeping, not the operation. Lab 35.2 shows a merge that became an ordinary one-parent commit this way, after which Git still considers the branch unmerged.

29.14 Command safety

Command	Label	What it changes	Preview	Recovery
The ten commands; <code>git ls-remote</code> ; <code>git merge-tree --write-tree</code> ; <code>git fsck</code> ; <code>cp -Rp</code> ; <code>git bundle create</code>	🟢 SAFE	No ref, index entry or tracked file (section 29.13 lists the small exceptions)	not needed	not needed
<code>git fetch</code>	🟢 SAFE	Adds objects; moves remote-tracking refs; with pruning, deletes stale ones	<code>git fetch --dry-run</code> ; <code>git ls-remote</code>	The relog of each remote-tracking ref, unless the ref was pruned
<code>git branch rescue/x <id></code> , <code>git update-ref refs/backup/x <id></code>	🟢 SAFE	Adds one ref	not needed	<code>git branch -D</code> , <code>git update-ref -d</code>
<code>git cherry-pick</code> , <code>git revert</code> , <code>git merge</code>	🟡 CAUTION	Add commits to the current branch; update index and working tree	<code>git merge-tree</code> ; <code>git log <range></code> ; <code>git show <id></code>	<code>git reset --keep ORIG_HEAD</code> or the branch relog
<code>git rebase</code>	🟡 CAUTION	Replaces commits of the current branch with new ones	<code>git log <upstream>..HEAD</code>	<code>git reset --keep ORIG_HEAD</code> ; a backup ref
<code>git merge --abort</code>	🔴 DANGEROUS	Ends the merge; resets index and working tree to HEAD. Destroys every resolution made so far and edits staged during the merge, and the manual warns that uncommitted changes present when the	<code>git status</code> , <code>git diff --cached --stat</code>	Staged content: <code>git fsck --lost-found</code> . Unstaged resolution edits: none. Appropriate when the merge

Command	Label	What it changes	Preview	Recovery
		merge started cannot always be reconstructed (Chapter 8, section 8.21)		was a mistake and the PRESERVE step is done
<code>git rebase --abort</code> , <code>git cherry-pick --abort</code> , <code>git revert --abort</code> , <code>git bisect reset</code>	⚠ CAUTION	End the operation; reset index and working tree to the starting commit; commits made inside a rebase leave the branch	<code>git status</code> , <code>git diff</code> , <code>git log <branch>..HEAD</code>	Commits: the HEAD reflog or a rescue ref. Uncommitted edits: none
<code>git push</code>	⚠ CAUTION	Moves a ref on the server; starts whatever listens for pushes	<code>git push --dry-run</code>	A revert commit; moving the ref back needs a force
<code>git reset --hard</code> , <code>git restore <path></code> , <code>git clean -f</code>	🔴 DANGEROUS	Overwrite or delete working tree files; uncommitted work is destroyed	<code>git status</code> , <code>git diff</code> , <code>git clean -n</code>	None for unstaged content; staged content through <code>git fsck --lost-found</code> . Appropriate when the working tree is known to be disposable
<code>git push --force</code> , <code>git push --force-with-lease</code>	🔴 DANGEROUS	Replaces the server's branch; discards commits others pushed	<code>git fetch</code> , then <code>git log HEAD..origin/<branch></code>	Another clone's copy; the old ID from the push output or the Activity view. Appropriate for your own unshared branch, with an explicit expected value
<code>git reflog expire --expire=now --all</code> , <code>git gc --prune=now</code>	🔴 DANGEROUS	Delete the reflogs and every unreachable object	<code>git fsck --unreachable</code> lists what would go	None. Appropriate only for purging sensitive data from a clone after rotation

29.15 Version notes

Version note. Older behavior: `git config --list`, `git config --get <key>`. Current behavior: `git config list`, `git config get`, with the old forms still accepted. Since: Git 2.46. Recommended: the subcommands; on an older Git use the dashed options (Chapter 14B, section 14B.3).

Version note. Older behavior: `git fsck` treated every reflog entry as a starting point. Current behavior: it skips reflog entries dated later than the real clock. Since: Git 2.53.0 (`git/git commit f6b262581a`; Chapter 3, section 3.8). Recommended: remember it when a machine's clock was wrong during an incident.

Version note. Older behavior: Events API `PushEvent` payloads listed the pushed commits. Current behavior: they carry `before`, `head` and `ref`, without commit summaries. Since: 7 October 2025 ([changelog](#)). Recommended: take the two IDs from the event and read the commits with Git.

29.16 Practice

- Labs 35.1 to 35.3 in the Module 35 lab manual: the ten-command diagnosis on three prepared repositories, five operations in progress read from `.git` alone, and "preserve evidence, then fix". Answers are in the Module 35 lab answers.
- Replay the two worked cases with `labs/run ch29/case-unfinished-rebase` and `labs/run ch29/case-wrong-upstream`, then work in the sandboxes they leave behind.
- A drill: after the replay of case 1, `rescue/latency-wip` still holds the conflict resolution. With `git log`, `git cherry` and `git range-diff` only, describe what option A would have published.
- Keep the troubleshooting playbook open during the incident drills of Chapter 30.

29.17 Interview questions

- A developer says "my commits are gone". Which commands do you run before you touch anything, in which order, and what does each tell you?
- `git push` prints "Everything up-to-date" and the commit is not on the server. Give three states that produce this and the command that separates them.
- Without running `git status`, how do you tell from the `.git` directory whether a merge, a rebase, a cherry-pick, a revert or a bisect is in progress? Name the files.
- During a rebase, when does the branch ref move, and what follows from that for commits made while the rebase is stopped?
- What does `git rebase --abort` do to commits created during the stopped rebase and to uncommitted edits? How do you protect each before running it?
- Compare a backup ref, a copy of the repository and a bundle: what does each preserve, and what does each miss?

7. A push is rejected, `git pull` says "Already up to date", and the push is rejected again. Explain the mechanism and the lowest-risk fix.
8. `main` on GitHub moved backwards overnight. Which evidence exists on your machine, which on GitHub, how long does each last, and which of them identifies the account that pushed?
9. Rank these fixes by risk and justify the order: `git revert`, `git reset --hard` followed by a force push, `git reset --keep`, `git cherry-pick`, a new branch.
10. What does it mean to verify a fix? Give the four checks and an example where the first passes and the third fails.
11. A pull request shows 400 changed files for a two-line change. List four mechanisms and the Git command that tests each one locally.

29.18 Sources

Primary sources

- The manual pages of Git 2.55.0, read locally with `git help -m <command>`: `git-status`, `git-rebase`, `git-merge`, `git-cherry-pick`, `git-revert`, `git-bisect`, `git-bundle`, `git-update-ref`, `git-for-each-ref`, `git-ls-remote`, `git-merge-tree`, `git-check-ignore`, `git-fsck`, `git-config`, `gitrevisions`, `gitrepository-layout`.
- `git-prompt.sh` as installed by Homebrew's Git at `/opt/homebrew/etc/bash_completion.d/git-prompt.sh`: the header comments document `__git_ps1`.
- GitHub Docs: [Using the activity view to see changes to a repository](#); [REST: list repository activities](#); [Issue event types](#); [GitHub event types: PushEvent](#); [REST API endpoints for events](#); [Viewing insights for rulesets](#); [Reviewing the audit log for your organization](#); [Audit log for an enterprise](#); [Deleting and restoring branches in a pull request](#); [Troubleshooting commits](#); [Troubleshooting required status checks](#).
- GitHub Changelog: [Upcoming changes to GitHub Events API payloads](#).

Secondary sources

- Phase 0 report, sections 12 and 13, and the research notes `github_platform.md`, from which every GitHub-side fact and flag in section 29.8 is taken.
- Yang et al., the ACM TOSEM study of Git-command questions ([author preprint](#)). The report flags the final volume and article number as not confirmed.

Videos (optional; the assessments in the Phase 0 report rest on captions, not on full viewing)

- [How to Undo Mistakes With Git Using the Command Line](#), Tobias Günther for freeCodeCamp, 55 minutes, 24 November 2020: restore, amend, revert, reset, recovery with the reflog. Caveat: `master` naming.
- [Lecture 5: Version Control and Git](#), MIT Missing Semester 2026, 1 hour 10 minutes: the data model before the commands, and the reflog as a safety net.

Further reading

- Chapter 13: Recovery, for every case in which something was lost.
- Chapter 12: Remote Operations, section 12.14, and Chapter 16: Authentication, sections 16.17 to 16.20, for the two other diagnosis procedures of this book.
- Chapter 18: Branch Protection and Rulesets, section 18.17, "Why can't I merge?".
- The troubleshooting playbook.

Chapter 30: Incident Response

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Transcripts are real output from `labs/incidents/`. GitHub-side steps are described from GitHub's documentation, with links, and were not run.

30.1 Why this matters

At some point a message like this arrives: "`main` is broken, the release is in an hour, and nobody knows what happened." The CTO does not want a command. The CTO wants four answers: what happened, how you know, what you changed and how you proved it worked, and why it will not happen again. Chapter 29: Production Troubleshooting gave you the method for one repository and one symptom. An incident adds three things the method alone does not cover: other people (whose clones hold evidence and who keep working while you investigate), time pressure (which makes the destructive shortcut attractive), and an audience that has to decide something on the basis of what you tell them.

This chapter has four parts. Sections 30.2 to 30.4 define an incident, a severity scale and the ten practice incidents. Sections 30.5 to 30.14 work through the ten incidents, each built by a script under `incidents/` so that you can attempt it before you read the answer. Sections 30.15 to 30.20 cover the parts that are not Git: the summary for a CTO, the three scenarios that define the senior standard, postmortems, and controls. The remaining sections are the reference tables.

No new Git mechanism is taught here. Every recovery uses techniques from earlier chapters, mostly Chapter 13: Recovery, Chapter 12: Remote Operations and Chapter 9: Rebase. What is new is the order in which you apply them when the report is wrong, the clock is running, and three clones disagree.

30.2 An incident, and the loop that handles one

In one sentence. An incident is a problem in shared state (a shared branch, a published history, a credential, a pipeline) that other people depend on while you are still working out what it is.

Analogy. A ward doctor follows a fixed sequence with an unstable patient: stabilise, record, diagnose, treat, confirm, hand over. The sequence exists because under pressure people skip to treatment. The analogy breaks in two places, both in your favor. A repository can be copied in seconds, so a treatment can be rehearsed on a copy. And Git almost never destroys committed work by itself, so "stabilise" usually means stopping people, not stopping a process.

Precisely. The loop has seven stages. It is the root-cause framework of Chapter 1, section 1.10, with the stages that only exist when other people are involved.

Stage	What you do	Typical commands	Risk
1. Stabilise	Stop the damage from spreading: ask people to stop pushing and pulling the affected branch; revoke a leaked credential	none in Git; a message, a provider console	none
2. Preserve	Give every state you may need a name before anything moves	<code>git branch rescue/<what> <id></code> , <code>git fetch</code> , a copy of a clone	●
3. Diagnose	Evidence, at least three hypotheses, one distinguishing command each	<code>git status -sb</code> , <code>git reflog show <ref></code> , <code>git ls-remote</code> , <code>git range-diff</code> , <code>git log --graph</code>	●
4. Recover	The change that destroys least and is simplest to undo	<code>git cherry-pick</code> , <code>git merge</code> , <code>git reset --keep</code> , <code>git push --force-with-lease=<ref>:<expect></code>	●, sometimes ●
5. Verify	Re-run the commands that showed the problem; run the check the affected system runs	<code>git merge-base --is-ancestor</code> , <code>git diff --stat</code> , <code>git ls-remote</code>	●
6. Communicate	Tell the team what to do with their clones; tell the CTO the four answers	none	none
7. Prevent	Turn the root cause into a control, and write the postmortem	a ruleset, a configuration default, a test, a checklist line	none

Two rules hold the loop together. Stages 2 and 3 use only commands that add refs or read state. And stage 6 starts early: a first message goes out as soon as you know which branch people must leave alone, long before you know the cause.

Inside `.git`. In an incident the evidence is spread over several repositories, and each kind lives in a specific place.

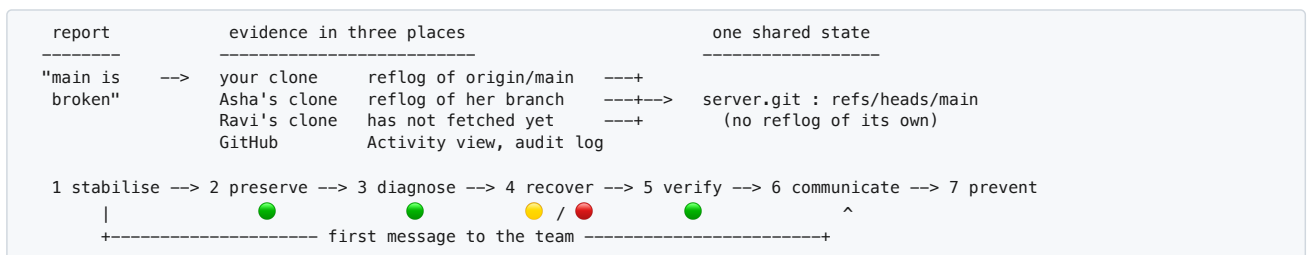
Evidence	Where it lives	Read it with	Lifetime
Every value a local branch had	<code>.git/logs/refs/heads/<branch></code> in that clone	<code>git reflog show <branch></code>	90 days; 30 for entries no longer reachable; gone when the branch is deleted
Every commit HEAD was on	<code>.git/logs/HEAD</code>	<code>git reflog</code>	same; survives branch deletion

Evidence	Where it lives	Read it with	Lifetime
What the server's branch was at each fetch or push from this clone	<code>.git/logs/refs/remotes/origin/<branch></code>	<code>git reflog show origin/<branch></code>	same; gone when the ref is pruned
What the server holds now	the server	<code>git ls-remote origin</code>	current value only
Staged content that was never committed	a dangling blob in <code>.git/objects</code>	<code>git fsck --lost-found</code>	two weeks after it becomes unreachable, once maintenance runs
Who pushed what, and force pushes	GitHub, not Git	the Activity view, <code>PushEvent</code> records, the audit log	see section 30.6

The retention periods are Git's defaults (Chapter 13, section 13.4); the lab configuration switches reflog expiry off. A bare repository, which is what a server holds, keeps no reflog unless `core.logAllRefUpdates` is set. So the history of a server-side branch exists only as the sum of what the clones remember, plus whatever the hosting platform records. That is why "preserve" includes asking colleagues not to run `git fetch --prune`, `git gc` or a re-clone until you have read their reflogs.

See it. Sections 30.5 to 30.14 are this loop run ten times, with real transcripts. The shortest complete example is incident 2 in section 30.6: one message, one fetch, two reflogs, two pushes.

Picture.



In production. An ML platform team deploys an inference service from the branch `production`. The deploy job refuses to run. Stage 1 is one sentence in the channel: "Nobody pushes to or resets `production` until I say so." Everything after that is incident 4 in section 30.8.

30.3 Severity

Severity decides who is told and how fast, not how interesting the Git problem is. The scale below is a working scale for repository incidents, to be adapted to your organisation's own. It asks three questions: which state was affected, could wrong code or a secret have reached users or outsiders, and is anything unrecoverable.

Level	Definition	Examples from this chapter	Who is told, when
SEV 1	A secret that unlocks production was exposed, or wrong code reached production, or shared history is unrecoverable	A live credential pushed anywhere (incident 3 before rotation); a rewritten production branch that was deployed	CTO and security at once; updates on a fixed rhythm
SEV 2	A shared default or release branch held wrong content, or a merged fix was silently lost; caught before or shortly after release	Incidents 2, 4 and 9	Engineering lead at once; CTO in the summary
SEV 3	A team is blocked or a pull request is unreviewable; no wrong content on a protected branch	Incidents 5, 6, 7 and 10	The team; lead on request
SEV 4	One person's local work	Incidents 1 and 8	The person; nobody else unless a habit needs changing

Three rules for using a scale. Assign the level on what could have happened in the window, not only on what did: incident 4 shipped nothing and is still SEV 2, because the only thing between the rewritten branch and production was one check. Raise the level the moment a secret is involved; lowering it later costs nothing. And never let severity depend on who caused the incident.

30.4 The ten incidents, and how to run them

Each incident is a directory under `incidents/` with a generator, a symptom report and a check script. The generator builds a bare repository `server.git` in the role of the server and clones named `you`, `asha` and `ravi`.

```
incidents/02-force-push-wrong-branch/generate.sh # prints the path of the sandbox
labs/shell "<that path>"                       # work in it; read SYMPTOMS.md first
incidents/02-force-push-wrong-branch/check.sh    # exit status 0 when the recovery is complete
```


The generator runs with the fixed lab clock, so the commit IDs in your sandbox are the IDs printed in this chapter. Attempt an incident before you read its section: a diagnosis you have read cannot be made again.

#	Incident	Mechanism	Layer of the cause	Decisive evidence	Recovery	Control
1	Accidental <code>git reset --hard</code>	A branch ref moved; index and working tree overwritten	Git	Branch relog; <code>git fsck --lost-found</code>	Anchor, <code>cherry-pick</code> , restore the blob	<code>git restore</code> , <code>reset --keep</code> , push early
2	Force push to the wrong branch	<code>push.default=upstream</code> plus <code>--force</code>	Git configuration; no GitHub rule	Reflow of <code>origin/main</code> ; <code>git branch -vv</code> in the pushing clone	New branch for the stray commits; restore with an explicit lease	Block force pushes on <code>main</code>
3	Committed secret	A deletion adds a snapshot and removes none	Git; issuer; GitHub retention	<code>git log --all --<path>; git branch -r --contains</code>	Rotate first; rewrite; prune everywhere	Ignore rule, push protection
4	Production history rewritten	Interactive rebase plus <code>--force</code> ; one commit dropped	Git; no GitHub rule	The deploy tag; tree diff of old and new tip	Restore, carry the newer commit over, realign clones	Ruleset on the production branch
5	Shared branch rebased	A merge pull joined the old and the rebased series	Git	The two relogs; <code>git range-diff</code>	Replay only the unpublished commits onto the rebased tip	<code>pull.rebase</code> , agreement before a rewrite
6	Pull request shows 500 changes	Another branch merged into the head branch	Git; GitHub displays it	<code>git log --merges main..head</code>	Rebuild the head without the merge	Update only from the base
7	CI passes locally, fails on Actions	Shallow clone without tags	GitHub Actions (a checkout default)	The failed step's one line; a depth-1 clone	<code>fetch-depth: 0</code>	Review tools with their inputs
8	Branch appears to have disappeared	Squash merge, head branch deleted, prune, <code>-D</code>	GitHub; Git	HEAD relog; tree diff against <code>main</code>	Anchor; cherry-pick the unmerged commit	New branch after every merge
9	Misunderstood merge conflict	<code>--ours</code> for a whole file	Git	<code>git show --remerge-diff</code>	One forward commit	Resolve hunks; review merges
10	Commit local, not remote	Two remotes; <code>origin</code> is a fork	Git	<code>git remote -v; git ls-remote</code>	Publish to the team repository; fix the upstream	One meaning of <code>origin</code>

Sections 30.5 to 30.14 use the same ten headings for every incident: symptoms, evidence, hypotheses, diagnostic commands, root cause, safe recovery, verification, prevention, communication, postmortem. The complete transcripts, with every step, are in `solutions/incident-NN-<slug>.md`. Incidents 3, 5 and 7 are the three scenarios of the senior standard; their sections here are summaries, and sections 30.16 to 30.18 work through them in full.

30.5 Incident 1: a developer runs `git reset --hard` by accident

Symptoms. Ravi: "I think `git pull` ate my work." Three or four commits of Friday are missing from `git log`; he "reset to what is on the server"; a new file had been added; an edit was in progress; he has committed once since. (`incidents/01-hard-reset`)

Evidence. `git status -sb` prints `## feature/escalation-rules` with no upstream, so there is no "this branch on the server". The branch relog is the record:

```
$ git relog show feature/escalation-rules
4e4c0b7 feature/escalation-rules@{0}: commit: Mention escalation in the README
2876d93 feature/escalation-rules@{1}: reset: moving to origin/main
0322a16 feature/escalation-rules@{2}: commit: Never escalate spam
a26c697 feature/escalation-rules@{3}: commit: Route escalated tickets to the on-call queue
95d110d feature/escalation-rules@{4}: commit: Add escalation predicate
44fa655 feature/escalation-rules@{5}: branch: Created from HEAD
# Ravi suspects "git pull". Does the HEAD relog record a pull at all?
$ git relog | grep -c pull
0
[exit status: 1]
```

Hypotheses. (1) A pull rewrote the branch: no `pull` line exists, the count is 0. (2) The commits were made elsewhere: no, three `commit:` lines are in this branch's relog. (3) A reset moved the branch away from them: `@{1}` says `reset: moving to origin/main`.

Diagnostic commands. `git status -sb`, `git branch -vv`, `git relog show <branch>`, then `git fsck --lost-found` for content that was staged and never committed.

Root cause. Ravi wanted to discard one edit and used a command that moves the branch, replaces the index and overwrites the working tree. `origin/main` was not his branch's counterpart; no counterpart existed. Layer: Git, one laptop.

Safe recovery. The lost work sits on three layers with three different answers.

What	Where Git has it	Recoverable
Three commits	reachable from the branch reflog, @ {2}	yes
rules/priority.yaml, staged, never committed	a dangling blob	yes, as content without a name
The unstaged edit to rules/routing.yaml	nowhere	no

Anchor first with `git branch rescue/before-reset 'feature/escalation-rules@{2}'`. Moving the branch back to `0322a16` would discard the commit made after the reset, so the three commits are added on top instead: `git cherry-pick origin/main..rescue/before-reset` 🟡. The staged file comes from the object database:

```
$ git fsck --lost-found
dangling blob 959c2056cb4f78bc321cf5bcf50cf2bf42d049bd
$ ls .git/lost-found/other
959c2056cb4f78bc321cf5bcf50cf2bf42d049bd
$ git cat-file -p 959c205
outage: 1
billing: 2
how-to: 3
```

`git cat-file -p 959c205 > rules/priority.yaml` writes it back. A blob has no file name; the name was in the index entry that the reset removed.

Verification. `git range-diff origin/main rescue/before-reset HEAD` shows three `=` pairs: each copy carries the same change as its original. The rescue branch is deleted only after that.

Prevention. `git restore <path>` discards an edit to one file. `git reset --keep` moves a branch and refuses to overwrite local changes (Chapter 11: Reset, Revert, Restore, section 11.6). A branch pushed on its first day has a second copy.

Communication. To Ravi: what is back (three commits, one file as untracked content), what is not (the unstaged edit, which Git never stored), and that `git pull` was not involved. Nobody else needs a message. SEV 4.

Postmortem. One line in the team notes: `reset --hard` is not "undo". The unrecoverable part was the part that was never given to Git.

30.6 Incident 2: a developer force-pushes to the wrong branch

Symptoms. `main` on the server ends in "WIP ... tests still red"; a merged commit is gone from its log. Asha pushed "only my branch" after an amend. Your `git status` says up to date. (`incidents/02-force-push-wrong-branch`)

Evidence. `git status` reports on `origin/main`, your clone's memory. `git ls-remote origin` asks the server, and the server's `main` is a commit your clone has never seen. After a fetch:

```
$ git fetch
From ../server
+ ca03e42...b4554be main      -> origin/main (forced update)
$ git status -sb
## main...origin/main [ahead 2, behind 2]
$ git reflog show origin/main
b4554be refs/remotes/origin/main@{0}: fetch: forced-update
ca03e42 refs/remotes/origin/main@{1}: update by push
27215c8 refs/remotes/origin/main@{2}: pull: fast-forward
f273cd3 refs/remotes/origin/main@{3}: update by push
```

`origin/main@{1}` is the value before the event. `git log origin/main..'origin/main@{1}'` lists what the server lost (two reviewed commits), and the reverse range what it gained (Asha's two WIP commits).

Hypotheses. (1) Her branch was merged into `main`: then the reviewed commits would still be there. (2) Someone reset `main` deliberately: no clone's local `main` moved. (3) Her push of `feature/dedupe` went to `main`.

Diagnostic commands. Look at the clone that pushed:

```
# Why did a push from feature/dedupe move main? Look at the clone that pushed.
$ git -C ../asha branch -vv
* feature/dedupe b4554be [origin/main] WIP config flag and readable dedupe, tests still red
  main          f273cd3 [origin/main: behind 2] Add row validation
$ git -C ../asha config get --show-origin push.default
file:.git/config      upstream
$ git -C ../asha reflog show origin/main
b4554be refs/remotes/origin/main@{0}: update by push
```

Root cause. `feature/dedupe` was created from `origin/main` and so has `origin/main` as upstream. With `push.default=upstream`, a bare `git push` updates the upstream branch whatever its name; `--force` removed the fast-forward check; the server accepted a forced update of `main`. The default, `simple`, would have refused (Chapter 12, section 12.5). Layers: Git configuration in one clone, and a missing GitHub rule.

Safe recovery. First confirm that your `origin/main@{1}` is the newest good value: Ravi's unfetched `origin/main` must be its ancestor. Then two pushes in this order.

```
git push origin origin/main:refs/heads/feature/dedupe # ● additive: Asha's commits get their own branch
```

```
$ git push --force-with-lease=main:b4554be origin rescue/main-before-force:main
To ../server.git
+ b4554be...ca03e42 rescue/main-before-force -> main (forced update)
$ git status -sb
## main...origin/main
```

● A forced update of `main`. It is appropriate here because the commits being removed are unreviewed, now live on their own branch, and nobody built on them. The lease `main:b4554be` makes it conditional: if anyone pushed to `main` meanwhile, it is refused. Had someone already built on the bad tip, the right move would be `git revert` of the two commits. In Asha's clone: `git branch --set-upstream-to=origin/feature/dedupe feature/dedupe` and `git config unset push.default`.

Verification. `git ls-remote origin` shows `main` at `ca03e42` and `feature/dedupe` at `b4554be`.

Prevention.

GitHub, not Git. "Block force pushes" in a ruleset on the default branch rejects this push; it is enabled by default in a new ruleset ([available rules](#); Chapter 18: Branch Protection and Rulesets). When no clone has the old value, GitHub offers instruments of its own: the repository's [Activity view](#) lists force pushes with the user and a comparison, a `PushEvent` in the [Events API](#) carries the `before` ID (last 300 events and 30 days), and the [references API](#) can create a branch at that ID:

```
gh api repos/OWNER/REPO/git/refs -f ref=refs/heads/recovered -f sha=<commit ID from before the force push>
```

Unverified. GitHub documents each instrument and not this combined procedure, publishes no retention period for commits that no ref reaches, and documents no "restore" action inside the Activity view (Phase 0 report, section 13). GitHub's own advice is the plain Git route: ask a collaborator who still has the commit to push it ([troubleshooting commits](#)).

Communication. At once: nobody pulls or pushes `main`. After the restore: `git fetch`, then `git status -sb`; a clone whose `main` shows ahead stops and asks. To the CTO: `main` pointed at unreviewed work for a stated number of minutes, nothing was lost, the rule goes on today. SEV 2.

Postmortem. The timeline comes from three clones' reflogs of `origin/main`, because the server kept none. The `(forced update)` line was printed and read as normal.

30.7 Incident 3: a secret is committed

Symptoms. Ravi committed `.env` with a password, pushed, deleted the file in a later commit, pushed again: "the branch is clean now, it was only my branch, the repository is private." (`incidents/03-committed-secret`; the string is a dummy.)

Evidence. `git log --all -- .env` finds the adding and the deleting commit; `git branch -r --contains 76aa6c4` lists two branches on the server, because Asha based her work on his.

Hypotheses. Each claim in the report is a hypothesis, and each is false: a deletion removes nothing from history; two branches reach the commit; "private" limits the audience and does not end the exposure.

Diagnostic commands. `git log --all -- <path>`, `git branch -r --contains <commit>`, `git show <commit>:<path>`, and the pushing clone's `git reflog show --date=iso origin/<branch>` for the time of the first push.

Root cause. An unignored file with a secret in the working tree, staged by `git add .`. Layers: Git stores it; the issuer accepts it; GitHub may keep it.

Safe recovery. Rotate the credential first, before any Git command. Then assess, rewrite the two unmerged branches, prune the old objects on the server and in every clone. Section 30.17 works through all six steps.

Verification. No ref in any repository reaches a commit with the file, and `git cat-file -t 76aa6c4` fails everywhere. Verification of the step that matters is in the provider's console: the old password is rejected.

Prevention. `.env` in `.gitignore`; staging by name; push protection for recognised formats.

Communication. The team gets exact instructions for their clones; the CTO gets the exposure window and what the provider's logs show. SEV 1 until rotation is confirmed.

Postmortem. Blameless and specific: the developer reported it himself. That behavior is the one to protect.

30.8 Incident 4: production branch history is rewritten

Symptoms. The release job refuses to deploy: the tagged, running commit is not an ancestor of `production`. Asha saw "diverged" without local commits, reset to the server on advice, and shipped a commit on top. Ravi "tidied" history: "same code, fewer commits". A finance ticket reports unrounded tax amounts. ([incidents/04-production-history-rewritten](#))

Evidence. After `git fetch`, your untouched `production` is ahead 4, behind 2: the server's branch was replaced. Three independent witnesses name the old tip:

```
# Three independent records of the old tip: my branch, the reflog of origin/production, the tag.
$ git rev-parse production 'origin/production@{1}' 'deploy-2026-09-07^{commit}'
3277739ce8e6fd4a425fdae7a124985414b5dd36
3277739ce8e6fd4a425fdae7a124985414b5dd36
3277739ce8e6fd4a425fdae7a124985414b5dd36
$ git merge-base --is-ancestor deploy-2026-09-07 origin/production
[exit status: 1]
$ git log --format='%h %an, committed by %cn: %s' production..origin/production
4fae70f Asha Rao, committed by Asha Rao: Add invoice PDF footer
f2783aa Lab User, committed by Ravi Menon: Tax calculation, formatting and logging
```

The last line also shows who did what. The folded commit keeps its original author and names Ravi as committer: in rewritten history, read `%cn` and the reflogs, not the author.

Hypotheses. (1) The tag was moved: no, it still names the tip every clone had. (2) The branch was reset to an older commit: no, the two new commits are not ancestors of anything old. (3) Rewritten with identical content: rewritten, yes; identical is a claim about trees, and it can be tested.

Diagnostic commands.

```
# "Same code, fewer commits" is a claim about trees. Compare the trees:
$ git diff --stat production origin/production
 billing/pdf.py | 2 ++
 billing/tax.py | 2 +-
 2 files changed, 3 insertions(+), 1 deletion(-)
$ git diff production origin/production -- billing/tax.py
diff --git a/billing/tax.py b/billing/tax.py
index 66a2912..0fce8dc 100644
--- a/billing/tax.py
+++ b/billing/tax.py
@@ -1,2 +1,2 @@
 def tax(amount, rate):
-     return round(amount * rate, 2)
+     return amount * rate
```

`billing/pdf.py` is Asha's new commit. `billing/tax.py` lost its rounding: the interactive rebase dropped the commit "Round tax to two decimals" when a line of the todo list was deleted. That is the finance ticket.

Root cause. A branch that deployment records refer to by commit ID accepted a forced update. The rewrite created new commits and lost one; `--force` replaced the server's ref; "reset to the server" then made a second clone adopt the rewrite. Layers: Git, and a missing GitHub rule.

Safe recovery. Two routes: keep the rewritten history and re-add the fix (no further force, but the deployed tag is never an ancestor again and every recorded ID points off-branch), or restore the old history and carry the one newer commit over. For a branch whose IDs are recorded elsewhere, restore.

```
git branch --no-track rescue/production-rewritten origin/production      # 🟢 name the state about to be replaced
git switch production && git cherry-pick origin/production              # 🟡 Asha's commit onto the real history
git push --force-with-lease=production:4fae70f origin production         # 🔴 conditional on the server's current value
```

Each clone that adopted the rewrite then proves that nothing of its own is lost before it moves:

```
$ cd ../asha
$ git fetch
From ../server
+ 4fae70f...a323df1 production -> origin/production (forced update)
$ git status -sb
## production...origin/production [ahead 2, behind 5]
# Is anything of mine missing from the server? "-" means the server has an equivalent change.
$ git cherry -v origin/production
+ f2783aa72335f59cef8731e0d5ed0dda990cd72 Tax calculation, formatting and logging
- 4fae70f86a56f9c739bd55c2e9830f4fba9145da Add invoice PDF footer
$ git reset --keep origin/production
$ cd ../ravi
$ git fetch
From ../server
+ f2783aa...a323df1 production -> origin/production (forced update)
```

```
$ git cherry -v origin/production production
+ f2783aa72335f59cef8731e0d5ed00dda990cd72 Tax calculation, formatting and logging
$ git reset --keep origin/production
```

`git cherry -v` marks with `-` a local commit whose change the server already has. The only `+` is the commit being retired on purpose.

Verification. `git merge-base --is-ancestor deploy-2026-09-07 origin/production` exits 0: the release job's own test. `git show origin/production:billing/tax.py` has the rounding.

Prevention. A ruleset on `production`: block force pushes, restrict deletions, require a pull request, empty bypass list (Chapter 18, section 18.18). In the handbook: "diverged, and I made no commits" means the server's history was replaced; stop and ask.

Communication. To the team: the new tip, and the two commands (`git cherry -v`, then `git reset --keep`) with the rule for any unexpected `+`. To the CTO: nothing bad was deployed because the release job refused; a dropped fix was caught; the missing rule goes on today. SEV 2.

Postmortem. The control that worked was an automated ancestry check. The control that was missing was one checkbox. The advice "reset to the server" turned one affected clone into two.

30.9 Incident 5: a developer rebases a shared branch

Symptoms. A pull request lists nine commits where it should list five; Asha's appear twice; a merge commit nobody intended joins them; the changed files look as before. Asha rebased and force-pushed with a lease. You ran `git pull` and pushed. (`incidents/05-rebased-shared-branch`)

Evidence. The commit list `main..head` has nine entries; the diff `main...head` has the same three files as before, because two copies of one change are one change in a tree.

Hypotheses. (1) A display problem on GitHub: `git ls-remote` and a local `git log` show the nine commits are real. (2) Someone undid the rebase: the rebased commits are all present. (3) The old series was merged with the rebased one.

Diagnostic commands. `git reflog show <branch>` and `git reflog show origin/<branch>` side by side; `git range-diff` between the old and the rebased series; `git config get pull.rebase`.

Root cause. Commits that another person had built on were rewritten, and that person's `git pull`, configured to merge, joined both versions. `--force-with-lease` is not at fault: it protects the server's commits, not the unpublished commits in a teammate's clone. Layer: Git.

Safe recovery. Keep the rebased series, replay only the unpublished commits onto it, publish with an explicit lease. Section 30.16 does this in the eleven steps.

Verification. Five commits, each subject once, no merge commit; the tree is identical to the tree before the repair.

Prevention. `pull.rebase true` or `pull.ff only`; a rewrite of a shared branch is announced with the command teammates need.

Communication. To Asha: nobody undid the rebase; `git pull --ff-only` brings her up to date. To reviewers: the diff never changed. SEV 3.

Postmortem. A pull that printed no error produced a nine-commit history, and it was pushed without a look at the graph.

30.10 Incident 6: a pull request suddenly shows 500 unrelated changes

Symptoms. Ravi pushed one small commit; his pull request went from three commits and two files to 500+ files and commits by Asha. He suspects a rewritten `main` or a changed base. The approval still stands. (`incidents/06-pr-500-changes`; the pull request is simulated as `main..head` and `main...head`, which is what GitHub computes, Chapter 17: Pull Requests, section 17.3.)

Evidence.

```
$ cd you
$ git fetch
From ../server
 * [new branch]      develop    -> origin/develop
 * [new branch]      feature/snippet-highlight -> origin/feature/snippet-highlight
# The commit list of the pull request, and the size of its diff:
$ git log --format='%h %an: %s' origin/main...origin/feature/snippet-highlight
5dc3109 Ravi Menon: Highlight every query term
461c3ea Ravi Menon: Merge branch 'develop' of ../server into feature/snippet-highlight
f94e7f0 Ravi Menon: Escape HTML in snippets
6d230a6 Ravi Menon: Test snippet highlighting
bdb4a59 Ravi Menon: Add snippet highlighting
069daa8 Asha Rao: Switch the tokenizer to ICU word breaking
5f0c2ae Asha Rao: Regenerate golden fixtures
796fd96 Asha Rao: Add golden fixture generator
$ git diff --shortstat origin/main...origin/feature/snippet-highlight
504 files changed, 517 insertions(+), 1 deletion(-)
```

```
$ git diff --dirstat=files,5 origin/main...origin/feature/snippet-highlight
99.2% tests/golden/
```

Hypotheses. The documented causes of an oversized pull request are listed in Chapter 17, section 17.12. Three fit the report. (1) `main` was rewritten: the relog of `origin/main` has one entry and the merge base equals the tip of `main`. (2) The head branch was force-pushed: the pushing clone's relog shows two ordinary pushes. (3) Another branch was merged into the head branch.

Diagnostic commands.

```
# Hypothesis 3: another branch was merged into the head branch.
$ git log --merges --format='%h %an: %s%n           parents: %p' origin/main...origin/feature/snippet-highlight
461c3ea Ravi Menon: Merge branch 'develop' of ../server into feature/snippet-highlight
           parents: f94e7f0 069daa8
$ git log --oneline --graph origin/main...origin/feature/snippet-highlight
* 5dc3109 Highlight every query term
* 461c3ea Merge branch 'develop' of ../server into feature/snippet-highlight
|\
| * 069daa8 Switch the tokenizer to ICU word breaking
| * 5f0c2ae Regenerate golden fixtures
| * 796fd96 Add golden fixture generator
* f94e7f0 Escape HTML in snippets
* 6d230a6 Test snippet highlighting
* bdb4a59 Add snippet highlighting
```

The merge commit `461c3ea` has the tip of `develop` as second parent. `git log --first-parent` shows the branch as Ravi experienced it, four commits and a merge, which is why "one small commit" felt true. His relog names the command: `pull --no-rebase origin develop`.

Root cause. A topic branch aimed at `main` was updated from `develop`, an unreleased long-running branch. A merge makes every commit of the merged branch an ancestor of the result, and a pull request lists everything reachable from the head and not from the base. Layer: Git. GitHub displayed the branch correctly.

Safe recovery. The tempting fix is `git revert -m 1 461c3ea`: no forced push, and the diff shrinks. It is wrong here. The commits of `develop` stay ancestors of the branch, so the commit list stays long; and once this branch is merged, `main` contains those commits together with a commit that undoes them, so the later release of `develop` brings nothing: the re-merge problem of Chapter 11, section 11.9. A reverted merge is not a removed merge. The branch has one author, so it is rebuilt:

```
git branch rescue/with-develop          # 🟢 anchor the current tip
git rebase --onto 461c3ea^1 461c3ea      # 🟡 replay what came after the merge onto its first parent
git push --force-with-lease --force-if-includes # 🟠 for a topic branch with one author: appropriate
```

Verification. `git log origin/main...<head>` lists four commits, `git diff --stat origin/main...<head>` two files, and `git merge-base --is-ancestor origin/develop <head>` exits 1.

Prevention. Update a topic branch from the branch it will be merged into, and name that branch in the command. Reviewers read the commit list, not only the files.

GitHub, not Git. Whether an approval survives a push depends on the rules: "dismiss stale pull request approvals" removes it when the diff changes (Chapter 17, section 17.5). Here an approval given to two files stood on 504. Treat it as void and review again.

Communication. To Ravi: GitHub showed what the branch contained; the relog line; the last commit has a new ID. To the lead: the near miss is the standing approval, not the file count. SEV 3.

Postmortem. Nothing was merged. Had it been, 500 fixtures and an unreleased tokenizer would have reached `main` under an approval for two files.

30.11 Incident 7: CI works locally but fails on GitHub Actions

Symptoms. The step "Compute version" fails on the runner with exit code 128; the script works on every laptop; re-runs fail identically; `main` has been red since the step was added. The reporter suspects a flaky runner and asks for an administrative merge. (`incidents/07-ci-passes-locally`; the run report is constructed for the exercise.)

Evidence. The workflow file and the description of the run. The failed step printed one line, and it is a Git message: `fatal: No names found, cannot describe anything.`

Hypotheses. (1) Flakiness: excluded, because the failure repeats and began with a commit. (2) The tag is missing on the server: it is there. (3) The runner's clone lacks the tag.

Diagnostic commands. The fixed investigation order of section 30.18; then a clone like the runner's, made with Git.

Root cause. `actions/checkout` fetches one commit and no tags by default; `git describe` needs history and tags. Layer: GitHub Actions, a documented default.

Safe recovery. `fetch-depth: 0` on the checkout step, on a branch, as a pull request. A second defect waits behind the first: a template name whose case differs between the code and the repository.

Verification. A green run on the fix branch, observed on GitHub. The files alone prove that the cause was removed, not that the job passes.

Prevention. A workflow change that adds a tool is reviewed with the inputs the tool reads. A red default branch is an incident on its first day.

Communication. The answer to "merge it anyway" is no, with the reason. SEV 3.

Postmortem. Five days of red on `main` hid a second, real defect.

30.12 Incident 8: a branch appears to have disappeared

Symptoms. Asha's `feature/prompt-versioning` is on neither the server nor her laptop; none of her commit messages are in `git log main`; she is sure everything was pushed; she needs her last commit, the variable validation. Ravi merged her pull request "with the green button". (`incidents/08-branch-disappeared`)

Evidence. No ref of that name exists anywhere. The reflog of the branch went with the branch; the reflog of HEAD did not:

```
# The branch reflog went with the branch. The HEAD reflog is still here:
$ git reflog show feature/prompt-versioning
fatal: ambiguous argument 'feature/prompt-versioning': unknown revision or path not in the working tree.
Use '--' to separate paths from revisions, like this:
'git <command> [<revision>...] -- [<file>...]'
[exit status: 128]
$ git reflog --format='%h %gd %gs'
1a7ca10 HEAD@{0} pull: Fast-forward
c8dbea7 HEAD@{1} checkout: moving from feature/prompt-versioning to main
70df7f7 HEAD@{2} commit: Validate prompt variables before save
ceaa8bc HEAD@{3} commit: Add version history
a93889a HEAD@{4} commit: Load a prompt by version
c12fb6d HEAD@{5} commit: Store every save as a new version
c8dbea7 HEAD@{6} checkout: moving from main to feature/prompt-versioning
c8dbea7 HEAD@{7} clone: from $LAB/incidents/solve-08-branch-disappeared/server.git
```

Hypotheses. (1) Renamed: no similar branch on the server. (2) Deleted by mistake, work lost. (3) Merged, and the head branch deleted after the merge. (4) "Everything was pushed."

Diagnostic commands. `git log -2 --format='%h author %an, committer %cn: %s' main` shows "Add prompt versioning (#42)", author Asha, committer Ravi: the shape of a squash merge, which keeps none of the original messages. Then the question "which work is where":

```
# The last commit made on the lost branch, from the HEAD reflog:
$ git log --oneline main..70df7f7
70df7f7 Validate prompt variables before save
ceaa8bc Add version history
a93889a Load a prompt by version
c12fb6d Store every save as a new version
# Patch IDs cannot see through a squash: every commit looks unmerged.
$ git cherry -v main 70df7f7
+ c12fb6d8347452b8124fe1d59b2f25c589155a6f Store every save as a new version
+ a93889ae2c557817889f98a9629eb7560ad5a984 Load a prompt by version
+ ceaa8bc13e03ca5676baa71d4401845ee3002318 Add version history
+ 70df7f70daac6ced8a343ab10d9509c1e363cf17 Validate prompt variables before save
# Trees can. What does the old tip have that main does not?
$ git diff --stat main 70df7f7
 registry/validate.py | 9 ++++++++
 1 file changed, 9 insertions(+)
$ git diff --stat main 70df7f7~1
```

`git cherry` reports all four commits as unmerged, which is misleading: a squashed commit equals none of its parts. Trees are reliable. `main` and the third commit have identical trees; the fourth commit adds one file and was never pushed.

Root cause. Four steps, each reasonable alone. GitHub squash-merged the pull request and deleted the head branch. `fetch.prune` removed the remote-tracking ref. Work had continued on the merged branch. `git branch -d` refused, correctly, because squashed commits are not ancestors of `main`, and `-D` overrode the refusal. Layers: GitHub for the first step, Git for the rest.

GitHub, not Git. A closed pull request offers **Restore branch** for its head branch ([deleting and restoring branches](#)). It restores what the server had. A commit that was never pushed is not on the server, and Support cannot produce it.

Safe recovery.

```
git branch rescue/prompt-versioning 70df7f7
git switch -c feature/prompt-validation main
git cherry-pick rescue/prompt-versioning
git push -u origin feature/prompt-validation
```

● anchor the tip from the HEAD reflog
● a new branch from the current main
● the one commit that is not merged

Recreating and pushing the old branch would publish three commits whose content is already in `main`: the reused-branch problem of Chapter 17, section 17.12.

Verification. `git log origin/main..origin/feature/prompt-validation` lists one commit, and `git diff --stat rescue/prompt-versioning feature/prompt-validation` prints nothing: the new branch has everything the lost one had.

Prevention. A new branch after every merge. Before `-D`: `git diff main <branch>`. "Pushed" is read from `git status -sb`, not from memory.

Communication. To Asha: nothing was deleted by mistake; three commits are in `main` as one; the fourth survived only in her reflog. SEV 4.

Postmortem. The only copy of a day's work was a reflog entry, which by default expires 30 days after its commit becomes unreachable.

30.13 Incident 9: a merge conflict is misunderstood

Symptoms. Two reviewed fixes by Asha are missing from `limiter/bucket.py` on `main`. Her commits are in `git log main`; there is no revert; `git log -- limiter/bucket.py` does not list them. Ravi resolved a conflict in that file and "kept ours". (`incidents/09-misunderstood-conflict`)

Evidence. `git merge-base --is-ancestor <her commit> main` exits 0, and no subject starts with "Revert". The commits are in the history and their effect is not in the file. Only a merge can do that without a diff of its own in the usual views.

Hypotheses. (1) Corruption: nothing supports it. (2) A revert: none. (3) A later commit overwrote the lines: none touches the file. (4) A merge resolution discarded her side.

Diagnostic commands. First, why her own search found nothing:

```
# The command Asha ran, and the same command without history simplification:
$ git log --oneline -- limiter/bucket.py
f8435ea Look up the rate per tenant
d40b668 Add token bucket limiter
$ git log --oneline --full-history -- limiter/bucket.py
fb67f49 Merge pull request #17 from feature/per-tenant-limits
d7497e2 Merge remote-tracking branch 'origin/main' into feature/per-tenant-limits
8a11540 Halve the default rate after the overload
bfd1f07 Cap the bucket at BURST tokens
f8435ea Look up the rate per tenant
d40b668 Add token bucket limiter
```

With a path, `git log` simplifies history: at a merge whose result for the path equals one parent, it follows only that parent. Her commits are on the discarded side, so the default view hides exactly the commits under investigation. Then the audit of the merge:

```
# What did the person who made this merge change, compared with what Git would have produced?
$ git show --remerge-diff --format='%h %an: %s' d7497e2
d7497e2 Ravi Menon: Merge remote-tracking branch 'origin/main' into feature/per-tenant-limits

diff --git a/limiter/bucket.py b/limiter/bucket.py
remerge CONFLICT (content): Merge conflict in limiter/bucket.py
index ebfcaa2..771c807 100644
--- a/limiter/bucket.py
+++ b/limiter/bucket.py
@@ -1,8 +1,4 @@
-<----- f8435ea (Look up the rate per tenant)
-  DEFAULT_RATE = 100
-=====
-RATE = 50
->>>>>> 8a11540 (Halve the default rate after the overload)
-  BURST = 200

@@ -13,5 +9,5 @@ def allow(key, now):

  def refill(bucket, now, rate):
  -   bucket.tokens = min(BURST, bucket.tokens + (now - bucket.ts) * rate)
  +   bucket.tokens = bucket.tokens + (now - bucket.ts) * rate
      bucket.ts = now
```

`--remerge-diff` re-runs the merge and shows what the human changed relative to Git's mechanical result. The first hunk was a real conflict, resolved for one side. The second hunk had no conflict: Git had merged the cap cleanly, and the recorded merge removes it. A resolution that

changes a hunk Git had already merged is the signature of taking one side of the whole file.

Root cause. `git checkout --ours limiter/bucket.py`. "Ours" was read as "the team's version". In `git merge X`, ours is the branch you are on, here the feature branch; in a rebase the roles are swapped (Chapter 8: Merge, section 8.15; Chapter 9, section 9.11). Layer: Git. GitHub merged what it was given.

Safe recovery. `main` is shared and four commits sit on the faulty merge, so nothing is rewritten and nothing is reverted (a revert of the merge would remove Ravi's feature). One new commit restores both changes, adapted to his rename, and its message names the merge and the two lost commits.

Verification. `git diff <Asha's last commit> <fix> -- limiter/bucket.py` shows only Ravi's two intended changes. A correct resolution contains each side's diff completely.

Prevention. Resolve hunks, not files. Review a merge with `git show --remerge-diff` before pushing it. A regression test for the cap would have failed on Ravi's branch.

Communication. To both: the repository is intact, nobody reverted anything, and why `git log -- <file>` hid it. To the CTO: the outage recurred because a merged fix was removed inside a merge commit, where review could not see it. SEV 2.

Postmortem. Detected by an outage, not by a control. The pull request diff could not show the loss, because after the bad merge the branch and the merge base agreed on those lines.

30.14 Incident 10: a commit exists locally but not remotely

Symptoms. A release candidate lacks a fix. Ravi sees the commit in `git log`, `git status` says up to date with `origin/main`, and the push printed `main -> main`. Asha cannot find it in the team repository. (incidents/10-commit-local-not-remote)

Evidence. Everything Ravi said is confirmed in his clone. "On the server" has a hidden parameter: which server.

Hypotheses. (1) Never pushed: `git status -sb` would show `ahead`. (2) Pushed to another branch. (3) Made in detached HEAD. (4) `origin` is not the repository the team means. (5) GitHub shows stale data.

Diagnostic commands.

```
$ git remote -v
origin ../ravi-fork.git (fetch)
origin ../ravi-fork.git (push)
upstream ../server.git (fetch)
upstream ../server.git (push)
$ git branch -vv
* main f3bb790 [origin/main] Guard against NaN in score aggregation
$ git reflog show origin/main
f3bb790 refs/remotes/origin/main@{0}: update by push
# Ask each server directly, without relying on any remote-tracking ref:
$ git ls-remote origin main
f3bb79056b4293f31ae8479851be75b1f71339a6      refs/heads/main
$ git ls-remote upstream main
ebd3afc65730975fcbd60abdc9302003fe8e5843     refs/heads/main
```

`git ls-remote` asks each server directly and bypasses every remote-tracking ref. The fork has the commit; the team repository does not.

Root cause. `origin` in this clone is Ravi's old fork, and `main` follows `origin/main`. A remote name is a local alias. Every local signal was true about the fork. Layer: Git configuration.

Safe recovery. The team repository has moved on, so the fix goes on top of its `main`, on a branch, as a pull request requires.

```
git fetch upstream
git switch -c fix/nan-aggregation && git rebase upstream/main      # 🟡 one unpublished commit, new ID
git push -u upstream fix/nan-aggregation                        # then the pull request
git switch main && git branch --set-upstream-to=upstream/main main
git cherry -v upstream/main main                                # "-": the team repository has an equivalent
git reset --keep upstream/main                                  # 🟡 after the merge of the pull request
```

Verification. `git ls-remote upstream main`, and `git merge-base --is-ancestor <fix> upstream/main`.

Prevention. Proof of a push is the `To <url>` line, `git ls-remote`, or the commit's page in the team repository; never `git status`. A release checklist checks promised fixes by ancestry.

Communication. To both: every command Ravi quoted told the truth about another repository. SEV 3.

Postmortem. The release manager looked for a specific commit instead of trusting "it is pushed". That check is now a line in the checklist.

30.15 The incident summary a CTO needs

A CTO reads the summary to decide three things: whether customers or outsiders are affected, whether anything is still at risk, and whether to spend money or attention on prevention. Commands do not help with any of them. The summary has four parts, in this order, and fits on one screen.

Part	The question it answers	Rules
1. What happened	What was affected, for how long, with what impact	Impact first, in business terms. Times with a time zone. No names
2. Root cause	Which mechanism, on which layer	One sentence, with the layer named: a Git default, a GitHub rule that was missing, a GitHub Actions default
3. What was done, and how it was verified	Is it over	The recovery in one sentence, and the check that proves it. State what is not yet verified
4. Prevention	Will it recur	The control, its owner, its date. A control, not a promise to be careful

Written for incident 4:

```
Subject: [Resolved] billing-api: production branch history rewritten (SEV 2, no customer impact)

What happened  Between <time> and <time> IST the branch "production" of billing-api pointed at a rewritten
                history. Nothing was deployed from it: the release job refused. A staging build from that
                branch showed unrounded tax amounts.
Root cause     A history clean-up (interactive rebase) on "production" was force-pushed. It replaced four
                commits by one and dropped the tax-rounding fix. Git allows this; our GitHub rules for the
                branch did not block force pushes.
What was done  The recorded history was restored from the deployment tag and the one change shipped in
                between was carried over. Verified: the deployed commit is again an ancestor of the branch
                (the release job's own check), and the rounding fix is in the file. All three known clones
                were realigned. Not yet verified: clones on CI caches; they are rebuilt tonight.
Prevention     A ruleset on "production" (no force pushes, no deletions, pull request required, no bypass).
                Owner: platform team. Active since <time> today.
```

Five habits make such a summary trustworthy. Separate what you observed from what you infer, and say which is which. Give every time and every ID a source: the reflog entry, the tag, the log line. Say "I do not know yet" early, with the time of the next update. Never write that something was verified unless you ran the check. And send a first, short version before the recovery, as soon as the team knows what to leave alone: silence during an incident is read as "it is worse than they say".

The same four parts, one sentence each, are the spoken answer when the CTO asks in a corridor.

30.16 Senior standard 1: a branch was rebased and force-pushed, and the pull request is broken

This is incident 5 again, worked in the eleven steps that define the standard. Steps 1 to 6 change nothing.

Step 1: inspect the state. Your clone first, then what the pull request shows. `git status -sb` says the branch equals `origin/feature/online-serving`: locally nothing looks wrong.

```
# What a pull request into main lists: the commits, then the changed files.
$ git log --oneline origin/main...origin/feature/online-serving
6e7ebbd Merge branch 'feature/online-serving' of ../server into feature/online-serving
d4fd03c Add cache warm-up job
5e57ed9 Decode online values
c8d1712 Add online lookup
57b5972 Decode batched values
af1f6de Add batched online lookup
4967e71 Add cache warm-up job
8f79c42 Decode online values
be7fb7a Add online lookup
$ git diff --stat origin/main...origin/feature/online-serving
 store/batch.py | 3 +++
 store/online.py | 3 +++
 store/warm.py  | 4 ++++
 3 files changed, 10 insertions(+)
```

Nine commits, with three subjects twice, and the same three files as before. The commit list and the file view answer different questions (Chapter 17, section 17.3), which is why one reviewer sees a broken pull request and another sees an unchanged one.

Step 2: inspect refs. `git for-each-ref refs/heads refs/remotes` for every ref and its value, and `git ls-remote origin` for the server's own answer. The local branch, the remote-tracking branch and the server agree on `6e7ebbd`. So this is not a stale view: the nine commits are what the repository contains.

Step 3: inspect the reflog. Two reflogs, read together from the bottom up.

```
$ git relog show feature/online-serving
6e7ebbd feature/online-serving@{0}: pull: Merge made by the 'ort' strategy.
57b5972 feature/online-serving@{1}: commit: Decode batched values
af1f6de feature/online-serving@{2}: commit: Add batched online lookup
4967e71 feature/online-serving@{3}: pull: Fast-forward
8f79c42 feature/online-serving@{4}: commit: Decode online values
be7fb7a feature/online-serving@{5}: commit: Add online lookup
65cb28c feature/online-serving@{6}: branch: Created from HEAD
$ git relog show origin/feature/online-serving
6e7ebbd refs/remotes/origin/feature/online-serving@{0}: update by push
d4fd03c refs/remotes/origin/feature/online-serving@{1}: pull: forced-update
4967e71 refs/remotes/origin/feature/online-serving@{2}: pull: fast-forward
8f79c42 refs/remotes/origin/feature/online-serving@{3}: update by push
$ git config get pull.rebase
false
```

Your branch: fast-forward to the shared tip, two commits of your own, then `pull: Merge made by the 'ort' strategy`. The remote-tracking branch: `pull: forced-update`. The server's branch was replaced, and your pull, configured with `pull.rebase=false`, merged the replacement with what you had.

Step 4: identify the old branch state. Three commits, by name: the shared tip before the rebase, `4967e71 (feature/online-serving@{3})`; the rebased tip, `d4fd03c (origin/feature/online-serving@{1})`; your tip before the pull, `57b5972 (feature/online-serving@{1})`. The merge commit confirms two of them as its parents: `git show --no-patch --format='%h parents: %p' prints 57b5972 d4fd03c`.

Step 5: understand what changed.

```
# Are the two copies the same changes? Compare the old series with the rebased series:
$ git range-diff origin/main~1..4967e71 origin/main..d4fd03c
1: be7fb7a = 1: c8d1712 Add online lookup
2: 8f79c42 = 2: 5e57ed9 Decode online values
3: 4967e71 = 3: d4fd03c Add cache warm-up job
$ git log -3 --format='%h author %ad committer %cd %s' --date=format:%H:%M 4967e71
4967e71 author 10:20 committer 10:20 Add cache warm-up job
8f79c42 author 10:11 committer 10:11 Decode online values
be7fb7a author 10:10 committer 10:10 Add online lookup
$ git log -3 --format='%h author %ad committer %cd %s' --date=format:%H:%M d4fd03c
d4fd03c author 10:20 committer 10:29 Add cache warm-up job
5e57ed9 author 10:11 committer 10:29 Decode online values
c8d1712 author 10:10 committer 10:29 Add online lookup
```

`git range-diff` pairs the old series with the rebased one: three = lines, the same changes under new IDs. The dates agree: a rebase keeps the author date and sets a new committer date (Chapter 9, section 9.3). The rebase lost nothing and changed nothing in content. The damage is entirely the merge that made both series reachable.

Step 6: preserve recoverable references. `git branch rescue/merged-state feature/online-serving` and `git branch rescue/my-work 57b5972`. The first makes the current state restorable and comparable; the second holds the only commits that exist in one clone alone.

Step 7: determine the safest recovery.

Option	Result	Verdict
Leave it; merge the pull request as it is	Duplicates and a pointless merge enter <code>main</code> (unless squashed)	Rejected: history that misleads <code>bisect</code> and <code>blame</code>
Force the old series back	Undoes Asha's rebase; the branch conflicts with <code>main</code> again; her clone diverges	Rejected: destroys a teammate's work
Keep the rebased series, replay only your two commits onto it	Five commits, once each; Asha's clone can fast-forward	Chosen

Step 8: restore the correct history.

```
# Back to my tip from before the pull (a local move), then replay only my two commits
# onto the rebased tip.
$ git reset --keep 57b5972
$ git rebase --onto d4fd03c 4967e71
Rebasing (1/2)
Rebasing (2/2)
Successfully rebased and updated refs/heads/feature/online-serving.
$ git log --online --graph origin/main~1..feature/online-serving
* 5e4c03f Decode batched values
* 021ec35 Add batched online lookup
* d4fd03c Add cache warm-up job
* 5e57ed9 Decode online values
* c8d1712 Add online lookup
* 8131d29 Lower the TTL to 15 minutes
```

`git reset --keep 57b5972` 🟡 is a local move back to your tip before the pull. `git rebase --onto d4fd03c 4967e71` 🟡 replays what lies after the old shared tip, your two commits, onto the rebased tip. Before publishing, two proofs: `git range-diff 4967e71..rescue/my-work`

d4fd03c..feature/online-serving pairs both commits with =, and `git diff --stat rescue/merged-state feature/online-serving` prints nothing. The repair changed history and no content.

Step 9: update the pull request safely. A pull request follows its head branch, so the update is a push to that branch.

```
$ git push --force-with-lease=feature/online-serving:6e7ebbd origin feature/online-serving
To ../server.git
+ 6e7ebbd...5e4c03f feature/online-serving -> feature/online-serving (forced update)
$ git log --oneline origin/main...origin/feature/online-serving
5e4c03f Decode batched values
021ec35 Add batched online lookup
d4fd03c Add cache warm-up job
5e57ed9 Decode online values
c8d1712 Add online lookup
$ git diff --stat origin/main...origin/feature/online-serving
store/batch.py | 3 +++
store/online.py | 3 +++
store/warm.py | 4 ++++
3 files changed, 10 insertions(+)
```

● The lease names `6e7ebbd`, the value you examined. If a teammate pushed after your fetch, the push is refused and you look again; a bare `--force` would overwrite their work. Tell the teammates before the push, not after.

GitHub, not Git. After a forced push to a head branch, review comments attached to replaced commits are shown as outdated, and an approval is dismissed if the rule "dismiss stale pull request approvals" is on and the diff changed (Chapter 17, sections 17.5 and 17.12). Here the diff is identical, and the commit list is what changed. Confirm on GitHub with `gh pr view <number> --json commits,changedFiles` and `gh pr diff <number> --name-only`.

Asha's clone stood at the rebased tip, of which the new tip is a descendant. For her, `git pull --ff-only` is the whole repair.

Step 10: explain what happened. Three sentences, no blame: "Asha rebased the shared branch onto `main` while I had two unpushed commits on the old version. My `git pull` merged the old and the rebased commits, and I pushed the result, so the pull request listed both. I rebuilt the branch as the rebased commits plus mine; no content changed, and reviews of the diff stay valid."

Step 11: prevent recurrence. `git config set pull.rebase true`: when an upstream was rewritten, `git pull --rebase` finds the old fork point in the reflog of the remote-tracking branch and replays only your commits (Chapter 9, sections 9.13 and 9.15). `pull.ff only` is stricter: the pull stops and you decide. And an agreement: whoever rewrites a shared branch announces it first, with the old tip and the command teammates need.

30.17 Senior standard 2: a secret was committed

The order is fixed and its first step is not Git: contain, assess, eradicate, recover, communicate, prevent (Chapter 21B: Repository Security and Secret-Leak Response, section 21B.14; Phase 0 report, section 14).

1. Contain: revoke or rotate the credential first. The damage happens where the secret is accepted, and automated scanners copy fast. Revocation works against every copy, including the ones you cannot reach. GitHub's documentation says that revoking or rotating may be sufficient and that a history rewrite may not be warranted ([removing sensitive data](#)). Every `git` command typed before this step is time in which the key still works.

2. Assess. Five facts for the record: which secret and what it can reach; the first commit and the time of the first push; which refs contain it; who could read it; whether it was used. Git answers the middle three:

```
# Which branches on the server can reach the commit that added it?
$ git branch -r --contains 76aa6c4
  origin/feature/digest-template
  origin/feature/email-digest
# In how many snapshots is the file present?
$ git grep -c SMTP_PASSWORD $(git rev-list --all) -- .env
fdd903cc43054ab432ffc9a7e35df247b713aded:.env:1
76aa6c4fe4793ad6b99759e54fe9b57cea95fc3d:.env:1
# Since when has it been on the server? The pushing clone recorded it:
$ git -C ../ravi reflog show --date=iso origin/feature/email-digest
bb4230f refs/remotes/origin/feature/email-digest@{2026-09-07 10:24:00 +0530}: update by push
fdd903c refs/remotes/origin/feature/email-digest@{2026-09-07 10:19:00 +0530}: update by push
# Is the branch "clean now"? The tip has no .env. The history under the tip has:
$ git cat-file -e origin/feature/email-digest:.env
fatal: path '.env' does not exist in 'origin/feature/email-digest'
[exit status: 128]
$ git show 76aa6c4:.env
SMTP_HOST=smtp.example.com
SMTP_USER=digest@example.com
SMTP_PASSWORD=lab-fixture-not-a-real-password
```

Two branches reach the commit, two snapshots contain the file, and the server has had it since the push at 10:19. Whether it was used is answered only by the issuer's logs.

3. Eradicate. Remove the secret from current code, and decide whether to rewrite history. Rewrite when the data stays harmful after rotation or the affected history is small and unmerged, as here. Do not rewrite by reflex for a revoked key on a busy `main`: a rewrite costs every collaborator their clone, changes every recorded commit ID and removes signatures (Chapter 21B, section 21B.16). The documented tool for a whole repository is `git-filter-repo` 2.47 or later with `--sensitive-data-removal`; it is not installed here, and one short branch needs only an interactive rebase that edits the adding commit and drops the deleting one. `git range-diff` shows the result:

```
$ git log --oneline main..feature/email-digest
4397ba9 Schedule the digest hourly
bd86515 Sort digest events by time
8d98701 Add SMTP sender
cb446bd Add digest builder
$ git log --oneline feature/email-digest -- .env
$ git range-diff 'feature/email-digest@{u}'...feature/email-digest
1: 76aa6c4 ! 1: 8d98701 Add SMTP sender
@@ Metadata
## Commit message ##
Add SMTP sender

- ## .env (new) ##
+ ## .gitignore ##
@@
+SMTP_HOST=smtp.example.com
+SMTP_USER=digest@example.com
+SMTP_PASSWORD=lab-fixture-not-a-real-password
+ __pycache__ /
+ .env

## notify/smtp.py (new) ##
@@
2: fdd903c = 2: bd86515 Sort digest events by time
3: c5d0303 < -: ----- Remove env file
4: bb4230f = 3: 4397ba9 Schedule the digest hourly
$ git status -sb --ignored
## feature/email-digest...origin/feature/email-digest [ahead 3, behind 4]
!! .env
```

After `git push --force-with-lease --force-if-includes`, every branch built on the old commits is transplanted with `git rebase --onto <new> <old> <branch>`. A merge or a plain pull from a stale clone brings the old commits back.

4. Recover. Deploy the new credential. Then remove the old objects, which a force push does not do:

```
$ cd ../you
# No ref on the server reaches the old commits now. The objects are still there:
$ git -C ../server.git cat-file -t 76aa6c4
commit
$ git -C ../server.git show 76aa6c4:.env
SMTP_HOST=smtp.example.com
SMTP_USER=digest@example.com
SMTP_PASSWORD=lab-fixture-not-a-real-password
```

In the sandbox you administer the server and run `git gc --prune=now` there. On GitHub that is a request to Support, who need the first changed commit and the number of affected pull requests and who assist only where rotation cannot mitigate the risk; GitHub states that old commits can otherwise stay reachable by ID in cached views, in forks and through pull requests (same page). Each clone keeps the objects through its reflogs until it is re-cloned or runs  `git reflog expire --expire=now --all && git gc --prune=now`, which destroys that clone's entire safety net.

5. Communicate. The team is told which branches were rewritten and exactly what to do with a clone that fetched them. The CTO is told the exposure window, what the credential could reach, what the issuer's logs show, and that rotation is complete. Concealment is the one choice that makes a leak worse.

6. Prevent. An ignore rule in the project template; staging by name and reading `git diff --cached`; push protection, which blocks recognised token formats and not free-form passwords ([push protection](#)); short-lived credentials.

30.18 Senior standard 3: GitHub Actions suddenly fails

"Suddenly" is a claim to test first. Either something in the repository changed (a commit to the workflow, the lock file, the code) or something outside it did (a runner image, an action behind a moving tag, a secret, a cache). `git log --oneline -- .github/workflows` and the list of recent runs separate the two in a minute:

```
gh run list --workflow ci.yml --branch main --limit 20      # where does green turn red?
gh run view <run-id> --json event,headBranch,headSha,workflowName
```

Then the investigation follows a fixed order, from "did the right thing start" to "what did it say" (Chapter 20B: Delivery, Runners, Cost and Debugging, section 20B.11). An answer early in the list makes the later ones irrelevant.

#	Area	The question	Typical finding
1	Workflow	Which file, from which commit, defined the run?	<code>schedule</code> uses the file on the default branch; <code>workflow_dispatch</code> uses the file on the ref it was dispatched against, and can be started only if the file exists on the default branch (Chapter 20B, section 20B.11)
2	Event	What triggered it, and which commit was checked out?	<code>pull_request</code> builds the merge ref, not your head commit
3	Permissions	What could the token do?	Unlisted scopes are <code>none</code> ; fork and Dependabot runs are read-only
4	Runner	Which image and size?	A <code>-latest</code> label moved; private repositories get smaller runners
5	Environment	Did the job reference one, and did its rules pass?	Waiting for approval; wrong branch for the environment
6	Dependencies	Were the same versions installed as locally?	No lock file, or not installed with <code>--locked</code>
7	Secrets	Were they present?	An unset or withheld secret is an empty string, not an error
8	Action versions	Which commit of each action ran?	An old major on a new Node runtime; a moved tag
9	Logs	What did the failed step print?	<code>gh run view <run-id> --log-failed</code>
10	Artifacts	Were the expected files produced and passed on?	Wrong name; expired
11	Cache	What was restored, under which key?	A broad <code>restore-keys</code> prefix restored a stale cache
12	Concurrency	Was the run cancelled or replaced?	Conclusion <code>cancelled</code> ; a shared group name

Applied to incident 7: areas 1 to 8 are answered from the workflow file and the run report without a log (one event, read-only token, fixed runner label, no environment, no secrets, both actions pinned by commit SHA). Area 9 gives one line, and it is a Git message. That points at the repository the runner had, and Git can build the same one:

```
# What the checkout step does by default, reproduced with Git: depth 1 and no tags.
$ git clone --quiet --depth 1 --no-tags "file://$PWD/../server.git" ../runner-checkout
$ cd ../runner-checkout
$ git rev-parse --is-shallow-repository
true
$ git rev-list --count HEAD
1
$ git tag --list
$ bash scripts/version.sh
fatal: No names found, cannot describe anything.
[exit status: 128]
```

One commit, no tags, exit status 128. `actions/checkout` fetches a single commit by default, with `fetch-tags` off; `fetch-depth: 0` fetches all history for all branches and tags ([checkout README](#), v7.0.1). The step was added without changing the checkout it depends on.

Two habits complete the standard. Look behind the first failure: the job never reached its tests, and `git ls-files` shows a template tracked as `summary.md.tmp1` while the code opens `Summary.md.tmp1`, which works only on a filesystem that ignores case. And refuse the shortcut: a red required check is not overridden by an administrator because the change "is only a docstring". The honest status after the fix is pushed is "cause removed in the files; confirmed when the run on the fix branch is green". Re-run with `gh run rerun <run-id> --failed --debug` only when the log is not enough; a re-run reuses the original commit and ref.

30.19 Blameless postmortems, with a template

In one sentence. A postmortem is a written record of an incident that explains how the system allowed it, so that the system can be changed; "blameless" means it treats every person's action as reasonable given what they knew.

Why blameless is a technical requirement. The evidence in this chapter came from people: Ravi's reflog, Asha's account of the reset, a developer saying "I committed a password". People who expect blame run `git gc`, re-clone, or stay silent, and the evidence is gone. Google's SRE book states the principle as assuming that everyone involved "had good intentions and did the right thing with the information they had" ([Postmortem Culture](#), a secondary source). The useful question is never "who ran the command" but "why did running it look right, and why did nothing stop it".

The test. Replace every name by a role. If the document still explains the incident, it is about the system. "Ravi force-pushed production" explains nothing. "The production branch accepted a forced update from any member with write access" names something that can be changed.

```
# Postmortem: <one-line title>          Severity: SEV n          Status: draft | reviewed | actions closed

## Summary          Four sentences: what happened, root cause with its layer, what was done, what prevents it.
## Impact          Who and what was affected, for how long. What could have happened in the window.
## Timeline          Time (with zone) | event | source of the fact (reflog entry, tag, log line, message)
## Detection          How it was noticed, by whom or what, and how long after it began.
## Root cause          The seven-line root-cause box of Chapter 1, section 1.10.
## Contributing conditions  What made the action look right. Which control was missing or bypassed.
## Recovery          What was done, in order, with risk labels. What was verified, and how.
## What went well          The controls and habits that limited the damage.
## What went badly          Without names.
## Actions          Control | type (prevent, detect, recover) | owner | date | how we will know it works
## Evidence          Transcripts, commit IDs, links. Kept with the document.
```

Write it within days, while reflogs and memories exist. Have the people involved review the timeline. An action without an owner and a date is a wish; a postmortem whose only action is "be more careful" has not found the cause.

30.20 Turning a root cause into a control

A control is something that still works when the person is new, tired or in a hurry. Controls differ in strength, and the order below is the order in which to look for one.

Strength	Kind of control	Example from this chapter	Why it ranks here
1	The server refuses	A ruleset that blocks force pushes on <code>main</code> and <code>production</code> (incidents 2, 4)	Applies to every client and tool, whatever their configuration
2	Automation checks	The release job's ancestry test (4); a regression test (9); push protection (3)	Runs every time, but only on what it was written to see
3	A safe default on the client	<code>push.default=simple</code> , <code>pull.rebase=true</code> , an ignore rule in the template (2, 5, 3)	Works until someone has another configuration
4	A step in a checklist or review	<code>--remerge-diff</code> for merges (9); read the commit list (6); ancestry of promised fixes (10)	Depends on the reviewer doing it
5	Training and habit	<code>git restore</code> instead of <code>reset --hard</code> (1); a new branch after a merge (8)	Decays, and does not reach new people

From root cause to control in four questions. Which layer could have refused the action: the platform, the pipeline, the client, a reviewer? What is the strongest control available on that layer and on your plan (some ruleset features are plan-gated, Chapter 18, section 18.15)? What will the control block that is legitimate, and what is the path for that case? And how will you know it works: a ruleset's insights, a test that fails when the fix is removed, an attempt in a sandbox.

Prefer one strong control to five weak ones. For incident 4 the list could have ten items; the one that matters is a checkbox on the server.

30.21 What can go wrong

What goes wrong	How you notice	Fix	Prevention
A second "fix" destroys the evidence (<code>reset --hard</code> , <code>gc</code> , re-clone, <code>fetch --prune</code>)	A reflog entry or dangling commit you counted on is missing	Another clone, <code>git fsck --lost-found</code> , a backup; often nothing	Stabilise first: ask for no cleanup before reflogs are read
The old value is restored from a stale clone	Commits pushed shortly before the incident are missing afterwards	Compare candidates with <code>git merge-base --is-ancestor</code> ; restore the newest	Collect the old tip from every clone and from tags before choosing
A bare <code>--force</code> during recovery overwrites a teammate's new push	Their commit is absent from the server	Their clone still has it; push it again	<code>--force-with-lease=<ref>:<expect></code> always, with the value you examined
A stale clone pushes removed history back	Old commits or a removed secret reappear after a rewrite	Repeat the clean-up; find the clone	Instructions per clone; rebase, never merge, old branches (Chapter 21B, section 21B.18)
A revert is used where a merge had to be removed	A later merge of the same branch "brings nothing"	Revert the revert, or rebuild (Chapter 11, section 11.9)	Decide revert or rebuild by asking what will be merged later
Git clean-up starts before a credential is rotated	The key still works while history is being rewritten	Rotate now	The fixed order of section 30.17
The summary claims more than was checked	A follow-up question has no answer	Correct it in writing	"Verified" only for checks that were run

30.22 When not to use it, and dangerous edge cases

- **Do not run an incident process for a SEV 4.** One person's local reset needs ten minutes of help and one note, not a channel and a document.
- **Do not rewrite when adding suffices.** Incident 9 was repaired with one commit. A rewrite of shared history is a second incident for everyone who has a clone.
- **Do not rewrite history to remove a secret that is already revoked** on a busy default branch, unless the data stays harmful. The cost is certain and the benefit is cosmetic.
- **Reflogs are per clone and expire.** By default an entry for an unreachable commit lasts 30 days, and a deleted branch loses its reflog at once. Evidence that is days old may be hours from deletion on a machine that runs maintenance.
- **A lease is only as good as its expected value.** The bare `--force-with-lease` trusts the remote-tracking branch, which a background fetch in an editor updates silently (Chapter 12, section 12.8). In an incident, give the expected ID explicitly.
- `git pull --rebase` **after a forced update can drop your own pushed commit.** With no upstream named on the command line, the rebase uses the fork point from the reflog of the remote-tracking branch. A commit you had pushed, and that a force push then removed from the server, counts as old upstream history and is not replayed. The pull prints "Successfully rebased" and the commit is off your branch; `ORIG_HEAD` and the reflog still name it. Lab 36.3 shows it. After a forced update, look first and integrate by hand.
- `git reflog expire --expire=now --all` **with** `git gc --prune=now` **is the one recovery step that is itself unrecoverable.** It belongs only at the end of a secret clean-up, in a clone with no other unpublished work.
- **GitHub keeps what Git has let go.** A commit removed by a force push may stay retrievable by ID on GitHub; this helps recovery and defeats removal. GitHub publishes no retention period.
- **The sandbox is kinder than production.** In `incidents/` you may read every clone and administer the server. In real life you ask a colleague to run `git reflog show origin/main` and paste it, and the server-side steps go through rules, the Activity view and Support.

30.23 Command safety

What the recovery commands of this chapter change:

Command	Working tree	Index	HEAD	Current branch ref	Other refs and files in .git	Remote	GitHub
<code>git branch rescue/x <id></code> ●	unchanged	unchanged	unchanged	unchanged	new ref and its reflog	unchanged	unchanged
<code>git cherry-pick <range></code> ●	updated	updated	moves with the branch	advances by the copied commits	reflogs; <code>CHERRY_PICK_HEAD</code> during a conflict	unchanged	unchanged
<code>git reset --keep <id></code> ●	updated; refuses to overwrite local changes	updated	moves with the branch	set to <id>	<code>ORIG_HEAD</code> , reflogs	unchanged	unchanged
<code>git rebase --onto <new> <old></code> ●	updated	updated	detached during the run, then back on the branch	set to the last replayed commit	<code>ORIG_HEAD</code> , reflogs, <code>.git/rebase-merge/</code> during the run	unchanged	unchanged
<code>git push origin <id>:refs/heads/<new></code> ●	unchanged	unchanged	unchanged	unchanged	remote-tracking ref created	new branch	a new branch; push rules apply
<code>git push --force-with-lease=<ref>:<expect></code> ●	unchanged	unchanged	unchanged	unchanged	remote-tracking ref updated	branch replaced if it still equals <expect>	pull requests on that branch follow; rules may reject it
<code>git reflog expire --expire=now --all</code> then <code>git gc --prune=now</code> ●	unchanged	unchanged	unchanged	unchanged	all reflog entries removed; unreachable objects deleted	unchanged	unchanged

Command	Label	What it can destroy	Preview	Recovery	When appropriate
<code>git push --force-with-lease=<ref>:<expect></code>	●	Commits on the server that only the server and stale clones have	<code>git log <local>..<expect></code> lists what will leave the branch	A clone's reflog of <code>origin/<ref></code> ; on GitHub the Activity view	Restoring a known good value; publishing a rewritten topic branch
<code>git push --force</code>	●	The same, without any condition	none	the same	Not in an incident
<code>git reset --hard <id></code>	●	Uncommitted changes to tracked files	<code>git status</code> , <code>git diff HEAD</code>	none for unstaged work	Use <code>--keep</code> instead during recovery
<code>git branch -D <branch></code>	●	The branch reflog; the only name of unmerged commits	<code>git log main..<branch></code> , <code>git diff main <branch></code>	HEAD reflog, until it expires	After the diff against the target is empty

Command	Label	What it can destroy	Preview	Recovery	When appropriate
<code>git reflog expire --expire=now --all, git gc --prune=now</code>		Every recovery path in that clone	<code>git fsck --unreachable</code> lists what would go	none	Final step of a secret clean-up
<code>git rebase --onto, git cherry-pick, git reset --keep</code>		Nothing permanently	<code>git log <old>..HEAD</code>	<code>ORIG_HEAD</code> , the branch reflog	The standard recovery tools
<code>git revert -m 1 <merge></code>		Nothing now; a later merge of the same branch is silently empty	<code>git show --stat <merge></code>	revert the revert	Only when the merged branch will never be merged again as it is

30.24 Version notes

Version note. Older behavior: an unconfigured `git pull` on diverged branches merged, silently before Git 2.27 and with a warning after. Current behavior: without `pull.rebase`, `pull.ff` or a command-line choice it stops and asks how to reconcile. Since: Git 2.33.1 and 2.34.0 (Chapter 12, section 12.6). Recommended: set `pull.rebase true` or `pull.ff only`; incident 5 needed an explicit `pull.rebase=false` to happen.

Version note. `--force-if-includes` exists since Git 2.30 and does nothing when the lease names an explicit expected value. `git show -r` `remerge-diff` exists since Git 2.36. The `git config get|set|unset` subcommands need Git 2.46 or later.

Version note. Git 2.56 adds `git add --resolved`, which stages resolved paths and aborts if conflict markers remain (added in Git 2.56, not run here; Phase 0 report, section 13). It would not have caught incident 9, where no marker was left behind.

GitHub, not Git. All GitHub facts are as of 1 October 2026: rulesets block force pushes by default when created; push protection for users has covered public repositories since 29 February 2024; `actions/checkout v7.0.1` defaults to `fetch-depth: 1` without tags. Interface labels change; follow the linked pages.

30.25 Practice

- Module 36: incident drills, local and history: incidents 1, 8, 10, 9 and 5.
- Module 37: incident drills, remote, platform, CI and security: incidents 2, 4, 6, 7 and 3.
- Module 38: communication, postmortems and the senior standard: a CTO summary, a postmortem, and the three scenarios against the clock.
- Keep the disaster-recovery playbook and the one-page emergency sheet at hand; both use only commands verified in this book.

Do every drill before reading its solution in `solutions/`.

30.26 Interview questions

1. `main` was force-pushed ten minutes ago. Before you restore it, which two things do you check, and what does the restoring command look like?
2. A colleague says "I deleted the file with the key and pushed, so we are fine". Give the order of the response and say what the deletion did and did not do.
3. Why is `--force-with-lease` not a defense against the situation in which a teammate has unpublished commits on a branch you rebased?
4. A pull request shows the same changed files as yesterday and three times as many commits. What happened, and which two comparisons explain why the views differ?
5. When is `git revert -m 1` the wrong way to take a mistaken merge out of a topic branch?
6. A fix is in `git log main` and not in the file. Which command shows what happened, and why did `git log -- <file>` hide the commits?
7. The server keeps no reflog. Where does the timeline of a server-side branch come from, on plain Git and on GitHub?
8. A job that runs `git describe` fails only on GitHub Actions. Name the default responsible and two remedies.
9. What are the four parts of an incident summary for a CTO, and what must never be claimed in the third?
10. Rank these controls by strength for "nobody force-pushes production": a handbook rule, a client alias, a ruleset, a review checklist. Justify the order.

11. After a squash merge, with the head branch deleted on the server and pruned from your clone, `git branch -d` refuses to delete the merged branch. Why, and what do you check before `-d`?
12. What makes a postmortem blameless in practice, and why does that matter for the evidence?

30.27 Sources

Primary sources

- Local manual pages of Git 2.55.0: `git help reflog`, `git help push` (the lease forms), `git help rebase` (`--onto`, `--empty`), `git help cherry`, `git help range-diff`, `git help fsck`, `git help log` (history simplification, `--remerge-diff`), `git help describe`.
- GitHub Docs: [Activity view](#); [deleting and restoring branches in a pull request](#); [Events API](#); [create a reference](#); [troubleshooting commits](#); [available rules for rulesets](#); [removing sensitive data from a repository](#); [remediating a leaked secret](#); [push protection](#).
- [actions/checkout README](#), v7.0.1.
- The Phase 0 report, sections 12 to 14, including its unverified flags for the GitHub-side recovery recipe.

Secondary sources

- [git-filter-repo manual](#), for the clean-up of other clones after a rewrite.
- Google, *Site Reliability Engineering*, [Postmortem Culture: Learning from Failure](#).
- Julia Evans, [what can go wrong with rebasing](#).
- Truffle Security, [commits orphaned by force pushes](#): an observation by researchers, not a statement by GitHub.

Videos

- ["Tag, You're Leaked: Surviving the tj-actions Supply Chain Attack"](#), BSides PDX 2025, 24 minutes. A 72-hour incident response for GitHub Actions; the Phase 0 report calls it the closest thing to a production incident-response case study, and notes that it found no conference talk covering end-to-end Git incident recovery with current tooling.

Further reading

- Chapter 13: Recovery, Chapter 29: Production Troubleshooting, Chapter 21B: Repository Security and Secret-Leak Response, Chapter 20B: Delivery, Runners, Cost and Debugging.

Chapter 31: Interview Preparation

Source file: `interview/senior-engineer-interview-guide.md`

Senior-engineer interview guide: Git and GitHub

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Every technical statement in this guide comes from the textbook, and each worked answer names the section that demonstrates it. Nothing here was captured from GitHub; platform behavior is described as the textbook describes it, from the documentation.

1. What this guide is for

You already have the material: thirty chapters, the labs, and a question bank with model answers. This guide is about the last step, which is saying it aloud to someone who will not help you. It covers:

1. how senior and staff interviews on Git and GitHub are run, and what is being assessed (section 2);
2. the structure of a strong answer (section 3);
3. twenty answers worked at three quality levels, with commentary (section 4);
4. scenario and system-design prompts, with what a strong response covers (section 5);
5. a rubric for grading yourself (section 6);
6. a four-week plan (section 7).

The rules of an oral session are in the interview-mode protocol. Read it once before section 4.

One warning. The weak and adequate answers in section 4 are not strawmen. They are what capable engineers say, and each contains something true. Read them for the exact point at which they stop.

2. How these interviews are run, and what is assessed

There is no standard "Git interview". At senior and staff level the subject appears inside other conversations, in four recurring forms. Prepare for the form, not for a list.

Form	What it sounds like	What is being assessed
The mechanism question	"What exactly is a branch?" "Why does a rebase change commit IDs?"	Whether your model of Git is the real one: objects, refs, HEAD, the index, reachability. A candidate who has only used Git describes commands. A candidate who understands it describes state
The incident walk-through	" <code>main</code> moved backwards overnight. Go."	Method under pressure: read-only first, evidence, more than one hypothesis, the lowest-risk fix, verification. The interviewer plays the repository and answers only what you ask
The design discussion	"Design branching and protection for this team." "Review this workflow file."	Judgment: whether you can state what each choice costs, name the layer that enforces it, and say when you would choose otherwise
The explanation to a non-specialist	"Tell me, as the CTO, why deleting the file did not remove the key."	Whether you can keep the mechanism exact while dropping the jargon, and whether you lead with impact and end with a control

Across all four, an experienced interviewer is listening for six things. They are the six dimensions on which this course grades every oral answer.

- **Correctness.** Every claim is true, and the layer is right. "GitHub rejected the push" and "Git rejected the push" are different statements with different fixes. In Chapter 12 the textbook shows that a non-fast-forward rejection is decided by your own client, before the server is asked to change anything.
- **Depth.** You explain what happens inside `.git`, and why the tool is designed that way. "Why Git does this" is a line of every root-cause box in the textbook because the design reason tells you where the behavior ends.
- **Terminology.** Commit ID, the index, working tree, ref, remote-tracking branch, upstream, merge base, fast-forward, relog, reachable. Loose words are not a matter of style: "the branch on GitHub" and "`origin/main`" are two different refs in two different repositories, and several incidents in this course come from confusing them.
- **Reasoning.** You get from state to symptom by a causal chain, and you can say which observation would prove you wrong.
- **Practical understanding.** You name the commands, in an order that is safe: look, preserve, preview, change, verify.
- **Production awareness.** You know who else is affected, what the fix can destroy, and which control would have refused the action.

Three behaviors lose an interview faster than a wrong fact.

1. **Reaching for a command before describing the state.** "I would run `git reset --hard`" as a first sentence tells the interviewer how you behave in an incident. Chapter 29 opens with the reason: most damage is done after the incident, by the first repair attempt.
2. **Certainty without evidence.** "It must be a force push." The stronger sentence is "three mechanisms produce this; the branch reflog separates them".
3. **Blaming the tool.** "Git is weird about that." Each surprising behavior in the textbook has a mechanism and a design reason. An answer that ends in "weird" has not reached either.

And three behaviors mark a senior answer even when a detail is missing: saying which layer acted; saying what you would check before you believe your own explanation; and saying "I do not know that; here is how I would find out", followed by a command that would in fact find out.

Git, not GitHub. Interviewers often test the boundary deliberately. A pull request, a review, a ruleset, a fork, a release and a workflow run are GitHub objects. Commits, trees, blobs, tags, refs, the index and reflogs are Git. A squash merge button is GitHub creating a Git commit on its servers. When a question mixes the two, separate them in your first sentence (Chapter 1, section 1.5).

3. The structure of a strong answer

A strong answer has five parts, in this order. Not every question needs all five at the same length, but none may contradict another, and the first two are never optional.

Part	The question it answers	Typical opening
State	What do the working tree, the index, HEAD, the refs and the server hold?	"A branch is a ref: a name that holds one commit ID."
Mechanism	What does Git, GitHub or GitHub Actions do with that state, and why is it designed so?	"A commit names its parent by ID, so a new parent means a new ID for every commit after it."
Evidence	Which read-only command shows it? What would the output look like if I were wrong?	" <code>git reflog show main</code> has a reset entry if the branch was moved."
Fix	What is the lowest-risk change that repairs the state, and what does it touch?	"Anchor the old tip with a branch first; nothing is rewritten."
Prevention	Which habit, setting or rule stops a silent recurrence, and on which layer?	"A ruleset that blocks force pushes; a client setting is weaker."

This is the seven-line root-cause box of Chapter 1, section 1.10, arranged for speech. "Observed behavior" and "Git state" become State; "Mechanism" and "Why Git does this" become Mechanism; "Root cause" is the sentence that joins State to Mechanism; "Correct fix" and "Prevention" keep their names. Evidence is added because an interviewer cannot see your terminal.

How the five parts scale. For a definition question ("What is HEAD?"), State and Mechanism are the answer, Evidence is one command, and Fix and Prevention shrink to one sentence about the mistake the definition prevents. For an incident question, State and Evidence come first and take most of the time, because you do not yet know the mechanism. For a design question, Prevention is the answer, and the other four are the justification: which state the control protects, which mechanism it interrupts, how you will know it works.

Length. Two minutes at most. The model answers in the bank are held to 180 words so that each can be said, unhurried, within that time. If you need longer, you are narrating commands.

Three rules for the first sentence.

1. Answer the question that was asked. "Why does a rebase change commit IDs?" starts with "because", not with a definition of rebase.
2. Name the object or ref that the answer turns on. "A reset moves a ref and deletes no object."
3. Name the layer if two are possible. "That is a GitHub Actions default, not Git."

One rule for the last sentence. End on the control or the limit, not on a summary. "What it cannot recover is work that was never staged, because no object was written" is a better ending than "so the reflog is very useful".

An example in the five parts. Question: "`git status` says the branch is up to date with `origin/main`, and a colleague pushed ten minutes ago. Is Git wrong?"

State. No. `origin/main` is a ref in my repository that records where the server's `main` was at my last fetch. *Mechanism.* `git status` compares two local refs and opens no connection, because every command except the transfer commands works offline. *Evidence.* `git ls-remote origin main` asks the server without changing anything; `git fetch` then moves `origin/main`, and status reports the branch as behind. *Fix.* None is needed; the statement was true about my copy. *Prevention.* Fetch before any decision that depends on the server, such as tagging or cutting a hotfix branch, and let scripts compare against `git ls-remote`, never against a remote-tracking ref of unknown age.

That is the content of Chapter 12, section 12.4, in about a hundred words.

4. Twenty worked answers at three quality levels

Each answer is given three times. **Weak** is what the interviewer most often hears. **Adequate** would pass a mid-level interview: it is correct and incomplete. **Strong** has the five parts. The commentary names what moved the answer from one level to the next, and the textbook section that backs the strong version. Cover the strong answer, give your own aloud, then compare.

4.1 What exactly is a branch?

Weak. "A branch is a separate copy of the code where you can work without affecting main."

Adequate. "A branch is a pointer to a commit. Creating one is cheap because nothing is copied. When you commit, the pointer moves forward."

Strong. "A branch is a ref: a name under `refs/heads/` that holds the ID of one commit, its tip. That is all it holds. There is no list of its commits, no record of where it started, no owner and no creation date; the history of a branch is whatever the tip reaches through parent links. Creating a branch writes one ref and no object. A commit made while HEAD names the branch moves the ref to the new commit. I can show it: after `git branch topic`, `git for-each-ref refs/heads` prints the same commit ID for both branches. I read refs with Git and not from the file system, because refs can be packed, and in a refable repository there is no file per branch. The practical consequence is that deleting a branch deletes a name, and that Git cannot tell me which branch another was created from."

Commentary. The weak answer is the model that makes people afraid of branches and surprised that deleting one loses nothing at once. The adequate answer has the mechanism and stops before its consequences. The strong answer states what a branch does not record, which is what explains real questions: why "which commits belong to this branch" has no stored answer, and why the base of a pull request is GitHub's record and not Git's. Reference: Chapter 7, sections 7.2 and 7.9.

4.2 What is HEAD, and how does it differ from a branch and from a remote-tracking branch?

Weak. "HEAD is the latest commit. A branch is your local version and `origin/main` is the branch on GitHub."

Adequate. "HEAD points at the branch you have checked out. A branch points at a commit. `origin/main` is your local copy of the remote's branch and is updated when you fetch."

Strong. "All three are refs in my own repository; they differ in what they hold and in what moves them. A branch holds one commit ID, and a commit made on it moves it. HEAD records where I am: normally it is a symbolic ref that holds the name of a branch, not a commit, and in the detached form it holds a commit ID directly. A commit does not rewrite a symbolic HEAD; it moves the branch that HEAD names. A remote-tracking branch such as `origin/main` lives under `refs/remotes/` and records where a branch in another repository was at my last fetch or push. My commits never move it. `git symbolic-ref HEAD` and `git for-each-ref` show all of this. So `main` on the server and `main` in my clone are two refs that share a name, and 'up to date with `origin/main`' describes my copy of the remote, never the remote."

Commentary. "HEAD is the latest commit" is the most common wrong model, and the textbook refutes it with a transcript: the content of `.git/HEAD` is the same before and after a commit, while the branch ref changes. The adequate answer is right and does not say what moves each ref, which is the part an incident turns on. The strong answer ends on the consequence that causes real failures. Reference: Chapter 2, section 2.8; Chapter 7, sections 7.2, 7.3 and 7.10.

4.3 How does Git store data? What are a blob, a tree and a commit?

Weak. "Git stores the changes between versions, so it only keeps the diffs. A commit is a set of changes."

Adequate. "Git stores snapshots, not diffs. A blob is a file, a tree is a directory and a commit points to a tree and to its parent. Everything is identified by a hash."

Strong. "Git is a content-addressed object database plus names. A blob is the bytes of one file and nothing else: no name, no mode. A tree is one directory listing: entries of mode, name and object ID, each naming a blob or another tree. A commit names exactly one top-level tree, zero or more parents, an author and a committer with times, and a message. It contains no diff and no branch name. Each object's ID is computed from its content, so the ID is also its checksum, and an object is never changed: a change files new objects and moves a name. Changing one file therefore creates a new blob, a new tree for each directory above it, and a new commit. Deltas exist only inside packfiles as a storage choice between similar objects; they change no ID, and every commit still reads back as a complete snapshot. `git cat-file -p` shows each object."

Commentary. The weak answer describes how Git displays history, not how it stores it; every later topic (why a rebase changes IDs, why a deleted file is still in history) fails on that model. The adequate answer has the right nouns. The strong answer says what each object deliberately leaves out and places deltas correctly, which is the point where confident candidates most often go wrong. Reference: Chapter 3, sections 3.3, 3.4 and 3.7.

4.4 Why does a rebase change commit IDs?

Weak. "Because rebase rewrites history, so the commits get new hashes."

Adequate. "A commit's ID is a hash of its content, including its parent. Rebase gives the commits a new parent, so the IDs change, and so do the IDs of all later commits."

Strong. "Because a commit ID is the hash of the commit object, and a rebase does not move commits: it writes new ones. The replayed commit differs from the original in three inputs. The parent, which is the purpose of the operation. The tree, because the new snapshot also contains what the new base added. And the committer line, which records who created this object and when. The author line and the message are carried over. Each commit names its parent by ID, so a new ID for one commit forces a new ID for every commit after it; commits before the first rewritten one keep theirs. `git cat-file -p` on an original and its copy shows the fields, and `git range-diff` pairs old with new. The consequence is outside Git: anything that recorded an old ID, such as a review approval, a CI status or a training run, now refers to a commit that is on no branch. So I do not rebase commits that others have, or that something has recorded."

Commentary. The weak answer restates the question. The adequate answer is correct and treats the commit as a diff with a parent; it misses that the tree usually changes and that the committer changes always. The strong answer names the three inputs, bounds the effect ("from the first rewritten commit onward"), and ends on the production consequence. Reference: Chapter 9, sections 9.3 and 9.6; Chapter 6, section 6.4.

4.5 Why does `git reset --hard` not necessarily delete commits permanently?

Weak. "It does delete them, but Git keeps a backup for 30 days in the reflog."

Adequate. "Reset only moves the branch pointer. The commits are still in the repository and you can find them with `git reflog` and reset back."

Strong. "Because a reset moves a ref and deletes no object. `git reset --hard <commit>` makes the current branch name another commit and rewrites the index and the tracked files to match. The commits that left the branch are still in the object database, and they still have names: the reset writes the previous tip to `ORIG_HEAD` and appends an entry to the reflog of `HEAD` and to the reflog of the branch. Git does not delete an object that a reflog entry names. So `git reflog` shows the old tip, and I anchor it with `git branch rescue/<what> <id>` before doing anything else. By default the entry for a commit the branch no longer reaches is kept for 30 days. What a hard reset does destroy is uncommitted work that was never staged: no object was ever written for it, so neither the reflog nor `git fsck` can find it. That is why `git reset --keep` is the safer way to move a branch back."

Commentary. "Backup" is the word that marks the weak answer: nothing is copied, and the candidate cannot say what is and is not covered. The adequate answer would recover the commits and lose the unstaged edit. The strong answer draws the line between what has an object and what does not, which is the difference between a recoverable incident and an unrecoverable one. Reference: Chapter 11, sections 11.4 to 11.6; Chapter 13, section 13.2.

4.6 What is reachability, why can a commit become unreachable, and what does garbage collection do about it?

Weak. "Unreachable commits are commits that were deleted. Garbage collection cleans them up."

Adequate. "A commit is reachable if a branch or tag leads to it. After a reset or a rebase the old commits are unreachable, and `git gc` removes them after a while."

Strong. "An object is reachable if I can arrive at it from a starting point by following the IDs stored inside objects. The starting points are the refs, `HEAD`, the index and every reflog entry. A commit becomes unreachable when the last name that leads to it moves or is removed: a reset, an amend, a rebase, a deleted branch, a dropped stash, commits left on a detached `HEAD`. Usually a reflog entry still reaches it, so it becomes truly unreachable only when that entry expires: by default after 90 days, or 30 for an entry the current tip no longer reaches. Unreachable is not deleted. `git fsck` still finds the object, and pointing a ref at it makes it reachable again. Deletion happens when a collection finds an unreachable object older than the grace period, two weeks by default. One current detail: since Git 2.54, automatic maintenance uses the geometric strategy and does not run `git gc`, so unreachable objects can stay longer than those numbers suggest."

Commentary. The weak answer reverses cause and effect: a name stopped pointing, and deletion comes later, if at all. The adequate answer leaves out the reflog as a starting point, which is the whole recovery window. The strong answer gives the chain (ref, reflog entry, grace period, collection) and one version-sensitive fact stated with its version. Do not add numbers you are unsure of; "weeks, by default" is a better answer than a wrong figure. Reference: Chapter 3, section 3.8; Chapter 13, sections 13.2 and 13.4; Chapter 26, section 26.3.

4.7 What happens during a three-way merge?

Weak. "Git compares the two branches and combines their changes. If the same line was changed in both, you get a conflict."

Adequate. "Git finds the common ancestor and compares each branch with it. Changes made on only one side are taken automatically; changes to the same lines on both sides conflict. The result is a merge commit with two parents."

Strong. "When both branches have commits the other lacks, Git merges three snapshots: the tree `HEAD` points to, called ours; the tree of the commit I named, theirs; and the tree of their merge base. It does not replay commits. For each path it compares three blob IDs. A path changed on one side only is taken from that side without reading the content. An identical change on both sides is taken once. Only a path that differs in all three goes to a line-level merge, where hunks that do not touch are both applied. If everything resolves, Git writes one commit with two parents and moves the current branch. If a region was changed differently on both sides, it stops: `MERGE_HEAD` is set, the index holds stages 1,

2 and 3 for the path, and the file gets conflict markers. `git ls-files -u` shows the stages. The merged tree is a snapshot that neither author wrote or tested, so CI has to run on the merge result."

Commentary. The weak answer has no third input, and without the merge base Git cannot tell a line that one side added from a line the other side deleted. The adequate answer is the standard one. The strong answer adds three things an incident needs: the merge reads snapshots and not commits, most paths never have their content read, and the conflict state lives in the index as stages. It ends on the consequence for testing. Reference: Chapter 8, sections 8.4 and 8.8.

4.8 Why can Git merge two files incorrectly from a human perspective?

Weak. "Sometimes Git's merge algorithm makes mistakes in complicated cases, so you have to check the result."

Adequate. "Git merges text line by line. If two changes do not overlap, it combines them even when they are logically incompatible, for example when one branch renames a function and the other adds a call to the old name. That is why you need CI."

Strong. "Because a merge is a computation on text, not on meaning. Git compares three snapshots, and a clean merge certifies exactly one property: no two changed regions overlapped. It says nothing about whether the result builds or passes its tests. In the textbook's example two green branches changed different files: one renamed a function, the other added a caller of the old name. By the three-way rule each path was changed on one side only and was taken as it is. No file-level merge ran, so there was nothing for Git to compare, and the merge commit was red. The dependency lives in the language's import system, not in the text of any one file. Git is correct by its rules. The control is on another layer: run CI on the merge result, require the branch to be up to date with `main`, or use a merge queue. And when I audit a hand-resolved merge afterwards, `git show --remerge-diff` shows what the resolver changed."

Commentary. "Mistakes" marks the weak answer: none of the documented failures needs a bug, and an answer that blames luck offers no control. The adequate answer is right. The strong answer says what a clean merge proves (one property), shows that in the worst case no content was even read, and puts the fix on the correct layer, which is GitHub and CI, not Git. Reference: Chapter 8, sections 8.15 and 8.16; Chapter 18, section 18.8.

4.9 Why does `--force-with-lease` exist, and how can it still fail?

Weak. "It is the safe version of force push. It checks that nobody else has pushed, so you should always use it."

Adequate. "A plain force push overwrites whatever is on the server. `--force-with-lease` only overwrites if the remote branch is where you expect it to be, so you do not destroy a teammate's commits. It can be fooled if you fetched in the meantime."

Strong. "It exists because `git push --force` checks nothing: the server's ref is set to my commit, and every commit there that is not in my history is dropped, including commits I never fetched. Rewriting my own published branch after a rebase is still legitimate, and it can never be a fast-forward. `--force-with-lease` is a compare-and-swap for that case: update the branch only if it still has the value I last read, and that value is my remote-tracking ref. The comparison is made by my client, not by the server. That is its weakness. Any fetch, including one that a tool runs in the background, moves the remote-tracking ref, so the lease passes although I never looked at the new commits. The defences are `--force-if-includes`, which also requires that I have integrated the remote tip, or an explicit expected value: `--force-with-lease=<branch>:<commit>`. And a branch that must never be rewritten needs a server rule, because the lease is a precaution of the pusher."

Commentary. The weak answer attributes the check to the server and concludes "always safe"; the textbook's root-cause box shows the lease overwriting a teammate's commit after a fetch. The adequate answer knows the failure and not its mechanism. The strong answer says where the comparison runs and against what, which makes both the failure and the two remedies follow without memorisation. Reference: Chapter 12, section 12.8.

4.10 Revert or reset: what is the difference, and which do you use on a shared branch?

Weak. "Reset deletes commits and revert undoes them. Revert is the safe one."

Adequate. "Reset moves the branch back, so the commits are no longer on it; that rewrites history. Revert creates a new commit that undoes an earlier one. On a shared branch you revert, because others already have the commits."

Strong. "`git reset` moves the current branch ref to another commit, and its mode decides whether the index and the working tree are rewritten. The commits after the target leave the branch. `git revert` adds one commit whose change is the inverse of the named commit; the branch only moves forward. So reset is for commits that exist nowhere else, and revert is the correction for shared history: everyone's next pull is a fast-forward, and nobody has to repair a clone. I decide with evidence, not memory: after a fetch, `git branch -r --contains <commit>` tells me whether a remote-tracking branch has it. Two edges matter. A revert is a three-way merge against the current files, so it can conflict. And reverting a merge removes the content and leaves the merge in the graph, so merging the repaired branch later brings only its new commits unless I revert the revert first. I write that procedure into the message of the revert commit."

Commentary. "Deletes" is wrong, since a reset deletes no object, and "safe" is not a mechanism. The adequate answer is the standard one. The strong answer adds the test that decides between the two, and the reverted-merge trap, which is the follow-up a CTO asks next because it has cost teams a release. Reference: Chapter 11, sections 11.4, 11.8 and 11.9.

4.11 Why does cherry-pick create a new commit?

Weak. "Because it copies the commit onto your branch, and the copy gets a new hash."

Adequate. "A commit's ID depends on its parent. The cherry-picked commit has a different parent, so it is a new commit with the same changes."

Strong. "Because a commit object is never modified, and its ID is a function of its content. A cherry-pick does not move or reuse the picked commit. It recomputes the change that commit made, as a three-way merge whose base is the picked commit's parent, and records the result on my branch. That result has a different tree, which is my branch's files plus the change; a different parent, the tip I picked onto; and a different committer line. Only the author, the author date and the message are carried over. Different content gives a different ID. Git stores no link from the copy to the original, so `git branch --contains` with the original's ID does not list my branch, and that is why `-x` exists: it writes the original ID into the message. To find out whether a fix was ported I compare changes, with `git cherry` or `git range-diff`, and I remember that a pick which applies cleanly can still break the build."

Commentary. The weak answer restates the observation. The adequate answer names the parent and not the tree, and misses that the operation is a merge, which is why a cherry-pick can conflict. The strong answer draws the consequence a release manager cares about: ancestry tests cannot see a backport. Reference: Chapter 10, sections 10.1, 10.3 and 10.9.

4.12 Why can a pull request show unexpected commits?

Weak. "Probably someone pushed to the branch, or GitHub has not refreshed. Rebasing usually fixes it."

Adequate. "The pull request shows every commit on the head branch that is not on the base. If the branch was created from the wrong place or contains a merge from another branch, those commits appear too."

Strong. "Because the page is a computation, not a record of intent. A pull request stores two names, a base and a head, and GitHub lists the commits reachable from the head and not from the base: `git log base..head`. Unexpected commits mean that the head reaches commits the base does not, or that the base is not where the work started. The cause I check first is a head branch reused after a squash merge. The squash put the content on `main` in a single-parent commit and recorded no ancestry, so the merge base never moved and the old commits are listed again. Others are a wrong base, another branch merged into the head, and a rewritten base. I reproduce it locally after a fetch with `git log --oneline origin/main..HEAD` and `git merge-base`, and for a reused branch I transplant the new commits with `git rebase --onto`. Prevention: delete the head branch after merging, and read the commit list before opening the pull request."

Commentary. The weak answer has two guesses and a command, in that order. The adequate answer knows the range. The strong answer separates the layers (GitHub displays, Git's reachability decides), names the most frequent cause with its mechanism, and reproduces the page locally, which is what turns a platform mystery into a Git question. Reference: Chapter 17, sections 17.3 and 17.12; Chapter 30, section 30.10.

4.13 What is the difference between `..` and `...`?

Weak. "Two dots is a range from one commit to another, and three dots is a bigger range that includes both sides."

Adequate. "In `git log`, `A..B` shows commits on B that are not on A, and `A...B` shows commits on either that are not on both. In `git diff`, three dots compares against the common ancestor."

Strong. "The dots mean different things in `git log` and in `git diff`. In `git log` they select commits by reachability: `A..B` is the commits reachable from B and not from A; `A...B` is the symmetric difference, the commits reachable from either tip and not from both. In `git diff` nothing is a range, because a diff compares exactly two snapshots. `git diff A..B` is the same as `git diff A B`: the trees of the two tips. `git diff A...B` compares the merge base of A and B with B. So the meanings cross: the diff that matches the log range `A..B` is the three-dot diff. The practical trap is `git diff main feature`, which shows what `main` gained after the fork as deletions that the feature never made. For review I use three dots, and in scripts the explicit `git diff --merge-base`."

Commentary. The weak answer treats history as a line with intervals; a range is a set defined by reachability. The adequate answer is correct and does not say that two-dot `diff` is not a range at all, which is the case that produces a misleading review. The strong answer states the crossing and the trap. Reference: Chapter 14A, sections 14A.2 and 14A.8.

4.14 How do you recover a deleted branch?

Weak. "Use `git reflog`, find the commit and check it out."

Adequate. "The commits still exist after the branch is deleted. Find the tip's ID in `git reflog` and recreate the branch with `git branch <name> <id>`."

Strong. "A branch is a ref, so recovery means finding the commit ID of its tip and recreating the ref; the commits never stopped existing. I look in this order. The terminal scrollback, because `git branch -D` prints the ID it deleted. Then the HEAD reflog, not the branch's own, because the reflog of a branch is deleted with the branch: the entry before I last switched away from it is the last commit HEAD had there. Then `git fsck --no-reflogs`, which lists the dangling tips that no ref reaches. I recreate it with `git branch <name> <id>`, verify with `git log --oneline main..<name>`, and push it if it matters. The limits: the recreated branch starts a new reflog, and a teammate's branch that I only fetched and then pruned was never in my HEAD reflog, so only `git fsck` can find it, and only until a collection removes the objects. On GitHub, a closed pull request offers to restore its deleted head branch."

Commentary. The weak answer would work on a good day and leaves you on a detached HEAD. The adequate answer says "the relog" without saying which, and the branch's own log is gone. The strong answer gives an ordered search, a verification, and the case where the relog cannot help. Reference: Chapter 13, sections 13.4 and 13.8; Chapter 29, section 29.8.

4.15 How do you investigate a leaked secret?

Weak. "Remove the file, rewrite the history with a filter tool and force-push so that the secret is gone."

Adequate. "Rotate the secret first, because it has to be considered compromised. Then find where it is in history, remove it, and add secret scanning so that it does not happen again."

Strong. "The order matters more than the tools. First contain: revoke or rotate the credential at its issuer. Until then it works for whoever copied it, and revocation is the only step that reaches clones, forks and caches. Second assess, in writing: what the secret can reach; the first commit that contains it, from `git log --all -S`; which refs contain it, from `git branch -a --contains` and `git tag --contains`; who could read it, including forks and CI logs; and whether it was used, which only the issuer's logs can say. Third eradicate: remove it from current code, and rewrite history only if the data stays harmful after rotation. Deleting the file changes the next snapshot and no earlier one. A rewrite on GitHub leaves unreachable commits, pull request refs, forks and clones, and a stale clone that merges and pushes brings the secret back. Then tell collaborators exactly what to do with their clones, and prevent recurrence with push protection, scanning in CI and short-lived credentials."

Commentary. The weak answer starts with the repository, where the damage does not happen. The adequate answer has the right first step and then goes vague. The strong answer orders the work by what reduces risk, lists the facts the assessment must produce, and states the limits of a rewrite on the platform, which is where "we cleaned it" turns out to be untrue. Reference: Chapter 21B, sections 21B.10, 21B.11, 21B.14 and 21B.18.

4.16 How do you secure GitHub Actions?

Weak. "Store credentials in secrets, pin your actions, keep dependencies updated and turn on Dependabot."

Adequate. "Set the token permissions to read-only, pin third-party actions to commit IDs, do not use `pull_request_target` with untrusted code, and avoid putting user input directly into scripts."

Strong. "I ask one question of every job: whose code runs in it, whose text does it process, and what can it reach. A job is a program started by GitHub with a credential for the repository. Then five controls, one for each part of that model. The token: a read-only `permissions` block at the top of every workflow, single write scopes on the one job that needs them. Triggers: build fork code under `pull_request`, which gets a read-only token and no secrets, and never execute pull request code under `pull_request_target`, `workflow_run` or `issue_comment`. Text: no `${{ }}` expression inside `run:`, because it is substituted into the script before the shell starts; pass the value through `env:`. Third-party code: pin each action to a full commit ID, because a tag can be moved. Credentials: each secret in one step behind an environment, and OIDC with an exact subject where possible. Masking is not a boundary. Then make it hold: organization policies, code-owner review of `.github/workflows/`, and a workflow scanner as a required check."

Commentary. The weak answer is a list of settings with no model, so it cannot say what each one stops. The adequate answer has four correct controls. The strong answer derives the controls from one question, explains the injection mechanism in a clause, and distinguishes what the platform can enforce from what a reviewer must read. Label the layer: none of this is Git. Reference: Chapter 21A, sections 21A.2 to 21A.9, 21A.16 and 21A.19.

4.17 The tests pass on your laptop and fail in CI "on the same commit". Why?

Weak. "CI has a different environment: other versions, other operating system. It works on my machine."

Adequate. "On a pull request, CI does not test your branch alone; it tests your branch merged with the base. If `main` changed something your code depends on, the merge result can fail while your branch passes."

Strong. "First I check whether it is the same commit. For a `pull_request` event, GitHub Actions sets the ref to the pull request's merge ref, and the checkout step uses it. So the runner's HEAD is a test merge of my head into the current base: a commit that does not exist in my clone. If the base gained a commit that changes behavior my code relies on, that merge is red while my branch is green. That is an Actions default, not a Git fault, and it asks the right question: is the result of merging safe. The evidence is the commit the job checked out, compared with `git rev-parse HEAD` in my clone: two different IDs. I reproduce it with `git fetch origin` and a local merge of `origin/main`. The fix is to update the branch and repair the code. If the commit is the same, the next hypotheses are the checkout defaults, one commit and no tags, and then the environment."

Commentary. The weak answer names the last hypothesis first and offers no test. The adequate answer has the mechanism. The strong answer starts by questioning the premise in the report ("the same commit"), names the layer, and orders the remaining hypotheses by how cheaply each is tested. Reference: Chapter 20A, section 20A.8; Chapter 30, section 30.11.

4.18 `.env` is listed in `.gitignore`, yet it keeps appearing in commits. Why?

Weak. "The `.gitignore` pattern must be wrong, or the Git cache needs to be cleared."

Adequate. ".gitignore only applies to untracked files. The file was committed before the rule was added, so Git keeps tracking it. You need `git rm --cached .env` and a commit."

Strong. "Because the file is tracked, and tracking wins. Ignore patterns are consulted only for paths that have no index entry: when `git status` lists untracked files and when `git add` walks a directory. A tracked path is compared with its index entry and no pattern is checked. The file was added before the rule existed. I confirm it: `git ls-files .env` prints the path, and `git check-ignore -v .env` prints nothing for a tracked file. The fix is `git rm --cached .env`, a commit, and the pattern stays. Three consequences belong in the same answer. The commit records a deletion, so teammates lose their copy of the file at the next pull. Every earlier commit still contains the file, so any credential in it is leaked and must be rotated. And `.gitignore` is not a security control. For prevention, write the ignore file before the first `git add .`, and let CI fail when `git ls-files -ci --exclude-standard` prints anything."

Commentary. "Clear the cache" is a ritual without a mechanism. The adequate answer is correct and stops at the fix. The strong answer explains when patterns are consulted, proves the diagnosis with two commands, and then states what the fix does to other people and to history, which is the part that becomes a security incident. Reference: Chapter 4, sections 4.5 and 4.6.

4.19 After a squash merge, is your branch merged?

Weak. "Yes, the pull request says merged. If `git branch -d` complains, use `-D`."

Adequate. "The content is on `main`, but Git does not see the branch as merged, because a squash creates a new commit instead of linking to yours. So `-d` refuses and you have to force the deletion."

Strong. "It depends on which meaning of 'merged' is asked for, and I say so. In Git, merged means reachable: the branch tip is an ancestor of `main`. A squash merge creates one commit with a single parent that carries the content and records no link to my commits, so the tip is not an ancestor. `git branch --merged` and `git merge-base --is-ancestor` answer no, and so does `git branch -d` once the remote-tracking branch is pruned (before that it compares with the upstream and deletes with a warning); they are right. On GitHub the pull request is merged, which is a platform record. The question I care about is whether anything would be lost, and that is a test of content, not ancestry: an empty `git diff main <branch>` right after the merge, or a merge-tree comparison once `main` has moved on. Only then `git branch -D`. The same mechanism explains why a reused branch lists its old commits in the next pull request. So the rule is one squash per branch: delete it, on the server and locally, and start again from `main`."

Commentary. The weak answer forces a deletion on the word of a web page. The adequate answer has the mechanism and still deletes without verifying. The strong answer separates three meanings of one word, picks the one that protects work, and verifies before a destructive command. Reference: Chapter 17, section 17.9; Chapter 27, section 27.3.

4.20 An evaluation run recorded its commit ID. You check out that commit and get a different metric. Why?

Weak. "Machine learning is not deterministic; it is probably the random seed or the GPU."

Adequate. "The commit only identifies the code. The data, the model weights, the dependencies and the configuration also have to be the same, so those need to be versioned as well."

Strong. "Before I suspect randomness I ask what the commit ID proves. `git rev-parse HEAD` names the last commit; it says nothing about the index or the working tree. If the run started from a tree with a modified tracked file, Python imported the file on disk, and the tracker recorded an ID for code that did not produce the number. A commit is a snapshot that was taken, and uncommitted work is in no snapshot. So the first evidence is whether the tracker stored `git status` and a diff with the run. If the patch was saved, I apply it to the commit and rerun. If it was not, the result cannot be reproduced, and I say so. Beyond the code, the commit does not pin data, weights, dependency versions or the container, which is why lock files, image digests and data versions are recorded with it. The control is in the launcher: refuse a tracked run from a dirty tree, or record status and diff, and treat untracked files the same way."

Commentary. The weak answer may be true and is untestable as stated. The adequate answer lists what else must be versioned and assumes that the code, at least, is pinned by the ID. The strong answer questions that assumption first, because it is the one a Git interview is probing, and it ends with a control that a launcher can enforce. Reference: Chapter 28, sections 28.7 and 28.8.

5. Scenario and system-design prompts

These prompts have no single right answer. They are graded on whether you state the context you are assuming, what each choice costs, which layer enforces it, and when you would choose otherwise. Take ten minutes for each, aloud, with paper. Then compare with "what a strong response covers". A response that lists controls without their costs is an adequate one at best.

5.1 Branching and governance for a described company

Prompt. "We are forty engineers. One hosted product deploys from `main` several times a day. We also ship an on-premises edition with two supported versions. An ML team trains models from the same repository. Design our branching model and the rules around it."

What a strong response covers.

- **The two questions first:** how many versions are live at once, and how often is each released. Here the answer is three rows of the context table at once: a continuously deployed service, several supported versions, and model releases that must be reproducible (Chapter 27, section 27.14).
- **The model:** one long-lived `main` with short-lived branches for the service; a `release/x.y` branch per supported on-premises version, because each supported version needs a line that can receive fixes. Add a branch only when you can name the version or the audit requirement that needs it.
- **The direction of fixes,** decided once and tested: either fixes land on the release branch and are merged upward, or they land on `main` and are picked down. Whichever it is, the check is a release gate, not a habit, because nothing in Git propagates a fix between branches (section 27.10).
- **Rules on the server:** a ruleset on `main` and on the release branches that blocks force pushes and deletions, requires a pull request with review, and requires status checks; a tag ruleset so that release tags cannot be moved or deleted (Chapter 18, sections 18.11, 18.12 and 18.18).
- **The merge method and its cost.** Squash gives one commit per pull request and discards ancestry, so head branches are deleted after merging and never reused; merge commits keep the reviewed IDs. Say which you choose for `main` and why the release branches may differ.
- **Ownership:** CODEOWNERS for the deployment workflow and the release configuration, with the limit that the file requests reviews and enforces nothing until a rule requires code-owner review (Chapter 19).
- **Reproducible model releases:** an annotated tag on the exact commit, with data and model versions recorded beside it, and deployment by commit ID or image digest, not by a name (Chapter 28, section 28.7).
- **What you leave out, and why:** for example no signed-commit rule yet, with the reason. And how you would roll the rules out: create them disabled or in evaluate mode where the plan allows, test with one pull request, then activate.

Follow-ups to expect. Who can bypass, and how is a bypass recorded? What happens to an emergency fix at night? Which of these rules depend on the GitHub plan?

5.2 A CI security review

Prompt. "Here is a workflow that labels and tests pull requests from forks. It triggers on `pull_request_target`, checks out the pull request's head, installs dependencies, runs the tests, and comments the result. It uses three third-party actions by version tag. Review it."

What a strong response covers.

- **The model before the findings:** a run is a program with a credential, and its safety depends on who controls its code and its inputs (Chapter 21A, section 21A.2).
- **The primary finding:** `pull_request_target` runs with the base repository's token and secrets. Checking out the fork's head does not by itself execute anything; the step that installs dependencies or runs the tests executes the outsider's code inside that privileged job. That step completes the vulnerability (section 21A.5).
- **The repair by separation:** build and test under `pull_request`, which gives a fork a read-only token and no secrets; do the privileged part (the comment, the label) in a job that never runs the fork's code.
- **Token scope:** a top-level read-only `permissions` block, and the single write scope on the one job that comments (section 21A.3).
- **Expressions:** any `${{ }}` inside `run:` that carries a title, a branch name or a body is script injection; pass it through `env:` (section 21A.6).
- **Third-party actions:** a tag can be moved; pin to a full commit ID with a version comment, and know what pinning does not cover (section 21A.7).
- **What is not a boundary:** log masking, and the approval gate for first-time contributors.
- **Process:** code-owner review for `.github/workflows/`, a workflow scanner as a required check, organization policies for what they can enforce, and the review checklist of section 21A.19.
- **The date:** the textbook records a platform change to this trigger's default rule that is enforced from 2 November 2026 for public repositories. State version-sensitive facts with their date and say you would re-verify them.

Follow-ups to expect. Which of your findings can an organization policy enforce, and which need a human reading the file? What changes if the runner is self-hosted?

5.3 Incident walk-through: the production branch moved backwards

Prompt. "It is Friday evening. The deploy job refuses to run. Someone says `production` 'looks different'. You have the room. Go." The interviewer answers only with what your chosen command would print.

What a strong response covers.

- **Stabilise first, with a sentence and not a command:** nobody pushes to or resets `production` until told otherwise. The first message goes out before the cause is known (Chapter 30, section 30.2).
- **Preserve:** fetch, and give the current and the previous state names with backup refs. Ask colleagues not to prune, collect garbage or clone again, because a server-side branch has no reflog of its own on plain Git and its history is the sum of what the clones remember.
- **Evidence, read-only:** `git reflog show origin/production` for the two values at fetch time; `git ls-remote` for what the server holds now; the deployment tag for what was last shipped; a tree diff between the old and the new tip. On GitHub, the Activity view and Rule

Insights say which account moved the ref and whether a rule evaluated it (Chapter 29, section 29.8).

- **At least three hypotheses** before a conclusion: a forced push after a rebase, a branch deleted and recreated, a reset pushed with force. Name the command that separates them.
- **The lowest-risk recovery**: restore the recorded history, carry over any legitimate newer commit, and if a forced push is the repair, state the expected old value explicitly with `--force-with-lease=<ref>:<expected>`.
- **Verification in every copy**: the deployed commit is again an ancestor of the branch; the release job's own check passes; IDs compared on the server and in the known clones. Say what is not yet verified.
- **Severity and communication**: assign the level on what could have happened in the window; give the four-part summary (section 30.15).
- **Prevention as a control**: a ruleset on the production branch, with an owner and a date, and a way to know it works.

5.4 Incident walk-through: a pull request suddenly shows five hundred changed files

Prompt. "A two-line change. The pull request shows five hundred files. The author says they changed nothing else." A strong response does not touch the branch until it has listed mechanisms: another branch merged into the head, a wrong base, a head reused after a squash merge, a two-dot comparison read as a review diff. It tests each locally (`git log --merges main..head`, `git merge-base`, `git log --oneline origin/main..HEAD`), names the layer (Git decides reachability; GitHub displays it), rebuilds the head without the foreign merge, and says what the forced push to the head branch will do to the existing review. Reference: Chapter 30, section 30.10; Chapter 17, section 17.12.

5.5 Explaining to an executive

Prompt. "You have ninety seconds with the chief executive. Explain why the leaked key is still a problem after the engineer deleted the file and pushed." A strong response has no Git vocabulary in its first two sentences and no inaccuracy anywhere: every saved version of the project is kept permanently, so removing the file from today's version leaves it in yesterday's; and every copy of the repository that was downloaded has it. Then the action in order: the key was revoked at its issuer at a stated time; here is what it could reach; here is whether the issuer's logs show use; here is the control that would have blocked the push. It ends with what is not yet known and when the next update comes. Reference: Chapter 21B, sections 21B.10 and 21B.14; Chapter 30, section 30.15.

6. Self-assessment rubric

Grade a recorded answer, not a remembered one. Score each dimension 0 to 2, as in the protocol, section 5; the descriptions below add the signs to listen for. Twelve points per answer.

Dimension	Signs of 0	Signs of 1	Signs of 2
Correctness	A wrong mechanism ("reset deletes commits"); the wrong layer; a number you made up	Right in outline, with one claim you could not defend if asked "how do you know"	Every claim agrees with the textbook; uncertain facts are marked as uncertain; the layer is named
Depth	You described what you type	You explained what Git does	You explained it in terms of objects, refs, HEAD and the index, and gave the design reason
Terminology	"The code", "the history", "SHA", "the remote branch" for three different things	Mostly exact; a loose word or two	Exact throughout: commit ID, the index, working tree, remote-tracking branch, upstream, merge base, reachable
Reasoning	A conclusion without a chain	State, then mechanism, then symptom	The chain, plus what you would observe if you were wrong, or which other hypotheses you ruled out
Practical understanding	No command, or a command that changes state as the first step	The right commands	The right commands in a safe order: read, preserve, preview, change, verify
Production awareness	The answer ends when your own repository is repaired	You mention teammates, the server or CI	You state what the fix can destroy, who must act afterwards, and the control that prevents recurrence, on its layer

The correctness gate. If correctness is 0, the answer is 0. Fluent vocabulary around a wrong mechanism is the most dangerous kind of answer, because it is believed.

Reading your totals.

Average over a session	What it means	What to do
10 to 12	Senior standard for these areas	Move to principal-level questions and to the prompts of section 5
7 to 9	Correct and thin	Find the dimension that is 1 in most answers. It is usually reasoning (no evidence) or production awareness (no control)
4 to 6	The model has gaps	Go back to the chapter's root-cause boxes and its "What can go wrong" table, then redo the labs for that area
Below 4, or any 0 on correctness for a destructive operation	Not yet safe to lead an incident in this area	Restudy the chapter from the start. Do not practise interview answers on a wrong model

Four checks that take one minute each. Did the first sentence answer the question? Did you name the layer where two were possible? Did you name one command that would show you were right? Did you end on a limit or a control? An answer with four "yes" is rarely below 9.

7. A four-week preparation plan

The plan assumes the textbook and labs are done and about seven hours a week. It uses the bank by area, the protocol for every session, and the weak-area tracker of the roadmap, section 12. Sessions are oral and recorded. Do not read model answers ahead of a session.

Week	Areas of the bank	Daily work (about an hour)	End-of-week session
1. The model	1 Fundamentals, 2 Git internals, 3 Branching, 8 Remote workflows	Ten questions a day from one area, foundational and working-engineer levels first. For every answer below 9 of 15, reread the cited section and its transcript, then draw the state on paper	Area interview on areas 1 to 3 and 8. Worked answers 4.1 to 4.3 and 4.9 from memory
2. Changing history, and getting it back	4 Merge, 5 Rebase, 6 Undo, 7 Recovery, 16 Debugging	Ten questions a day at working-engineer and senior levels. Two of the generated incidents under <i>incidents/</i> as debugging rounds, attempted before reading their sections	Area interview on areas 4 to 7, at the 90% threshold. Worked answers 4.4 to 4.8, 4.10, 4.11, 4.13 and 4.14
3. The platform	9 GitHub, 10 Pull requests, 11 GitHub Actions, 12 Security, 13 Open source, 14 AI/ML workflows	Ten questions a day. For each GitHub fact with a date or a plan gate, note the date; re-verify the ones you would quote in an interview against the documentation the textbook links	Area interview on areas 9 to 12, at the 90% threshold for Security. Prompts 5.1 and 5.2. Worked answers 4.12, 4.15 to 4.17, 4.19, 4.20
4. Judgment	15 Production incidents, 17 Architecture, 18 CTO interview, and every tracked question	Senior and principal levels only. One debugging round a day. One prompt from section 5 a day, ten minutes aloud	Two full CTO interviews of twenty questions on different days, each drawing on at least ten areas. Prompts 5.3 to 5.5

Rules for the four weeks.

- Tracked questions come first.** Every session opens with the questions in the tracker that are due. A question leaves the tracker after two consecutive scores of 12 or more out of 15.
- One wrong mechanism stops the week.** If you score 0 on correctness for the same concept twice, stop interview practice for that area and return to the chapter and its lab. Practising delivery on a wrong model makes it more convincing and no more correct.
- Draw before you speak.** For any question about history, draw the commit graph with the refs and HEAD before answering. In a real interview, ask for the whiteboard.
- Say the layer.** Make "that is Git", "that is GitHub", "that is GitHub Actions" a habit in the first sentence until it is automatic.
- Version-sensitive facts carry their date.** Say "as of October 2026" for platform facts, and "since Git 2.54" where the textbook gives a version. If you are not sure of a number, say what it depends on instead.
- The last two days are for the twenty.** Give each worked answer of section 4 once more, in two minutes, and grade it with the four checks of section 6.

When you are ready. Two full interviews in week 4 at or above the threshold, no tracked question older than a week, and every prompt of section 5 answered once with its costs. The capstone simulation of Module 41 comes after that.

8. Where each part of this guide comes from

Part	Source in the course
The five-part answer	Chapter 1, section 1.10 (the root-cause framework and its seven-line box)
Read-only diagnosis, the risk ladder, verification	Chapter 29, sections 29.2, 29.7, 29.9 and 29.10
The incident loop, severity, the summary for a CTO, controls	Chapter 30, sections 30.2, 30.3, 30.15 and 30.20
The six grading dimensions and the conduct of a session	Roadmap, section 3.4; interview-mode protocol
The worked answers	The sections named in each commentary; the same questions appear in the bank, listed under "The questions from your brief"
The field versions of the method	Troubleshooting playbook, disaster-recovery playbook

Chapter 32: Capstone

Source file: `capstone/README.md`

Capstone: eight incidents at Tessaly

Baseline. Git 2.55.0 on macOS; GitHub facts as of 1 October 2026. Everything runs locally in the lab sandbox. The company, the people, the product and every alert are fictional and were constructed for this course.

This is the last piece of work in the course. You join a small team for eight working days. On each day something goes wrong in or around the team's repository, and you are the person who is asked to sort it out. Nobody tells you the cause. What you are told is what a teammate, a manager or an alert would tell you: incomplete, sometimes wrong, never the mechanism.

The capstone applies the method of Chapter 29: Production Troubleshooting and the incident loop of Chapter 30: Incident Response. It adds nothing new to learn. It tests whether you can use what you learned when the situation does not announce which chapter it belongs to, when the states of earlier days are still in the repository, and when you have to write down what you did for people who were not there.

1. The company and the product

Tessaly sells a customer-support platform. One of its backend services is `intent-router`: it receives a customer message, decides which `intent` the message has (`billing_refund`, `order_status`, `account_access`), and routes it to the queue of the desk that handles that intent. A message that fits no intent well enough goes to a human triage agent.

The decision uses two signals per candidate intent, both between 0 and 1:

- a **keyword score**: how many of the intent's keywords occur in the message (`router/keywords.py`);
- an **embedding score**: the similarity between the message and the intent's examples, computed by a separate embedding service.

`router/scoring.py` blends the two, and `router/classify.py` picks the best intent above a threshold. The code is small on purpose. You will read all of it within minutes, and no stage requires more Python than a three-line change.

2. The team

Person	Role	Clone	What to expect
You	Backend engineer, three months in the team	<code>you/</code>	Identity <code>Lab User <you@example.com></code>
Nandini Iyer	Tech lead; reviews and merges most pull requests; owns releases	<code>nandini/</code>	Precise, asks for evidence, will not bypass a rule on a guess
Kabir Sethi	ML engineer; owns scoring and the embedding client	<code>kabir/</code>	Fast, helpful, sometimes helps before asking
Tanvi Desai	Platform engineer; owns CI and deployment; on call this fortnight	<code>tanvi/</code>	Under time pressure; proposes the quickest route
Leela Varma	CTO	<code>none</code>	Reads your summaries; asks about customers, risk and recurrence

The people are written to be reasonable. Each action that causes an incident looked right to the person who took it. Your postmortems are expected to show why (Chapter 30, section 30.19).

3. The sandbox

`capstone/setup.sh` builds everything under `$GIT_MASTERY_LABS/capstone/intent-router` (`~/git-mastery-labs/capstone/intent-router` by default):

<code>server.git</code>	a bare repository: the server, the part GitHub plays in real life
<code>pr</code>	a script that plays the pull-request side of GitHub (section 6)
<code>you/</code>	your clone
<code>nandini/</code> <code>kabir/</code> <code>tanvi/</code>	your teammates' clones, each with its own <code>user.name</code> and <code>user.email</code>
<code>evidence/</code>	files that a stage writes for you: an alert, a run report, a pasted script
<code>home/</code>	the isolated Git configuration of the sandbox

You are allowed to look into and work in every clone, the way you would sit down at a colleague's machine during an incident, unless a briefing says that a person is away. `git -C ../tanvi <command>` runs a command in Tanvi's clone without leaving yours. You are also the administrator of `server.git`; where you use that power, your write-up must say what the equivalent step on GitHub is and who could perform it.

Nothing touches a network, your real Git configuration, or anything outside the lab root.

4. The repository

```
README.md
.gitignore
.github/CODEOWNERS          who must review which path
.github/workflows/ci.yml     tests on every pull request and on pushes to main and release/**
.github/workflows/release.yml builds the archive when a tag v* is pushed
router/__init__.py
router/keywords.py          keyword hits and the keyword score
router/scoring.py           the blend of keyword score and embedding score
router/classify.py          picks the intent
router/routes.yml           intent -> queue and priority
scripts/test.sh             runs the unit tests; prints OK or the failing tests
scripts/evaluate.py         offline evaluation against the evaluation set
scripts/version.sh          "git describe" for the release archive
data/eval-messages.jsonl.dvc a pointer file: hash, size and name of the evaluation set
data/.gitignore             keeps the evaluation set itself out of Git
tests/test_scoring.py tests/test_classify.py
```

Three things in this layout deserve a sentence each.

- **The data pointer.** The evaluation set (48 MB of labelled messages) is not in Git. `data/eval-messages.jsonl.dvc` is a small text file in the format DVC uses: the hash, the size and the path of the real file, which lives in object storage. Git versions the pointer; the data tool fetches the data. Chapter 28 explains why. No stage requires the data.
- **CODEOWNERS.** `/router/scoring.py` and `/router/keywords.py` belong to `@tessaly/ml`, `/.github/` and `/scripts/` to `@tessaly/platform`, everything else to `@tessaly/intent-router`. On GitHub a ruleset would require a review from the owners; in the sandbox the file is something you read to decide whom to tell.
- **The workflows.** Both reference actions by full commit SHA with the version as a comment, taken from `workflows/ACTION_PINS.md`; both set `permissions: contents: read`; the checkout steps set `persist-credentials: false`. They were written from documented syntax and parse as YAML. They were never executed on GitHub, and the sandbox cannot execute them. Stage 4 gives you a constructed description of a run instead.

`bash scripts/test.sh` needs `python3` and nothing else. It prints one line per failing test and a last line, `OK` or `FAILED`.

5. Branching and release conventions

The team works trunk-based (Chapter 27, section 27.8).

Convention	Rule
<code>main</code>	Always releasable in principle. Nobody pushes to it. Every change arrives through a pull request. Nobody rewrites it
Feature branches	<code>feature/<topic></code> , <code>fix/<topic></code> , <code>chore/<topic></code> , <code>docs/<topic></code> ; short-lived; pushed early
Merge method	Squash by default: one commit on <code>main</code> per pull request, titled <code><title> (#<number>)</code> . A merge commit (<code>Merge pull request #<number> from <branch></code>) when the individual commits should stay visible, for example a revert next to its test
After a merge	The server deletes the head branch
Releases	An annotated tag <code>vMAJOR.MINOR.PATCH</code> on the released commit, created by whoever ships it and pushed with <code>git push origin <tag></code> . The release workflow refuses a tag that is not annotated
Maintenance lines	<code>release/MAJOR.MINOR</code> , created at the release tag the first time a released version needs a fix that <code>main</code> cannot deliver
Direction of fixes	A fix lands on <code>main</code> first and is copied to the maintenance line with <code>git cherry-pick -x</code> (Chapter 10, section 10.9; Chapter 27, section 27.10)
Shared feature branches	Allowed. Whoever rewrites one announces it first
Staging deployments	The deploy script marks the deployed commit with a lightweight tag <code>staging/<date></code>

GitHub, not Git. "Nobody pushes to `main`" is a rule that only a platform can enforce (a ruleset, Chapter 18). A bare repository accepts any push from anyone who can write to it. In the sandbox the rule is a convention, and the check scripts verify that you kept it: every commit on the first-parent history of `main` must have been created by `./pr merge`.

6. Pull requests in the sandbox

GitHub is not Git, and a bare repository has no pull requests. The script `pr` in the sandbox root keeps the few facts the simulation needs, with plain Git: one small text file per pull request next to `server.git`, and the two refs that Chapter 17, section 17.2 describes,

`refs/pull/<number>/head` and, when head and base merge without conflict, `refs/pull/<number>/merge`. A post-receive hook in `server.git` runs `pr sync` after every push, so the refs follow the branches as they do on GitHub. `receive.hideRefs` keeps clients from pushing to `refs/pull/`.

Run it from any clone as `./pr`:

```
./pr list          # open pull requests; --all includes merged and closed ones
./pr view 12       # title, state, the commits (base..head), the changed files (base...head)
./pr open feature/x # after pushing the branch; --base <branch> and --title '<title>' are optional
./pr merge 12      # squash merge; --merge for a merge commit. Deletes the head branch on the server
./pr close 12      # close without merging
./pr reopen 12     # only when the head branch exists again
```

What it imitates and what it does not:

Behavior	In the sandbox	On GitHub
A pull request gets its number when it is opened	yes	yes
<code>refs/pull/N/head</code> follows the head branch; <code>refs/pull/N/merge</code> is a test merge of head into base	yes, rebuilt after every push	yes; the test merge is rebuilt on a push, when the merge base changes, or when it is older than 12 hours (Chapter 20A, section 20A.8)
A pull request with conflicts cannot be merged and has no merge ref	yes	yes
Deleting the head branch closes the pull request	yes, and <code>refs/pull/N/head</code> keeps the last head commit	the pull request is closed, and a closed pull request offers Restore branch (Chapter 30, section 30.12)
Who is recorded on a merge	the person who runs <code>./pr merge</code> is the committer; a squash commit has the first commit's author as its author	the committer of a web merge is GitHub; who becomes the author of a squash commit is not stated in the documentation (Chapter 17, section 17.8)
Reviews, approvals, required checks, rulesets, the Activity view, the audit log	not simulated	yes

These are plain Git commands chosen to produce the documented result. GitHub does not publish the commands it runs. Where a stage depends on something in the last row, the briefing gives you a constructed piece of evidence, and your write-up names the real instrument.

7. The history you inherit

`capstone/setup.sh --stage 0` builds the repository with no incident applied. Look at it once before you start; you will be expected to know your way around.

The first-parent history of `main`, from Monday 7 to Friday 11 September 2026: two direct commits by the tech lead from before the pull-request rule, then pull requests #1 to #11, nine of them squash merges and two merge commits, and four annotated tags, `v1.0.0` to `v1.3.0`. Two feature branches have open pull requests: #12 (yours) and #13 (Kabir's). The walkthrough prints this history as a real transcript.

The lab clock makes every run of `setup.sh` produce the same commit IDs. The incidents then happen on fixed days:

Stage	Day (2026)	Directory	What the briefing says
1	Mon 14 Sep	<code>stage-01-bug-in-production</code>	"The billing queue is flooded with junk since the release"
2	Tue 15 Sep	<code>stage-02-merge-conflict</code>	"I resolved your conflicts for you, press the button"
3	Wed 16 Sep	<code>stage-03-leaked-secret</code>	"I deleted the file, so the branch is clean now"
4	Thu 17 Sep	<code>stage-04-failed-ci</code>	"It passes on my machine; it must be the Python version"
5	Fri 18 Sep	<code>stage-05-lost-work</code>	"I fetched, and my three commits were gone"
6	Tue 22 Sep, morning	<code>stage-06-deleted-branch</code>	"The branch for Thursday's demo is not in the list"
7	Tue 22 Sep	<code>stage-07-broken-pull-request</code>	"The pull request shows every commit twice"
8	Wed 23 Sep	<code>stage-08-hotfix-and-backport</code>	"Production returns errors, and <code>main</code> is not signed off"

8. How to run the simulation

All commands are typed in the course root.

```
capstone/setup.sh          # builds the sandbox with stage 1 applied; prints its path
labs/shell "<the path it printed>/you" # a shell with the isolated lab configuration
```

Then, for each stage:

1. Read the stage's `BRIEFING.md`. Read nothing else in the stage directory.
2. Work in the sandbox. Keep your evidence log as you go (`DELIVERABLES.md`).
3. Run the stage's check from the course root: `capstone/stage-NN-<slug>/check.sh`. It prints one line per condition and ends with `PASS` (exit status 0) or `NOT YET` (exit status 1).
4. Write the stage's deliverables before you go on. The check sees Git state. It cannot see whether you understood what you did.
5. Apply the next stage: `capstone/stage-NN-<slug>/inject.sh`. It refuses to run until the previous check passes.

The stages are cumulative. Stage 5 happens in a clone whose reflogs you may have expired in stage 3. Stage 8 has to deal with everything that reached `main` in stages 2, 4 and 7. That is the point of doing them in order, in one sandbox.

File in a stage directory	What it is	When to read it
<code>BRIEFING.md</code>	What you are told and what you are asked for	First
<code>check.sh</code>	Read-only verification of the Git state the stage must reach	Run it; read it after the stage
<code>inject.sh</code>	Builds the incident on top of the current state. It is the answer to "what happened"	After the stage

To retake or skip a stage: `capstone/setup.sh --stage N` deletes the sandbox and builds the state at the start of stage N directly: the company repository, then stages 1 to N-1 solved with the model solutions, then the incident of stage N. The result is deterministic, and its commit IDs are the ones printed in `solutions/capstone-walkthrough.md`. Your own solutions will have other IDs for the commits you create, because `labs/shell` uses the real clock.

If a repair goes wrong: rebuild with `capstone/setup.sh --stage N` and start the stage again. A sandbox is the one place where a destructive mistake costs nothing. In the evaluation, a rebuild counts as what it would have been in production.

9. The rules of the simulation

1. **Evidence before action.** Until you can state the cause, use only commands that read state or add refs (Chapter 29, section 29.2). Write at least three hypotheses per stage, with the command that separates them.
2. **Preserve before you change.** Every state you may need gets a name first: `git branch rescue/<what> <commit ID>`.
3. **The lowest-risk fix.** Among the changes that repair the cause, choose the one that destroys least and is simplest to undo, and write down the options you rejected.
4. **`main` is never rewritten, and never pushed to.** Changes reach it through `./pr merge`.
5. **A forced push names what it expects.** `--force-with-lease=<branch>:<commit ID>`, or `--force-with-lease --force-if-includes` on a branch you examined a moment ago. A bare `--force` is a finding against you.
6. **Verify with the commands that showed the problem,** and run the project's tests on the commit that will be used.
7. **Say what you did not verify.** A local sandbox cannot show what a GitHub run, a provider console or a production system would show. Name those steps; do not claim them.
8. **No names in causes.** A cause is a mechanism on a layer (Git, GitHub, GitHub Actions, a team convention), not a person.
9. **The solutions stay closed** until your own deliverables for the stage are written: the walkthrough, the `inject.sh` scripts and the scripts under `labs/capstone/`.

10. What you hand in, and how it is judged

`DELIVERABLES.md` lists what you hand in for each stage (evidence log, root cause, recovery, verification, prevention, a message to the team, a message to the CTO) and the final postmortem. `EVALUATION.md` is the rubric: seven dimensions, a four-level scale with observable evidence for each level, and the rule by which the final judgment, "demonstrates senior-level mastery" or "not yet", is reached. The model solution is `solutions/capstone-walkthrough.md`. Read it after your own attempt, stage by stage.

Plan about ten to fourteen hours in total: roughly one hour per stage in the sandbox, half an hour per stage for the write-up, and two hours for the postmortem.

11. For maintainers

File	Purpose
<code>capstone/lib/capstone-lib.bash</code>	People, clock, clones, the stage list, <code>cap_build_to</code>
<code>capstone/lib/base.bash</code>	The company repository: file contents and the history up to <code>v1.3.0</code>
<code>capstone/lib/pr.sh</code>	The pull-request stand-in; copied to the sandbox as <code>pr</code>
<code>capstone/lib/check-lib.bash</code>	Helpers of the check scripts
<code>labs/capstone/model-NN-<slug>.bash</code>	The model solution of a stage as a replay body. <code>setup.sh --stage N</code> applies the earlier ones silently

File	Purpose
labs/capstone/stage-NN-<slug>.sh	Replay of one stage: builds its start state, asserts that the check fails, plays the model solution with transcript capture, asserts that the check passes
labs/capstone/end-to-end.sh	All eight stages in one sandbox, with the check before and after each

Every stage starts at a fixed time on the lab clock, so its commit IDs do not depend on how many commands the earlier stages used.

labs/verify-all.sh capstone re-runs all replays and compares the output with the transcripts printed in the walkthrough. It takes a few minutes, because each replay builds its start state from the beginning.

Chapter 33: Cheat Sheets

Source file: cheatsheets/git-cheat-sheet.md

Git cheat sheet

Baseline. Git 2.55.0. ● SAFE reads state or only adds objects · ● CAUTION moves refs or rewrites local history, recoverable through the reflog · ● DANGEROUS can destroy uncommitted work, remote history, or the safety net itself (Chapter 1, section 1.8). Every command is taught in the textbook; the number in brackets is the section. Full forms: Git command reference. The five answers for every ● and ● command: command safety. When something is already lost: emergency recovery.

Before anything else: look

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git status -sb</code>	Branch, upstream, ahead and behind, changed paths (4.4)	<code>git status -sb</code>	●	Reading "clean" as "equal to the commit": ignored files are not shown	<code>git status --ignored</code>
<code>git log --oneline --graph --decorate --all</code>	Every ref as a graph (14A.9)	<code>git log --oneline --graph --decorate --all -20</code>	●	Forgetting <code>--all</code> : the walk starts at HEAD only	Not needed
<code>git diff</code> , <code>git diff --cached</code>	Index against working tree; HEAD against index (5.4)	<code>git diff --cached</code>	●	Committing without reading the staged diff	Not needed
<code>git reflog</code>	Every commit HEAD was on, and the command that moved it (13.3)	<code>git reflog -10</code>	●	Expecting one in a fresh clone or on CI	The clone where the work was done
<code>git remote -v</code> , <code>git branch -vv</code>	Where pushes go; which upstream each branch has (12.2, 12.5)	<code>git branch -vv</code>	●	Trusting <code>origin/<branch></code> without fetching: it is a cache	<code>git fetch</code>

Daily use

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git init</code>	Creates <code>.git</code> (1.9)	<code>git init shop</code>	●	Running it in the wrong directory	Remove the stray <code>.git</code> if it holds nothing
<code>git clone</code>	New repository with <code>origin</code> , remote-tracking refs and one checked-out branch (12.3)	<code>git clone <url></code>	●	<code>--depth 1</code> where history is needed	<code>git fetch --unshallow</code>
<code>git add <path></code>	Stores content as a blob and points the index entry at it (5.3)	<code>git add src/router.py</code>	●	Editing after <code>git add</code> : the commit lacks the edit	<code>git add</code> again
<code>git add -p</code>	Stages hunk by hunk (5.5)	<code>git add -p</code>	●	Following it with <code>git commit -a</code>	<code>git reset HEAD~1</code> ● while private, stage again
<code>git commit -m</code>	New commit from the index; moves the current branch (6.3)	<code>git commit -m "Add rate limits"</code>	●	A missing file: it was never staged	A further commit
<code>git commit --amend</code>	Replaces the tip commit with a new one (6.7)	<code>git commit --amend --no-edit</code>	●	Amending a commit that is already pushed	<code>git reset --soft 'HEAD@{1}'</code>
<code>git restore --staged <path></code>	Unstages: index entry copied from HEAD (5.9)	<code>git restore --staged notes.txt</code>	●	Using <code>git rm --cached</code> , which stages a deletion	<code>git add <path></code>
<code>git restore <path></code>	Overwrites the file from the index (4.7)	<code>git restore -p src/router.py</code>	●	Not reading <code>git diff --<path></code> first	None for content never staged
<code>git rm --cached <path></code>	Stops tracking; the file stays on disk (4.6)	<code>git rm --cached .env</code>	●	Forgetting that the commit deletes the file for everyone who pulls	<code>git restore --staged <path></code> before committing
<code>git mv <a> </code>	Renames the file and its index entry (4.8)	<code>git mv old.py new.py</code>	●	Renaming and editing in one commit: history of the file stops at the rename	Rename and edit in separate commits

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git stash push -u</code>	Parks index and working tree as commits under <code>refs/stash</code> (11.11)	<code>git stash push -u -m "wip retry"</code>	●	Leaving long-lived work in the stash	<code>git stash pop --index</code>
<code>git stash pop</code>	Applies the newest entry and drops it (11.11)	<code>git stash pop --index</code>	●	Popping onto local edits: conflict, entry kept	<code>git reset --merge</code> , then <code>git stash branch <name></code>
<code>git switch <branch></code>	Moves HEAD, index and working tree; refuses to lose edits (7.6)	<code>git switch -c feature/streaming</code>	●	Committing on a detached HEAD and leaving	<code>git branch <name> <id></code> from <code>git reflog</code>
<code>git config get, list</code>	Reads configuration with its origin (14B.3; Git 2.46 or later)	<code>git config list --show-origin --show-scope</code>	●	Looking in one scope only	Not needed
<code>git config set</code>	Writes one configuration file (14B.3; Git 2.46 or later)	<code>git config set --global pull.ff only</code>	●	A mistyped key is stored and ignored	<code>git config unset</code>

History

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git log <A>..</code>	Commits reachable from B and not from A (14A.8)	<code>git log --oneline origin/main..HEAD</code>	●	Confusing it with <code>git diff A..B</code> , which compares two tips	Not needed
<code>git log <A>... --left-right</code>	Commits on either side and not both (14A.14)	<code>git log --oneline --left-right main...feature/x</code>	●	Reading three dots in <code>log</code> like three dots in <code>diff</code>	Not needed
<code>git diff <A>...</code>	Merge base against B: what a branch adds (14A.2)	<code>git diff --stat main...feature/x</code>	●	Two dots for review	Not needed
<code>git show <commit></code>	One commit with its diff; a file at a revision (3.5)	<code>git show HEAD~1:config/limits.yaml</code>	●	Reading a merge's combined diff as the whole merge	<code>git show --remerge-diff <merge></code>
<code>git log -S<string></code>	Commits that changed the number of occurrences (14A.11)	<code>git log -S'PASS_MARK' --oneline</code>	●	Taking the hit for the cause without reading the patch	Check with <code>-G</code> and <code>git show</code>
<code>git log -L</code>	History of a range of lines or a function (14A.12)	<code>git log -L :score:scoring/metrics.py</code>	●	Anchoring on line numbers	A <code>/regex/</code> or <code>:funcname range</code>
<code>git log --follow -- <path></code>	History of one file across renames (14A.10)	<code>git log --follow --oneline --src/app.py</code>	●	Expecting it to survive a rename plus rewrite	Read on under the old path
<code>git blame</code>	The commit that last changed each line (14A.16)	<code>git blame -w -C -L 10,30 <file></code>	●	Stopping at a formatting commit	<code>--ignore-rev</code> , <code>--ignore-revs-file</code>
<code>git shortlog -sn <rev></code>	Commit counts per author (14A.15)	<code>git shortlog -sn --no-merges HEAD</code>	●	Omitting the revision in a script: it reads standard input	Pass the revision
<code>git describe</code>	Version string from the nearest tag (14B.12)	<code>git describe --tags --dirty</code>	●	Shallow clone without tags	<code>git fetch --unshallow --tags</code>
<code>git range-diff</code>	Compares two versions of a commit series (9.14)	<code>git range-diff main ORIG_HEAD HEAD</code>	●	Pushing a rebase without it	Not needed

Branches and integration

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git branch <name> [<start>]</code>	Adds one ref (7.5)	<code>git branch rescue/before-rebase</code>	●	Branching on an unborn branch	Make the first commit
<code>git branch -d</code>	Deletes a merged branch and its reflog (7.5)	<code>git branch -d feature/x</code>	●	Reading the refusal after a squash merge as a bug	<code>git branch <name> <id></code> from the <code>was <id></code> line
<code>git branch -D</code>	Deletes with no merge check (7.5)	<code>git branch -D feature/x</code>	●	Deleting the only name of unmerged commits	HEAD reflog, while the objects exist
<code>git merge <other></code>	Fast-forward or three-way merge into the current branch (8.3, 8.4)	<code>git merge --no-ff feature/x</code>	●	Whole-file <code>--ours</code> or <code>--theirs</code> in a conflict	<code>git reset --merge ORIG_HEAD</code> ; pushed: <code>git revert -m 1</code>

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git merge --abort</code>	Back to the state before the merge (8.10)	<code>git merge --abort</code>	🔴	Losing resolutions and edits staged during the merge	<code>git fsck</code> for staged content
<code>git merge-tree --write-tree</code>	A test merge without touching anything (8.17)	<code>git merge-tree --write-tree --name-only HEAD feature/x</code>	🟢	The deprecated three-argument form	Not needed
<code>git rebase <upstream></code>	Re-creates your commits on top of the upstream; new IDs (9.2)	<code>git rebase origin/main</code>	🟡	Rebasing a branch others push to	<code>git reset --hard <branch>@{1}</code> 🔴
<code>git rebase -i</code>	Reorder, squash, edit, drop commits (9.6)	<code>git rebase -i --autosquash origin/main</code>	🟡	Deleting a line: the commit is dropped silently	Same reset; <code>rebase.missingCommitsCheck=error</code>
<code>git rebase --onto <new> <old></code>	Replays only the commits after <old> (9.5)	<code>git rebase --onto main feature/base</code>	🟡	The wrong boundary: conflicts in files you never changed	<code>git rebase --abort</code>
<code>git rebase --continue</code> , <code>--abort</code>	Go on after resolving; or give up (9.11)	<code>git rebase --continue</code>	🟡	<code>--amend</code> at a conflict stop; forgetting that "ours" is the new base	Reflog of the branch
<code>git cherry-pick -x <commit></code>	Copies one change as a new commit (10.2)	<code>git cherry-pick -x <id></code>	🟡	No <code>-x</code> on a backport	<code>git cherry-pick --abort</code> ; pushed: <code>git revert</code>
<code>git tag -a <name></code>	Annotated tag: a tag object plus a ref (14B.8)	<code>git tag -a v1.0.0 -m "1.0.0"</code>	🟢	Moving it after publishing	<code>git tag -d <name></code> while unpublished
<code>git worktree add</code>	A second working tree on the same repository (25.2)	<code>git worktree add ../hotfix release/1.2</code>	🟢	Removing it with <code>rm -rf</code>	<code>git worktree prune</code>

Remotes

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git fetch</code>	Updates remote-tracking refs; never your branches (12.4)	<code>git fetch origin</code>	🟢	Believing it changed your branch	Not needed
<code>git fetch --prune</code>	Also deletes stale remote-tracking refs and their reflogs (12.11)	<code>git fetch --prune</code>	🟡	Pruning the last name of commits you still need	<code>git fsck --lost-found</code> , then <code>git branch <name> <id></code>
<code>git pull --ff-only</code>	Fetch, then fast-forward or stop (12.6)	<code>git pull --ff-only</code>	🟡	Plain <code>git pull</code> on diverged branches without a setting: fatal	<code>git reset --hard ORIG_HEAD</code> 🔴, at once
<code>git pull --rebase</code>	Fetch, then re-create your unpushed commits (9.13)	<code>git pull --rebase</code>	🟡	After someone's forced push it can drop commits	<code>git reset --hard <branch>@{1}</code> 🔴
<code>git push -u origin <branch></code>	Publishes the branch and sets its upstream (12.5)	<code>git push -u origin feature/x</code>	🟡	Everything up-to-date while the commit is on another branch	Push the branch that holds it
<code>git push --dry-run</code>	Shows <source> -> <destination> without sending (12.7)	<code>git push --dry-run</code>	🟢	Skipping it before a push that matters	Not needed
<code>git push --force-with-lease --force-if-includes</code>	Replaces the server's branch only if it is what you last integrated (12.8)	<code>git push --force-with-lease --force-if-includes</code>	🔴	Using it on a branch others push to	The old tip from your reflog, pushed back
<code>git push --force</code>	Replaces the server's branch unconditionally (12.8)	none: use the guarded form	🔴	Answering "non-fast-forward" with it	A clone that still has the old tip
<code>git push origin --delete <branch></code>	Deletes a ref on the server (12.7)	<code>git push origin --delete feature/x</code>	🔴	Deleting a branch with unmerged commits	<code>git push origin <id>:refs/heads/<name></code>
<code>git push origin <tag></code>	Publishes one tag (14B.10)	<code>git push origin v1.0.0</code>	🟡	<code>--tags</code> , which publishes every local tag	None that is clean
<code>git ls-remote</code>	What the server has now (12.4)	<code>git ls-remote --tags origin</code>	🟢	Deciding from stale remote-tracking refs	Not needed
<code>git remote set-url</code>	Changes where a remote points (12.2)	<code>git remote set-url origin <url></code>	🟡	A token inside the URL	Set the previous URL; revoke the token

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git submodule update --init --recursive</code>	Checks out the recorded submodule commits (23.3)	<code>git submodule update --init --recursive</code>	●	Work on the detached HEAD vanishes at the next update	<code>git -C <path> branch <name> <id></code>

Undo

Situation	Command	Example	Risk	Common mistake	Recovery
Unstage a file	<code>git restore --staged <path></code> (5.9)	<code>git restore --staged notes.txt</code>	●	<code>git reset --hard</code> instead	<code>git add <path></code>
Discard edits to a file	<code>git restore <path></code> (4.7)	<code>git restore -p config.yaml</code>	●	No <code>git diff</code> first	None unless staged once
Get one file from another commit	<code>git restore --source=<commit> <path></code> (11.3)	<code>git restore --source=HEAD~2 -- config.yaml</code>	●	A <code>.</code> as the path: deletes files the commit lacks	<code>git restore .</code> for the index versions
Fix the last commit, unpushed	<code>git commit --amend</code> (11.7)	<code>git commit --amend</code>	●	Unrelated staged changes are swallowed	<code>git reset --soft 'HEAD@{1}'</code>
Take back commits, keep the changes	<code>git reset --soft <commit></code> or <code>git reset <commit></code> (11.4)	<code>git reset --soft HEAD~1</code>	●	Doing it to pushed commits	<code>git reset --soft ORIG_HEAD</code> , at once
Move the branch, keep local edits safe	<code>git reset --keep <commit></code> (11.6)	<code>git reset --keep origin/main</code>	●	Reaching for <code>--hard</code>	<code>ORIG_HEAD</code> , the reflog
Throw away commits and edits	<code>git reset --hard <commit></code> (11.5)	<code>git reset --hard origin/main</code>	●	Uncommitted work in the tree	Commits: reflog. Staged: <code>git fsck --lost-found</code> . Unstaged: none
Undo a pushed commit	<code>git revert <commit></code> (11.8)	<code>git revert --no-edit <id></code>	●	Resetting and force-pushing instead	Revert the revert
Undo a pushed merge	<code>git revert -m 1 <merge></code> (11.9)	<code>git revert -m 1 <id></code>	●	Merging the same branch again later: its changes are missing	Revert the revert, then merge
Remove untracked files	<code>git clean -n</code> , then <code>git clean -f</code> (4.14)	<code>git clean -n -d</code>	●	<code>-X</code> in a repository with ignored data	None through Git
Leave a merge, rebase or pick	<code>--abort</code> (29.6)	<code>git rebase --abort</code>	● (merge: ●)	Aborting before anchoring new commits	HEAD reflog

Recovery

Situation	Command	Example	Risk	Common mistake	Recovery
Name a state before touching anything	<code>git branch rescue/<what> <id></code> (13.7)	<code>git branch rescue/before-fix</code>	●	A second fix before the anchor	<code>git branch -D</code> when done
Find where a branch was	<code>git reflog show <branch></code> (13.3)	<code>git reflog show main</code>	●	Using <code>ORIG_HEAD</code> later than the next command	The branch reflog
Commits lost by reset, amend or rebase	<code>git reset --keep <id></code> or <code>git cherry-pick <range></code> (13.8)	<code>git reset --keep 'main@{1}'</code>	●	<code>--hard</code> during recovery	<code>ORIG_HEAD</code>
A deleted branch	<code>git branch <name> <id></code> (13.8)	<code>git branch feature/x <id></code>	●	Looking for the deleted branch's reflog	HEAD reflog; <code>git fsck --no-reflogs</code>
Staged content that was overwritten	<code>git fsck --lost-found</code> (13.9)	<code>git fsck --lost-found</code>	●	Running <code>git gc --prune=now</code> first	<code>git cat-file -p <id> > <path></code>
A dropped stash	<code>git stash store -m <message> <id></code> (13.9)	<code>git fsck --unreachable</code> to find the ID	●	<code>git stash clear</code>	While the objects exist
A teammate force-pushed	<code>git reflog show origin/<branch></code> (13.10)	<code>git branch rescue/server-before origin/main@{1}</code>	●	Pulling first	Push the old tip back, with a lease
A backup that can travel	<code>git bundle create <file> --all</code> (13.14)	<code>git bundle verify repo.bundle</code>	●	Expecting reflogs or uncommitted files in it	<code>cp -Rp</code> of the repository for those
The point of no return	<code>git reflog expire --expire=now --all</code> , <code>git gc --prune=now</code> (13.13)	none: only at the end of a secret clean-up	●	Running them as "maintenance"	None

Debugging

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git bisect start, good, bad</code>	Binary search for the commit that changed a property (14A.20)	<code>git bisect start HEAD v1.2.0</code>	●	A dirty tree; a verdict on a commit that does not build	<code>git bisect reset</code>
<code>git bisect run <command></code>	Lets a script give the verdicts; exit 125 skips (14A.21)	<code>git bisect run ./test.sh</code>	●	A script that fails for other reasons than the bug	<code>git bisect log</code> , <code>edit</code> , <code>git bisect replay</code>
<code>git check-ignore -v</code>	The pattern that ignores a path (4.5)	<code>git check-ignore -v build/out.log</code>	●	Asking about a tracked file	<code>--no-index</code>
<code>git ls-files --stage</code>	The index, entry by entry; stages 1 to 3 in a conflict (5.11)	<code>git ls-files -u</code>	●	Missing the lower-case tag of <code>-v</code> for assume-unchanged	<code>git update-index --no-assume-unchanged</code>
<code>git rev-parse</code>	A name as an object ID; facts about the repository (3.6)	<code>git rev-parse --abbrev-ref HEAD</code>	●	Reading <code>.git</code> files in scripts	Not needed
<code>git cat-file -p, -t</code>	Content and type of any object (3.5)	<code>git cat-file -p HEAD</code>	●	Assuming 40-digit IDs	Not needed
<code>git config list --show-origin --show-scope</code>	Every setting and the file it comes from (14B.2)	<code>git config get --show-origin user.email</code>	●	Forgetting includes and environment variables	<code>git var GIT_AUTHOR_IDENT</code>
<code>GIT_TRACE=1 git <command></code>	What Git actually ran (14B.7)	<code>GIT_TRACE=1 git st</code>	●	Pasting a trace that holds credentials	Not needed
<code>git diff --check</code>	Conflict markers and whitespace errors (14A.6)	<code>git diff --cached --check</code>	●	Committing a merge without it	A new commit that removes the markers
<code>git fsck</code>	Integrity of the object database (3.8)	<code>git fsck</code>	●	Reading dangling as damage	Not needed

Advanced

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git commit --fixup= <commit> with git rebase --autosquash</code>	A correction that finds its target at the next rebase (9.7)	<code>git commit --fixup= <id></code>	●, then ●	Forgetting the rebase before merging	Fold it in, or drop it
<code>git rebase --update-refs</code>	Moves every local branch in the range: stacks stay stacked (9.9; Git 2.38 or later)	<code>git rebase --update-refs main</code>	●	It moves branches you did not name	Each branch from its reflog
<code>git rerere</code>	Replays recorded conflict resolutions (14C.3)	<code>git rerere diff</code>	●, ● for forget	A wrong resolution is replayed too	<code>git rerere forget <path></code> , <code>git restore --merge <path></code> ●
<code>git add --renormalize .</code>	Restages every file under the current attributes (14C.5)	<code>git add --renormalize .</code>	●	Mixing it with other changes	<code>git restore --staged .</code>
<code>git hook list <event></code>	Which hooks would run, and from which scope (14C.11)	<code>git hook list --show-scope pre-commit</code>	●	Treating hooks as enforcement	Enforce on the server
<code>git sparse-checkout set</code>	Limits the working tree to a cone of directories (24.4)	<code>git sparse-checkout set services/gateway</code>	●	Ignored files in directories that leave the cone are deleted	None for those files
<code>git clone --filter=blob:none</code>	A blobless partial clone: full history, blobs on demand (26.12)	<code>git clone --filter=blob:none <url></code>	●	Going offline without <code>git backfill</code>	Reconnect
<code>git maintenance run</code>	Packs, commit-graph, multi-pack-index (26.3)	<code>git maintenance run -task=commit-graph</code>	●	Running <code>git gc</code> beside it	None for expired reflog entries
<code>git gc --prune=now</code>	Deletes all unreachable objects at once (13.13)	none: see Recovery	●	"It fixes a slow repository"	None in this repository
<code>git worktree remove</code>	Deletes a clean working tree and its HEAD reflog (25.6)	<code>git worktree remove ../hotfix</code>	● (● with --force)	Detached commits made there lose their name	<code>git fsck --lost-found</code>
<code>git lfs track, git lfs migrate info</code>	Which paths become pointers; what a migration would rewrite (22.4, 22.9)	<code>git lfs migrate info --above=1mb</code>	●	Tracking after the file is in history	<code>git lfs migrate import</code> on unpushed commits ●

Command	Purpose	Example	Risk	Common mistake	Recovery
<code>git history reword</code>	Rewrites one commit and moves descendant branches (14D.6; experimental, Git 2.54 or later)	<code>git history reword --dry-run HEAD~2</code>	●	Using it on published commits	Each branch from its reflog
<code>git update-ref <ref> <new> [<old>]</code>	Writes one ref and nothing else (7.2)	<code>git update-ref refs/backup/main <id></code>	● (● with -d)	Moving the current branch with it	<code>git update-ref <ref> "<ref>@{1}"</code>

Chapter 33 (continued): Emergency recovery, one page

Source file: `cheatsheets/emergency-recovery-one-page.md`

Emergency recovery: one page

Git 2.55.0. ● reads or only adds · ● moves refs, recoverable through the reflog · ● can destroy work or the safety net. Every command was run in this book's labs. Details: disaster-recovery playbook.

Stop. No `git gc`, `git prune`, `git clean`, `git stash clear`, `git fetch --prune`, re-clone or second fix until the lost state has a name.

Look (always first)

Command		Shows
<code>git status -sb</code>	●	branch, upstream, ahead and behind, operation in progress
<code>git reflog show <branch></code>	●	every value the branch had, and the command that set it
<code>git reflog</code>	●	every commit HEAD was on (survives branch deletion)
<code>git reflog show origin/<branch></code>	●	the server's values as this clone saw them; @{1} is the one before the last change
<code>git ls-remote origin</code>	●	what the server holds now
<code>git fsck --lost-found</code>	●	dangling commits and blobs; copies in <code>.git/lost-found/</code>

Anchor (before anything moves)

Command		
<code>git branch rescue/<what> <ID></code>	●	a named commit cannot expire

Recover

Disaster	Command	
Reset, amend or rebase took commits away	<code>git reset --keep rescue/<what></code>	●
The same, but new commits were made since	<code>git cherry-pick <branch>..rescue/<what></code>	●
Branch deleted (Deleted branch x (was <ID>))	<code>git branch <name> <ID></code>	●
Rebase in progress, going wrong	<code>git rebase --abort</code>	●
Merge in progress, going wrong	<code>git merge --abort</code> (discards the resolutions made so far)	●
Bad merge committed, not pushed	<code>git reset --keep ORIG_HEAD</code>	●
Bad commit on a shared branch	<code>git revert --no-edit <ID></code>	●
Bad merge on a shared branch	<code>git revert -m 1 <merge></code> (a later re-merge then brings nothing)	●
Staged file overwritten	<code>git cat-file -p <blob ID> > <path></code>	●
Stash dropped	<code>git stash store -m 'recovered' <ID></code>	●
Server branch force-pushed: keep the newcomers	<code>git push origin origin/<branch>:refs/heads/<new-name></code>	●
Server branch force-pushed: restore	<code>git push --force-with-lease=<branch>:<bad ID> origin rescue/<what>:<branch></code>	●
Server branch deleted	<code>git push origin <name></code>	●
My clone after the server was restored	<code>git cherry -v origin/<branch> <branch></code> , then <code>git reset --keep origin/<branch></code>	● ●
Corrupt index	<code>mv .git/index .git/index.corrupt && git reset</code>	●
Missing or corrupt objects	<code>git fetch --refetch origin</code>	●
Secret committed	Rotate the credential first. Then the playbook, page 11	

Verify

Command		Proves
<code>git range-diff <base> <old> <new></code>	●	= on every line: the same commits came back
<code>git diff --stat <old> <new></code>	●	no output: identical content
<code>git merge-base --is-ancestor <ID> <branch></code>	●	exit 0: the commit is on the branch
<code>git ls-remote origin <branch></code>	●	the server has the ID you intended

Never in an emergency

Command		Why
<code>git reset --hard</code>	●	destroys uncommitted changes; use <code>--keep</code>
<code>git push --force</code>	●	no condition; use <code>--force-with-lease=<branch>:<ID></code>
<code>git reflog expire --expire=now --all, git gc --prune=now</code>	●	deletes the safety net itself
<code>git clean -f -d</code>	●	deletes untracked files; no undo

Chapter 33 (continued): Command safety

Source file: `reference/command-safety.md`

Command safety: every and command of the book

Baseline. Git 2.55.0 on macOS; GitHub CLI 2.88.1; GitHub facts as of 1 October 2026. This file is compiled from the "Command safety" tables and the inline risk labels of Chapters 1 to 30. It adds nothing to them. The section number in the last column is where the textbook answers the questions in full.

1. How to read this file

The three labels are defined in Chapter 1, section 1.8, and they describe the worst case:

Label	Meaning	Examples from 1.8
 SAFE	Reads state or only adds objects. Nothing is lost	<code>git status</code> , <code>git log</code> , <code>git fetch</code> , <code>git reflog</code> , <code>git commit</code>
 CAUTION	Moves refs or rewrites local history. Recoverable through the reflog if you know how	<code>git reset --soft</code> , <code>git rebase</code> , <code>git commit --amend</code>
 DANGEROUS	Can destroy uncommitted work, remote history, or the safety net itself	<code>git reset --hard</code> , <code>git clean -fd</code> , <code>git push --force</code> and its conditional forms <code>--force-with-lease</code> and <code>--force-if-includes</code> , <code>git reflog expire --expire=now --all</code> , <code>git gc --prune=now</code>

Before any  command the book answers five questions: what it changes, what it can destroy, how to preview it, how to recover, when it is appropriate. The tables below give those five answers for every  and every  command.  commands are not listed here; they are in the Git command reference.

Three rules follow from the tables and are worth knowing before you read them:

1. **The preview is always a  command.** Run it, read every line, then decide.
2. **"Reflog" in the Recover column means a local reflog.** A fresh clone and a CI runner have none (Chapter 13, section 13.16), tags and remote-tracking refs have none by default (Chapter 7, section 7.11), and the lab configuration switches reflog expiry off while real repositories expire entries after 90 days, or 30 days when unreachable (Chapter 13, section 13.4).
3. **Content that was never staged and never committed is in no object.** No reflog and no `git fsck` returns it (Chapter 13, section 13.12). That is the common reason for a  label.

Earlier drafts of the chapters labelled a few commands in two ways. One label per command has since been chosen, and section 12 records every decision and its reason.

2. Working tree and index

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git restore <path></code> , <code>git restore -p <path></code> , <code>git checkout -- <path></code>		Working tree files, from the index	Every unstaged change in the named paths, without confirmation	<code>git diff -- <path></code>	None for content that was never staged; <code>git fsck</code> for content that was	After you have read the diff and decided the changes are worthless; <code>-p</code> to discard hunk by hunk	4.7, 11.3
<code>git restore --source=<commit> [--staged] [--worktree] <path></code>		Files (and index entries) from a commit; deletes tracked paths the commit lacks	Unstaged edits in those paths	<code>git status --short -- <path></code> ; <code>git diff <commit> -- <path></code>	None for unstaged edits	Bringing back a deleted file, where no file of that name exists; name the paths you mean	4.7, 11.3, 14A.13
<code>git restore <path></code> on a file with the assume-unchanged bit		Overwrites the hidden edit without warning	The edit the bit was hiding	Clear the bit, then <code>git diff</code>	None	Never as a way to keep local edits: the bits are not an ignore mechanism	5.12
<code>git restore <notebook></code> , <code>git switch</code> across a notebook	 for local outputs	Rewrites the file from the stripped blob	Notebook outputs, which were never in Git	<code>git status</code> ; copy the file first	None; rerun the notebook	When the working copy holds nothing that exists only there	28.18

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
change, with a clean filter that strips outputs							
<code>git clean -f [-d] [-X or -x]</code>	●	Deletes untracked files; with <code>-d</code> untracked directories; with <code>-x</code> or <code>-X</code> ignored files	Files Git has no copy of: <code>.env</code> , downloaded datasets, model checkpoints, virtual environments	The same options with <code>-n</code> in place of <code>-f</code>	None through Git	To prove a build from a pristine tree or reset a CI workspace; prefer <code>-X</code> to <code>-x</code> , pass a pathspec, protect files with <code>-e</code>	4.14, 11.10
<code>git switch --discard-changes, git checkout -f</code>	●	Index and working tree, to the target	Uncommitted edits in the files it touches	<code>git status</code> , <code>git diff</code>	None for content that was never staged	Only when you have decided to throw the edits away	7.15
<code>git switch <branch></code> when an ignored file is tracked on that branch	●	Overwrites the ignored file	The ignored file's content	<code>git ls-tree -r --name-only <branch></code> compared with <code>git status --ignored</code>	None; prevent with <code>--no-overwrite-ignore</code>	Not a choice: know the case before switching	4.17
<code>git rm -f <path></code>	●	As <code>git rm</code> , without the check	Uncommitted changes in the file	<code>git diff HEAD -- <path></code>	None for unstaged content	Only after reading that diff	4.8
<code>git rm <path></code>	●	Deletes the file and its index entry; refuses if the content is not in HEAD	Nothing: the content is in HEAD	<code>git rm -n <path></code>	<code>git restore --staged --worktree <path></code>	Removing a tracked file from the project	4.8
<code>git rm --cached <path></code>	●	Removes the index entry; the file on disk stays	The next commit deletes the path for everyone who pulls	<code>git rm --cached -n <path></code> ; <code>git status</code>	<code>git restore --staged <path></code> before committing	A file that was tracked by mistake; announce the commit with the restore command	4.6, 5.9, 21B.10, 28.9
<code>git restore --staged <path></code> , <code>git reset [-] <path></code>	●	Index entries, copied from HEAD	The name of a staged version that differed from the file	<code>git diff --cached -- <path></code>	<code>git add <path></code> ; <code>git fsck --lost-found</code> for a staged version that differed from the file	Unstaging	5.9, 11.3
<code>git update-index --add --cacheinfo <mode>, <id>, <path></code>	●	One index entry	Nothing	<code>git ls-files --stage</code>	<code>git restore --staged <path></code>	Building a commit by hand to learn the plumbing	2.6
<code>git update-index --assume-unchanged, --skip-worktree</code>	●	A flag bit that hides later edits from <code>git status</code>	Nothing at once; sets up the ● row above	<code>git ls-files -v</code>	The <code>--no-</code> forms	Not as an ignore mechanism	5.12
<code>git add --renormalize .</code>	●	Stages every tracked file whose stored form changes under the current attributes and filters	Nothing	<code>git ls-files --eol</code> ; <code>git status</code> before committing	<code>git restore --staged .</code> before committing	One renormalizing commit when attributes or filters change	14C.5, 20B.18, 28.18
<code>git apply <patch></code>	●	Working tree files (and the index with <code>--index</code>)	Nothing that the reverse patch does not restore	<code>git apply --check, git apply --stat</code>	<code>git apply -R</code>	Applying a diff that is not a commit	14D.11, 28.18
<code>rm .git/index.lock</code>	●	Removes a lock; unsafe while a Git process is writing	A write in progress	<code>pgrep -fl git</code>	None needed for a stale lock	Only when no Git process is running	5.15
<code>rm .git/index</code> , then <code>git reset</code>	●	Discards staging state, flag bits and conflict stages; outside Git's locking	What was staged and not committed loses its index entry	<code>git diff --cached</code> ; move the file aside instead of deleting it	<code>git fsck</code> for blobs that were staged; move the file back	"index file corrupt", "index file smaller than expected"	5.15, 13.18

3. Stash

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git stash push [-u]</code>	●	Records index and working tree as commits that only	Nothing while the entry exists	<code>git status -s</code>	<code>git stash pop --index</code>	Parking work for minutes; a branch	11.11, 14C.2

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
		<code>refs/stash</code> reaches, then resets both to HEAD; with <code>-u</code> also removes untracked files				is better for work that must survive	
<code>git stash apply</code> , <code>git stash pop</code> , <code>git stash branch <name></code>	●	Merges an entry into the files; can conflict	Nothing: the entry is kept after a conflict	<code>git stash show -p</code>	<code>git reset --merge</code> backs out; then <code>git stash branch <name></code>	Pop soon; clear local edits before popping	11.11
<code>git stash drop</code> , <code>git stash clear</code>	●	Deletes one line, or all lines, of the log of <code>refs/stash</code>	The stashed changes, once the unreachable commits are pruned	<code>git stash show -p 'stash@{n}'</code>	<code>git stash store <id></code> while the objects exist; without the ID, the <code>git fsck --unreachable</code> recipe of Lab 8.6	drop after <code>git stash show -p</code> ; <code>clear</code> is almost never appropriate	11.11, 14C.15

4. Commits, branches and refs

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git commit --amend</code> (with or without <code>--no-edit</code> , <code>--only</code> , <code>--reset-author</code>)	●	Replaces the tip commit with a new one that has a new ID	Nothing locally; a published commit that others have is not replaced for them	<code>git diff --cached</code> , <code>git log -1</code>	<code>git reset --soft 'HEAD@{1}'</code>	Unpublished commits only	6.7, 11.7
<code>git commit --no-verify</code>	●	As <code>git commit</code> , without the <code>pre-commit</code> and <code>commit-msg</code> hooks	Nothing; the checks did not run	Run the hook's checks by hand	Amend, or a further commit; the server may still refuse	When a hook is wrong and you have run its checks yourself	6.15, 14C.11, 28.18
<code>git branch -m <name></code>	●	Renames the ref, its reflog and its configuration section	Nothing	<code>git branch -vv</code>	Rename it back	Before the first push	7.5
<code>git branch -f <name> <commit></code> , <code>git switch -C</code>	●	Moves a ref; the old value stays in the reflog	Nothing while the reflog entry lives	<code>git branch -vv</code> , <code>git rev-parse <ref></code>	<code>git branch -f <ref> <ref>@{1}</code>	Putting a branch where you have decided it belongs	7.5
<code>git branch -d <name></code>	●	Deletes the ref and its reflog after a merge check against the upstream or HEAD	The branch reflog	<code>git branch --merged</code> , <code>git branch -vv</code>	<code>git branch <name> <id></code> from the was <code><id></code> output or from <code>git reflog</code>	Merged branches	7.5, 27.21
<code>git branch -D <name></code>	●	Deletes the ref and its reflog with no merge check	The only name of unmerged commits	<code>git log --oneline main..<branch></code> ; <code>git diff main <branch></code>	<code>git branch <name> <id></code> while the objects exist; the HEAD reflog if the commits were checked out	After the diff against the target is empty (a squash-merged branch)	7.5, 13.8, 17.9, 30.23
<code>git tag -f <name></code> , <code>git tag -d <name></code>	●	Moves or deletes a local tag; tags have no reflog by default	The only record of where the tag pointed	<code>git rev-parse <tag></code> , and write the ID down	<code>git tag <name> <old-id></code> from the was <code><id></code> output; <code>git fsck</code> lists a dangling tag object	Unpublished tags	7.11, 14B.21
<code>git update-ref <ref> <new> [<old>]</code>	●	One ref and its reflog, and nothing else: not the index, not the working tree	Nothing while the reflog entry lives	<code>git rev-parse <ref></code> ; pass <code><old></code> so that a surprise stops the command	<code>git update-ref <ref> "<ref>@{1}"</code>	Scripts; full names that begin with <code>refs/</code>	2.6, 3.17, 7.2
<code>git update-ref -d <ref></code>	●	Deletes the ref and its reflog, even for the current branch	The only name that leads to those commits	<code>git rev-parse <ref></code> ; write the ID down	<code>git update-ref <ref> <id></code> ; without the ID, <code>git fsck --no-reflogs</code> lists the commits as dangling while they exist	Scripts that remove refs they created themselves	2.14
<code>git symbolic-ref HEAD <ref></code>	●	Which branch is current, without	Nothing	<code>git symbolic-ref HEAD</code>	<code>git symbolic-ref HEAD <previous></code> ; <code>git</code>	Repairs and demonstrations;	3.17, 7.3

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
		touching the index or the working tree			reflog	porcelain otherwise	
Writing a file under <code>.git/refs/</code> by hand	●	A ref without validation, locking or reflog	A typo produces a broken ref that <code>git log --all</code> refuses to read	Use <code>git update-ref</code> instead	<code>git update-ref</code> after deleting the broken file (Lab 4.1)	Never	7.15
<code>rm</code> of a file under <code>.git/refs</code>	●	Removes a ref outside Git's locking	The ref's value	Move the file aside instead of deleting it	Move it back	An empty or unreadable ref file that <code>git update-ref -d</code> cannot delete	13.11, 13.18
Deleting or editing files under <code>.git/objects</code>	●	The object database	One object can be part of every snapshot in the history	None	Write the same content back, or copy the object from another clone (Lab 1.2)	Never	2.14
<code>rm -rf .git</code>	●	Deletes the repository and leaves the working tree files	Every commit that was never pushed, every local branch, stash and reflog	<code>git log --online --all</code> , <code>git branch -vv</code>	None through Git; another clone or a disk backup	A repository created by mistake that holds no commit you need	1.14

5. Reset

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git reset --soft <commit></code>	●	The branch ref	Nothing	<code>git log --online <commit>..HEAD</code>	The same mode with <code>ORIG_HEAD</code> , or the reflog	Regrouping private commits; <code>git reset --soft @{u}</code> after a rejected push of an amend	11.4, 21B.23
<code>git reset [--mixed] <commit></code>	●	The branch ref and the index	Nothing in files; staged versions lose their index entry	As above, and <code>git diff --cached</code>	As above	While the commits are private	11.4
<code>git reset --keep <commit></code>	●	The branch ref, the index, files that differ between the two commits; refuses when it would overwrite a local change	Nothing	<code>git log --online <commit>..HEAD</code>	<code>ORIG_HEAD</code> , the branch reflog	The reset to use during recovery, in place of <code>--hard</code>	11.6, 13.7, 30.23
<code>git reset --merge [<commit>]</code>	●	As <code>--keep</code> for unstaged changes; discards staged changes	Staged changes	<code>git diff --cached</code>	Commits: reflog. Staged content: <code>git fsck --lost-found</code>	Backing out of a merge, a pop or a pick that left the index unmerged	8.21, 11.6
<code>git reset --hard <commit></code> , <code>git reset --hard ORIG_HEAD</code>	●	The branch ref, the index, and every tracked file that differs from the target	Every uncommitted change to tracked files, and untracked files that are in the target's way	<code>git status -s</code> , <code>git diff HEAD --stat</code> , <code>git log --online <commit>..HEAD</code>	Commits: reflog. Staged content: <code>git fsck --lost-found</code> . Unstaged: none	When the working tree is known to be disposable; <code>git stash push -u</code> first when unsure; <code>ORIG_HEAD</code> only as the very next command	9.16, 11.5, 13.7

6. Merge, rebase, cherry-pick, revert, bisect

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git merge <other></code> (fast-forward or true merge), <code>--no-ff</code> , <code>--ff-only</code>	●	Moves the current branch; rewrites index and working tree	Nothing permanently	<code>git merge-tree --write-tree --name-only HEAD <other></code> ; <code>git diff HEAD...<other></code> ; <code>git log HEAD...<other></code>	Before pushing: <code>git reset --merge ORIG_HEAD</code> or <code>git reset --keep ORIG_HEAD</code> (check it first, or take the commit from the reflog). After pushing: <code>git revert -m 1</code>	From a clean tree	8.3, 8.4, 8.12, 27.21

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git merge -X ours, -X theirs, -X ignore-space-change</code>	●	As above, and silently drops one side of every conflicting hunk	The dropped side's changes, in the result	Merge without the option first	As above; audit with <code>-rmerge-diff</code>	Only when one side's hunks are the decision	8.6
<code>git merge -s ours <other></code>	●	Records <other> as merged and takes none of its content	Nothing in objects; the branch counts as merged while its changes are missing	<code>git diff HEAD...<other></code> shows what is discarded	As for <code>git merge</code>	Deliberately superseding a branch	8.6
<code>git merge --squash <other>, git merge --no-ff --no-commit <other></code>	●	Index and working tree; no commit	Nothing	As for <code>git merge</code>	<code>git reset --merge</code>	Reviewing or editing the result before committing	8.12
<code>git merge --continue, git commit</code> during a merge	●	Creates the merge commit and moves the branch	Nothing	<code>git diff --cached -check</code> ; run the tests	<code>git reset --merge ORIG_HEAD</code> before pushing	After every conflict is resolved and checked	8.10
<code>git merge --quit</code>	●	Deletes the merge state only	Nothing in files; no command resumes the merge	<code>git status</code>	<code>git reset --merge</code>	Rarely	8.10
<code>git merge --abort, git reset --merge</code>	●	Resets index and working tree to HEAD; deletes the merge state	Every resolution made so far, and any uncommitted edit that was staged during the merge	<code>git status, git diff --cached --stat</code>	Staged content survives as unreferenced objects that <code>git fsck</code> lists (Lab 6.7); unstaged resolution edits are gone	The merge was a mistake or needs a different approach	8.10, 8.21
<code>git restore --ours --theirs --merge <path>, and the git checkout</code> forms	●	Overwrites the working tree file with one side, or with a fresh marker version	Your hand edits to that file, which exist nowhere else until you <code>git add</code> them	<code>git diff --ours, git diff --theirs, git diff <path></code>	<code>git restore --merge <path></code> brings back the starting point, not your edits	The whole file from one side is the decision; a resolution, yours or rerere's, is wrong	8.10, 8.21, 14C.15
<code>git rebase <upstream>, --onto, -i, --autosquash, --keep-base, --rebase=merges</code>	●	New commits; the branch ref, at the end	Nothing permanently	<code>git log --oneline <upstream>..HEAD</code> lists what will be replayed	<code>git reset --hard <branch>@{1}</code> , or <code>git reset --keep ORIG_HEAD</code> ; a backup ref	Your own unpublished commits; ask before rebasing a branch others push to	9.2, 9.5, 9.6
<code>git rebase --update-refs</code>	●	Also every local branch inside the range	Nothing permanently	With <code>-i</code> , the <code>update-ref</code> lines	Each branch from its own reflog	Stacked branches	9.9
<code>git rebase --root</code>	●	Every commit of the branch	Nothing locally	<code>git log --oneline</code>	Reflog; for a published repository there is no clean recovery	Unpublished repositories	9.23
<code>git rebase --continue</code>	●	Commits the index as the copy, runs on	Nothing	<code>git status, git diff --cached</code>	Reflog, after the rebase	After <code>git add</code> of the resolution; never <code>--amend</code> at a conflict stop	9.11
<code>git rebase --skip</code>	●	Omits the stopped commit	Nothing: the original is in the old history	<code>git rebase --show-current-patch</code>	Redo	A commit that is already upstream	9.11
<code>git rebase --abort</code>	●	Index and working tree back to the old tip	Resolution work in progress; commits made inside the rebase leave the branch	<code>git status, git diff, git log <branch>..HEAD</code>	Commits: the HEAD reflog or a rescue ref. Uncommitted edits: none	After anchoring new commits with a branch	9.11, 29.3
<code>git rebase --quit</code>	●	Removes the state directory; HEAD stays detached	Nothing	<code>git status</code>	<code>git switch <branch></code>	Rarely; leave with <code>-abort</code>	9.11

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git pull --rebase</code>	●	Fetches; re-creates your unpushed commits on the fetched upstream	Nothing permanently; after someone's forced push it can drop commits	<code>git fetch</code> , then <code>git log --oneline @{{u}}..HEAD</code>	<code>git reset --hard <branch>@{{1}}</code>	Unpushed local commits; look before integrating after a forced update	9.13, 12.6
<code>git pull --ff-only</code> , <code>git pull --no-rebase</code>	●	Fetch, then moves the branch forward or creates a merge commit; working tree	Nothing permanently	<code>git fetch</code> ; <code>git log ..@{{u}}</code> ; <code>git log --oneline --graph HEAD @{{u}}</code>	<code>git reset --hard ORIG_HEAD</code> (itself ●); the reflog	Not in automation: a job that pulls can create merge commits nobody reviewed	12.6, 12.16
<code>git cherry-pick <commit></code> , with or without <code>-x</code> , <code>-e</code> , <code>-m</code>	●	Adds commits to the current branch; updates index and working tree; refuses to overwrite local changes	Nothing	<code>git show <commit></code> ; <code>git merge-tree --write-tree --merge-base=<commit>^ HEAD <commit></code> ; <code>git cherry -v</code>	<code>git reset --keep ORIG_HEAD</code> or the branch reflog; <code>git revert</code> if already pushed	Backports (<code>-x</code> always), and recovery of single commits	10.2, 10.4, 13.7
<code>git cherry-pick -n <commit></code>	●	Index and working tree; no commit, no in-progress state	Nothing	<code>git show <commit></code>	<code>git restore --staged --worktree <paths></code>	Combining several staged changes into one commit	10.4
<code>git cherry-pick --skip</code> , <code>--abort</code> , <code>--quit</code>	●	<code>--skip</code> discards the stopped pick; <code>--abort</code> returns branch, index and working tree to the state before the sequence; <code>--quit</code> removes the sequencer state only	Resolution work in progress	<code>git status</code>	Completed copies are in the reflog; resolution work is gone	An interrupted sequence	10.6
<code>git revert <commit></code> , <code>git revert -m 1 <merge></code>	●	Adds one commit that applies the inverse change	Nothing now; after <code>-m 1</code> a later merge of the same branch is silently empty	<code>git show <commit></code> ; <code>git show --stat <merge></code>	Revert the revert; before pushing, <code>git reset --keep HEAD~1</code>	The undo for published history; <code>-m 1</code> only when the merged branch will never be merged again as it is, or with the re-merge procedure in the message	10.11, 11.8, 11.9, 30.23
<code>git revert -n <commit></code>	●	Index and files	Nothing	<code>git show <commit></code>	<code>git revert --abort</code>	Several reverts in one commit	11.16
<code>git merge --abort</code> , <code>git rebase --abort</code> , <code>git cherry-pick --abort</code> , <code>git revert --abort</code> , <code>git bisect reset</code> as "leave" commands	● (merge: ●, above)	End the operation; reset index and working tree to the starting commit	Uncommitted edits made during the operation	<code>git status</code> , <code>git diff</code> , <code>git log <branch>..HEAD</code>	Commits: the HEAD reflog or a rescue ref. Uncommitted edits: none	After the PRESERVE step of the troubleshooting method	29.6, 29.14
<code>git bisect start</code> , <code>good</code> , <code>bad</code> , <code>skip</code> , <code>run</code> , <code>replay</code>	●	Detaches HEAD and checks out other commits; writes <code>refs/bisect/*</code> and <code>BISECT_*</code>	Nothing	<code>git status</code> must be clean; <code>git rev-list --count <good>..<bad></code>	<code>git bisect reset</code>	Finding the commit that changed a property	14A.20
<code>git bisect reset</code>	●	Checks out the starting branch; deletes the bisect state	The session, unless you saved it	<code>git bisect log > file</code> to keep the session	<code>git bisect replay file</code>	At the end of a session	14A.22
<code>git history reword</code> , <code>fixup</code> , <code>split</code> (experimental; <code>reword</code> and <code>split</code> since Git 2.54, <code>fixup</code> since 2.55)	●	New commits; every descendant local branch moves	Nothing locally	<code>--dry-run</code> ; <code>git branch -r --contains <commit></code> first	Each branch's reflog	Local cleanup only	14D.6
<code>git replay --onto</code> , <code>--advance</code> , <code>--</code>	●	New commits; refs move in one transaction	Nothing where a reflog exists	<code>--ref-action=print</code>	Reflog, where one exists; none by default in a bare repository	Rebase and cherry-pick without a working tree	14D.7

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
revert (experimental; since Git 2.44, updates refs itself since 2.53, <code>--revert</code> since 2.54)							
<code>git am</code>	●	New commits on the current branch	Nothing	<code>git apply --check</code>	<code>git am --abort</code> while it is stopped; afterwards the reflog of the branch	The patch-based workflow	14D.11

7. Remotes, pushes and tags

Git, not GitHub. The rows in this section describe what Git does against any server. On GitHub, rulesets and branch protection can reject a push that Git would perform (Chapter 18), and a default bare Git server keeps no reflog (Chapter 12, section 12.8).

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git push</code> , <code>git push -u</code> , <code>git push --follow-tags</code>	●	Refs on the server (fast-forward or new), your remote-tracking refs; starts whatever listens for pushes	Nothing; published commits are not taken back quietly once others have fetched	<code>git push --dry-run</code> ; <code>git log @{u}..</code>	A revert commit and another push; moving the ref back needs a force	After reading <code><source> -> <destination></code> in the dry run	1.12, 12.7
<code>git push origin <tag></code> , <code>git push --tags</code>	●	Creates tags on the server; on GitHub starts every workflow with a matching tag filter	Nothing in Git; a published name cannot be taken back cleanly	<code>git push --dry-run</code> ; <code>git ls-remote --tags origin</code>	None that is clean: others may have fetched; deleting the tag does not unpublish what a workflow built	Push single tags or <code>--follow-tags</code> , not <code>--tags</code>	14B.10, 20A.18
<code>git push <remote> --delete <ref></code> , <code>git push <remote> :<ref></code>	●	Deletes a ref on the server, and your remote-tracking ref	The only name the branch's commits have there; a bare server keeps no record of the deletion	<code>git ls-remote <remote>; git log --oneline origin/<branch> --not origin/main</code>	From any clone that has the commit: <code>git push <remote> <id>:refs/heads/<name></code>	Branches that are merged, or abandoned by agreement	12.7, 27.21
<code>git push --force</code> , <code>git push -f</code> , <code>git push <remote> +<ref></code>	●	Sets the server's ref with no check, and your remote-tracking ref	Every commit on the server's branch that is not in your history, including commits you have never seen	<code>git fetch</code> , then <code>git log --oneline HEAD..origin/<branch>; git push --dry-run --force</code>	The old tip from any clone that had it (a teammate's branch, the reflog of anybody's remote-tracking ref), or the server's unreachable object until it is collected; push it back	None for the plain form: use a guarded form. Not in an incident	9.17, 12.8, 30.23
<code>git push --force-with-lease</code> alone	●	The server's ref, if it equals your remote-tracking ref; the check passes wrongly after a background fetch	As <code>--force</code> , when the remote-tracking ref was updated without your looking	<code>--dry-run</code> ; <code>git log HEAD..origin/<branch></code>	Only from a clone that has the commits	None: add <code>--force-if-includes</code> or an explicit value	9.17, 9.23
<code>git push --force-with-</code>	●	The server's ref, only if it	Commits on the server	<code>--dry-run</code> ; <code>git log <local>..<expect></code> lists	The old tip is in your own reflogs; a clone's reflog of	A branch that only you use, such as	9.17, 12.8,

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>lease --force-if-includes, git push --force-with-lease=<ref>: <expect></code>		still has the value you rebased from or named	that only the server and stale clones have; with these forms everything removed is something your clone has	what will leave the branch	<code>origin/<ref></code> ; on GitHub the Activity view (Chapter 13, section 13.15)	your own pull request branch after a rebase; restoring a known good value in an incident; tell teammates before the push	27.3, 30.23
<code>git push --mirror, git push --prune, git push --force --mirror origin</code>	●	Creates, forces and deletes server refs to match yours; from an ordinary clone <code>--mirror</code> would also publish your remote-tracking refs	Branches and tags that exist only on the remote	<code>git push --dry-run --force --mirror origin</code>	From other clones, ref by ref; on GitHub, the instruments of Chapter 13	Once, after a freeze, at the end of a verified rewrite, from a <code>--mirror</code> clone	12.16, 21B.17
<code>git push --force origin <tag></code>	●	Replaces a published tag	The meaning of a release name for everyone who fetches later; a workflow that names the tag runs different code	<code>git ls-remote --tags origin</code>	Force the old tag object back from a clone that has it	Putting a tag back at its published position; a name that is meant to move should be a branch	14B.11, 21A.7
<code>git push origin --delete <tag></code>	●	Deletes a published tag	The only published name of a release commit, and the tag under a GitHub release	<code>git ls-remote --tags origin</code>	Push it again from a clone that kept it	A tag pushed by mistake minutes ago, announced	14B.10
<code>git fetch --prune, git remote prune <remote></code>	●	Deletes stale remote-tracking refs and their reflogs	The reflog that recorded the server's earlier values	<code>git remote prune --dry-run <remote></code>	<code>git fsck --lost-found</code> , then <code>git branch <name> <id></code> , before garbage collection	Routine; not while a lost remote state still has no name	7.13, 12.11, 13.18
<code>git fetch --tags --force</code>	●	Replaces local tags with the server's	Your local value of a moved tag	<code>git ls-remote --tags; git fetch --tags --dry-run; note git rev-parse <tag> first</code>	Re-create from that ID; the old tag objects stay, dangling, until pruned	After confirming the server is right	12.4, 14B.11
<code>git fetch --prune --prune-tags</code>	●	Deletes every local tag the remote lacks	Private lightweight tags, which have no reflog and no object to find them by	Add <code>--dry-run</code>	Annotated tags: <code>git fsck</code> ; lightweight: only the IDs in the command's output	A mirror or a CI cache that must equal the server	14B.10
<code>git remote set-url, set-head, set-branches, rename; git branch -u; git config set push.default</code>	●	Where later fetches, pulls and pushes go	Nothing	<code>git remote -v; git config list --local</code>	Set the previous value	Deliberate changes of topology	12.2, 12.5, 12.10
<code>git remote remove <name></code>	●	The remote, its remote-tracking refs and reflogs,	The reflogs of the remote-tracking refs	<code>git branch -r, git branch -vv</code>	Add and fetch again; the reflogs are lost	A remote you no longer use	12.10

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
		upstream settings					

8. Reflogs, the object database and maintenance

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git gc, git maintenance run --task=gc</code>	●	Packs objects and refs; expires reflog entries by the configured periods; deletes unreachable objects older than two weeks	What was past the retention periods	<code>git fsck --unreachable; git reflog expire --dry-run --all; git count-objects -v; git prune -n</code>	Within the periods nothing is lost; beyond them, another clone or a backup	Rarely by hand: maintenance is automatic (geometric strategy by default since Git 2.54)	3.17, 13.13, 26.3
<code>git maintenance run</code>	●	Packs, packed-refs, commit-graph, multi-pack-index; expires reflog entries by the configured periods	Expired reflog entries	<code>git maintenance is-needed</code> (Git 2.53 or later); <code>GIT_TRACE=1</code>	None for expired reflog entries	Hundreds of packs or thousands of loose objects	26.3
<code>git maintenance start, git maintenance register</code>	●	Global configuration and the system scheduler	Nothing	<code>git help maintenance</code>	<code>git maintenance stop, git maintenance unregister</code>	A deliberate choice on your own machine; never in the labs	26.3
<code>git gc --prune=now, git prune</code>	●	Deletes unreachable objects at once, with no grace period	Everything that only the grace period was keeping: dropped stashes, staged blobs, commits whose reflog entries are gone	<code>git fsck --unreachable; git prune -n</code>	None in this repository; another clone or a backup	The purge after a deliberate history rewrite, with no other process using the repository	3.17, 13.13, 26.5
<code>git repack -a -d</code>	●	Rewrites packs; drops the unreachable objects of the packs it replaces	Unreachable objects in those packs	<code>git fsck --unreachable; git count-objects -v</code>	Another clone or a backup only	After a deliberate history rewrite in a repository you have copied first	3.17
<code>git repack --geometric=2 -d</code>	●	Rewrites packs	Nothing reachable	<code>git count-objects -v</code>	Not needed for reachable objects	What automatic maintenance does for you	26.4
<code>git reflog expire, git reflog delete <ref>@{<n>}, git reflog drop <ref></code> (drop: Git 2.50 or later)	●	Remove reflog entries	The record of where a ref used to point	<code>--dry-run --verbose</code>	<code>git fsck</code> , until a prune	Almost never by hand	13.4, 13.18
<code>git reflog expire --expire=now --all</code> , usually followed by <code>git gc --prune=now</code>	●	Empties every reflog, the stash list included; then deletes every unreachable object	Every recovery path in that clone: every commit, stash and staged blob that only a reflog, or nothing at all, was keeping	<code>git reflog expire --dry-run --expire=now --all; git fsck --unreachable --no-reflogs</code> lists what would go	None locally	The final step of a secret clean-up, in a cleanup clone, after rotation; in a stale clone after its owner has saved their work	3.17, 13.13, 21B.17, 30.23
<code>git refs migrate --ref-format=<format></code> (Git 2.46 or later)	●	The ref storage format; values are kept	Nothing	<code>--dry-run</code>	Migrate back, or restore the copy of <code>.git</code> you made first	A clone that hits a case-insensitive ref conflict; throwaway repositories	3.13, 14D.5
<code>git filter-repo ... git filter-branch ...</code>	●	Every affected commit and all descendants, on every rewritten ref	Signatures; with a wrong path argument, files you meant to keep	<code>git-filter-repo --dry-run</code> ; run in a fresh clone	The untouched server and other clones, until you push	Data that stays harmful after rotation; for an unpushed mistake on a private branch a rebase is the smaller tool	21B.17

9. Configuration, hooks and recorded resolutions

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git config set, unset, edit</code> (Git 2.46 or later), in any scope	●	One configuration file, which has no history	The previous value	<code>git config get --show-origin <key>;</code> <code>git config list --show-origin --show-scope</code>	Set the old value again; keep the global file under version control	Deliberate settings; the scope that should own the key	14B.3
<code>git config set --global init.default*, git config set --global safe.bareRepository explicit</code>	●	The behavior of later commands: new repositories; Git commands inside bare repositories then need <code>--git-dir</code> or <code>GIT_DIR</code>	Nothing	<code>git config list --show-origin</code>	Unset the keys	<code>init.defaultBranch</code> and <code>safe.bareRepository</code> now; the object and ref formats per repository, not globally	14D.4, 21B.23
<code>git config set --global --append safe.directory <dir></code>	●	Makes Git trust that directory's configuration and hooks	Nothing; removes a protection	Read <code><dir>/.git/config</code> and <code><dir>/.git/hooks</code> first	<code>git config unset --global --value=<dir> safe.directory</code>	A directory whose contents you have read	21B.23
<code>git config set core.hooksPath, hook, <name>.*, filter.*, diff.*, merge.*</code>	●	Which programs Git runs on your machine	Nothing	<code>git hook list --show-scope <event>;</code> <code>git config list --show-origin</code>	Unset the keys	Programs you have read	14C.11, 14C.12, 28.18
<code>git hook run <event>; --no-verify</code>	●	Runs the local checks; skips them	Nothing	<code>git hook list <event></code>	The server may still refuse	Testing a hook; a hook that is wrong	14C.11
<code>git rerere forget <path>, git rerere clear, git rerere gc</code>	●	Deletes recorded resolutions	The recorded resolution	<code>git rerere diff; ls .git/rr-cache</code>	Resolve again	A recorded resolution that is wrong	14C.3
<code>git lfs install</code> without <code>--local</code>	●	The filter keys go into your global configuration; hooks in the current repository	Nothing	<code>git config list --show-origin</code>	<code>git lfs uninstall</code>	In this book it is always run with <code>--local</code> (●)	22.3

10. Submodules, subtrees, LFS, sparse-checkout, worktrees

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git submodule update</code> (default checkout)	●	Moves the submodule's HEAD (detached) and its files; refuses to overwrite local edits to files it must change	Nothing; commits made on the detached HEAD lose their name	<code>git submodule status; git -C <path> status</code>	The submodule's reflog: <code>git -C <path> reflog; git -C <path> branch <name> <id></code>	After every pull that moved a gitlink	23.3, 23.5
<code>git submodule update --remote</code>	●	As above, to the tip of the remote-tracking branch; the superproject then shows a modified gitlink	Nothing	<code>git -C <path> fetch && git -C <path> log --oneline HEAD..origin/HEAD</code>	<code>git submodule update</code> returns to the recorded commit	Moving the pointer on purpose, in a dedicated branch	23.6
<code>git submodule update --force, git submodule deinit --force</code>	●	Overwrites modified tracked files in the submodule	Uncommitted changes inside the submodule	<code>git submodule foreach 'git status --short'</code>	None for uncommitted edits	To get back to exactly the recorded state	23.9, 23.16
<code>git submodule deinit <path></code>	●	Empties the working directory; removes the section from <code>.git/config</code> ; refuses if there are local modifications unless <code>--force</code>	Nothing	<code>git -C <path> status</code>	<code>git submodule update --init</code>	A submodule you do not need locally	23.11

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>git rm <submodule path></code>	●	Removes the gitlink, the <code>.gitmodules</code> section and the working directory	Nothing	<code>git status</code>	<code>git restore - --staged -- worktree .gitmodules <path></code> , then <code>git submodule update --init</code>	Removing the submodule from the project	23.11
Deleting <code>.git/modules/<name></code>	●	Deletes a repository	Commits in the submodule that were never pushed	<code>git -C <path> status; git -C <path> reflog</code>	None	Removing a submodule completely from your clone, after checking for unpushed commits	23.11, 23.16
<code>git push --recurse- submodules=on-demand</code>	●	Pushes in the submodules, then the superproject	As for any push	<code>git submodule foreach 'git status --short -- branch'</code>	As for any push	When the superproject commit needs submodule commits that are not published	23.8
<code>git subtree add, git subtree pull</code>	●	New commits on the current branch (a merge); needs a clean working tree	Nothing	<code>git ls-remote <repository> <ref></code>	<code>git reset -- hard ORIG_HEAD</code> if not pushed (● for uncommitted work); <code>git revert -m 1</code> if pushed	Vendoring a dependency's files into your own history	23.13
<code>git subtree push</code>	●	Creates or updates a branch in the library's repository	Nothing in yours	<code>git subtree split</code> first and inspect the result	Delete or reset that branch in the library's repository	Sending changes back upstream	23.13
<code>git lfs push</code> , and <code>git push</code> with the pre-push hook	●	Uploads LFS objects; on GitHub they are billed and cannot be deleted by you	Nothing	<code>git lfs push -- dry-run <remote> <ref>; git lfs status</code>	None on GitHub short of deleting the repository (22.11)	After deciding what belongs in LFS	22.6
<code>git lfs prune</code>	●	Deletes local LFS objects that are pushed and not needed	Nothing while the remote has the objects	<code>--dry-run -- verbose; --verify- remote</code>	<code>git lfs fetch</code> , if the remote has them	Reclaiming disk space	22.8
<code>git lfs migrate import, export</code> (default scope or named branches)	●	Rewrites unpushed commits; the working tree is left with pointers after import	Nothing locally	<code>git lfs migrate info</code> with the same options	Branch reflogs: <code>git reset -- hard <branch>@{1}</code>	Unpushed commits that contain large files	22.9
<code>git lfs migrate import --everything</code> , then a forced push	●	Rewrites published history	Nothing locally; breaks every other clone	<code>git lfs migrate info --everything</code> ; rehearse in a fresh clone	Before the push: reflogs. After: the old history from another clone	A coordinated rewrite	22.9, 22.13
<code>git sparse-checkout set, add, init --cone</code> (<code>init</code> is deprecated: <code>set</code> does everything)	●	Working tree, skip-worktree bits, pattern file, <code>config.worktree</code> ; deletes ignored files in directories that leave the cone	Ignored files in directories that leave the cone, such as a local <code>.env</code> or a downloaded evaluation set	<code>git status --short --ignored</code>	<code>set</code> again or <code>disable</code> ; ignored files are gone	After checking for ignored files you cannot regenerate	24.4, 24.5
<code>git sparse-checkout reapply, disable</code>	●	Removes, or writes, tracked files	Nothing tracked	<code>git ls-files -t</code>	<code>set</code> or <code>add</code>	After conflicts outside the cone	24.4
<code>git sparse-checkout set --sparse-index</code>	●	Rewrites the index in a form some tools cannot read	Nothing	Test the tools first	<code>--no-sparse- index</code>	Very large repositories	24.6
<code>git sparse-checkout clean -f</code> (Git 2.52 or later)	●	Deletes directories outside the cone that hold untracked files	Untracked files there, which were never in Git	<code>--dry-run, -- verbose</code>	None	After reading the preview and moving away what you need	24.5

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>scalar clone</code>	●	A clone, and also global configuration, a scheduler unless <code>--no-maintenance</code> , a monitor daemon	Nothing	<code>git help scalar</code>	<code>scalar unregister</code> or <code>scalar delete</code> ; <code>git fsmonitor--daemon stop</code>	A deliberate choice on your own machine	24.7
<code>git worktree move</code>	●	The directory and both pointers	Nothing	<code>git worktree list</code>	Move it back	Reorganising directories	25.6
<code>git worktree remove</code>	●	Deletes a clean working tree and its entry, including its HEAD reflog	The HEAD reflog of that worktree; the name of detached commits made there	<code>git -C <path> status --short</code> ; <code>git -C <path> log --oneline -3</code>	<code>git worktree add</code> again; detached commits via <code>git fsck --lost-found</code>	A finished task; create a branch before removing	25.6
<code>git worktree remove --force</code>	●	As above, and deletes uncommitted and untracked files	Files that were never committed	<code>git -C <path> status --short --ignored</code>	None for files that were never committed	Only after looking	25.6, 25.9
<code>git worktree prune</code>	●	Deletes entries whose working tree is missing, including their HEAD reflogs	Those HEAD reflogs	<code>git worktree prune --dry-run --verbose</code>	<code>git worktree add</code> again; a moved tree: <code>git worktree repair</code> before pruning	After a working tree was deleted with <code>rm -rf</code>	25.6
<code>git worktree add --force</code> for a branch that is already checked out	●	Two indexes on one ref	The consistency of the stale worktree: staged changes nobody made	None; do not use	<code>git reset --hard</code> in the stale worktree after saving its work	Never	25.4, 25.9
<code>python3 tools/dataref.py checkout</code> (and <code>dvc checkout</code>)	● for unrecorded data edits	Overwrites data files with the versions the pointers name	Unrecorded edits to data files	<code>python3 tools/dataref.py verify</code>	None for unrecorded edits; record them first with <code>add</code>	After <code>verify</code> reports "ok"	28.18

11. GitHub: the GitHub CLI, authentication, rules and Actions

GitHub, not Git. Every ● command here changes GitHub objects that no clone contains (Chapter 15, section 15.22). The preview is always a read with the same tool. Flags were checked with `gh <command> --help` of version 2.88.1; nothing was run against GitHub. The full command tables are in the GitHub CLI cheat sheet.

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>gh repo create --source=.</code> , <code>--push</code>	●	A repository on GitHub, a remote, a push	Nothing; what reached a public repository is published	<code>git log</code> , <code>git status</code> ; <code>check -public</code> or <code>--private</code>	<code>gh repo delete</code>	After reading what the push will publish	15.22
<code>gh issue create</code> , <code>gh pr create</code> , <code>gh label create</code> , <code>gh release create --verify-tag --draft</code>	●	GitHub objects that notify people; <code>gh pr create</code> may push the branch	Nothing; notifications are not recalled	<code>gh pr create --dry-run</code> ; <code>--draft</code>	Close or delete; <code>gh pr close</code>	Ordinary collaboration	15.22, 17.21
<code>gh repo edit</code> (features, merge methods), <code>gh repo fork</code> , <code>gh repo sync</code>	●	Settings; a new repository; a fast-forward	Nothing	<code>gh repo view --json</code> ; <code>git log HEAD..upstream/main</code>	Set the previous value; delete the fork	Deliberate changes	15.22

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>gh repo sync --force</code>	●	Hard reset of the destination branch	Commits on the destination branch that the source lacks	<code>git log upstream/main..origin/main</code> after a fetch; <code>git rev-list --left-right --count upstream/main...origin/main</code>	Push the old commits back from a clone that still has them	A fork branch that holds nothing of its own	15.7, 17.21, 27.21
<code>gh api</code> with POST, PATCH, PUT, DELETE	●	Whatever the endpoint says, with all your permissions	Depends on the endpoint	The GET first	Depends on the endpoint; often none	After the GET, with the saved JSON at hand	15.22
<code>gh release create TAG</code> without <code>--verify-tag</code>	●	May create a tag on the default branch and start tag workflows	Nothing existing; publishes a version name	<code>git ls-remote --tags origin TAG</code>	Delete release and tag; consumers may have fetched it	Use <code>--verify-tag</code>	15.22
<code>gh release delete --cleanup-tag, git push origin --delete TAG</code>	●	A published version's name	The release and its tag	<code>gh release view TAG</code>	Re-create from the recorded commit ID	A release published by mistake	15.22
<code>gh repo edit --visibility, gh repo delete, gh repo rename, gh repo archive</code>	●	Visibility with its side effects; the repository, its URL, its writability	The side effects of a visibility change cannot be undone	Section 15.5; ask who depends on the URL	Visibility can be changed back, its side effects cannot; restore within 90 days unless the fork network is not empty	Planned, announced changes	15.5, 15.22
<code>gh pr update-branch, gh pr edit --base, gh pr merge</code>	●	Move the head branch, change the base, or write to the base branch; can dismiss approvals	Nothing	<code>gh pr view, gh pr checks --required</code>	Change the base back; <code>gh pr revert</code>	The normal end of a reviewed pull request	17.21
<code>gh pr merge --admin, a bypass merge</code>	●	Writes to a protected branch without the required reviews or checks	Nothing in Git; the control	<code>gh pr checks --required</code>	<code>gh pr revert</code> ; record why	A planned migration or a declared emergency, by a named person, with the reason written down where the audit will find it	17.16, 18.21
<code>gh api --method POST repos/ORG/REPO/rulesets --input file.json</code>	●	Creates a ruleset; if Active, it binds everyone at once, you included	Nothing	Create it with "enforcement": "disabled", then <code>gh ruleset check</code>	Set <code>enforcement</code> to <code>disabled</code> , or delete the ruleset	Rules as reviewed JSON	18.21
<code>gh api --method PUT .../rulesets/ID</code>	●	Replaces settings of a ruleset	The previous settings	<code>gh api .../rulesets/ID</code> first and keep the JSON	PUT the saved JSON back	Changing a rule	18.21
<code>gh api --method DELETE .../rulesets/ID</code>	●	Removes the protection for every ref it targeted, immediately	Nothing directly; it makes force pushes and deletions possible	Save the JSON first	Re-create from the saved JSON; ruleset history exists only on Enterprise	A planned migration or a declared emergency	18.21
<code>git push --force</code> to a shared base branch, or to a branch whose protection was just removed	●	Removes commits from the remote branch; corrupts open pull requests	Commits others pushed	<code>git log <branch>..origin/<branch></code> and the reverse	Chapter 13; Chapter 30	Not appropriate	17.21, 18.21
Editing <code>CODEOWNERS</code> in a pull request	●	Who is requested, and under a rule who can block, from the next merge on	Nothing	The errors endpoint with <code>ref</code>	Revert the commit	With review by the current owners	19.14

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
Removing the owner of <code>/.github/</code>	●	Lets any writer change reviewers and workflows without owner review	No data; removes a control	Review the diff of the file	Restore the line; audit what merged in between	Never without a replacement owner	19.14
<code>gh workflow run <file> --ref <branch></code>	●	Starts a run, which uses minutes and may deploy if the workflow deploys	Nothing in Git	Read the workflow file at that ref: <code>gh workflow view --yaml</code>	<code>gh run cancel <run-id></code> ; deploy the previous commit	A <code>workflow_dispatch</code> workflow you have read	20A.18, 20B.18
<code>gh run rerun <run-id>, --failed</code>	●	Starts a new attempt on the original commit	Nothing	<code>gh run view RUN_ID --json headSha</code> and compare with <code>main</code>	Cancel the attempt; start a new run from the current commit	A flaky failure, not a stale commit	20A.18, 20B.18
<code>gh run cancel</code>	●	Stops a run, possibly mid-deployment	The consistency of a half-finished deployment	<code>gh run view</code>	Re-run; check the target's state by hand	A run that should not finish	20B.18
<code>gh secret set, gh variable set</code>	●	Overwrites the stored value	The old secret value, which cannot be read back	<code>gh secret list --env NAME</code>	Set the previous value again from your secret store	Rotation	20B.18
<code>gh api -X PUT .../environments/NAME</code>	●	Replaces the environment's protection settings with the body sent	The previous settings	<code>gh api .../environments/NAME</code>	Send the previous configuration again	Environment rules as code	20B.18
<code>gh cache delete --all</code>	●	Removes every cache of the repository	Nothing permanent	<code>gh cache list</code>	Caches are rebuilt by the next runs, slowly	A poisoned or oversized cache	20B.18
<code>git tag -f -a v1</code> , then <code>git push --force origin v1</code>	●, then ●	Moves a local tag; replaces a published tag that workflows may name	What uses: <code>...@v1</code> meant yesterday	<code>git rev-parse 'v1^{commit}'; git ls-remote --tags origin v1</code>	Re-create the tag at the old commit; push the old tag object back, if you still have it	The reason to pin actions by full commit ID: see the action pins	21A.7
Pushing a change under <code>.github/workflows/</code>	●	Git: an ordinary commit. GitHub Actions: the next matching event runs the new file with the repository's token and secrets	Nothing in Git	Review the diff with section 21A.19	Revert the commit; rotate any secret the changed workflow could read	After review by the owner of <code>/.github/</code>	21A.22
<code>ssh-keygen -t ed25519 -f NEW</code>	●	Creates two files; overwrites an existing key only after asking	An overwritten private key	<code>ls ~/.ssh</code>	None for an overwritten private key	A new key under a new file name	16.23
<code>ssh-add, ssh-add --apple-use-keychain</code>	●	The agent's key list; a keychain entry	Nothing	<code>ssh-add -l</code>	<code>ssh-add -d</code>	Loading the key you mean	16.23
<code>ssh-keygen -R HOST</code>	●	Removes a host's lines from <code>known_hosts</code> , keeping a <code>.old</code> copy	Nothing	<code>ssh-keygen -F HOST</code>	The <code>.old</code> file	After comparing the published fingerprint	16.11, 16.23

Command	Label	What it changes	What it can destroy	Preview	Recover	When appropriate	Section
<code>StrictHostKeyChecking no</code> , or deleting <code>known_hosts</code> wholesale	●	Removes the host key check	The protection against a wrong host	None	Install the published lines instead	Never; on a CI runner install the published lines	16.11
<code>git config set --global credential...</code> , <code>gh auth setup-git</code> , <code>gh auth login</code> , <code>gh auth switch</code>	●	Which credential answers, for every repository	Nothing	<code>git config get --show-origin --all credential.helper</code>	Set the previous value; <code>git config unset</code>	Deliberate changes of identity	16.23
<code>git credential reject</code> , <code>git credential-osxkeychain erase</code>	●	Deletes a stored credential	The stored credential	<code>git credential fill</code> shows what is stored	Log in again	A stale or wrong stored credential	16.23
<code>gh auth token</code> , <code>gh auth status --show-token</code>	●	Prints a live token to the terminal and its scrollbar	The secrecy of the token	Ask whether you need the value at all	Revoke the token	Rarely; never in a shared or recorded terminal	16.23
<code>gh ssh-key delete</code> , deleting a token or a deploy key in the web interface	●	Every machine and job that used it stops	Access for those machines and jobs	List where it is used	Create and distribute a new one	Rotation, with the list of users at hand	16.23
Writing a token into a URL, a file or a command line	●	Publishes the secret to logs, history and backups	The secrecy of the token	None	Revoke it; cleaning is not enough	Never	16.23

For the response to a leaked secret, the order of the steps matters more than any single command: see the security cheat sheet and Chapter 21B. For recovery after any ● command, see the emergency recovery page and the disaster-recovery playbook.

12. Where chapters label the same command differently

Earlier drafts of the chapters labelled fifteen commands in two ways, because each chapter judged the worst case in its own context. The editorial pass of 6 October 2026 chose one label per command from the definitions in section 1, and the chapters, this file, the command reference, the cheat sheets and the playbooks now agree. The table records each decision. Git behavior named in the last column was checked on Git 2.55.0; the `gh` rows were checked against `gh <command> --help` of 2.88.1 and not run against GitHub.

One test settled most rows: a command is ● only if no ref moves and nothing is lost. A command that adds commits to the current branch through the merge machinery moves that branch and rewrites index and working tree, which is the reason `git merge` is ● in 8.21; a command that overwrites a setting loses the previous value, because a configuration file keeps no history.

Command	Label now	Sections changed (label before)	Reason
<code>git cherry-pick <commit></code> , <code>git cherry-pick --continue</code>	●	2.10, 2.15, 10.2, 10.14, 27.9, 27.21 (●)	It moves the current branch and rewrites index and working tree, as <code>git merge</code> does; it adds commits and removes none
<code>git revert <commit></code> , <code>git revert -m 1 <merge></code>	●	10.11, 10.14, 11.8, 11.16, 27.15, 27.21 (●)	As above; and after <code>-m 1</code> a later merge of the same branch is silently empty
<code>git merge --abort</code>	●	29.14, the two playbooks, the emergency page (●)	It discards every resolution made so far and every edit staged during the merge, and the manual warns that uncommitted changes present when the merge started cannot always be reconstructed. <code>git rebase --abort</code> , <code>git cherry-pick --abort</code> and <code>git revert --abort</code> stay ● in their own rows
<code>git push --force-with-lease</code> <code>--force-if-includes</code>	●	1.8 (●, as an example)	Every forced push replaces history on a remote, where you have no reflog (9.17, 12.8). The conditions make it the right form of a ● command, not a ● one
<code>git push --force-with-lease</code>	●	17.21 (●)	As above; alone, the check passes wrongly after a background fetch
<code>git update-ref -d <ref></code>	●	3.17 (●)	It deletes the ref together with its reflog, which is the safety net for that ref
<code>git repack -a -d</code>	●	26.18 (●)	It drops the unreachable objects of the packs it replaces, at once and without the two-week grace period. <code>git repack --geometric=2 -d</code> stays ●
<code>rm .git/index</code> , then <code>git reset</code>	●	5.18 (●)	It removes the index outside Git's locking and discards the record of what was staged, the flag bits and the conflict stages; no reflog

Command	Label now	Sections changed (label before)	Reason
			holds them. Moving the file aside first (<code>mv .git/index .git/index.corrupt</code> , 13.11) is  , because the file can be moved back
<code>git config set <key></code> , <code>unset</code> , <code>edit</code> , in any scope		14A.17, 14A.27 (<code>blame.ignoreRevsFile</code> , <code>diff.algorithm</code> , <code>blame.markIgnoredLines</code>), 14D.4 (<code>init.default*</code>), 21B.23 (<code>safe.bareRepository</code>) ()	It rewrites a file that keeps no history, and it changes the behavior of later commands. This is the label of 1.15 and 14B.21
<code>git lfs checkout</code>		22.7 ()	It replaces unmodified pointer files with their content from the local store and never overwrites a modified file: no ref moves and nothing is lost. <code>git lfs pull</code> is <code>fetch</code> plus <code>checkout</code> and carries the same label (22.14). The row this file had for the command is removed, because  commands are not listed here
<code>git subtree add</code>		23.13 ()	It creates a merge on the current branch
<code>gh pr create</code>		17.2, 17.21 ()	It may push the branch, and it creates a GitHub object that notifies people
<code>gh api</code> with <code>POST</code> , <code>PATCH</code> , <code>PUT</code> or <code>DELETE</code> , including the ruleset calls of 18.21 and the environment call of 20B.18		18.21 (<code>POST</code> and <code>PUT</code> on rulesets), 20B.18 (<code>PUT</code> on an environment) ()	The call does whatever the endpoint and the body say, with all your permissions (15.22). A <code>PUT</code> replaces settings that exist nowhere else unless you saved the JSON first; the <code>POST</code> that creates a ruleset destroys nothing and keeps the label of the command, with that stated in its row

Three labels of the GitHub CLI cheat sheet that no chapter table carried were confirmed in the same pass:

Command	Label	Reason
<code>gh ssh-key add</code>		It attaches a credential to your account that can act as you until it is deleted; <code>gh ssh-key delete</code> undoes it (16.8)
<code>gh repo set-default <repository></code>		It records a setting in the clone that decides which repository later <code>gh</code> commands act on; <code>--view</code> is  and <code>--unset</code> undoes it (15.16)
<code>gh api graphql</code>	 with a <code>query</code> ,  with a <code>mutation</code>	A GraphQL <code>query</code> only reads, although the request is sent as a <code>POST</code> ; a <code>mutation</code> writes, and falls under the rule of 15.22

13. Chapter files

The section numbers in the tables point into these files.

Sections	File
1.x	Chapter 1: Fundamentals
2.x	Chapter 2: The Mental Model
3.x	Chapter 3: Git Internals
4.x	Chapter 4: The Working Tree
5.x	Chapter 5: The Index
6.x	Chapter 6: Commits
7.x	Chapter 7: Branches
8.x	Chapter 8: Merge
9.x	Chapter 9: Rebase
10.x	Chapter 10: Cherry-pick
11.x	Chapter 11: Reset, Revert, Restore
12.x	Chapter 12: Remote Operations
13.x	Chapter 13: Recovery
14A.x	Chapter 14A: History investigation
14B.x	Chapter 14B: Configuration, Aliases, Tags and Signing
14C.x	Chapter 14C: Stash Internals, Rerere, Attributes, Hooks
14D.x	Chapter 14D: The Frontier
15.x	Chapter 15: GitHub
16.x	Chapter 16: Authentication

Sections	File
17.x	Chapter 17: Pull Requests
18.x	Chapter 18: Branch Protection and Rulesets
19.x	Chapter 19: CODEOWNERS
20A.x	Chapter 20A: GitHub Actions Fundamentals
20B.x	Chapter 20B: GitHub Actions: delivery, runners, cost and debugging
21A.x	Chapter 21A: GitHub Actions security
21B.x	Chapter 21B: Repository security, identity, the Git client, and secret-leak response
22.x	Chapter 22: Git LFS
23.x	Chapter 23: Submodules and Subtrees
24.x	Chapter 24: Monorepos
25.x	Chapter 25: Worktrees
26.x	Chapter 26: Performance
27.x	Chapter 27: Open source and team workflows
28.x	Chapter 28: AI/ML workflows
29.x	Chapter 29: Production Troubleshooting
30.x	Chapter 30: Incident Response

Chapter 34: Glossary

Source file: `reference/glossary.md`

Glossary

Baseline. Git 2.55.0; GitHub terms as of 1 October 2026. 293 terms, each defined in one to three sentences in the textbook's own words and checked against `git help glossary` where Git defines the term. The last column is where the term is taught.

Use it in three ways. Look a word up when a chapter, an error message or a colleague uses it. Read one letter before a mastery gate and say each definition aloud before you read it. When two definitions seem to say the same thing (a branch and a tag, an upstream and a remote-tracking branch, a release and a tag), read both: the difference is usually the lesson.

The **Layer** column says whose term it is, because GitHub is not Git:

Layer	Meaning
Git	A term of Git itself: it means the same against any server
GitHub	A term of the GitHub platform, not of Git: no clone contains the thing it names
Actions	A term of GitHub Actions
LFS	A term of Git LFS, a separate program
Course	A term of practice or of this course's method, defined in the chapter named

Terminology follows the textbook: commit ID or object ID (not "SHA" alone), the index (also called the staging area), working tree (not working directory), remote-tracking branch, upstream branch, HEAD in capitals.

Symbols and notation

Term	Layer	Definition	Taught in
<code>A...B</code> (three dots)	Git	In <code>git log</code> : commits reachable from either side and not from both. In <code>git diff</code> : the diff from the merge base of A and B to B, which is what a pull request's "Files changed" tab shows.	Chapter 14A, section 14A.2
<code>A..B</code> (two dots, in <code>git log</code>)	Git	The set of commits reachable from B and not from A. In <code>git diff</code> the same spelling compares the two tips, which is a different operation.	Chapter 14A, section 14A.8
<code>--</code> (double dash)	Git	Separates revisions and options from paths on a command line: everything after it is a path.	Chapter 14A, section 14A.7
<code>--force-if-includes</code>	Git	A condition on a forced push (Git 2.30): the server's tip must be reachable from a reflog entry of your branch, that is, you integrated what you are about to overwrite.	Chapter 12, section 12.8
<code>--force-with-lease</code>	Git	A forced push that succeeds only if the server's ref still has the expected value: by default your remote-tracking ref, or a value you name.	Chapter 12, section 12.8
<code>HEAD~<n></code> , <code><rev>^<n></code>	Git	<code>~<n></code> follows the first parent <i>n</i> times; <code>^<n></code> selects the <i>n</i> -th parent of a merge commit.	Chapter 14A, section 14A.7
<code>+</code> in a refspec	Git	Allows a non-fast-forward update of the destination ref: a forced update for that refspec.	Chapter 12, section 12.12
<code>:<n>:<path></code>	Git	The index entry of a path at stage <i>n</i> : 1 the base, 2 "ours", 3 "theirs" during a conflict; stage 0 is the normal entry.	Chapter 5, section 5.13
<code><ref>@{<n>}</code> , <code><ref>@{<time>}</code>	Git	The value a ref had <i>n</i> changes ago, or at a time, read from its reflog. It exists only in the clone whose reflog recorded it.	Chapter 13, section 13.3
<code><rev>:<path></code>	Git	The blob or tree at a path in the tree of a revision, for example <code>HEAD~1:config/limits.yaml</code> .	Chapter 3, section 3.6
<code><rev>^{commit}</code> , <code><rev>^{tree}</code>	Git	Peels a name to the object of the given type, for example from an annotated tag to its commit, or from a commit to its tree.	Chapter 3, section 3.6
<code>--update-refs</code>	Git	A rebase option (Git 2.38) that moves every local branch pointing at one of the replayed commits, not only the branch you are on.	Chapter 9, section 9.9
<code>@{u}</code> , <code>@{upstream}</code>	Git	The upstream branch of the current branch; <code>@{push}</code> is where a push would go.	Chapter 12, section 12.5

A

Term	Layer	Definition	Taught in
action	Actions	A packaged, reusable step that a workflow calls with <code>uses: owner/repo@ref</code> ; it downloads and runs someone else's code inside your job.	Chapter 20A, section 20A.2
<code>actions/checkout</code>	Actions	The action that fetches the repository onto the runner: by default one commit, without tags, for the ref of the event, with a detached HEAD on <code>pull_request</code> events.	Chapter 20A, section 20A.8
Activity view	GitHub	A page of a repository that lists pushes, force pushes, merges, branch creations and deletions, with the user and a comparison. Described from the documentation.	Chapter 13, section 13.15
ahead, behind	Git	Counts of commits that one branch has and the other lacks, counted from their merge base; <code>git status -sb</code> shows them for a branch and its upstream.	Chapter 7, section 7.8
alias	Git	A configuration value under <code>alias.<name></code> that Git substitutes for <code><name></code> when it is the first word of a command line, appending any further arguments. A value that starts with <code>!</code> is a shell command.	Chapter 14B, section 14B.6
allowed-signers file	Git	The file named by <code>gpg.ssh.allowedSignersFile</code> that maps identities to SSH public keys; without it Git cannot verify SSH signatures.	Chapter 14B, section 14B.16
amend	Git	<code>git commit --amend</code> writes a new commit that takes the place of the current one: same parent, new tree and message as you choose, new ID.	Chapter 6, section 6.7
annotated tag	Git	A tag ref that points at a tag object, which names the tagged object and records who tagged it, when and why. <code>git describe</code> uses annotated tags by default.	Chapter 14B, section 14B.8
approval	GitHub	A review state attached to the diff as it was when the reviewer approved; settings decide whether a later push dismisses it.	Chapter 17, section 17.5
artifact	Actions	A set of files that a job uploads to GitHub, stored with the run, downloadable by later jobs or by people, and expiring after a retention period.	Chapter 20A, section 20A.12
assume-unchanged bit	Git	A flag on an index entry that tells Git not to examine the working tree file. It is a performance promise, not a way to keep private edits to a tracked file.	Chapter 5, section 5.12
attribute	Git	A named property attached to paths by a <code>.gitattributes</code> file, consulted whenever Git converts, compares, merges or archives a file.	Chapter 14C, section 14C.4
audit log	GitHub	An organization's record of events; the organization audit log covers 180 days. Git events such as pushes are restricted as Chapter 21B, section 21B.8 describes.	Chapter 15, section 15.19
authentication	GitHub	Establishing which account a connection belongs to. Authorization, a separate step, decides what that account may do to this repository.	Chapter 16, section 16.2
author	Git	The person who wrote the change, recorded in a commit with a name, an email address, a time and a time-zone offset. It is text supplied by whoever creates the commit.	Chapter 6, section 6.5
auto-merge	GitHub	A stored instruction on a pull request: merge with this method as soon as every requirement is met.	Chapter 17, section 17.10
<code>AUTO_MERGE</code>	Git	A tree written during a conflicted merge that holds the automatic result with conflict markers; <code>git diff AUTO_MERGE</code> shows what your resolution changed.	Chapter 8, section 8.8
autosquash	Git	<code>git rebase --autosquash</code> reads the subjects of <code>fixup!</code> , <code>squash!</code> and <code>amend!</code> commits and arranges the todo list so that each lands on its target.	Chapter 9, section 9.7
autostash	Git	<code>--autostash</code> puts uncommitted changes into a stash commit before a rebase or merge and applies it to the result; the final apply can conflict.	Chapter 9, section 9.8

B

Term	Layer	Definition	Taught in
backport	Git	A fix that lands on the development line first and is then copied to a maintenance line with <code>git cherry-pick -x</code> , so that the copy says where it came from.	Chapter 10, section 10.9
bare repository	Git	A repository with no working tree, whose directory is itself the Git directory; the server in every demo is one.	Chapter 12, section 12.2
base branch	GitHub	The branch a pull request asks to merge into; the head branch holds the commits offered.	Chapter 17, section 17.2
bisect	Git	<code>git bisect</code> finds the commit that changed a property by checking out a commit in the middle of the suspects, asking for a verdict, and halving the suspects until one is left.	Chapter 14A, section 14A.20
bitmap	Git	A file that stores, for selected commits, one bit per object in the pack saying whether that commit reaches it, so that a server can answer which objects a client needs without walking the graph.	Chapter 26, section 26.8
blame	Git	<code>git blame</code> annotates every line of a file, as it is in one revision, with the commit that last changed that line. It names the last change, not the cause.	Chapter 14A, section 14A.16
blob	Git	The object type that holds the bytes of one file, and nothing else: no name, no mode.	Chapter 3, section 3.4

Term	Layer	Definition	Taught in
blobless clone	Git	A partial clone made with <code>---filter=blob:none</code> : all commits and trees, file contents fetched when a command needs them.	Chapter 26, section 26.12
branch	Git	A ref under <code>refs/heads/</code> that holds the ID of one commit, the tip; it moves forward when you commit on it. The branch's history is whatever that commit reaches through its parents.	Chapter 7, section 7.2
branch protection rule (classic)	GitHub	The older rule mechanism: one rule per branch-name pattern, visible to administrators, with its own bypass behavior, still supported and still evaluated alongside rulesets.	Chapter 18, section 18.14
bundle	Git	A file that holds a pack and the refs that go with it; an incremental bundle names commits the receiver must already have.	Chapter 26, section 26.14
bypass list	GitHub	The actors a ruleset excepts. A ruleset applies to everyone, administrators included, except the actors on its bypass list.	Chapter 18, section 18.5

C

Term	Layer	Definition	Taught in
cache (Actions)	Actions	A directory stored under a key so that a later run with the same key can restore it; an optimization that may be absent, and untrusted input for a release job.	Chapter 20A, section 20A.11
cache (Git)	Git	An old name for the index, still visible in options such as <code>---cached</code> . See index.	—
case-insensitive filesystem trap	Git	Git treats <code>Config.py</code> and <code>config.py</code> as two paths while the default macOS filesystem treats them as one file; <code>core.ignoreCase</code> reconciles the two and hides case-only renames.	Chapter 4, section 4.12
check, status check	GitHub	A result reported for a commit by a workflow or an external service. A required status check is a rule that holds a list of check names which must pass on the newest commit.	Chapter 17, section 17.6
checkout	Git	Writing a commit's files into the working tree and the index. <code>git checkout</code> is the older command that both switched branches and restored files; <code>git switch</code> and <code>git restore</code> (Git 2.23) do the two jobs separately.	Chapter 7, section 7.6
cherry-pick	Git	<code>git cherry-pick <commit></code> takes the difference between a commit and its parent, merges it into the current branch with a three-way merge, and commits the result as a new commit with a new ID.	Chapter 10, section 10.2
<code>CHERRY_PICK_HEAD</code>	Git	The root ref that names the commit being picked while a cherry-pick is stopped; a sequence of several picks also has <code>.git/sequencer/</code> .	Chapter 10, section 10.6
clean filter	Git	The command of a filter driver that rewrites a file's content on its way from the working tree into the repository, at <code>git add</code> .	Chapter 14C, section 14C.8
clone	Git	A new repository created by <code>git clone</code> : the source registered as the remote <code>origin</code> , everything the default refspec covers fetched, and one branch checked out.	Chapter 12, section 12.3
code owner	GitHub	A user or team that a <code>CODEOWNERS</code> line assigns to a path pattern; a suggested reviewer unless a rule requires the owner's approval.	Chapter 19, section 19.6
code scanning	GitHub	Analysis of code for vulnerabilities and coding errors with CodeQL or third-party tools that upload SARIF; free on public repositories, licensed on private ones as section 21B.13 states.	Chapter 21B, section 21B.13
<code>CODEOWNERS</code>	GitHub	A text file in the repository that maps path patterns to users and teams, which GitHub uses to request reviews automatically and, if a rule says so, to require them. It is read from the base branch of the pull request.	Chapter 19, section 19.2
commit	Git	An immutable object that names one complete snapshot of the project (a tree), the commit or commits it was built on, who wrote the change and when, who created the commit and when, and a message. It records a state, not a change.	Chapter 6, section 6.2
commit graph (history)	Git	The graph formed because every commit names its parents; "the history of X" is everything reachable from X by following those links. Not the same thing as the commit-graph file.	Chapter 2, section 2.7
commit ID	Git	The hash of the commit object; one ID names the snapshot, the metadata and, through the parent IDs, the whole history behind the commit.	Chapter 6, section 6.4
commit-graph	Git	A derived file that stores, for every commit, its parents, root tree, date and generation, and optionally a summary of changed paths, so that history walks stop decompressing commit objects.	Chapter 26, section 26.6
committer	Git	The person who created this commit object, recorded with name, email, time and offset; it differs from the author after a rebase, a cherry-pick or an amend by someone else.	Chapter 6, section 6.5
community health files	GitHub	Tracked files with well-known names (such as <code>CONTRIBUTING.md</code> or <code>SECURITY.md</code>) that GitHub finds and uses to guide the people who interact with a repository.	Chapter 15, section 15.13
composite action	Actions	An action defined in <code>action.yml</code> with <code>runs.using: "composite"</code> that replaces several steps inside a job and runs on the caller's runner.	Chapter 20B, section 20B.6
concurrency group	Actions	A name; GitHub lets at most one run or job with that name execute at a time and decides what happens to the others.	Chapter 20B, section 20B.5
conditional include	Git	<code>includeIf.<condition>.path</code> : reads another configuration file only when a condition about the current repository is true, for example <code>gitdir:~/work/</code> .	Chapter 14B, section 14B.4

Term	Layer	Definition	Taught in
cone mode	Git	The default mode of sparse-checkout, in which you name directories and Git includes them recursively plus the files of their parent directories.	Chapter 24, section 24.4
configuration scope	Git	One of the places a setting is read from: system, global, local, worktree, and the command line; when a key appears more than once the value read last wins.	Chapter 14B, section 14B.2
conflict	Git	The state in which a merge cannot decide the content of a path: both sides changed the same lines, or lines with no unchanged line between them, and did not make the identical change.	Chapter 8, section 8.7
conflict markers	Git	The <<<<<<, ===== and >>>>>> lines that Git writes into a working tree file to delimit the competing versions of a conflicting region.	Chapter 8, section 8.8
conflict style	Git	How much of the three versions is written between the markers: merge shows ours and theirs, diff3 adds the base, zdiff3 (Git 2.35) adds the base and moves shared edge lines out of the block.	Chapter 8, section 8.9
content addressing	Git	Naming an object by a hash of its type and content, so that identical content has the identical ID in every repository.	Chapter 2, section 2.4
context	Actions	A named object of facts about a run (github, matrix, steps, secrets and others), read in expressions.	Chapter 20A, section 20A.5
credential helper	Git	A program named by credential.helper that stores and returns the username and token for HTTPS remotes; Git asks each configured helper in turn before prompting.	Chapter 16, section 16.4
criss-cross merge	Git	A history in which two branches have each merged the other, so that they have two best common ancestors; Git merges those ancestors first and uses the result as the base.	Chapter 8, section 8.5
cruft pack	Git	A pack that holds only unreachable objects, with a companion file recording when each was last touched, so that garbage can wait out its grace period.	Chapter 26, section 26.5

D

Term	Layer	Definition	Taught in
dangling object	Git	An unreachable object that no other object refers to: the tip of lost history, or a blob that lost its index entry. git fsck reports it as information, not as damage.	Chapter 13, section 13.6
default branch	GitHub	The branch a repository presents first and that clones check out; several workflow events use the workflow file on it. In Git, the name of the first branch comes from init.defaultBranch.	Chapter 15, section 15.5
delta	Git	The storage of an object inside a pack as a difference from a similar object. It is a compression detail: a commit still records a full snapshot.	Chapter 3, section 3.7
Dependabot	GitHub	Three features that share a name: alerts about vulnerable dependencies, security updates, and version updates.	Chapter 21B, section 21B.13
deploy key	GitHub	An SSH key attached to one repository, not to a person; read-only unless write access is allowed, with no expiry.	Chapter 16, section 16.14
detached HEAD	Git	The state in which HEAD holds a commit ID instead of a branch name; Git works normally, and new commits advance HEAD itself rather than any branch.	Chapter 7, section 7.7
diff driver	Git	A configuration section, selected by the diff attribute, that tells Git how to show changes to a path; textconv converts a file to text before diffing.	Chapter 14C, section 14C.6
directory (in Git)	Git	Not tracked as such: a directory exists in a commit only because a file path passes through it.	Chapter 4, section 4.11
distributed version control	Git	A system in which every developer's copy contains the history, and "the server" is one more copy that the team agrees to treat as the meeting point.	Chapter 1, section 1.3
divergence	Git	Two branches have diverged when neither tip is an ancestor of the other.	Chapter 7, section 7.8
draft pull request	GitHub	A pull request state that signals work in progress before "ready for review"; a state in GitHub's data, not in Git.	Chapter 17, section 17.4
DVC	Course	One of the data-versioning tools of Chapter 28: Git holds a small file that names the data by checksum and another store holds the bytes. Not installed in the lab; its commands are given from its documentation.	Chapter 28, section 28.6

E

Term	Layer	Definition	Taught in
empty commit	Git	A commit that has the same tree as its parent: a point in history with a message and no change.	Chapter 6, section 6.8
enforcement status	GitHub	Whether a ruleset is enforced. The documentation for Free, Pro and Team lists Active and Disabled; the Enterprise Cloud documentation adds Evaluate.	Chapter 18, section 18.3

Term	Layer	Definition	Taught in
enterprise	GitHub	A container above organizations in GitHub's account model; several features named in the chapters are gated to Enterprise Cloud.	Chapter 15, section 15.3
environment	Actions	A named GitHub object that a job can reference, holding protection rules the job must pass before it starts and secrets and variables it can read only after that.	Chapter 20B, section 20B.2
event	Actions	What starts a workflow run (<code>push</code> , <code>pull_request</code> , <code>schedule</code> , <code>workflow_dispatch</code> and others); each event fixes which commit and ref the run is about and which copy of the workflow file is used.	Chapter 20A, section 20A.4
expression	Actions	A formula written <code>\${{ ... }}</code> over contexts that GitHub Actions replaces with its value before the step starts; with attacker-controlled values this is script injection.	Chapter 20A, section 20A.5

F

Term	Layer	Definition	Taught in
fast-forward	Git	A merge in which the current commit is an ancestor of the commit merged: Git moves the branch ref to that commit and creates no new object.	Chapter 8, section 8.3
feature flag	Course	A condition in the code that keeps unfinished or unreleased behavior switched off, so that the code can be merged long before the feature is released.	Chapter 27, section 27.11
fetch	Git	<code>git fetch</code> asks a remote where its refs point, downloads the objects you lack, and moves your remote-tracking refs; it never moves a local branch, the index or the working tree.	Chapter 12, section 12.4
<code>FETCH_HEAD</code>	Git	A pseudoref written by every fetch that lists what was fetched; one of the two names the glossary still calls pseudorefs.	Chapter 3, section 3.10
file mode	Git	What Git records about a file besides content: its type (regular, symbolic link, gitlink) and whether it is executable.	Chapter 4, section 4.10
file-system monitor	Git	A daemon (<code>core.fsmonitor</code>) that tells Git which paths changed since the last command, replacing the scan of the working tree by a question.	Chapter 26, section 26.9
filter driver	Git	A pair of commands selected by the <code>filter</code> attribute: <code>clean</code> on the way into the repository, <code>smudge</code> on the way into the working tree. Git LFS is one.	Chapter 14C, section 14C.8
fine-grained personal access token	GitHub	A token (prefix <code>github_pat_</code>) limited to one owner, optionally to selected repositories, with per-permission read or write.	Chapter 16, section 16.6
first-parent history	Git	The history obtained by following only the first parent of each merge commit: the commits the receiving branch itself went through.	Chapter 8, section 8.13
fixup commit	Git	A correction recorded as a commit of its own whose subject (<code>fixup! <subject></code>) names the earlier commit it belongs to.	Chapter 9, section 9.7
force push	Git	A push that sets the server's ref to a commit that is not a descendant of its current value, removing commits from that branch on the server.	Chapter 12, section 12.8
fork	GitHub	A second repository on GitHub with its own refs, settings and permissions, connected to the repository it was made from, and storing its Git data together with it. Git has no such concept.	Chapter 15, section 15.7
fork network	GitHub	A repository and all its forks, whose Git data is stored together; an object pushed to one member can be reachable by ID through another.	Chapter 15, section 15.7
four layers of protection	Course	Refs, reflogs, the grace period, and other repositories: a recovery is always a move from a lower layer back to the first one.	Chapter 13, section 13.2
<code>fsck</code>	Git	<code>git fsck</code> verifies that everything reachable is present and well-formed, and lists what nothing reaches.	Chapter 3, section 3.8

G

Term	Layer	Definition	Taught in
garbage collection	Git	Packing objects and refs, expiring reflog entries by age, and deleting unreachable objects older than the grace period: <code>git gc</code> , and the tasks of <code>git maintenance</code> .	Chapter 26, section 26.3
geometric repacking	Git	Keeping packs in a sequence where each is at least twice as large as the next, by merging only the small ones; the default automatic maintenance strategy since Git 2.54.	Chapter 26, section 26.4
Git	Git	A program on your machine that stores history in a <code>.git</code> directory. It knows nothing about accounts, pull requests or rules.	Chapter 1, section 1.5
Git Flow	Course	A branching model with two long-lived branches, <code>main</code> for released code and <code>develop</code> for integration, plus feature, release and hotfix branches. The names are conventions; Git gives none of them a meaning.	Chapter 27, section 27.7
Git LFS	LFS	A separate program that replaces large files in commits by pointer files through a <code>clean</code> and a <code>smudge</code> filter and keeps the content in an LFS object store.	Chapter 22, section 22.3

Term	Layer	Definition	Taught in
<code>.git</code> directory	Git	The repository: an object database, names for objects, a log of how those names moved, one staging file, and settings. The files next to it are a working tree that Git can rebuild from it.	Chapter 3, section 3.2
<code>.gitattributes</code>	Git	A versioned file that attaches attributes to paths; it reaches every clone, while the programs that some attributes name are defined in each clone's configuration.	Chapter 14C, section 14C.4
GitHub	GitHub	A hosted service that stores Git repositories and surrounds them with its own objects: accounts, permissions, pull requests, reviews, rulesets and workflows.	Chapter 1, section 1.5
GitHub App	GitHub	An integration that acts with its own identity; its installation token (prefix <code>ghs_</code>) lasts 1 hour and reaches the repositories and permissions of the installation.	Chapter 16, section 16.14
GitHub CLI (<code>gh</code>)	GitHub	A client for GitHub's API that knows which repository you are in. Version 2.88.1 is installed here.	Chapter 15, section 15.16
GitHub Flow	Course	One long-lived branch that is always deployable, short branches off it, a pull request for each, and deployment right after the merge.	Chapter 27, section 27.6
GitHub-hosted runner	Actions	A fresh virtual machine, built from a published image, that runs one job and is then destroyed.	Chapter 20B, section 20B.8
<code>GITHUB_TOKEN</code>	Actions	The short-lived installation token every job gets for the repository that contains the workflow; the <code>permissions</code> key decides what it may do.	Chapter 21A, section 21A.3
<code>.gitignore</code>	Git	A file of ignore patterns. It affects only untracked paths: a file that is already tracked stays tracked.	Chapter 4, section 4.5
gitlink	Git	A tree entry of mode 160000 that records the ID of a commit in another repository: what a superproject stores for a submodule.	Chapter 23, section 23.2
<code>.gitmodules</code>	Git	The versioned file in a superproject that records each submodule's path and URL.	Chapter 23, section 23.2
grace period	Git	The time an unreachable object is kept before a collection may delete it: two weeks by default (<code>gc.pruneExpire</code>).	Chapter 13, section 13.4

H

Term	Layer	Definition	Taught in
head branch	GitHub	The branch whose commits a pull request offers to the base branch.	Chapter 17, section 17.2
<code>HEAD</code>	Git	The ref that says where you are: normally a symbolic ref that names the current branch, and in detached state a direct reference to a commit.	Chapter 7, section 7.3
hook	Git	An executable file with a fixed name in the hooks directory (or, since Git 2.54, a command named in configuration) that Git runs at a defined point of a command. Hooks are local, are not cloned, and cannot enforce policy.	Chapter 14C, section 14C.9
host key	GitHub	The key with which an SSH server proves who it is; <code>ssh</code> compares it with the one remembered in <code>~/.ssh/known_hosts</code> .	Chapter 16, section 16.11
hotfix	Course	A fix made for a released version; under release branches it is committed on <code>main</code> first and cherry-picked down, or on the oldest branch and merged upward.	Chapter 27, section 27.10
hunk	Git	One contiguous block of changed lines in a diff, with its context; <code>git add -p</code> stages hunk by hunk.	Chapter 5, section 5.5

I

Term	Layer	Definition	Taught in
ignored file	Git	A path that has no index entry and matches an ignore pattern. Ignored is the opposite of tracked, so no pattern can ignore a tracked file.	Chapter 4, section 4.3
incident	Course	A problem in shared state (a shared branch, a published history, a credential, a pipeline) that other people depend on while you are still working out what it is.	Chapter 30, section 30.2
index	Git	One binary file, <code>.git/index</code> , that records for every tracked path a mode, a blob ID, a stage number and cached <code>stat</code> data. It is the proposed next commit, also called the staging area.	Chapter 5, section 5.2
indirect merge	GitHub	A pull request that GitHub marks as merged because its head commits became reachable from its base branch by another route.	Chapter 17, section 17.13
intent-to-add	Git	<code>git add -N <path></code> gives a new file an index entry without staging its content, so that index-against-working-tree comparisons stop overlooking it.	Chapter 5, section 5.6
interactive rebase	Git	<code>git rebase -i</code> : Git shows the todo list before executing it, and you may edit, reorder, extend or shorten it.	Chapter 9, section 9.6

Term	Layer	Definition	Taught in
issue	GitHub	A numbered record in GitHub's database for a unit of work or a report, which since 2025 can have a type, a parent, children and dependencies.	Chapter 15, section 15.8

J

Term	Layer	Definition	Taught in
job	Actions	A list of steps that run in order on one fresh runner; jobs of a workflow run in parallel unless <code>needs</code> orders them.	Chapter 20A, section 20A.2

K

Term	Layer	Definition	Taught in
<code>known_hosts</code>	GitHub	The file <code>~/.ssh/known_hosts</code> in which <code>ssh</code> remembers server host keys. Deleting it wholesale removes the check.	Chapter 16, section 16.11

L

Term	Layer	Definition	Taught in
label	GitHub	A GitHub tag on issues and pull requests, stored in GitHub's database; <code>gh label clone</code> copies a label set between repositories.	Chapter 15, section 15.9
lease	Git	The expected value of the server's ref in <code>--force-with-lease</code> . Without an explicit value it is your remote-tracking ref, which a background fetch can update without your looking.	Chapter 12, section 12.8
lightweight tag	Git	A ref under <code>refs/tags/</code> that names an object directly: no tag object, no tagger, no message.	Chapter 7, section 7.11
line-ending normalization	Git	A path with the <code>text</code> attribute is stored with LF endings in the repository whatever its endings are on disk; <code>*text=auto</code> lets Git decide per file which files are text.	Chapter 14C, section 14C.5
linear history	GitHub	As a rule ("Require linear history"): GitHub rejects an update that adds a commit with two parents to the ref.	Chapter 18, section 18.11
linked worktree	Git	An additional working tree created by <code>git worktree add</code> : the same objects and refs as the main worktree, with a HEAD and an index of its own.	Chapter 25, section 25.2
loose object	Git	One compressed file whose content is a short header, a NUL byte and the data, and whose file name is the hash of those bytes.	Chapter 3, section 3.3
loose ref	Git	A ref stored as a small file under <code>refs/</code> in the default <code>files</code> format; the same ref may also be a line in <code>packed-refs</code> .	Chapter 3, section 3.9

M

Term	Layer	Definition	Taught in
machine user	GitHub	A user account for automation: it occupies a seat, and somebody has to own its password and second factor.	Chapter 16, section 16.14
<code>main</code>	Git	The example default branch of this course, set through <code>init.defaultBranch</code> ; unconfigured Git 2.55 still creates <code>master</code> . A name by convention only.	Chapter 1, section 1.9
maintenance	Git	<code>git maintenance</code> runs a configurable set of tasks (<code>gc</code> , <code>commit-graph</code> , <code>prefetch</code> , <code>loose-objects</code> , <code>incremental-repack</code> and others); since Git 2.54 the maintenance Git starts on its own no longer runs <code>git gc</code> .	Chapter 26, section 26.3
master	Git	The name unconfigured Git still gives the first branch. See <code>main</code> .	—
matrix	Actions	A strategy that turns one job definition into one job per combination of the values you list, each with its own runner and <code>matrix</code> context.	Chapter 20A, section 20A.10
merge	Git	Joining another history into the current branch: by fast-forward, or by a three-way merge of the two tips against their merge base, recorded as a commit with two parents.	Chapter 8, section 8.4
merge base	Git	The best common ancestor of two commits: the most recent commit that both histories contain, and the third input of every merge.	Chapter 8, section 8.2
merge commit	Git	An ordinary commit object with more than one <code>parent</code> line; the order of those lines records which branch received the merge.	Chapter 8, section 8.13
merge driver	Git	The file-level merge chosen by the <code>merge</code> attribute for a path: the normal text merge, a refusal (<code>-merge</code>), the built-in <code>union</code> driver, or a program you define.	Chapter 14C, section 14C.7
merge method	GitHub	One of the three merge buttons: a merge commit that keeps your commits, a squash that replaces them by one new commit, or a rebase that puts new copies of your commits in a straight line.	Chapter 17, section 17.8

Term	Layer	Definition	Taught in
merge queue	GitHub	A mechanism that merges pull requests one group at a time and runs the required checks on the exact commit that will become the new tip of the base branch; workflows must listen to the <code>merge_group</code> event.	Chapter 17, section 17.11
merge strategy	Git	The algorithm that turns the inputs of a merge into a result tree, selected with <code>-s .ort</code> is the default since Git 2.34.	Chapter 8, section 8.6
MERGE_HEAD	Git	The pseudoref that names the commit being merged while a merge is in progress; it is what makes the next commit a merge commit.	Chapter 8, section 8.8
milestone	GitHub	A GitHub object that groups issues and pull requests toward a date and shows a completion percentage. It is not a Git tag and not a release.	Chapter 15, section 15.9
monorepo	Course	A repository that keeps many separately deployed projects together so that one commit can change all of them; a polyrepo layout gives each project its own history, access rules and release cadence.	Chapter 24, section 24.2
multi-pack-index	Git	One sorted index over the objects of many packs, so that finding an object costs one lookup however many packs there are.	Chapter 26, section 26.7

N

Term	Layer	Definition	Taught in
non-fast-forward	Git	A ref update in which the new value is not a descendant of the old one. A normal push refuses it.	Chapter 12, section 12.7

O

Term	Layer	Definition	Taught in
OAuth app token	GitHub	A token (prefix <code>gho_</code>) granted to an app on behalf of a user, by scope; <code>gh auth login</code> produces one.	Chapter 16, section 16.6
object	Git	A unit of Git's storage, named by the hash of its type and content. There are four types: blob, tree, commit and tag.	Chapter 2, section 2.5
object database	Git	The store of all objects under <code>.git/objects</code> , as loose files and packs.	Chapter 3, section 3.2
object ID	Git	The hash that names an object: 40 hex digits with SHA-1, the default, and 64 with SHA-256. Never assume the length.	Chapter 2, section 2.4
octopus merge	Git	A single merge commit with three or more parents, made by a separate strategy that refuses any conflict.	Chapter 8, section 8.14
OIDC federation	Actions	In place of a stored cloud key, the job asks GitHub for a signed statement of what it is, and the cloud exchanges it for a credential valid for that job only.	Chapter 21A, section 21A.10
organization	GitHub	A container that owns repositories and decides who may do what; people sign in to user accounts.	Chapter 15, section 15.3
ORIG_HEAD	Git	One file holding the commit HEAD pointed at before the last "drastic" command (<code>git am</code> , <code>git merge</code> , <code>git rebase</code> , <code>git reset</code> ; also <code>git pull</code> and <code>git stash push</code>). It has one slot and no reflog: use it only as the very next command.	Chapter 13, section 13.5
origin	Git	The name <code>git clone</code> gives the remote it cloned from. A convention, not a keyword.	Chapter 12, section 12.3
"ours" and "theirs"	Git	The two sides of a three-way merge: stage 2 and stage 3. In a merge, "ours" is your branch. During a rebase "ours" is the new base plus the copies made so far and "theirs" is your own commit.	Chapter 9, section 9.11

P

Term	Layer	Definition	Taught in
pack index	Git	The <code>.idx</code> file beside a packfile that finds any object in it by ID. Derived data: <code>git index-pack</code> rebuilds it.	Chapter 3, section 3.7
packed-refs	Git	One file that holds many refs as lines, written by <code>git pack-refs</code> ; a loose file for the same ref takes precedence.	Chapter 3, section 3.9
packfile	Git	One file that stores many objects and may store an object as a delta from a similar object. None of this changes what an object is.	Chapter 3, section 3.7
parent (of a commit)	Git	A commit named in another commit's <code>parent</code> line; these lines are the only links in the history graph.	Chapter 6, section 6.6
partial clone	Git	A clone that has all of the history's structure and leaves out objects of a chosen kind, which a promisor remote has promised to deliver when a command needs them.	Chapter 26, section 26.12

Term	Layer	Definition	Taught in
patch ID	Git	A hash of a change's diff, ignoring line numbers and whitespace, by which <code>git cherry</code> , <code>git log --cherry-mark</code> and rebase recognize the same change under another commit ID.	Chapter 10, section 10.10
pathspec	Git	A pattern that limits a command to paths; <code>.</code> means this directory and below.	Chapter 5, section 5.8
permissions (workflow key)	Actions	The key that decides what the job's <code>GITHUB_TOKEN</code> may do; least privilege is declared here.	Chapter 21A, section 21A.3
personal access token (classic)	GitHub	A token (prefix <code>ghp_</code>) that reaches, by scope, every repository and organization the user can reach; long-lived, no expiry required.	Chapter 16, section 16.6
pickaxe	Git	<code>git log -S<string></code> finds commits that changed how many times a string occurs; <code>-G<regex></code> finds commits whose patch has an added or removed line matching a pattern.	Chapter 14A, section 14A.11
pinning (an action)	Actions	Writing <code>uses: owner/repo@<full commit ID></code> : only a full commit ID guarantees that the code is the code you reviewed, because tags can move.	Chapter 21A, section 21A.7
plumbing	Git	Commands with stable output that do one low-level operation each (<code>git hash-object</code> , <code>git update-ref</code>); porcelain commands such as <code>git add</code> and <code>git commit</code> are built from them.	Chapter 2, section 2.6
pointer file	LFS	The three-line blob that Git stores for an LFS-tracked path: the specification version, the SHA-256 of the real content, and its size.	Chapter 22, section 22.3
porcelain	Git	The user-facing commands, such as <code>git add</code> and <code>git commit</code> , as opposed to plumbing.	Chapter 2, section 2.6
postmortem	Course	A written record of an incident that explains how the system allowed it, so that the system can be changed; "blameless" means every person's action is treated as reasonable given what they knew.	Chapter 30, section 30.19
private vulnerability reporting	GitHub	A way for outsiders to report a security problem to maintainers without a public issue; a <code>SECURITY.md</code> file states how to report.	Chapter 21B, section 21B.13
promisor remote	Git	The remote that has promised to deliver the objects a partial clone left out.	Chapter 26, section 26.12
protocol version 2	Git	The wire protocol that is the default since Git 2.26; the server no longer lists every ref first.	Chapter 12, section 12.13
prune	Git	To delete: unreachable objects (<code>git prune</code> , <code>git gc --prune=<date></code>), stale remote-tracking refs (<code>git fetch --prune</code>), or entries of missing worktrees (<code>git worktree prune</code>).	Chapter 13, section 13.13
pseudoref	Git	Since Git 2.46 only <code>FETCH_HEAD</code> and <code>MERGE_HEAD</code> : files that look like refs and are not refs in the strict sense. <code>ORIG_HEAD</code> , <code>CHERRY_PICK_HEAD</code> and the others are root refs.	Chapter 3, section 3.10
pull	Git	<code>git pull</code> is <code>git fetch</code> followed by one integration of the fetched upstream into the current branch: a fast-forward, a merge or a rebase.	Chapter 12, section 12.6
pull request	GitHub	A GitHub object that proposes merging a head branch into a base branch. Opening one moves no branch; it creates <code>refs/pull/N/head</code> and, when the merge is clean, a test merge commit at <code>refs/pull/N/merge</code> .	Chapter 17, section 17.2
<code>pull_request_target</code>	Actions	An event that runs the workflow from the default branch, with the repository's token and secrets, in response to a pull request; safe only as long as it never executes the pull request's code.	Chapter 21A, section 21A.5
push	Git	<code>git push</code> asks a remote to make some of its refs point at commits of yours and sends the objects it lacks; each ref succeeds only if both your Git and the remote accept the update.	Chapter 12, section 12.7
push protection	GitHub	A secret-scanning feature that blocks a push containing a recognized secret before it reaches the repository.	Chapter 21B, section 21B.12
"pwn request"	Actions	The attack on a workflow with a privileged trigger such as <code>pull_request_target</code> that checks out and executes the code of an untrusted pull request.	Chapter 21A, section 21A.5

R

Term	Layer	Definition	Taught in
range	Git	A set of commits defined by reachability: everything reachable from the included tips, minus everything reachable from the excluded ones.	Chapter 14A, section 14A.8
range-diff	Git	<code>git range-diff</code> compares two versions of a series of commits by pairing each old commit with its new counterpart.	Chapter 9, section 9.14
reachability	Git	An object is reachable if you can arrive at it from a starting point (a ref, the index, a reflog entry) by following the IDs stored inside objects.	Chapter 3, section 3.8
rebase	Git	<code>git rebase <upstream></code> creates a copy of each commit your branch has and the upstream lacks, on top of the upstream, and then points your branch at the last copy. Every copy has a new ID.	Chapter 9, section 9.2
<code>REBASE_HEAD</code>	Git	The root ref that names the commit being replayed while a rebase is stopped.	Chapter 9, section 9.11

Term	Layer	Definition	Taught in
ref	Git	A name that holds an object ID or the name of another ref: branches, tags, remote-tracking branches, HEAD.	Chapter 3, section 3.9
reflog	Git	A local, append-only list of the values a ref has had, with the time, the person and the command behind each change. Entries expire after 90 days, or 30 when unreachable, by default.	Chapter 13, section 13.3
refs/pull/N/head , refs/pull/N/merge	GitHub	Read-only refs GitHub creates in the base repository for pull request N: the head commit, and the test merge commit when the merge is clean.	Chapter 17, section 17.2
refspec	Git	[+]<source>:<destination>: the rule that decides which refs a fetch or a push touches; the source is on the sending side.	Chapter 12, section 12.12
reftable	Git	A ref storage format that keeps refs and reflogs in binary tables under .git/reftable/; available since Git 2.45 and planned as the default for new repositories in Git 3.0.	Chapter 3, section 3.13
release	GitHub	A GitHub record that points at a tag name and adds a title, notes, files and flags. The tag is Git's; the release is not.	Chapter 15, section 15.12
release branch	Course	A branch cut from the trunk shortly before a release, which receives only fixes and is where the release and its patches are tagged.	Chapter 27, section 27.8
remote	Git	A name in your repository's configuration that stands for another repository: its URL, and the rules for copying refs between that repository and yours.	Chapter 12, section 12.2
remote-tracking branch	Git	A ref in your own repository, such as origin/main, that records the last position of a branch in another repository as of your last fetch or push.	Chapter 7, section 7.10
rename detection	Git	A commit does not record renames; git diff and git log conclude that a path was renamed when one path disappeared, another appeared, and their contents are similar enough.	Chapter 4, section 4.9
repository	Git	The .git directory: objects, refs, reflogs, the index and configuration. On GitHub, a Git repository surrounded by records in GitHub's database that Git cannot copy.	Chapter 3, section 3.2
required review	GitHub	The pull request rule that turns advisory reviews into a gate: a number of approvals, with sub-options that each close one loophole.	Chapter 18, section 18.7
rerere	Git	"Reuse recorded resolution": with rerere.enabled, Git records the text of every conflict together with your resolution and writes it again when the same conflict text appears.	Chapter 14C, section 14C.3
rescue branch	Course	A branch such as rescue/<what> created on a lost or endangered commit before anything else moves: a named commit cannot expire.	Chapter 13, section 13.7
reset	Git	git reset <commit> makes the current branch point at <commit>; the mode decides whether the index (--mixed) and the working tree (--hard) are rewritten to match.	Chapter 11, section 11.4
restore	Git	git restore copies a stored version of the named paths into the working tree, into the index, or into both, and moves no ref.	Chapter 11, section 11.3
reusable workflow	Actions	A workflow file with on: workflow_call that another workflow calls as if it were a single job.	Chapter 20B, section 20B.6
revert	Git	git revert <commit> adds a commit whose change is the opposite of what <commit> changed, and removes nothing.	Chapter 11, section 11.8
review	GitHub	A claim about a specific diff at a specific commit: a comment, an approval or a request for changes.	Chapter 17, section 17.17
risk labels	Course	🟢 SAFE reads state or only adds objects; 🟡 CAUTION moves refs or rewrites local history; 🔴 DANGEROUS can destroy uncommitted work, remote history, or the safety net itself.	Chapter 1, section 1.8
role	GitHub	A named level of access to a repository. In an organization, what a person can do is the highest of every grant that reaches them.	Chapter 15, section 15.4
root commit	Git	A commit without a parent line.	Chapter 6, section 6.6
root ref	Git	A ref that lives directly in the repository directory, with a name in capital letters such as HEAD, ORIG_HEAD, CHERRY_PICK_HEAD or REBASE_HEAD.	Chapter 3, section 3.10
root-cause framework	Course	The seven-line box this course uses for surprising behavior: observed behavior, Git state, mechanism, root cause, why Git does this, correct fix, prevention.	Chapter 1, section 1.10
ruleset	GitHub	A named list of rules with a target (which refs), an enforcement status and a bypass list. All active rulesets that target a ref are added together, and for each rule the strictest version wins.	Chapter 18, section 18.3
runner	Actions	The machine that runs one job: GitHub-hosted, or self-hosted on a machine you operate.	Chapter 20A, section 20A.2

S

Term	Layer	Definition	Taught in
safe.bareRepository	Git	A setting: with explicit, Git uses a bare repository only when you name it with --git-dir or GIT_DIR. Planned as the default in Git 3.0.	Chapter 21B, section 21B.3

Term	Layer	Definition	Taught in
<code>safe.directory</code>	Git	The list of directories Git trusts although a different operating-system user owns them; without an entry Git refuses to read such a repository's configuration.	Chapter 21B, section 21B.3
Scalar	Git	<code>scalar</code> , a command shipped with Git since 2.38 that sets up a blobless, sparse clone together with settings and scheduled maintenance.	Chapter 24, section 24.7
script injection	Actions	An expression in <code>\${{ }}</code> is replaced by its value in the text of a script before the shell starts, so a value an outsider controls becomes part of your program.	Chapter 21A, section 21A.6
secret (Actions)	Actions	An encrypted value stored in GitHub settings and read through the <code>secrets</code> context; it is as exposed as the least trustworthy code in any job that receives it, and masking is not a boundary.	Chapter 21A, section 21A.9
secret scanning	GitHub	A GitHub feature that scans the entire Git history on all branches, plus issues, pull requests, discussions and wikis, for recognized secrets; free on public repositories.	Chapter 21B, section 21B.12
self-hosted runner	Actions	The runner program on a machine you operate, which asks GitHub for jobs and runs them with whatever access that machine has; it keeps state between jobs unless made ephemeral.	Chapter 20B, section 20B.9
Semantic Versioning	Course	The version scheme <code>MAJOR.MINOR.PATCH</code> of the 2.0.0 specification, used for the tag names in this course.	Chapter 14B, section 14B.13
sequencer	Git	The state under <code>.git/sequencer/</code> that holds the rest of the plan when a cherry-pick or revert of several commits has stopped.	Chapter 10, section 10.6
SHA-1, SHA-256	Git	The two object formats. SHA-1 with collision detection is the default; SHA-256 repositories (supported since Git 2.42) cannot exchange objects with SHA-1 repositories, and a repository on GitHub stays SHA-1.	Chapter 3, section 3.14
shallow clone	Git	A clone that contains the commits within a given distance of the requested tips and a file, <code>.git/shallow</code> , that tells Git to pretend the history ends there.	Chapter 26, section 26.11
sign-off	Git	The <code>Signed-off-by:</code> trailer added by <code>git commit -s</code> . It is a line of text, not a cryptographic signature.	Chapter 6, section 6.9
signature	Git	A detached cryptographic signature over the bytes of one commit or one annotated tag, produced by an external program and stored inside that object. It proves that a key holder signed, not who wrote the change.	Chapter 14B, section 14B.15
skip-worktree bit	Git	A flag on an index entry that tells Git not to write or examine the working tree file; sparse-checkout uses it for paths outside the cone.	Chapter 5, section 5.12
smudge filter	Git	The command of a filter driver that rewrites content on its way from the repository into the working tree, at checkout.	Chapter 14C, section 14C.8
snapshot	Git	The complete state of every tracked file at one moment, recorded by a commit as the ID of one tree. Git stores snapshots, not diffs.	Chapter 2, section 2.3
sparse index	Git	An index in which all entries of a directory outside the cone are replaced by one entry naming that directory's tree.	Chapter 24, section 24.6
sparse-checkout	Git	A working tree that holds only chosen directories, while the repository, its history and its index still contain every file.	Chapter 24, section 24.4
squash	Git	Combining several commits into one: in an interactive rebase, with <code>git merge --squash</code> , or with GitHub's "Squash and merge", each of which creates a new commit.	Chapter 8, section 8.12
SSH key	GitHub	A key pair with which you prove who you are by signing a challenge; the private key never leaves your machine and GitHub holds only the public key.	Chapter 16, section 16.8
SSO (SAML single sign-on)	GitHub	An Enterprise Cloud feature: in an organization that uses it, a credential must be authorized for that organization in addition to being valid.	Chapter 16, section 16.15
stacked pull requests	GitHub	A chain of pull requests in one repository in which each targets the branch of the one below, so that a large change is reviewed as small layers.	Chapter 17, section 17.14
stage (number)	Git	The slot of an index entry: 0 normally; during a conflict 1 holds the base, 2 "ours" and 3 "theirs".	Chapter 5, section 5.13
staging area	Git	Another name for the index. See index.	—
stale branch	Git	A ref that nobody needs any more; Git offers three tests for it (age, reachability from <code>main</code> , a deleted upstream), each with a blind spot.	Chapter 7, section 7.13
stash	Git	<code>git stash push</code> records the index and the working tree as commits that only <code>refs/stash</code> reaches, then resets both to HEAD. The stash list is the reflog of <code>refs/stash</code> .	Chapter 11, section 11.11
stat data	Git	The size and timestamps an index entry remembers from when Git last saw the file match it, so that "has this file changed?" normally costs one <code>lstat</code> call.	Chapter 5, section 5.14
step	Actions	One entry of a job: a shell command (<code>run</code>) or an action (<code>uses</code>). Every <code>run</code> step is a new shell process.	Chapter 20A, section 20A.2
strategy option	Git	An option passed with <code>-X</code> that adjusts how one merge strategy behaves, for example <code>-X ours</code> , which silently takes one side of every conflicting hunk.	Chapter 8, section 8.6
submodule	Git	Another repository checked out in a subdirectory of yours, of which your repository records exactly one thing in its history: the ID of the commit that should be checked out there.	Chapter 23, section 23.2

Term	Layer	Definition	Taught in
subtree	Git	<code>git subtree</code> copies another project's files into a subdirectory as ordinary tracked files and records in commit messages where they came from.	Chapter 23, section 23.13
superproject	Git	The repository that contains a submodule.	Chapter 23, section 23.2
symbolic ref	Git	A ref that holds the name of another ref instead of an object ID; HEAD is normally one.	Chapter 3, section 3.9

T

Term	Layer	Definition	Taught in
tag	Git	A ref under <code>refs/tags/</code> that is not expected to move; lightweight or annotated. Every clone keeps its own copy and never updates it by itself.	Chapter 14B, section 14B.8
tag object	Git	The fourth object type: a named, dated pointer to another object with a tagger and a message, optionally signed.	Chapter 3, section 3.4
team	GitHub	A group of organization members that can be granted a role on repositories and named as a code owner or required reviewer.	Chapter 15, section 15.4
ten-command diagnosis	Course	The fixed, read-only sequence of ten commands with which every investigation in this course opens.	Chapter 1, section 1.11
test merge commit	GitHub	The commit at <code>refs/pull/N/merge</code> that GitHub computes when a pull request merges cleanly; <code>actions/checkout</code> checks it out on <code>pull_request</code> events.	Chapter 17, section 17.2
textconv	Git	A diff driver setting that names a program converting a file to text before Git diffs it.	Chapter 14C, section 14C.6
three trees	Git	The three places a version of a file can be at the same time: the commit that HEAD names, the index, and the working tree.	Chapter 2, section 2.9
three-way merge	Git	Combining two versions against their common base, path by path and then line by line: a change made on one side only is taken, and overlapping different changes conflict.	Chapter 8, section 8.4
tip	Git	The commit a branch ref currently holds.	Chapter 7, section 7.2
todo list	Git	The list of instructions (<code>pick</code> , <code>reword</code> , <code>edit</code> , <code>squash</code> , <code>fixup</code> , <code>drop</code> , <code>exec</code> and others) that a rebase executes from top to bottom.	Chapter 9, section 9.6
token	GitHub	A string that stands for an account with a subset of its rights; the prefix says what kind it is, and the kind decides how far a leak reaches. Over HTTPS, GitHub accepts a token where HTTP expects a password, and nothing else.	Chapter 16, section 16.6
tracked file	Git	A path that has an index entry.	Chapter 4, section 4.3
tracking branch	Git	Loose usage for either a remote-tracking branch or a local branch that has an upstream. See remote-tracking branch and upstream.	—
trailer	Git	A <code>Key: value</code> line in the last paragraph of a commit message, placed where Git and other tools can find and parse it.	Chapter 6, section 6.9
tree	Git	The object type that holds one directory listing: for each entry a mode, a name and the ID of a blob or another tree.	Chapter 3, section 3.4
trunk-based development	Course	Developers integrate into one branch, the trunk, and avoid other long-lived development branches; larger teams use short-lived feature branches.	Chapter 27, section 27.8
two-factor authentication	GitHub	Required by GitHub since March 2023 for everyone who contributes code on GitHub.com.	Chapter 16, section 16.15

U

Term	Layer	Definition	Taught in
unborn branch	Git	The branch HEAD names in a new repository before the first commit: the name exists in HEAD, the ref does not.	Chapter 7, section 7.3
unreachable object	Git	An object that no ref, no reflog entry and the index do not reach. It stays in the object database until a collection deletes it after the grace period.	Chapter 13, section 13.2
untracked cache	Git	A cache (<code>core.untrackedCache</code>) that remembers which directories contained no untracked files.	Chapter 26, section 26.9
untracked file	Git	A path in the working tree that has no index entry and matches no ignore pattern.	Chapter 4, section 4.3

Term	Layer	Definition	Taught in
upstream (branch)	Git	The branch on a remote that <code>git status</code> compares a local branch with and that <code>git pull</code> integrates from; two lines of configuration.	Chapter 12, section 12.5
upstream (repository)	GitHub	In the fork workflow, the project's repository from which a fork was made, by convention the remote named <code>upstream</code> .	Chapter 27, section 27.2

V

Term	Layer	Definition	Taught in
variable (Actions)	Actions	A plain value stored in GitHub settings and read through the <code>vars</code> context; unlike a secret it is neither encrypted nor masked.	Chapter 20A, section 20A.6
version control system	Git	A system that records successive states of a set of files so that you can recall any state, compare any two, and see who recorded each one, when, and why.	Chapter 1, section 1.2
vigilant mode	GitHub	The account setting "Flag unsigned commits as unverified": with it, unsigned commits that name you are marked; without it an unsigned commit gets no badge at all.	Chapter 21B, section 21B.6
visibility	GitHub	Who can read a repository at all: public or private; internal repositories need Enterprise Cloud.	Chapter 15, section 15.5

W

Term	Layer	Definition	Taught in
webhook	GitHub	A registration of a URL and events on a repository, an organization or a GitHub App: GitHub sends an HTTP request with a JSON payload when one occurs.	Chapter 15, section 15.18
workflow	Actions	A YAML file under <code>.github/workflows/</code> that says "when this event happens, run these jobs".	Chapter 20A, section 20A.2
<code>workflow_dispatch</code>	Actions	The event that starts a run by hand, from the web interface or with <code>gh workflow run</code> .	Chapter 20A, section 20A.4
working tree	Git	The directory of ordinary files that you edit, build and run: one checked-out version of the project, plus whatever else is lying in that directory.	Chapter 4, section 4.2
worktree	Git	A working tree together with its own HEAD and index; a repository has one main worktree and any number of linked worktrees. A branch can be checked out in at most one.	Chapter 25, section 25.2

Z

Term	Layer	Definition	Taught in
zdiff3	Git	See conflict style.	—

See also

- Git command reference: every command taught, with the forms, a mistake and the recovery.
- GitHub reference: limits, roles, rule types, merge methods, token types, events.
- Command safety: every 🟡 and 🔴 command.
- References: the sources behind the chapters, including [gitglossary](#), on your machine as `git help glossary`.

Chapter 35: References

Source file: `reference/references.md`

References

Baseline. Sources as cited on 1 October 2026 for Git 2.55.0 (2.56.0 where a chapter says so) and for GitHub. This file is the "Sources" section of every chapter, collected in the order of the modules. No source was added: every link, date, caveat and video below stands in a chapter, and the chapters took them from the Phase 0 research report, its research notes, the local manual pages, or official pages fetched while the chapter was written.

How to use this list

- **Primary sources** are the Git manual pages and source, GitHub's documentation and changelog, and the tools' own repositories. When a chapter and a primary source disagree, the primary source for your version wins: on your machine, `git help <command>` is the manual of the Git you run.
- **Secondary sources** are books, blog posts and surveys. Each carries the caveat its chapter gave it (age, sample, vendor interest).
- **Videos** are optional and come only from the tables of the Phase 0 report. The assessments there rest on captions and chapter lists, not on full viewing, and several videos predate `git switch` and `git restore` or assume a branch called `master`; the note beside each video says so.
- **Further reading** points to other chapters and to deeper material.

GitHub-side facts carry dates because the platform changes: before you rely on a limit, a price, a plan gate or a label, open the linked page. The re-verification calendar is section 10 of the roadmap.

Module index

Modules	Chapters
Modules 0 and 1: what Git is, the mental model	Chapter 1: Fundamentals, Chapter 2: The Mental Model
Module 2: working tree, index, HEAD	Chapter 4: The Working Tree, Chapter 5: The Index
Module 3: commits	Chapter 6: Commits
Module 4: refs, branches, HEAD	Chapter 7: Branches
Modules 5, 13 and 24 (local part): configuration, tags and versions, signing	Chapter 14B: Configuration, Aliases, Tags and Signing
Module 6: merge	Chapter 8: Merge
Module 7: remotes	Chapter 12: Remote Operations
Module 8: undo	Chapter 11: Reset, Revert, Restore
Module 9: rebase	Chapter 9: Rebase
Module 10: cherry-pick and ranges	Chapter 10: Cherry-pick
Modules 10 (ranges) and 11: history forensics	Chapter 14A: History investigation
Module 12: recovery	Chapter 13: Recovery
Module 14: hooks, rerere, attributes, worktrees	Chapter 14C: Stash Internals, Rerere, Attributes, Hooks, Chapter 25: Worktrees
Module 15: submodules, subtrees, LFS	Chapter 22: Git LFS, Chapter 23: Submodules and Subtrees
Modules 16 and 17: the object database, the index, refs and the Git directory	Chapter 3: Git Internals
Modules 16 and 18: maintenance, transfer and scale	Chapter 26: Performance, Chapter 24: Monorepos
Modules 19 and 25: the GitHub platform, the CLI and the API	Chapter 15: GitHub
Module 20: authentication and SSH	Chapter 16: Authentication
Modules 21 and 22: pull requests, forks, merge methods, releases	Chapter 17: Pull Requests
Module 23: governance	Chapter 18: Branch Protection and Rulesets, Chapter 19: CODEOWNERS
Module 26: Actions fundamentals	Chapter 20A: GitHub Actions Fundamentals
Modules 27 and 28: build, package, deliver; runners and debugging CI	Chapter 20B: GitHub Actions: delivery, runners, cost and debugging
Module 29: Actions security	Chapter 21A: GitHub Actions security

Modules	Chapters
Modules 24 (GitHub part), 30 and 31: repository security and secret-leak response	Chapter 21B: Repository security, identity, the Git client, and secret-leak response
Modules 32 and 34: branching and release strategy, practices and design review	Chapter 27: Open source and team workflows
Module 33: AI/ML workflows	Chapter 28: AI/ML workflows
Module 35: the diagnosis method	Chapter 29: Production Troubleshooting
Modules 36 to 38: incident drills, communication and postmortems	Chapter 30: Incident Response
Module 42: the frontier	Chapter 14D: The Frontier

Modules 0 and 1: what Git is, the mental model

Chapter 1: Fundamentals

From section 1.19 of Chapter 1: Fundamentals.

Primary sources

- [git\(1\)](#): description, environment variables, `--no-optional-locks`.
- [gitrepository-layout](#): every file in `.git`.
- [gitglossary](#): bare repository, unborn branch, plumbing, porcelain.
- [gitdatamodel](#): the official data model page, on your machine as `git help datamodel`.
- [git-init](#), [git-status](#), [git-config](#), [git-rev-parse](#), [git-ls-files](#), [git-rm](#).
- Git source at the v2.55.0 tag: [builtin/rm.c](#), [submodule.c](#) and [builtin/fsck.c](#), read for the two root-cause notes in sections 1.7 and 1.13.
- Pro Git, second edition: [About Version Control, A Short History of Git, What is Git?](#). The book is hosted on the official site and has been frozen since May 2024, so its command style is older than this chapter's.
- GitHub: [changelog entry on CLI telemetry](#), [gh environment variables](#).

Secondary sources

- Phase 0 research report, sections 1, 2, 4 and 12: versions, dates, the two Gits, and the Git and GitHub layers, each with a link to its primary source.

Videos

The report assessed these from captions and chapter lists, not by full viewing.

- ["Learn Git & GitHub for Beginners \(2026\) Tutorial"](#), Coder Coder, 56 minutes, 28 June 2026. Current commands, including `git switch` and `git restore`. Beginner scope only.
- ["Git Tutorial for Beginners: Learn Git in One Video"](#), CodeWithHarry, Hindi, 2 hours 32 minutes, 12 July 2026. Current commands. It teaches no reset, revert or reflog, uses a local `master`, and first calls a branch a copy before correcting that to a pointer.
- ["Tech Talk: Linus Torvalds on git"](#), 2007, and ["Two decades of Git"](#), 2025. Design intent and history. Polemical; his 2025 remark that the SHA-256 work was needless churn is his opinion, not the project's position.

Further reading

- [gittutorial](#) and [giteveryday](#), also available as `git help tutorial` and `git help everyday`.
- [The Git User's Manual](#).

Chapter 2: The Mental Model

From section 2.19 of Chapter 2: The Mental Model.

Primary sources

- [gitdatamodel](#): objects, refs, the index and reflogs; on your machine as `git help datamodel`.
- [gitglossary](#): DAG, reachable, dangling object, HEAD, ref, symref.
- [gitrevisions](#): `<rev>^{tree}`, `<rev>:<path>`, `:<path>`, and the lookup order of a short ref name.
- Manual pages: [git-hash-object](#), [git-update-index](#), [git-write-tree](#), [git-commit-tree](#), [git-update-ref](#), [git-cat-file](#), [git-ls-tree](#), [git-fsck](#), [git-patch-id](#), [git-cherry-pick](#).
- Git source at the v2.55.0 tag: [refs.c](#), read for the root-cause box in section 2.13.
- Pro Git, second edition, chapter 10: [Git Objects](#), [Git References](#). Frozen since May 2024; the object model is current, the commands use `master` and `git checkout`.
- GitHub Docs: [pull requests reference](#), [removing sensitive data from a repository](#).

Secondary sources

- Phase 0 research report: section 12 for the misconceptions, sections 1 and 4 for versions.
- Julia Evans: [how cherry-pick and revert work](#), [branches: intuition and reality](#), [confusing Git terminology](#), [poll results](#). The polls are self-selected samples: direction, not population figures.

Videos

The report assessed these from caption searches, chapter lists and descriptions, not by full viewing.

- "Lecture 5: Version Control and Git", MIT Missing Semester 2026, 1 hour 10 minutes, 19 February 2026, with [notes and exercises](#). The data model before the commands. SHA-1 only; the demo starts on `master`; nothing on GitHub.
- "Git Internals by John Britton of GitHub - CS50 Tech Talk", 58 minutes, 11 April 2018. Blobs, trees, commits, hashing, refs as files. Correct and clear; `master`, `git checkout`, SHA-1 only.
- "Complete git and Github course in Hindi", Chai aur Code, Hindi, 2 hours 55 minutes, 8 June 2024, from 55:16. Commit, tree and blob by name; HEAD as a pointer to the current branch. Local `master`; no revert, restore or cherry-pick.

Further reading

- [gitcore-tutorial](#), the Git project's own walk through the plumbing, and [The Git User's Manual](#).

Module 2: working tree, index, HEAD

Chapter 4: The Working Tree

From section 4.21 of Chapter 4: The Working Tree.

Primary sources

- [git-status](#), [gitignore](#), [git-check-ignore](#), [git-restore](#), [git-rm](#), [git-mv](#), [git-clean](#), [git-ls-files](#), [git-checkout](#), [git-diff](#), [git-config](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that the transcripts were checked against.
- [gitfaq](#): "I asked Git to ignore various files, yet they are still tracked" and "Why do I have a file that's always modified?".
- [gitglossary](#) and [gitattributes](#).
- [Git User's Manual](#): renames are not recorded; only the executable bit is.
- Release notes: [2.23.0](#), [2.51.0](#), and 1.8.2 and 1.8.5 as shipped with Git 2.55.0 (`Documentation/RelNotes/` in the Git source tree).

Secondary sources

- Pro Git, [Recording Changes to the Repository](#). Caveat: its status transcripts mix the current `git restore --staged` hint with the older `git reset HEAD <file>` hint.
- GitHub Docs, [Ignoring files](#), and the [github/gitignore](#) template collection.
- GitHub Docs, [Removing sensitive data from a repository](#), for what a committed secret requires beyond `git rm --cached`.
- Perez De Rosso and Jackson, [Purposes, Concepts, Misfits, and a Redesign of Git](#) (OOPSLA 2016): file tracking, untracking a file, file rename and empty directory are four of its seven "operational misfits".
- The Phase 0 report of this course, sections 4, 12 and 15, for the Stack Overflow figures and the ML ignore list.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Git Tutorial for Beginners: Learn Git in One Video](#), CodeWithHarry, Hindi, 12 July 2026: covers staging, ignoring files and tracking empty directories with `git restore --staged`. Caveats from the report: no reset, revert or reflog; local `master`; an early "branch is a copy" phrase that the video corrects later.
- [How to Undo Mistakes With Git Using the Command Line](#), Tobias Günther for freeCodeCamp, 24 November 2020: `git restore`, including `-p`. Caveat: `master` naming.

Further reading

- [git-update-index](#), section "Untracked cache", and the "Untracked files and performance" section of `git-status`, for why status is slow in very large working trees. Chapter 26 (Performance) continues from there.

Chapter 5: The Index

From section 5.22 of Chapter 5: The Index.

Primary sources

- [git-add](#), [git-diff](#), [git-commit](#), [git-restore](#), [git-reset](#), [git-rm](#), [git-ls-files](#), [git-update-index](#), [git-checkout-index](#), [git-write-tree](#), [git-diff-files](#), [git-status](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text used for the transcripts.
- [gitdatamodel](#), [gitformat-index](#), [gitrevisions](#) and [gitfaq](#) ("How do I ignore changes to a tracked file?").
- [racy-git](#): the cached stat data and its one known weakness. [lockfile.h](#): the lock-file protocol.
- Release notes [2.0.0](#), [1.6.1](#), [2.11.0](#), [2.19.0](#); 1.5.4 and 2.38.0 as shipped with Git 2.55.0.

Secondary sources

- Pro Git, [Interactive Staging](#). Caveat: its prompt and its `git stash save --patch` come from an older Git.
- Pro Git, [Reset Demystified](#): the three trees, with the index as "your proposed next commit". Caveat: it says "working directory" for the working tree.
- The Phase 0 report of this course, sections 11 and 12: the Stack Overflow figure and the survey of ten beginner resources (outline-based; transcripts were not reviewed).

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Lecture 5: Version Control and Git](#), MIT Missing Semester 2026: the staging area as part of the data model; the [notes](#) list `add -p`. Caveats: describes object IDs as SHA-1 only; the demo starts on `master`.
- [Git for Professionals](#), Tobias Günther for freeCodeCamp, September 2021: commit craft with `git add -p`. Caveat: 2021 interface.

Further reading

- [git-sparse-checkout](#), the feature that the skip-worktree bit was built for.

Module 3: commits

Chapter 6: Commits

From section 6.19 of Chapter 6: Commits.

Primary sources

- [git-commit](#), [git-show](#), [git-log](#), [git-cat-file](#), [git-commit-tree](#), [git-interpret-trailers](#), [git-shortlog](#), [git-var](#), [git-am](#), [git-gc](#), [githooks](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that the transcripts were checked against.
- [gitdatamodel](#), [gitrevisions](#), [gitformat-signature](#).
- The Git project's [SubmittingPatches](#): separate commits, the message, sign-off and trailers. [trailer.c](#) at [2.55.0](#) for the trailers Git generates.
- Release notes [2.29](#), [2.32](#) and [2.56](#).
- GitHub Docs: [troubleshooting commits](#), [creating a commit with multiple authors](#), [setting your commit email address](#), [about commit signature verification](#).

Secondary sources

- Pro Git, [Git Internals: Git Objects](#) and the commit guidelines in [Contributing to a Project](#). Caveat: both use `master`.
- Chris Beams, [How to Write a Git Commit Message](#) (2014): the widely quoted seven rules.
- [Conventional Commits 1.0.0](#), a title convention whose footers follow the trailer format.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Lecture 5: Version Control and Git](#), MIT Missing Semester 2026: snapshots, history as a graph, content-addressed objects. Caveats: describes object IDs as SHA-1 only; the demo starts on `master`.
- [RubyConf 2018 - Branch in Time](#), Tekin Süleyman: why history quality matters, told as one story. Nothing in it depends on a Git version.
- [How to Undo Mistakes With Git Using the Command Line](#), Tobias Günther for freeCodeCamp, 2020: includes amend. Caveat: `master` naming.

Further reading

- Chapter 9, section 9.3, and Chapter 10, section 10.3: the "new object, new ID" rule applied to rebase and cherry-pick.

Module 4: refs, branches, HEAD

Chapter 7: Branches

From section 7.20 of Chapter 7: Branches.

Primary sources

- [git-branch](#), [git-switch](#), [git-checkout](#) ("Detached HEAD"), [git-symbolic-ref](#), [git-update-ref](#), [git-for-each-ref](#), [git-rev-parse](#), [git-merge-base](#), [git-rev-list](#), [git-tag](#), [git-check-ref-format](#), [git-pack-refs](#), [git-merge-tree](#), [git-fetch](#), [git-clone](#), [git-worktree](#), [git-submodule](#), [git-config](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that the transcripts were checked against.
- [gitrevisions](#), [gitglossary](#), [gitdatamodel](#).
- Git source at 2.55.0, [wt-status.c](#), for how `git status` chooses "detached at", "detached from" and "Not currently on any branch".
- [git-branch at 2.56.0](#) and release notes [2.22](#), [2.23](#), [2.47](#), [2.51](#), [2.55](#), [2.56](#).
- GitHub Docs: [pull requests reference](#), [events that trigger workflows](#).

Secondary sources

- Pro Git, [Branches in a Nutshell](#) and [Git References](#). Caveats: `master` throughout, and branch switching with `git checkout`.
- Julia Evans, [git branches: intuition & reality](#) and [how HEAD works in git](#); the [2024 poll results](#) quoted in section 7.1.
- The Phase 0 report of this course, sections 1 and 12, for the reftable rationale and the misconception table.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Git Internals by John Britton of GitHub - CS50 Tech Talk](#), 2018: refs as files holding an ID, branches as pointers. Caveats: `master` and `checkout`; SHA-1 only.
- [Git For Beginners](#), Telusko, 2023: asks whether a branch copies the project and explains why it does not. Caveat: no undo or rebase.
- [Intern DELETED a Git Branch!](#), Chai aur Code, Hindi, 2026: recovering a deleted branch through the reflog. Caveat: a single scenario.

Outdated advice. Several popular tutorials introduce a branch as a copy of the project and HEAD as the latest commit; the report lists [Apna College's 2023 tutorial](#) among them. Both statements fail the transcripts of sections 7.2 and 7.3.

Further reading

- Chapter 8, section 8.2, for merge bases with more than one candidate; Chapter 12 for everything about `origin/`; Chapter 13 for the reflog as a recovery tool.

Modules 5, 13 and 24 (local part): configuration, tags and versions, signing

Chapter 14B: Configuration, Aliases, Tags and Signing

From section 14B.25 of Chapter 14B: Configuration, Aliases, Tags and Signing.

Primary sources

- [git-config](#), [git](#), [git-var](#), [gitcredentials](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text the transcripts were checked against.
- [git-tag](#) (including "On Re-tagging"), [git-describe](#), [git-for-each-ref](#), [git-check-ref-format](#), [git-push](#), [git-fetch](#), [gitworkflows](#), [githooks](#).
- [gitformat-signature](#), [git-verify-commit](#), [git-verify-tag](#), [git-log](#) (pretty formats), [git-merge](#), the [gpg configuration reference](#), and [ssh-keygen\(1\)](#), "ALLOWED SIGNERS".
- Git source at 2.55.0, read for behavior the manual does not state: [receive-pack.c](#) (the `receive.deny*` settings apply to `refs/heads/` only), [http.c](#) (`GIT_CURL_VERBOSE`), [editor.c](#) (`VISUAL` and dumb terminals).
- Release notes [2.34](#), [2.45](#), [2.46](#), [2.49](#), [2.54](#), [2.56](#).
- [Semantic Versioning 2.0.0](#).
- GitHub Docs: [about commit signature verification](#), [telling Git about your signing key](#), [about releases](#), [associating text editors with Git](#).

Secondary sources

- Pro Git: [Git Configuration](#), [Git Aliases](#), [Tagging](#), [Signing Your Work](#), [Environment Variables](#). Caveats: legacy `git config` syntax, GPG only, `master`.
- Julia Evans, [Popular git config options](#) (2024): a crowd-sourced list that overlaps the table in section 14B.5.
- Taylor Blau, [Highlights from Git 2.34](#): the introduction of SSH signing.
- Russ Cox, [the xz attack timeline](#); Gruntwork, [How to spoof any user on GitHub](#).

Videos

(optional; assessments in the Phase 0 report rest on captions and chapter lists, not on full viewing)

- [So You Think You Know Git - FOSDEM 2024](#), Scott Chacon, 47 minutes: `includeIf` at 08:23, SSH signing and the "Verified" badge from 19:00. Caveats: a survey, not a deep dive; about one minute of product pitch.
- [Simplify signing Git commits and tags with SSH keys](#), Andy Feller, Git Merge 2022 workshop, 56 minutes. Caveat: assessed from automatic captions and the description.
- [Tag, You're Leaked: Surviving the tj-actions Supply Chain Attack](#), BSides PDX 2025, 24 minutes: what a moved tag costs at scale.

Further reading

- The "DISCUSSION" section of `git-tag(1)`: three short essays on re-tagging, tag following and backdating.

Module 6: merge

Chapter 8: Merge

From section 8.25 of Chapter 8: Merge.

Primary sources

- [git-merge](#), [git-merge-base](#), [git-merge-tree](#), [git-merge-file](#), [git-read-tree](#), [gitrevisions](#), [git-status](#), [git-restore](#), [git-checkout](#), [git-log](#), [git-diff](#), [git-revert](#), [git-rerere](#), [gitattributes](#), [gitglossary](#), [gitfaq](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that every quotation and transcript was checked against.
- Git source and documentation at fixed tags: [merge strategies](#), [merge configuration](#), [git-add at 2.56](#), [builtin/merge.c at 2.55](#).
- Release notes: [2.34](#), [2.35](#), [2.36](#), [2.38](#), [2.50](#), [2.56](#).
- GitHub: [Pull request merges](#), [Events that trigger workflows](#), [changelog of 12 September 2022](#), [Scaling merge-ort across GitHub](#) (27 July 2023).

Secondary sources

- Pro Git, [Basic Branching and Merging](#) and [Advanced Merging](#). Caveat: see the "Outdated advice" note in section 8.22.
- Julia Evans, [Git poll results](#) (2024). Self-selected samples.
- [nbdime](#) documentation, for notebook diffs and merges.
- The Phase 0 report of this course, sections 1, 4, 12 and 13.

Videos

(optional; the assessments in the Phase 0 report rest on captions and chapter lists, not on full viewing)

- David Mahler, ["Branching and Merging"](#) (29 min, 2017). Fast-forward against three-way merges drawn on the commit graph. Conceptually excellent; uses `master` and the commands that predate `git restore`.
- ThePrimeagen for Boot.dev, ["Git and GitHub - Full Course"](#) (2024): merge from 1:03:27, merge conflicts from 2:41:25. Current commands; a digressive commentary style.
- Elijah Newren, the author of `ort`, at Git Merge [2022](#) and [2025](#): the merge backend and `--remerge-diff`. Contributor level; watch after this chapter.

Further reading

- Chapter 9 for the other way to integrate, Chapter 11 for undoing merges, Chapter 14C for rerere and merge drivers, Chapter 17 for merge methods on GitHub.

Module 7: remotes

Chapter 12: Remote Operations

From section 12.21 of Chapter 12: Remote Operations.

Primary sources

- [git-clone](#), [git-fetch](#), [git-pull](#), [git-push](#), [git-remote](#), [git-ls-remote](#), [git-branch](#), [git-merge-base](#) (fork point), [git-receive-pack](#), [git-bundle](#), [git-url-parse](#), [gitrevisions](#), [gitglossary](#), [gitdatamodel](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that every flag in this chapter was checked against.
- [gitprotocol-v2](#) and [gitprotocol-pack](#): the fetch and push conversations of section 12.13, including "wants can be anything".
- Configuration reference at 2.56: [push](#), [pull](#), [fetch](#), [core](#).
- Release notes: [1.8.3](#), [1.8.4](#), [1.8.5](#), [2.0.0](#), [2.4.0](#), [2.5.0](#), [2.20.0](#), [2.26.0](#), [2.29.0](#), [2.34.0](#), [2.48.0](#), [2.54.0](#), [2.55.0](#). Homebrew installs the same files under `share/doc/git-doc/RelNotes/`.

Secondary sources

- Pro Git: [Working with Remotes](#), [Remote Branches](#), [The Refspec](#), [Transfer Protocols](#). Caveats: `master` and `git checkout` throughout; the first still says that `git pull` only warns when `pull.rebase` is unset, which was true from 2.27 to 2.33.0; the last does not mention protocol version 2.
- GitHub Docs: [About remote repositories](#), [Forks](#), [Syncing a fork](#), [Checking out pull requests locally](#), [Renaming a branch](#), [Troubleshooting commits](#).
- Julia Evans, [Confusing git terminology](#), cited by the Phase 0 report for what "Your branch is up to date with 'origin/main'" does and does not say.
- The Phase 0 report of this course, sections 1, 4, 12 and 13, for the dated defaults, the misconception table and the force-push safety facts.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Introduction to Git - Remotes](#), David Mahler, 31 minutes, 27 March 2018: clone, fetch versus pull and tracking branches drawn on the commit graph. Caveat: `master` throughout.
- [Git and GitHub - Full Course](#), ThePrimeagen for Boot.dev, 12 November 2024: the chapter on remotes and GitHub starts at 1:54:18, followed by forks. Caveat: digressive style.
- [Complete Git and GitHub Tutorial](#), Kunal Kushwaha, 1 August 2021: fork, upstream remote, keeping a fork in sync. Caveats from the report: recorded days before password removal, uses a plain force push, mixes `master` and `main`, and could not be transcript-audited.

Further reading

- [gitprotocol-http](#), for what "smart HTTP" does with the conversation of section 12.13.
- [git-maintenance](#), task `prefetch`: Git's own scheduled fetch writes to `refs/prefetch/` so that it does not move remote-tracking branches, and therefore does not renew a lease.
- Chapter 13 (Recovery) for `reflogs` and `git fsck`, Chapter 26 (Performance) for shallow and partial clones and bundle URIs, and Chapter 30 (Incident response) for the force-push drill.

Module 8: undo

Chapter 11: Reset, Revert, Restore

From section 11.20 of Chapter 11: Reset, Revert, Restore.

Primary sources

- [git-reset](#) (with the tables in its "Discussion" section), [git-revert](#), [git-restore](#), [git-clean](#), [git-stash](#), [git-commit](#), [git-rebase](#), [git-fsck](#), [git-config](#), [gitrevisions](#) and [git\(1\)](#). The transcripts were checked against the local Git 2.55.0 copies (`git help -m <command>`).
- [How to revert a faulty merge](#), the how-to that the revert manual points to.
- [gitfaq](#), the entries on undoing a change that is already in the main branch and on a change reverted on one of two branches.
- Release notes [2.23.0](#), [2.35.0](#), [2.43.0](#), [2.51.0](#), and 1.7.1 as shipped with Git 2.55.0 (directory `RelNotes` under the path that `git --html-path` prints).
- GitHub Docs: [Reverting a pull request](#) and [Available rules for rulesets](#); GitHub CLI manual, [gh pr revert](#).

Secondary sources

- Julia Evans, [How git cherry-pick and revert use 3-way merge](#).
- Pro Git, [Reset Demystified](#). Caveat: it contrasts `reset` with `git checkout` and does not mention `git restore`.
- Pro Git, [Undoing Things](#) (old and new spellings side by side) and [Stashing and Cleaning](#). Caveat: the stash chapter mostly uses bare `git stash` and does not cover `--staged`.
- The Phase 0 report of this course, sections 4, 12 and 13, for the Stack Overflow figures and the retention defaults.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [How to Undo Mistakes With Git Using the Command Line](#), Tobias Günther for freeCodeCamp, 55 minutes, 24 November 2020: `git restore` including `-p`, `amend`, `revert`, `reset`, `reflog` recovery from 24:02. Caveat: `master` naming.
- [Revert and Reset Commits Like a Pro](#), ProCodrr episode 6, Hindi, 56 minutes, March 2023: `reset` versus `revert`, with `reflog` recovery of a hard-reset commit. Caveat: `master`.

Further reading

- [Oh Shit, Git!?!](#), a short scenario list. Caveat: it uses the `checkout` and `reset` spellings.

Module 9: rebase

Chapter 9: Rebase

From section 9.27 of Chapter 9: Rebase.

Primary sources

- [git-rebase](#), including its sections "Behavioral differences", "Recovering from upstream rebase" and "Rebasing merges"; [git-range-diff](#); [git-push](#); [git-pull](#); [git-commit](#) for `--fixup` and `--squash`; [git-cherry](#); [git-patch-id](#); [gitrevisions](#) for `ORIG_HEAD` and `REBASE_HEAD`; [gitglossary](#); [githubooks](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that the transcripts were checked against.
- [git-replay](#) and [git-history](#); the pages on [git-scm.com](#) describe 2.56.
- Release notes: [2.26.0](#), [2.34.0](#), [2.38.0](#), [2.44.0](#), [2.46.0](#), [2.54.0](#).
- GitHub Docs: [about merge methods](#), [pull request merges](#), [available rules for rulesets](#).

Secondary sources

- Pro Git, [Rebasing](#) and [Rewriting History](#). Caveats: `master` throughout, and `git checkout` in the first; neither mentions `--update-refs`, `--force-with-lease` or `git range-diff`; the second still demonstrates `git filter-branch`, with a warning.
- Julia Evans, [git rebase: what can go wrong?](#), [Confusing git terminology](#) and [Some Git poll results](#). The polls are self-selected samples.
- [git rebase in depth](#), a sandbox walkthrough. Its author and year could not be confirmed from the page, and it does not cover `exec`.
- The Phase 0 report of this course, sections 1, 4, 12 and 13.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- Philomatics, ["git rebase - Why, When & How to fix conflicts"](#), 10 minutes, June 2024. Current; short and opinionated.
- Tobias Günther for freeCodeCamp, ["Advanced Git Tutorial"](#), 34 minutes, November 2021. Interactive rebase and cherry-pick; mixes `master` and `main` and uses some `checkout`.
- Scott Chacon, ["So You Think You Know Git Part 2"](#), 23 minutes, March 2024. Fixup commits and `--update-refs` for stacked branches.
- Colt Steele, ["Git Rebase Vs. Merge"](#), 20 minutes, March 2021. Uses `git switch`; `master` naming.

Further reading

- Chapter 10 for the single replay that a rebase repeats; Chapter 12 for push and pull; Chapter 13 for reflogs and retention; Chapter 14C for rerere.

Module 10: cherry-pick and ranges

Chapter 10: Cherry-pick

From section 10.18 of Chapter 10: Cherry-pick.

Primary sources

- [git-cherry-pick](#), [git-cherry](#), [git-patch-id](#), [git-revert](#), [gitrevisions](#), [gitworkflows](#), [git-commit](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that the transcripts were checked against; the `--cherry-mark`, `--cherry-pick` and `--left-right` options are described in `git help -m log`.
- Release notes: [2.45.0](#), [2.46.0](#), [2.56.0](#).

Secondary sources

- Julia Evans, [How git cherry-pick and revert use 3-way merge](#), the clearest short account of the base used by each.
- Derrick Stolee, [Commits are snapshots, not diffs](#) (GitHub Blog, 2020), on how cherry-pick and rebase replay changes although commits store snapshots.
- [Branch for release](#) on [trunkbaseddevelopment.com](#) and Microsoft's [Release Flow](#), for the "fix on the main line, then cherry-pick" convention.
- The Phase 0 report of this course, sections 1, 4, 12 and 13.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- Tobias Günther for freeCodeCamp, "[Advanced Git Tutorial](#)", 34 minutes, November 2021. Includes cherry-picking; mixes `master` and `main`.
- ProCodrr, [episode 9 on cherry-pick](#), 8 minutes, March 2023, in Hindi. Mostly current; `master` naming.

Further reading

- Chapter 9, where the same replay runs once per commit; Chapter 11 for revert; Chapter 14A for range notation and `git log` as a query language.

Modules 10 (ranges) and 11: history forensics

Chapter 14A: History investigation

From section 14A.31 of Chapter 14A: History investigation.

Primary sources

- [git-diff](#), [gitrevisions](#), [git-log](#), [gitdiffcore](#), [git-blame](#), [git-bisect](#), [git-grep](#), [git-shortlog](#), [git-describe](#), [git-config](#). The local copies (`git help -m <command>`) are the Git 2.55.0 text that every option in this chapter was checked against.
- Christian Couder, [Fighting regressions with git bisect](#), distributed with Git: the bisection and skip algorithms. The step estimate is in [bisect.c](#) at v2.55.0.
- Release notes: [2.51.0](#), [2.52.0](#), [2.53.0](#), [2.56.0](#); [BreakingChanges](#); [git-whatchanged](#); [git-last-modified](#); [merge strategies](#).
- GitHub Docs, described and not run here: [three-dot and two-dot comparisons](#); [ignore commits in the blame view](#).

Secondary sources

- The Phase 0 report of this course, sections 1, 4 and 12: the deprecation table, and the two meanings of the dots among commonly misunderstood topics.
- Julia Evans, [Confusing git terminology](#), cited by the report for several of those misunderstandings.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- Tekin Süleyman, "[RubyConf 2018 - Branch in Time](#)", 27 minutes, December 2018. Why history quality matters, told through `git blame`, `git log -S`, interactive rebase and `--force-with-lease`. Nothing in it depends on a Git version.
- ThePrimeagen for Boot.dev, "[Git and GitHub - Full Course](#)", November 2024; the bisect chapter starts at 3:54:53. Current; digressive commentary style.
- gliich.stream, "[Chad level git: advanced concepts \(2025\)](#)", 1 hour 14 minutes, January 2025. Includes bisect, hands-on.

Further reading

- Pro Git, part "Git Tools": [Revision Selection](#), [Searching](#) and [Debugging with Git](#).
- Chapter 5 for the three everyday diffs; Chapter 7 for counting divergence; Chapter 8 for `--remerge-diff`; Chapter 9 for `git range-diff`; Chapter 10 for patch IDs; Chapter 13 for the reflog as a recovery tool.

Module 12: recovery

Chapter 13: Recovery

From section 13.22 of Chapter 13: Recovery.

Primary sources

- [git-reflog](#), [git-fsck](#), [git-gc](#) (with its NOTES section), [git-prune](#), [git-bundle](#), [git-stash](#) (the recovery recipe in EXAMPLES), [git-branch](#), [git-fetch](#), [gitrevisions](#), [git-maintenance](#), [git-pack-objects](#), [git-unpack-objects](#), [gitfaq](#).
- Configuration reference at the 2.56.0 tag: [gc](#), [core](#), [maintenance](#).
- Git source at the 2.55.0 tag: [reflog.c](#) (the expiry rules and the stash exemption), [builtin/stash.c](#).
- [How to recover a corrupted blob object](#), a how-to in Git's documentation, written in 2007; its method is unchanged.
- [Release notes of Git 2.36](#) for `git fetch --refetch`.
- GitHub Docs: [Activity view](#), [deleting and restoring branches in a pull request](#), [events API](#), [create a reference](#), [troubleshooting commits](#).

Secondary sources

- Pro Git, [Maintenance and Data Recovery](#). Caveats: it gives "around 7,000" loose objects as the automatic threshold where the configuration reference says 6700, and it predates cruft packs.
- The Phase 0 report of this course, sections 1, 12 and 13, for the retention defaults, the traced maintenance run and the GitHub instruments.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [How to Rescue Your Commits with Git Reflog](#), System Crafters, 14 minutes, November 2024: a deleted branch, commits in detached HEAD, a rebase mistake. Caveat: `master` and `checkout`.
- [Intern DELETED a Git Branch!](#), Chai aur Code, Hindi, 9 minutes, 4 March 2026: a deleted branch recovered through the reflog. A single scenario.
- [The BIGGEST Git Mistake Every Intern Makes](#), Chai aur Code, Hindi, 14 minutes, 4 March 2026: commits on the wrong branch, moved with cherry-pick and reset. A single scenario.

Further reading

- Chapter 30: Incident Response, for the same accidents at team scale, and the disaster-recovery playbook that grows out of the method of section 13.7.

Module 14: hooks, rerere, attributes, worktrees

Chapter 14C: Stash Internals, Rerere, Attributes, Hooks

From section 14C.20 of Chapter 14C: Stash Internals, Rerere, Attributes, Hooks.

Primary sources

- [git-stash](#), [git-rerere](#), [gitattributes](#), [git-check-attr](#), [git-archive](#), [git-add](#), [git-restore](#), [githooks](#), [git-hook](#), [git-receive-pack](#), [git-config](#), [git-maintenance](#), [gitfaq](#) and the security section of [git](#). Every command and option in this chapter was checked against the local Git 2.55.0 copies (`git help -m <command>`).
- [Rerere technical notes](#) for conflict normalization and conflict IDs; [contrib/rerere-train.sh](#).
- [Maintenance configuration](#) and [hook configuration](#).
- Release notes [2.51.0](#), [2.54.0](#) and [2.55.0](#).
- GitHub Docs: [About rulesets](#), [About push protection](#), [About pre-receive hooks](#) (GitHub Enterprise Server).
- [pre-commit](#), the documentation of the framework, with [pre-commit-hooks](#) and [ruff-pre-commit](#).

Secondary sources

- Pro Git, [Git Hooks](#), [Git Attributes](#) and [Rerere](#). Caveat: the second edition predates `git restore`, `git hook` and configuration-defined hooks, and uses `master`.
- The Phase 0 report of this course, sections 1, 4, 14 and 15, for release dates, the hook count, the fail-open argument and the pre-commit facts.
- [nbstripout](#), the tool whose idea the notebook filter of section 14C.8 imitates in twelve lines.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [So You Think You Know Git Part 2](#), Scott Chacon, DevWorld 2024, 23 minutes, 20 March 2024: hooks, attributes, smudge and clean filters. Caveat from the report: no depth on hook security.
- [So You Think You Know Git](#), Scott Chacon, FOSDEM 2024, 47 minutes, 8 February 2024: rerere among many other settings. Caveat: a survey, with about one minute of product pitch.

Further reading

- Chapter 8 for conflicts, Chapter 9 for rebase, Chapter 11 for the stash commands, Chapter 13 for unreachable objects, Chapter 14D for `safe.bareRepository`.

Chapter 25: Worktrees

From section 25.14 of Chapter 25: Worktrees.

Primary sources

- [git-worktree](#): commands, the REFS, CONFIGURATION FILE, DETAILS and BUGS sections. The local copy (`git help -m worktree`) is the Git 2.55.0 text that the transcripts were checked against. Source of the current manual: [git-worktree.adoc](#).
- `gc.worktreePruneExpire`, `extensions.worktreeConfig`, `worktree.guessRemote`, `worktree.useRelativePaths`, `checkout.defaultRemote`: `git help -m config`.
- [git-rebase](#) for `--update-refs` and branches checked out in a worktree; [git-refs](#) for the migration limitation.
- Release notes: [2.48.0](#), [2.56.0](#).

Secondary sources

- The Phase 0 report of this course, section 1 (the table of newer commands and the maintenance defaults) and section 13.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- Scott Chacon, "[So You Think You Know Git Part 2 - DevWorld 2024](#)" (23 min, 2024): worktrees among other topics. Current.
- Edward Thomson, "[You Don't Know Git](#)", NDC London 2025 (1 h 02 min): `reflog`, `rerere`, worktrees, rebase variants. Current.
- glich.stream, "[Chad level git: advanced concepts \(2025\)](#)" (1 h 14 min): hands-on, includes worktrees and Git LFS. Current.

Further reading

- [gitrepository-layout](#) for the `worktrees/` directory and the `commondir` and `gitdir` files.

Module 15: submodules, subtrees, LFS

Chapter 22: Git LFS

From section 22.18 of Chapter 22: Git LFS.

Primary sources

- Git LFS: the [specification](#), [server discovery](#) and [custom transfers](#) in the project's repository, read on 2 October 2026; the help texts of the installed client 3.7.1 (`git lfs <command> --help`), which the transcripts were checked against; [releases](#).
- GitHub Docs, read on 2 October 2026: [About large files on GitHub](#), [About Git Large File Storage](#), [Git LFS billing](#), [Collaboration with Git LFS](#), [Removing files from Git LFS](#), [Repository limits](#).
- [gitattributes](#) for filter drivers; `actions/checkout` [action.yml](#) at v7.0.1.

Secondary sources

- The Phase 0 report of this course: section 2 (the large-files paragraph), section 13 ("Scale, submodules and large files") and section 15 ("Data and model versioning"), with the research notes on AI/ML workflows.

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- glich.stream, "[Chad level git: advanced concepts \(2025\)](#)" (1 h 14 min): hands-on, includes Git LFS. Current.
- Vikash Das, "[MLOps: Day 4 - Data Versioning using DVC](#)" (1 h 25 min, 2024, Hindi): why Git alone does not suit data, and DVC beside Git. Mostly current; DVC CLI versions not checked.

Further reading

- Chapter 28 for data and model versioning beyond LFS; Chapter 26 for partial clone, which is Git's own way of not downloading every blob.

Chapter 23: Submodules and Subtrees

From section 23.21 of Chapter 23: Submodules and Subtrees.

Primary sources

- [git-submodule](#), [gitsubmodules](#), [gitmodules](#), [git-push](#) (`--recurse-submodules`). The local copies (`git help -m submodule`, `git help -m gitsubmodules`, `git help -m config`) are the Git 2.55.0 text that the transcripts were checked against.
- [git-subtree](#) in Git's `contrib/` directory, read on 2 October 2026; locally, `git subtree -h`.
- [Protocol configuration](#); release notes [2.38.1](#) and [2.30.6](#).
- Security advisories [GHSA-8h77-4q3w-gfgv](#) (CVE-2024-32002) and [GHSA-vwqx-4fm8-6qc9](#) (CVE-2025-48384).
- `actions/checkout` [action.yml](#) and [README](#) at v7.0.1, the README read on 2 October 2026.

Secondary sources

- Pro Git, [Git Tools: Submodules](#), which documents the failure modes of sections 23.5 to 23.10 from the user's side.
- The Phase 0 report of this course, section 12 (the misconception table), section 13 ("Scale, submodules and large files") and section 14 ("The Git client").

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- Tobias Günther for freeCodeCamp, "[Advanced Git Tutorial](#)" (34 min, 2021): includes a submodules segment. Concepts current; mixes `master` and `main` and uses `checkout`.

Further reading

- [gitrepository-layout](#) for `modules/`; Chapter 24 for the alternative of one repository.

Modules 16 and 17: the object database, the index, refs and the Git directory

Chapter 3: Git Internals

From section 3.21 of Chapter 3: Git Internals.

Primary sources

The local manual pages for the Git you run: `git help gitformat-loose`, `gitformat-pack`, `gitformat-index`, `gitformat-signature`, `gitrepository-layout`, `gitdatamodel`, `gitglossary`, `gitrevisions`, and the pages of every command used above. Online, at the 2.56.0 tag: [gitdatamodel](#), [gitformat-loose](#), [gitformat-pack](#), [gitformat-index](#), [gitformat-signature](#), [gitrepository-layout](#), [glossary](#), [revisions](#), [git-refs at 2.55](#), [the reftable design](#), [repository format versions](#), [hash-function-transition](#), [BreakingChanges](#), the release notes for 2.13, 2.29, 2.42, 2.45, 2.46, 2.51 and 2.53, and the commit "[fsck: snapshot default refs before object walk](#)".

Secondary sources

Pro Git, chapter 10: [objects](#), [packfiles](#) and [maintenance and recovery](#); frozen since May 2024, so it predates reftable and the glossary change. The GitHub Blog, [Git's database internals, part I](#). The Phase 0 report of this course for the hosting facts of section 3.14.

Videos

Derrick Stolee, "[Git Internals: a Database Perspective](#)" (Git Merge 2022, 27 min): packfiles and the storage layer; current. John Britton, "[Git Internals](#)" (CS50, 2018, 58 min): the object model, correct and clear; `master`, `checkout`, SHA-1 only, almost nothing on packfiles. Patrick Steinhart, "[Reftable Backend](#)" (2025, 20 min) and Emily Shaffer, "[SHA-256 at a Hyperscaler](#)" (Git Merge 2026, 32 min): both assume this chapter's level; the second had almost no views when the report checked it.

Further reading

James Coglan, *Building Git*, teaches the formats by reimplementing them; "[Write yourself a Git](#)" does the same in Python. Both predate reftable.

Modules 16 and 18: maintenance, transfer and scale

Chapter 26: Performance

From section 26.22 of Chapter 26: Performance.

Primary sources

The local manual of Git 2.55.0: `git help maintenance`, `gc`, `repack`, `commit-graph`, `multi-pack-index`, `fsmonitor--daemon`, `update-index`, `clone`, `fetch`, `rev-list`, `backfill`, `bundle`, `config`. Online: [git-maintenance](#), [git-gc](#), [git-repack](#), [git-commit-graph](#), [gitformat-commit-graph](#), [git-multi-pack-index](#), [bitmap-format](#), [gitformat-pack](#), [git-fsmonitor--daemon](#), [shallow](#), [partial-clone](#), [git-backfill](#), [git-bundle](#), [bundle-uri](#), [gitprotocol-v2](#), [api-trace2](#). Git source at the 2.55.0 tag: [builtin/gc.c](#) (strategies, tasks and thresholds) and [odb/source-loose.c](#) (the loose-object estimate).

Secondary sources

Derrick Stolee, Git's database internals, GitHub Blog, 2022: [I, the packed object store](#), [II, commit history queries](#), [III, file history queries](#), [IV, distributed synchronization](#). [Get up to speed with partial clone and shallow clone](#) (2020). Taylor Blau, [Scaling monorepo maintenance](#) (2021). Jeff Hostetler, [the file system monitor](#) (2022). The practitioner reports of section 26.15: each from the publisher's own environment and date, not comparable with one another. The Phase 0 report of this course, sections 1, 12 and 13.

Videos

(optional; assessments from the Phase 0 report). Derrick Stolee, ["Git Internals: a Database Perspective"](#) (Git Merge 2022, 27 min): pack indexes, the multi-pack-index, bitmaps, `git maintenance start`; current, and the best compact talk on this layer. Derrick Stolee, ["Git at Scale for Everyone"](#) (32 min, 2020): partial clone and background maintenance; the Scalar packaging shown is the one of 2020.

Further reading

Chapter 3, section 3.7, for packs and deltas; Chapter 13, sections 13.4 and 13.13, for retention; Chapter 24 for the working-tree side; Chapter 29 for traces in diagnosis.

Chapter 24: Monorepos

From section 24.18 of Chapter 24: Monorepos.

Primary sources

The local manual of Git 2.55.0: `git help sparse-checkout` with its "Internals" sections, `git help scalar`, `git help clone`, `git help backfill`. Online: [git-sparse-checkout](#), [scalar](#), [git-backfill](#), [partial-clone](#), [git-describe](#), the [sparse-index design notes](#) and [scalar.c](#) at the 2.55.0 tag. Potvin and Levenberg, [Why Google Stores Billions of Lines of Code in a Single Repository](#) (2016; not Git). Engineering at Meta, [Sapling](#) (2022). GitHub Docs as linked in sections 24.8 and 24.9.

Secondary sources

GitHub Blog: [sparse-checkout](#) (2020), [sparse index](#) (2021), [The Story of Scalar](#) (2022), [Git's database internals V](#) (2022) on ways to split a repository. [Software Engineering at Google, chapter 16](#). The practitioner reports linked in section 24.2, each from the publisher's own environment. The Phase 0 report of this course, sections 1, 12 and 13.

Videos

(optional; assessments from the Phase 0 report). Derrick Stolee, ["Git at Scale for Everyone"](#) (32 min, 2020): partial clone and a cone-mode demo; concepts current, the Scalar packaging is the one of 2020. Scott Chacon, ["So You Think You Know Git"](#) (FOSDEM 2024, 47 min): a survey, with about a minute of product pitch. Stolee's [Scalar talk](#) (2021) and ["Scaling Git at Microsoft"](#) (2017) describe superseded designs: history only.

Further reading

Chapter 26 for maintenance, the commit-graph and clone shapes; Chapter 25 for per-worktree configuration.

Modules 19 and 25: the GitHub platform, the CLI and the API

Chapter 15: GitHub

From section 15.26 of Chapter 15: GitHub.

Primary sources

- GitHub Docs, read on 2 October 2026: [types of accounts](#), [repository roles](#), [base permissions](#), [repository visibility](#), [forks](#), [linking a pull request to an issue](#), [about releases](#), [immutable releases](#), [issue forms](#), [community health files](#), [repository limits](#), [large files](#), [Packages](#), [API versions](#), [rate limits](#), [webhooks](#).
- The GitHub CLI: `gh <command> --help` of the installed 2.88.1 for every command and flag in this chapter; the [manual](#) and [release notes](#) for newer versions.
- The GitHub Changelog entries linked in the sections.
- Git: [gitnamespaces](#) for the model in section 15.7; [git-describe](#), [git-rev-list](#), [git-cat-file](#), as installed with Git 2.55.0.

Secondary sources

- The Phase 0 report of this course, sections 2, 3, 4 and 12, and its research notes on the GitHub platform: the dated changes, the plan gates and the flags marked unverified here.
- Check Point Research, [Stargazers Ghost Network](#), cited by the report.

Videos

(optional; the report's assessments rest on captions and chapter lists, not on full viewing)

- [The ultimate beginner's guide to GitHub in 2026](#), GitHub, 51 minutes, 22 September 2025. Caveats: mechanics only; compiled from 2024 episodes.
- [Git and GitHub - Full Course](#), ThePrimeagen for Boot.dev, 12 November 2024: remotes and GitHub from 1:54:18, then forks.
- The report found no verified video on roles, fork networks, releases or the API at this depth.

Further reading

- [GitHub flow](#) and [best practices for organizations](#).
- Chapter 18 for governance, Chapter 21B for fork networks during a secret leak, Chapter 27 for open-source practice.

Module 20: authentication and SSH

Chapter 16: Authentication

From section 16.27 of Chapter 16: Authentication.

Primary sources

- GitHub Docs, read on 2 October 2026: [about authentication](#), [managing personal access tokens](#), [credential types](#), [token expiration and revocation](#), [caching credentials](#), [generating an SSH key](#), [testing your SSH connection](#), [GitHub's SSH key fingerprints](#), [SSH over the HTTPS port](#), [Permission denied \(publickey\)](#), [Host key verification failed](#), [troubleshooting cloning errors](#), [managing multiple accounts](#), [deploy keys](#), [single sign-on](#), [mandatory two-factor authentication](#).
- GitHub Blog and Changelog: [token authentication requirements](#), [improving Git protocol security](#), [the RSA host key](#), [security improvements for SSH](#).
- Git 2.55.0: [gitcredentials](#), [git-credential](#), [git-config](#) (`credential.*`, `transfer.credentialsInUrl`, `url.<base>.insteadOf`, `includeIf`), as installed (`git help -m <page>`); the source files [http.c](#) and [remote-curl.c](#) for what Git does on 401, 403 and 404.
- OpenSSH as installed (`man ssh`, `man ssh_config`, `man ssh-keygen`, `man ssh-add`); online at [man.openbsd.org](#).
- The GitHub CLI: `gh auth <command> --help` of 2.88.1, and [helper_config.go](#) for what `gh auth setup-git` writes.

Secondary sources

- The Phase 0 report of this course, section 2 (authentication, the error table, the flags) and section 4, and its research notes on the GitHub platform, section 4.
- Pro Git, [Credential Storage](#): the helper protocol with a custom helper. Caveat: predates the GitHub CLI and fine-grained tokens.

Videos

(optional; the report's assessments rest on captions and chapter lists, not on full viewing)

- [Git & GitHub Crash Course 2025](#), Traversy Media, 49 minutes, 13 January 2025: SSH key setup. Caveat: beginner scope.
- [Complete git and Github course in Hindi](#), Chai aur Code, 8 June 2024: states that passwords no longer work and sets up SSH. Caveat: audited through auto-generated captions.
- No verified video covers credential helpers, token types or authentication diagnosis at this depth.

Further reading

- [Authenticating to the REST API](#) and [troubleshooting the REST API](#), for the 401, 403 and 404 rules that Git's HTTPS errors inherit.
- Chapter 21A for the workflow token and OIDC, Chapter 21B for credential blast radius, token forensics and the response to a leaked secret, Chapter 29 for the troubleshooting playbook.

Modules 21 and 22: pull requests, forks, merge methods, releases

Chapter 17: Pull Requests

From section 17.25 of Chapter 17: Pull Requests.

Primary sources

(docs.github.com, the GitHub Changelog and cli.github.com; read on 1 and 2 October 2026)

- Reference pages: [Pull requests](#), [Branches](#), [Pull request merges](#), [Pull request reviews](#), [Status checks](#), [Merge conflicts](#), [Forks](#), [Stacked pull requests](#)

- How-to pages: [Checking out pull requests locally](#), [Merging a pull request](#), [Troubleshooting required status checks](#), [Syncing a fork](#); the others are linked where they are quoted.
- Repository pages: [About merge methods on GitHub](#), [Managing a merge queue](#), [Repository limits](#), [About commit signature verification](#), [REST API endpoints for pull requests](#), [Events that trigger workflows](#)
- Changelog entries, linked where they are used: 19 February 2026 (test merge commits), 30 July 2026 (stacked pull requests), 1 October 2026 (asynchronous merge API).
- GitHub CLI: `gh pr --help` and its subcommands (2.88.1, local); [gh pr manual](#).
- Git 2.55 manual pages: `git help merge-tree`, `git help rebase`, `git help branch`, `git help cherry`, `git help config` (`receive.hideRefs`); the [gitfaq](#) entry on squash merges and long-lived branches.

Secondary sources

- The Phase 0 report of this course, sections 2, 3, 12 and 13, and its notes on the GitHub platform.
- GitHub Engineering, [Scaling merge-ort across GitHub](#) (2023): why a server needs a merge without a working tree.

Videos

(from the Phase 0 report, with its caveats)

- ["The ultimate beginner's guide to GitHub in 2026"](#), GitHub, 22 September 2025: the pull request and merge chapters, 40:44 to 47:14. Mechanics only; compiled from 2024 episodes. The report found no verified video that teaches code review end to end in the 2026 interface.

Further reading

- Chapter 8: Merge, sections 8.12, 8.15 and 8.17; Chapter 9: Rebase, sections 9.9 and 9.14; Chapter 12: Remote Operations, sections 12.8, 12.10 and 12.12; Chapter 14A: History investigation, sections 14A.2, 14A.8, 14A.18 and 14A.22.
- [Pro Git](#), ["Contributing to a Project"](#): the fork-and-pull flow at a book's pace; its screenshots predate the current interface.

Module 23: governance

Chapter 18: Branch Protection and Rulesets

From section 18.25 of Chapter 18: Branch Protection and Rulesets.

Primary sources

(docs.github.com and the GitHub Changelog; read on 1 and 2 October 2026)

- Rulesets: [About rulesets](#) and its [Enterprise Cloud view](#), [Available rules for rulesets](#), [Creating rulesets for a repository](#), [Managing rulesets for a repository](#), [Troubleshooting rules](#), [Converting branch protections to rulesets](#), [Creating rulesets for repositories in your organization](#)
- Classic protection: [About protected branches](#)
- Checks: [Troubleshooting required status checks](#), [Status checks](#), [Skipping workflow runs](#)
- API and CLI: [REST API endpoints for rules](#), [rule suites](#), [branch protection](#); `gh ruleset --help` and `gh api --help` (2.88.1, local); [gh ruleset manual](#)
- Roles: [Repository roles for an organization](#)
- Changelog entries are linked where they are used.
- Ruby's `File.fnmatch`, the function GitHub's documentation links to; Git 2.55: `git help hooks` (`pre-receive`), `git help config` (`receive.denyNonFastForwards`).

Secondary sources

- The Phase 0 report of this course, sections 2, 3, 4 and 13, and its notes on the GitHub platform.
- [github/ruleset-recipes](#): GitHub's importable example rulesets.

Videos

(from the Phase 0 report, with its caveats)

- ["Introduction to GitHub Actions - Part 6 - Repository Rulesets"](#), Mickey Gousset, 6 December 2024, 15 minutes: a ruleset that requires a status check, the bypass list, rulesets versus classic rules. Current; slow and careful.
- ["Securely building GitHub on GitHub"](#), GitHub Universe 2024, 41 minutes: rulesets, Rule Insights, Evaluate mode and bypass lists at scale. Predates the 2025 and 2026 ruleset additions.

Further reading

- Chapter 12: Remote Operations, section 12.7 (server-side rules in plain Git); Chapter 14B, sections 14B.11 and 14B.15 to 14B.17; Chapter 17: Pull Requests, sections 17.5, 17.8 and 17.11.

Chapter 19: CODEOWNERS

From section 19.18 of Chapter 19: CODEOWNERS.

Primary sources

(docs.github.com and the GitHub Changelog; read on 1 and 2 October 2026)

- [About code owners](#): locations, syntax, the example file, size limit, write-access requirement, forks, branch protection.
- [Available rules for rulesets](#): code owner review and the required reviewers sub-option.
- [REST API: list CODEOWNERS errors](#).
- [Changelog, 17 February 2026: required reviewer rule generally available](#).
- [Stacked pull requests \(reference\)](#).
- Git 2.55: [gitignore pattern format](#) (`git help ignore`), `git help check-ignore`.

Secondary sources

- The Phase 0 report of this course, section 2 ("CODEOWNERS requests reviews; only a rule makes them mandatory"), and its notes on the GitHub platform, section 3.

Videos

- The Phase 0 report lists no verified video that teaches CODEOWNERS; its survey of popular beginner courses found that none covers it.

Further reading

- Chapter 17: Pull Requests, sections 17.3 to 17.5; Chapter 18: Branch Protection and Rulesets, sections 18.5 and 18.7; Chapter 4: The Working Tree, for `.gitignore` itself.

Module 26: Actions fundamentals

Chapter 20A: GitHub Actions Fundamentals

From section 20A.22 of Chapter 20A: GitHub Actions Fundamentals.

Primary sources

(docs.github.com, the GitHub Changelog, and each action's repository at the pinned tag; read on 1 and 2 October 2026)

- Reference pages: [workflow syntax](#), [events that trigger workflows](#), [contexts](#), [expressions](#), [variables](#), [secrets](#), [workflow commands](#), [dependency caching](#), [limits](#), [GitHub-hosted runners](#), [reusing workflow configurations](#).
- Other documentation: [workflow artifacts](#), [GITHUB_TOKEN](#), [skipping workflow runs](#), [troubleshooting required status checks](#), [publishing a package with Actions](#).
- Action READMEs and `action.yml` files at the pinned versions: [checkout v7.0.1](#), [setup-python v7.0.0](#), [setup-java v6.0.1](#), [cache v6.1.0](#), [upload-artifact v7.0.1](#), [download-artifact v8.0.1](#), [setup-uv v10.2.0](#), [docker/login-action v4.6.0](#), [docker/metadata-action v6.2.0](#), [docker/build-push-action v7.4.0](#), [docker/setup-buildx-action v4.4.1](#).
- Tool documentation: [uv with GitHub Actions](#), [Docker: cache management with GitHub Actions](#).
- Changelog entries, linked where they are used.
- Git 2.55 manual pages: `git help clone`, `git help fetch`, `git help describe`, `git help diff`. GitHub CLI 2.88.1: `gh run --help`, `gh workflow --help`, `gh cache --help`.

Secondary sources

- The Phase 0 report of this course, sections 2, 3, 4, 12 and 13, and its notes on GitHub Actions, including the flags carried into this chapter as "Unverified".

Videos

(from the Phase 0 report, with its caveats)

- ["Complete GitHub Actions Course - From BEGINNER to PRO"](#), Sid Palas, DevOps Directive, 3 h 43 min, 24 September 2025. The most complete current free course on workflow mechanics. It predates the Node 24-only runtime and `actions/checkout v7`, says "branch protections" and never mentions rulesets.

- ["GitHub Actions tutorial in Hindi"](#), CODERS NEVER QUIT, 1 h 51 min, 27 September 2024, Hindi. Jobs, expressions, checkout, caching and artifacts. Mostly current in concept; the action versions shown were not checked; no pinning.

Further reading

- Chapter 17: Pull Requests, sections 17.2, 17.6 and 17.11; Chapter 18: Branch Protection and Rulesets on required status checks; Chapter 19: CODEOWNERS on protecting the workflows directory; Chapter 26: Performance, section 26.11 on shallow clones; Chapter 14A: History investigation on two-dot and three-dot ranges.

Modules 27 and 28: build, package, deliver; runners and debugging CI

Chapter 20B: GitHub Actions: delivery, runners, cost and debugging

From section 20B.22 of Chapter 20B: GitHub Actions: delivery, runners, cost and debugging.

Primary sources

- GitHub Docs: [deployments and environments](#), [managing environments](#), [control deployments](#), [reviewing deployments](#), [REST: deployment environments](#).
- GitHub Docs: [workflow syntax](#), [reusing workflow configurations](#), [reuse workflows](#), [metadata syntax](#), [custom actions](#).
- GitHub Docs: [GitHub-hosted runners](#), [self-hosted runners](#), [secure use](#), [limits](#), [billing](#), [runner pricing](#).
- GitHub Docs: [dependency caching](#), [using secrets](#), [enable debug logging](#), [re-running workflows and jobs](#), [troubleshooting workflows](#), [skipping workflow runs](#), [troubleshooting required status checks](#).
- Action documentation at the pinned versions: [actions/checkout v7.0.1](#), [actions/cache v6.1.0](#), [actions/upload-artifact v7.0.1](#), [actions/download-artifact v8.0.1](#), [astral-sh/setup-uv v10.2.0](#).
- GitHub CLI manual: [gh run view](#), [gh run rerun](#), [gh run watch](#), [gh workflow run](#), and the `--help` output of the installed 2.88.1.
- The local Git manual: `git help describe`, `git help ls-files`, `git help gitattributes`, `git help config (core.ignoreCase)`.

Secondary sources

- [actionlint](#) and [act: unsupported functionality](#), third-party tools, not installed for this course.
- The GitHub changelog entries linked in the text for each dated change.

Videos

(from the Phase 0 report, with its caveats)

- ["Complete GitHub Actions Course - From BEGINNER to PRO"](#), Sid Palas, DevOps Directive, 3 h 43 min, 24 September 2025. The most complete current free course; it has sponsor segments, says "branch protections" and never mentions rulesets, and pre-dates Node 24-only runners and `actions/checkout v7`.
- ["Introduction to GitHub Actions - Part 6 - Repository Rulesets"](#), Mickey Gousset, 15 min, 6 December 2024. A ruleset that requires a status check.

Further reading

- Chapter 18, section 18.8, for required status checks from the ruleset side; Chapter 21A for the security model of everything in this chapter.

Module 29: Actions security

Chapter 21A: GitHub Actions security

From section 21A.26 of Chapter 21A: GitHub Actions security.

Primary sources

- GitHub Docs: [secure use reference](#); [securely using pull_request_target](#); [script injections](#); [GITHUB_TOKEN](#); [workflow syntax](#); [OIDC reference](#); [OIDC in AWS](#); [dependency caching](#); [secrets reference](#); [managing GitHub Actions settings for a repository](#); [about Actions policies](#); [keeping your actions up to date with Dependabot](#).
- GitHub changelog entries, linked where each change is described (sections 21A.3, 21A.5, 21A.7, 21A.8, 21A.10, 21A.11, 21A.15).
- [actions/checkout README at v7.0.1](#).
- Advisories and post-mortems: [CISA on tj-actions](#); [GHSA-mrrh-fwg8-r2c3](#); [Nx post-mortem](#); [GHSA-69fq-xp46-6x23](#); [Aqua incident notice](#); [TanStack post-mortem](#); [PyPI on Ultralytics](#); [Codecov security update](#).

Secondary sources

- [GitHub Security Lab: preventing pwn requests](#) (2021; parts 2 to 4 of the series were not consulted).
- Vendor and researcher analyses: [Unit 42](#); [GitGuardian on GhostAction](#); [Wiz on Shai-Hulud 2.0](#); [Datadog Security Labs](#); [StepSecurity on hackerbot-claw](#); [John Stawinski on PyTorch](#).
- Secondary reporting only: [Techzine](#); [The Hacker News](#).
- Tools: [zizmor audits](#); [actionlint](#); [OpenSSF Scorecard checks](#).

Videos

(from the Phase 0 report, with its caveats)

- [Adnan Khan, "The dark side of GitHub Actions"](#), RomHack 2024, 53 min. The vulnerability classes are current; GitHub defaults have changed since.
- [Niek Palm, "Beyond the Commit: Weaponizing and Hardening GitHub Actions"](#), NDC Security 2026, 55 min. The most current defender-oriented talk; it pre-dates the June to September 2026 platform changes.
- ["Tag, You're Leaked: Surviving the tj-actions Supply Chain Attack"](#), BSides PDX 2025, 24 min. An incident-response case study.

Further reading

- [GitHub's 2026 security roadmap for Actions](#): announced work, not shipped features.
- `workflows/ACTION_PINS.md` in this course: the pins and how to re-verify them.

Modules 24 (GitHub part), 30 and 31: repository security and secret-leak response

Chapter 21B: Repository security, identity, the Git client, and secret-leak response

From section 21B.27 of Chapter 21B: Repository security, identity, the Git client, and secret-leak response.

Primary sources

- Git 2.55 manual pages: `git help git` (SECURITY), `git help config` (`safe.*`, `protocol.*`), `git help filter-branch` (WARNING), `git help hook`, `git help githooks`.
- [Git security advisories](#); [safe configuration](#); [protocol configuration](#); [githooks](#).
- GitHub Docs, fetched 2 October 2026: [removing sensitive data from a repository](#); [remediating a leaked secret](#); [about push protection](#); [Dependabot options reference](#). Through the Phase 0 report: [about secret scanning](#); [supported patterns](#); [commit signature verification](#); [credential types](#); [security features](#).
- [git-filter-repo manual](#) (fetched 2 October 2026).

Secondary sources

- [GitGuardian, State of Secrets Sprawl 2026](#); [GitGuardian on leaked GitHub App private keys](#).
- [Russ Cox, timeline of the xz open source attack](#).
- The research, incident reports and articles linked where they are used, in sections 21B.4, 21B.8, 21B.10, 21B.14 and 21B.15.

Videos

(from the Phase 0 report, with its caveats)

- ["git-filter-repo for rewriting Git history"](#), Elijah Newren, Git Merge 2024, 22 min. A contributor talk on what the tool does and how it compares with filter-branch and BFG; not a step-by-step incident tutorial.
- [Git Merge 2022 workshop on SSH commit signing](#), listed by the roadmap for Module 24.
- ["Day-2: DevSecOps for Git and GitHub"](#), Abhishek Veeramalla, 46 min, 22 January 2026. Shows a branch ruleset, CODEOWNERS, gitleaks through pre-commit and in Actions, and Dependabot together; no signing, OIDC or action pinning.

Further reading

- Chapter 13: Recovery for reflogs, pruning and the GitHub-side recovery instruments; Chapter 14B for local signing; Chapter 16 for credential mechanics; Chapter 12 for force pushes.

Modules 32 and 34: branching and release strategy, practices and design review

Chapter 27: Open source and team workflows

From section 27.25 of Chapter 27: Open source and team workflows.

Primary sources

- Git 2.55 manual pages: `git help workflows` ([gitworkflows](#)), `git help cherry-pick`, `git help rev-list` (`--cherry-pick`, `--left-right`), `git help branch`, `git help merge-tree`, `git help for-each-ref`.
- GitHub Docs: [GitHub flow](#), [Forks](#), [Syncing a fork](#), [About merge methods](#), [Managing a merge queue](#), [Setting guidelines for repository contributors](#), [About releases](#).
- GitHub CLI 2.88.1, local help: `gh repo fork --help`, `gh repo sync --help`, `gh pr create --help`.
- The authors of the models: Vincent Driessen, [A successful Git branching model](#) with the 2020 note; Scott Chacon, [GitHub Flow](#); GitLab, [What is GitLab Flow?](#); [trunkbaseddevelopment.com](#); Microsoft, [How Microsoft develops with DevOps](#).
- DORA capability pages: [trunk-based development](#), [working in small batches](#), [version control](#).
- Conventions: [How to Write a Git Commit Message](#), [Conventional Commits 1.0.0](#), [Semantic Versioning 2.0.0](#), [Keep a Changelog 1.1.0](#).

Secondary sources

- The Phase 0 report of this course, sections 6, 12 and 13, and section 6 of its notes on books, documentation and misconceptions.
- Martin Fowler, [Patterns for Managing Source Code Branches](#); Pete Hodgson, [Feature Toggles](#); Rouan Wilsenach, [Ship / Show / Ask](#).
- Lopes, Accioly, Borba and Menezes, interview-and-survey study of branching workflows, [arXiv 2507.08943](#).
- Vendor and company material, linked where used, each describing its own environment: GitHub and Shopify on merge queues, Atlassian on Gitflow.

Videos

(from the report's tables, with its caveats)

- [3 Git Workflows Every Developer Should Know](#), TechWorld with Nana, 13 January 2026, 32 minutes: a neutral comparison of Git Flow, GitHub Flow and trunk-based development. Caveat: it has a sponsored segment.
- [Continuous Integration vs Feature Branch Workflow](#), Dave Farley, 6 January 2021, 18 minutes: the argued case for trunk. Caveat: deliberately one-sided advocacy; watch it after the first, and with section 27.13 beside you.
- [Implementing a Strong Code-Review Culture](#), Derek Prior, RailsConf 2015, 38 minutes: review as teaching and context-giving. Tool-independent and still valid.

Further reading

- Jackson Gabbard, [Stacked Diffs Versus Pull Requests](#), and Gergely Orosz, [Stacked Diffs](#).
- *Accelerate* (Forsgren, Humble, Kim, 2018), the book-length statement of the DORA research.
- Chapter 28: AI/ML workflows applies this chapter to repositories that also hold notebooks, data pointers and evaluation runs.

Module 33: AI/ML workflows

Chapter 28: AI/ML workflows

From section 28.22 of Chapter 28: AI/ML workflows.

Primary sources

- Git 2.55 manual pages: `git help attributes` (filters, text, merge), `git help ignore`, `git help check-ignore`, `git help describe`, `git help status` (porcelain format), `git help worktree`, `git help apply`.
- Notebooks: [nbformat](#), [nbstripout](#), [nbdime](#) and its [Git integration](#), [jupyter](#), [ReviewNB](#), GitHub's [rich notebook diff preview](#).
- Data and models: [About large files on GitHub](#), [About Git LFS](#), [DVC: get started](#), [DVC joins lakeFS](#), [lakeFS licence change](#), [Hugging Face download guide](#), [MLflow model registry](#).
- Reproducibility: [MLflow system tags](#), [W&B code saving](#), [Hydra output directory](#), [uv project layout](#) and [locking](#), [Docker build best practices](#), [git-describe](#).
- Hooks and CI: [pre-commit](#), [pre-commit-hooks](#), [ruff-pre-commit](#), [uv with GitHub Actions](#), [larger runners](#), [push protection](#), [promptfoo GitHub Action](#), [DeepEval in CI](#).
- Structure, Docker and Java: [Cookiecutter Data Science](#), [PyPA on src and flat layouts](#), [Docker build secrets](#), [GitHub's Python ignore template](#), [Gradle Wrapper](#), [Dependabot ecosystems](#).

Secondary sources

- The Phase 0 report of this course, sections 14 and 15, and its notes on AI/ML workflows and security. Every version, date, licence and status in this chapter comes from there; its flags are carried as "Unverified" callouts.
- GitGuardian, [How to handle secrets in Jupyter notebooks](#) and [State of Secrets Sprawl 2026](#): one vendor's measurements.
- DORA, [2025 report](#) and [version control capability](#).
- StepSecurity and Techzine on agents in workflows, linked in section 28.15.

Videos

(from the report's tables, with its caveats)

- [MLOps: Day 4 - Data Versioning using DVC](#), Vikash Das, in Hindi, 7 August 2024, 1 h 25 min: why Git alone does not suit data, DVC beside Git, an S3 remote, `dvc push` and `dvc pull`. Caveats: DVC CLI versions were not checked; no pipelines or CI.
- [How to build a ML project using MLOps](#), CampusX, in Hindi, 25 November 2024, 1 h 21 min. Use it as a concept map only: the report notes no workflow YAML, no `dvc push` or `dvc pull`, and no remote storage.

Further reading

- Chapter 14C for filters, drivers and hooks in full; Chapter 22 for Git LFS; Chapter 25: Worktrees for the worktree used in Lab 33.3.
- Chapter 27: Open source and team workflows for branching, release tags and the fork model that section 28.11 collides with.

Module 35: the diagnosis method

Chapter 29: Production Troubleshooting

From section 29.18 of Chapter 29: Production Troubleshooting.

Primary sources

- The manual pages of Git 2.55.0, read locally with `git help -m <command>`: `git-status`, `git-rebase`, `git-merge`, `git-cherry-pick`, `git-revert`, `git-bisect`, `git-bundle`, `git-update-ref`, `git-for-each-ref`, `git-ls-remote`, `git-merge-tree`, `git-check-ignore`, `git-fsck`, `git-config`, `gitrevisions`, `gitrepository-layout`.
- `git-prompt.sh` as installed by Homebrew's Git at `/opt/homebrew/etc/bash_completion.d/git-prompt.sh`: the header comments document `__git_ps1`.
- GitHub Docs: [Using the activity view to see changes to a repository](#); [REST: list repository activities](#); [Issue event types](#); [GitHub event types: PushEvent](#); [REST API endpoints for events](#); [Viewing insights for rulesets](#); [Reviewing the audit log for your organization](#); [Audit log for an enterprise](#); [Deleting and restoring branches in a pull request](#); [Troubleshooting commits](#); [Troubleshooting required status checks](#).
- GitHub Changelog: [Upcoming changes to GitHub Events API payloads](#).

Secondary sources

- Phase 0 report, sections 12 and 13, and the research notes `github_platform.md`, from which every GitHub-side fact and flag in section 29.8 is taken.
- Yang et al., the ACM TOSEM study of Git-command questions ([author preprint](#)). The report flags the final volume and article number as not confirmed.

Videos

(optional; the assessments in the Phase 0 report rest on captions, not on full viewing)

- [How to Undo Mistakes With Git Using the Command Line](#), Tobias Günther for freeCodeCamp, 55 minutes, 24 November 2020: restore, amend, revert, reset, recovery with the reflog. Caveat: `master` naming.
- [Lecture 5: Version Control and Git](#), MIT Missing Semester 2026, 1 hour 10 minutes: the data model before the commands, and the reflog as a safety net.

Further reading

- Chapter 13: Recovery, for every case in which something was lost.
- Chapter 12: Remote Operations, section 12.14, and Chapter 16: Authentication, sections 16.17 to 16.20, for the two other diagnosis procedures of this book.
- Chapter 18: Branch Protection and Rulesets, section 18.17, "Why can't I merge?".
- The troubleshooting playbook.

Modules 36 to 38: incident drills, communication and postmortems

Chapter 30: Incident Response

From section 30.27 of Chapter 30: Incident Response.

Primary sources

- Local manual pages of Git 2.55.0: `git help reflog`, `git help push` (the lease forms), `git help rebase` (`--onto`, `--empty`), `git help cherry`, `git help range-diff`, `git help fsck`, `git help log` (history simplification, `--remerge-diff`), `git help describe`.
- GitHub Docs: [Activity view](#); [deleting and restoring branches in a pull request](#); [Events API](#); [create a reference](#); [troubleshooting commits](#); [available rules for rulesets](#); [removing sensitive data from a repository](#); [remediating a leaked secret](#); [push protection](#).
- [actions/checkout README, v7.0.1](#).

- The Phase 0 report, sections 12 to 14, including its unverified flags for the GitHub-side recovery recipe.

Secondary sources

- [git-filter-repo manual](#), for the clean-up of other clones after a rewrite.
- Google, *Site Reliability Engineering*, [Postmortem Culture: Learning from Failure](#).
- Julia Evans, [what can go wrong with rebasing](#).
- Truffle Security, [commits orphaned by force pushes](#): an observation by researchers, not a statement by GitHub.

Videos

- ["Tag, You're Leaked: Surviving the tj-actions Supply Chain Attack"](#), BSides PDX 2025, 24 minutes. A 72-hour incident response for GitHub Actions; the Phase 0 report calls it the closest thing to a production incident-response case study, and notes that it found no conference talk covering end-to-end Git incident recovery with current tooling.

Further reading

- Chapter 13: Recovery, Chapter 29: Production Troubleshooting, Chapter 21B: Repository Security and Secret-Leak Response, Chapter 20B: Delivery, Runners, Cost and Debugging.

Module 42: the frontier

Chapter 14D: The Frontier

From section 14D.18 of Chapter 14D: The Frontier.

Primary sources

- [BreakingChanges](#), read in the copy installed with Git 2.55.0, and the [release notes directory](#).
- [git-init](#), [git-config](#) (`init.*`, `safe.bareRepository`), [git-refs](#), [hash-function-transition](#), [git-fast-export](#), [git-fast-import](#).
- [git-history](#), [git-replay](#), [git-last-modified](#), [git-repo](#); for 2.56-only parts, [git-history](#) at 2.56 and [git-replay](#) at 2.56.
- [git-format-patch](#), [git-am](#), [git-apply](#), [git-range-diff](#), [git-patch-id](#), [git-send-email](#), [SubmittingPatches](#), [MyFirstContribution](#), [maintain-git](#).
- The maintainer's message of 28 September 2026, through a [mailing-list mirror](#), as cited in the Phase 0 report.
- The [GitHub Changelog](#), with the entry on [stacked pull requests](#); the [Homebrew formula for Git](#).

Secondary sources

- [LWN on the road to 3.0](#) and [GitLab on Git 2.56](#): the only sources for calendar months. The Phase 0 report records a conflict between LWN's paraphrase and the primary message.
- The Phase 0 report of this course, section 1, with its table of flags.
- [Git Rev News](#); the repositories and documentation of [Jujutsu](#), [Sapling](#) and [git-branchless](#).

Videos

(optional; assessments in the Phase 0 report rest on captions, not on full viewing)

- [Reftable Backend: What it is, where it's headed and why should you care](#), Patrick Steinhardt, 20 minutes, 3 September 2025. Caveat: assumes knowledge of internals.
- [SHA-256 at a Hyperscaler](#), Emily Shaffer, Git Merge 2026, 32 minutes, uploaded 1 October 2026. Caveat from the report: almost no views when checked, so there is no secondary quality signal.
- [You Don't Know Git](#), Edward Thomson, NDC London 2025, 1 hour 2 minutes: `rerere`, the `reflog`, `worktrees` and `rebase` variants.

Further reading

- Chapter 3 for the formats, Chapter 9 for `rebase`, `--update-refs` and `git range-diff`, Chapter 10 for patch IDs, Chapter 14C for hooks and their security.

The video path as a whole

The chapters cite videos one at a time. The complete, ordered path, with length, date, level, what to learn from each and what is outdated in it, is section 7 of the Phase 0 research report. Its finding is that a complete path exists only by combining about a dozen sources, and that no single video covers the full list of topics.

See also

